ARCH LINUX 分册教材

第二册

安装、启动与软件包管理

从 Live 环境到可启动系统，以及 pacman 与打包体系

依据 Arch Linux 中文维基全量条目编写

共 9 册 · 997 篇 · 安装 / 系统 / 网络 / 硬件 / 桌面 / 软件 / 开发 / 安全

第一册讲的是 Arch 为什么长成这个样子；这一册开始动手，把一台空机器变成能开机、能装软件、能长期更新的系统。

从 Live 环境到一台能启动的机器

你最先拿到的不是系统，而是一个 Live 环境：U 盘或光盘里的临时系统，好用，却不是你最终要用的东西。本册要打通三段链路：启动上，机器通电后固件怎么找到内核、把控制权交出去，引导加载程序装在哪里；安装上，分区、挂载、写出一个最小可启动系统，初始化与 initramfs 如何接续；软件上，包管理器怎么从仓库取包、怎么记账、升级时怎么保持依赖对齐。

读完这一册，你能独立完成一次安装，说得清每一步在干什么；机器起不来时，能判断问题卡在引导、内核参数还是 initramfs；日常能熟练地同步、安装、卸载、查询，并排查依赖与冲突；看得懂构建脚本，能把 AUR 里的东西装到本机，也能自己打包交出去；并且明白打包准则为什么把某些要求写成硬性规定。

在九册里的位置

前置是第一册《Arch Linux 与它的社区》。它交代了发行版的取舍、社区的协作方式，以及把 Arch Wiki 当第一手资料的阅读习惯——本册大量篇幅就是照着文档做事，不接受那份契约的人会觉得处处别扭。

后继以第三册《系统管理与内核》为主：本册装出的分区、引导、内核与 initramfs，在那里展开成文件系统、内核与系统恢复的完整图景。再往后，第四册的网络与服务、第六册的桌面环境与图形栈，都建立在“系统已经装好、而且能装包”这个前提上。

阅读顺序与取舍

主线是安装、启动、初始化、包管理这条链。按序读，每章都在补上一章留下的悬念：装完为什么起不来，该往哪里看，起来之后软件从哪装。

打包准则排在本册开头、篇幅最大，但它更像标准而非教程：首次安装只读其中依赖、许可证与包命名的通则，真要打包时再逐条查。包开发、非官方用户仓库与 AUR 助手属于同一条线，只用官方仓库可以整段跳过；函数式包管理器、启动屏幕这类单章话题，缺什么补什么，不影响主线。

前置知识与常见误区

前置要求不高：会用终端，大致知道分区、挂载、文件系统指什么，愿意耐心读英文文档，这些在第一册都有铺垫。

误区之一是把安装当成背命令。命令会随文档和工具变化，值得记住的是顺序与判据：为什么要先挂载再安装，引导程序为什么装在那里。误区之二是以为装完就结束了：滚动更新之下，定期更新、读公告、绝不做部分更新才是日常。误区之三是把 AUR 当成官方仓库——那里是用户提交的构建脚本，编译发生在你自己机器上，没有人替你兜底。误区之四是动不动就重装：多数故障能顺着引导、内核参数、initramfs 与日志逐层排查修回来，本册教的正是这条路径。

安装流程

八个阶段，一步不可省

1. **准备安装介质**
   下载 ISO / netboot，校验 SHA256 与 PGP 签名
2. **启动 Live 环境**
   UEFI 或 BIOS 引导，进入 root shell
3. **分区与格式化**
   分区表 → 文件系统 → 挂载到 /mnt
4. **安装基础系统**
   pacstrap 写入 base、内核与固件
5. **生成 fstab**
   genfstab 固化挂载关系
6. **chroot 配置**
   时区、本地化、主机名、用户、initramfs
7. **安装引导程序**
   GRUB / systemd-boot / EFISTUB
8. **重启与验证**
   卸载、重启、登录、联网、装桌面

从准备介质到首次登录的完整安装流程

启动流程

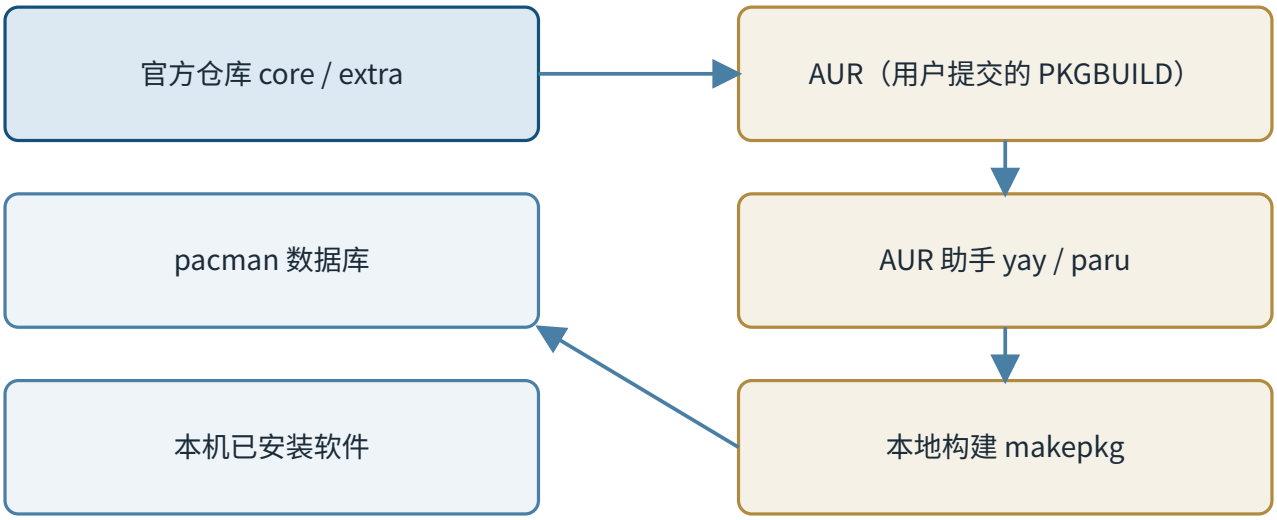
从按下电源到登录提示符



固件、引导加载程序、内核、initramfs 与 systemd 的接力

包管理生态

官方仓库与 AUR 的分工



官方仓库由 Arch 团队构建；AUR 只提供构建脚本，编译在你自己的机器上完成

pacman 与 AUR 助手的关系：构建发生在本地

目　录

第 1 章　Arch 打包准则（21 节）

1.1 PHP 打包准则　1.2 32 位软件包打包准则　1.3 Free Pascal 打包准则　1.4 KDE 软件打包准则　1.5 Haskell 打包准则　1.6 CLR 软件打包准则　1.7 命令行解释器（Shell）打包准则　1.8 字体打包指南　1.9 R 软件打包准则　1.10 Node.js 打包准则　1.11 DKMS 软件包打包准则　1.12 Go 语言软件打包准则　1.13 Electron 打包准则　1.14 Rust 软件打包准则　1.15 Java 打包准则　1.16 交叉编译工具打包准则　1.17 Arch 打包准则/安全　1.18 VCS 软件打包准则　1.19 CMake 软件打包准则　1.20 Python 打包准则　1.21 Arch 打包准则

第 2 章　启动过程（11 节）

2.1 Bootchart　2.2 E4rat　2.3 静默启动　2.4 性能优化/启动过程　2.5 PXE　2.6 Fstab　2.7 EFI 系统分区　2.8 Arch 的启动流程　2.9 Arch + Windows 双系统　2.10 UEFI　2.11 UEFI/安全启动

第 3 章　安装过程（11 节）

3.1 安装指南　3.2 Ventoy　3.3 Archinstall　3.4 Genfstab　3.5 无盘系统　3.6 在可移动设备上安装 Arch Linux　3.7 多引导 USB 设备　3.8 从现有 Linux 发行版安装 Arch Linux　3.9 Install Arch Linux with Fake RAID　3.10 U 盘安装介质　3.11 在 ZFS 上安装 Arch Linux

第 4 章　启动加载程序（9 节）

4.1 Clover　4.2 EFISTUB　4.3 统一内核映像　4.4 REFInd　4.5 Systemd-boot　4.6 GRUB Legacy　4.7 GRUB/技巧和窍门　4.8 Syslinux　4.9 GRUB

第 5 章　包开发（8 节）

5.1 Clang　5.2 .SRCINFO　5.3 Debuginfod　5.4 Ccache　5.5 OCaml 打包准则　5.6 Eclipse 插件打包准则　5.7 调试/获取跟踪信息　5.8 PKGBUILD

第 6 章　包管理（8 节）

6.1 Pkgstats　6.2 Pkgfile　6.3 Reflector　6.4 元软件包和软件包组　6.5 Powerpill　6.6 降级软件包　6.7 Local Mirror　6.8 软件包补丁

第 7 章　初始化（6 节）

7.1 BusyBox　7.2 Systemd/FAQ　7.3 OpenRC　7.4 SysVinit　7.5 Init　7.6 Systemd

第 8 章　包管理器（6 节）

8.1 Pacman/恢复本地数据库　8.2 Pacman/pacnew 与 pacsave　8.3 Pacman/软件包签名　8.4 Pacman/各软件包管理器命令对应关系　8.5 Pacman　8.6 Pacman/提示和技巧

第 9 章　Initramfs（4 节）

9.1 Booster　9.2 Mkinitcpio/极简 initramfs　9.3 Dracut　9.4 Mkinitcpio

第 10 章　配置文件（4 节）

10.1 Xsettingsd　10.2 X 资源　10.3 桌面项　10.4 XDG 基本目录

第 11 章　非官方用户仓库（4 节）

11.1 Arch Linux 中文社区仓库　11.2 Arch Linux 中文社区仓库历史存档　11.3 ALHP　11.4 非官方用户仓库

第 12 章　AUR 助手（1 节）

12.1 Yay

第 13 章　函数式包管理器（1 节）

13.1 Nix

第 14 章　启动屏幕（1 节）

14.1 Plymouth

Arch 打包准则

本章解决"同一个上游软件，包该叫什么、版本怎么写、依赖怎么填、文件放哪里"这四类规范，以及各语言与构建系统的例外。建议先读同名那节打包准则，把命名、版本号、依赖、关联与软件源过一遍，再读安全那一节；Python、CMake、Go、Rust、Java 按项目挑着看，其余章节等遇上再翻。

最容易的误解是把语言专章当成可有可无的附录：KDE 要强制指定 Qt 版本，Electron 要先决定是否复用系统 electron，VCS 包的版本必须由 pkgver() 生成。漏掉这些，编译虽过，包名、依赖与文件位置却不合准则。

1.1　PHP 打包准则

难度 入门 | **篇幅** 2.7 KB | **所属** Arch 打包准则

本节将讲到

- 软件包名称
- 打包文件的位置
- 体系架构

本文档介绍了为 PHP 库创建 PKGBUILD 的过程。本文档的目标读者是 PHP 库的打包人员。对于 PHP Web 应用程序，请参阅 Web application package guidelines

软件包名称

对于模块，软件包名称应以 `php-` 开头，其余名称应通过将库名称转换为小写字母并用连字符分隔来从库名称构造。 例如，与 `File iterator` 相对应的软件包名称将是 `php-file-iterator`。

打包文件的位置

PHP 软件包应将文件安装到 `/usr/share/php/` 中。此路径应位于 php.ini include_path 或 open_basedir 指令中，以便能够在 PHP Web 应用程序中包含库文件。

体系架构

在大多数情况下，`arch` 数组应包含 `'any'`，因为大多数 PHP 软件包都是与体系架构无关的。

1.2　32 位软件包打包准则

难度 入门 | **篇幅** 3.2 KB | **所属** Arch 打包准则

本节将讲到

- 软件包命名
- 变量和参数

可以在不同架构的计算机（例如 x86_64）上构建和安装老式 32 位软件。本文介绍了此类软件包的打包和约定。

软件包命名

- 给 32 位版本的软件包加上前缀 lib32-。
- 给使用 x32 ABI 的软件包加上后缀 `-x32`。
- 软件包描述应与原生版本的相区别，即 `pkgdesc+=" (32-bit)"`。

变量和参数

lib32

在 PKGBUILD 中指定这些 bash 变量来告诉编译器输出 32 位代码：

```
export CFLAGS+=" -m32"
export CXXFLAGS+=" -m32"
export LDFLAGS+=" -m32"
export PKG_CONFIG_PATH='/usr/lib32/pkgconfig'
```

文件放置

确保 lib32 软件包文件与本机软件包文件不冲突，并且包括所有必需的文件，例如特定于架构的包含文件。例如，如果使用 GNU Autoconf构建软件包，则向 `configure` 指定以下内容：

```
--program-suffix="-32" \
--lib{exec,}dir=/usr/lib32 \
--includedir=/usr/include/"$pkgbase"32 \
--build=i686-pc-linux-gnu
```

1.3 Free Pascal 打包准则

难度 入门 | **篇幅** 3.6 KB | **所属** Arch 打包准则

本节将讲到

- Free Pascal

本页说明如何为使用 Free Pascal Compiler (FPC) PKGBUILD。当前存在用于构建 Linux 软件的两个选项，以及使用 FPC 交叉编译器在其他目标上构建软件的少数选项：

- fpc包 提供了仅针对您的主机 CPU（仅x86_64）的编译器。

Free Pascal

软件包命名

仅项目名称通常就足够了。但是，在交叉编译的情况下，当从 multilib 以 i686 Linux 为目标时，包的前缀应该是 `fpc32-` ，当以非 Arch Linux 系统为目标时，包的名称应该是 `fpc-*cpu*-*system*-*pkgname*`。

有用的代码片段

- 确定 FPC 的版本以及要输出的单元的 CPU 和 OS：

```
_unitdir=`fpc -iSP`-`fpc -iSO`
_fpcver=`fpc -iV`
```

打包

制作基于 FPC 的软件包时，请遵守以下规定：

- 始终将 fpc包 添加到 `makedepends` 或 `depends`
- 始终将所有编译单元（\.*a*, \.compiled, \.*o*, \.ppu, \.*res*, \.rst) ）放在 `/usr/lib/fpc/*$_fpcver*/units/*$arch*-linux` 下面。
- 如果安装导入库，请将 `staticlibs` 添加到 `options` 中。

交叉编译

- 始终将上述相应的交叉编译器包（`fpc-*cpu*-*system*-rtl`) ）添加到 `depends`
- 对于非类 Unix 的系统总是添加 `!strip` 到 `options`
- 始终将所有已编译的单元（\.*a*, \.compiled, \.*o*, \.ppu, \.*res*, \.rst) 放在 `/usr/lib/fpc/*$_fpcver*/units/*$_unitdir*`下（或者如果是 multilib，放在 `/usr/lib/fpc/*$_fpcver*/units/i386-linux`）
- 始终使用 `any` （如果是 multilib，则为 `x86_64`）作为体系结构
- 如果安装导入库，请将 `staticlibs` 添加到 `options` 中。

1.4 KDE 软件打包准则

难度 入门 | **篇幅** 4.3 KB | **所属** Arch 打包准则

本节将讲到

- 构建目录
- 安装所需的前缀参数
- 构建类型
- 强制指定 Qt 版本
- KF5 软件包命名

Arch Linux 上的 KDE 软件包符合特定的模式。

构建目录

构建 CMake 软件包的推荐做法是在项目根目录之外新建一个构建目录，并从该目录运行 cmake。PKGBUILD 应如下所示：

```
prepare() {
  mkdir -p build
}

build() {
  cd build
  cmake ../${pkgname}-${pkgver}
}
```

安装所需的前缀参数

每个软件包都必须设置 `CMAKE_INSTALL_PREFIX` 变量，但考虑到 KDE 的自定义构建版本，请使用以下参数：

```
-DCMAKE_INSTALL_PREFIX=$(kf5-config --prefix)
```

当软件包被移动到 *extra* 仓库时，上述参数应改为：

```
-DCMAKE_INSTALL_PREFIX=/usr
```

构建类型

一般来说不应指定构建类型，这样能使 CMake 接受 `CFLAGS`、`CPPFLAGS` 等环境变量。

强制指定 Qt 版本

KF5 软件包命名

Plasma 挂件

Plasma 挂件 (之前名为 Plasmoids) 软件包应当被命名为 `plasma5-applets-*widgetname*`，这样能被识别为与 Plasma 5 相关的软件包，且易于与官方软件包区分。

Runners

Plasma runners 软件包应当被命名为 `plasma5-runners-*runnername*` ，这样能被识别为与 Plasma 5 相关的软件包，且易于与官方软件包区分。

服务菜单

服务菜单软件包应当被命名为 `kf5-servicemenus-*servicename*` ，这样能被识别为与 KF5 相关的软件包。

主题

Plasma 主题软件包应当被命名为 `plasma5-themes-*themename*` ，这样能被识别为与 Plasma 5 相关的软件包。

图标和 .desktop 文件安装

一些 KDE 软件在 hicolor 图标主题和 `.desktop` 文件中提供了图标（必须通过 pacman 钩子安装）。应当避免在 `.install` 中对这些类型的文件使用安装命令，因为这将导致不必要的重复执行。

1.5 Haskell 打包准则

难度 入门 | **篇幅** 4.4 KB | **所属** Arch 打包准则

本节将讲到

- 软件包命名
- 体系架构
- 来源
- 重建顺序
- PKGBUILD 库示例

本文档旨在涵盖在Arch上创建优质 Haskell 软件包标准和准则。

在撰写本文之前，请联系 User:Felixonmars

软件包命名

对于 Haskell 库，使用 `haskell-*libraryname*`，通常与 hackage 相同。

注意：软件包名称应完全小写。

体系架构

参见 PKGBUILD#arch。

每个 Haskell 库或程序都依赖于体系结构。

来源

Haskell程序或库的首选来源是 hackage。PKGBUILD#source `source=()` 数组应使用以下 URL 模板：

```
:https://hackage.haskell.org/packages/archive/$_hkgname/
$pkgver/$_hkgname-$pkgver.tar.gz
```

请注意，因为 Haskell 软件包通常以 haskell- 为前缀，所以使用自定义 _hkgname 变量而不是 pkgname 。通常可以如下定义此变量：

```
_hkgname=stm-delay
```

重建顺序

当 Haskell 库更改时，它的构建标志或被更新，所有依赖包都需要重建。

PKGBUILD 库示例

打包 Haskell 库与打包 Haskell 程序不同，Arch Linux 中打包的库供打包的 Haskell 程序使用。

```
PKGBUILD
# Contributor: Your Name <youremail@domain.com>
_hkgname=stm-delay
pkgname=haskell-stm-delay
arch=('x86_64')
url="https://hackage.haskell.org/package/$hkgname"
depends=(ghc-libs)
makedepends=(ghc)
source=("https://hackage.haskell.org/packages/archive/$_hkgname/$pkgver/$_hkgname-
$pkgver.tar.gz")

build() {
  cd $_hkgname-$pkgver

  runhaskell Setup configure -O --enable-shared --enable-executable-dynamic --disable-
library-vanilla \
    --prefix=/usr --docdir=/usr/share/doc/$pkgname --enable-tests \
    --dynlibdir=/usr/lib --libsubdir=\$compiler/site-local/\$pkgid \
    --ghc-option=-optl-Wl\,-z\,relro\,-z\,now \
    --ghc-option='-pie'

  runhaskell Setup build
  runhaskell Setup register --gen-script
  runhaskell Setup unregister --gen-script
  sed -i -r -e "s|ghc-pkg.*update[^ ]* |&'--force' |" register.sh
  sed -i -r -e "s|ghc-pkg.*unregister[^ ]* |&'--force' |" unregister.sh
}

check() {
  cd $_hkgname-$pkgver
  runhaskell Setup test
}

package() {
  cd $_hkgname-$pkgver

  install -D -m744 register.sh "$pkgdir"/usr/share/haskell/register/$pkgname.sh
  install -D -m744 unregister.sh "$pkgdir"/usr/share/haskell/unregister/$pkgname.sh
  runhaskell Setup copy --destdir="$pkgdir"
  install -D -m644 "LICENSE" "${pkgdir}/usr/share/licenses/${pkgname}/LICENSE"
  rm -f "${pkgdir}/usr/share/doc/${pkgname}/LICENSE"
}
```

1.6 CLR 软件打包准则

难度 入门 | **篇幅** 4.6 KB | **所属** Arch 打包准则

本节将讲到

- 打包注意
- PKGBUILD 示例

本文档定义了在 Arch Linux 下打包 Common Language Runtime （.NET）项目的标准。当前，只有 Mono 能够为多个系统提供可用的，高效的 CLR runtime，并且该标准将反映其用法。请注意，许多 CLR 程序是在考虑到 Microsoft .NET 的情况下开发的，因此，由于 P/Invoke 调用和 Microsoft 数字版权管理（DRM）API 等 .NET 专有的因素，它们可能在 Mono 下运行，也可能无法在 Mono 下运行。因此不会为 Arch Linux 提供可用的软件包。但是，如果与 1.5.6 版（？）或更高版本的 Wine 结合使用，则您的程序包可能有机会在其下运行。如果是这种情况，请参阅 Wine PKGBUILD Guidelines 以获取更多信息。

打包注意

- 始终将 mono包 添加到 `depends`
- 始终设置 `arch` 为 `any`。Mono 尚不支持编译（运行）64-bit 程序集。
- 始终添加 `!strip` 到 `options`
- 如果程序包是一个库（DLL），则要将该程序包用作依赖项，请考虑将其安装到 Mono 的全局程序集缓存（GAC）中。
- 如果程序集是预编译的，并且带有程序调试数据库文件（Foo.dll.pdb），请考虑将其转换为：{ic|pdb2mdb Foo.dll}}
- 如果要执行该程序包（EXE），请确保安装到 `/usr/bin`，使用 shell 脚本以运行它，类似于以下程序：

```
#!/bin/sh
mono foo.exe "$@"
```

签名的程序集

如果要将软件包安装到 GAC 中，请确保它具有签名的密钥文件。如果没有，您可以生成一个这样的：`sn -k 1024 Foo.snk`。随后，嵌入密钥文件到组件的最简单方法是拆卸它，是这样的：`monodis Foo.dll --output=Foo.il`。然后，像这样重新组装它：`ilasm /dll /key:Foo.snk Foo.il`

PKGBUILD 示例

以下示例将尝试涵盖一些最常见的约定和构建系统。

xbuild

未签名的 DLL

```
# Maintainer: yourname <yourmail>
pkgname=foo
pkgver=1.0
pkgrel=1
```

```
pkgdesc="Fantabulous library for .Net"
arch=('any')
url="http://www.foo.bar"
license=('GPL')
depends=('mono')
options=('!strip')
source=("http://www.foo.bar/foobar.tar.gz")
md5sums=('4736ac4f34fd9a41fa0197eac23bbc24')

build() {
  cd "${srcdir}/foobar"

  xbuild Foo.sln

  # if the package is unsigned, do the following:
  cd "/bin/x86/Debug"
  monodis Foo.dll --output=Foo.il
  sn -k 1024 Foo.snk
  ilasm /dll /key:Foo.snk Foo.il
}

package() {
  cd "${srcdir}/foobar/bin/x86/Debug"

  install -Dm644 Foo.dll "$pkgdir/usr/lib/foobar/Foo.dll"
  install -Dm644 Foo.dll.mdb "$pkgdir/usr/lib/foobar/Foo.dll.mdb"

  # Register assembly into Mono's GAC
  gacutil -i Foo.dll -root "$pkgdir/usr/lib"
}
```

1.7 命令行解释器（Shell）打包准则

难度 入门 | **篇幅** 4.6 KB | **所属** Arch 打包准则

本节将讲到

- 安装
- Shell 自动补全
- Shell 函数或模块

安装

Shell 必须在 `/etc/shells` 中列出，以使用户能够更改默认 Shell。大部分的 shell 软件包都带有如下安装脚本：

```
post_install() {
    grep -Fqx /bin/shellname /etc/shells || echo /bin/shellname >>/etc/shells
    grep -Fqx /usr/bin/shellname /etc/shells || echo /usr/bin/shellname >>/etc/shells
}

post_upgrade() {
    post_install
}

post_remove() {
    sed -i -r '/^(\/usr)?\/bin\/shellname$/d' etc/shells
}
```

Shell 自动补全

大部分 shell 都为少部分常用命令内置了一套自动补全，并支持扫描至少一个系统路径下由其它软件包提供的函数。 以下表格列出了软件包的补全文件存放路径和命名规则：

Shell	目录	文件名
Bash	`/usr/share/bash-completion/completions/`	`binary_name`
Elvish	`/usr/share/elvish/lib/`	`binary_name.elv`
Fish	`/usr/share/fish/vendor_functions.d/`	`binary_name.fish`
Zsh	`/usr/share/zsh/site-functions/`	`_binary_name`

其它 shell：

- Nushell 自带了部分补全功能，但暂时不支持通过全局目录提供补全。对于提供 Nushell 自动补全函数的软件包，可以将它们打包到 `/usr/share/nushell/autoload/binary_name.nu`，然后使用 `post_install()` 函数输出提示，让用户自行将 `use /path/to/file` 添加到配置文件中。

提示：一般来说，软件包不得在 *depends*\ 或 *optdepends*\ 中包含 shell。就算软件提供了自动补全文件，也不代表它们对于 shell 有依赖关系。一种例外是软件包不提供自身的补全，且 shell 软件包中也不提供其补全，而是由补充集合包（例如 bash-completion包 和 zsh-completions包）提供。在这种情况下，需要将它们添加到 `optdepends` 中。

Shell 函数或模块

Shell	目录	文件名
Elvish	`/usr/share/elvish/lib/`	`module_name.elv`
Fish	`/usr/share/fish/vendor_functions.d/`	`function_name.fish`
Zsh	`/usr/share/zsh/site-functions/`	`function_name`

1.8 字体打包指南

难度 入门 | **篇幅** 4.6 KB | **所属** Arch 打包准则

本节将讲到

- 通用指引
- 打包
- 示例包
- 参见

本文档提供为字体撰写 PKGBUILD 的建议规范。

通用指引

包命名

- TTF 字体使用`ttf-`前缀。
- OTF 字体使用`otf-`前缀。

若为可变字体，使用`-variable`后缀。

包名应使用全小写。

包描述

至少要表明它是一个字体，并注明字体类别：衬线体、非衬线体、等宽等。

架构

字体是与架构无关的。使用`arch=(any)`。

依赖

字体没有依赖项。但是许多仓库里的包依赖于 fontconfig包 和 xorg-mkfontscale包，这些依赖是以前字体包还需要使用安装脚本更新字体缓存时的历史遗留。现在可以用 pacman 钩子来完成这项工作。安装上述两个包时，`/usr/share/fonts/`下的所有字体都会被缓存，因此无需再强制使用 fontconfig 或 mkfontscale。

提供

许多应用依赖于虚拟包`ttf-font`。若你的字体家族满足该页面的要求，可添加`provides=('ttf-font')`。

来源

可按下列顺序查询：

- 官方网站，或某个如 GitHub 的托管平台
- Google Fonts: `https://fonts.google.com/download?family=*Name*`
- Font Squirrel、Font Library、FFonts、1001 Fonts、 DaFont

以下来源不推荐：

- FontSpace（校验码因不明原因变动）

打包

下列片段是以 OFL（SIL Open Font License）授权的 OTF 字体的示例片段：

```
license=('OFL')
...
```

```
package() {
  install -Dm644 -t "$pkgdir/usr/share/fonts/OTF" "path/to/font/*.otf"
  install -Dm644 -t "$pkgdir/usr/share/licenses/$pkgname" "path/to/license/OFL.txt"
}
```

- OFL理论上是个自定义许可，因此字体包的许可证文件需要安装在`/usr/share/licenses/[包名]`。
- 若有许多个文件，目标目录名可考虑使用包名而非`OTF`。

示例包

- ttf-merriweather-sansAUR
- otf-commissionerAUR - OTF、TTF、可变三种包分离

参见

- 论坛讨论

1.9 R 软件打包准则

难度 进阶 | **篇幅** 6.2 KB | **所属** Arch 打包准则

本节将讲到

- 软件包命名
- 软件包版本号
- Arch
- 依赖
- 来源

本文档涵盖了为 R 软件包编写 PKGBUILD 的标准和指南。大多数信息可以通过查看软件包的 `DESCRIPTION` 文件获得。在 R 中运行 `tools::CRAN_package_db()` 可以获得大部分信息。您还可以访问 CRAN、Bioconductor link1与Bioconductor link2，了解所有 R 软件包的信息。

软件包命名

软件包应命名为 `r-pkgname`，其中 pkgname 取自 `DESCRIPTION` 文件中的`Package`字段。软件包名称应小写。

软件包版本号

从`Version`字段中获取。R 允许软件包在版本号中使用冒号和连字符，但在 PKGBUILD 中不允许。将其转换为句号或下划线。

Arch

请参见 PKGBUILD#arch。如果软件包的 CRAN 网页上有 `NeedsCompilation: yes`，那么它很可能是特定架构的。否则，很可能不是，参见 PKGBUILD#arch。

依赖

在软件包 `DESCRIPTION` 文件的`Depends`、`Imports`或 `LinkingTo` 字段中列出的 R 软件包应列在依赖项下。

`Suggests` 中列出的 R 软件包应列为 optdepends。

有些软件包需要外部工具，这些工具列在 `SystemRequirements` 下。

某些软件包需要依赖 gcc-fortran包，但它并不总是列在 `DESCRIPTION` 文件中。

来源

CRAN 上的所有 R 软件包都可以在网站 `https://cran.r-project.org/src/contrib/*cranname*_*cranversion*.tar.gz` 上找到，其中 `*cranname*` 是 CRAN 上软件包的名称，`*cranversion*` 是 cran 的版本。

Bioconductor 上可用的 R 软件包可从网站 `https://bioconductor.org/packages/release/bioc/src/contrib/*bcname*_*bcname*.tar.gz` 或 `https://bioconductor.org/packages/release/data/annotation/src/contrib/*bcname*_*bcname*.tar.gz` 上获取，其中 `*bcname*` 是 Bioconductor 上软件包的名称，`*bcver*` 是版本。

构建与打包

R 内置支持构建软件包。下面是三个软件源的 `PKGBUILD` 模板： MRAN、CRAN 和 Bioconductor。MRAN 是 CRAN 的快照镜像，使用该模板即使软件包过期也能构建。

MRAN

```
_cranname=
_cranver=
_updatedate=YYYY-MM-DD
pkgname=r-${_cranname,,}
pkgver=${_cranver//[:-]/.}
pkgrel=1
pkgdesc=""
arch=()
url="https://cran.r-project.org/package=${_cranname}"
license=()
depends=(r)
makedepends=()
optdepends=()
source=("https://cran.microsoft.com/snapshot/${_updatedate}/src/contrib/${_cranname}_${_cranver}.tar.gz")
sha256sums=()

build() {
```

```
  R CMD INSTALL ${_cranname}_${_cranver}.tar.gz -l "${srcdir}"
}

package() {
  install -dm0755 "${pkgdir}/usr/lib/R/library"

  cp -a --no-preserve=ownership "${_cranname}" "${pkgdir}/usr/lib/R/library"
}
```

CRAN

```
_cranname=
_cranver=
pkgname=r-${_cranname,,}
pkgver=${_cranver//[:-]/.}
pkgrel=1
pkgdesc=""
arch=()
url="https://cran.r-project.org/package=${_cranname}"
license=()
depends=(r)
makedepends=()
optdepends=()
source=("https://cran.r-project.org/src/contrib/${_cranname}_${_cranver}.tar.gz")
sha256sums=()

build() {
  R CMD INSTALL ${_cranname}_${_cranver}.tar.gz -l "${srcdir}"
}

package() {
  install -dm0755 "${pkgdir}/usr/lib/R/library"

  cp -a --no-preserve=ownership "${_cranname}" "${pkgdir}/usr/lib/R/library"
}
```

Bioconductor

```
_bcname=
_bcver=
pkgname=r-${_bcname,,}
pkgver=${_bcver//[:-]/.}
pkgrel=1
pkgdesc=""
arch=()
url="https://bioconductor.org/packages/${_bcname}"
license=()
depends=(r)
makedepends=()
optdepends=()
source=("https://bioconductor.org/packages/release/bioc/src/contrib/${_bcname}
_${_bcver}.tar.gz")
# or
# source=("https://bioconductor.org/packages/release/data/annotation/src/contrib/
${_bcname}_${_bcver}.tar.gz")
sha256sums=()

build() {
  R CMD INSTALL ${_bcname}_${_bcver}.tar.gz -l "${srcdir}"
}

package() {
  install -dm0755 "${pkgdir}/usr/lib/R/library"

  cp -a --no-preserve=ownership "${_bcname}" "${pkgdir}/usr/lib/R/library"
}
```

技巧与窍门

Bioconductor 存储库

欲轻松访问bioconductor软件包，可以添加 bioarchlinux 软件源。

1.10 Node.js 打包准则

难度 进阶 | **篇幅** 6.7 KB | **所属** Arch 打包准则

本节将讲到

- 软件包命名
- 源代码
- 使用 npm
- 使用 nvm

本文档包含了给 Node.js 软件包编写 PKGBUILD 的规范与指导。

软件包命名

提示：可以用自定义变量 `_pkgname` 代替 `pkgname`，一般这个变量可以这样定义：`_pkgname=${pkgname#nodejs-}`

Node.js 库的软件包名应该以 `nodejs-` 开头。对于独立的应用，只使用程序名。

源代码

npm 提供了可靠的命名规范和下载 URL。PKGBUILD#source `source=()` 列表可以使用下面的 URL 模板：

```
https://registry.npmjs.org/$_pkgname/-/$_pkgname-$pkgver.tgz
```

使用 npm

使用 npm 安装时，应当将其添加为构建时依赖。

```
makedepends=('npm')
```

不需要解压压缩包：

```
noextract=("${_pkgname}-${pkgver}.tgz")
```

这是一个最小的 package) 函数：

```
package() {
    npm install -g --prefix "${pkgdir}/usr" "${srcdir}/${_pkgname}-${pkgver}.tgz"

    # npm 会把所有文件的所有权交给构建用户
    # https://bugs.archlinux.org/task/63396
    chown -R root:root "${pkgdir}"
}
```

设置临时缓存

当 *npm* 处理 `package.json` 时，npm 会为了构建软件包，下载依赖到默认的缓存目录 `$HOME/.npm`。为了避免污染用户的 home 目录，可以用 `--cache` 选项，将缓存目录临时设为其他目录。

下载依赖到 `${srcdir}/npm-cache`，然后安装它们到打包目录：

```
npm install --cache "${srcdir}/npm-cache"
```

继续正常打包：

```
npm run packager
```

软件包包含向 $srcdir/$pkgdir 的引用

很不幸，*npm* 会创建指向 `$srcdir` 和 `$pkgdir` 的引用。这是一个已知问题。但你可以手动删除这些引用，反正也不会用到它们。

所有依赖都会在 `_where` 属性里，包含指向 `$pkgdir` 的引用，你通常可以用一点 *sed* 魔法删除它们：

```
find "$pkgdir" -name package.json -print0 | xargs -r -0 sed -i '/_where/d'
```

你的主软件包也会包含一些其他引用，删除它们最简单的办法就删除所有有下划线的属性，但用 *sed* 不是很方便。你可以使用 jq包 得到想要的结果：

```
local tmppackage="$(mktemp)"
local pkgjson="$pkgdir/usr/lib/node_modules/$_pkgname/package.json"
jq '.|=with_entries(select(.key|test("_.+")|not))' "$pkgjson" > "$tmppackage"
mv "$tmppackage" "$pkgjson"
chmod 644 "$pkgjson"
```

另一个你会找到对 `$pkgdir` 引用的地方，是软件包的 `man` 属性。如果你不在乎 man 页面（反正它们不会被作为依赖安装），你可以像这样删除它们：

```
find "$pkgdir" -type f -name package.json | while read pkgjson; do
    local tmppackage="$(mktemp)"
    jq 'del(.man)' "$pkgjson" > "$tmppackage"
    mv "$tmppackage" "$pkgjson"
    chmod 644 "$pkgjson"
done
```

一个使用了上面全部三个技巧的例子：readability-cliAUR。

使用 nvm

当一个基于 node.js 的应用需要**构建**或**打包**使用不同的 node.js 版本时，可以使用 nvmAUR。

警告：这只适用于应用的构建 / 打包时依赖，不影响运行时依赖.

把它加入构建时依赖：

```
makedepends=('npm' 'nvm')
```

nvmAUR 使用 `NVM_DIR` 环境变量来查找前缀（译注：指 nvm 的安装路径，下同）。如果不在 nvmAUR 初始化前指定，前缀就会被设置为 `$HOME/.nvm`。

你可以用下面的函数创建自定义前缀，并将用户目录和前缀隔离开：

```
_ensure_local_nvm() {
    # let's be sure we are starting clean
    which nvm >/dev/null 2>&1 && nvm deactivate && nvm unload
    export NVM_DIR="${srcdir}/.nvm"

    # The init script returns 3 if version specified
    # in ./.nvrc is not (yet) installed in $NVM_DIR
    # but nvm itself still gets loaded ok
    source /usr/share/nvm/init-nvm.sh || [[ $? != 1 ]]
}
```

应该在使用 nvmAUR、npm包 或其他基于特定版本 Node.js 的程序前，调用这个函数。

示例 PKGBUILD 用法

```
prepare() {
    _ensure_local_nvm
    nvm install 14.15.0
}

build() {
    _ensure_local_nvm
    npm install
}
```

或者，执行不带参数的 `nvm install`，会在当前目录的 `.nvrc` 文件中查找版本字符串。

这个用法的一个例子可以在 insomniaAUR 找到。

1.11 DKMS 软件包打包准则

难度 进阶 | **篇幅** 6.9 KB | **所属** Arch 打包准则

本节将讲到

- 包名
- 依赖
- 源代码构建位置
- 打补丁
- .install 中模块的自动加载

创建一个新的 DKMS 包时，可以参考下面的指导方针。

包名

DKMS 包的命名方式是：原始包名加"\-*dkms*"后缀。

通常在 `$pkgname` 后面使用 `$_pkgname` 记录不包含 "\-*dkms*" 后缀的软件包名 (例如 `_pkgname=${pkgname%-*}`). 这样可以在原始的软件包 PKGBUILD 和 DKMS 编译文件之间保持相似性。

依赖

依赖的包应该是原来软件包的基础上，加上 dkms包。这一点很重要，因为它将提供工具和 Hook 文件，在内核更新时重建由 \-*dkms* 软件包提供的内核驱动程序。

不要在 PKGBUILD 包含 linux-headers包 或任何其他的Linux头文件包。这些头文件已经作为 dkms包 的可选依赖，同时每个内核包都有自己的头文件包，因此在 \-*dkms* 软件包包含这些头文件包是不必要的重复和限制的。

源代码构建位置

构建模块所需源代码需要放在（这是DKMS构建模块时使用的默认目录）：

```
/usr/src/PACKAGE_NAME-PACKAGE_VERSION
```

在软件包目录，要包含一个 `dkms.conf` 配置文件，告诉 DKMS 如何编译。这个配置文件需要包含：

- `PACKAGE_NAME` - 实际的项目名称，通常使用 `$_pkgname` （见#包名）或 `$_pkgbase`.
- `PACKAGE_VERSION` - 通常使用 `$pkgver`.

注意：不需要编译内核模块的源文件，也不需要安装它们；由于 dkms包 中的 pacman#钩子，每当 Linux 内核更新时，这些工作都会自动完成。

打补丁

为内核模块源代码打补丁既可以直接在 PKGBUILD 中进行，也可以通过`dkms.conf` 来进行。

若通过 `dkms.conf` 打补丁，请确保将补丁安装在 `/usr/src/*PACKAGE_NAME*-*PACKAGE_VERSION*/patches/` 目录下，并为每个补丁添加 `PATCH[*number*]=*patch_filename*` 条目，记得将 `*number*` 替换为以0起始的数字。查阅 dkms(8) § DKMS.CONF 以获取相关信息

.install 中模块的自动加载

模块的加载和卸载必须由用户自己来执行，设想一下，某个模块可能在加载的时候崩溃。

另外，**不要在 .install 中调用 dkms**，该操作会由 dkms包 提供的 Pacman 钩子 自动完成。Pacman 会自动执行 `dkms install` 和 `dkms remove` 钩子。

`dkms install` 会确保过程结束时执行 `depmod`。`dkms install` 依赖 `dkms build` (针对当前内核编译源码)，build 依赖 `dkms add` (添加从 `/var/lib/dkms/<package>/<version>/source` 到 `/usr/src/<package>` 的链接)。

例子

这儿有个根据包名字和版本来对`dkms.conf`进行编辑，并安装模块黑名单配置文件的例子。

其它示例请参考官方软件仓库中的 -dkms 软件包和 AUR 中的 -dkms 包。

PKGBUILD

```
PKGBUILD
# Maintainer: foo <foo(at)example(dot)org>
# Contributor: bar <bar(at)example(dot)org>

_pkgbase=example
pkgname=example-dkms
pkgver=1
pkgrel=1
pkgdesc="The Example kernel modules (DKMS)"
arch=('x86_64')
url="https://www.example.org/"
license=('GPL2')
depends=('dkms')
conflicts=("${_pkgbase}")
install=${pkgname}.install
source=("${url}/files/tarball.tar.gz"
        'dkms.conf'
        "${pkgname}.conf"
        'linux-3.14.patch')
md5sums=(use 'updpkgsums')

prepare() {
  cd ${_pkgbase}-${pkgver}

  # Patch
  patch -p1 -i "${srcdir}"/linux-3.14.patch
}

package() {
  # Copy dkms.conf
  install -Dm644 dkms.conf "${pkgdir}"/usr/src/${_pkgbase}-${pkgver}/dkms.conf

  # Set name and version
  sed -e "s/@_PKGBASE@/${_pkgbase}/" \
      -e "s/@PKGVER@/${pkgver}/" \
      -i "${pkgdir}"/usr/src/${_pkgbase}-${pkgver}/dkms.conf

  # Copy sources (including Makefile)
  cp -r ${_pkgbase}/* "${pkgdir}"/usr/src/${_pkgbase}-${pkgver}/

  # Blacklists conflicting module
  install -Dm644 ${pkgname}.conf "${srcdir}/usr/lib/modprobe.d/${pkgname}.conf"
}
```

dkms.conf

```
dkms.conf
PACKAGE_NAME="@_PKGBASE@"
PACKAGE_VERSION="@PKGVER@"
MAKE="make --uname_r=$kernelver"
CLEAN="make clean"
BUILT_MODULE_NAME="@_PKGBASE@"
DEST_MODULE_LOCATION="/kernel/drivers/misc"
AUTOINSTALL="yes"
```

.install

该示例会在安装或升级后显示一条信息，建议用户在不想立即重启系统时，卸载冲突模块（`example-conflicting-module`）并加载该包提供的模块（`example`）。

```
example.install
post_install() {
  cat<<EOF

Unload and load kernel modules:

  rmmod example-conflicting-module
  modprobe example

EOF
}

post_upgrade() {
  post_install
}
```

Module blacklist conf

已知模块 `example-conflicting-module` 会与模块 `example` 冲突时，应将其加入模块黑名单：

```
example-dkms.conf
blacklist example-conflicting-module
```

1.12 Go 语言软件打包准则

难度 进阶 | **篇幅** 7.5 KB | **所属** Arch 打包准则

本节将讲到

- 通用准则
- 构建
- PKGBUILD 范例
- 软件包示例

本文涵盖了为 Go 软件包编写 PKGBUILD 时须遵循的标准和指引。

通用准则

命名

如果软件包提供的程序与 Go 生态存在强关联，请使用 `go-*模块名称*`。对于其它应用，请仅使用程序名。

注意：软件包名称只允许使用小写字符。

构建

依赖

Go 1.11 为 go 模块提供了初步支持，使得上游的 Go 代码可以声明依赖并将其固定到特定项目版本。目前，我们的打包工作利用了这一点来满足 go 模块依赖关系。

上游项目不使用 go 模块

对于不使用 Go 模块的上游代码，可以使用以下变通方法，并请考虑向上游提出 issue：

```
PKGBUILD
url=https://github.com/upstream_user/upstream_project

prepare() {
  cd "$pkgname-$pkgver"
  go mod init "${url#https://}" # strip https:// from canonical URL
  go mod tidy
}
```

注意：由于模块可能会在软件的各次构建期间出现变化，该方法将导致软件包的构建不具有可复现性。

上游项目使用 go 模块

go 默认会使用 `GOPATH` 来下载和存放 go 模块，并导致用户的 `~/go` 目录大小增加。

为将所有 go 模块保留在构建环境中，可以在准备步骤（prepare）中配置 `GOPATH="${srcdir}"`，然后将 go 模块下载到软件包源码目录（srcdir）中：

```
PKGBUILD
prepare() {
  cd "${pkgname}-${pkgver}"
  export GOPATH="${srcdir}"
  go mod download -modcacherw
}
```

构建标志和选项

Go 不会自动将系统的构建标志（例如 `CFLAGS` 和 `LDFLAGS`）传递给 C 工具链。为使用 RELRO 和其它加固标志构建 Go 二进制文件，需要在构建环境中显式指定 `CGO_CFLAGS` 和 `CGO_LDFLAGS` 等相关变量：

```
export CGO_CPPFLAGS="${CPPFLAGS}"
export CGO_CFLAGS="${CFLAGS}"
export CGO_CXXFLAGS="${CXXFLAGS}"
export CGO_LDFLAGS="${LDFLAGS}"
export GOFLAGS="-buildmode=pie -trimpath -ldflags=-linkmode=external -mod=readonly -
modcacherw"

# 或者也可以通过命令行选项指定部分标志
go build \
    -trimpath \
    -buildmode=pie \
    -mod=readonly \
    -modcacherw \
    -ldflags "-linkmode external -extldflags \"${LDFLAGS}\"" \
    .
```

注意：大多数用于 Go 应用的 Makefile 会覆盖掉 `GOFLAGS`，不会将上述标志传递给编译器。如果上游项目使用了 Makefile，则需要通过补丁使其遵循传入的标志，或是直接调用 `go build` 来绕过该文件。

标志功能

- `-buildmode=pie` 会启用 PIE 编译，用于加固二进制文件。
- `-trimpath` 用于可复现构建，可以防止嵌入完整构建路径和模块路径。
- `-mod=readonly` 可以保证模块文件不会在 go 的任何操作中被更改。
- `-modcacherw` 不重要，但是可以保证 go 模块会创建一个可写的路径。默认只会创建一个只读的路径。

提示：如果软件的 `modules.txt` 中包含了一个 `vendor` 目录，那可以将 `-mod` 标志修改为 `-mod=vendor`。

警告：包维护者需要确保构建标志被正确传递给了编译器，请务必检查源码的 `Makefile`。

支持调试包

要启用带有源代码和符号查找的调试包，需要对默认构建标志进行一些修改。

- 删除 `-trimpath` 来确保源码路径被写入二进制文件。
- 由于目前的工具不支持压缩的 DWARF 头，因此需要在 `-ldflags` 中包含 `-compressdwarf=false` 来确保工具能解析 DWARF 头。
- 使用 `-linkmode=external`，因为 go 内部的链接器不会把 build-id 嵌入二进制文件。
- 包含 `GOPATH="${srcdir}"` 以使 *makepkg* 能包含所有模块的源码。

使用上述选项应该能生成包含正确的分离符号和源代码，并能被调试器正确读取的调试包。

```
export CGO_CPPFLAGS="${CPPFLAGS}"
export CGO_CFLAGS="${CFLAGS}"
export CGO_CXXFLAGS="${CXXFLAGS}"
export CGO_LDFLAGS="${LDFLAGS}"
export GOPATH="${srcdir}"
export GOFLAGS="-buildmode=pie -mod=readonly -modcacherw"

go build -ldflags "-compressdwarf=false -linkmode external" .
```

输出目录

有多种方法可以构建项目中的所有 go 二进制文件。

```
build(){
    cd "$pkgname-$pkgver"
    go build -o output-binary .
}
```

`...` 是一个简写，它会让编译器递归遍历所有目录，并找到所有二进制文件。可以将其搭配输出目录使用以构建所有文件：

```
prepare(){
```

```
    cd "$pkgname-$pkgver"
    mkdir -p build
}

build(){
    cd "$pkgname-$pkgver"
    go build -o build ./cmd/...
}
```

PKGBUILD 范例

```
pkgname=foo
pkgver=0.0.1
pkgrel=1
pkgdesc='Go PKGBUILD Example'
arch=('x86_64')
url="https://example.org/$pkgname"
license=('GPL')
makedepends=('go')
source=("$url/$pkgname-$pkgver.tar.gz")
sha256sums=('1337deadbeef')

prepare(){
  cd "$pkgname-$pkgver"
  mkdir -p build/
}

build() {
  cd "$pkgname-$pkgver"
  export CGO_CPPFLAGS="${CPPFLAGS}"
  export CGO_CFLAGS="${CFLAGS}"
  export CGO_CXXFLAGS="${CXXFLAGS}"
  export CGO_LDFLAGS="${LDFLAGS}"
  export GOFLAGS="-buildmode=pie -trimpath -ldflags=-linkmode=external -mod=readonly -
modcacherw"
  go build -o build ./cmd/...
}

check() {
  cd "$pkgname-$pkgver"
  go test ./...
}

package() {
  cd "$pkgname-$pkgver"
  install -Dm755 build/$pkgname "$pkgdir"/usr/bin/$pkgname
}
```

软件包示例

- podman包
- k9s包
- helm包

1.13 Electron 打包准则

难度 进阶 | **篇幅** 8.7 KB | **所属** Arch 打包准则

本节将讲到

- Using the system electron
- Architecture

- Directory structure
- Getting version of Electron

This document covers standards and guidelines on writing PKGBUILDs for Electron.

Using the system electron

Arch Linux provides versioned electron* packages and an electron包 metapackage for the latest version. They can be used to run an electron application via a shell script wrapper:

```
#!/bin/sh

exec /usr/bin/electron34 /path/to/appname/ "$@"
```

The `*appname*/` directory, or alternatively a file bundle called `*appname*.asar`, can be found in a prebuilt electron application as the `resources/app/` folder (or `resources/app.asar`). Everything else is just a copy of the electron runtime and can be removed from the final package.

注意：Applications that require `ELECTRON_RUN_AS_NODE=1`, e.g. Visual Studio Code, cannot use `/usr/bin/electron*`. See code.sh.

Editing version on package.json

It is dangerous to edit version of Electron in `package.json` or `package-lock.json` using `sed`. You can use `npm pkg set devDependencies.electron=$(cat /usr/lib/electron*/version)` instead.

**本文或本章节的事实准确性存在争议。**

原因： Editing package.json may not needed and unrelated to building modules correctly.（在 Talk:Electron 打包准则 中讨论）

Building compiled extensions against the system electron

Some electron applications have compiled native extensions which link to the electron runtime, and must be built using the correct electron version. Since npm/yarn will always build against a private prebuilt copy of electron, patch the electron dependency from `package.json` to reference the same version as the system electron dependency. The build system will download the prebuilt copy it requires, compile the native extensions, and package everything into a final distribution, but this can be pruned during the `package()` step as usual.

Alternatively, you can remove the electron dependency from `package.json` and set the correct environment variables before running npm:

```
export npm_config_target=$(tail /usr/lib/electron/version)
export npm_config_arch=x64
export npm_config_target_arch=x64
export npm_config_disturl=https://electronjs.org/headers[失效链接 2023-10-29 ⓘ]
export npm_config_runtime=electron
export npm_config_build_from_source=true
HOME="$srcdir/.electron-gyp" npm install
```

Set `HOME` to a path inside the `$srcdir` so the build process does not place any files in your real `HOME` directory. Make sure to adjust the path for all further commands that make use of the `.electron-gyp` cache.

(more details in Electron docs).

Using electron-builder with system electron

Many projects use **electron-builder** to build and package the Javascript file and Electron binaries. By default electron-builder downloads the entire electron version that is defined in the package management file (e.g. `package.json`). This might not be desired if you want to use the system electron and save the bandwidth since you are going to throw away the electron binaries anyway. The electron-builder provides the configurations `electronDist` and `electronVersion`, to specify a custom path of Electron and the version the application is packaged for respectively.

Find the electron-builder configuration file (e.g. `electron-builder.json`) and add the following settings:

- `electronDist` to `/usr/lib/electron34` for electron34包
- `electronVersion` to the contents of `/usr/lib/electron34/version` (without the leading `v` if exists)

Packages that apply this: rocketchat-desktopAUR ubports-installer-gitAUR

electron-builder configuration\[失效链接 2024-10-12 ⓘ\]

Alternatively you can use the CLI to change/add these settings like this:

```
./node_modules/.bin/electron-builder --linux --x64 --dir $dist -
c.electronDist=$electronDist -c.electronVersion=$electronVer
```

Note that you have to specify all these options or it will not work.

**本文或本章节的事实准确性存在争议。**

原因： Current setting still copies Electron from $electronDist . It is expired to add the method to stop it.（在 Talk:Electron 打包准则 中讨论）

Architecture

See PKGBUILD#arch.

An Electron package that contains compiled native extensions is architecture-dependent. Otherwise it is most likely architecture-independent.

If the package contains a prebuilt copy of electron, it is always architecture-dependent.

Directory structure

If the package is architecture-dependent, install the `resources/app/` directory to `/usr/lib/*appname*/`. Otherwise use `/usr/share/*appname*/`.

If the package contains a prebuilt copy of electron, copy the final distribution in its entirety to `/opt/*appname*`.

Getting version of Electron

Version of Electron could be given by `npm pkg get devDependencies.electron` in the directory containing `package.json` or `package-lock.json`.

Prebuild or Nonfree application hides version of electron from `package.json`, `package-lock.json`, and `version` files. In such case, you may get version by replacing `app` or `app.asar` with `/usr/lib/electron/resources/default_app.asar` and running `*pathto/electron-binary* --version`.

注意：Avoid doing it at PKGBUILD. This should be last resort to make packages with system-wide electron.

1.14 Rust 软件打包准则

难度 进阶 | **篇幅** 8.9 KB | **所属** Arch 打包准则

本节将讲到

- 软件包命名
- 源码
- 依赖关系
- 准备工作
- 构建

本文档涵盖了为 Rust 软件编写 PKGBUILD 的标准和指南。

软件包命名

打包 Rust 项目时，软件包的名称应基本与生成的二进制文件名一致。请注意，打包库单元包 (library crate) 没有任何意义，只有带二进制文件的单元包 (crate) 才需要打包。对于生成多个二进制文件的包，通常使用上游单元包 (crate) 的名称较为合适。在任何情况下软件包的名称都应该小写。

源码

大多数 Rust 项目可以从 tarball、源归档文件（例如 GitHub Release 上的源链接）或其他已发布的源构建。另外，许多项目发布在 crates.io 上，该网站给 cargo 提供了稳定的 URL 下载方案。如果需要，PKGBUILD#source 可使用以下模板：

```
source=("$pkgname-$pkgver.tar.gz::https://static.crates.io/crates/$pkgname/$pkgname-$pkgver.crate")
```

依赖关系

虽然有些 Rust 项目有外部依赖，但大多数项目只使用在最终二进制文件中静态链接的 Rust 生态系统库。因此，大多数项目不需要指定很多 `depends` (依赖)，只需要 `makedepends` (构建依赖)。绝大多数 Rust 项目都使用 cargo 依赖管理器构建，它会协调下载库以满足编译时的依赖关系，并执行所有必要的调用，使用真正的 Rust 编译器 `rustc` 进行编译。目前 *cargo* 和 *rustc* 都由 rust包 提供，但也有其他方法可同时获取或单独获取这两个包，包括 rustup包。因此，大多数 PKGBUILD 都会调用 *cargo* 工具，应该直接依赖它。

```
makedepends=(cargo)
```

如果项目需要使用开发版 Rust 工具链，请使用：

```
makedepends=(cargo-nightly)
```

准备工作

Rust 依赖管理器 cargo 能提前下载构建项目所需的所有库。在 `prepare()` 阶段进行下载，能使后续的 `build()` 和其他阶段完全离线运行。

```
prepare() {
    export RUSTUP_TOOLCHAIN=stable
    cargo fetch --locked --target "$CARCH-unknown-linux-gnu"
}
```

其中：

- `RUSTUP_TOOLCHAIN=stable` 确保默认工具链被设置为稳定版，以防用户改变默认值。当然，如果在上游项目有特殊要求，则应被设置为开发版。此操作可避免在未使用 chroot 环境构建时受用户配置影响。另外，如果上游项目的源代码中有 `rust-toolchain` 文件或 `rust-toolchain.toml` 文件可实现此目的，则无需此步骤。
- `--locked` 确保严格遵守 `Cargo.lock` 文件中指定的版本，防止其更新依赖关系。这对可重现构建很重要。
- `--target "$CARCH-unknown-linux-gnu"` 确保只获取正在构建的特定目标平台所需的依赖项，从而减少下载量（参见 PKGBUILD#arch 和 Rust 平台支持）。

注意：在为上游源项目构建 VCS 软件包时，如果该项目在发布周期之间未保持 `Cargo.lock` 文件与 `Cargo.toml` 同步，请在运行 `cargo fetch` 之前添加 `cargo update`。尽管结果并非完全可重现，但有关构建的其他所有方面都应符合上述内容，因为依赖关系将在构建时得到解决。

构建

构建 Rust 软件包。

```
build() {
    export RUSTUP_TOOLCHAIN=stable
    export CARGO_TARGET_DIR=target
    cargo build --frozen --release --all-features
}
```

其中：

- `--release` 确保使用发布模式编译 (默认使用调试模式编译)
- `--frozen` 确保 *cargo* 保持离线状态，只使用 `Cargo.lock` 文件中指定的版本和 `prepare()` 阶段缓存的版本。这在功能上与 `--locked --offline` 一致。这对可重现构建很重要。
- `--all-features` 确保编译时启用软件包的所有特性。另外，如果只需启用指定特性，可使用 `--features *FEATURE1*, *FEATURE2*` 。
- `CARGO_TARGET_DIR=target` 指定输出路径为当前目录下的 *target* 目录，以防未使用 chroot 环境构建时受用户配置影响。

注意：对于 Arch 仓库的软件包来说不需要这两个环境变量，因为 Arch 仓库的软件包总是在 chroot 环境下使用默认设置下构建。但为了方便 AUR 用户，此处仍提到了上述环境变量，因为 AUR 用户可能没有意识到改变用户默认设置和在构建软件包时不使用 chroot 的后果。

检查

大多数 Rust 项目提供了运行测试的简单方法。

```
check() {
    export RUSTUP_TOOLCHAIN=stable
    cargo test --frozen --all-features
}
```

还应该检查仓库是否是 cargo 工作空间。只需打开 `/Cargo.toml`，检查是否包含 `[workspace]` 部分。如果包含 `[workspace]`，则应当在 `cargo test` 后添加 `--workspace`，以确保所有工作区成员的测试都能运行。

运行测试时应避免使用 `--release` 标志，否则二进制文件将会使用 *bench* 参数重新编译，并覆盖 `build()` 生成的文件。或者也可以保留 `--release` 标志，但使用不同的 `CARGO_TARGET_DIR`。但需要注意的是，使用发布模式也会启用编译器优化，并禁用一些功能，如整数溢出检查和 `debug_assert!()` 宏，所以理论上可能最终问题会更少。这两种方法都会重新编译依赖项，从而使总编译时间略微增加。

打包

Rust 在 `target/release` 目录内构建二进制文件，将二进制文件安装到 `/usr/bin` 很容易。

```
package() {
    install -Dm0755 -t "$pkgdir/usr/bin/" "target/release/$pkgname"
}
```

如果一个软件包在 /usr/bin 内有多个可执行文件，可使用 find 命令：

```
package() {
    find target/release \
        -maxdepth 1 \
        -executable \
        -type f \
        -exec install -Dm0755 -t "$pkgdir/usr/bin/" {} +
}
```

使用 cargo install 的注意事项

有些软件包需要安装更多的文件，如手册页面或其他必要的文件。如果没有其他方式来安装这些文件，可使用 cargo install。在这种情况下不必使用 build()，因为即使该软件包已通过 cargo build 构建，cargo install 仍会强制重新构建。但仍然可使用 prepare() 提前获取源代码：

```
package() {
    cd "$pkgname-$pkgver"
    export RUSTUP_TOOLCHAIN=stable
    cargo install --no-track --frozen --all-features --root "$pkgdir/usr/" --path .
}
```

应始终使用 --no-track 参数，否则 cargo install 将创建不必要的文件，如 /usr/.crates.toml 或 /usr/.crates2.json。

软件包示例

PKGBUILD 样例请参阅软件包页面上的 *Package Actions* > *Source Files*。

- cbindgen包
- ripgrep包

1.15 Java 打包准则

难度 进阶 | **篇幅** 9.4 KB | **所属** Arch 打包准则

本节将讲到

- 介绍
- 典型 Java 应用程序的结构
- 在 Arch Linux 中打 Java 包

本文档定义了 Arch Linux 下打包 Java 程序的建议标准。Java 程序是出了名的难以在没有重叠的依赖关系的情况下干净利落地打包。本文档描述了一种处理这种情况的方法。这些指南十分灵活，以涵盖处理 Java 程序时遇到的许多不同情况。

介绍

Arch Linux 的打包人员似乎无法就如何处理 Java 包达成一致。官方和非官方软件库以及 AUR 的 PKGBUILD 中使用了各种方法。这包括把所有东西扔到 /opt 里，把

Shell 脚本放在 /usr/bin 或把配置文件放在 /etc/profile 中。其他的则是放在 /usr/share 目录中，脚本放在 /usr/bin 中。许多做法在系统 CLASSPATH 和 PATH 中添加了不必要的文件。

典型 Java 应用程序的结构

大多数桌面 Java 应用程序结构类似。它们是由一个不依赖系统（但依赖软件包！）的安装程序安装的。这通常会把所有东西都安装在包含 bin、lib、jar、conf等子目录的目录下。一般会有一个包含主要可执行类的主 jar 文件。通常还会提供一个 Shell 脚本来运行主类，这样用户就不必直接调用 Java 解释器。这个 Shell 脚本通常相当复杂，因为它需要在不同的发行版中通用，而且经常包括不同系统的特殊情况（例如 Cygwin）。

lib 目录通常包含捆绑的 jar 文件来满足 Java 程序的依赖。这使用户安装程序变得简单（其中包含了所有依赖），但却是软件包开发者的噩梦。几个软件包捆绑相同的依赖会浪费空间。这在过去并不是一个大问题，当时桌面 Java 应用程序和库较少，而那些存在的应用程序和库本身往往也非常大。但现在情况不同了……

运行程序所需的其他文件通常与主 jar 文件存放在同一目录或其子目录下。由于 Java 程序不知道它们的类是从哪里加载的，所以通常需要在这个目录中运行（即 Shell 脚本应 cd 进入该目录），或者设置一个环境变量来指示这个目录的位置。

在 Arch Linux 中打 Java 包

在 Arch 中打包 Java 应用程序将会比现在更费劲。然而，这是值得的，它可以使文件系统更干净，捆绑的依赖更少（随着越来越多的Java 库被重构为单独的包，打包会变得更容易）。在创建 Arch Linux Java 包时，应遵循以下准则：

- 如果 Java 库有一个通用的名字，那么包的名字应该在前面加上 java- 以与其他库区分。对于命名独特的包（如 JUnit）、终端用户程序（如 Eclipse）或可以用其他前缀唯一描述的库（如 jakarta-commons-collections 或 apache-ant），则不需要这样做。
- 将所有随程序分发的 jar 文件（而不是其他文件）放在 /usr/share/java/*程序*目录下。这包括随程序分发的所有依赖 jar 文件。然而，应该努力将常见的或大型的依赖放到单独的包中。这只有在程序不依赖某个特定版本的库时才能实现。

: 这个规则使得迭代重构依赖关系成为可能。也就是说，软件包和它的所有依赖可以先放在一个目录中。经过测试后，主要的依赖可以一个一个地被重构出来。请注意，有

些应用程序在主 jar 文件中包括捆绑的依赖。也就是说，他们将捆绑的依赖解包，并将其包含在主 jar 中。这样的依赖通常非常小，重构它们没有什么意义。

- 如果程序是要由用户运行的，编写一个自定义的 Shell 脚本运行主 jar 文件。这个脚本应该放在 `/usr/bin` 中。库一般不需要 Shell 脚本。从头开始编写 Shell 脚本，不要使用程序捆绑的脚本。删除检查自定义环境（如 Cygwin）的代码，以及试图确定 `JAVA_HOME` 是否已经设置的代码（Arch 不使用 `JAVA_HOME`，而是使用 `archlinux-java` 来设置 `/usr/bin/java` 符号连接）。

: 寻找 jar 文件的脚本应该类似这样：：```

exec /usr/bin/java -jar '/usr/share/java/程序/程序.jar' "$@"

```
: 寻找单个类文件的应该类似这样：
: ```
#!/bin/sh
exec /usr/bin/java '/usr/share/java/程序/程序类名' "$@"
```

- 使用 `-cp` 选项为 Java 解释器设置 `CLASSPATH`，除非有明确的理由不这样做（即：`CLASSPATH` 用于插件机制）。`CLASSPATH` 应该包括 `/usr/share/java/*程序*目录下的所有 jar 文件，以及已被重构到其他目录中的依赖库的 jar 文件。可以使用类似以下代码：

: ``` for name in /usr/share/java/程序/*.jar ; do CP=$CP:$name done
CP=$CP:/usr/share/java/dep1/dep1.jar java -cp $CP 程序.java.MainClass

```
- 确保 Shell 脚本可执行!

- 随软件包一起分发的其他文件应存放在 `/usr/share` 下以该软件包命名的目录中。你可能需要在 Shell 脚本中的 `项目_HOME` 等变量中设置该目录的位置。本准则假设程序希望所有文件都在同一目录下（这是 Java 软件包的标准做法）。如果配置文件更适合放在其他地方（例如，把日志放在 `/var/log` 中），则可以这样做。

: 请记住，在某些系统上 `/usr` 可能被挂载为只读。如果应用程序需要写入共享目录中的文件，它们可能需要被重新定位到`/etc`、`/var` 或用户的主目录。

- 与其他 Arch Linux 软件包的标准一样，如果遵守上述标准需要做大量的工作，则应以其偏好的方式安装软件包，并将生成的目录放在 `/opt` 中。这对于捆绑 JRE、包含自定义版本的依赖或做其他奇怪和痛苦的事情的程序很有用。

### 多个API实现

如果软件包分发了常用的 API 实现（比如 jdbc 驱动），则应该把库放在 `/usr/share/java/*API_名称*` 下。这样，允许用户选择实现的应用程序会知道在哪里寻找它们。这个位置只用于库包本身。如果这样的实现是应用程序分发的一部分，**不要**将这个 jar 文件放在公共位置，而是使用普通的包结构。

### 目录结构示例

为了说明问题，下面是一个假想的名为 `foo` 程序的目录结构示例。由于 `foo` 是一个常见的名字，所以软件包被命名为 `java-foo`，但注意目录结构中并没有体现：
```

```
- `/usr/share/java/foo/`
- `/usr/share/java/foo/foo.jar`
- `/usr/share/java/foo/bar.jar`（`java-foo` 包含的依赖）
- `/usr/share/foo/`
- `/usr/share/foo/*.*`（`java-foo` 需要的一些文件）
- `/usr/bin/foo`（可执行的 Shell 脚本）

### 依赖

Java packages might specify `java-runtime` or `java-environment` as dependency, based on what they need. For most packages, `java-runtime` (*Java Runtime Environment*) is what is needed to simply run software written in *Java*. `java-environment` (*Java Development Toolkit*) is needed by packages that will need to compile *Java* source code into bytecode.

更多信息参见 [Java](../../06_软件与应用/Oracle/Java.md)。
```

1.16 交叉编译工具打包准则

难度 进阶 | **篇幅** 9.7 KB | **所属** Arch 打包准则

本节将讲到

- 重要提示
- 版本兼容性
- 构建交叉编译器
- 软件包命名
- 放置文件

本页描述了如何打包交叉编译器工具链。另一种交叉编译的方法是在多种架构上使用 distcc。参见 Distcc#使用 distcc 交叉编译。

提示：打包交叉编译器还可使用 crosstool-ng，以完全自动化的方式创建自己的工具链。详情请见 crosstool-ng包。

重要提示

受以下软件包启发，本文介绍新的打包方法：

- mingw-w64-gcc包 以及 `mingw-w64-*` 系列的其他软件包
- arm-none-eabi-gcc包 以及 `arm-none-eabi-*` 系列的其他软件包
- `arm-wince-cegcc-*` 系列的其他软件包

版本兼容性

警告：使用不兼容版本的软件包编译工具链必然会失败。一般来说，不同版本之间都不兼容。

建议按照以下策略选择 gcc、binutils、内核和 C 库的兼容版本：

- 总则：
- gcc 和 binutils 的版本相关，建议使用同时发布的版本；

- 建议使用最新的内核头文件来编译 libc，但使用 `--enable-kernel`（只保证适用于 glibc，其他 C 库可能约定不同）能强制在旧内核上工作；
- 官方仓库：可能需要额外的修正，但 Arch Linux（或其特定架构的分支）使用的版本很可能可以协同工作；
- 软件文档：所有 GNU 软件都有 `README` 和 `NEWS` 文件，涵盖了依赖关系的最低版本要求等内容；
- 其他发行版：同样也进行交叉编译
- https://trac.clfs.org 涵盖了构建交叉编译器的必要步骤以及一些依赖的最新版本。

构建交叉编译器

构建交叉编译器的一般方法是：

1. binutils：构建为目标架构链接、处理的 cross-binutils。
2. 头文件：安装目标架构的 C 库和内核头文件。
3. 参考 linux-api-headers包，并将 `ARCH=*target-architecture*` 参数传给 *make*
4. 打包 libc 头文件 (此处描述了 Glibc 的处理方法)
5. gcc 第 1 阶段：构建基本的（第 1 阶段）gcc 交叉编译器。这将被用来编译 C 库，但是几乎不能编译其他任何东西（因为无法链接不存在的 C 库）。
6. libc：构建交叉编译的 C 库（使用第 1 阶段的交叉编译器）。
7. gcc 第 2 阶段：构建完整的（第 2 阶段）C 交叉编译器。

头文件和 libc 的源代码因平台而异。

提示：根据需求的不同，具体操作步骤也不尽相同。例如，如果想创建内核和 glibc 版本与 Arch Linux 系统相同的 "克隆版"，可不构建头文件直接将 `--with-build-sysroot=/` 传给 `configure`。

软件包命名

软件包名称*不*应以 `cross-` 开头（之前建议使用前缀 `cross-`，但可能由于包名太长，在正式的软件包中未被采用），而应由软件包名称组成，前缀为 GNU triplet，不含厂商字段或在厂商字段中含 "unknown"。例如：`arm-linux-gnueabihf-gcc`。如果存在更短的命名规则（例如：`mips-gcc`），可以使用但并不建议。

放置文件

最新版本的 gcc 和 binutils 对系统根目录和库使用不冲突的路径。可执行文件应放在 `/usr/bin/` 中。为防止冲突，应在所有文件前加上架构名称。

通常来说，`./configure` 应至少含以下参数：

```
_target=your_target
_sysroot=/usr/lib/${_target}
...
./configure \
    --prefix=${_sysroot} \
    --sysroot=${_sysroot} \
    --bindir=/usr/bin
```

其中 `*your_target*` 可自行修改，例如修改为 "i686-pc-mingw32"。

示例

此处以 MinGW 的 binutils 的 PKGBUILD 为例。需要注意的是：

- 指定交叉编译环境的根目录
- 使用 `${_pkgname}`、`${_target}` 和 `${_sysroot}` 能使代码更易读
- 删除重复或冲突的文件

```
# Maintainer: Allan McRae <allan@archlinux.org>

# cross toolchain build order: binutils, headers, gcc (pass 1), w32api, mingwrt, gcc
(pass 2)

_target=i686-pc-mingw32
_sysroot=/usr/lib/${_target}

pkgname=${_target}-binutils
_pkgname=binutils
pkgver=2.19.1
pkgrel=1
pkgdesc="MinGW Windows binutils"
arch=('i686' 'x86_64')
url="http://www.gnu.org/software/binutils/"
license=('GPL')
depends=('glibc>=2.10.1' 'zlib')
options=('!libtool' '!distcc' '!ccache')
source=(http://ftp.gnu.org/gnu/${_pkgname}/${_pkgname}-${pkgver}.tar.bz2)
md5sums=('09a8c5821a2dfdbb20665bc0bd680791')

build() {
  cd ${srcdir}/${_pkgname}-${pkgver}
  mkdir binutils-build && cd binutils-build

  ../configure --prefix=${_sysroot} --bindir=/usr/bin \
    --with-sysroot=${_sysroot} \
    --build=$CHOST --host=$CHOST --target=${_target} \
    --with-gcc --with-gnu-as --with-gnu-ld \
    --enable-shared --without-included-gettext \
    --disable-nls --disable-debug --disable-win32-registry
  make
  make DESTDIR=${pkgdir}/ install

  # clean-up cross compiler root
  rm -r ${pkgdir}/${_sysroot}/{info,man}
}
```

注意：在交叉编译工具链构建过程中，每次都从独立的目录中执行 `configure` 和 *make* 命令（所谓的树外编译），并在 PKGBUILD 发生变化后删除整个 `src` 目录（即使变化很小）。

疑难解答

为什么不安装到 /opt?

有以下两个原因：

1. 首先，根据文件系统层次结构标准 (FHS)，这些文件只位于 `/usr` 的某个位置。
2. 其次，安装到 `/opt` 是退而求其次的选择。

"路径外的可执行文件 (out-of-path executables)" 是什么?

这看起来很奇怪，但能让交叉编译更容易。有时项目的 Makefile 不使用 `CC` 或其他变量，而是直接使用 **gcc** 。在尝试交叉编译这样的项目时，编辑 Makefile 可能会很麻烦。然而，将 `$PATH` 改为优先使用我们的可执行文件即可解决此问题，只需将 `make` 改为执行 `PATH=/usr/*arch*/bin/:$PATH make`。

故障排除

编译失败但没有明确的提示应该如何解决?

如果是在运行 `configure` 时发生的错误，参见 `$srcdir/*pkgname* - build/config.log`。如果是在编译过程中发生的错误，请向上滚动控制台日志或搜索 "error" 一词。

\[error message\] 错误意味着什么?

很可能犯了一些不明显的错误：

- 配置标志位过多或过少。尝试使用已证明有效的标志位。
- 依赖关系被破坏。例如，binutils 文件放错位置或丢失，可能导致 gcc 配置过程中出现晦涩难懂的错误信息。
- 未在 `build()` 函数中添加 `export CFLAGS=""`（参见 GCC Bugzilla 中的 bug 25672）。
- 某些 `--prefix` 或 `--with-sysroot` 参数组合可能要求目录可写。
- 系统根目录缺少内核头文件或 libc 头文件。
- 如果通过搜索引擎仍无法解决，建议立刻放弃当前配置，尝试更稳定、更成熟的配置。

为什么文件安装位置会不正确?

执行 `make install` 命令的各种方法会产生不同的结果。例如，有些 make 目标可能不提供 `DESTDIR` 支持，而是要求使用`install_root`。对于 `tooldir`、`prefix` 和其他类似参数也是如此。有时提供参数而不是环境变量，例如

```
./configure CC=arm-elf-gcc
```
而不是

```
CC=arm-elf-gcc ./configure
```
反之亦然，可能会导致不同的结果（通常是因为 configure/make 递归地调用自身）。

另请参阅

- https://wiki.osdev.org/GCC_Cross-Compiler (English)

1.17 Arch 打包准则/安全

难度 进阶 | **篇幅** 10.3 KB | **所属** Arch 打包准则

本节将讲到

- 用法
- RELRO
- Stack canary
- NX
- PIE

本文描述了适用于 Arch Linux 软件包的打包安全指引。对于 C/C++ 项目，可以为编译器和链接器应用安全加固选项。Arch Linux 默认会启用 PIE、FORTIFY_SOURCE、栈保护、nx 和 relro。

用法

可以使用 checksec包 检查强化加固保护选项：

```
$ checksec --file=/usr/bin/cat
```
提示：Namcap 也可以报告本文描述的一些安全问题。

RELRO

RELRO 是用于加固 ELF 二进制文件和进程的一种通用缓解技术。在程序加载时，链接器需要写入几个 ELF 内存区段，但在将控制权交给程序前，可以将这些区段设为只读，由此防止攻击者覆盖掉部分 ELF 区段。有两种 RELRO 模式：

- 部分 RELRO（`-Wl,-z,relro`）：程序加载后部分区段被设为只读，但 GOT（`.got.plt`）仍为可写。
- 完全 RELRO（`-Wl,-z,now`）：程序加载时解析所有动态符号，使得整个 GOT 都可被设为只读。

如果应用汇报使用了部分 RELRO，需检查构建工具是否使用了我们传递的 LDFLAGS，以及是否允许覆写 LDFLAGS。对于 Go 软件包，需检查构建方法是否将 `build.go` 作为了纯 golang Makefile 替代使用，该方法不允许传递 LDFLAGS。

Haskell

暂不清楚如何为 Haskell 实现完全 RELRO。

Go

参见 Go 语言软件打包准则#Flags and build options。

Stack canary

Stack canary（又称栈警惕标志）由编译器添加在栈的缓冲区和控制数据之间。如果该已知值被破坏，就代表出现了缓冲区溢出，运行中的程序就会报出段错误（segfaults），以防止执行潜在的错误代码。

gcc包 包默认通过 \--enable-default-ssp 编译选项启用了栈保护。

NX

C/C++

可执行空间保护会将内存区域标记为非可执行，使得在该区域执行机器码时会出现报错。该功能使用了类似 NX 位（No-eXecute bit，禁止执行位）的硬件功能，有时也会使用软件模拟这些特性。

PIE

C/C++

gcc包 包默认通过 \--enable-default-pie 选项为 C/C++ 启用了该特性。

Golang

将以下标志传递给 `go build`：

```
export GOFLAGS='-buildmode=pie'
export CGO_CPPFLAGS="-D_FORTIFY_SOURCE=3"
export CGO_LDFLAGS="-Wl,-z,relro,-z,now"
```

Haskell

将以下标志传递给 `runhaskell Setup.hs configure`：

```
--ghc-option='-pie'
```

RPATH/RUNPATH

RUNPATH/RPATH 为包含该参数的对象提供了更多的搜索路径（适用于可执行文件和共享对象）：

```
$ objdump -x /usr/bin/perl | grep -E 'RPATH|RUNPATH'
```

如果 RPATH 的值包含了攻击者可控制的路径，那就有可能通过对应目录下的恶意库执行代码（例如 CVE-2006-1566 和 CVE-2005-4280）。更多信息请参考 Debian:RpathIssue。

RPATH 是通过向 LDFLAGS 传递类似 `-Wl,-rpath -Wl,/usr/local/lib` 的参数，然后由链接器负责设置的。如果要设置 RUNPATH，请将 `--enable-new-dtags` 添加到链接器标志中。

FORTIFY

Fortify source 是一个宏，可以为执行内存和字符串操作的函数添加缓冲区溢出保护。它会检测是否有攻击者尝试通过负责更多的字节来溢出缓冲区，并停止程序的运行。该保护项已在默认 `CPPFLAGS` 中启用：

```
makepkg.conf
CPPFLAGS="-D_FORTIFY_SOURCE=3"
```

更多信息请参考 makepkg#配置。

systemd 服务

**此页面或章节适合移动到 systemd/Sandboxing。**

附注： 内容主题与 systemd#应用程序环境沙盒 相同，两者可被汇总到单独页面，草稿可参考 User:NetSysFire/systemd sandboxing。（在 Talk:Security#systemd unit hardening and system.conf tweaks 讨论）

如果上游不提供 systemd 服务文件，使得需随软件包附带，请检查是否需要使用以下 systemd 服务加固功能。Systemd 提供并为服务启用了分析安全特性的方法：

```
$ systemd-analyze security reflector.service
```

文件访问

可以通过限制文件系统访问来加固服务。

为执行的进程配置一个新的文件系统命名空间，并在内部挂载私有 `/tmp` 和 `var/tmp` 目录，使得其不与命名空间外部的进程共享。该方法适用于会向 `/tmp` 目录写入数据的程序：

```
PrivateTmp=true
```

ProtectSystem 参数对于执行中进程有三种只读挂载方式，其中 “full” 选项会将 `/usr`、`/boot` 和 `/etc` 目录挂载为只读。ProtectHome 会使进程无法访问 `/home`、`/root` 和 `/run/user` 目录：

```
ProtectSystem=strict
ProtectHome=true
```

PrivateDevices 会为执行中的进程配置新的 `/dev` 命名空间，其中只包含类似 `/dev/null`、`/dev/zero` 和 `/dev/random` 的 API 伪设备，不包含任何物理设备、系统内存、系统端口和其它设备。该选项可以防止进程直接向物理设备进行写入，systemd 还会为 `@raw-io` 集添加系统调用过滤器：

```
PrivateDevices=true
```
以下选项会使执行中进程无法修改可通过 `/proc/sys` 和 `/sys` 等路径访问的内核变量。ProtectControlGroups 会使 `/sys/fs/cgroup` 层次结构变为只读：

```
ProtectKernelTunables=true
ProtectControlGroups=true
```
以下方法可禁止访问部分文件路径：

```
InaccessiblePaths=/etc
```
更多信息请参考 systemd.exec(5)。

用户

确保执行中的进程和其子进程无法通过 execve(2) 获得新权限：

```
NoNewPrivileges=true
```
内存

禁止创建同时为可写和可执行的内存映射、将映射修改为可执行和创建可执行共享内存。该方法会将进程沙箱化，防止攻击者向可执行的内存部分写入数据。注意，该方法与使用 JIT) 的应用不兼容：

```
MemoryDenyWriteExecute=true
```
系统调用

锁定 personality(2) 系统调用，使内核执行域无法被修改：

```
LockPersonality=true
```
还可以限制服务的系统调用，以下命令将显示 systemd 可以过滤的系统调用：

```
$ systemd-analyze syscall-filter
```
可以使用预定义组，以下示例为使用推荐的系统服务调用白名单配置起点：

```
SystemCallFilter=@system-service
```
还可以按架构来限制系统调用，例如限制在 64 位设备上执行 32 位二进制文件（即非原生架构二进制文件）：

```
SystemCallArchitectures=native
```
网络

如果进程不需要访问网络，可以为进程配置一个新的网络命名空间，并只配置回环接口，以此完全禁用网络访问：

```
PrivateNetwork=true
```
如果需要网络，还可以限制 socket(2) 系统调用可使用的地址族类型，以只允许 UNIX socket 为例：

```
RestrictAddressFamilies=AF_UNIX
```
也可以对只需连接到 localhost（本机）或特定 IP 段的情况进行限制，以只允许 localhost 为例：

```
IPAddressAllow=localhost
IPAddressDeny=any
```

更多网络过滤相关的信息请参考 systemd.resource-control(5)。

其它

为执行的进程配置新的 UTS 命名空间，并禁止修改主机名或域名：

```
ProtectHostname=true
```

1.18 VCS 软件打包准则

难度 进阶 | **篇幅** 11.0 KB | **所属** Arch 打包准则

本节将讲到

- 包名
- 版本
- 冲突、提供与依赖
- 认证与安全
- VCS 源代码

版本控制系统可用于获取常规静态版本软件包的源代码，以及开发分支的最新（主干）版本。

提示：使用 pacman包 软件包提供的 `/usr/share/pacman/PKGBUILD-vcs.proto` 原型文件。

包名

除非软件包获取的是特定发布版本，否则应添加后缀 `-bzr`、`-cvs`、`-darcs`、`-git`、`-hg` 或 `-svn` 等。

版本

若因修改依赖关系、URL 或源代码导致生成的软件包发生变化，请将 `pkgver` 更新至最新版本。若自上次更新 `PKGBUILD` 以来 `pkgver` 未发生变化，则应递增 `pkgrel` 的值。

建议采用以下版本格式：`*`发行（`release`）`*.r*`修订（`revision`）`*`，其中 `*`修订`*` 是唯一标识源代码树的单调递增数字（VCS 修订号即符合此特性）。若无公开发行版本且仓库无标签，可用 0 作为发行版本号，或完全省略 `*`发行`*` 部分直接使用形如 `r*`修订`*` 的版本号。若有公开发行版本但仓库未打标签，开发者需通过解析项目文件等方式获取发行版本号。

修订号分隔符（`*`修订`*` 之前的 `r`）至关重要，可避免上游首次发布版本或使用不同组件数量的版本时出现的问题。例如，若在修订版 `455` 时上游决定发布 `0.1` 版本，修

订号分隔符能保持版本单调性：`0.1.r456 > r454`。若无分隔符将导致单调性失效：`0.1.456 < 454`。

提示：可使用 `--holdver` 选项阻止 makepkg 更新 `pkgver`，详见 makepkg(8)。

冲突、提供与依赖

- 明确声明软件包冲突与提供，例如 fluxbox-gitAUR 应包含：

: ``` conflicts=('fluxbox') provides=("fluxbox=${pkgver}")

: > **注意：**关于版本控制的通用建议（尤其适用于 VCS 软件包），请参阅 [PKGBUILD#provides](../包开发/PKGBUILD.md#provides)。

- `replaces=()` 通常会引起不必要的问题，应避免使用。

- 在 `makedepends=()` 中包含对应的版本控制工具——[cvs](https://archlinux.org/packages/?name=cvs)包、[subversion](https://archlinux.org/packages/?name=subversion)包、[git](https://archlinux.org/packages/?name=git)包 等。

认证与安全

- 使用 cvsroot 时，优先采用 `anonymous:@` 而非 `anonymous@`，以避免输入空密码；若必须提供密码则使用 `anonymous:password@`。
- 由于源代码并非静态文件，需在 `sha256sums=()` 中通过添加 `'SKIP'` 跳过硬校验。

VCS 源代码

> **注意：**Arch 官方不支持使用稀疏克隆（sparse clone）或浅克隆（shallow clone），相关讨论参见 [FS#34677](https://bugs.archlinux.org/task/34677) 及[邮件列表](https://lists.archlinux.org/archives/list/aur-requests@lists.archlinux.org/thread/4IM3GOPMSIW4IULKB5DTIY3FAOIQKIN6/#O3OB7N3R2G7QMRBDCPE7SYCYVS7VL5JS)。

VCS 源代码需在 `source` 数组中指定，其处理方式与其他类型源代码相同。makepkg 会将仓库克隆/检出/分支（clone/checkout/branch）到 `$SRCDEST` 目录（若未在 [makepkg.conf(5)](https://man.archlinux.org/man/makepkg.conf.5) 中设置，则默认使用 `$startdir`），然后以特定于各 VCS 的方式将其复制到 `$srcdir`。本地仓库不会被修改，因此无需使用 `-build` 目录。

`source` 数组的通用格式为：
source=('[folder::][vcs+]url[#fragment]')

- `folder`（可选）—文件夹名称，用于将默认仓库名称更改为更具意义的名称（例如替代 `trunk`），或保留先前源代码路径
- `vcs+`—当 URL 未体现 VCS 类型时需要添加，例如 `git+https://some_repo`
- `url`—远程或本地仓库的 URL
- `#fragment`（可选）—片段，用于拉取特定分支或提交。各 VCS 支持的片段格式请参阅 [PKGBUILD(5) § USING VCS SOURCES](https://man.archlinux.org/man/PKGBUILD.5#USING_VCS_SOURCES)

使用 [Git](../../07_开发与工具/版本控制系统/Git.md) 时的 `source` 数组示例：
source=('project_name::git+https://project_url#branch=project_branch')

> **警告：**请勿在 `folder` 字段中使用 `pkgver` 变量，因为该变量可能在 `pkgver()` 函数调用时被修改，导致后续函数无法访问已创建的文件夹。

pkgver() 函数

通过专用的 `pkgver()` 函数可实现自动版本更新。这种方式能更精细地控制版本号，维护者应尽量采用有意义的版本格式。使用该函数时，仍需预先声明更改 PKGBUILD 时最新的 `pkgver` 变量值，makepkg 工具会自动调用 `pkgver()` 并更新该变量。

Bazaar

pkgver() { cd "$pkgname" printf "r%s" "$(bzr revno)" }

r830

Git

基于最后提交可达的最近的带注释的标签生成版本号：

pkgver() { cd "$pkgname" git describe --long --abbrev=7 | sed 's/\([^-]*-g\)/r\1/;s/-/./g' }

2.0.r6.ga17a017

基于最后提交可达的最近的未注释的标签生成版本号：

pkgver() { cd "$pkgname" git describe --long --tags --abbrev=7 | sed 's/\([^-]*-g\)/r\1/;s/-/./g' }

0.71.r115.gd95ee07

若 [git-tag(1)](https://man.archlinux.org/man/git-tag.1) 名称不含连字符，可使用简化 [sed(1)](https://man.archlinux.org/man/sed.1) 表达式 `sed 's/-/.r/;s/-/./'`。

当标签包含前缀（如 `v` 或项目名）时需进行截取：

pkgver() { cd "$pkgname"

git describe --long --abbrev=7 | sed 's/^foo-//;s/\([^-]*-g\)/r\1/;s/-/./g' }

6.1.r3.gd77e105

无可用标签时使用历史提交计数：

pkgver() { cd "$pkgname" printf "r%s.%s" "$(git rev-list --count HEAD)" "$(git rev-parse --short=7 HEAD)" }

r1142.a17a017

省略 [SHA-1](https://zh.wikipedia.org/wiki/SHA-1) 哈希，仅保留版本号与修订号（注意此时若版本号混乱，将无法快速定位到具体修订版本）：

git describe --long --abbrev=7 --tags | sed 's/\([^-]\)-g./r\1/;s/-/./g'

可组合使用两种方法以支持开始无标签、后期添加标签的仓库（使用了 bash 特性）：

pkgver() { cd "$pkgname" (set -o pipefail git describe --long --abbrev=7 2>/dev/null | sed 's/\([^-]*-g\)/r\1/;s/-/./g' || printf "r%s.%s" "$(git rev-list --count HEAD)" "$(git rev-parse --short=7 HEAD)") }

0.9.9.r27.g2b039da # 存在标签时 r1581.2b039da # 无标签时回退方案

Mercurial

pkgver() { cd "$pkgname" printf "r%s.%s" "$(hg identify -n)" "$(hg identify -i)" }

r2813.75881cc5391e

Subversion

pkgver() { cd "$pkgname" local ver="$(svnversion)" printf "r%s" "${ver//[[:alpha:]]}" }

r8546

> **注意：**应优先采用项目正式发布版本（如果有），而非使用 `0.` 占位符。

回退方案

当无法从仓库提取有效版本时，可使用当前日期：
pkgver() { date +%Y%m%d }

20130408

> **注意：**此方法无法唯一标识代码状态，应尽量避免使用。

提示与技巧

Git 子模块

Git 子模块的处理需要特别注意。核心思路是将子模块自身的 URL 直接添加到 `sources` 数组，然后在 `prepare()` 阶段进行引用。

下游项目开发者可能不会使用与上游仓库相同的子模块命名。要查看 Git 子模块名称，需访问项目仓库中的 `.gitmodules` 文件。例如，上游开发者命名为 `*lib-dependency*` 的仓库，在下游 `.gitmodules` 中可能注册为 `*libs/libdep*` 子模块。
[submodule "libs/libdep"] path = libs/libdep url = https://example.org/lib-dependency/lib-dependency.git

source=("git+https://example.org/main-project/main-project.git" https://example.org/lib-dependency/lib-dependency.git)

prepare() { cd main-project git submodule init git config submodule.libs/libdep.url "$srcdir/lib-dependency" git -c protocol.file.allow=always submodule update }

Git 大文件存储（LFS）

Git 大文件存储（LFS）需要一些额外配置：
makedepends=(... 'git-lfs')

prepare() { git lfs install --local git remote add network-origin https://example.org/upstream/lfs/repo git lfs fetch network-origin git lfs checkout }

这在子模块中使用 LFS 时也可用：
prepare() { git submodule init git config submodule.libs/libdep.url "$srcdir/lib-dependency" git -c protocol.file.allow=always submodule update

git -C libs/libdep lfs install --local git -C libs/libdep remote add network-origin https://example.org/upstream/lfs/repo git -C libs/libdep lfs fetch network-origin git -C libs/libdep lfs checkout }

Git 校验和

当通过 `git+https://domain.invalid/repository.git#tag=v1.0.0` 引用稳定的 Git 标签或特定提交作为源时，可以在 `PKGBUILD` 中指定校验和。只需使用 `makepkg -g` 或 `updatepkgsums` 生成校验和，操作方式与其他非 Git 源相同。

1.19 CMake 软件打包准则

难度 进阶 | **篇幅** 12.9 KB | **所属** Arch 打包准则

本节将讲到

- 典型用法
- CMake 的不良行为
- 前缀和库安装目录
- 提示和技巧
- 模板

本文描述了为使用 cmake包 的软件编写 PKGBUILD 时需遵循的标准与指引。

摘自 CMake 网站：

: CMake is an open-source, cross-platform family of tools designed to build, test and package software. CMake is used to control the software compilation process using simple platform and compiler independent configuration files, and generate native makefiles and workspaces that can be used in the compiler environment of your choice.

典型用法

典型用法是运行 `cmake` 命令，然后执行构建命令。`cmake` 命令通常会设置一些参数，检查所需的依赖关系并创建构建文件，从而使该软件可以通过诸如 `make` 和 `ninja` 的其他工具进行构建。

CMake 的不良行为

由于其内部的生成构建文件的特性，有时 CMake 可能会以不良方式运行。因此，在为基于 CMake 的软件编写 PKGBUILD 时，应注意一些步骤。

CMake 可能会自动覆盖默认的编译器优化标志

使用 `-DCMAKE_BUILD_TYPE=Release` 选项运行 CMake 是很常见的。一些上游项目甚至无意中在其构建指令中包含此选项，但这会产生不希望的行为。

每个构建类型都会导致 CMake 自动将一组标志附加到 `CFLAGS` 和 `CXXFLAGS`。当使用通用 `Release` 构建类型时，它会自动附加 `-O3` 编译器优化标志，这将覆盖当前为 `-O2`（在 makepkg 配置文件中定义）的默认 Arch Linux 标志。这是不可取的，因为它偏离了 Arch Linux 的目标优化级别。

关于 -O3 的注意事项

使用 `-O3` 不能保证软件的性能会更好，有时甚至会减慢程序的速度。在某些情况下，它也会破坏软件。Arch Linux 开发者选择 `-O2` 作为目标优化级别是经斟酌后的结果，不应进行改动。除非你确切知道自己在做什么，或者如果上游明确告诉或暗示需要 `-O3`，否则应该避免在我们的软件包中使用它。

修复自动优化标志覆盖

由于 CMake 的灵活性，以 100% 完全解决这个问题并不简单。请注意，不存在可适用于所有情况的标准解决方案。本节将讨论可能的解决方案和应遵守的一些要点。

默认的 CMake 构建类型是 `None`，默认情况下它不会向 `CFLAGS` 和 `CXXFLAGS` 附加任何标志，因此只要省略 `CMAKE_BUILD_TYPE` 就默认使用 `None` 类型。但请注意，忽略此选项并不能保证解决问题，因为如果命令行中未设置 `CMAKE_BUILD_TYPE`，许多软件项目会自动将 CMake 文件中的生成类型设置为 `Release` 或其他类型。另外请注意，由于 `None` 构建类型中缺少 `NDEBUG` 定义，软件包结果中可能会有对源文件的引用，并导致 makepkg 输出 `WARNING: Package contains reference to $srcdir` 警告。

由于默认的 `None` 构建类型在默认情况下不会向 `CFLAGS` 和 `CXXFLAGS` 追加任何标志，因此也可以使用 `-DCMAKE_BUILD_TYPE=None` 选项。一般来说，使用 `-DCMAKE_BUILD_TYPE=None` 选项比省略 `CMAKE_BUILD_TYPE` 要好，它默认不会附加任何标志，也很少有软件会为 `None` 构建类型设置不需要的标志。

不幸的是，该问题不是简单使用 `-DCMAKE_BUILD_TYPE=None` 就能解决的。当使用 `None` 构建类型修复 `-O3` 问题时，可能会遇到另一个问题。很多软件项目会在 CMake 文件中为 `Release` 构建类型定义一些必需的编译器标志（例如 `CMAKE_C_FLAGS_RELEASE` 和 `CMAKE_CXX_FLAGS_RELEASE` CMake 变量）。如果使用 `None` 构建类型，在没有这些上游定义标志的情况下编译时，此类软件可能会损坏或出现错误行为。为了确定是否缺少某些标志，需要检查 CMake 文件，或者可以比较 `make VERBOSE=1` 对于 `None` `Release` 构建类型的输出。如果 `None` 构建类型导致某些上游定义的标志丢失，该怎么办？这时存在两种情况：如果你使用了 `Release` 构建类型，那么就可能会使用到不需要的 `-O3` 标志；如果使用 `None` 构建类型，那就会丢失必需的上游定义标志。该问题不存在标准解决方法，需要逐案分析。如果上游为 `Release` 构建类型定义了 `-O2`，则可以使用 `-DCMAKE_BUILD_TYPE=Release` （参见下文）。否则，可以尝试修补 CMake 文件。

一些软件项目在其 CMake 文件中为 `Release` 构建类型硬编码了 `-O2`，因此，如果您确定 `-O2` 是正在使用的优化级别，那么在这种情况下可以安全地设置 `-DCMAKE_BUILD_TYPE=Release`。

警告：

- 使用 `None` 类型时，某些软件可能会崩溃。当使用 `None` 时请测试软件，检查软件是否损坏或缺少功能。
- 某些软件可能只适用于 `Release` 构建类型。你需要实验和测试软件。

验证修复

通过启用生成工具的详细模式，可以验证 CMake 是否正确应用了修复。例如，当使用 `make`（这是 CMake 的默认值）时，可以通过将 `VERBOSE=1` 添加到 `make` 中（比如 `make VERBOSE=1`）来完成。这将使 `make` 能够输出正在执行的编译器命令。然后，您可以运行 makepkg 并检查输出，看看编译器是否使用了 `-D_FORTIFY_SOURCE=2` 和 `-O2` 标志。如果在每个命令行中显示多个优化标志，则该行中的最后一个标志将是编译器使用的标志（这意味着 `-O2` 需要是最后一个优化标志才能生效）。

前缀和库安装目录

标准 Arch Linux `/usr` 前缀可以由 `-DCMAKE_INSTALL_PREFIX=/usr` CMake 选项指定。通常需要这样做，因为许多软件默认将文件安装到 `/usr/local` 前缀中。

一些上游项目将 CMake 文件设置为将库安装到 `/usr/lib64` 目录中。如果是这种情况，可以使用 `-DCMAKE_INSTALL_LIBDIR=lib` CMake 选项将库安装目录正确设置为 `/usr/lib`。

提示和技巧

指定目录

自从 CMake 3.13 版本开始，可以使用 `-B` 来自动创建构建目录。这样可以避免使用单独的 `mkdir` 或 `install` 命令创建生成目录。`-S` 选项指定源目录（即搜索 `CMakeLists.txt` 文件的位置），并避免在执行 `cmake` 之前使用 `cd` 进入源码树。将这两个选项结合在一起，可以快捷指定生成目录和源目录。

构建 CMake 项目一般需要使用很多选项，可以把它们放到构建函数的本地数组中进行指定。这可以避免使用反斜杠将长命令切割为多行，还可以为各选项单独进行注释：

```
PKGBUILD
build() {
```

```
  local cmake_options=(
    -B build
    -S $pkgname-$pkgver
    # Any other options required to build a project may follow
    [other_cmake_options]
  )
  cmake "${cmake_options[@]}"
  cmake --build build
}
```

减少可能不需要的输出

`-Wno-dev` 选项将抑制某些警告的输出，这些警告仅适用于编写 `CMakeLists.txt` 文件的上游项目开发人员。删除这些警告将使 CMake 输出更流畅，并减少检查它的负担。一般来说，打包者通常可以被安全地忽略这些警告。

从二进制文件中删除不安全的 RPATH 引用

有时生成的二进制文件可能在 `RPATH` 中包含不安全的引用。这可以通过在构建的包上运行 Namcap 来验证，并且这是一个需要修复的安全问题。使用 `CMAKE_SKIP_INSTALL_RPATH=YES` 或 `CMAKE_SKIP_RPATH=YES` CMake 选项很有可能解决此问题。你需要尝试这两个选项，并查看哪一个选项在对应软件中可用（不需要同时使用）。

获取所有可用的 CMake 选项

要获取软件项目可用的所有“可见” CMake 选项，请在源码树（主 `CMakeLists.txt` 文件所在位置）中执行 `cmake -LAH`。

如果要保存输出以供以后参考，可以将其重定向到文件：

```
$ cmake -LAH >options.txt 2>&1
```

避免在构建时使用 FetchContent 进行下载

CMake 提供了 FetchContent 模块，可以在构建时下载额外的资源和子项目。但在理想情况下，所有资源都应在 `sources` 数组中指定，并由 makepkg 在构建之前获取。可以通过 FETCHCONTENT_SOURCE_DIR_<uppercaseName> 选项指定需获取的文件的路径，另外还可以使用 FETCHCONTENT_FULLY_DISCONNECTED=ON 跳过所有构建时的下载，无视任何 `FetchContent` 定义。

示例

假设项目需要获取的资源名为 `foo`：

```
CMakeLists.txt
FetchContent_Declare(
    foo
    URL https://example.com/foo.tar.gz
    URL_HASH SHA256=cf051bf611a94884ba5e4c2d03932d14e83875c5b77f0fdf55c404cad0e4a6e6
)
FetchContent_MakeAvailable(foo)
```

可以将该资源添加到 `sources` 数组并在生成构建文件时进行声明，而不是在构建时下载：

```
PKGBUILD
sources=(
    ...
    "https://example.com/foo.tar.gz"
)

sha256sums=(
    ...
    "cf051bf611a94884ba5e4c2d03932d14e83875c5b77f0fdf55c404cad0e4a6e6"
)
$ cmake -B build -S "$pkgname-$pkgver" -DFETCHCONTENT_FULLY_DISCONNECTED=ON -
DFETCHCONTENT_SOURCE_DIR_FOO="$srcdir/foo"
```

模板

下面是 `build()` 函数的通用模板，它是基于 CMake 的包的起点。假设软件基于 C 和 C++，且在 CMake 文件中没有为 `Release` 构建类型定义必需编译器标志：

注意：check 函数只在 `CMakeLists.txt` 中使用了 `enable_testing()` 和/或 `add_test()` 功能时可用。

```
PKGBUILD
build() {
  local cmake_options=(
    -B build
    -S $pkgname-$pkgver
    -W no-dev
    -D CMAKE_BUILD_TYPE=None
    -D CMAKE_INSTALL_PREFIX=/usr
  )
  cmake "${cmake_options[@]}"
  cmake --build build
}

check() {
  local excluded_tests=""
  local ctest_flags=(
    --test-dir build
    # show the stdout and stderr when the test fails
    --output-on-failure
    # execute tests in parallel
    --parallel $(nproc)
    # exclude problematic tests
    --exclude-regex "$excluded_tests"
  )
  ctest "${ctest_flags[@]}"
}

package() {
  DESTDIR="$pkgdir" cmake --install build
}
```

不要忘记将 cmake包 添加到 makedepends。

范例包

- i2pd包
- libmysofa包

另请参见

- CMake 文档
- 最新版本的文档参考
- 维基百科文章
- cmake(1)

1.20 Python 打包准则

难度 进阶 | **篇幅** 15.1 KB | **所属** Arch 打包准则

本节将讲到

- 包名
- 架构
- 源代码
- 安装方式
- 测试

本文档描述了使用 PKGBUILD 来为 Python 程序进行打包时须遵守的标准和指引。

包名

对于 Python 3 的**库**模块，或与 Python 生态紧密耦合的**应用程序**（如 *pip* 和 *tox*），使用 `python-*`模块名或应用程序名`*` 命名。对于其他**应用程序**，则仅使用程序原名。

注意：包名应全为小写。

架构

参见 PKGBUILD#arch。

包含 C 扩展的 Python 软件包与架构相关。反之则通常无关于架构。

使用 setuptools 构建的软件包通过在 `setup.py` 中定义 `ext_modules` 关键字来声明其使用的 C 扩展。

源代码

注意：根据 RFC0020，默认应使用上游提供的源代码压缩包，而非 PyPI 提供的 sdist 压缩包。

PyPI 网站提供的下载链接包含不可预测的哈希值，每次更新软件包时需重新从 PyPI 获取。这导致其不适合直接用于 PKGBUILD。PyPI 提供了以下稳定方案：PKGBUILD#source `source=()` 数组应使用以下 URL 模板：

源代码包：：`https://files.pythonhosted.org/packages/source/${_name::1}/${_name//-/_}/${_name//-/_}-$pkgver.tar.gz` **纯 Python wheel 包**：`https://files.pythonhosted.org/packages/py2.py3/${_name::1}/$_name/${_name//-/_}-$pkgver-py2.py3-none-any.whl`（同时兼容 Python 2 和 Python 3）：`https://files.pythonhosted.org/packages/py3/${_name::1}/$_name/${_name//-/_}-$pkgver-py3-none-any.whl`（仅兼容 Python 3）：注意分发包名称（distribution name，在 PyPI 注册的名称）可能包含连字符，但在 wheel 文件名中需转换为下划线。 **架构相关的 wheel 包**：可通过追加下划线和架构名添加额外数组（如 `source_x86_64=('...')`）。另可使用 `_py=cp310` 避免重复 Python 版本：：`https://files.pythonhosted.org/packages/$_py/${_name::1}/$_name/${_name//-/_}-$pkgver-$_py-${_py}m-manylinux1_x86_64.whl`

注意使用自定义变量 `**_name**` 代替 `pkgname`，因 Python 软件包通常带有 `python-` 前缀。该变量可通过以下方式定义：

```
_name=${pkgname#python-}
```

安装方式

Python 中的包通常使用 Python 专用的包管理器（如 pip）进行安装，这类工具会从在线仓库（通常是 PyPI —— Python 软件包索引）获取软件包并追踪相关文件。

然而，在使用 `PKGBUILD` 管理 Python 软件包时，需要将 Python 包“安装”到临时位置 `*$pkgdir*/usr/lib/python*<Python 版本>*/site-packages/*$pkgname*`。

对于使用标准元数据（standard metadata）在 `pyproject.toml` 中指定了构建后端（build backend）的 Python 包，使用 python-build包 和 python-installer包 是完成上述“安装”过程最容易的方式。旧版本软件包可能未声明使用 setuptools，仅提供需手动调用的 `setup.py`。

注意：必须将元数据中的依赖关系声明在 `depends` 数组中，否则相关依赖不会被安装。

基于标准（PEP 517）的安装方式

提示：当从上游提供的源代码 tarball 构建时，若项目依赖 git 生成版本字符串，在构建 wheel 前需根据使用的工具将特定的环境变量设为 `$pkgver`：

- python-flit-core包、python-hatch-vcs包 或 python-setuptools-scm包：`SETUPTOOLS_SCM_PRETEND_VERSION`
- python-pbr包：`PBR_VERSION`

- python-pdm-backend包：`PDM_BUILD_SCM_VERSION`

基于标准的工作流程很简单：使用 python-build包 构建一个 wheel，再使用 python-installer包 将其安装到 `$pkgdir`：

注意：除 python-build包 和 python-installer包 外，还需将软件包使用的构建后端（build backend）添加至 `makedepends`。仓库中所有可用的构建后端均属于 python-build-backend包组。请检查项目的 `pyproject.toml` 文件中 `build-system.build-backend` 配置项的值，该值即为此项目实际使用的构建后端，若未配置则默认使用 python-setuptools包。

```
makedepends=(python-build python-installer python-wheel)

build() {
    cd $_name-$pkgver
    python -m build --wheel --no-isolation
}

package() {
    cd $_name-$pkgver
    python -m installer --destdir="$pkgdir" dist/*.whl
}
```

其中：

- `--wheel` 表示仅构建 wheel 文件，不生成源代码分发包
- `--no-isolation` 表示使用系统已安装的包（含 `depends` 中声明的依赖）进行构建，默认会创建隔离的虚拟环境执行构建
- `--destdir="$pkgdir"` 可防止直接尝试安装到宿主系统（而非软件包目录），避免权限错误
- 可将 `--compile-bytecode=……` 或 `--no-compile-bytecode` 传递给 `installer`，但默认值较为合理，无需手动指定

注意：不建议跳过 `build` 阶段而直接将 `.whl` 文件放入 `source` 数组，应优先采用源代码构建。仅在无法通过源码构建时（例如项目**仅**提供 wheel 源）才应使用此方式。

提示：若软件包为 VCS 包（`python-……-git`），请在 `prepare` 函数中添加 `git -C "${srcdir}/${pkgname}" clean -dfx` 命令。此操作可清除旧版 wheel 及其他构建产物，避免后续问题。另请参阅 setuptools 与 Poetry 的上游 issue。

使用 setuptools 或 distutils 的安装方式

若项目中不存在 `pyproject.toml` 文件，或该文件未包含 `[build-system]` 表，则说明项目使用遗留格式，即通过 *setup.py* 文件调用 *setuptools* 或 *distutils.core* 的 `setup` 函数。

此类软件包通常仍可使用上述 python-build包 和 python-installer包 方法构建安装（推荐方式），但需在 `makedepends` 中添加 python-setuptools包。

通过直接运行 *setup.py* 的旧方式（见下）仍可构建安装，但此方法已弃用，仅建议在 PEP 517 兼容方式因故不可用时作为备选方案。

需注意，使用此方法构建时会在 `package` 阶段输出以下警告：

```
SetuptoolsDeprecationWarning: setup.py install is deprecated.
```

另请注意，Python 3.12 及后续版本的标准库中已移除 *distutils*。对于仍使用 *setup.py* 的项目，必须在 `makedepends` 中添加 python-setuptools包（该软件包提供了自带的 *distutils* 实现）。

```
makedepends=('python-setuptools')

build() {
    cd $_name-$pkgver
    python setup.py build
}

package() {
    cd $_name-$pkgver
    python setup.py install --root="$pkgdir" --optimize=1
}
```

其中：

- `--root="$pkgdir"` 作用同前文 `--destdir`
- `--optimize=1` 预生成优化字节码文件（*.opt-1.pyc*）而非在宿主系统按需生成，以便由 pacman 追踪这些文件
- 添加 `--skip-build` 可以跳过不必要的重复构建步骤（如 `build()` 函数已执行过的构建过程）

若软件包使用 python-setuptools-scm包，构建时可能报错如下：

```
LookupError: setuptools-scm was unable to detect version for /build/python-
jsonschema/src/jsonschema-3.2.0.

Make sure you're either building from a fully intact git repository or PyPI tarballs.
Most other sources (such as GitHub's tarballs, a git checkout without the .git folder)
don't contain the necessary metadata and will not work.
```

需将 `SETUPTOOLS_SCM_PRETEND_VERSION` 环境变量设为 `$pkgver` 以完成构建：

```
export SETUPTOOLS_SCM_PRETEND_VERSION=$pkgver
```

测试

注意：

- 避免使用 `tox` 运行测试套件，因其明确设计用于依据从 PyPI 下载的可复现的、固定的配置环境测试，而不会测试软件包实际安装后的版本，若使用则将使 *check* 函数失去意义
- 避免在 **checkdepends** 中添加用于代码检查（lint）、覆盖率或类型检查的 pytest 插件（详见 #禁用 pytest 附加选项章节）。这些工具会增加引导难度且非发行版打包必需，因其不验证功能正确性

- 在为测试配置自定义 `PYTHONPATH` 时，避免使用相对路径，否则可能导致问题出现。请使用绝对路径。

大多数提供测试套件的 Python 项目使用 unittest 运行器、nosetests 或 pytest（分别由 python包、python-nose包 和 python-pytest包 提供），通过检测特定格式的文件/目录名和类/函数名（如含 `test` 的命名）自动发现测试套件。通常直接运行 `nosetests` 或 `pytest` 即可执行测试套件。

```
check(){
    cd $_name-$pkgver

    # 使用内置的 unittest
    python -m unittest discover -vs .

    # 使用 nosetests
    nosetests

    # 使用 pytest
    pytest
}
```

若存在已编译的 C 扩展模块，测试时需通过 `$PYTHONPATH` 指定与当前 Python 主次版本匹配的构建路径以正确加载模块。

```
check(){
    cd $_name-$pkgver
    local python_version=$(python -c 'import sys; print("".join(map(str,
sys.version_info[:2])))')

    # 使用内置的 unittest
    PYTHONPATH="$PWD/build/lib.linux-$CARCH-cpython-$python_version" python -m unittest
discover -vs .

    # 使用 nosetests
    PYTHONPATH="$PWD/build/lib.linux-$CARCH-cpython-$python_version" nosetests

    # 使用 pytest
    PYTHONPATH="$PWD/build/lib.linux-$CARCH-cpython-$python_version" pytest
}
```

提示和技巧

获取 Python 版本号

在准备、构建、测试或安装过程中，如需引用系统 Python 的主次版本号（如 `3.9` 或 `3.10`），请勿硬编码版本号，而应通过 Python 解释器动态获取并存储至局部变量：

```
check(){
  local python_version=$(python -c 'import sys; print(".".join(map(str,
sys.version_info[:2])))')
  ...
}
```

获取 site-packages 路径

在构建、测试或安装过程中，如需引用系统的 `site-packages` 目录，请勿硬编码路径，而应通过 Python 解释器动态获取并存储至局部变量：

```
check(){
  local site_packages=$(python -c "import site; print(site.getsitepackages())")
  ...
}
```

避免在 site-package 中放置测试目录

避免在 `site-packages/` 下直接安装名为 `tests/` 的目录（即 `/usr/lib/python*X*.*Y*/site-packages/tests/`），否则可能引发软件包冲突。某些使用 *setuptools* 的 Python 项目存在错误配置，可能将其测试目录作为顶层 Python 包包含。若发现此类问题，请向项目提交 issue 请求修复，例如此 issue。

禁用 pytest 附加选项

运行 pytest 时，通常应禁用额外插件。特别是代码检查（lint）和覆盖率（coverage）插件在打包场景中可能适得其反，因其行为变更可能导致测试失败。

推荐通过命令行覆盖配置选项（而非修改 pytest 配置文件）来禁用 `addopts` 等参数，以减少维护成本。要清空所有附加选项，可使用：

```
pytest -o addopts=""
```

修复 meson-python 的可复现性问题

使用 meson-python包 作为 PEP 517 构建后端时，其随机生成的构建目录路径会导致可复现性问题。可通过 `-Cbuild-dir` 参数硬编码构建目录来规避此问题：

```
python -m build --wheel --no-isolation -Cbuild-dir=build
```

使用已安装的软件包运行测试

有些软件包需要安装后才能正常测试。在这种情况下（例如 python-narwhals包），可以将构建的软件包安装到*虚拟环境*下，并在该环境中运行测试：

```
python -m venv --system-site-packages test-env
test-env/bin/python -m installer dist/*.whl
test-env/bin/python -P -m pytest
```

1.21 Arch 打包准则

难度 进阶 | **篇幅** 17.1 KB | **所属** Arch 打包准则

本节将讲到

- 打包规则
- 软件包命名
- 软件包版本号
- 软件包依赖
- 软件包关联

Arch 打包准则

---

32 位 – 安全 – CLR – CMake – DKMS – Eclipse – Electron – Free Pascal – GNOME – Go – Haskell – Java – 交叉编译工具 – KDE – Lisp – Meson – MinGW – 内核模块 – Node.js – Nonfree – OCaml – Perl – PHP – Python – R – Ruby – Rust – Shell – VCS – Web – Wine – 字体

翻译状态：

- 本文（或部分内容）译自 Arch packaging standards，最近一次同步于 2026-05-02，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

在为 Arch Linux 构建软件包时，您应该遵循以下的**软件包指导原则**，尤其是当打算贡献新软件包至 Arch Linux 时。同时需要阅读 PKGBUILD(5) 和 makepkg(8) man 手册。

本页中列出的重要项将不再于其它软件包指导页中指出，这些指导页将作为以下准则的附加内容。

可在 `/usr/share/pacman/` 目录下的 *.proto* 文件中查看 PKGBUILD 示例。

注意：提交到 Arch 用户软件仓库 (AUR) 的软件包需要额外遵守 AUR 提交准则。

打包规则

- **永远别**将软件包安装至 `/usr/local/`
- **除非非用不可，否则绝对不要在 `PKGBUILD` 中自定义和使用新的变量**，以避免和 *makepkg* 本身的变量冲突。
- 在非用不可的情况下，需要**给自定义变量名前加上下划线 (_)**。例如：

```
_customvariable=
```

- 任何情况下都要**避免**使用 `/usr/libexec/`，应该使用 `/usr/lib/${pkgname}/`。
- 包信息文件中的 `packager` 字段可以通过修改 `/etc/makepkg.conf` 文件由编译者进行**自定义**。使用 `~/.makepkg.conf`也可以达到此目的。
- **不要使用 makepkg 子程序**（例如 `error`，`msg`，`msg2`，`plain` 和 `warning`），这些随时都可能会更改。请使用 `printf` 或 `echo` 来输出内容。
- 所有安装过程中所需输出的重要的信息，都要放到 **.install** 文件中。比如说如果某软件包需要扩展的安装步骤才能正常运行，你可以将这些步骤的介绍包含在 .install 文件中。
- **依赖包**是最容易出现错误的地方。请花时间仔细核对，例如对动态可执行文件执行 ldd(1)，`ldd --unused --function-relocs` 和 readelf(1) §

dynamic，检查脚本需要的工具，查看软件文档等。namcap 可以帮助你，它可以解析 PKGBUILD 文件和构建出的软件压缩包，报告权限错误、缺少依赖、过多依赖等常见问题。

- 任何运行该软件包不需要，或者该软件包的通用功能不需要的**可选依赖**不要加入到 **depends** 中。这些信息应该加入 **optdepends** 数组，例如：

: ``` optdepends=('cups: printing support' 'sane: scanners support' 'libgphoto2: digital cameras support' 'alsa-lib: sound support' 'giflib: GIF images support' 'libjpeg: JPEG images support' 'libpng: PNG images support')

: 例子取自 [wine](https://archlinux.org/packages/?name=wine)包 软件包。这些信息在安装和升级时会自动打印，所以**不要**将这些信息加入 .install 文件。

- 在填写**软件包描述（description）**时，请不要使用下定义的方式描述包名称。比如说，“Nedit is a text editor for X11” 应当简写为 "A text editor for X11"。顺便注意保持描述在 80 个字符以内.

- 尽量保持 `PKGBUILD` 文件中**每行**不超过100字符。

- 如果可能的话，从 `PKGBUILD` 文件中**去掉空行**（没有设置变量值的行）（如 `provides`、`replaces` 等）

- 通常实践建议按照 [PKGBUILD](../包开发/PKGBUILD.md) 一文中的示例**安排 `PKGBUILD` 各变量顺序**。当然这不是强制性的，这里唯一强制要求的是满足**正确的 Bash 语法**。

- 变量名可能包含空格时，请使用**引号**，例如 `"$pkgdir"` 和 `"$srcdir"`。

- 请确保软件包的**完整性**，确保[校验变量](../包开发/PKGBUILD.md#完整性检验)包含正确的数值，可以通过 `updpkgsums` 工具进行更新。

软件包命名

- 软件包应当仅包含**字母或数字**以及 `@`，`.`，`_`，`+`，`-`，不能以 `-` 和 `.` 开头，所有的字母应当保持**小写**。

- 软件包的名称不应该包含上游的主版本号，例如如果上游软件包是 libfoo v2.3.4，不应该命名为 libfoo2。这样在上游发布新的大版本时，就可以保留软件包名和依赖可以保持不变。只有个别软件包是例外，例如图形库 GTK 和 Qt。这些软件升级后，应用程序需要花很长时间和经历才能移植完成，所以系统需要同时安装多个版本，软件包名应该包含大版本号，比如 gtk3、gtk4、qt5、qt6。如果出现大部分软件包都可以同步升级，只有个别软件没有被移植的情况，可以把老的软件包命名为 libfoo1，而新的软件包继续使用 libfoo。

软件包版本号

- 软件包[版本号](../包开发/PKGBUILD.md#pkgver)（即 `pkgver`）应当**和作者发行版号保持一致。**
- 如果需要的话，版本号可以包含字母（例如 `2.54BETA32`）。
- **版本号里不能包含连字符**，只能允许字母、数字、下划线、点号。如果上游版本号中出现了连字符，需要将其替换为下划线。

- 软件包[发行号](../包开发/PKGBUILD.md#pkgrel)（即 `pkgrel`）**仅和 Arch 相关**. 这样用户就可以区分不同的编译版本。**发布号从1开始**，软件包被**重新（打包）**发布时 ，**发行号将增加1**。
- 当新版本发布的时候，发行号（release）自动回到1。
- 软件包发布标记和软件包版本标记**遵从同样的规则**。

软件包依赖

```
- **不要依赖 [PKGBUILD#依赖关系](../包开发/PKGBUILD.md#依赖关系) 中的[传递依存关系](https://en.wikipedia.org/wiki/Transitive_dependency)**，因为依赖更新可能会导致关系中断。
- 列出所有直接依赖库，可以通过 [find-libdeps(1)](https://man.archlinux.org/man/find-libdeps.1) ([devtools](https://archlinux.org/packages/?name=devtools)包 的组件）来完成。

## 软件包关联

- 不要将 `$pkgname` 添加到 [PKGBUILD#provides](../包开发/PKGBUILD.md#provides)，软件包无法与自身冲突。
- 在 [PKGBUILD#provides](../包开发/PKGBUILD.md#provides) 中列出所有外部共享库（如 `'libsomething.so'`），可通过 [find-libprovides(1)](https://man.archlinux.org/man/find-libprovides.1) ([devtools](https://archlinux.org/packages/?name=devtools)包 的组件）来完成。

## 软件源

- 尽可能使用 HTTPS 源：压缩包使用 `https://`，git 源使用 `git+https://`
- 尽可能使用 PGP 签名对源进行验证（如果上游对提交进行了签名，但没有对压缩包签名，可能需要使用 git 标签而不是源码压缩包进行构建。）

> [![](https://wiki.archlinuxcn.org/wzh/images/d/d6/Tango-inaccurate.png)](https://wiki.archlinuxcn.org/wzh/File:Tango-inaccurate.png)**本文或本章节的事实准确性存在争议。**
>
> **原因：** commit# is not required in recent pacman as proper checksum is supported for git sources. See. gitea package also has been updated to use below approach（在 [Talk:Arch 打包准则](https://wiki.archlinuxcn.org/wiki/Talk:Arch_%E6%89%93%E5%8C%85%E5%87%86%E5%88%99) 中讨论）

- 使用 git 标签进行构建时，不要使用标签名，要使用 `git rev-parse` 获取到的对象哈希值

: ```
_tag=1234567890123456789012345678901234567890 # git rev-parse "v$pkgver"
source=(git+https://$url.git?signed#tag=$_tag)

pkgver() {
    cd "$pkgname"
    git describe
}
```

: 具体案例可参考 gitea包 包。该操作的原因是，标签可以被强制推送改变指向到的提交，从而导致构建出的包产生变化。强制推送会改变标签的哈希值，从而使用标签的对象哈希值可以确保源的完整性。使用 `pkgver()` 函数可避免在没有更新 `_tag` 的情况下不小心修改掉 `pkgver`。关于 VCS 源格式的更多信息请参考 VCS 软件打包准则 #VCS 源。

- **Do not diminish the security or validity of a package** (e.g. by removing a checksum check or by removing PGP signature verification), because an upstream release is broken or suddenly lacks a certain feature (e.g. PGP signature missing for a new release)

- 源在 `srcdir` 下的名称不能相同（可能需要在下载时进行重命名，例如 `"${pkgname}-${pkgver}.tar.gz::https://${pkgname}.tld/download/${pkgver}.tar.gz"`）
- 避免对如 sourceforge 等平台的链接使用特定镜像源进行下载，因为它们有可能会变得不可用
- 使用 SSH 密钥签名的 Git 对象（例如标签和提交等）可以使用 git 命令进行验证，用法为通过 `gpg.ssh.allowedSignersFile` 指定包含可能的签名密钥的文件。具体用例请参考 。

与上游协作

It is considered best-practice to work closely with upstream wherever possible. This entails reporting problems about building and testing a package.

- Report problems to upstream right away.
- Upstream patches wherever possible.
- Add comments with links to relevant (upstream) bug tracker tickets in the PKGBUILD (this is particularly important, as it ensures, that other packagers can understand changes and work with a package as well).

It is recommended to track upstream with tools such as nvchecker包, nvrsAUR or urlwatch包 to be informed about new stable releases.

目录

- **配置文件**应该放置到 `/etc` 目录。如果有多个配置文件，可以使用**子目录**，以保持 `/etc` 简洁。例如，如果包名是 `*pkg*`，那就应使用 `/etc/*pkg*`（或者其他合适的名称，比如apache使用的就是 `/etc/httpd/`）。
- 软件包文件应该安装在下列**常用目录**：

: <table class="wikitable"><tbody><tr><td><code>/etc</code></td><td>系统关键配置文件</td></tr><tr><td><code>/usr/bin</code></td><td>二进制文件</td></tr><tr><td><code>/usr/lib</code></td><td>库</td></tr><tr><td><code>/usr/include</code></td><td>头文件</td></tr><tr><td><code>/usr/lib/pkg</code></td><td>模块，插件等</td></tr><tr><td><code>/usr/share/doc/pkg</code></td><td>应用程序文档</td></tr><tr><td><code>/usr/share/info</code></td><td>GNU Info 系统文件</td></tr><tr><td><code>/usr/share/licenses/pkg</code></td><td>Application licenses</td></tr><tr><td><code>/usr/share/man</code></td><td>手册</td></tr><tr><td><code>/usr/share/pkg</code></td><td>程序数据</td></tr><tr><td><code>/var/lib/pkg</code></td><td>应用持久数据

</td></tr><tr><td><code>/etc/pkg</code></td><td>pkg 应用的配置文件</td></tr><tr><td><code>/opt/pkg</code></td><td>大的独立程序，例如 Java</td></tr></tbody></table>

- 软件包不应该在下面目录添加任何文件:
- `/bin`
- `/sbin`
- `/dev`
- `/home`
- `/srv`
- `/media`
- `/mnt`
- `/proc`
- `/root`
- `/selinux`
- `/sys`
- `/tmp`
- `/var/tmp`
- `/run`

Makepkg 的任务

当您使用 makepkg 构建软件包时，makepkg 会自动执行如下功能:

- 检查软件包的**依赖**和**构建依赖**是否已安装
- 从服务器中**下载源文件**
- **校验**源文件的完整性
- **解压**源文件
- 打上必要的**补丁（patch）**
- **构建**软件并安装到 fake root
- 从可执行文件中**去掉**符号标记
- 从库文件中**去掉**调试标记
- **压缩**手册页和/或 info 页
- 生成**包信息文件**（包含软件包的基本信息）
- **压缩** fake root 成为软件包文件（*.pkg.tar.xz）
- 生成的软件包文件**保存**在指定的目录中（默认在当前目录）

架构

如果该软件包是针对特定架构编译的，那么 `arch` 数组应该包含 `'x86_64'`，否则使用 `'any'` 生成架构无关的包。

授权协议

有两种与 Arch 软件包相关的许可证：

PKGBUILD 的许可证字段

`PKGBUILD` 中的 `license` 字段列出了被打包软件的上游许可证，注意这不是软件打包代码的许可证。该字段必须遵循 SPDX 许可证格式，更多信息请参考 PKGBUILD#license

软件打包代码许可证

软件打包代码的许可证。根据 RFC40，Arch Linux 指定了软件打包代码需以 BSD Zero Clause License (`0BSD`) 许可证进行授权，并参照 RFC52 使用 REUSE 来执行这一点。

总结一下就是：

1. 在软件包源码的根目录下添加一个 `LICENSE` 文件，其内容与该文件完全一致，即 Arch Linux 用于软件包的 `0BSD` 许可证。
2. 在软件包源码的根目录下添加一个 `REUSE.toml` 文件，可以使用 `pkgctl license setup` 生成一个后在其上进行修改。
3. 确保执行 `pkgctl license check` 命令时不会有任何报错。

如果有其它需要配置授权的文件，你需要为它们选择一个合理的许可证。方法通常很简单：

- 如果文件（例如类似 `launcher.sh` 的启动脚本或类似 `myunit.service` 的 systemd 服务文件）是完全由你或其他 Arch 人员创建的，则需使用 `0BSD` 许可证。

> **注意**：如果你编写了该补丁，同时也将其提交到了上游，那可以其针对 Arch 将其以 `0BSD` 许可证分发，并在提交到上游时让他们应用自己的许可证。

- 如果文件时从上游获取的（例如类似 `tool.png` 的图标或类似 `fix.patch` 的补丁），那需要应用上游的许可证。

另请参考介绍了 pkgctl-license(1) 的这篇 Arch Linux 开发博客文章。

可复现构建

Arch 正努力使所有软件包具有可复现性。打包人员可通过 devtools包 的 `makerepropkg` 或 archlinux-repro包 的 `repro` 来检查包是否有可复现性：

```
$ makerepropkg $pkgname-1-1-any.pkg.tar.zst
```

或

```
$ repro -f $pkgname-1-1-any.pkg.tar.zst
```

如果在构建时需要使用时间戳，可以使用 `SOURCE_DATE_EPOCH` 环境变量，具体格式可参考上游文档。

第 2 章　共 11 节　250 KB

启动过程

从固件到登录提示符，启动要依次穿过引导加载程序、内核、initramfs 和早期用户空间；本章把这些环节拆开，讲清各自负责什么、出错时该先查哪一环。必读《Arch 的启动流程》《Fstab》与《EFI 系统分区》：一节是全章骨架，后两节讲文件系统如何挂载、EFI 系统分区如何检查与创建；装 Windows 的读者再加双系统一节。Bootchart、E4rat、静默启动、性能优化与 PXE 按需查阅，不打算分析或压缩启动时间、不联网装机即可跳过；《UEFI/安全启动》只在确实要启用安全启动时才读。

最容易踩的坑是没确认固件类型就动手分区，以及把 EFI 系统分区的容量与挂载点随手一设——引导加载程序装得进去却起不来；双系统还要挑对时间标准，否则两边时间会互相拨乱。

2.1　Bootchart

难度 入门 | **篇幅** 3.3 KB | **所属** 启动过程

本节将讲到

- 安装Bootchart
- 运行Bootchart
- 生成分析结果图表
- 参考资料

**这篇文章或章节的翻译质量不佳。**

原因：Last updated in 2017 (490242), out of sync with English page（在 Talk:Bootchart# 中讨论）

使用 Bootchart 可以很方便地分析 Linux 启动流程，分析结果可以用来优化启动速度。包含的 bootchartd 服务负责记录及展示分析结果。

注意：Bootchart 已经成为 systemd 的一部分，请参考 Improve boot performance#Analyzing the boot process 页面。本文介绍的是合并之前的老版本和bootchart2。

安装Bootchart

安装 bootchartAUR。

运行Bootchart

要运行 bootchart，需要将他添加到引导器的初始化进程选项，或者手动在init脚本（通常是rc.sysinit）中手动添加。不过需要注意的是，如果你是手动添加到init脚本的，那么也要手动停止它，总之，这种情况需要特别留意！

启动引导器设置

下面我们介绍常用的方法，即将原有引导选项复制一份，并在内核项后面添加 `init=/usr/bin/bootchartd`. 方法参阅kernel parameters. 然后通过启动引导器引导bootchart。这样bootchart会在登录提示符出现的时候自动停止。

生成分析结果图表

你可以通过运行下面的命令来生成分析结果图：

```
bootchart-render
```

确保运行命令的目录有写权限，程序就会生成一个名为'bootchart.png'的图像，这就是分析结果图。

你需要事先安装Java运行环境并且在此之前设置正确。

问题解决

Bootchart-render 如果无法生成 'bootchart.png' 图片并显示如下错误信息：

```
/var/log/bootchart.tgz not found
```

主要原因是 bootchartd 无法检测到启动过程何时停止。如该使用非 KDM 或 GDM 的启动管理器如 SLIM 或 entrance 时会发生这个问题。可以打开 `/sbin/bootchartd` 脚本并将这些程序加到 `exit_proc` 变量中：

```
# The processes we have to wait for
local exit_proc="gdmgreeter gdm-binary kdm_greet kdm slim"
```

如果没有使用启动管理器，修改 `exit_proc` 变量为：

```
# The processes we have to wait for
local exit_proc="login"
```

参考资料

- Bootchart主页\[失效链接 2021-11-07 ⓘ\]
- 如何快速启动上网本的 LWN 文章写得很好，提供了许多快速启动的技巧，尽管许多不适合一般用户使用。(修改 X.org 内核 kernel 等)。

2.2 E4rat

难度 进阶 | **篇幅** 7.8 KB | **所属** 启动过程

本节将讲到

- 机制
- 安装
- 配置
- 其他 init 程序搭配 e4rat
- 启动流程图

翻译状态：

- 本文（或部分内容）译自 E4rat，最近一次同步于 2012-07-11，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

**这篇文章或章节的翻译质量不佳。**

原因：Last updated in 2012（在 Talk:E4rat# 中讨论）

e4rat，“Ext4 - Reducing Access Times”（减少ext4访问次数）之略，是一款优化ext4文件系统、加速系统启动的工具。该项目由 Andreas Rid 和 Gundolf Kiefer 发起。e4rat工具系列包含e4rat-collect、e4rat-realloc、e4rat-preload。

目前最新版本是0.21。

机制

如果你用bootchart监视开机过程，会发现CPU和磁盘均未全速运转。e4rat将改变这一状况，使系统启动时CPU和磁盘全速运转，从而加速启动过程。此优化包括三步：

- **e4rat-collect** - 收集文件，在特定时间（默认120s，可调整）内收集文件信息
- **e4rat-realloc** - 文件再分配，在磁盘上整理文件
- **e4rat-preload** - 预读取文件

对谁有效?

经证实，e4rat对一般用户——直接进入X图形界面——特别有效，但对于服务器用户——启动到命令行——效果不怎么明显。此外，此工具对SSD（固态磁盘）用户也没用，因为SSD基本没有读取延迟。Ureadahead 可能会有效。

注意：e4rat 官方手册 声称 ureadahead 与 e4rat 冲突，在 Ubuntu 中确实这样，但是 Arch 中并无冲突，但有 ureadahead 的情况下，速度提升效果有限。

为保险起见，请及时备份数据。

安装

从 AUR 安装软件包 e4ratAUR.

配置

现在进入正题。

e4rat-collect

首先，e4rat需要收集开机时预读的文件列表。添加如下内容到`/boot/grub/menu.lst`（grub）启动项的`linux`行末尾，或`/boot/grub/grub.cfg`（grub2）启动项的`kernel`行：

```
kernel /vmlinuz0linux root=/dev/disk/by-label/ARCH init=/sbin/e4rat-collect ro 5
```

)

由于该过程只需进行一次，所以我建议直接在grub命令行修改即可，而不必修改启动菜单文件。

启动后，e4rat-collect默认会监视系统120秒。所以请在两分钟内打开经常使用的程序，e4rat-collect会记录下来（译者注：我不建议预读太多东西，这样反而会拖慢开机速度）。可以在`/etc/e4rat.conf`中修改默认的120秒收集时间（`timeout 120`这一行，去掉注释）。或者手动停止e4rat-collect：

```
e4rat-collect -k
```

或者：

```
pkill e4rat-collect
```

收集完成后，应该会出现这个文件：`/var/lib/e4rat/startup.log`

最后，不要忘记从`/boot/grub/grub.cfg`或`/boot/grub/menu.lst`中移除最开始添加的内容。

e4rat-realloc

接着上一步进行文件再分配（通俗说就是磁盘整理）。先切换启动级别1：

```
sudo init 1
```

输入root密码，执行：

```
e4rat-realloc  /var/lib/e4rat/startup.log
```

根据`startup.log`中文件的多少，或长或短要等一段时间。

注意：使用纯 systemd 的用户不需要修改 runlevels，用 root 登录后执行 *e4rat-collect* 即可。

e4rat-preload

永久性地，添加如下内容到`/boot/grub/grub.cfg`（grub）启动项的`linux`行末尾，或`/boot/grub/menu.lst`（grub2）启动项的`kernel`行：

```
init=/sbin/e4rat-preload
```

注意：如果你正在使用grub2，内核参数最好添加到`/etc/default/grub`-

```
GRUB_CMDLINE_LINUX="..."
```

另一选择：e4rat-preload-lite

论坛用户jlindgren提供了一个优化版本的preload程序，也许能帮你节约几秒的开机时间：

优化之处在于：

- 启动`/sbin/init`前，只预读前100个文件（inode和文件内容），启动`/sbin/init`后再并行地预读其他文件。

从AUR中安装该工具（译者注，不要卸载e4rat，该包只提供preload）：e4rat-preload-liteAUR\[损坏的链接：package not found\]

添加（或替换）如下内容到`/boot/grub/grub.cfg`（grub）启动项的`linux`行末尾，或`/boot/grub/menu.lst`（grub2）启动项的`kernel`行：

```
init=/usr/sbin/e4rat-preload-lite
```

重启即可。

其他 init 程序搭配 e4rat

e4rat-collect默认会使用`/sbin/init`替换自己。如果使用其他init程序（比如 `/bin/systemd`），修改`/etc/e4rat.conf`中的`init`参数，并去掉前面的分号（注释）即可。

启动流程图

通过bootchart绘制使用e4rat前后的启动流程图，可以直观地看到巨大的优化。

bootchart 0.9-9

添加如下内容到`grub.cfg`或`menu.lst`的启动菜单项即可：

```
init=/sbin/bootchartd bootchart_init=/sbin/e4rat-preload
```

该版本bootchart会在登录管理器启动后停止监视。

bootchart2

添加如下内容到`grub.cfg`或`menu.lst`的启动菜单项：

```
init=/sbin/bootchartd
```

然后，手动修改`/sbin/bootchartd`的`init="/sbin/init"`这一行为`init="/sbin/e4rat-preload`。（译者注：虽然程序说明上说，应该通过添加启动参数设置init，但我试了，不行。）

`/etc/bootchartd.conf`的`EXIT_PROC`中可以设置哪些程序启动后停止监视：

```
EXIT_PROC="kdm_greet xterm konsole gnome-terminal metacity mutter compiz ldm icewm-session enlightenment"
```

留空的话，需要手动停止监视。

疑难解答

如果出现问题，请参考以下内容。

startup.log 未创建

- 在`/etc/rc.conf`注释掉`auditd`。
- 检查下列命令的输出：

```
dmesg | grep e4rat
```

- 在`/etc/e4rat.conf`设置`loglevel`为`31`获取详细调试信息。

e4rat 错误地报告文件系统为 ext2

- 添加如下内容到`/boot/grub/grub.cfg`启动项的`linux`行末尾，或`/boot/grub/menu.lst`启动项的`kernel`行末尾：

```
rootfstype=ext4
```

无法读取 startup.log

- 这说明你的`/var`和根目录不在同一分区，因而开机时为挂载。可以修改`startup.log`文件位置（比如{ic|/etc/startup.log}}），方法是修改`/etc/e4rat.conf`：

```
startup_log_file /etc/e4rat/startup.log
```

减少启动时的信息输出

在 `/etc/e4rat.conf` 中将 `loglevel` 减小到 1。

2.3 静默启动

难度 进阶 | **篇幅** 8.5 KB | **所属** 启动过程

本节将讲到

- 内核参数
- 移除控制台光标闪烁
- sysctl
- agetty
- startx

翻译状态：

- 本文（或部分内容）译自 Silent boot，最近一次同步于 2023-7-2，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

本页适用于那些出于美观或其他原因而倾向于将其系统启动的记录严格限制在最低限度的人。遵循本指南将删除启动过程中的所有文本。演示视频

内核参数

在你的引导加载程序配置选项中更改内核参数，以包括以下参数：

`quiet`

注意：将 `vga=current` 添加到内核参数可以避免类似 FS#32309 的奇怪行为。请记住，这与KMS相冲突，因此只有当您受到上述错误的影响时才应使用此参数。

如果您仍发现消息被打印到控制台，则可能是dmesg向您发送了它认为重要的消息。您可以使用 `quiet loglevel=*level*` 来更改打印这些消息的级别，其中 `*level*` 是0到7之间的任意数字，其中0代表只有最关键的，7代表打印的调试级别（debug levels）。

`quiet loglevel=3`

请注意，只有同时使用 `quiet` 和 `loglevel=*level*` 且 quiet 在前才应当有效。loglevel 参数只会更改打印到控制台的内容，*dmesg* 本身的级别不会受到影响，并且仍然可以通过日志（journal）和 *dmesg* 使用。有关详细信息，请见内核参数。

如果您还想在引导时阻止 systemd 打印其版本号，您还应该将 `udev.log_level=3` 附加到内核参数中。如果在 initramfs 中使用 systemd，请改为附加 `rd.udev.log_level=3` 。见 systemd-udevd.service(8) § KERNEL COMMAND LINE 获取详细信息。

如果您在 initramfs 中使用 `systemd` 钩子，则可能会在 initramfs 初始化期间获得 systemd 消息。您可以传递 `systemd.show_status=false` 来禁用它们，或者传递 `systemd.show_status=auto` 只阻止成功的消息（因此在出现错误时，您仍然可以看到它们）。事实上，当使用 `quiet` 时，`auto` 已经被传递给了 `systemd.show_status=auto`，但是出于某些原因，有时 initramfs 中的 systemd 无法获得它。以下是您需要传递给内核的参数，以便在 initramfs 中使用 systemd 进行完全干净的启动：

`quiet loglevel=3 systemd.show_status=auto rd.udev.log_level=3`

还可以用 `touch ~/.hushlogin` 删除最后登录消息。

移除控制台光标闪烁

如果按照以上说明，启动时的控制台光标会一直闪烁。这可以通过向内核传递 `vt.global_cursor_default=0` 来解决 。

要恢复 TTY 中的光标，请运行：

```
# setterm -cursor on >> /etc/issue
```

sysctl

要从控制台隐藏任何内核消息，请根据 添加或修改 `kernel.printk` 行：

```
/etc/sysctl.d/20-quiet-printk.conf
kernel.printk = 3 3 3 3
```

agetty

要从控制台隐藏 agetty 打印的问题（issue）和“login:”提示行 ，创建 `getty@tty1.service` 的附加片段。

```
/etc/systemd/system/getty@tty1.service.d/skip-prompt.conf
[Service]
ExecStart=
ExecStart=-/usr/bin/agetty --skip-login --nonewline --noissue --autologin username --noclear %I $TERM
```

startx

要隐藏 `startx` 消息，您可以在你的 .bash_profile 中重定向它的输出到 `/dev/null`，类似于这样：

注意：无根（rootless）登录导致重定向中断。见 Xorg#重定向Xorg会话日志。

```
$ [[ $(fgconsole 2>/dev/null) == 1 ]] && exec startx -- vt1 &> /dev/null
```

fsck

要在引导期间隐藏 fsck 消息，请让 systemd 检查根文件系统。为此，请将 *udev* 钩子替换为 *systemd*：

```
HOOKS=( base systemd fsck )
```

在 `/etc/mkinitcpio.conf` 中并重新生成 initramfs。

现在编辑 `systemd-fsck-root.service` 和 `systemd-fsck@.service` 并配置 `StandardOutput` 和 `StandardError` 类似于这样：

```
[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/lib/systemd/systemd-fsck
StandardOutput=null
StandardError=journal+console
TimeoutSec=0
```

见 systemd-fsck@.service(8) 获得可以传递给 `systemd-fsck` 的选项的更多信息，您可以更改服务检查（或不检查）文件系统的频率。

使 GRUB 静默

要隐藏 GRUB 欢迎和引导消息，您可以安装非官方的 grub-silentAUR 包。

安装完成后，需要先将 GRUB 重新安装到必要的分区中。

然后，以 `/etc/default/grub.silent` 为例，对 `/etc/default/grub` 进行必要的更改。

以下三行是必要的：

```
GRUB_DEFAULT=0
GRUB_TIMEOUT=0
GRUB_RECORDFAIL_TIMEOUT=$GRUB_TIMEOUT
```

注意：如果设置了 `GRUB_TIMEOUT=0` 和 `GRUB_HIDDEN_TIMEOUT=1`（或任何正值），请设置 `GRUB_RECORDFAIL_TIMEOUT=$GRUB_HIDDEN_TIMEOUT`，而不是 `GRUB_RECORDFAIL_TIMEOUT=$GRUB_TIMEOUT`。否则，在启动时按 `Esc` 以显示 GRUB 菜单将无法工作。

最后，重新构建 `grub.cfg` 文件。

保留或禁用 BIOS 中的供应商徽标

现代 UEFI 系统在启动时显示供应商标志，直到将控制权移交给引导加载程序；例如，联想笔记本电脑上显示一个鲜红色的联想标志。这个供应商标志通常被引导加载程序（如果使用标准 GRUB）或内核覆盖。

为了避免内核覆盖供应商徽标，Linux 4.19 引入了一个新的配置选项 `FRAMEBUFFER_CONSOLE_DEFERRED_TAKEOVER`，该选项将保留帧缓冲区的内容，直到需要在帧缓冲区控制台上打印文本为止。截至2018年11月（Linux 4.19.1），官方的 Arch Linux 内核使用 `CONFIG_FRAMEBUFFER_CONSOLE_DEFERRED_TAKEOVER=y` 编译。

当与低日志级别相结合（以防止打印文本）时，可以在系统初始化时保留供应商徽标。请注意，标准配置中的 GRUB 会清空屏幕；考虑使用 EFISTUB 引导来直接引导到内核中，从而利用延迟接管（deferred takeover）。

视频演示

内核命令行应该使用上面提到的 `loglevel=3` 或 `rd.udev.log_level=3`。请注意，如果您经常在内核日志中收到 `Core temperature above threshold, cpu clock throttled` 消息，则需要使用日志级别2在启动时使这些消息静音。或者，如果您编译自己的内核，请调整 `arch/x86/kernel/cpu/mcheck/therm_throt.c` 中消息的日志级别。

如果你使用 Intel 图形处理器，另见 Intel 图形处理器#Fastboot。

延伸阅读：

- Phoronix: Linux 4.19 Adds Deferred Console Takeover Support For FBDEV - Cleaner Boot Process
- Hans de Goede: Adding deferred fbcon console takeover to the Fedora kernels

禁用延迟接管（deferred takeover）

如果新行为导致问题，可以使用 `fbcon=nodefer` 内核参数禁用延迟接管。

2.4 性能优化/启动过程

难度 进阶 | **篇幅** 9.4 KB | **所属** 启动过程

本节将讲到

- 启动过程分析
- 编译自定义内核
- Initramfs
- 选择合适的服务启动方式
- 分批启动

翻译状态：

- 本文（或部分内容）译自 Improving performance/Boot process，最近一次同步于 2025-04-03，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面/Boot_process_(%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87)?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

提升系统的启动性能可以缩短启动等待时长，还能学习系统文件和脚本见交互的方式。本文整合了数种提升 Arch Linux 启动性能的方式。

启动过程分析

使用 systemd-analyze

systemd 提供了一个名为 `systemd-analyze` 的工具，可用于显示有关启动过程的计时细节, 包括一个显示各单元等待其依赖项的 svg 图。您可以看到哪些单元文件导致启动过程减慢, 然后可以相应地优化系统.

要查看启动时在内核/用户空间中消耗的时间, 只需使用：

```
$ systemd-analyze
```

提示：如果系统使用 UEFI 引导，且启动引导器实现了 systemd 的启动引导器接口（目前只有 GRUB 和 systemd-boot 实现了），**systemd-analyze** 还可以显示 EFI 固件和启动引导器自身花费的时间。

按照耗费时间顺序，列出启动各个单元耗费的时间：

```
$ systemd-analyze blame
```

在启动过程的一些时刻，需要特定的单元成功启动了才能继续。要查看在启动链中哪些单元处于这些关键点，可以执行：

```
$ systemd-analyze critical-chain
```

类似于 Bootchart，还可以生成 SVG 形式的启动流程图：

```
$ systemd-analyze plot > plot.svg
```

更多信息请参考 systemd-analyze(1)。

使用 bootchart2

你还可以使用 Bootchart2 将启动流程可视化。

编译自定义内核

使用自定义内核可以减少启动时间和内存占用，但由于 Linux 内核的模块化特性以及 64 位架构的逐渐标准化，这些好处可能没有预期的那么大。具体信息请参考内核#编译。

官方内核的模块默认压缩等级为 `ZSTD_CLEVEL=19`，但 SSD 可能更适合使用 `ZSTD_CLEVEL=1`。

建议将会用到的存储和文件系统模块嵌入到内核中，以使用 Arch 的启动流程#Running_without_initramfs。

Initramfs

最佳做法是 Arch 的启动流程#Running_without_initramfs。

mkinitcpio 默认使用 `base` 和 `udev` 钩子，将它们替换为 `systemd`可以提升启动速度，具体细节请参考 Mkinitcpio#常用钩子。如果要替换 `fsck` 钩子，请参考 Fsck#启动时检查。

与#编译自定义内核类似，initramfs 也可以被精简。最简单的方法是包含 mkinitcpio 的 `autodetect` 钩子。Booster 可以生成比 mkinitcpio 和 dracut 更小的 initramfs，并使用更快的单二进制文件 init. 参阅极简 initramfs 或 Booster#Removing modules 以获得更多信息。

取决于你的硬件（处理器和存储速度）不同，由于启动时更快的解压速度通常可以抵消掉从硬盘读取稍大的 initramfs 的负面影响，使用 lz4包 取代默认的 zstd包 压缩算法有可能会更快，具体细节请参考 Mkinitcpio#压缩方式(COMPRESSION))。

选择合适的服务启动方式

systemd 的一个核心功能是 D-Bus 和 socket 激活。多数情况下建议使用该功能，它会使服务只在初次访问时启动，通常来说这是件好事（例如 `cups.service` 在桌面环境通常没必要开机启动，而启用 `cups.socket` 会使只在打印时启动服务）。

但如果有个服务（以 upower包 为例）始终会在开机时启动，那尽早将其启动有可能会减少启动耗时，该操作可通过启用 `upower.service` 达成（前提是已配置了启动文件，多数情况下都是如此）。

这样 systemd 就会尽早启动 UPower，而不会引起 socket 或 D-Bus 激活的竞争。

分批启动

有些硬件使用了分批启动（Staggered spin-up，SSP），会使系统按顺序探测 ATA 接口，从而使硬盘逐个轮流启动，以降低耗电峰值。这会降低启动速度，且由于多数消费级硬件在通电后就会立即启动，因此并没有提供什么优势。执行以下命令可以检查是否启用了分批启动：

```
# dmesg | grep SSS
```

无输出表示没有使用该功能。

可以将 `libahci.ignore_sss=1` 添加到内核参数以禁用该功能。

挂载文件系统

借助 mkinitcpio 的 `fsck` 钩子，你可以通过在内核命令行中将 ro 改为 rw 来避免对 root 分区进行影响性能的重挂载：一个选择是使用 `rootflags=**rw**,*other_mount_options*` 来设置相关选项。必须从 `/etc/fstab` 中移除对应的条目，否则 `systemd-remount-fs.service` 仍会尝试应用这些设置。或者，也可以尝试屏蔽（mask）此服务单元。

如果根文件系统是 Btrfs，就不需要和其他文件系统一样每次启动都进行 `fsck`。在这种情况下，可以移除 mkinitcpio 的 fsck 钩子。你也可以选择屏蔽（Mask）`systemd-fsck-root.service`，或者使用内核命令行`fsck.mode=skip`来告诉根文件系统跳过 `fsck`。没有 mkinitcpio 的 fsck 钩子，systemd 仍会使用 `systemd-fsck@.service` 对任何相关的文件系统进行 fsck。

你也可以移除`/etc/fstab`中的 API 文件系统，因为 systemd 会自动挂载它们（列表见 `pacman -Ql systemd | grep '\.mount$'`）。用户可能看到从 sysvinit 迁移过来的 `/tmp` 条目，但你可能已经注意到，从上面的命令可以看出，systemd 已经处理了这个问题。因此，`/tmp` 条目可以安全地移除。

其他文件系统，比如 /home 或者 EFI 系统分区，可以使用自定义的挂载单元进行挂载。向挂载参数中添加 noauto,x-systemd.automount 会延迟挂载分区，直到首次访问时才进行 fsck 和挂载，从而减少启动过程中需要 fsck 或挂载的文件系统数量，帮助加快启动速度。

注意：

- 这将使你的 /home 文件系统类型变成 autofs，默认情况下，locate 会忽略它。自动挂载 /home 的加速效果可能不会超过一两秒，具体取决于你的系统，因此这个技巧可能并不值得尝试。
- 如果系统安装在一个 *btrfs* 子卷中（具体来说：根目录 / 本身是一个子卷），并且 /home 是一个独立的文件系统，你可能还需要防止创建一个 /home 子卷。可以通过屏蔽 home.conf 临时文件来实现：ln -s /dev/null /etc/tmpfiles.d/home.conf。

精简启动阶段输出信息

对于某些用户，特别是 SSD 用户，TTY 的龟速实际上成为了性能瓶颈，精简输出信息实际上有利于提高性能。具体建议请参考静默启动。

替换启动引导器

替换引导加载程序（例如换成 systemd-boot 这种简单的引导器）可能会减少数秒的启动时间。

在可用的情况下，只使用一个 EFI boot stub 可以进一步降低启动耗时。

内核参数

有些内核参数可以提升启动性能。

如果不需要 systemd-gpt-auto-generator，使用 systemd.gpt_auto=no 可能会减少 0.2~3 秒的启动时长。如果根分区是 Btrfs 且没有使用 initramfs 进行启动，那 systemd-gpt-auto-generator 不会起效。

cryptomgr.notests 会禁用自测试，但效果可能不大。

挂起到内存

减少启动时长的最好方法是完全不用启动，可以考虑转而将系统挂起到内存。

2.5 PXE

难度 进阶 | **篇幅** 17.1 KB | **所属** 启动过程

本节将讲到

- 准备

- Pixiecore
- 服务器搭建
- 安装
- 排障

维基百科：预启动执行环境：

: 预启动执行环境（Preboot eXecution Environment，PXE，也被称为预执行环境）提供了一种使用网络接口（Network Interface）启动计算机的机制。这种机制让计算机的启动可以不依赖本地数据存储设备（如硬盘）或本地已安装的操作系统。

在本指南中，PXE 用于通过支持 PXE 的 Option ROM 在目标机器上启动安装介质，适用于安装了服务器的情况。

准备

概览

在开始前，需要先了解下 PXE 启动流程，以理解#服务器搭建，目标设备上的#安装流程以及所需的 Arch Linux 文件。

目标设备会先广播数据包，请求 DHCP 服务器并包含特定的 PXE 选项。接下来，DHCP 服务器会发送包含网络信息（分配给目标设备的 IP 地址）的回应，并通过 DHCP 的特定引导协议（bootstrap protocol，BOOTP）参数提供如 TFTP 服务器地址，初始网络引导程序（initial network bootstrap program，NBP）的下载路径或启动配置文件名称等额外信息。

NBP 会通过 TFTP 从 PXE 服务器传输到目标设备上，然后加载到内存中并执行。内核以及 initramfs 也是通过一样的方式进行传输。

接下来根文件系统会通过以下协议之一进行传输：HTTP，NFS 或 NBD。

警告：PXELINUX 不支持以任何形式对通过 TFTP 或 HTTP 收到的数据进行验证，具体信息请参考 RFC 5071 section8。替代方法是使用网络引导（Netboot）映像，它可以使用 iPEX 中嵌入的代码签名凭证来验证签名。注意，网络引导映像需要放置在本地设备上。

启动安装介质

首先从下载页获取最新的官方安装映像，以获取启动时需要从服务器传输到客户端的文件。

接下来需要挂载映像：

```
# mount --mkdir -o loop,ro archlinux-release_date-x86_64.iso /mnt/archiso
```

其中 `*release_date*` 是 ISO 文件名中的发布日期，例如 `2022.10.01`。

注意：Arch ISO 现在只支持 **BIOS** 风格的 PXE 引导。更多信息请参考 archlinux/archiso#55。

从网络引导启动

Arch Linux 网络引导映像可以在系统启动时即时下载最新发布的 Arch Linux 版本。请下载适用于你的配置的版本。

注意：Arch 网络引导映像同时支持 **BIOS** 和 **UEFI** 引导方式。

Pixiecore

pixiecore 提供了一体化解决方案：

1. 安装 pixiecore-gitAUR
2. 以根用户身份执行 `pixiecore quick arch --dhcp-no-bind`
3. 通过 PXE 进行引导

服务器搭建

你需要搭建一个 DHCP 服务器和一个 TFTP 服务器来传输 NBP，以及下列服务之一来传输根文件系统：HTTP 服务器，NFS 或 NBD。

网络

激活网卡，并分配一个合适的地址。

```
# ip link set eth0 up
# ip addr add 192.168.0.1/24 dev eth0
```

你还可以使用网络管理器来配置静态 IP。

DHCP + TFTP

为了在安装目标上配置网络并在 PXE 服务端和客户端之间传输文件，需要搭建 DHCP 和 TFTP 服务器。dnsmasq 能同时做到这两点，也很容易配置。

首先安装 dnsmasq包。

接下来需要配置 *dnsmasq*，请参考 dnsmasq#TFTP 服务器和 dnsmasq#PXE 服务器。

以下为一些常用配置，其中 *tftp_root* 为 Arch ISO 的挂载目录（例如 `/mnt/archiso`）或网络引导程序的挂载路径：

```
# /etc/dnsmasq.conf
# Listen only to the specified interface
interface=eth0

# Do not function as DNS server
port=0

# TFTP server setup
enable-tftp
```

```
tftp-root=tftp_root

# Log extra information about DHCP transactions (for debug purposes)
log-dhcp
```

要启用 DHCP 服务器并分配 IPv4 地址段，需在配置文件中添加如下一行：

```
dhcp-range=192.168.0.50,192.168.0.150
```

如果已有 DHCP 服务器并希望与其搭配使用，请参考 dnsmasq#Proxy DHCP。

下文提供了两种通过不同方式启动到安装介质的示例。

配置完成后，启动 `dnsmasq.service`。

使用 BIOS 启动到安装介质

在配置文件中，使用 `dhcp-boot` 选项定义要发送的初始引导程序的路径：

```
dhcp-boot=/boot/syslinux/lpxelinux.0
```

要发送如配置文件路径等特定引导协议（*boot*strap *p*rotocol，BOOTP）参数，需使用 `dhcp-option-force=*flag*,*value*` 配置：

```
dhcp-option-force=209,archiso_pxe.cfg # this file might be
under /mnt/archiso/boot/syslinux
dhcp-option-force=210,
```

使用 UEFI 从网络引导启动

要根据架构发送文件（此处为用于 UEFI 引导的网络引导映像），请使用：

```
pxe-service=BC_EFI, "Boot from network BC EFI", ipxe.efi
pxe-service=X86-64_EFI, "Boot from network X86-64 EFI", ipxe.efi
```

剩余的服务器搭建环节仅适用于 Arch ISO，如果使用了网络引导（netboot），那以下部分将不再适用。

传送 archiso 根文件系统

得益于 archiso 中的 `archiso_pxe_http`，`archiso_pxe_nfs` 和 `archiso_pxe_nbd` initcpio 钩子，你可以选择使用 HTTP，NFS 或 NBD 进行启动。这三种方式的启动时间基本没有区别，但使用 HTTP 方式能以百分比形式查看 `airootfs.sfs` 的下载进度。

HTTP

在所有备选方案中，*darkhttpd* 是最容易设置的（也是最轻量的）。

首先，安装 darkhttpd包。

然后以 `/mnt/archiso` 作为文件根目录启动 *darkhttpd*：

```
# darkhttpd /mnt/archiso
darkhttpd/1.8, copyright (c) 2003-2011 Emil Mikulic.
listening on: http://0.0.0.0:80/
```

注意：请确保服务器使用的是 `**80**` 端口。如果你没有以根用户启动 *darkhttpd*，那么它会默认使用 `**8080**` 端口，导致目标设备尝试访问无效的 80 端口并启动失败。

NFS

您需搭建 NFS 服务器并将安装镜像的挂载点作为出口（export）。如果您按照#准备段落做了的话，出口就是 `/mnt/archiso`。服务器搭建起来后，往 `/etc/exports` 写入这行：

```
/etc/exports
/mnt/archiso 192.168.0.0/24(ro,no_subtree_check)
```

如果服务器已经在运行了，用 `exportfs -r -a -v` 重新导出文件系统。

安装程序会在 `/run/archiso/bootmnt/` 查找 NFS，因此您需要编辑启动选项。在启动菜单按下 Tab，参考如下修改 `archiso_nfs_srv`：

```
archiso_nfs_srv=${pxeserver}:/mnt/archiso
```

或者，您也可以整个过程中都使用 `/run/archiso/bootmnt`。

在内核加载后，Arch bootstrap 镜像会通过 NFS 复制根文件系统到引导主机（booting host）。这需要一定的时间。一旦复制完成，您就有可运作的系统了。

NBD

安装 nbd包 并配置：

```
/etc/nbd-server/config
[generic]
[archiso]
    readonly = true
    exportname = /srv/archlinux-release_date-x86_64.iso
```

其中 `*release_date*` 是 ISO 文件名中的发布日期，例如 `2022.10.01`。

启动 `nbd.service`。

已有的 PXE 服务器

如果已经有了 PXELINUX 配置（例如 DHCP + TFTP）的 PXE 服务器，可以通过在 `/tftpboot/pxelinux.cfg/default` 文件中添加以下菜单项来通过偏好的方式启动 Arch。

鉴于 PXELINUX 支持 HTTP，可以只通过 TFTP 发送引导加载程序，其它的都通过 HTTP 进行传输，例如：

```
LABEL archlinux
        MENU LABEL Arch Linux x86_64
        LINUX http://httpserver/path/to/extracted/Arch/ISO/arch/boot/x86_64/vmlinuz-
linux
        INITRD http://httpserver/path/to/extracted/Arch/ISO/arch/boot/x86_64/initramfs-
linux.img
        APPEND archisobasedir=arch
archiso_http_srv=http://httpserver/path/to/extracted/Arch/ISO/ cms_verify=y
        SYSAPPEND 3
        TEXT HELP
        Arch Linux 2022.10.01 x86_64
        ENDTEXT
```

如果使用 NFS 和 NBD，那么必须通过 TFTP 传输内核及 initramfs。以 NFS 为例：

```
LABEL archlinux
        MENU LABEL Arch Linux x86_64
        LINUX /path/to/extracted/Arch/ISO/arch/boot/x86_64/vmlinuz-linux
        INITRD /path/to/extracted/Arch/ISO/arch/boot/x86_64/initramfs-linux.img
        APPEND archisobasedir=arch archiso_nfs_srv=pxeserver:/run/archiso/bootmnt
cms_verify=y
        SYSAPPEND 3
        TEXT HELP
        Arch Linux 2022.10.01 x86_64
        ENDTEXT
```

其中 `LINUX` 和 `INITRD` 路径是相对于 TFTP 根路径的。对于 NBD，需要将上面的 `archiso_nfs_srv` 替换为 `archiso_nbd_srv`。可以参考一下 Arch Linux ISO 中的 `boot/syslinux/archiso_pxe.cfg` 示例文件。

无论使用了哪种方法，都必须在尝试通过网络挂载安装介质前通过 `ip=` 参数让内核拉起网络接口。如果目标设备上有多个有线网卡，或是需要在 archiso 启动后预配置 `resolv.conf`，就需要传递 `BOOTIF=` 参数。你可以通过 sysappend mask `3`（即 `1` + `2`）来自动传递这些参数。所有可用启动参数请参考 README.bootparams。

安装

在进行该部分操作前，需要知道如何让你的目标设备进行 PXE 引导，通常在正常引导时屏幕角落会提示进行 PXE 引导需要按下的按键。例如在 IBM x3650 上按下 `F12` 会出现一个引导菜单，可以选择第一项 *Network*；在 Dell PE 1950/2950 上按下 `F12` 会直接进行 PXE 引导。

启动

通过查看 PXE 服务器的 journald，可以了解 PXE 早期启动阶段时的具体流程：

```
# journalctl -u dnsmasq.service -f
dnsmasq-dhcp: DHCPDISCOVER(eth1) 00:1a:64:6a:a2:4d
dnsmasq-dhcp: DHCPOFFER(eth1) 192.168.0.110 00:1a:64:6a:a2:4d
dnsmasq-dhcp: DHCPREQUEST(eth1) 192.168.0.110 00:1a:64:6a:a2:4d
dnsmasq-dhcp: DHCPACK(eth1) 192.168.0.110 00:1a:64:6a:a2:4d
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/pxelinux.0 to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/archiso.cfg to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/whichsys.c32 to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/archiso_pxe_choose.cfg to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/ifcpu64.c32 to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/archiso_pxe_both_inc.cfg to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/archiso_head.cfg to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/archiso_pxe32.cfg to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/archiso_pxe64.cfg to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/archiso_tail.cfg to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/vesamenu.c32 to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/boot/syslinux/splash.png to 192.168.0.110
```

在通过 TFTP 加载了 `pxelinux.0` 和 `archiso.cfg` 后，应该会出现带有多个选项的 syslinux 菜单，这时候需选择 *Boot Arch Linux (x86_64) (HTTP)*。

接下来将通过 TFTP 传输架构对应的内核与 initramfs：

```
dnsmasq-tftp: sent /mnt/archiso/arch/boot/x86_64/vmlinuz to 192.168.0.110
dnsmasq-tftp: sent /mnt/archiso/arch/boot/x86_64/initramfs-linux.img to 192.168.0.110
```

如果一切正常，你将可以在 darkhttpd 上看到 PXE 目标设备的一系列活动。这时 PXE 目标设备将加载内核并初始化：

```
1348347586 192.168.0.110 "GET /arch/aitab" 200 678 "" "curl/7.27.0"
1348347587 192.168.0.110 "GET /arch/x86_64/root-image.fs.sfs" 200 107860206 ""
"curl/7.27.0"
1348347588 192.168.0.110 "GET /arch/x86_64/usr-lib-modules.fs.sfs" 200 36819181 ""
"curl/7.27.0"
1348347588 192.168.0.110 "GET /arch/any/usr-share.fs.sfs" 200 63693037 "" "curl/7.27.0"
```

在通过 HTTP 下载根文件系统后，最终将看到正常 live 环境的根用户 zsh 提示符。

启动后

除非你想让所有流量都路由过 PXE 服务器（在正确配置前都无法使用），否则就需要停止 `dnsmasq.service` 服务，并根据网络环境为安装目标获取新的租约。

你还可以杀掉 *darkhttpd*：目标已经下载好了根文件系统，不再需要该进程。在进行该操作时，可以顺便卸载掉安装映像：

```
# umount /mnt/archiso
```

这时候就可以根据安装指南进行安装了。

小内存设备

`copytoram` initramfs 选项可以控制在启动早期时是否将整个根文件系统复制到内存中。

强烈建议不去改动该选项，且只在必要的时候禁用（物理内存小于 ~256MB）。如果要这么做，需向内核参数加入 `copytoram=n` 设置。

注意：由于该操作需要从已挂载的远程文件系统循环挂载 squashfs，因此 `copytoram=n` 和 `[archiso_pxe_http](#HTTP)` 不能同时使用。

与 PXE 目标设备共享网络

如果 PXE 目标设备使用的是内部网络（例如 192.168.1.0/24），但还需要访问互联网（如需要安装软件包），那么就需要配置 masquerade/source nat。PXE 服务器上必须有连接到互联网的单独网卡。可以使用如下命令让目标设备访问到互联网：

```
iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -j MASQUERADE
```

要将配置持久化，需使用如下命令：

```
iptables-save -f /etc/iptables/iptables.rules
```

然后启用 `iptables.service`。

具体信息请参考 Simple stateful firewall#Setting up a NAT gateway 和网络分享#启用 NAT。

排障

DHCP interface rename bug

FS#36749 causes default predictable network interface renaming to fail and then DHCP client to fail because of it. A workaround is to add the kernel boot parameter `net.ifnames=0` to disable predictable interface names.

VirtualBox cannot boot while real machines can

When using VirtualBox to test your configuration, the virtual machine may get stuck at:

```
Probing EDD (edd=off to disable)... ok
```

While PXE booting with a real machine works fine. The problem may be because you have set several CPU cores to your client machine, and you set its type as *Other* and version as *Other/Unknown (64 bit)*. So VirtualBox does not know which paravirtualization interface to use by default.

Adding `loglevel=7` to the kernel parameters lets you see where it actually got stuck:

```
[    0.063697] smp: Bringing up secondary CPUs...
[    0.103768] x86: Booting SMP configuration:
```

To resolve this, either use one CPU core, or go to *Machine* > *Settings* > *System* > *Acceleration* and set one of the following paravirtualization interface: *Minimal*, *Hyper-V* or *KVM*.

2.6 Fstab

难度 高级 | **篇幅** 21.1 KB | **所属** 启动过程

本节将讲到

- 用法
- 文件系统标识
- 提示和技巧
- 参见

fstab(5)文件可用于定义如何挂载硬盘分区以及各种块设备和远程文件系统。

每个文件系统都分别在单独一行中定义，这些定义将在系统引导或重新加载系统管理器配置时被动态转换为 systemd 挂载单元。默认设置将在需要挂载文件系统的服务启动之前自动进行 fsck 并挂载文件系统。例如，systemd 会自动确保类似 NFS 和 Samba 的远程文件系统仅在网络可用后才会进行挂载。因此，`/etc/fstab` 中指定的本地和远程文件系统挂载应能开箱即用。具体信息请参考 systemd.mount(5)。

如果仅提供了单个目录或是设备作为参数，那么 `mount` 会调用 fstab 来填充其它参数。在这种情况下，将调用 fstab 中列出的挂载参数。

用法

一个简单的 `/etc/fstab`，使用了文件系统 UUID：

```
/etc/fstab
# <device>                                 <dir> <type> <options>
<dump> <fsck>
UUID=0a3407de-014b-458b-b5c1-848e92a327a3 /     ext4 defaults
0      1
UUID=CBB6-24F2                           /boot vfat
defaults,nodev,nosuid,noexec,fmask=0177,dmask=0077 0       2
UUID=f9fe0b69-a280-415d-a03a-a32752370dee none  swap defaults
0      0
UUID=b411dc99-f0a0-4c87-9e05-184977be8539 /home ext4 defaults
0      2
```

- `<device>` 描述了要挂载的特定块设备或远程文件系统，详见 #文件系统标识。
- `<dir>` 指定了要将文件系统挂载到的目录，即挂载点。必须提前创建该目录。
- `<type>` 是文件系统类型。
- `<options>` 是文件系统挂载选项，详见 mount(8) § FILESYSTEM-INDEPENDENT MOUNT OPTIONS 和 ext4(5) § Mount options for ext4。
- `<dump>` 会被 dump(8) 工具检查。该字段通常设置为 `0`，以禁用检查。
- `<fsck>` 设置了引导时检查文件系统的顺序，具体信息请参考 fsck(8)。对于根设备，该字段需设置为 `1`。对于其它分区该字段应该设置为 `2`，或设置为 `0` 以禁用检查。

提示：

- `auto` 类型会让 mount 命令猜测设备使用的文件系统。这对于光学介质（CD/DVD/Blu-ray）非常有用。
- 如果根文件系统是 btrfs 或 XFS，那么 fsck 顺序应该设置为 `0` 而不是 `1`。具体信息请参考 fsck.btrfs(8) 和 fsck.xfs(8)。

除非指定 `noauto` 选项，否则在启动以及使用带有 `-a` 标识的 mount(8) 命令时,所有在 `/etc/fstab` 里指定的设备都会自动被挂载。除非使用 `nofail` 选项，否则在列表中出现但实际不存在的设备会导致错误。

更多信息请参考 fstab(5) § DESCRIPTION。

文件系统标识

在 `/etc/fstab` 配置文件中，有多种方法可用于标识文件系统：内核名称、文件系统标签或者 UUID，对于 GPT 硬盘还可以使用 GPT 分区标签和 UUID。不应使用内核名称，而 UUID 或分区 UUID（PARTUUID）应该优于标签。更多信息请参考块设备持久化命名。建议在继续阅读本文前先阅读那篇文章。

在本节中，我们将通过示例描述如何使用所有可用的挂载方法来挂载文件系统。以下示例中使用的命令 `lsblk -f` 和 `blkid` 的输出可在块设备持久化命名一文中找到。

内核名称

运行 `lsblk -f` 以列出分区，并在 *NAME* 列中的值的前面加上 `/dev/`。

```
/etc/fstab
# <device> <dir> <type> <options>                                          <dump> <fsck>
/dev/sda2 /     ext4   defaults                                            0      1
/dev/sda1 /boot vfat   defaults,nodev,nosuid,noexec,fmask=0177,dmask=0077 0      2
/dev/sda3 /home ext4   defaults                                            0      2
/dev/sda4 none  swap   defaults                                            0      0
```

警告：块设备的内核名称不是持久化的，可能在每次启动系统时都不同，因此不能被用在配置文件中（包括 `/etc/fstab`）。

文件系统标签

运行 `lsblk -f` 以列出分区，并在 LABEL 列中的值的前面加上 `LABEL=`；或者运行 `blkid` 并使用不带引号的 LABEL 的值：

```
/etc/fstab
# <device>    <dir> <type> <options>                                          <dump>
<fsck>
LABEL=System /     ext4   defaults                                            0      1
LABEL=ESP    /boot vfat   defaults,nodev,nosuid,noexec,fmask=0177,dmask=0077 0      2
LABEL=Data   /home ext4   defaults                                            0      2
LABEL=Swap   none  swap   defaults                                            0      0
```

注意：如果你的标签中包含空格，请参考 #路径名有空格。

文件系统 UUID

运行 `lsblk -f` 以列出分区，并在 UUID 列中的值的前面加上 `UUID=`；或者运行 `blkid` 并使用不带引号的 UUID 的值：

```
/etc/fstab
# <device>                                 <dir> <type> <options>
<dump> <fsck>
UUID=0a3407de-014b-458b-b5c1-848e92a327a3 /     ext4 defaults
0      1
UUID=CBB6-24F2                           /boot vfat
defaults,nodev,nosuid,noexec,fmask=0177,dmask=0077 0      2
UUID=b411dc99-f0a0-4c87-9e05-184977be8539 /home ext4 defaults
0      2
UUID=f9fe0b69-a280-415d-a03a-a32752370dee none  swap defaults
0      0
```

GPT 分区标签

运行 `blkid` 来列出分区,并使用不带引号的 PARTLABEL 的值：

```
/etc/fstab
# <device>                           <dir> <type> <options>
<dump> <fsck>
PARTLABEL=GNU/Linux                  /     ext4 defaults
0      1
PARTLABEL=EFI\040system\040partition /boot vfat
defaults,nodev,nosuid,noexec,fmask=0177,dmask=0077 0      2
PARTLABEL=Home                       /home ext4 defaults
0      2
```

```
PARTLABEL=Swap                          none  swap defaults
0       0
```

注意：如果你的分区标签中包含空格，请参考 #路径名有空格。

GPT 分区 UUID

运行 `blkid` 来列出分区，并使用不带引号的 PARTUUID 的值：

```
/etc/fstab
# <device>                                  <dir> <type> <options>
<dump> <fsck>
PARTUUID=98a81274-10f7-40db-872a-03df048df366 /     ext4 defaults
0       1
PARTUUID=d0d0d110-0a71-4ed6-936a-304969ea36af /boot vfat
defaults,nodev,nosuid,noexec,fmask=0177,dmask=0077 0       2
PARTUUID=7280201c-fc5d-40f2-a9b2-466611d3d49e /home ext4 defaults
0       2
PARTUUID=039b6c1c-7553-4455-9537-1befbc9fbc5b none  swap defaults
0       0
```

提示和技巧

通过 systemd 自动挂载

请参考 systemd.mount(5) 获取所有 systemd 挂载选项。

本地分区

在使用大分区时，可以在 *fsck* 进行检查时启动不依赖该分区的服务以提高效率。这可以通过向 `/etc/fstab` 的分区条目中添加以下选项来完成：

```
x-systemd.automount
```

这会使分区被首次访问时才运行 fsck 和 mount，并且内核会在它可用前缓冲所有对它的文件访问。 该方法适用于如 `/home` 分区非常大的情况。

注意：这会将文件系统类型设为 `autofs`，并默认被 locate 忽略。

远程文件系统

对于远程文件系统的挂载也是一样的。如果你想让它们只在访问时被挂载，就需要使用 `x-systemd.automount` 参数。另外,你还可以使用 `x-systemd.mount-timeout=` 选项来指定 systemd 应该在命令完成前等待多久。另外，`_netdev` 选项可确保 systemd 明白该挂载需要网络，并且在网络就绪后再执行挂载：

```
x-systemd.automount,x-systemd.mount-timeout=30,_netdev
```

加密文件系统

如果你有一个使用密钥文件的加密附属文件系统，可以将 `nofail` 参数添加到 `/etc/crypttab` 和 `/etc/fstab` 中的对应项中。*systemd* 将不会在启动时等待 cryptsetup 完成解密并挂载，而是等到甚至是抵达 default.target 后再完成挂载。通过忽略无需启动后立即可用的加密附属文件系统，该操作可避免由此带来的启动延迟。cryptsetup 的具体配置请参考 dm-crypt/系统配置#非阻塞挂载。

由于挂载服务默认仅会为文件系统可用等待 90 秒，因此任何对密钥文件可用性的延迟都可能导致挂载失败。为避免该情况出现，可以在 fstab 中添加 `x-systemd.mount-timeout=0` 选项，以使挂载服务一直等待到分区解锁完成。

```
/etc/fstab
UUID=0a3407de-014b-458b-b5c1-848e92a327a3 /data ext4 defaults,nofail,x-systemd.device-timeout=0    0 2
```

自动卸载

可以通过 `x-systemd.idle-timeout` 参数来为挂载设置一个空闲超时时间。举个例子：

```
x-systemd.automount,x-systemd.idle-timeout=1min
```

这会使 systemd 在设备空闲 1 分钟后卸载它。

外部设备

**本文或本章节的事实准确性存在争议。**

原因： nofail 选项无法按以下描述工作。（在 Talk:Fstab#3.2 External Device options 中讨论）

对于仅在可用时挂载，未找到时忽略的外部设备，需要使用 `nofail` 选项。该选项可在启动时防止出现报错。例如：

```
/etc/fstab
LABEL=MyExternalDrive /media/backup    jfs    nofail,x-systemd.device-timeout=5    0  2
```

建议将 `nofail` 选项搭配 `x-systemd.device-timeout` 选项使用，原因是默认设备超时时间是 90 秒，因此除非按如上进行配置，否则仅包含 `nofail` 选项且未连接的外部设备将使启动时间延长 90 秒。务必确保超时时间没有设为 0 秒，这会将超时时间设为无限。

路径名有空格

因为空格在 `fstab` 中用于划分列，如果任意一列（*PARTLABEL*，*LABEL* 或挂载点）中含有空格，这些空格必须被替换为转义符 `\` 后面跟着三位数八进制代码 `040`：

```
/etc/fstab
UUID=47FA-4071          /home/username/Camera\040Pictures   vfat  defaults      0  0
LABEL=Storage\040drive /media/100\040GB\040(Storage)       ext4  defaults,user 0  2
```

atime 参数

下列 atime 选项可能影响驱动器性能。

- `strictatime` 选项会在文件每次被访问时更新它们的访问时间。这更多用于 Linux 服务器，而对于桌面用户并没有多少价值。`strictatime` 选项的缺点是即使从页面缓存中读取文件（从内存而不是从驱动器读取）也会导致一次写入。

- `noatime` 选项将在读取文件时不将文件访问时间写入驱动器。开启这个选项对多数应用不会产生影响，除了那些需要知道文件自上次修改后是否被访问的应用。在启用此选项的情况下，只要写入文件，文件的写入时间信息就会更新。
- `nodiratime` 选项仅对目录禁用文件访问时间的写入，而其他文件仍会写入访问时间。

> **注意**：`noatime` 包含了 `nodiratime`。你不需要同时启用两个选项。

- `relatime` 仅当先前访问时间早于当前修改时间时才会更新访问时间。此外，从 Linux 2.6.30 开始，如果与之前的访问时间间隔超过 24 小时，则始终会更新访问时间。在使用 `defaults` 选项，`atime` 选项（即使用内核默认值 `relatime`；具体信息请参考 mount(8) 和 Once upon atime）或没指定任何选项时，将启用该选项。

在使用 Mutt 或其它需要知道一个文件自上次修改后是否被读取的应用时，不应该使用 `noatime` 选项；可以使用 `relatime`，同时还能获得性能提升。

自从内核 4.0 版本起还有另一个相关的选项：

- `lazytime` 通过仅在内存中维护对 inode 时间戳的更改（包括访问、修改和创建时间）来减少对磁盘的写入。硬盘上的时间戳仅在以下情况下更新：(1) 文件 inode 需要针对与文件时间戳无关的某些更改进行更新，(2) 发生与硬盘的同步，(3) 从内存中逐出未删除的 inode 或 (4) 自上次将内存中的副本写入硬盘已超过 24 小时。

: > **警告**：如果系统崩溃，硬盘上的访问和修改时间可能会距当前时间过时最多 24 小时。

注意，`lazytime` 并不用于替代上述 `*atime` 选项，而是与它们**搭配**使用。默认会与 `relatime` 搭配，但在搭配 `strictatime` 时的硬盘写入损耗也较纯 `relatime` 选项相同设置更低。

重新挂载根分区

如果出于某些原因根分区被错误挂载为只读，通过以下命令可以将根分区以读写访问重新挂载：

```
# mount -o remount,rw /
```

GPT 分区自动挂载

在根据可发现分区规范进行了分区的情况下，使用 UEFI/GPT 时可以在 `/etc/fstab` 中省略部分分区，并让 systemd-gpt-auto-generator(8) 自动进行挂载。具体信息请参考 systemd#GPT分区自动挂载。

可以将 `/dev/gpt-auto-root` 作为设备名称来为根分区指定挂载选项：

```
/etc/fstab
/dev/gpt-auto-root   /   ext4   defaults,noatime   0   1
```

Bind 挂载

你可以使用 `bind` 选项链接目录：

```
/etc/fstab
# <device>                              <dir>                          <type> <options>
<dump> <fsck>
UUID=94649E22649E06E0                   /media/user/OS/                ntfs
defaults,rw,errors=remount-ro  0  0
/media/user/OS/Users/user/Music/        /home/user/Music/              none
defaults,bind 0   0
/media/user/OS/Users/user/Pictures/     /home/user/Pictures/           none
defaults,bind 0   0
/media/user/OS/Users/user/Videos/       /home/user/Videos/             none
defaults,bind 0   0
/media/user/OS/Users/user/Downloads/    /home/user/Downloads/          none
defaults,bind 0   0
/media/user/OS/Users/user/Documents/    /home/user/Documents/          none
defaults,bind 0   0
/media/user/OS/Users/user/projects/     /home/user/projects/windows/   none
defaults,bind 0   0
```

详情请见 mount(8) § Bind mount operation。

通过 genfstab 自动生成 fstab

可以使用 *genfstab* 来创建 fstab 文件，具体信息请参考 genfstab.

GUI 工具

以下为一系列可用于修改挂载点的工具，它们的 fstab 编辑功能可能不完整，但常用的功能都有，可以简化你的工作流：

- **GNOME Disks** — 用于处理存储设备的 GNOME 工具，是 gnome包组 的一部分。

: https://apps.gnome.org/DiskUtility/ || gnome-disk-utility包

- **KDE Partition Manager** — 可用于管理电脑上的硬盘、分区和文件系统的工具，是 kde-system包组 的一部分。

: https://apps.kde.org/partitionmanager/ || partitionmanager包

修改用户权限和所有权

如果你希望所有用户都能挂载某个硬盘，可以考虑在 fstab 的对应项中添加以下挂载选项：

注意：同样可用于没有文件权限功能的文件系统，效果为硬盘的所有者和权限被设为挂载该盘的用户。

- `users` - 允许任意用户挂载和卸载（即使由其他用户挂载）对应的文件系统。该选项隐式包含了 noexec，nosuid 和 nodev 选项（除非被类似 users，exec，dev 和 suid 的选项覆盖）。
- `user` - 允许一个普通用户挂载对应的文件系统，且仅允许同一个用户进行卸载。该选项隐式包含了 noexec，nosuid 和 nodev 选项（除非被类似 users，exec，dev 和 suid 的选项覆盖）。

对于没有文件权限功能的文件系统（例如 FAT 和 exFAT），可以显式指定整个硬盘和其中的文件的用户和用户组。可以通过 `/etc/passwd` 查找特定用户的 ID，其中第三列是用户 ID（UID），第四列是组 ID（GID）。

- `uid` - 设置硬盘的所有者 ID
- `gid` - 设置硬盘的组 ID

对于 ext4，btrfs 和其他有权限功能的文件系统，其他用户可能没有查看硬盘的权限。请检查硬盘下的文件权限，并根据需要进行修改。

验证修改

可以通过 `findmnt --verify --verbose` 检查 fstab 中的语法错误和无效选项。

参见

- 完整的设备列表，包括块设备
- 文件系统层次标准
- 通过 TMPFS 获取 30 倍更快的缓存与网站浏览速度

2.7 EFI 系统分区

难度 高级 | **篇幅** 26.4 KB | **所属** 启动过程

本节将讲到

- 检查现有的分区
- 创建分区
- 格式化分区
- 挂载分区
- 提示和技巧

EFI 系统分区（也称为 ESP）是一个与操作系统无关的分区，其中存储了由 UEFI 固件启动的 UEFI 引导加载器、应用程序和驱动，是 UEFI 启动所必须的。

检查现有的分区

如果你正将 Arch Linux 安装到支持 UEFI 且已安装操作系统的计算机上，例如与 Windows 10 双启动，那么你很可能已有 EFI 系统分区。

要查看磁盘分区表和系统分区，以 root 的身份对你想要启动的磁盘使用 fdisk：

```
# fdisk -l /dev/sdx
```

命令将返回：

- 磁盘的分区表：如果分区表是 GPT，则会显示 `Disklabel type: gpt`；如果是 MBR，则会显示 `Disklabel type: dos`。
- 磁盘上分区的列表：在列表中搜索 EFI 系统分区，它通常大小不小于 100 MiB，且类型为 `EFI System` 或 `EFI (FAT-12/16/32)`。要确认这个分区是 ESP，mount 它，然后看看是否包含一个名为 `EFI` 的目录，如果有，那这肯定就是 ESP。

提示：要判断是 FAT12、FAT16 还是 FAT32 文件系统，参阅 FAT#检测FAT类型。

警告：双启动时不要重新格式化 ESP，因为它可能包含启动其他操作系统所需的文件。

如果你找到了现有的 EFI 系统分区，前往#挂载分区，否则，你将需要创建一个，前往#创建分区。

创建分区

下两节介绍如何创建 EFI 系统分区（ESP）。

警告：EFI 系统分区必须是磁盘主要分区表上的物理分区，不能处于 LVM 或软件 RAID 等等之下。

分区大小应足以储存启动加载器和启动所需要的其他文件。

推荐创建1 GiB大小的EFI系统分区以确保其提供足够的空间存放多个内核或统一内核镜像、引导加载器、固件升级文件以及任何其他操作系统或OEM文件。如果还有疑虑，**4 GiB**应该足够任何人使用，比如有些工具像Limine 启动引导器带有用于Btrfs的Snapper集成（英语：Limine#Snapper snapshot integration for Btrfs），支持创建多个可启动的快照。

注意：当然也可以使用一个较小的分区，但要注意潜在的兼容问题：

- 对于早期和/或古怪的 UEFI 实现，可能最少需要 512 MiB。
- 如果你打算把分区挂载到/boot并且只安装一个内核，那么400 MiB足够使用。
- 如果是Arch + Windows 双系统，逻辑扇区大小为4096（高级格式化4Kn 设备）的分区应该至少300 MiB，其他情况应该至少100 MiB。

- 为确保分区能够被格式化为FAT32，逻辑扇区大小512字节的分区大小应该至少 36 MiB，4096字节的分区至少 260 MiB。
- 如果与这些问题都无关，分区大小最小可达 2 MiB，此时只能放得下启动加载器。

GPT 分区磁盘

GUID Partition Table 中 EFI 系统分区以分区类型 GUID `C12A7328-F81F-11D2-BA4B-00A0C93EC93B` 标识。

从以下方法中**任选其一**在 GPT 分区的磁盘上创建 ESP：

- fdisk：创建分区，然后使用命令`t`并指定`uefi`别名将分区类型更改为 `EFI System`。
- gdisk：创建分区类型为 `EF00` 的分区。
- GNU Parted：创建文件系统类型为 `fat32` 的分区，并设置 `esp` 标志。

创建分区之后，应当格式化为一种文件系统。前往#格式化分区。

MBR 分区磁盘

警告：强烈建议使用 GPT 而不是 MBR

- 某些固件可能不支持 UEFI/MBR 启动，因为它不受 Windows Setup 支持。
- *bootctl* 不支持将 systemd-boot 安装到MBR 分区的磁盘，参见 systemd issue 1125。

另请参阅 Partitioning#选择 GPT 还是 MBR 查看 MBR 的限制和 GPT 的优点。

主引导记录中 EFI 系统分区以分区类型 ID `EF` 标识。

从以下方法中**任选其一**在 MBR 分区的磁盘上创建 ESP：

- fdisk: 创建一个主分区，然后使用命令`t`将分区类型更改为 `EFI (FAT-12/16/32)`。
- GNU Parted: 创建文件系统类型为 `fat32` 的分区，并设置 `esp` 标志。

创建分区之后，应当格式化为一种文件系统。前往#格式化分区。

格式化分区

UEFI 规范要求支持 FAT12、FAT16 和 FAT32 文件系统（参见 UEFI specification version 2.10, section 13.3.1.1），但任何合规的厂商都可以支持额外的文件系统。例如，Apple Mac 的固件支持 HFS+ 文件系统。

为避免与其他操作系统的潜在问题，同时既然 UEFI 规范声称 UEFI "encompasses the use of FAT32 for a system partition, and FAT12 or FAT16 for removable

media"\[specs/UEFI/2.10/13_Protocols_Media_Access.html#file-system-format\]，建议使用 FAT32。使用 dosfstools包 中的 mkfs.fat(8) 工具：

```
# mkfs.fat -F 32 /dev/sdxY
```

如果你收到消息 `WARNING: Not enough clusters for a 32 bit FAT!`并且不能创建更大的EFI系统分区，运行 `mkfs.fat -s2 -F32 ...` 或 `-s1` 减小簇大小。否则 UEFI 可能无法读取分区。参见 mkfs.fat(8) 查看支持的簇大小。

小于 32 MiB 的分区可能无法使用 FAT32。这种情况下，格式化为 FAT16 甚至是 FAT12。例如，2 MiB 的 ESP 只能支持 FAT12：

```
# mkfs.fat -F 12 /dev/sdxY
```

挂载分区

内核、initramfs 文件，在大多数情况下还有处理器的微码，都需要能被引导加载程序或 UEFI 本身访问才能成功启动系统。因此，如果想要设置简单，那引导加载程序的选择就会限制 EFI 系统分区可能的挂载点。

注意：如果EFI系统分区没有挂载到`/boot`，确保在升级内核时，没有使用systemd自动挂载机制（包括systemd-gpt-auto-generator）。每次系统或内核升级前都要手动挂载EFI系统分区，否则升级后可能会无法挂载，导致你的内核停留在当前版本并无法更新EFI系统分区中的内核文件。

或者在启动时预加载需要的内核模块，例如：

```
/etc/modules-load.d/vfat.conf
vfat
nls_cp437
nls_ascii
```

典型挂载点

有三种挂载EFI系统分区的典型情况：

- 挂载 EFI系统分区 到 `/boot`：
- 便于系统维护和管理，`/boot`是微码包安装CPU微码initramfs文件和mkinitcpio安装内核与initramfs镜像的默认位置。
- FAT在挂载时设置了全局属性，这会阻止设置文件特定的权限和拓展属性
- 通常安装在`/boot`中的文件与EFI相关文件共享EFI系统分区，提高了EFI系统分区的大小需求
- 双启动的情况下，系统特定的启动文件会处在被其它系统修改操作的潜在危险中
- 无法加密/boot，因为固件需要读取EFI相关文件
- 挂载 EFI系统分区到`/efi`：

- 当EFI系统分区包含其他系统的文件时最好和操作系统相关的文件分开，这确保了操作系统相关和EFI相关文件的分离。
- 只有EFI二进制文件（引导加载程序（和可选驱动））和（或）统一内核镜像会安装在EFI系统分区，避免了安装在/boot中的文件对EFI系统分区的大小需求，节约了EFI系统分区的空间。
- 允许保留/boot中文件的Linux特定的文件系统权限，避免了FAT的限制。
- 允许根据需求单独挂载EFI系统分区，例如需要升级引导加载程序时。
- 如果加密整个系统并且配置恰当，除少数需要文件没有被加密，/boot中的文件能够被加密保护：内核及其他文件储存在加密分区，统一内核镜像或引导加载程序通过相应的文件系统驱动来访问这些文件。
- 挂载 EFI系统分区到/efi， 然后再挂载一个“拓展引导加载器分区”（XBOOTLDR）分区到 /boot。在以前创建的 ESP 太小而无法容纳多个引导加载程序以及内核但 ESP 又无法轻松调整大小时（例如在 Windows 之后将 Linux 安装到双引导（多引导）时），这可能非常有用。至少在 systemd-boot#使用XBOOTLDR安装时支持此方法。

注意：

- /efi是/boot/efi的替代挂载点，/boot/efi在过去被使用但现在不推荐。
- /efi在安装一开始时不存在，需要先用 mkdir(1) 创建再挂载EFI系统分区到该目录。

替代挂载点

如果不使用#典型挂载点中的方法，就需要将引导文件复制到 ESP（以下称为 *esp*）。

```
# mkdir -p esp/EFI/arch
# cp -a /boot/vmlinuz-linux esp/EFI/arch/
# cp -a /boot/initramfs-linux.img esp/EFI/arch/
# cp -a /boot/initramfs-linux-fallback.img esp/EFI/arch/
```

注意：如果你使用了外部微码镜像，这些文件同样还需要复制到复制到启动项的位置。

此外，还需要使 ESP 中的文件在以后的内核更新中保持最新。否则可能会导致系统无法启动。以下部分讨论了几种自动化的机制。

使用bind挂载

除了将EFI系统分区挂载到/boot你也可以使用bind挂载将分区中的目录挂载到/boot（参考mount(8)）。这样pacman就可以直接更新内核文件并保持EFI系统分区的规划。

注意：这需要内核和引导加载程序兼容FAT32。通常安装的Arch没有这个问题，但可能存在于其他发行版中（也就是说需要在 /boot中创建软连接，参见。

按照#替代挂载点节内容，复制所有引导文件到你的EFI系统分区，分区挂载点在/boot外面。然后bind挂载目录：

```
# mount --bind esp/EFI/arch /boot
```

检查生效后，编辑Fstab使修改持续有效：

```
/etc/fstab
esp/EFI/arch /boot none defaults,bind 0 0
```

使用 systemd

Systemd具备事件触发型任务能力。在本特定场景中，该系统利用路径监控功能来检测/boot/目录下EFISTUB内核与初始化内存盘文件的更新情况，并在文件被更新时执行同步操作。选择监视initramfs-linux-fallback.img文件是因为该文件由mkinitcpio最后生成，可确保所有文件构建完成后再启动复制流程。需要创建的*systemd*路径单元文件及服务单元文件包括：

```
/etc/systemd/system/efistub-update.path
[Unit]
Description=Copy EFISTUB Kernel to EFI system partition

[Path]
PathChanged=/boot/initramfs-linux-fallback.img

[Install]
WantedBy=multi-user.target
WantedBy=system-update.target
/etc/systemd/system/efistub-update.service
[Unit]
Description=Copy EFISTUB Kernel to EFI system partition

[Service]
Type=oneshot
ExecStart=/usr/bin/cp -af /boot/vmlinuz-linux esp/EFI/arch/
ExecStart=/usr/bin/cp -af /boot/initramfs-linux.img esp/EFI/arch/
ExecStart=/usr/bin/cp -af /boot/initramfs-linux-fallback.img esp/EFI/arch/
```

随后启用并启动efistub-update.path。

提示：若需使用自有密钥实现安全启动，您可以配置该服务，使其额外使用 sbsigntools包 对映像进行签署：

```
ExecStart=/usr/bin/sbsign --key /path/to/db.key --cert /path/to/db.crt --output
esp/EFI/arch/vmlinuz-linux /boot/vmlinuz-linux
```

使用文件系统事件

文件系统事件可用于在内核更新后运行脚本同步 EFISTUB 内核。下面是一个使用 incron（英语：incron） 的示例。

```
/usr/local/bin/efistub-update
#!/bin/sh
cp -af /boot/vmlinuz-linux esp/EFI/arch/
cp -af /boot/initramfs-linux.img esp/EFI/arch/
cp -af /boot/initramfs-linux-fallback.img esp/EFI/arch/
```

注意：第一个参数 /boot/initramfs-linux-fallback.img 是要监视的文件。第二个参数 IN_CLOSE_WRITE 是要监视的动作。第三个参数 /usr/local/bin/efistub-update 是要执行的脚本。

```
/etc/incron.d/efistub-update.conf
/boot/initramfs-linux-fallback.img IN_CLOSE_WRITE /usr/local/bin/efistub-update
```

要使用这个方法，启用 `incrond.service`。

使用 mkinitcpio preset

因为`/etc/mkinitcpio.d/`中的preset支持shell脚本，编辑presets可以复制内核和initramfs到ESP。

替换 mkinitcpio 钩子

编辑文件 `/etc/mkinitcpio.d/linux.preset`：

```
/etc/mkinitcpio.d/linux.preset
# mkinitcpio preset file for the 'linux' package

# Directory to install the kernel, the initramfs...
ESP_DIR="esp/EFI/arch"

#ALL_config="/etc/mkinitcpio.conf"
ALL_kver="${ESP_DIR}/vmlinuz-linux"

PRESETS=('default' 'fallback')

#default_config="/etc/mkinitcpio.conf"
default_image="${ESP_DIR}/initramfs-linux.img"
default_options=""

#fallback_config="/etc/mkinitcpio.conf"
fallback_image="${ESP_DIR}/initramfs-linux-fallback.img"
fallback_options="-S autodetect"
```

要测试它，只需运行：

```
# rm /boot/initramfs-linux-fallback.img /boot/initramfs-linux.img
# mv /boot/vmlinuz-linux esp/EFI/arch/
# mkinitcpio -p linux
```

另一个例子

```
/etc/mkinitcpio.d/linux.preset
ESP_DIR="esp/EFI/arch"
#ALL_config="/etc/mkinitcpio.conf"
ALL_kver="$ESP_DIR/vmlinuz-linux$suffix"
PRESETS=('default')
default_config="/etc/mkinitcpio.conf"
default_image="$ESP_DIR/initramfs-linux$suffix.img"
/etc/mkinitcpio.d/linux-zen.preset
suffix='-zen'
source /etc/mkinitcpio.d/linux.preset
```

使用 mkinitcpio post 钩子

mkinitcpio post 钩子能够在initramfs生成后，复制内核和initramfs镜像到所需的位置。

创建下面的文件并使文件可执行:

```
/etc/initcpio/post/copy-kernel-and-initramfs
#!/usr/bin/env bash

kernel="$1"
initrd="$2"
target_dir="esp/EFI/arch"
files_to_copy=()
```

```
for file in "$kernel" "$initrd"; do
       if [[ -n "$file" ]] && ! cmp -s -- "$file" "${target_dir}/${file##*/}"; then
              files_to_copy+=("$file")
       fi
done

(( ! ${#files_to_copy[@]} )) && exit 0

cp -af -- "${files_to_copy[@]}" "${target_dir}/"
```

使用 pacman 钩子

最后一个选项依赖于在事务结束时运行的 pacman 钩子。

第一个文件是一个监控相关文件的钩子，如果文件在前一个事务中被修改，钩子就会运行。

```
/etc/pacman.d/hooks/999-kernel-efi-copy.hook
[Trigger]
Type = Path
Operation = Install
Operation = Upgrade
Target = usr/lib/modules/*/vmlinuz
Target = usr/lib/initcpio/*
Target = boot/*-ucode.img

[Action]
Description = Copying linux and initramfs to EFI directory...
When = PostTransaction
Exec = /usr/local/bin/kernel-efi-copy.sh
```

第二个文件是脚本本身。创建文件并使其可执行：

```
/usr/local/bin/kernel-efi-copy.sh
#!/bin/sh
#
# Copy kernel and initramfs images to EFI directory
#

ESP_DIR="esp/EFI/arch"

for file in /boot/vmlinuz*
do
        cp -af "$file" "$ESP_DIR/$(basename "$file").efi"
        [ $? -ne 0 ] && exit 1
done

for file in /boot/initramfs*
do
        cp -af "$file" "$ESP_DIR/"
        [ $? -ne 0 ] && exit 1
done

[ -e /boot/intel-ucode.img ] && cp -af /boot/intel-ucode.img "$ESP_DIR/"
[ -e /boot/amd-ucode.img ] && cp -af /boot/amd-ucode.img "$ESP_DIR/"

exit 0
```

提示和技巧

替换为更大的分区

如果硬盘已经预装操作系统，那么EFI系统分区大小可能会比#创建分区中推荐的小。例如Windows安装程序会在一个非4Kn设备上创建一个很小的100 MiB EFI系统分区。

这种情况下最好新创建一个更大的EFI系统分区以避免存储空间耗尽。

在Windows里为新分区腾出空间

在Windows下，使用磁盘管理(`diskmgmt.msc`)或在命令行使用`diskpart.exe`工具管理分区。

以管理员权限运行`diskmgmt.msc`。

1. 右键C盘分区，然后选择*压缩卷*
2. 输入`4096`作为压缩的空间量，并点击*压缩*。

之后在C盘后方应该会出现4 GiB的未分配空间。

引导进入Arch Linux或Arch Linux安装介质环境为下一步创建新分区做准备。

删除旧的分区并创建一个新的ESP

首先确保备份好原来EFI系统分区的内容，若EFI系统分区的挂载点为*esp*：

```
# cp -a esp /esp_backup
```

卸载EFI系统分区：

```
# umount esp
```

注意：在本地安装的系统下你可能还需要停止 `*esp*.mount` 和 `*esp*.automount` 单元以防止系统再次自动挂载它。

运行*blkid*并记下旧分区的UUID和PARTUUID，下一步这些将值用于新分区上。

```
# blkid
/dev/sdxY: UUID="XXXX-XXXX" BLOCK_SIZE="512" TYPE="vfat" PARTLABEL="EFI system
partition" PARTUUID="YYYYYYYY-YYYY-YYYY-YYYY-YYYYYYYYYYYY"
```

使用gptfdisk包的sgdisk删除旧分区：

```
# sgdisk --delete=Y /dev/sdx
```

在最大的未分配空间上创建新分区并指定PARTLABEL和使用旧分区的PARTUUID：

```
# sgdisk --align-end --largest-new=0 --typecode=0:ef00 --change-name=0:'EFI system
partition' --partition-guid=0:YYYYYYYY-YYYY-YYYY-YYYY-YYYYYYYYYYYY /dev/sdx
```

使用*fdisk*确认新创建的大小为4 GiB的EFI系统分区：

```
# fdisk -l /dev/sdx
...
Device          Start        End    Sectors  Size Type
/dev/sdx1  158099456 166488063   8388608    4G EFI System
/dev/sdx2     206848    239615     32768   16M Microsoft reserved
/dev/sdx3     239616 158099455 157859840 75.3G Microsoft basic data
/dev/sdx4  166488064 167768063   1280000  625M Windows recovery environment
/dev/sdx5  167768064 176156671   8388608    4G Linux swap
/dev/sdx6  176156672 243265535  67108864   32G Linux root (x86-64)
Partition table entries are not in disk order.
```

分区编号在删除和创建分区后没有重新排列，所以EFI系统分区编号应该和之前一样。

将新分区格式化为FAT32，并使用旧的UUID（需要删除UUID中的“-”横线符）：

```
# mkfs.fat -F 32 -i XXXXXXXX /dev/sdxY
```

最后挂载新分区并恢复原有内容：

```
# mount /dev/sdxY esp
# cp -a /esp_backup/. esp/
```

如果你先前停止了`*esp*.automount`，再次启动它。

通过牺牲相邻的swap分区来扩大ESP

如果swap正好在EFI系统分区后面，你可以牺牲swap空间用于扩大EFI系统分区。例如你的分区布局类似下面的例子：

```
# fdisk -l /dev/sdx
...
Device        Start       End   Sectors  Size Type
/dev/sdx1      2048    616447    614400  300M EFI System
/dev/sdx2    616448   9005055   8388608    4G Linux swap
/dev/sdx3   9005056 125827071 116822016 55.7G Linux root (x86-64)
```

首先停用swap分区，并把它从fstab里删除。

使用fdisk删除swap分区并增大EFI系统分区。

1. 运行：

```
    # fdisk -l /dev/sdx
```

1. 使用`d`命令删除swap分区（在示例中swap分区的分区号是`2`）。
2. 使用`e`命令增大EFI系统分区（在示例中EFI系统分区的分区号是`1`）。使用给出的默认值作为新分区的大小并按下`Enter`确认。
3. 通过{{ic|w}命令将修改落盘并退出fdisk。

分区大小修改后需要修改分区内的文件系统大小。因为fatresize(1) 存在问题并且 libparted 不能修改FAT卷的大小为确切的值，唯一的办法就是备份文件系统的文件，然后创建新的分区来利用所有分区空间。

记下文件系统的UUID：

```
$ lsblk -dno UUID /dev/sdxY
XXXX-XXXX
```

备份好原来EFI系统分区的内容，若EFI系统分区的挂载点为*esp*：

```
# cp -a esp /esp_backup
```

卸载EFI系统分区：

```
# umount esp
```

注意：在本地安装的系统下你可能还需要停止 `*esp*.mount` 和 `*esp*.automount` 单元以防止系统再次自动挂载它。

从分区中擦除文件系统的signature以避免受到旧文件系统的影响：

```
# wipefs -af /dev/sdxY
```

将新分区格式化为FAT32，并使用旧的UUID（需要删除UUID中的“-”横线符）：

```
# mkfs.fat -F 32 -i XXXXXXXX /dev/sdxY
```
最后挂载新分区并恢复原有内容：

```
# mount /dev/sdxY esp
# cp -a /esp_backup/. esp/
```
如果你先前停止了`*esp*.automount`，再次启动它。

由于现在没有了swap分区，将swap放在一个交换文件上。

故障排除

ESP在软RAID1上

将ESP放在RAID1阵列上是可能的，但这么做也会带来数据损坏的风险，创建ESP时也要做额外的考虑，详情参见和还有UEFI booting and RAID1。

整个方案的关键点是使用`--metadata 1.0`将RAID metadata放在分区尾部，否则固件无法访问ESP：

```
# mdadm --create --verbose --level=1 --metadata=1.0 --raid-
devices=2 /dev/md/ESP /dev/sdaX /dev/sdbY
```
或者如果你的ESP不会频繁修改，可以在进行相关更新时把主要ESP的修改复制到不同磁盘上的备用ESP上。备用ESP的引导启动项可以用efibootmgr手动添加。参见debian wiki。需要注意这个办法虽然避免了RAID方法的风险，但只在使用单系统时有用。

固件看不到 EFI 目录

如果要给 FAT 文件系统一个卷名（即文件系统标签），请不要将其命名为 `EFI`。卷名和 EFI 目录名称相同可能会触发某些固件中的错误，导致固件表现得好像 EFI 目录不存在一样。

参见

- EFI 系统分区与默认启动行为
- Multi Boot Linux With One Boot Partition | John Ramsden

2.8 Arch 的启动流程

难度 高级 | **篇幅** 26.9 KB | **所属** 启动过程

本节将讲到

- 固件类型
- 系统初始化
- 引导加载程序
- 内核
- initramfs

为了启动 Arch Linux，必须配置一个与 Linux 兼容的引导加载程序。引导加载程序负责在初始化启动进程之前，加载好内核和 initramfs，具体过程因 BIOS 和 UEFI 系统而异。

固件类型

固件是开机时最先执行的程序。

提示：

- 本文时常以 *BIOS* 和 *UEFI* 代称*固件*。
- 勿与 Linux 固件混淆。

UEFI

统一可扩展固件接口（Unified Extensible Firmware Interface，UEFI）支持读取分区表和文件系统。UEFI 不从主引导记录（MBR）中启动任何引导代码（无论其是否存在），相反，UEFI 的启动过程依赖非易失性随机访问存储器（NVRAM）中的引导条目。

UEFI 规范要求支持 FAT12、FAT16 和 FAT32 文件系统（参见 UEFI 规范 2.11 版 13.3.1.1 小节），但每个符合规范的厂商可以选择添加对其它文件系统的支持；比如，苹果的固件支持 HFS+ 或 APFS 文件系统。UEFI 的一些实现方案还支持光盘的 ISO 9660 文件系统。

UEFI 会启动 EFI 应用程序，例如引导加载程序、引导管理器和 UEFI Shell 等等。这些应用程序通常以文件形式存储在 EFI 系统分区中。厂商可以将其特定文件存储在 EFI 系统分区中的 `/EFI/*vendor_name*` 文件夹下。应用程序可以通过在 NVRAM 中添加引导项或从 UEFI shell 中启动。

UEFI 规范通过兼容性支持模块（Compatibility Support Module，CSM）来支持传统 BIOS 引导。如果在 UEFI 中启用了 CSM，UEFI 会为所有驱动器生成 CSM 引导项。如果选择从某一个 CSM 引导项启动，UEFI 的 CSM 会尝试从这个磁盘的 MBR 引导代码启动。

注意：英特尔正逐渐取消对 CSM 的支持。以后也许不能再依赖此特性。

BIOS

BIOS，又称基本输入输出系统（Basic Input-Output System），大多数情况下储存在主板自身的一块闪存内，独立于其它系统存储。其最早是为 IBM PC 开发，用于处理硬件初始化和启动过程。从 2010 年起已逐渐被技术上没有类似限制的 UEFI 替换。

系统初始化

系统上电时，会执行加电自检（Power-on self-test，POST）。详细信息可参考 Hugo Landau 的 Modern CPUs have a backstage cast 一文。

在 UEFI 下的情况

1. 加电自检后，UEFI 初始化引导所需的硬件（硬盘、键盘控制器等等）。
2. 固件读取 NVRAM 中的引导项，以决定要启动哪一个 EFI 应用程序，以及从哪启动（比如从哪一个硬盘和分区）。
3. 一个引导项可能对应的只是一块硬盘。在这种情况下，固件会寻找硬盘上的 EFI 系统分区，并尝试在后备引导路径 `\EFI\BOOT\BOOTx64.EFI` 处（在 IA32（32 位）UEFI 的系统上为 `BOOTIA32.EFI`）查找 EFI 应用程序。这就是 UEFI 可引导可移除介质的工作原理。
4. 固件启动 EFI 应用程序。
5. 这可以是一个#引导加载程序，或者是使用 EFISTUB 的 Arch 内核本体。
6. 还可以是一些其他的 EFI 应用程序，比如 UEFI shell 或引导管理器（例如 systemd-boot 或 rEFInd）。

如果启用了安全启动，启动过程将会通过签名验证 EFI 二进制文件的真实性。

注意：一些 UEFI 系统只能从后备引导路径启动。

多重引导

由于每个操作系统或厂商都可以维护自己在 EFI 系统分区中的文件，同时不影响其他系统，所以 UEFI 的多重引导的原理就是启动不同的、与特定操作系统引导加载程序所对应的 EFI 应用程序。这避免了依赖一个#引导加载程序去加载另一个操作系统的链式加载机制。

另请参阅 Arch + Windows 双系统。

在 BIOS 下的情况

1. 上电自检后，BIOS 初始化引导所需的硬件（硬盘、键盘控制器等等）。
2. BIOS 启动在“BIOS 硬盘顺序”中第一块硬盘上的前 440 字节代码（即主引导记录引导代码区域）。
3. 引导加载程序在 MBR 引导代码的第一阶段，之后会从下列任意一处启动第二阶段代码（如果有的话）：
4. MBR 之后的下一个磁盘扇区，即所谓 MBR 后间隙（post-MBR gap，仅在 MBR 分区表上有）。
5. 分区或者无分区磁盘的卷引导记录（Volume Boot Record，VBR）。

6. GRUB 特定 BIOS 引导分区_特殊操作)（仅限 GPT 分区硬盘上的 GRUB，用于 GPT 上没有 MBR 后间隙的情况）
7. 真正的#引导加载程序启动。
8. 随后，引导加载程序通过链式加载或直接加载操作系统内核的方式加载操作系统。

引导加载程序

引导加载程序（boot loader，又称引导加载器、启动加载器或启动引导器）是由计算机固件（BIOS 或 UEFI）启动的软件，负责用指定内核参数加载内核和其它外置 initramfs 映像。

引导管理器（boot manager，又称启动管理器）让用户使用启动选项菜单或其他方式控制启动过程——也就是说，仅用于运行其他 EFI 应用程序。

提示：一些程序，例如 GRUB 及 GRUB Legacy 兼具引导加载程序和引导管理器的功能。

在 UEFI 的情况下，内核本身可以由 UEFI 使用 EFI boot stub（英语：EFI boot stub） 直接启动。要在引导前编辑内核参数，可以使用引导管理器或是单独的引导加载程序。

使用 32 位 IA32 UEFI 固件的系统需要使用支持混合启动模式的引导加载程序。

警告：引导加载程序必须能够访问通常位于 `/boot` 目录下的内核和 initramfs 映像才能成功引导 Arch 系统。也就是说，引导加载程序必须解决从块设备、堆叠块设备（LVM、RAID、dm-crypt、LUKS 等）开始，到内核和 initramfs 映像所在文件系统为止的访问。

因为几乎没有引导加载程序支持堆叠块设备，并且文件系统引入的一些新特性可能尚未有任何引导加载程序支持（例如 archlinux/packaging/packages/grub#7、FS#79857、FS#59047、FS#58137、FS#51879、FS#46856、FS#38750、FS#21733 和 fscrypt 加密目录），所以用广泛支持的文件系统（例如 FAT32）单独创建 /boot 分区通常更可行。

功能比较

注意：

- 由于 GPT 是 UEFI 规范的一部分，因此所有的 UEFI 引导加载程序都支持 GPT 磁盘。在 BIOS 上使用 GPT 磁盘是可行的，可以使用 Hybrid MBR 的“混合引导（hybrid booting）”，或者使用新的纯 GPT 协议。但是这个协议可能在某些 BIOS 实现上会出问题，详情请参考 Rodsbooks。
- 作为 UEFI 规范的一部分，所有 UEFI 引导加载程序都支持安全启动，但有可能会存在一些限制。

名称	固件	分区表	多重引导	文件系统	注意		
BIOS	UEFI	MBR	GPT				
Clover	是	是	否	是	是	可扩展 2,5	可以在过时 BIOS 系统上模拟 UEFI。

名称	固件	分区表	多重引导	文件系统	注意		
EFI boot stub（英语：EFI boot stub）	–	是1	是	是	–	继承自固件2	内核是有效的 EFI 可执行文件，可直接从 UEFI 或其它 UEFI 引导加载器启动。
GRUB	是	是3	是	是	是	内置	支持 RAID，LUKS（Argon2 PBKDFs 除外）和 LVM（但是不支持精简配置卷）。平台相关限制请参考 GRUB。
Limine	是	是3	是	是	是	有限	
rEFInd	否	是	是	是	是4	可扩展2,5	支持自动检测内核和参数，无需明确配置，并支持快速启动。
Syslinux	是	部分1	是	是	部分	有限	不支持某些文件系统功能。 只能访问自身所处的文件系统。
systemd-boot	否	是3	手动	是	是4	可扩展2,5	无法从 ESP 或扩展引导加载程序分区（XBOOTLDR）以外的分区启动二进制文件。 支持自动检测放入 `*esp*/EFI/Linux/` 的统一内核映像。
统一内核映像	–	是3	是	是	–	继承自固件2	systemd-stub(7)、内核、initramfs、内核命令行打包而成的 EFI 可执行文件，可直接从 UEFI 固件或另一个引导加载程序加载。
GRUB Legacy	是	否	是	否	是	有限	停止开发，转为 GRUB。
LILO	是	否	是	部分	是	有限	因为局限性（如与 Btrfs、GPT 和 RAID 搭配使用时）已停止开发。

1. 虽然二进制文件可以被签名用于安全启动，但它不会进行后续验证，从而破坏了信任链。
2. 文件系统支持是从固件继承的。UEFI 规范要求支持 FAT12，FAT16 和 FAT32 文件系统，但厂商可选择添加对其他文件系统的支持。比如说，苹果 Mac 中的固件支持 HFS+ 文件系统。如果固件提供在启动时加载 UEFI 驱动程序的接

口，则可以通过加载文件系统驱动程序（需单独获取）的方式添加对其他文件系统的支持。
3. 支持混合模式启动，即可以在 32 位 IA32 UEFI 固件上启动 64 位 x86_64 Linux 内核
4. 一种启动管理器。它只能启动其他的 EFI 应用程序，例如，使用 `CONFIG_EFI_STUB=y` 参数编译的 Linux 内核映像和 Windows Boot Manager `bootmgfw.efi`。
5. 支持加载 UEFI 文件系统驱动。

另请参见维基百科：引导加载程序比较。

内核

引导加载器会启动包含内核的 vmlinux 映像。

内核是操作系统的核心。它运行于一个叫*内核空间*的底层上，负责机器硬件和应用程序之间的交流。在继续进入用户空间前，内核会首先执行硬件枚举和初始化。具体细节请参考zhwp:内核和zhwp:Linux内核。

initramfs

initramfs（初始内存文件系统，***init***ial ***RAM f***ile ***s***ystem）映像是一个 cpio 存档文件，为*早期用户空间*（见下文）启动晚期用户空间提供了必要的文件。这包括了所有用于定位，访问和挂载根文件系统的内核模块、用户空间工具、相关库文件、类似 udev 规则的支持文件等。得益于 initramfs 的概念，它可以处理更加复杂的配置场景，例如从外置硬盘启动，堆叠设备（例如逻辑卷，软 RAID，压缩和加密），或是在早期用户空间中运行一个微型 SSH 服务器，以供远程解锁或为根文件系统执行维护任务。

绝大部分内核模块都将在初始化流程的后期阶段，由 udev 在根切换到根文件系统后加载。

具体流程如下：

1. `/` 下的根文件系统原本是一个空的 rootfs，它是一个特殊的 tmpfs 或 ramfs 实例。这里就是 initramfs 会解压到的临时根文件系统。
2. 内核会将其内置 initramfs 解压到临时根文件系统下。Arch Linux 官方支持的内核使用空白存档作为内置 initramfs，即构建内核时的默认行为。
3. 然后，内核会按照引导加载器传递的命令行参数指定的顺序解压外置 initramfs 映像，覆盖掉之前内置 initramfs 或其它解压出来的文件。注意，可以将多个 initramfs 映像合并为一个文件，内核会按照文件内的顺序加载映像。

4. 如果首个 initramfs 映像未经压缩，那么内核会在解包该映像后
   在 `/kernel/x86/microcode/` 目录查找 CPU 微码更新，
   在 `/kernel/firmware/acpi/` 目录查找 ACPI 表更新。
5. 在适用的情况下，在处理完 CPU 微码和 ACPI 表更新后，内核会继续解压剩余的 initramfs 映像。

initramfs 映像是 Arch Linux 推荐的早期用户空间配置方法，并可通过 mkinitcpio，dracut 或 booster 来生成。

不使用 initramfs

从 6.13.8 版本开始，官方支持的内核已内置 Btrfs 和 Ext4 的驱动。

因此内核可以直接使用这些文件系统格式的根分区，然后再加载其它需要的外部模块。但有几点需要注意：

- 不能使用 GPT 分区自动挂载，必须使用 `root` 内核参数。
- 只能使用 `PARTUUID` 和 `PARTLABEL` 作为根分区（`root`）的块设备持久化名称。
- `rootflags` 的挂载选项比较有限，例如 `noatime` 就无法使用。要绕过该问题，可以先以只读进行挂载（`rootflags=ro`），然后使用 fstab 进行重新挂载来应用所需的挂载选项。
- 由于 GPT 分区自动挂载不可用，无需启用 systemd-gpt-auto-generator(8)，启用甚至还会导致问题出现，可以通过 `systemd.gpt_auto=no` 将其禁用。

注意：目前只内置了 SCSI/SATA/AHCI 的模块，其它存储设备（例如 NVMe（英语：NVMe），USB，device mapper 等）不适用于该节内容。另外，LVM 和加密也必须搭配 initramfs 使用，这两者需要用户空间工具才能正常工作。

另外，早期微码加载也必须搭配 initramfs 使用，但没必要为其构建完整映像，Arch 可以将微码放置在单独的 initramfs 文件中，以供单独使用。

即使没有提供 initramfs 映像，内核仍会包含一个空映像以供启动，以防止根分区固定出现问题。

早期用户空间

早期用户空间阶段（亦称“initramfs 阶段”）在由 #initramfs 映像提供文件的 rootfs 中进行，始于内核以 PID 1 执行 `/init`。

早期用户空间的功能可以进行配置，但主要是引导系统到能够访问真正根文件系统的状态，这主要包括：

- systemd-modules-load(8) 加载内核模块（基于 systemd 的 initramfs），例如挂载真正根文件系统所需的任何块设备模块。
- 构建访问真正根文件系统所需的存储栈（例如通过 dm-crypt、dm-verity（英语：dm-verity）、mdadm、LVM、systemd-repart（英语：systemd-repart）等）。
- 解密真正根文件系统（若适用）。
- udev 将块设备持久化名称解析为实际设备。
- 加载 DRM 模块（因为默认启用的 KMS 早启动）。

需要注意的是，早期用户空间不仅仅用于设置真正根文件系统。有些任务只能在挂载真正根文件系统之前执行，例如 fsck和从休眠中恢复。

在早期用户空间的最后阶段，真正根文件系统会被挂载到 `/sysroot/`（基于 systemd 的 initramfs）或 `/new_root/`（基于 BusyBox 的 initramfs），然后通过 `systemctl switch-root`（基于 systemd 的 initramfs）或 switch_root(8)（基于 BusyBox 的 initramfs）切换到真正根文件系统。最后通过执行真正根文件系统中的 init 程序启动晚期用户空间。

晚期用户空间

晚期用户空间从 init 进程开始。Arch 官方支持的 systemd 基于单元和服务的概念，但这里描述的功能在很大程度上与其它 init 系统重叠。

getty

init 会为每个虚拟终端（通常有六个）调用一次 getty，它会初始化终端并保护其免受未授权访问。在提供用户名和密码后，*getty* 会对照 `/etc/passwd` 和 `/etc/shadow` 检查是否正确。如果正确，就接着调用 login(1)。

login

login 会根据 `/etc/passwd` 设置环境变量并启动用户 shell，从而为用户配置一个会话。在成功登录后，启动登录 shell 前，*login* 程序会显示 /etc/motd)（***m***essage ***o***f ***t***he ***d***ay）的内容，你可以用它来显示服务条款以提醒用户你的本地策略，也可以显示其它提示信息。

shell

用户的 shell 启动后，在显示命令行提示符前，通常会执行一个运行时配置文件（例如 bashrc）。如果用户账户配置为在登录时自动启动 X，那么运行时配置文件会调用 startx 或 xinit，具体内容请参考#图形会话（Xorg）。

显示管理器

另外，在特定虚拟终端下，init 可配置为启动显示管理器，而不是 *getty*。要达成该效果，需要手动启用其 systemd 服务文件，之后显示管理器就会启动一个图形会话。

图形会话（Xorg）

xinit 会调用用户的 xinitrc 运行时配置文件，后者一般会启动一个窗口管理器或桌面环境。如果用户退出了窗口管理器，*xinit*、*startx*、shell、login 就会依次中断，返回到 *getty* 或显示管理器。

参见

- Wikipedia:Booting process of Linux
- Inside the Linux boot process
- Rod Smith - Managing EFI Boot Loaders for Linux
- NeoSmart: The BIOS/MBR Boot Process
- Lennart Poettering - Linux Boot Partitions and How to Set Them Up
- Wikipedia:initramfs
- Early Userspace in Arch Linux
- Kernel Newbie Corner: initrd and initramfs
- bootup(7)（主要涉及 systemd 早期用户空间部分）

2.9 Arch + Windows 双系统

难度 高级 | **篇幅** 31.7 KB | **所属** 启动过程

本节将讲到

- 重要信息
- 安装
- 时间标准
- 蓝牙配对
- 参见

重要信息

Windows 在 UEFI 和 BIOS 上的限制

不同版本的 Windows 对固件启动模式和分区表有不同的需求。以下依版本列出。

注意：下列情形是 Windows安装程序 的限制。但 Windows 本身仍可能在 Windows安装程序 不支持的情况下正常运行。例如，在绕过相关安装检测后，Windows 11 仍可在 BIOS/MBR 的配置上运行。

- **Windows 8/8.1** 和 **10** 的 **x86 32**位版本只支持从 GPT 磁盘启动IA32 UEFI模式，或者从 MBR 磁盘启动 BIOS 模式。它们不支持从 GPT/MBR 磁盘启动 x86_64 UEFI，从MBR 磁盘启动 x86_64 UEFI，或者从 GPT 磁盘启动

BIOS。目前市面上，唯一已知装载 IA32 (U)EFI 的系统是一些较老的 Intel Mac（2010年前的型号？）以及 Intel Atom 系统级芯片（Clover Trail 和 Bay Trail）的Windows平板电脑，这些设备只支持在IA32 UEFI模式下并且只从 GPT 磁盘启动。

- **Windows 8/8.1** 和 **10** 的 **x86_64**版本只支持从 GPT 磁盘启动 x86_64 UEFI 模式，或者从 MBR 磁盘启动 BIOS 模式。它们不支持IA32 UEFI启动，从 MBR 磁盘启动 x86_64 UEFI，或者从 GPT 磁盘启动 BIOS。
- **Windows 11** 只支持 **x86_64** 并从 GPT 磁盘以 UEFI 模式启动。

对于预装系统的计算机：

- 所有预装 Windows XP、Vista 或 7 32位的系统，无论服务包级别、位宽度、版本（SKU），或固件中是否支持UEFI，都默认以 BIOS/MBR 模式启动。
- 大部分预装 Windows 7 x86_64的系统，无论服务包级别、位宽度还是版本（SKU），默认均以 BIOS/MBR 模式启动。极少数较新的预装 Windows 7 的系统默认以 x86_64 UEFI/GPT 模式启动。
- 所有预装 Windows 8/8.1、10 和 11 的系统都默认以 UEFI/GPT 模式启动。在 Windows 10 之前，固件位宽与 Windows 位宽匹配，即 x86_64 Windows 在 x86_64 UEFI 模式下启动，32位 Windows 在IA32 UEFI模式下启动。

判断 Windows 启动模式的简单方式：

- 启动到Windows
- 按`Win+R`打开“运行”
- 在“运行”对话框中输入`msinfo32`并回车
- 系统信息->系统摘要->BIOS模式
- 如果值为 `UEFI`，Windows 以 UEFI/GPT 模式启动。如果值为 `Legacy`，Windows 以 BIOS/MBR 模式启动。

通常，Windows 安装程序基于固件模式强制选择磁盘分区类型：UEFI 启动只能装到 GPT 盘，BIOS 启动只能装到 MBR 盘。这是Windows安装程序强加的限制，截至2014年4月，没有官方（Microsoft）支持的 在UEFI/MBR或BIOS/GPT配置中安装 Windows 的方法。因此，Windows只支持UEFI/GPT启动或BIOS/MBR配置。

提示：Windows 10的1703版本及以后的版本支持使用MBR2GPT.EXE从BIOS/MBR转换为UEFI/GPT。

Linux 内核没有这样的限制，但可能取决于使用的引导加载程序以及/或者 其配置。如果用户希望从同一磁盘启动Windows和Linux，那么应该考虑Windows的这个限制，因为boot loader的安装过程取决于固件类型和磁盘分区配置。建议遵循Windows使用的

方法，即选择UEFI/GPT启动或BIOS/MBR启动。请参见 https://support.microsoft.com/kb/2581408获取更多信息。

UEFI 和 BIOS 引导加载程序的限制

大多数某类（UEFI 或 BIOS 之一）固件上的 Linux 引导加载程序，不能启动或链式加载另一种固件上的引导加载程序。例如，如果 Arch 以 UEFI 模式安装在一块硬盘下、Windows 以 BIOS 模式安装在另一块硬盘下，Arch 的 UEFI 引导加载程序不能链式加载 Windows 的 BIOS 引导加载程序。这与分区表类型无关。反之亦然。

有几个例外：

- 装在苹果电脑上的 GRUB。这里，UEFI 模式的 GRUB 可以用`appleloader`命令启动 BIOS 模式安装的操作系统（在非苹果电脑上无效）。
- rEFInd 理论上也支持从 UEFI 固件启动 BIOS 操作系统，但作者称在非苹果电脑上不总是正常工作。
- 一块硬盘上的 BIOS/GPT Arch 所使用的引导加载程序可以启动另一块硬盘上的 BIOS/MBR Windows，如果该引导加载程序本身支持链式加载其他硬盘的系统。

注意：如果要在同一硬盘上与Windows双启动，Arch应该遵循与Windows安装相同的固件引导模式和分区组合。

Windows 安装程序会创建一个100 MiB的EFI系统分区（对于先进格式化 4K本地驱动器，它会创建一个300 MiB的ESP），因此多内核使用受到限制。解决方法包括：

- 将ESP挂载到`/efi`，并使用具有文件系统驱动程序的boot loader，它能够启动位于其他分区的内核。
- 减小 Windows 分区的大小，并替换现有的 ESP 为一个新的、更大的分区。
- 备份并删除`esp/EFI/Microsoft/Boot/Fonts/`中不需要的字体 。
- 在`*esp*/EFI/Microsoft/Boot/`中备份和删除不必要的语言目录（例如，仅保留`en-US`）。
- 使用更高但更慢的initramfs 压缩方式，并确保解压缩可加载的内核模块和固件。例如：

```
COMPRESSION="xz"
COMPRESSION_OPTIONS=(-9e)
MODULES_DECOMPRESS="yes"
```

UEFI 安全启动

一切预装的 Windows 8/8.1/10/11 系统都默认使用 UEFI/GPT 方式启动且默认启用 UEFI 安全启动 (Secure Boot)。对于所有 OEM 预装系统 Microsoft 都强制要求这么做。

目前 Arch Linux 的安装媒介尚不支持安全启动。参见 UEFI/安全启动#引导安装介质。

目前建议的做法是，在试图启动 Arch Linux 之前在固件设置中手动禁用 UEFI 安全启动。这样，Windows 8/8.1/10/11 **应该**也还能正常启动。然而， Microsoft 已经明确禁止在预装的Windows 8/8.1及以上系统中以远程或编程的方式（从操作系统内部）禁用安全启动，因此，这种做法的唯一问题是需要操作者能够亲自坐在目标电脑前并直接访问该电脑。。

注意：

- 如果使用 BitLocker 加密 Windows 且将密钥保存在 TPM 内以便启动时自动解锁，则一旦禁用安全启动，Windows 将无法启动，并显示 BitLocker 恢复屏幕。虽然如此，该问题并不永久，只要再次启用安全启动即可再次启动 Windows。亦可在禁用加密后安装 Arch Linux。
- 如果使用的是 Windows 11，在安装后禁用安全启动不会导致问题，只要TPM正常工作即可。

警告：

**本文或本章节的事实准确性存在争议。**

原因：

安全启动的更改不应影响 BitLocker：问题可能是自签名 Windows 引导加载程序并从引导加载程序链式加载它：只要 Windows 引导加载程序保持使用 MS 密钥签名，并且 Microsoft 证书已注册，它应该没问题。 问题是禁用它并启动 Windows，还是禁用、重新启用它然后启动 Windows？ 第一个是可以理解的，第二个是需要警告的。

（在 Talk:Arch + Windows 双系统 中讨论）

如果您打算也为 Linux 使用安全启动，您可能需要对安全启动设置进行更改。这些更改会阻止在没有恢复密钥的情况下解锁 BitLocker 磁盘，导致**永久数据丢失**。在继续之前，请检查是否是这种情况，如果尚未完成，请备份您的 BitLocker 恢复密钥。如果 Windows 是由供应商预装的，这一点尤其重要。

快速启动和休眠

两个系统都可以休眠，既可以休眠 Windows 而启动 Linux （或者另一个 OS），又可以休眠 Linux 而启动 Windows，或者同时休眠两个系统。

警告：如果在 Windows 休眠时使用双启动进入另一 OS 并在二者皆可读/写的（如 NTFS）且已经被 Windows 挂载的某个文件系统上做出更改，有可能发生数据损失。（参见。）同样，如果 Linux 休眠而启动另一个 OS，也可能出现数据损失，如此之类。在选择关闭 Windows 时，Windows 也有可能实际上在休眠而不是在关机。参见#Windows 设置。

同样的原因，如果 Windows 和 Linux 使用同一个 EFI 系统分区，如果休眠（或在启用快速启动时关机）Windows 并启动 Linux，或者休眠 Linux 并启动 Windows， EFI 系统分区可能遭到破坏。

ntfs-3g包 加入了一种保护机制以阻止以读/写方式挂载已休眠的 NTFS 文件系统，但是 Linux 内核中的 NTFS 驱动并没有该机制。

一般情况下，Windows 是无法读取像目前广泛用于 Linux 的 ext4 之类的文件系统的。除非你为这些 Windows 所不支持的文件系统安装 Windows 驱动，否则这些文件系统可以忽略。

Windows 设置

快速启动在 Windows 8 引入，其在关机时会休眠而非真正意义上关闭 Windows 以加快启动速度。

在之后的章节中会谈到关于快速启动和休眠的 Windows 设置项。

- 关闭快速启动并关闭休眠
- 关闭快速启动并启用休眠
- 启用快速启动并启用休眠

禁用快速启动的方法：Windows 8，Windows 10, Windows 11。无论如何，如果你要禁用某项设置，请确保在禁用该设置之后关闭 Windows，再安装 Linux；请注意不应只是重启。

关闭快速启动并关闭休眠

这是最安全的方法，并且如果对该问题不确定，那么推荐使用该方法。因为该方法在重启进入另一 OS 时对用户感知要求并不高。这时可以让 Windows 和 Linux 共享 EFI 系统分区。

可在具有管理员权限的 Windows 命令行 shell 中用下面的指令代替鼠标操作：

```
> powercfg /H off
```

关闭快速启动并启用休眠

该方法在重启进入另一 OS 时要求用户感知。如果你想在 Windows 休眠时启动 Linux（这种情况很常见）那么：

- 必须为 Windows 和 Linux 使用不同的 EFI 系统分区 (ESP)，并确保 Windows 在休眠前没有挂载 Linux 的 ESP。由于**每个硬盘只能有一个 ESP**，Linux 的 ESP 所处的盘必须与 Windows 的不同。即使在这种情况下， 如果你将Linux使

用的ESP放在与Linux根分区不同的硬盘上，Windows 和 Linux 仍然可以安装在同一硬盘的不同分区中。

- 不能在 Linux 下以 读/写方式 挂载任何在 Windows 休眠时已经由 Windows 挂载的任何文件系统。必须非常小心对待此事，对此还应考虑Automount行为。
- 如果完全关闭 Windows 而非将其休眠，那么这些文件系统可以读/写挂载。

注意：在 Windows 中将某盘挂载为可移动磁盘并在休眠前将其弹出可以为该盘避免此问题。

启用快速启动并启用休眠

此情况下，应该考虑与“关闭快速启动并启用休眠”时同样的情况。但是由于此时无法将 Windows 完全关闭，而只能将其休眠，在 Windows 休眠时由 Windows 挂载的分区将永远不能读/写挂载。

注意：Windows Update 更新可能导致快速启动被重新启用。见\[失效链接 2024-07-30 ⓘ\]

Windows 文件名限制

Windows 限制文件路径小于260 个字符。

由于一些可以追溯到 DOS 的原因，Windows 同时禁止文件名中出现某些字符.aspx#naming_conventions)：

- < (小于号)
- > (大于号)
- : (半角冒号)
- " (半角双引号)
- / (反斜杠)
- \ (斜杠)
- | (竖杠)
- ? (半角问号)
- * (星号)

这属于 Windows 而非 NTFS 本身的限制——任何使用 NTFS 分区的其它 OS 都不受影响。Windows 将无法检测到这些文件，并且运行 `chkdsk` 很可能会使它们被删除，造成数据丢失的风险。

如果使用`windows_names`选项，NTFS-3G将会把和 Windows 一样的限制应用于新文件的名字上：ntfs-3g(8) § Windows_Filename_Compatibility。（见fstab）

安装

设置 Linux/Windows 双启动的推荐方式是，先安装 Windows， 但是只为其分区划一部分硬盘空间。在 Windows 设置完毕后，启动 Linux 的安装环境以便为 Linux 创建分区并调整分区大小，但 Windows 分区保持原样。Windows 的安装已经创建了一个能够用于你的 Linux 启动引导程序的 EFI 系统分区。如果您从头开始安装 Windows，请注意 Windows 安装程序创建的 EFI 系统分区对于大多数用例来说太小。请参见 #Windows 安装程序创建的 EFI 系统分区太小。

先 Windows 后 Linux

BIOS 系统

使用 Linux 启动引导程序

你可以使用任何具有多启动支持的 BIOS 启动引导程序。

使用 Windows Vista/7/8/8.1 启动引导程序

本章节将讲述如何在某一分区而非 MBR 上安装一个 Linux 启动引导程序、将该程序复制到某个可被 Windows 引导程序读取的分区，以及用 Windows 引导程序启动该 Linux 引导程序。

注意：某些文档声明能被 Windows 引导器读取的分区必须是主分区，但是扩展分区的使用也有成功记录。

- 在安装引导程序时，请将其安装在你的`/boot`分区而非 MBR 上。如果使用 GRUB，请见GRUB/技巧和窍门#安装到分区上或者无分区磁盘上；如果是 Syslinux，见Syslinux#Manual install处的附注；如果是 LILO，见LILO#Install to partition or partitionless disk。
- 建立一份 VBR 的拷贝：

```
dd if=/dev/disk of=/path/to/linux.bin bs=512 count=1
```

其中`/dev/*disk*`是你安装启动引导程序的分区的路径，`/path/to/`是你想要 Windows 启动引导程序可以读取该拷贝的且已经挂载的文件系统。

- 现在你的 linux.bin 应该可以在 Windows 下访问了。以管理员权限运行 **cmd**（进入*开始菜单-全部应用程序-(Windows)附件*。右键*命令提示符*并选择*以管理员权限运行*）并执行以下命令：

```
bcdedit /create /d "Linux" /application BOOTSECTOR
```

BCDEdit 将会返回该项的UUID。该 UUID 在下面的步骤中将会用`*UUID*`代替。

```
bcdedit /set UUID device partition=X: (X:为linux.bin所处分区的分卷号)
bcdedit /set UUID path \path\to\linux.bin
bcdedit /displayorder uuid /addlast
bcdedit /timeout 30
```

重启之后，Windows 和 Linux 应该都显示在 Windows 的启动引导程序当中。

注意：在某些电脑上，Windows 引导程序用另一电源按钮启动另一 OS（例如：Dell Precision M4500）。

参见 https://www.iceflatline.com/2009/09/how-to-dual-boot-windows-7-and-linux-using-bcdedit/

UEFI 系统

如果你已经安装 Windows，它已经在一张GPT格式的硬盘上创建如下分区：

- 一个WinRE（**W**indows **R**ecovery **E**nvironment）分区，一般大小为 499 MiB，包含启动 Windows 所需的文件，等同于 Linux 下的`/boot`分区;
- 一个具有 FAT32 文件系统的 EFI 系统分区;
- 一个MSR 分区(Microsoft Reserved Partition);
- 一个格式化为 NTFS 的 Microsoft Basic Data 分区，对应`C:`;
- 可能是系统恢复/备份/非主要数据分区（对应`D:`等）;

在 Windows 下，使用磁盘管理工具查看各分区的标签以及报告的类型。这样能使人清楚哪些分区属于必要的 Windows 分区，以及可能随时改变用途的其它分区。Windows 磁盘管理同样可以缩小 Windows (NTFS) 分区以便为 Linux 腾出空间。

警告：上述分区中前 4 种为关键系统分区，不能删除。

现在可以继续按需进行分区。引导加载程序需要支持链式加载其他EFI应用程序，以实现Windows和Linux的双重启动。

不应创建额外的EFI系统分区，因为这可能会导致Windows无法启动。直接挂载现有分区即可。

注意：只有 Linux 被安装在另一块硬盘并在磁盘上创建了新的 EFI 系统分区时才会出现此问题。

启动引导程序须支持链式载入其它 EFI 应用程序以进行 Windows-Linux 双启动。

提示：\ *rEFInd和systemd-boot都会自动检测*Windows Boot Manager*(`\EFI\Microsoft\Boot\bootmgfw.efi`)并将其显示于自己的启动菜单中。如果使用 GRUB，见GRUB#Windows installed in UEFI/GPT mode（手动添加启动项）或GRUB#Detecting other operating systems（自动生成配置文件）。

- 为了节省EFI系统分区的空间，特别是对于多个内核，请增加initramfs的压缩率)。

预装较新版本的 Windows 的电脑通常启用安全启动。需要进行额外操作以禁用安全启动或使你的安装媒体支持安全启动（详见上文）。

先 Linux 后 Windows

即使设置 Windows-Linux 双启动时一般推荐先安装 Windows，换成 Linux 在前也可以实现双启动。与 Windows 在前的方式相比，这种方式需要在启动 Windows 安装之前创建并搁置一个（比如 40GB 或者更高的）Windows 分区，或者预留一些空闲的未分区空间，或者从 Linux 安装中创建/调整 Windows 分区。

UEFI 固件

Windows 将会使用已有的EFI 系统分区。 假设安全启动已禁用。

1. 启动 Windows 安装。注意让安装器使用你希望其使用的分区，然后让它如同没有安装 Linux 一样工作。
2. 按照#快速启动和休眠的说明操作。
3. 恢复 Linux 启动（可参阅#安装 Windows 后 Linux 无法启动）。前面已经提到过，某些 Linux 引导启动程序会自动检测*Windows Boot Manager*。即使较新的 Windows 安装都有一个能用于启动到 Linux 的“高级启动”选项，仍然建议使用 Arch 安装媒体或 LiveCD 之类的方式启动 Linux。

Windows 10 和 GRUB

这里假设使用GRUB作为启动引导程序（对于其它引导程序大致思路也相似）且 Windows 10 将会安装到一个已经有 EFI 系统分区的 GPT 块设备上。详见这篇 Microsoft 文档的“系统分区”章节。

用`gdisk`在块设备上创建如下分区：（更精确的分区大小见）

<table class="wikitable"><caption>标题文本</caption><tbody><tr><th>最小尺寸</th><th>分区类型代码</th><th>分区类型名</th><th>文件系统</th></tr><tr><td>16 MB</td><td>0C01</td><td>Microsoft reserved</td><td>N/A</td></tr><tr><td>约 40 GB或更大</td><td>0700</td><td>Microsoft basic data</td><td>NTFS</td></tr><tr><td>300 MB</td><td>2700</td><td>Windows RE</td><td>NTFS</td></tr></tbody></table>

用来自ntfs-3g包的`mkntfs`工具在新创建的 Microsoft basic data 和 Windows RE 分区上创建新的 NTFS 文件系统。

将系统重启到 Windows 10 安装媒体。选择“自定义”安装选项并将 Windows 安装到先前创建的 Microsoft basic data 分区上。这样应该也会将 Microsoft EFI 文件安装到 EFI 分区中。

安装完毕后（不需要 OOBE 和登录），重启进入 Linux 并生成一个 GRUB 配置文件以便 Windows 的启动管理器在下次启动时启动在 GRUB 菜单中。

故障排除

无法创建新分区或无法找到已有分区

见 #Windows 在 UEFI 和 BIOS 上的限制。

安装 Windows 后 Linux 无法启动

见 Unified Extensible Firmware Interface#Windows changes boot order

恢复 Windows 启动记录

按照惯例（且为了更简易的安装过程），Windows 通常安装在第一分区并将其分区表和其启动引导程序的引用安装到该分区的第一扇区。如果不慎把一个 GRUB 之类的启动引导器安装到 Windows 分区上，或者用其它什么方式弄坏了 Windows 的启动记录，就需要使用工具对其进行修复。Microsoft 在他们的恢复光盘（有些时候，安装光盘）上提供一个叫做`FIXBOOT`的启动扇区修复工具和一个名为`FIXMBR`的 MBR 修复工具。这种方式可以分别更正第一分区的启动扇区对启动引导程序的引用以及 MBR 上对第一分区的引用。在此之后需要按原先计划将 GRUB 重新安装到 MBR 上（GRUB 可以配置为链式加载 Windows 启动引导程序）。

如果你希望回退到 Windows，你可以用`FIXBOOT`命令链接 MBR 和第一分区启动扇区以恢复 Windows 的自动启动。

有一个叫做ms-sysAUR的工具可以用于安装 MBR。但是，该工具目前只能写新 MBR 或 FAT 文件系统的启动扇区（又称为启动记录；等效于`FIXBOOT`）。许多 LiveCD 默认都没有该工具，所以其需要提前安装，或者可以考虑包含该工具的急救 CD，如 Parted Magic。

首先，重新写一遍分区信息（表）：

```
# ms-sys --partition /dev/sda1
```

接下来，写 Windows 2000/XP/Server 2003 MBR：

```
# ms-sys --mbr /dev/sda  #查阅 options 列表以获取不同版本的信息
```

最后，写启动扇区：

```
# ms-sys -(1-6) #查阅 options 列表以查找正确的 FAT 记录类型
```

`ms-sys`同样可以写 Windows 98/ME 及 Windows Vista/7 MBR。详见`ms-sys -h`。

恢复意外删除的EFI系统分区

如果您的磁盘采用GPT分区，并且擦除了（例如使用 `mkfs.fat -F32 /dev/sd*x*` 擦除）EFI系统分区，您会发现Windows引导管理器要么从启动选项中消失，要么选择它会将您送回UEFI界面。

欲解决这个问题，请使用Windows安装介质启动，按下 `Shift+F10` 打开控制台（或点击 *NEXT* > *Repair Computer* > *Troubleshoot...* > *Advanced* > *Command Prompt*），然后启动diskpart实用程序：

```
X:\Sources> diskpart
DISKPART> list disk
```

选择适当的硬盘：

```
DISKPART> select disk number
```

确保存在一个系统类型的分区（EFI系统分区）

```
DISKPART> list partition
```

选择该分区，

```
DISKPART> select partition number
```

并为其分配一个临时驱动器号：

```
DISKPART> assign letter=G:
DiskPart successfully assigned the drive letter or mount point.
```

为确保正确分配驱动器号，请检查一下：

```
DISKPART> list vol
Volume ###  Ltr  Label        Fs     Type        Size     Status     Info
 ----------  ---  -----------  -----  ----------  -------  ---------  --------
 Volume 0     E                       DVD-ROM         0 B  No Media
 Volume 1     C                NTFS   Partition    195 GB  Healthy    Boot
 Volume 2         WINRE        NTFS   Partition    400 MB  Healthy    Hidden
 Volume 3     G                FAT32  Partition    499 MB  Healthy    System
```

关闭diskpart

```
DISKPART> exit
```

导航到 `C:\`（或者你的系统驱动器卷标）：

```
 X:\Sources> cd /d C:\
```

接下来是"magic"命令，重新创建BCD存储（使用 `/s` 作为挂载点， `/f` 作为固件类型，可选择添加 `/v` 以获取详细信息）：

```
C:\> bcdboot C:\Windows /s G: /f UEFI
```

提示：如果一分钟后卡住了，点击 `Ctrl+c`。有时会发生这种情况，但你会收到一个类似 `boot files successfully created` 的消息，而且它会正常工作。

现在，您应该已经将 *Windows引导管理器（Windows Boot Manager）*设置为启动选项，从而可以访问Windows。只需确保再也不要格式化您的EFI系统分区！

注意：删除分配给EFI系统分区的G盘符，以防止它在*我的电脑*中显示出来。

参见 , 和 .

Windows 安装程序留下的 EFI 系统分区太小

默认情况下，Windows 安装程序会创建一个 100 MiB 的 EFI 系统分区（在Advanced Format盘上为 300 MiB）。一般来说，这种容量并不足以放下所需要的所有文件。您可以替换现有的 EFI 系统分区为一个新的、更大的分区。

不过，如果你从头开始安装Windows，就可以在安装过程中指定EFI系统分区的大小：

1. 选择您的安装目标，并确保它没有分区。
2. 点击*"New"*按钮，然后点击*"Apply"*按钮。Windows安装程序将生成预期的分区（将几乎所有内容分配给其主分区），并将仅分配100MB给EFI。
3. 用安装程序的图形界面删除 `System` 、 `MSR` 和 `Primary` 分区。保留 `Recovery` 分区（如果存在）不变。
4. 按下 `Shift+F10` 键打开命令提示符。
5. 输入 `diskpart.exe` 并按下 `Enter` 打开磁盘分区工具。
6. 输入 `list disk` 并按下 `Enter` 列出您的磁盘。找到您打算修改的磁盘并记录其磁盘编号。
7. 输入 `select disk *disk_number*` 和磁盘号以进行修改。
8. 输入 `create partition efi size=*size*` 和所需的ESP大小（以 Mebibytes（MiB）为单位），然后按 `Enter` 。请参阅EFI 系统分区#创建分区获取EFI的推荐大小。
9. 输入 `format quick fs=fat32 label=System` 并按 `Enter` 键以格式化 ESP
10. 输入 `exit` 并按 `Enter` 键退出磁盘分区工具，然后再次输入 `exit` 和 `Enter` 。

一旦安装了Windows，你就可以在Windows中调整主分区的大小，然后重新启动并进行常规的Arch安装，填充刚刚创建的空间。

或者，可以将 Windows 安装到盘上之前就用 Arch 安装媒体创建一个自定大小（且较大）的EFI 系统分区。Windows 安装程序将会使用你自己创建的 EFI 分区，而非再为自己创建一个。

无法在BIOS系统上安装Windows累积更新

在BIOS系统上，Windows累积更新可能会失败，并显示错误信息：无法完成更新。正在撤销更改。请勿关闭计算机。在这种情况下，在Windows中，您需要将Windows分区设置为活动分区。

```
C:\> diskpart
DISKPART> list disk
DISKPART> select disk number
DISKPART> list partition
DISKPART> select partition number
DISKPART> active
DISKPART> exit
```

成功安装Windows更新后，再使用上述命令将Linux分区标记为活动状态。

时间标准

- 建议：将 Arch Linux 和 Windows 皆设置为UTC时间（见系统时间#Windows 系统使用 UTC）。某些 Windows 版本在设置为自动同步网络时间时会将 RTC（硬件时钟） 回退到本地时间（*localtime*）。该问题在 Windows 10 中似乎已修复。
- 不建议：将Arch Linux的硬件时钟模式设为*localtime*并禁用任何时钟同步服务。这将让Windows负责硬件时钟的校正，并且你需要在一年中至少启动Windows两次(分别在春季与秋季)以正确应用夏令时。所以请不要忘记启动 Windows 而在论坛问为什么时钟会快/慢一个小时。

蓝牙配对

对于将蓝牙设备同时与 Linux 和 Windows 配对，两个系统均使用相同的 MAC 地址，但是配对过程中会使用不同的 *link keys*，导致在蓝牙设备与其中一个系统配对后就无法连接到另一个系统。如果想要使蓝牙设备不用重新配对就在两个系统下都可用，见 Bluetooth#Dual boot pairing。

参见

- Booting Windows from a desktop shortcut
- One-time boot into Windows partition from desktop shortcut
- Windows 7/8/8.1/10 ISO to Flash Drive burning utility for Linux (MBR/GPT, BIOS/UEFI, FAT32/NTFS)

2.10 UEFI

难度 高级 | **篇幅** 39.0 KB | **所属** 启动过程

本节将讲到

- UEFI 固件位数
- UEFI 变量
- UEFI Shell

- UEFI 驱动
- UEFI 可启动介质

统一可扩展固件接口（**Unified Extensible Firmware Interface，简称 UEFI**）是操作系统和固件之间的接口。UEFI 提供了启动操作系统或运行预启动程序的标准环境。

UEFI 有别于传统 BIOS 系统所使用的 MBR 引导代码。二者的区别和使用 UEFI 启动的过程见 Arch 的启动流程。要配置 UEFI 引导加载程序，请参见 Arch 的启动流程#引导加载程序。

注意：

- 主板制造商的早期 UEFI 实现可能相比其 BIOS 版本有着更多的错误。如果遇到无法解决的问题，请考虑对这类系统使用传统 BIOS 引导。
- 苹果的 UEFI 实现不符标准。除非特别支持，本文内容皆为通用指南，部分可能不适用或与 Mac 上的操作有所不同。

UEFI 固件位数

UEFI 下每一个程序，无论它是某个 OS 引导器还是某个内存测试或数据恢复的工具，都要兼容于 EFI 固件位数或体系结构。

无论是系统引导器还是工具（例如内存测试或数据恢复工具），UEFI 下的所有程序都是 EFI 应用，都需要兼容对应的 UEFI 固件位数或架构。

当前大部分 x86_64 架构系统（包括较新的苹果 Mac）都使用了 x64（64 位）UEFI 固件。目前已知还在使用 IA32（32 位）UEFI 固件的有 2008 年前生产的苹果 Mac、Intel Atom SoC 设备（截至 2013 年 11 月 2 号）和一些使用 Intel EFI 1.10 固件的 Intel 服务器主板。

与 x86_64 Linux 和 Windows 系统不同，x64 UEFI 固件不兼容 32 位 EFI 程序，所以必须针对特定固件处理器的位数和架构编译 EFI 应用。

注意：IA32 UEFI 平台上的系统需要使用支持混合引导模式的引导加载程序。

检查固件位数

可以在已启动的系统上检查固件位数。

Linux 系统

在运行 Linux 4.0 及以上版本内核的发行版中，可以通过 sysfs 接口查看 UEFI 系统位数。试着运行：

```
$ cat /sys/firmware/efi/fw_platform_size
```

如果返回 `64` 则代表 64 位（x64） UEFI 系统，如果返回 `32` 则代表 32 位（IA32）UEFI 系统。如果文件不存在，那么代表并没有以 UEFI 模式启动。

macOS 系统

2008 年以前的 Mac 大都使用 IA32 EFI 固件，2008 年及以后大都使用 x64 EFI。有能力运行 Mac OS X Snow Leopard 64 位内核的 Mac 使用的都是 x64 EFI 1.x 版的固件。

在 Mac OS X 的终端输入以下命令可以找出该 Mac 的 EFI 固件架构：

```
$ ioreg -l -p IODeviceTree | grep firmware-abi
```

如果命令返回 `EFI32`。则对应的是 IA32（32 位）EFI 固件，返回 `EFI64` 对应的则是 x64 EFI 固件。由于苹果的 EFI 实现不完全符合 UEFI 2.x 标准，因此大部分 Mac 没有 UEFI 2.x 固件。

Windows 系统

64 位的 Windows 系统不支持在 32 位 UEFI 上启动。所以如果你在 UEFI 模式下启动了 32 位 Windows，那么你使用的是 32 位 UEFI。

可以通过运行 `msinfo32.exe` 来查看固件位数。请看*系统摘要*条目下“系统类型”和“BIOS 模式”对应的值。

如果是 64 位 UEFI 上的 64 位 Windows，则会显示 系统类型：基于`x64`的电脑 和 `BIOS 模式：UEFI`；如果是 32 位 UEFI 上的 32 位 Windows，则会显示 系统类型：基于`x86`的电脑 和 `BIOS 模式：UEFI`。如果“BIOS 模式”不是 `UEFI`，那么 Windows 并没有用 UEFI 模式启动。

UEFI 变量

UEFI 定义了一些变量，操作系统可以通过它们与固件进行交互。UEFI 引导变量只在早期系统启动时由引导加载程序和操作系统使用。UEFI 运行时变量允许操作系统管理固件的某些设置，例如 UEFI 引导管理器或 UEFI 安全启动协议的密钥等。你可通过下面的命令来获得变量列表：

```
$ efivar -l
```

Linux 内核中的 UEFI 变量支持

从 3.8 版本开始，Linux 内核通过 **efivarfs**（**EFI VAR**iable **F**ile**S**ystem，EFI 变量文件系统）接口（`CONFIG_EFIVAR_FS`），使用 `efivarfs` 内核模块将 UEFI 变量数据挂载到了 `/sys/firmware/efi/efivars` 下，并暴露到用户空间。它不限制单个变量的大小，并支持 UEFI 安全启动变量。

UEFI 变量正常工作的需求

1. 内核需通过 EFI boot stub（可选使用引导管理器）或 UEFI 引导加载程序启动，而不是 BIOS/CSM（Compatibility Support Module，兼容性支持模块）或者苹果同为 CSM 的 Boot Camp 启动。
2. 内核需带有 EFI 运行时服务支持（`CONFIG_EFI=y`，可运行 `zgrep CONFIG_EFI /proc/config.gz` 来检查是否满足）。
3. **不应**通过内核命令行参数禁用 EFI 运行时服务，即**不应使用** `noefi` 内核参数。
4. `efivarfs` 文件系统应被挂载在 `/sys/firmware/efi/efivars`，否则参考下文 #挂载 efivarfs 部分。
5. `efivar` 在使用 `-l`/`--list` 选项列出 UEFI 变量时不应出现任何报错。

如果 EFI 变量支持在满足以上条件后仍有问题，尝试以下解决方案：

1. 如果在列出 UEFI 变量时出现以下报错：`efivar: error listing variables: Function not implemented`，且系统启动到了实时内核，尝试添加 `efi=runtime` 到内核参数并重启（这类内核默认会禁用 efivarfs）。
2. 更多排障信息请参考 #Userspace tools are unable to modify UEFI variable data。

挂载 efivarfs

如果 `efivarfs` 启动时并没有被 systemd 自动挂载
在 `/sys/firmware/efi/efivars`，你需要通过手动挂载来把 UEFI 变量暴露给类似 *efibootmgr* 的用户空间工具：

```
# mount -t efivarfs efivarfs /sys/firmware/efi/efivars
```

注意：如果适用，以上命令需要在 chroot **之外（之前）*和*之内**都执行一次。

更多内核文档请参考 efivarfs.html。

用户空间工具

有多个工具可以读取/修改 UEFI 变量：

- **efivar** — 操作 UEFI 变量的库和工具（被 efibootmgr 用到）

: https://github.com/rhboot/efivar || efivar包

- **efibootmgr** — 操作 UEFI 固件启动管理器设置的工具

: https://github.com/rhboot/efibootmgr || efibootmgr包

1. **uefivars** — 转储 UEFI 变量和 PCI 相关信息（内部使用 efibootmgr 源码）

: https://github.com/fpmurphy/Various/tree/master/uefivars-2.0 || uefivars-gitAUR

1. **efitools** — 控制 UEFI 安全启动平台的工具

: https://git.kernel.org/pub/scm/linux/kernel/git/jejb/efitools.git || efitools包

1. **Ubuntu 固件测试套件** — 对 Intel/AMD 主机固件进行检查的测试套件

: https://wiki.ubuntu.com/FirmwareTestSuite/ || fwts-gitAUR

- **QEFI 启动项管理器** — 这是一个基于 Qt 的图形界面应用，用于管理 EFI 启动项和 EFI 系统分区

: https://github.com/Inokinoki/QEFIEntryManager || qefientrymanagerAUR

efibootmgr

首先需要安装 efibootmgr包。

注意：

- 如果 *efibootmgr* 完全无效，你可以重启进入 #UEFI Shell 使用 `bcfg` 命令来给引导器创建一个启动条目。
- 如果你不能使用 `efibootmgr`，某些 UEFI 固件允许用户用内建的启动时界面管理启动条目。例如，有些固件带有“Add New Boot Option”选项，能让你选择本地 EFI 系统分区并手动输入 EFI 应用位置（例如 `\EFI\refind\refind_x64.efi`）。
- 下面的命令用 rEFInd 引导器作为例子。

要通过 *efibootmgr* 添加新的启动参数，需要确认：

1. 包含 EFI 系统分区（ESP）的磁盘编号，例如：`/dev/sda`，`/dev/nvme0n1`
2. ESP 分区在其硬盘上的分区编号，即 `/dev/sda*Y*` 或 `/dev/nvme0n1p*Y*` 中的 `*Y*`。
3. EFI 程序相对 ESP 根目录的路径

假设要为 `/efi/EFI/refind/refind_x64.efi` 添加启动项，其中 `/efi` 是 ESP 的挂载目录，需要执行：

```
$ findmnt /efi
TARGET SOURCE    FSTYPE OPTIONS
/efi   /dev/sda1 vfat   rw,flush,tz=UTC
```

在以上示例中，findmnt(8) 指出 `/dev/sda` 上 ESP 分区的编号为 1，EFI 程序相对于 ESP 根的路径是 `/EFI/refind/refind_x64.efi`。因此，需要使用以下命令创建启动项：

```
# efibootmgr --create --disk /dev/sda --part 1 --loader '\EFI\refind\refind_x64.efi' --label 'rEFInd Boot Manager' --unicode
```

查看所有启动项和启动顺序：

```
# efibootmgr --unicode
```

如需配置启动顺序，使用：

```
# efibootmgr --bootorder XXXX,XXXX --unicode
```

其中 *XXXX* 是上述 *efibootmgr* 命令输出显示的数字。

如需删除不需要的启动项，使用：

```
# efibootmgr --delete-bootnum --bootnum XXXX --unicode
```

更多信息请参考 efibootmgr(8) 或 efibootmgr README。

注意：UEFI 标准使用反斜杠 `\` 作为路径分隔符，不过 *efibootmgr* 会自动把 UNIX 的 `/` 路径分隔符转换为 `\`。

禁止访问 UEFI 变量

UEFI 变量访问有可能会导致超出系统范围的破坏。有些 UEFI 漏洞（例如 LogoFAIL）会让黑客获得系统的完整权限，在有些不完善的 UEFI 实现中甚至能导致硬件故障。

因此，鉴于日常使用系统时不会访问 UEFI 变量，可以选择将其禁用，以防止潜在安全漏洞和误操作产生破坏。

可选方法有：

- 使用 fstab 以只读模式挂载 `efivars`，例如：

```
efivarfs /sys/firmware/efi/efivars efivarfs ro,nosuid,nodev,noexec 0 0
```

- 使用 `noefi` 内核参数完全禁用系统对 UEFI 的访问

注意：禁用后将无法使用 UEFI 用户空间工具，因此需要在进行前完成所有必要配置。另外，UEFI 相关命令也将无法使用（例如 `systemctl reboot --firmware-setup`）。

UEFI Shell

UEFI Shell 是一个用于固件的命令行终端，可用于启动如 UEFI 引导器等 EFI 程序。除此之外，它还可用于采集固件和系统的各种信息，例如内存映射（memmap），修改启动管理器变量（bcfg），运行分区程序（diskpart），加载 UEFI 驱动，编辑文本文件（edit），十六进制编辑等等。

获取 UEFI Shell

可以从 TianoCore EDK2 项目获取到以 BSD 许可证发布的 UEFI Shell：

你可从 Intel 的 Tianocore UDK/EDK2 Sourceforge.net 工程下载以 BSD 许可证发布的 UEFI Shell.

- Shell v2：
- 在 Arch 安装媒介上的位置为 `/shellx64.efi`，在构建 ISO 时从 `/usr/share/edk2-shell/x64/Shell_Full.efi` 复制而来。
- edk2-shell包 - 从 TianoCore EDK2 最新版本编译而来，为 x64（64 位）UEFI 提供了 x64 Shell，以及为 IA32（32 位）UEFI 提供了 IA32 Shell。
- uefi-shell-gitAUR - 从 TianoCore EDK2 最新源码编译而来，为 x64（64 位）UEFI 提供了 x64 Shell，以及为 IA32（32 位）UEFI 提供了 IA32 Shell。
- Shell v1：
- 源自 TianoCore 的预编译 UEFI Shell v1 二进制文件（从 2014 年 1 月 10 号开始上游就不再更新）。
- 打过补丁的 shells：
- 源自 Clover EFI 启动引导器的预编译 UEFI Shell v2 二进制文件\[失效链接 2023-07-30 ⓘ\]，修改了 bcfg 以与 UEFI pre-2.3 固件配合使用。
- 源自 OpenCore 启动引导器的预编译 UEFI Shell v2 二进制文件，与大量固件兼容，可从发布包中的 `EFI/OC/Tools/OpenShell.efi` 位置获取。

Shell v2 在 UEFI 2.3+ 系统上表现最好，在这些系统上建议使用 v2 版本。Shell v1 应在所有 UEFI 系统上都可用，与固件遵循的 UEFI 标准无关。更多信息请参考 ShellPkg 和 EDK2 邮件清单 - Inclusion of UEFI shell in Linux distro iso。

启动 UEFI Shell

从 Sandy Bridge 开始，少数华硕和其它使用 AMI Aptio x64 UEFI 固件的主板提供了一个叫做 *Launch EFI Shell from filesystem device* 的选项。对于这些主板，需要将 x64 UEFI Shell 复制到 EFI 系统分区的根目录下，并重命名为 `shellx64.efi`。

提示：

- Arch Linux 安装媒介的根目录包含 `shellx64.efi`。
- 如果 EFI 系统分区的根目录下存在 `shellx64.efi`，那么 rEFInd 和 systemd-boot 会自动为其添加启动项。

Phoenix SecureCore Tiano UEFI 固件已内嵌 UEFI Shell, 可按 `F6`, `F11` 或 `F12` 键来启动。

注意：如果你用以上方法不能启动 UEFI Shell，可以创建一个 FAT32 格式的 U 盘，并把 EFI 二进制文件复制到 `*/U_盘挂载点*/EFI/BOOT/BOOTx64.EFI`。这个 U 盘将会出现在固件的启动菜单里，启动它就会启动到 UEFI Shell。

重要 UEFI Shell 命令

UEFI Shell 命令通常支持 `-b` 选项，它会在输出的每页末尾暂停。运行 `help -b` 可以列出所有可用命令，这些命令都是内置在 shell 或单独的 EFI 应用中。

详细信息请参考 Intel 脚本编写指南 2008\[失效链接 2023-07-30 ⓘ\] 和 Intel“教程”2011\[失效链接 2023-07-30 ⓘ\]。

bcfg

bcfg 命令用于修改 UEFI NVRAM 条目，它能让用户改变启动条目或驱动器选项，在 UEFI Shell Specification 2.2 PDF 文档的 96 页(Section 5.3) 有详细说明。

注意：

- 仅当 *efibootmgr* 无法创建启动条目时才推荐尝试 *bcfg*。
- UEFI Shell v1 官方二进制文件不支持 `bcfg` 命令，可以参考#获取 UEFI Shell下载一个修改过的 UEFI Shell v2 二进制文件，有可能在 UEFI 2.3 版本前的固件上可用。

转储当前启动条目：

```
Shell> bcfg boot dump -v
```

以为 rEFInd 为例，将其添加为第 4 个（从 0 开始计数）启动项：

```
Shell> bcfg boot add 3 FS0:\EFI\refind\refind_x64.efi "rEFInd Boot Manager"
```

其中 `FS0:` 是 EFI 系统分区映射到的路径，`FS0:\EFI\refind\refind_x64.efi` 是要启动的文件。

如需添加直接启动到系统而非通过引导加载器的条目，参见 EFI boot stub#bcfg.

删除第 4 个启动选项：

```
Shell> bcfg boot rm 3
```

把第 3 个启动选项移动到第 0 个（也就是第 1 个选项，是 UEFI 启动菜单的默认启动选项）：

```
Shell> bcfg boot mv 3 0
```

bcfg 帮助信息：

```
Shell> help bcfg -v -b
```

或:

```
Shell> bcfg -? -v -b
```

map

`map` 显示设备的映射列表，即可用的文件系统（`FS0`）和存储设备（`blk0`）

在执行 `cd` 或 `ls` 等文件系统命令前，你需要输入其名称来进入相应的文件系统。

```
Shell> FS0:
FS0:\> cd EFI/
```

edit

`edit` 命令提供了类似于 nano 界面的基本编辑器，但是功能略少一点。它以 UTF-8 编码，并且兼容 LF 和 CRLF 行尾结束符。

本例中，编辑在固件 EFI 系统分区 (`fs0:` 中 rEFInd 的 `refind.conf`：

```
Shell> edit FS0:\EFI\refind\refind.conf
```

按下 `Ctrl+e` 可显示帮助信息。

UEFI 驱动

UEFI 驱动是一些用于提供额外功能的软件。例如，UEFI shell 通常无法访问 NTFS 格式的分区，而 efifs包 提供了从 EFI shell 访问多种文件系统读取文件的驱动。其中一种用法是先将驱动复制到 UEFI shell 能访问的分区，然后在 UEFI shell 中执行如下命令：

```
Shell> load ntfs_x64.efi
Shell> map -r
```

执行命令后，用户就可以在 UEFI shell 中访问 NTFS 格式的分区。

提示：

- systemd-boot 会自动从 `*esp*/EFI/systemd/drivers/` 加载 UEFI 驱动。
- rEFInd 会自动从其 ESP 下的安装目录中的 `drivers` 和 `drivers_x64` 子目录加载 UEFI 驱动，例如 `*esp*/EFI/refind/drivers_x64/`。可以配置扫描更多其它目录。

UEFI 可启动介质

从光学介质里移除 UEFI 启动支持

注意：

- 本部分内容为从**仅 CD/DVD**（利用 EL Torito 的光学介质启动）介质移除 UEFI 启动支持，而不是 U 盘。
- 为了隐藏USB启动盘上的 UEFI 设备，在将 ISO 文件复制到闪存驱动器后，使用分区编辑器。删除类型为 `EF` 的分区。**切勿**接受转换为 GPT 的提示。

大部分 32 位 EFI Mac 和部分 64 位 EFI Mac 无法从 UEFI（X64）+ BIOS 可启动 CD/DVD 启动。如果希望使用光学介质完成安装，可能首先需要移除 UEFI 启动支持。

提取 ISO 内容，并跳过 UEFI 特定目录：

```
$ mkdir extracted_iso
$ bsdtar -x --exclude=EFI/ --exclude=loader/ -f archlinux-version-x86_64.iso -C extracted_iso
```

确保已经设置了正确的卷标，比如`ARCH_202103`; 可以在原 ISO 中运行 file(1) 获得。然后重新构建 ISO，并使用 libisoburn包 的 xorriso(1) 移除 UEFI 光学介质启动支持。

请确保已设置正确的卷标（例如 `ARCH_202103`），可以对原始 ISO 使用 file(1) 获取：

```
$ xorriso -as mkisofs \
    -iso-level 3 \
    -full-iso9660-filenames \
    -joliet \
    -joliet-long \
    -rational-rock \
    -volid "ARCH_YYYYMM" \
    -appid "Arch Linux Live/Rescue CD" \
    -publisher "Arch Linux <https://archlinux.org>" \
    -preparer "prepared by $USER" \
    -eltorito-boot syslinux/isolinux.bin \
    -eltorito-catalog syslinux/boot.cat \
    -no-emul-boot -boot-load-size 4 -boot-info-table \
    -isohybrid-mbr "extracted_iso/syslinux/isohdpfx.bin" \
    -output archlinux-version-x86_64-noUEFI.iso extracted_iso/
```

把 `archlinux-*version*-x86_64-noUEFI.iso` 烧录进光学介质并照常完成安装。

无原生支持情况下测试 UEFI

虚拟机使用 OVMF

OVMF 是一个为虚拟机添加 UEFI 支持的 TianoCore 项目。OVMF 包含了一个 UEFI 固件示例，和单独一个用于 QEMU 的非易失性变量存储。

可以从 extra 仓库安装 edk2-ovmf包。

建议先将虚拟机的非易失变量存储本地保存一份：

```
$ cp /usr/share/edk2/x64/OVMF_VARS.4m.fd my_OVMF_VARS.4m.fd
```

要使用 OVMF 固件和该变量存储，请在 QEMU 命令中添加以下内容：

```
-drive if=pflash,format=raw,readonly,file=/usr/share/edk2/x64/OVMF_CODE.4m.fd \
-drive if=pflash,format=raw,file=my_OVMF_VARS.4m.fd
```

示例:

```
$ qemu-system-x86_64 -enable-kvm -m 1G -drive
if=pflash,format=raw,readonly,file=/usr/share/edk2/x64/OVMF_CODE.4m.fd -drive
if=pflash,format=raw,file=my_OVMF_VARS.4m.fd …
```

仅 BIOS 的系统使用 DUET

DUET 是一个 TianoCore 项目，使得可以从 BIOS 设备连锁启动到完整 UEFI 环境，与 BIOS 操作系统启动类似。该方法在这里有广泛讨论。预构建的 DUET 可以从代码库\[失效链接 2023-04-07 ⓘ\]下载，使用指南请参考这里\[失效链接 2023-04-07 ⓘ\]。然而，从 2018 年 11 月开始，DUET 代码库已被从 TianoCore git 仓库移除。

也可考虑 Clover，它提供的 DUET 镜像可能包含了一些系统的专用补丁，并且比 gitlab 源更新得更频繁。

疑难问题

困在 Windows 时启动到 Arch Linux

当被困在 Windows 时，要启动到 Arch Linux，可以通过 Windows PowerShell 命令 `shutdown /r /o` 进入*高级启动*，或是通过*设置* > *更新与安全* > *恢复* > *高级启动*，选择*现在重启*。当你进入*高级启动*菜单时，选择*使用设备*（该项中为 UEFI 启动项，包括 U 盘，CD 或硬盘上的操作系统），并选择“Arch Linux”。

无法使用热键进入固件设置

在一些笔记本电脑上，比如联想小新15are 2020，按 F2 或 F12 等按键可能没有任何反应。这种情况有可能通过返厂让OEM修复主板信息来解决，但有时这既不可能也不被希望。不过，还有其他方法可以进入固件设置：

- 使用 systemctl:

```
$ systemctl reboot --firmware-setup
```

这会让电脑重启到固件。

- 使用 GRUB: 按 `c` 进入 GRUB 命令行并输入 `fwsetup`
- 在 Windows 中进入: 使用 Windows 高级重启，参见 #Boot back to Arch Linux when stuck with Windows.

用户空间工具无法修改 UEFI 变量

如果任何用户空间工具无法修改 UEFI 变量，检查是否存在 /sys/firmware/efi/efivars/dump-* 文件。如果这些文件存在，删除它们，重启后再尝试。如果上述步骤无法解决问题，尝试使用内核参数 `efi_no_storage_paranoia` 启动系统，以禁用内核对UEFI变量存储空间的检查，这种检查可能会阻止对 UEFI 变量的写入或修改。

警告：`efi_no_storage_paranoia` 应仅在必要时使用，不应作为正常的启动选项保留。该内核命令行参数的作用是关闭一个保护机制，该机制旨在防止当 NVRAM 过满时导致设备变砖。更多信息请参见 FS#34641。

efibootmgr 无法创建启动条目

某些内核与 efibootmgr 版本组合可能会拒绝创建新的启动项。这可能是 NVRAM 空间不足所致。你可以尝试章节 #Userspace tools are unable to modify UEFI variable data 中提供的解决方案。

你也可以尝试将 efibootmgr downgrade 到版本0.11.0。该版本兼容 Linux 内核 4.0.6。更多信息请参见错误讨论 FS#34641，尤其是 closing comment 。

Windows 改变了启动次序

如果你使用 Arch + Windows 双系统，且主板直接启动到了 Windows，没有出现 EFI 应用选择菜单，那么有多个可能的原因和解决方案。

- 确保 Windows 电源选项中的"快速启动"选项已禁用
- 如果引导管理器没有签名过，请确保固件中的安全启动选项已禁用
- 确保 Windows Boot Manager 不是 UEFI 启动首项，可以使用 efibootmgr 或查看 UEFI 配置工具中的信息。有些主板会默认覆盖掉 Windows 通过 efibootmgr 做出的任何修改（例如 Packard Bell 笔记本）。
- 如果你的主板启动到了默认启动路径（`\EFI\BOOT\BOOTx64.EFI`），那么该文件有可能会被 Windows boot loader 覆写。请尝试使用 efibootmgr 配置正确的启动路径。
- 如果以上步骤无效，还可以直接让 Windows boot loader 运行其它 EFI 应用。进入 Windows 管理员提示符终端，执行 `bcdedit /set "{bootmgr}" path "\EFI\*path*\*to*\*app.efi*"`
- 或者，以根用户权限执行 `efibootmgr -A -b *bootnumber*`（`*bootnumber*` 为 Windows Boot Manager 的启动编号）禁用 Windows Boot Manager，效果可通过不带参数执行 `efibootmgr` 查看。
- 或者可以在 Windows 上配置一个启动脚本，在每次启动 Windows 时配置正确的启动顺序：
- 使用管理员权限打开命令提示符终端，执行 `bcdedit /enum firmware` 并找到目标启动项。
- 复制启动项的标识符（包括大括号），例如 `{31d0d5f4-22ad-11e5-b30b-806e6f6e6963}`
- 创建一个 batch 脚本文件，内容为 `bcdedit /set "{fwbootmgr}" DEFAULT "*{复制来的启动项标识符}*"`
- 打开 *gpedit.msc*，进入 *Local Computer Policy* > *Computer Configuration* > *Windows Settings* > *Scripts (Startup/Shutdown)*，选择 *Startup*
- 在 *Scripts* 选项卡下，点击 *Add* 按钮，选择上面创建的 batch 文件

: : 注意：Windows 10 家庭版默认不包含 gpedit.msc，但有非官方方法可以手动进行安装。

- 或者也可以使用 Windows 任务管理器执行启动脚本：
- 参照步骤 1-3 创建 batch 文件
- 执行 *taskschd.msc*，然后在 *Action* 菜单中选择 *Create Task...*。

- 在 *General* 选项卡中：

: 输入合适的 *Name* 和 *Description*。 : 确保选择的管理员账号属于“管理员”，而不是“标准用户”。 : 选中 "*Run whether user is logged in or not*"。 : 选中 "*Run with highest privileges*"。

1. 在 *Triggers* 选项卡下，在菜单中选择“*At startup*“，然后点击 *OK*。
2. 在 *Actions* 选项卡下，点击 *New...*， *Browse...*，然后选择步骤 1 创建的 batch 文件。
3. 在 *Conditions* 选项卡下，取消选择 *Power* 选项，使得在使用电池时（例如笔记本）也会执行该脚本。
4. 点击 *OK*，在出现提示框时输入步骤 4 选择的用户账户的密码。

USB 介质卡在黑屏界面

可能是 KMS 的问题，从 USB 启动时请尝试禁用 KMS。

固件中不显示引导加载器

有些固件不支持自定义启动项，它们只会从硬编码的启动项启动。

一种常见的解决方法是不依赖NVRAM中的启动项，而是将引导加载程序安装到EFI系统分区上的常见回退路径之一。

以下章节将介绍这些回退路径。

适用于可移除硬盘的默认启动路径

UEFI 标准规定了从可移除媒介启动时的默认 EFI 二进制文件路径：

- x64 UEFI 固件 - `*esp*/EFI/BOOT/BOOTx64.EFI`
- IA32 UEFI 固件 - `*esp*/EFI/BOOT/BOOTIA32.EFI`

虽然标准只针对了可移除硬盘，但大多数固件都支持从任意硬盘启动。

关于如何安装或从引导加载器迁移到默认/后备启动路径的信息，请参考对应的引导加载程序文章。

Microsoft Windows boot loader location

On certain UEFI motherboards like some boards with an Intel Z77 chipset, adding entries with `efibootmgr` or `bcfg` from the UEFI Shell will not work because they do not show up on the boot menu list after being added to NVRAM.

This issue is caused because the motherboards can only load Microsoft Windows. To solve this you have to place the *.efi* file in the location that Windows uses.

Copy the `BOOTx64.EFI` file from the Arch Linux installation medium (`FS0:`) to the Microsoft directory your ESP partition on your hard drive (`FS1:`). Do this by booting into EFI shell and typing:

```
Shell> mkdir FS1:\EFI\Microsoft
Shell> mkdir FS1:\EFI\Microsoft\Boot
Shell> cp FS0:\EFI\BOOT\BOOTx64.EFI FS1:\EFI\Microsoft\Boot\bootmgfw.efi
```

After reboot, any entries added to NVRAM should show up in the boot menu.

UEFI/BIOS is stuck on loading screen

This is a recurring problem with Acer laptops, which occurs if `.efi` files have not been manually authorized. See Laptop/Acer#Firmware Setup became inaccessible after Linux installation.

Boot entries created with efibootmgr fail to show up in UEFI

efibootmgr can fail to detect EDD 3.0 and as a result create unusable boot entries in NVRAM. See efibootmgr issue 86 for the details.

To work around this, when creating boot entries manually, add the `-e 3` option to the *efibootmgr* command. E.g.

```
# efibootmgr --create --disk /dev/sda --part 1 --loader '\EFI\refind\refind_x64.efi' --label 'rEFInd Boot Manager' --unicode -e 3
```

To fix boot loader installers, like `grub-install` and `refind-install`, create a wrapper script `/usr/local/bin/efibootmgr` and make it executable:

```
/usr/local/bin/efibootmgr
#!/bin/sh

exec /usr/bin/efibootmgr -e 3 "$@"
```

UEFI boot entry disappears after removing its referenced drive

Some firmware will remove boot entries referencing drives that are not present during boot. This could be an issue when frequently detaching/attaching drives or when booting from a removable drive.

The solution is to install the boot loader to the default/fallback boot path.

Boot entries are randomly removed

Some motherboards may remove boot entries due to lack of free space in the NVRAM instead of giving an error at creation. To prevent this from occurring, reduce the amount of boot entries being added by minimizing your entry creation process, as well as reducing the amount of automatic drive boot entries by the Compatibility Support Module (CSM) by disabling it from your UEFI settings. See BBS#1608838.

Another reason why boot entries might have been removed is the fact that UEFI specification allows OEMs to do "NVRAM maintenance" during boot process. Those manufacturers do it simply: they just look up for EFI applications in predefined, hardcoded paths on the device. If they fail to find any, they conclude there is no operating system on the device and wipe all boot entries from NVRAM associated with it, because they assume the NVRAM contains some corrupted or outdated data.

If you do not plan to install Windows and still want to load the Linux kernel directly from the firmware, one possible workaround is to create an empty file `*esp*/EFI/BOOT/BOOTx64.EFI`:

```
# mkdir -p esp/EFI/BOOT
# touch esp/EFI/BOOT/BOOTx64.EFI
```

And restore the deleted boot entry. Now after reboot the motherboard will see the "Fake OS" and should not wipe other boot entries from NVRAM. You can change the fake operating system loader with an actual EFI application if you want, of course, as long as you keep the standard fallback name.

Lenovo ThinkPad: boot entries not persistent due to "OS Optimized Defaults"

**本文或本章节可能需要合并到Lenovo。**

附注： Brand-specific issues should be on the dedicated page.（在 Talk:UEFI 中讨论）

On recent Lenovo ThinkPad laptops (e.g. T16 Gen 2 AMD models), users report that custom UEFI boot entries (created with `efibootmgr` or `bootctl`) are automatically deleted at each boot, with only Windows Boot Manager and Lenovo's own entries (PXE, Recovery, Diagnostics) restored.

This is caused by the BIOS option "Restart / OS Optimized Defaults", which resets the UEFI boot variables at each reboot to defaults optimized for Windows.

Solution: Disable "OS Optimized Defaults" in the BIOS/UEFI setup. After doing so, manually created boot entries persist correctly, allowing systemd-boot or other custom boot managers to work as intended.

参见

- UEFI Forum - contains the official UEFI Specifications - GUID Partition Table is part of UEFI Specification
- UEFI boot: how does that actually work, then? - A blog post by AdamW
- Linux Kernel UEFI documentation for x86_64 platforms
- Intel's page on EFI
- Intel Architecture Firmware Resource Center\[失效链接 2023-07-30 ⓘ\]
- Matt Fleming - The Linux EFI Boot Stub
- Matt Fleming - Accessing UEFI Variables from Linux
- Rod Smith - Linux on UEFI: A Quick Installation Guide
- UEFI Boot problems on some newer machines (LKML)
- LPC 2012 Plumbing UEFI into Linux\[失效链接 2021-05-17 ⓘ\]
- LPC 2012 UEFI Tutorial : part 1\[失效链接 2021-05-17 ⓘ\]
- LPC 2012 UEFI Tutorial : part 2\[失效链接 2021-05-17 ⓘ\]
- Intel's TianoCore Project for Open-Source UEFI firmware which includes DuetPkg for direct BIOS based booting and OvmfPkg used in QEMU and Oracle VirtualBox
- FGA: The EFI boot process

- Microsoft's Windows and GPT FAQ
- Convert Windows x64 from BIOS-MBR mode to UEFI-GPT mode without Reinstall
- Create a Linux BIOS+UEFI and Windows x64 BIOS+UEFI bootable USB drive
- Rod Smith - A BIOS to UEFI Transformation
- EFI Shells and Scripting - Intel Documentation
- UEFI Shell - Intel Documentation
- UEFI Shell - bcfg command info
- The bootstrap process on EFI systems

2.11 UEFI/安全启动

难度 高级 | **篇幅** 59.0 KB | **所属** 启动过程

本节将讲到

- 检查安全启动状态
- 引导安装介质
- 实施安全启动
- 保护安全启动
- 提示与技巧

安全启动（Secure Boot）是UEFI标准中的一项安全功能，旨在为pre-boot process添加一层保护；通过维护被授权或禁止的在启动时运行的经过加密签署的二进制文件列表，它有助于使得核心引导组件（引导管理器、内核、initramfs）不被篡改。

因此，它可以被视为对个人使用环境的securing所做努力的延续或补充，减少其他软件安全解决方案（如系统加密）无法轻易覆盖的攻击面，同时完全不同且不依赖于它们。安全启动只是作为当前安全实践的一个组成部分独立存在，具有自己的一套优缺点。

注意：为了更深入地了解Linux中的安全启动。见Rodsbooks' Secure Boot与other online resources。文章重点介绍了如何在 Arch Linux 中设置安全启动。

检查安全启动状态

在启动操作系统之前

此时，必须查看固件设置。如果机器已启动并正在运行，则在大多数情况下必须重新启动。

您可以在引导过程中通过按特殊键来访问固件配置。使用的特殊键取决于固件。它通常是`Esc`、`F2`、`Del`或者`F*n*`键中的一个。有时在启动过程中特殊键的名称会显示一小段时间。主板说明书往往有关于此的记载。如果你想要按下特殊键，请在启动机器

后立即按下该键，甚至是在屏幕实际显示任何内容之前。 进入固件设置后，请注意不要擅自更改任何设置。 通常在每个设置的底部都会有导航说明和设置的简短帮助。设置本身可能由几个页面组成，您必须导航到正确的位置。有一些安全启动设置可能简单地表示为安全启动选项，可以将其设置为打开或关闭 .

在启动操作系统之后

使用systemd检查系统上的安全启动状态的一种简单方法是使用bootctl(1)：

注意：以下命令，并非要求你强制使用systemd-boot,作为你的引导启动管理器，才能正确让你得到你需要的内容（安全启动是否开启），它更类似于其他*ctl systemd实用程序（localectl、timedatectl……），此处即为有关启动事项的综合管理工具，并且不会影响你的配置。

```
$ bootctl
System:
      Firmware: UEFI 2.80 (American Megatrends 5.26)
 Firmware Arch: x64
   Secure Boot: enabled (user)
  TPM2 Support: yes
  Measured UKI: yes
  Boot into FW: supported
...
```

在这里，我们看到安全启动已启用并强制执行用户模式（user mode）。Secure Boot的其他值包括`disabled`，Setup Mode的其他值包括`setup`。

检查机器是否使用安全启动的另一种方法是使用以下命令：

```
$ od --address-radix=n --format=u1 /sys/firmware/efi/efivars/SecureBoot-8be4df61-93ca-11d2-aa0d-00e098032b8c
```

如果启用了安全启动，此命令将返回`1`作为五个列表中的最后一个整数，例如：

```
6   0   0   0   1
```

但是请注意，如果使用了功能缺失的引导加载程序，内核可能不知道安全引导（即使它在固件中启用）。 这可以通过在系统启动后不久检查内核消息来验证：

```
# journalctl -kg 'secure boot'
kernel: Secure boot disabled
kernel: Secure boot could not be determined
```

能启用的内核会打印`Secure boot enabled`。

引导安装介质

官方安装镜像不支持安全启动（archlinux/archiso#69）。对于安全启动的支持最早在`archlinux-2013.07.01-dual.iso`中引入，但随后在`archlinux-2016.06.01-dual.iso`中被移除。当时，*prebootloader* 被替换为了efitools包，尽管后者使用的是未签名的 EFI 二进制文件。自那之后，官方安全介质就再未支持过安全启动。

为了在开启安全启动的系统上引导安装介质，你需要禁用安全启动，或者修改镜像以添加已签名的引导加载程序（Bootloader）。

Archboot 镜像提供了一种在安装介质中使用安全启动的方式。

禁用安全启动

可以通过 UEFI 固件接口禁用安全启动功能。如何访问固件配置请访问#在启动操作系统之前.

如果使用热键不起作用并且您可以启动 Windows，您可以通过以下方式强制重新启动到固件配置（对于 Windows 10）：*设置* > *更新和安全* > *恢复* > *高级启动（立即重新启动）* > *故障排除* > *高级选项* > *UEFI 固件设置* > *重启*。

请注意，某些主板（Packard Bell 笔记本电脑就是这种情况）只有在您设置了管理员密码（之后可以删除）时才允许禁用安全启动。 另请参阅 Rod Smith 的禁用安全启动。

重新打包安装镜像

参见#ISO 重新打包。

编辑安装介质

如果使用的是 U 盘安装介质，则可以通过手动编辑介质上的 EFI 系统分区来添加对安全启动的支持。

插入 U 盘，然后挂载该分区：

```
# mount /dev/disk/by-label/ARCHISO_EFI /mnt
```

接着按照使用自己的密钥中的说明安装任意已签名的 Bootloader。例如，若要安装 #PreLoader：

```
# mv /mnt/EFI/BOOT/BOOTx64.EFI /mnt/EFI/BOOT/loader.efi
# cp /usr/share/preloader-signed/PreLoader.efi /mnt/EFI/BOOT/BOOTx64.EFI
# cp /usr/share/preloader-signed/HashTool.efi /mnt/EFI/BOOT/HashTool.efi
```

实施安全启动

为了确保理想的安全*启动*设置，有以下几个特定条件：

1. UEFI基本被认为是可信的（尽管有一些众所周知的批评和漏洞），且必须被强密码保护。
2. 不使用默认的制造商或第三方密钥，因为它们被证明会大大削弱安全启动的安全模型。

3. UEFI直接加载用户签名的、兼容EFI boot stub（英语：EFI boot stub）的统一内核镜像（无需引导加载程序），包含微码（如果适用）以及 initramfs。以便在整个启动过程维持安全启动建立的信任链并减少攻击面。
4. 使用完整驱动器加密，这样在创建内核镜像和签署的过程中所涉及的工具与文件就无法被具有机器物理访问权限的人访问并篡改。
5. 可以使用TPM（可信平台模块）获得进一步的提升。

#使用自己的密钥中描述了一个简单且完全自力更生的设置，而#使用已签署的引导加载程序使用了第三方签署的中间工具。

使用 GRUB 引导加载程序在启用安全启动前需要额外的步骤，参见GRUB#启用安全启动。

使用自己的密钥

警告：使用自定义密钥替换平台密钥（PK）会瘫痪某些机器（比如笔记本）上的硬件，甚至无法进入UEFI/BIOS设置恢复。这是因为某些硬件（例如显卡）的固件（OpROMs）在启动时微软使用微软第三方 CA 证书进行签署。许多联想 Thinkpad X、P 和 T 系列笔记本电脑以及华硕 TUF Gaming 笔记本电脑均属于这种情况。

实现安全启动需要下列密钥：

平台密钥 Platform Key （PK）：最高级的密钥。 密钥交换密钥 Key Exchange Key （KEK）：用于签署签名数据库和禁止的签名数据库更新的密钥。 签名数据库 Signature Database （db）：包含密钥和/或被允许的EFI二进制文件的散列值。 禁止的签名数据库 Forbidden Signatures Database （dbx）：包含密钥和/或被禁止的EFI二进制文件的散列值。

详细解释请阅读所有UEFI密钥的含义。

为了使用安全启动，你至少需要**PK**，**KEK**和**db**密钥。 虽然你可以添加多个KEK，db和dbx证书，但只允许一个平台密钥（PK）。

一旦安全启动处于“用户模式”，密钥只能通过用更高级别的密钥签署（使用*sign-efi-sig-list*）来更新。平台密钥可以自己签署。

确定 OpROM 签名

按以下步骤检查是否拥有任何已签名的 OpROM。

列出所有带 OpROM 的设备：

```
$ find /sys/devices/ -name rom
/sys/devices/pci0000:80/0000:80:01.0/0000:81:00.0/rom
...
```

以上展示了一个带有已签名 EFI ROM 的 Intel DC SSD 示例。对每个设备重复以下步骤。

```
$ lspci -s 81:00.0
81:00.0 Non-Volatile memory controller: Intel Corporation PCIe Data Center SSD (rev 01)
```

转储（Dump）ROM 内容：

```
# echo 1 > /sys/devices/pci0000:80/0000:80:01.0/0000:81:00.0/rom
# cat /sys/devices/pci0000:80/0000:80:01.0/0000:81:00.0/rom > ~/intel_ssd.rom
# echo 0 > /sys/devices/pci0000:80/0000:80:01.0/0000:81:00.0/rom
```

获取并编译 TianoCore 的 EDK II 工具：

```
$ make -C BaseTools
```

解析 ROM 文件：

```
$ ./BaseTools/Source/C/bin/EfiRom -d ~/intel_ssd.rom
Image 1 -- Offset 0x0
  ROM header contents
    Signature               0xAA55
    PCIR offset             0x001C
    Signature                PCIR
    Vendor ID                0x8086
    Device ID                0x0953
    Length                   0x001C
    Revision                 0x0003
    DeviceListOffset         0x00
    Class Code               0x000000
    Image size               0x6A00
    Code revision:           0x0000
    MaxRuntimeImageLength    0x00
    ConfigUtilityCodeHeaderOffset 0x00
    DMTFCLPEntryPointOffset 0x00
    Indicator                0x80   (last image)
    Code type                0x03   (EFI image)
  EFI ROM header contents
    EFI Signature           0x0EF1
    Compression Type        0x0001 (compressed)
    Machine type            0x8664 (X64)
    Subsystem               0x000B (EFI boot service driver)
    EFI image offset        0x0038 (@0x38)
```

结果显示，该 OpROM 在偏移量（Offset）0x0 处的 Image 1 偏移量 0x0038 位置包含一个压缩的 EFI 镜像。提取该 EFI 区域：

```
$ dd if=intel_ssd.rom ibs=1 skip=$(( 0x00 + 0x38 )) of=intel_ssd.rom.efi.compressed
```

解压缩该镜像：

```
$ ./BaseTools/Source/C/bin/TianoCompress -d --uefi -v ~/intel_ssd.rom.efi.compressed -o
~/intel_ssd.rom.efi
$ file ../intel_ssd.rom.efi
../intel_ssd.rom.efi: PE32+ executable for EFI (boot service driver), x86-64, 6
sections
```

使用 sbsigntools包 检查签名，可以看到它是使用微软的 DB 证书签名的：

```
$ sbverify --list ~/intel_ssd.rom.efi
signature 1
image signature issuers:
 - /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft Corporation UEFI
CA 2011
image signature certificates:
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/OU=MOPR/CN=Microsoft
Windows UEFI Driver Publisher
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Corporation UEFI CA 2011
```

```
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Corporation UEFI CA 2011
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Corporation Third Party Marketplace Root
```

警告：为了防止设备软变砖（Soft brick），如果发现显卡带有已签名的 ROM，请确保在录入自定义密钥后，你的 Secure Boot DB（允许数据库）中依然保留了必要签发者（Issuers）证书。

备份当前变量

在创建新的密钥和修改EFI变量之前，建议备份当前的变量，以便在发生错误时可以恢复。

安装 efitools包 软件包后，运行以下命令来备份所有四个主要的安全启动变量。

```
$ for var in PK KEK db dbx ; do efi-readvar -v $var -o old_${var}.esl ; done
```

如果你在新电脑或主板上执行这条命令，你提取的变量很可能是微软提供的。

创建密钥

手动流程

要生成密钥，请执行以下步骤。

你将需要多种格式的私钥和证书。

.key : PEM格式的**私钥**，用于签署EFI二进制和EFI签名列表。 ***.crt*** : PEM格式的证书，用于sbsign(1)、sbvarsign(1)与sign-efi-sig-list(1)。 ***.cer*** : DER格式的证书，用于固件。 ***.esl*** : EFI签名列表的证书，用于sbvarsign(1)、efi-updatevar(1)、*KeyTool*与固件。 ***.auth*** : EFI签名列表中的证书，带有认证头（即签名的证书更新文件），用于efi-updatevar(1)、*sbkeysync*、*KeyTool*与固件。

创建一个GUID用于自我识别：

```
$ uuidgen --random > GUID.txt
```

平台密钥：

```
$ openssl req -newkey rsa:4096 -nodes -keyout PK.key -new -x509 -sha256 -days 3650 -
subj "/CN=my Platform Key/" -out PK.crt
$ openssl x509 -outform DER -in PK.crt -out PK.cer
$ cert-to-efi-sig-list -g "$(< GUID.txt)" PK.crt PK.esl
$ sign-efi-sig-list -g "$(< GUID.txt)" -k PK.key -c PK.crt PK PK.esl PK.auth
```

签署一个空白文件用来在“用户模式”中移除平台密钥：

```
$ sign-efi-sig-list -g "$(< GUID.txt)" -c PK.crt -k PK.key PK /dev/null no_PK.auth
```

密钥交换密钥：（生成KEK.auth的指令没有加-a，所以文件只能用来update KEK实现替换。如果要生成用于add添加的KEK.auth，则需要在指令里添加-a，同理db等。PK只能有一个所以直接替换就好，删除的话用本页面介绍的指令生成的rm_PK.auth。）

```
$ openssl req -newkey rsa:4096 -nodes -keyout KEK.key -new -x509 -sha256 -days 3650 -
subj "/CN=my Key Exchange Key/" -out KEK.crt
$ openssl x509 -outform DER -in KEK.crt -out KEK.cer
$ cert-to-efi-sig-list -g "$(< GUID.txt)" KEK.crt KEK.esl
$ sign-efi-sig-list -g "$(< GUID.txt)" -k PK.key -c PK.crt KEK KEK.esl KEK.auth
```

签名数据库密钥：

```
$ openssl req -newkey rsa:4096 -nodes -keyout db.key -new -x509 -sha256 -days 3650 -subj "/CN=my Signature Database key/" -out db.crt
$ openssl x509 -outform DER -in db.crt -out db.cer
$ cert-to-efi-sig-list -g "$(< GUID.txt)" db.crt db.esl
$ sign-efi-sig-list -g "$(< GUID.txt)" -k KEK.key -c KEK.crt db db.esl db.auth
```

辅助脚本

关于这个主题的参考页的作者提供了一个辅助/快捷脚本（需要python）。一个轻度修改的版本也被打包为sbkeysAUR。

为了使用它，请在安全位置创建一个文件夹（比如`/etc/efi-keys/`，如果你接下来计划统一内核镜像的创建与签署自动化过程）并且运行：

```
# mkdir /etc/efi-keys
# cd !$
# curl -L -O https://www.rodsbooks.com/efi-bootloaders/mkkeys.sh
# chmod +x mkkeys.sh
# ./mkkeys.sh
<输入一个通用名，以嵌入至密钥中，比如 "Secure Boot">
```

这将产生所需的不同格式的文件。

更新密钥

一旦安全启动进入“用户模式”，任何对KEK、db以及dbx的改变都需要更高级的密钥签署。

比如，如果你想要更新你的db密钥：

1. 创建新的密钥,
2. 转换其为EFI签名列表，
3. 签署EFI签名列表，
4. 登记已签署的证书更新文件。

```
$ cert-to-efi-sig-list -g "$(< GUID.txt)" new_db.crt new_db.esl
$ sign-efi-sig-list -g "$(< GUID.txt)" -k KEK.key -c KEK.crt db new_db.esl new_db.auth
```

如果你想**添加**另一个密钥至签名数据库中且不替换你的db密钥，你需要使用选项`-a`（请看sign-efi-sig-list(1)）：

```
$ sign-efi-sig-list -a -g "$(< GUID.txt)" -k KEK.key -c KEK.crt db new_db.esl new_db.auth
```

当`*new_db*.auth`被创建，登记它。

签署EFI二进制文件

当*安全启动*激活时（比如在“用户模式”中），只有已被签署的EFI二进制文件（比如应用程序、驱动、统一内核镜像）可以被启动。

使用sbsigntools手动操作

安装sbsigntools包来使用sbsign(1)签署EFI二进制文件。

提示：

- 为了检查一个二进制文件是否被签署并列出它的签名，使用`sbverify --list */path/to/binary*`。
- rEFInd启动管理器的`refind-install`脚本可以签署rEFInd的EFI二进制文件并把它们与db证书一起拷贝到ESP中。 请参考rEFInd#使用你自己的密钥。

注意：如果不带`--output`使用*sbsign*的话，生成文件将会是`*filename*.signed`。详情请看sbsign(1)。

使用*sbsign*来签署你的内核与启动管理器，比如：

```
# sbsign --key db.key --cert db.crt --output /boot/vmlinuz-linux /boot/vmlinuz-linux
# sbsign --key db.key --cert db.crt --output esp/EFI/BOOT/BOOTx64.EFI esp/EFI/BOOT/BOOTx64.EFI
```

警告：只对内核进行签署并不能保护initramfs不被篡改。查看统一内核镜像以了解如何生成一个组合镜像并通过*sbsign*手动签署。

使用pacman钩子签署内核

你也可使用mkinitcpio的pacman钩子在安装与更新时签署一个内核。

复制`/usr/share/libalpm/hooks/90-mkinitcpio-install.hook`到`/etc/pacman.d/hooks/90-mkinitcpio-install.hook`以及复制`/usr/share/libalpm/scripts/mkinitcpio-install`到`/usr/local/share/libalpm/scripts/mkinitcpio-install`。

在`/etc/pacman.d/hooks/90-mkinitcpio-install.hook`里，将：

```
Exec = /usr/share/libalpm/scripts/mkinitcpio-install
```

替换为：

```
Exec = /usr/local/share/libalpm/scripts/mkinitcpio-install
```

在`/usr/local/share/libalpm/scripts/mkinitcpio-install`里，将：

```
install -Dm644 "${line}" "/boot/vmlinuz-${pkgbase}"
```

替换为：

```
sbsign --key /path/to/db.key --cert /path/to/db.crt --output "/boot/vmlinuz-${pkgbase}" "${line}"
```

如果你正在使用systemd-boot，有一个专用的pacman钩子会半自动的做这个任务。

用sbupdate全自动生成和签署统一内核

sbupdate是一个专门为在Arch Linux上自动生成和签署统一内核镜像而制作的工具。它通过Pacman#钩子处理内核的安装、移除和更新。

安装sbupdate-gitAUR并按照项目主页上的说明进行配置。

提示：如果正在使用systemd-boot，设置`OUT_DIR="EFI/Linux"`可以让你的已签署的内核镜像直接被识别而不需要必须的配置。请看systemd-boot(7) § FILES与Systemd-boot#增加启动选项。

一旦完成配置，只需要以root运行`sbupdate`以完成首次镜像生成。

注意：*sbupdate*的输出经常包含错误比如`warning: data remaining[26413568 vs 26423180]: gaps between PE/COFF sections?`。那些是无害的且可以被安全的无视掉。

将固件置于 "设置模式 "下

当平台密钥被删除时，安全启动处于设置模式。要将固件置于设置模式，请进入固件设置工具，并找到删除或清除证书的选项。#在启动操作系统之前中介绍了如何进入设置工具。

在固件中登记密钥

使用下列某种方式登记'*db*、*KEK*和*PK*证书。

提示：如果**dbx**（禁止的签名数据库）是空白的，在下面的说明中，它可以被安全地排除在外。

警告：登记平台密钥会使安全启动进入“用户模式”，也就意味着会离开“设置模式”，所以登记应该完成在所有流程的最后。

使用sbkeysync

安装sbsigntools包。按照以下目录结构创建文件夹`/etc/secureboot/keys`：

```
/etc/secureboot/keys
├── db
├── dbx
├── KEK
└── PK
```

比如，这样用：

```
# mkdir -p /etc/secureboot/keys/{db,dbx,KEK,PK}
```

接下来复制之前生成的每一个*.auth*文件到它们所对应的位置（比如，把`PK.auth`放进`/etc/secureboot/keys/PK`，以此类推）。

检查`sbkeysync`会对你系统的UEFI密钥库做什么改变。

```
# sbkeysync --pk --dry-run --verbose
```

最后，使用`sbkeysync`来登记你的密钥。

```
# sbkeysync --verbose
# sbkeysync --verbose --pk
```

提示：

- 如果`sbkeysync`返回了写错误，那么在执行`sbkeysync`前先运行`chattr -i /sys/firmware/efi/efivars/{PK,KEK,db}*`来暂时改变文件属性，使EFI密钥可以写入`efivars`目录中。详情见chattr(1)。
- 如果为`PK.auth`得到了一个`permission denied`错误，你可以使用命令`efi-updatevar -f /etc/secureboot/keys/PK/PK.auth PK`登记密钥。
- 如果都失败了，这可能是因为固件对BIOS设置上锁了。对于戴尔和联想的系统，你或许需要重置你的BIOS密码：Sysfs固件认证文档

对于联想系统：

```
# cat > /sys/class/firmware-attributes/thinklmi/authentication/Admin/current_password
my-super-secret-password
^D
```

对于戴尔系统：

```
# cat > /sys/class/firmware-attributes/dell-wmi-
sysman/authentication/Admin/current_password
my-super-secret-password
^D
```

然后再次尝试。

下一次启动时，UEFI应该会切回用户模式并且执行安全启动策略。

使用固件设置工具

复制所有的`*.cer`、`*.esl`、`*.auth`文件（除了`noPK.auth`）到一个FAT格式的文件系统（你可以使用EFI 系统分区）。

警告：不要把`noPK.auth`文件复制到你个人电脑的EFI 系统分区（ESP）！ 如果你这样做了，假如有人偷了你的电脑，这个人可以删除UEFI安全启动固件中的个人平台密钥，再一次打开你电脑上的“设置模式”，然后用他们自己的平台密钥替换你的安全启动密钥（PK、KEK、db、dbx），这将完全违背UEFI安全启动的目的。 应当只有你自己可以替换平台密钥，只有你自己可以访问`noPK.auth`文件。因此把`noPK.auth`文件保存在一个只有你自己能访问的安全的地方。这些地方对于`noPK.auth`文件来说都可以是安全的：

- 当你使用`KeyTool`时，可以使用带有EFI 系统分区的外部U盘。要注意，`KeyTool`只能读取未加密存储中的文件。
- 当你使用`sbkeysync`时，运用静态数据加密。

个人电脑的EFI 系统分区一定要按照UEFI的规范进行加密，否则就可以在另一台电脑上被挂载且被读取（如果你的电脑被偷了，硬盘可以被取出来挂载到另一台电脑上）。复制`noPK.auth`到你电脑的ESP后再删除也是不可取的，因为在FAT32格式的EFI 系统分区上被删除的文件可以通过像是PhotoRec这样的工具被还原。

启动固件设置工具，登记**db**、**KEK**与**PK**证书。 固件会有各种不同的界面，请看使用固件的设置工具更换密钥，以了解如何登记密钥。

如果使用的工具支持，最好使用*.auth*和*.esl*而不是*.cer*。

使用KeyTool

`KeyTool.efi`包含在efitools包包中，复制它到ESP。为了能在登记密钥后使用它，请用`sbsign`对它签署。

```
# sbsign --key db.key --cert db.crt --output esp/KeyTool-
signed.efi /usr/share/efitools/efi/KeyTool.efi
```

使用固件设置工具，引导加载程序或者UEFI Shell启动`KeyTool-signed.efi`，然后登记密钥。

参见使用KeyTool更换密钥了解KeyTool的菜单选项。

与其他操作系统进行双重启动

微软 Windows

通常情况下，启用安全引导下，仅使用私人自定义密钥签署微软引导程序（`EFI/Microsoft/Boot/bootmgfw.efi`）而没有在UEFI安全启动变量中登记“Microsoft Windows Production PCA 2011”不可能启动Windows：

- 如果`bootmgfw.efi`**同时**包含来自“Microsoft Windows Production PCA 2011”**和**你自己的安全启动数据库密钥（也就是说有**两个签名**），那么UEFI固件实现（比如*INSYDE Corp. 4096.01 (UEFI Version 2.31, Version F.70, Release Date: 07/18/2016, BIOS Revision 15.112, Firmware Revision: 29.66)*）将不会启动`bootmgfw.efi`，并且会抛出一个安全违规错误（**`Selected boot image did not authenticate. Press ENTER to continue.`**）。像是这样的UEFI固件实现可能只能读取**第一个**签名而忽略**第二个**。只有第二个签名的证书被注册到UEFI安全启动变量中，所以安全启动验证失败。
- 如果从`bootmgfw.efi`中剥离/移除“Microsoft Windows Production PCA 2011”证书，而只在文件中添加了你自己的安全启动数据库密钥的签名，虽然UEFI会启动该文件但Windows将启动一个恢复/修复环境。Windows会抱怨说：Windows的安装被破坏了（因为`bootmgfw.efi`中缺失了“Microsoft Windows Production PCA 2011”证书）。

所以为了实现双系统：

- 你必须把`bootmgfw.efi`的散列值添加到`DB`变量中允许的散列值列表中，而且每次Windows更新改变`bootmgfw.efi`时，你都必须更新DB变量。这非常繁琐，容易出错，而且不被微软支持，例如，Bitlocker在这种设置下将无法正常工作（Bitlocker每次解密Windows分区时都会要求你提供恢复密码）。
- 或者你必须将微软的证书添加到UEFI安全启动的变量中，微软有两个C0证书和一个C1证书：
- - 为了使Windows操作系统可以被引导，Microsoft Windows Production PCA 2011 证书必须被包含进`DB`变量。
- 为了使用第三方二进制，比如UEFI驱动、其他ROM、shim包等，Microsoft Corporation UEFI CA 2011 证书（又叫作微软第三方UEFI CA证书）必须被包含进`DB`变量。

- 为了“通过更新DBX和潜在更新DB来撤销不良镜像对新的Windows签名镜像做准备”，Microsoft Corporation KEK CA 2011 证书应当被包含进KEK变量。然而没有“Microsoft Corporation KEK CA 2011”证书，Windows也可以启动。

使用微软的GUID（77fa9abd-0359-4d32-bd60-28f4e78f784b），根据微软的多个DER格式的DB证书创建多个EFI签名列表。然后为了方便，把它们合并进一个文件中：

```
$ sbsiglist --owner 77fa9abd-0359-4d32-bd60-28f4e78f784b --type x509 --output
MS_Win_db.esl MicWinProPCA2011_2011-10-19.crt
$ sbsiglist --owner 77fa9abd-0359-4d32-bd60-28f4e78f784b --type x509 --output
MS_UEFI_db.esl MicCorUEFCA2011_2011-06-27.crt
$ cat MS_Win_db.esl MS_UEFI_db.esl > MS_db.esl
```

可选操作（为了严格符合的微软UEFI安全启动要求）：使用微软的GUID（77fa9abd-0359-4d32-bd60-28f4e78f784b），根据微软的KEK格式的DB证书创建EFI签名列表：

```
 $ sbsiglist --owner 77fa9abd-0359-4d32-bd60-28f4e78f784b --type x509 --output
MS_Win_KEK.esl MicCorKEKCA2011_2011-06-24.crt
```

使用你自己的KEK签署一个DB变量更新。使用带有-a选项的sign-efi-sig-list来**添加**而不是替换一个DB证书：

```
$ sign-efi-sig-list -a -g 77fa9abd-0359-4d32-bd60-28f4e78f784b -k KEK.key -c KEK.crt db
MS_db.esl add_MS_db.auth
```

可选操作（为了严格符合微软的UEFI安全启动要求）：使用你自己的PK签署一个KEK变量更新。使用带有-a选项的sign-efi-sig-list来**添加**而不是替换一个KEK证书：

```
$ sign-efi-sig-list -a -g 77fa9abd-0359-4d32-bd60-28f4e78f784b -k PK.key -c PK.crt KEK
MS_Win_KEK.esl add_MS_Win_KEK.auth
```

按照#在固件中登记密钥的内容来登记add_MS_db.auth，为了严格符合微软的UEFI安全启动要求，将add_MS_Win_KEK.auth放入UEFI安全启动数据库的变量中。

使用已签署的引导加载程序

使用已签署的引导加载程序意味着使用由**微软的密钥签署的**引导加载程序。有两个已知的签署的引导加载程序：PreLoader和shim。它们的目的是连锁加载其他EFI二进制文件（通常是引导加载程序）。由于微软不会签署一个自动启动任何未签署的二进制文件的引导加载程序，PreLoader和shim使用一个叫做机器所有者密钥列表的白名单，缩写为MokList。如果二进制文件的SHA256散列值（Preloader和shim）或签署二进制文件的密钥（shim）在MokList中，它们就执行它，如果没有，他们就启动一个密钥管理工具，允许注册散列值或密钥。

警告：微软所谓的“Secured-core PCs”在出厂时并没有登记微软第三方UEFI CA证书（Microsoft Corporation UEFI CA 2011）。唯一登记的DB证书是Microsoft Windows Production PCA 2011证书，该证书专门用于签署Windows的引导加载程序。

微软第三方UEFI CA证书的登记需要在固件设置中启用，以启动用该证书签署的EFI二进制文件和OpROM。

PreLoader

运行时，PreLoader尝试启动`loader.efi`。如果`loader.efi`的散列值不在MokList中，PreLoader将会启动`HashTool.efi`。在HashTool中，你必须登记你想要启动的EFI二进制的散列值，也就是你的引导加载程序（`loader.efi`）以及内核的散列值。

注意：每次你更新任何二进制文件（如引导加载程序或内核）时，你都需要登记它们的新散列值。

提示：rEFInd加载管理程序的`refind-install`脚本可以复制rEFInd与PreLoader的EFI二进制文件到ESP。请看REFInd#使用_PreLoader。

设置PreLoader

注意：efitools包包中的`PreLoader.efi`与`HashTool.efi`没有被签署，所以它们的作用是有限的。你可以从preloader-signedAUR获取被签署的`PreLoader.efi`与`HashTool.efi`，或者手动下载它们。

安装preloader-signedAUR和`HashTool.efi`然后复制`PreLoader.efi`和`HashTool.efi`到引导加载程序目录中。对于systemd-boot，使用：

```
# cp /usr/share/preloader-signed/{PreLoader,HashTool}.efi esp/EFI/systemd
```

现在复制一份引导加载程序的二进制文件并重命名为`loader.efi`。对于systemd-boot，使用：

```
# cp esp/EFI/systemd/systemd-bootx64.efi esp/EFI/systemd/loader.efi
```

最终，创建一个新的NVRAM引导项来启动`PreLoader.efi`：

```
# efibootmgr --unicode --disk /dev/sdX --part Y --create --label "PreLoader" --
loader /EFI/systemd/PreLoader.efi
```

把`*X*`替换为驱动器字符并把`*Y*`替换为EFI 系统分区的分区号。

这个启动项应当添加进启动列表中并作为第一项启动，用`efibootmgr`命令检查，必要时调整启动顺序。

回撤

如果启动这个自定义NVRAM启动项时出现问题，复制`HashTool.efi`和`loader.efi`到UEFI系统自动启动的默认加载程序位置：

```
# cp /usr/share/preloader-signed/HashTool.efi esp/EFI/BOOT/
# cp esp/EFI/systemd/systemd-bootx64.efi esp/EFI/BOOT/loader.efi
```

复制一份`PreLoader.efi`并重命名：

```
# cp /usr/share/preloader-signed/PreLoader.efi esp/EFI/BOOT/BOOTx64.EFI
```

对于特别顽固的UEFI实现，可以将`PreLoader.efi`复制到Windows系统使用的默认加载程序位置：

```
# mkdir -p esp/EFI/Microsoft/Boot
# cp /usr/share/preloader-signed/PreLoader.efi esp/EFI/Microsoft/Boot/bootmgfw.efi
```

注意：如果与Windows双启动，请先备份原始的`bootmgfw.efi`，因为替换它可能会导致Windows更新的问题。

如前所述，将`HashTool.efi`和`loader.efi`复制到`'esp*/EFI/Microsoft/Boot/*`。

当系统在启用安全启动下启动时，按照上述步骤登记`loader.efi`和`/vmlinuz-linux`（或任何使用的内核镜像）。

如何在启动时使用?

当出现一条消息说：`Failed to Start loader... I will now execute HashTool.`。要使用HashTool来登记`loader.efi`和`vmlinuz.efi`的散列值，请遵循以下步骤。这些步骤假定了一个重制的archiso安装介质的项目。你得到的确切的项目取决于你的引导加载程序的设置。

- 选择 *OK*
- 在HashTool的主菜单，选择*Enroll Hash*，选择`\loader.efi`然后选*Yes*确认。同样，再选择*Enroll Hash*和`archiso`来进入archiso的目录，然后选择`vmlinuz.efi`，用*Yes*确认。接下来选择*Exit*返回至启动设备选单。
- 在启动设备选单中选择*Arch Linux archiso x86_64 UEFI CD*

移除PreLoader

注意：既然你要删除东西，那么最好先备份。

卸载preloader-signedAUR然后

并简单地删除复制的文件并恢复配置。 对于systemd-boot使用：

```
# rm esp/EFI/systemd/{PreLoader,HashTool}.efi
# rm esp/EFI/systemd/loader.efi
# efibootmgr --unicode --bootnum N --delete-bootnum
# bootctl update
```

这里`*N*`指的是用来启动`PreLoader.efi`所创建的NVRAM启动项。 如有必要，用*efibootmgr*命令检查并调整启动顺序。

注意：上述命令涵盖了最简单的情况；如果你创建、复制、重命名或编辑了更多的文件，那么你也必须处理它们。如果PreLoader是你的操作引导项，你还需要#禁用安全启动。

删除已登记的散列值

在MOK数据库中登记的每一条散列值都会吃掉NVRAM的一小块空间。你可能想删除无用的散列值，以释放空间并防止过时的程序启动。

安装efitools包然后复制`KeyTool.efi`：

```
# cp /usr/share/efitools/efi/KeyTool.efi esp/EFI/systemd/KeyTool.efi
```

引导启动至Preloader，你会看到KeyTool引导项。然后你就可以在MOK数据库中编辑散列值。

shim

在运行时，shim会尝试启动`grubx64.efi`。如果MokList没有包含`grubx64.efi`的散列值或它所签署的密钥，shim将会启动MokManager（`mmx64.efi`）。你必须在MokManager里登记你想要启动的EFI二进制文件（你的引导加载程序`grubx64.efi`与内核）的散列值，或者登记你想要签署的密钥。

注意：

- 如果你使用#shim与散列值，每次你升级任何二进制文件（比如引导加载程序或内核）后都需要重新登记它们的散列值。
- 在版本15.3之后，shim不再启动没有合法`.sbat`部分的EFI二进制文件。想要验证一个EFI二进制文件是否具有这个部分，运行`objdump -j .sbat -s */path/to/binary.efi*`。详情看：SBAT文档。
- 或许值得注意的是，说实话，如果你对安全启动带来的安全性并不感兴趣，之所以想要开启它仅仅是为了满足Windows 11的要求，你或许可以考虑关闭shim的验证过程：`mokutil --disable-validation`。这种情况下你不必签署grub（或许sbat仍然需要）以及内核镜像，且还可以通过`chainloader`从grub启动Windows。

设置shim

提示：rEFInd引导程序中的`refind-install`脚本可以签署rEFInd的EFI二进制文件并与shim和MOK证书一并复制到ESP中。详情见rEFInd#使用 shim。

安装shim-signedAUR.

重命名当前的引导加载程序为`grubx64.efi`

```
# mv esp/EFI/BOOT/BOOTx64.EFI esp/EFI/BOOT/grubx64.efi
```

复制*shim*与*MokManager*到ESP中的启动引导文件夹，然后将`shimx64.efi`重命名为引导加载程序之前的文件名：

注意：

- 确保你**不要复制**同一目录下的fbx64.efi，除非你真的有一个有效的bootx64.csv可以使用。否则shim不会执行grubx64.efi，这将无法正常工作，只会重置机器。

```
# cp /usr/share/shim-signed/shimx64.efi esp/EFI/BOOT/BOOTx64.EFI
# cp /usr/share/shim-signed/mmx64.efi esp/EFI/BOOT/
```

最终，创建一个新的NVRAM引导项来启动`BOOTx64.EFI`：

```
# efibootmgr --unicode --disk /dev/sdX --part Y --create --label "Shim" --
loader /EFI/BOOT/BOOTx64.EFI
```

*shim*可以通过机器所有者密钥或MokList所存储的散列值验证二进制文件。

机器所有者密钥 - Machine Owner Key (MOK)：一个由用户生成并被用来签署EFI二进制文件的密钥。 **散列值 - hash**：一个EFI二进制文件的SHA256散列值。

使用散列值更简单但是每次升级你的引导加载器或内核后需要添加它们新的散列值进MokManager。而使用MOK你只需要添加密钥一次，但是每次引导加载器或内核升级后都需要重新签署。

shim与散列值

如果*shim*无法找到在MokList中找到`grubx64.efi`的SHA256散列值，它将会启动MokManager（`mmx64.efi`）。

在*MokManager*中选择*Enroll hash from disk*，找到`grubx64.efi`然后添加它到MokList。重复这一步骤添加你的内核`vmlinuz-linux`。完成之后，选择*Continue boot*，你的引导加载程序便会启动并成功加载内核。

shim与密钥

安装sbsigntools包。

你需要：

.key：PEM格式的私有密钥，用于签署EFI二进制文件。 ***.crt***：PEM格式的证书，用于*sbsign*。 ***.cer***：DER格式的证书，用于*MokManager*。

创建一个机器所有者密钥（MOK）：

```
$ openssl req -newkey rsa:4096 -nodes -keyout MOK.key -new -x509 -sha256 -days 3650 -
subj "/CN=my Machine Owner Key/" -out MOK.crt
$ openssl x509 -outform DER -in MOK.crt -out MOK.cer
```

签署你的引导加载程序（名为`grubx64.efi`）以及内核：

```
# sbsign --key MOK.key --cert MOK.crt --output /boot/vmlinuz-linux /boot/vmlinuz-linux
# sbsign --key MOK.key --cert MOK.crt --output esp/EFI/BOOT/grubx64.efi
esp/EFI/BOOT/grubx64.efi
```

每次它们更新后你都需要这么做。你可以使用一个Pacman钩子自动化签署过程，比如：

```
/etc/pacman.d/hooks/999-sign_kernel_for_secureboot.hook
[Trigger]
Operation = Install
Operation = Upgrade
Type = Package
Target = linux
Target = linux-lts
Target = linux-hardened
Target = linux-zen

[Action]
Description = Signing kernel with Machine Owner Key for Secure Boot
When = PostTransaction
Exec = /usr/bin/find /boot/ -maxdepth 1 -name 'vmlinuz-*' -exec /usr/bin/sh -c
'if ! /usr/bin/sbverify --list {} 2>/dev/null | /usr/bin/grep -q "signature
certificates"; then /usr/bin/sbsign --key MOK.key --cert MOK.crt --output {} {}; fi' ;
```

```
Depends = sbsigntools
Depends = findutils
Depends = grep
```

或者使用一个 mkinitcpio钩子：

```
/etc/initcpio/post/kernel-sbsign
#!/usr/bin/env bash

kernel="$1"
[[ -n "$kernel" ]] || exit 0

# use already installed kernel if it exists
[[ ! -f "$KERNELDESTINATION" ]] || kernel="$KERNELDESTINATION"

keypairs=(/path/to/db.key /path/to/db.crt)

for (( i=0; i<${#keypairs[@]}; i+=2 )); do
    key="${keypairs[$i]}" cert="${keypairs[(( i + 1 ))]}"
    if ! sbverify --cert "$cert" "$kernel" &>/dev/null; then
        sbsign --key "$key" --cert "$cert" --output "$kernel" "$kernel"
    fi
done
```

复制`MOK.cer`到一个FAT格式的文件系统（你可以使用EFI 系统分区）。

重新启动并启用“安全启动”。如果*shim*未能找到在MokList中找到签署的`grubx64.efi`证书，它将启动MokManager（`mmx64.efi`）。

在*MokManager*中选择*Enroll key from disk*，找到`MOK.cer`并把它添加到MokList。完成之后，选择*Continue boot*，你的引导加载程序便会启动并成功加载任何由你的机器所有者密钥签署的二进制文件。

shim与密钥和GRUB

**本文或本章节可能需要合并到GRUB。**

附注： 关于构建自己的EFI二进制文件并选择模块或许应该放进GRUB页面。（在 Talk:UEFI/安全启动 中讨论）

只有GRUB的EFI二进制文件包含所有能够读取vmlinuz与initramfs镜像所在文件系统的模块，才能在安全启动模式下成功启动。

从GRUB版本`2.06.r261.g2f4430cc0`之后，通过`insmod`在安全启动下读取模块不再被允许，因为这将违反不侧载任意代码的构想。如果GRUB模块没有被嵌入进EFI二进制文件，GRUB会尝试通过`insmod`侧载它们，这会在启动时报出如下错误信息：

```
error: prohibited by secure boot policy
```

根据Ubuntu的官方构建脚本，Ubuntu嵌入了下列GRUB模块在它已签署的GRUB EFI二进制文件`grubx64.efi`中。

- “base”模块，为了从CD或者单一分区硬盘中启动所必需含有： `all_video`, `boot`, `btrfs`, `cat`, `chain`, `configfile`, `echo`, `efifwsetup`, `efinet`, `ext2`, `fat`, `font`, `gettext`, `gfxmenu`, `gfxterm`, `gfxterm_background`,

gzio, halt, help, hfsplus, iso9660, jpeg, keystatus, loadenv, loopback, linux, ls, lsefi, lsefimmap, lsefisystab, lssal, memdisk, minicmd, normal, ntfs, part_apple, part_msdos, part_gpt, password_pbkdf2, png, probe, reboot, regexp, search, search_fs_uuid, search_fs_file, search_label, sleep, smbios, squash4, test, true, video, xfs, zfs, zfscrypt, zfsinfo

- “platform-specific”模块，为了x86_64-efi架构所必需含有，例如：
- play：为了在启动中播放声音
- cpuid：为了启动中的CPU支持
- linuxefi：为了支持UEFI启动
- tpm：为了支持度量启动（Measured boot）与可信平台模块
- “advanced”模块，包含下列模块：
- cryptodisk：为了从plain模式加密的硬盘启动
- gcry_*algorithm*：为了支持特定的散列和加密算法
- luks：为了从LUKS加密的硬盘启动
- lvm：为了从LVM逻辑分卷硬盘启动
- mdraid09, mdraid1x, raid5rec, raid6rec：为了从RAID虚拟硬盘中启动

你必须以一个shell变量的形式构建你的GRUB模块列表，我们将它表示为GRUB_MODULES。你可以使用最新的Ubuntu脚本作为一个出发点，并修剪掉你系统中不需要的模块。修剪模块将使启动过程相对更快，并在ESP分区上节省一些空间。

如果GRUB从UEFI shim加载器启动，为了提高安全性，你还需要在EFI二进制文件中包含一个安全启动高级目标（SBAT）文件/部分。这个SBAT文件/部分包含了关于GRUB二进制文件的元数据（版本、维护者、开发者、上游URL），并且使shim更容易阻止某些有安全漏洞的GRUB版本被加载，正如UEFI shim引导加载程序的安全启动生命流程的提升文档中所解释的那样。

如果grubx64.efi中的SBAT部分丢失，UEFI shim引导加载程序的第一阶段将无法启动grubx64.efi。

如果安装了GRUB，/usr/share/grub/sbat.csv中提供了一个SBAT的*.csv*文件样本。

使用提供的/usr/share/grub/sbat.csv文件和所有需要的GRUB_MODULES重新安装GRUB并签署：

```
# grub-install --target=x86_64-efi --efi-directory=esp --modules=${GRUB_MODULES} --
sbat /usr/share/grub/sbat.csv
# sbsign --key MOK.key --cert MOK.crt --output esp/EFI/GRUB/grubx64.efi
esp/EFI/GRUB/grubx64.efi
```

```
# cp esp/GRUB/grubx64.efi esp/boot/grubx64.efi
```

重新启动，在*MokManager*中选择密钥，安全启动就应该可以工作了。

移除shim

卸载shim-signedAUR，移除被复制的*shim*与*MokManager*文件并重命名为你的引导加载程序默认名。

保护安全启动

防止有物理访问权限的人禁用安全启动的唯一方法是用密码保护固件设置。

大多数UEFI固件提供这样的功能，通常列在固件设置中的“安全”部分。

提示与技巧

sbctl

sbctl包是一种用户友好的设置安全启动和签署文件方式。

注意：sbctl并不能在所有的硬件上工作。它的实际工作效果取决于制造商。

创建并登记密钥

在开始之前，进入你的固件设置将安全启动模式设置为*设置模式*。每个设备都有所不同。

当你重新登录后，检查安全启动的状态：

```
$ sbctl status
```

你应当看到sbctl并未安装，并且“设置模式”（Setup Mode）已经开启，以及安全启动已关闭。

若显示“设置模式”（Setup Mode）关闭，请进入BIOS手动重置进入Setup Mode。

接下来创建你的自定义安全启动密钥：

```
# sbctl create-keys
```

（在sbctl 0.8以及更新版本之后，执行此命令默认会把密钥创建在/var/lib/sbctl/keys/文件夹内）

将你的密钥与微软的密钥登记入UEFI中：

```
# sbctl enroll-keys -m
```

警告：启用安全启动时，一些固件会用微软的密钥进行签署和验证。不对设备进行验证可能会使它们变砖。如果想仅登记你的密钥而不登记微软的，请运行：`# sbctl enroll-keys`。**只有在你清楚了解自己在做什么的情况下才可以这样做。**

再次检查安全启动状态：

```
$ sbctl status
```

sbctl现在应该已经安装好了，但是在用你刚刚创建的密钥对启动文件进行签署之前，安全启动将无法工作。

签署

为了安全启动工作，检查需要被签署的文件：

```
# sbctl verify
```

现在签署所有的未被签署的文件。通常是内核与引导加载程序需要被签署。比如：

```
# sbctl sign -s /boot/vmlinuz-linux
# sbctl sign -s /boot/EFI/BOOT/BOOTX64.EFI
```

需要被签署的文件将取决与你系统的结构，内核以及引导加载程序。

现在万事大吉！重新启动你的系统，并在固件设置中重新打开安全启动。如果引导加载程序与操作系统被加载，安全启动应当开始工作了。用下列命令确认：

```
$ sbctl status
```

使用pacman钩子自动签署

sbctl带有一个Pacman钩子可以在内核、systemd或者引导加载程序更新后自动地签署所有的新文件。

提示：如果你通过Systemd-boot启用了`systemd-boot-update.service`，那么引导加载程序只会在重启后升级，导致sbctl的Pacman钩子因此而不签署新的文件。 变通的方法是直接在`/usr/lib/`中小心的签署引导加载程序，这样`bootctl install`与`update`将会自动识别并复制*.efi.signed*（如果有的话）到ESP，而不是普通的*.efi*文件。详情见bootctl(1)。

```
# sbctl sign -s -o /usr/lib/systemd/boot/efi/systemd-
bootx64.efi.signed /usr/lib/systemd/boot/efi/systemd-bootx64.efi
```

用自定义密钥签署官方ISO

使用自定义密钥的安全启动支持可以添加到官方ISO中，只需提取出引导加载程序（`BOOTx64.EFI`与`BOOTIA32.EFI`）、内核与UEFI Shell，并对它们进行签署，然后用被签署的文件重新打包ISO。

安装libisoburn包、mtools包与sbsigntools包。

首先解压出相关的文件以及El-Torito启动镜像：

```
$ osirrox -indev archlinux-YYYY.MM.DD-x86_64.iso \
       -extract_boot_images ./ \
       -extract /EFI/BOOT/BOOTx64.EFI BOOTx64.EFI \
       -extract /EFI/BOOT/BOOTIA32.EFI BOOTIA32.EFI \
       -extract /shellx64.efi shellx64.efi \
       -extract /shellia32.efi shellia32.efi \
       -extract /arch/boot/x86_64/vmlinuz-linux vmlinuz-linux
```

*mkarchiso*所使用的，xorrisofs(1)的`-rational-rock`选项，会使ISO 9660中的文件变为只读，且在解压后仍然如此。我们需要让这些文件变为可写，以便它们可以被修改：

```
$ chmod +w BOOTx64.EFI BOOTIA32.EFI shellx64.efi shellia32.efi vmlinuz-linux
```

使用一个已登记的数据库密钥和证书签署这些文件：

```
$ sbsign --key db.key --cert db.crt --output BOOTx64.EFI BOOTx64.EFI
$ sbsign --key db.key --cert db.crt --output BOOTIA32.EFI BOOTIA32.EFI
$ sbsign --key db.key --cert db.crt --output shellx64.efi shellx64.efi
$ sbsign --key db.key --cert db.crt --output shellia32.efi shellia32.efi
$ sbsign --key db.key --cert db.crt --output vmlinuz-linux vmlinuz-linux
```

将引导加载程序与UEFI Shell复制到`eltorito_img2_uefi.img`中。它将被用作为EFI系统分区，并被列为一个El-Torito UEFI启动镜像。`eltorito_img2_uefi.img`的大小是固定的，但*mkarchiso*增加了1 MiB的空余空间，用于对齐以及为后续的扇区保留，所以签署所带来的大小增加应该不是问题。

```
$ mcopy -D oO -i eltorito_img2_uefi.img BOOTx64.EFI BOOTIA32.EFI ::/EFI/BOOT/
$ mcopy -D oO -i eltorito_img2_uefi.img shellx64.efi shellia32.efi ::/
```

使用修改过的El-Torito UEFI引导映像，重新打包ISO，并在ISO 9660中加入签署的引导加载程序文件、UEFI Shell和内核。

```
$ xorriso -indev archlinux-YYYY.MM.DD-x86_64.iso \
       -outdev archlinux-YYYY.MM.DD-x86_64-Secure_Boot.iso \
       -boot_image any replay \
       -append_partition 2 0xef eltorito_img2_uefi.img \
       -map BOOTx64.EFI /EFI/BOOT/BOOTx64.EFI \
       -map BOOTIA32.EFI /EFI/BOOT/BOOTIA32.EFI \
       -map shellx64.efi /shellx64.efi \
       -map shellia32.efi /shellia32.efi \
       -map vmlinuz-linux /arch/boot/x86_64/vmlinuz-linux
```

启动生成的`archlinux-*YYYY.MM.DD*-x86_64-Secure_Boot.iso`文件。

另见

- Understanding the UEFI Secure Boot Chain by tianocore
- Wikipedia:Unified Extensible Firmware Interface#Secure boot
- Dealing with Secure Boot by Rod Smith
- Controlling Secure Boot by Rod Smith
- UEFI secure booting (part 2) by Matthew Garrett
- UEFI Secure Boot by James Bottomley
- efitools README
- Will your computer's "Secure Boot" turn out to be "Restricted Boot"? — Free Software Foundation
- Free Software Foundation recommendations for free operating system distributions considering Secure Boot
- Intel's UEFI Secure Boot Tutorial
- Secure Boot, Signed Modules and Signed ELF Binaries
- National Security Agency docs: UEFI Defensive Practices Guidance and unclassified UEFI Secure Boot customization
- sbkeysync & maintaining uefi key databases by Jeremy Kerr
- Secure your boot process: UEFI + Secureboot + EFISTUB + Luks2 + lvm + Arch Linux (2020-07)

- How is hibernation supported, on machines with UEFI Secure Boot? (Security StackExchange)
- Authenticated Boot and Disk Encryption on Linux by Lennart Poettering (2021-09-23)

安装过程

安装是 Arch 绕不开的手工环节，介质、分区与引导三件事必须对齐。本章沿准备介质、开始安装、配置系统、重启、安装后收尾走完一次完整安装，再展开变体：U 盘安装介质与 Ventoy 决定从哪里装，Archinstall 与 genfstab 负责自动化与挂载收尾，无盘系统、多引导 USB、Fake RAID、ZFS 对应特殊场景。

顺序上《安装指南》必读，是其余各节的地基；《U 盘安装介质》《Archinstall》《Genfstab》紧随其后，多数人靠这几节就能装完。其余按需取用：无盘系统、多引导 USB、Fake RAID、ZFS，以及从现有发行版安装和在可移动设备上安装，第一遍都可以跳过。最容易踩的坑是照抄别人的命令，分区、引导加载程序与挂载点必须匹配自己的固件类型与磁盘布局，套错了重启后便无法引导。

3.1　安装指南

难度 高级 | **篇幅** 41.0 KB | **所属** 安装过程

本节将讲到

- 安装前的准备
- 开始安装系统
- 配置系统
- 重新启动计算机
- 安装后的工作

本文将指导如何使用官方安装映像启动的 Live 系统安装 Arch Linux。它可以提供无障碍辅助功能，对此使用无障碍功能安装 Arch Linux 页面有详细描述。关于其它的安装方法，请您参阅安装过程分类下的其他条目。

建议在安装前阅读常见问题。对于本文中使用的惯用术语，请参阅阅读帮助。请将代码、命令中*斜体*（也有可能是楷体、仿宋体字）的部分按实际情况替换成具体内容。

本指南旨在简洁且通俗易懂，建议您严格遵循每一节中的顺序与指示执行安装。有关更详细的说明，请您仔细阅读本指南内相应的 ArchWiki 文章或各类程序的 Man 手册或 Info 手册并掌握核心工具和部分软件包的基础用法。若您需要交互帮助，可以访问 Arch Linux 中文社区的聊天群组或论坛甚至国际社区。

Arch Linux 被设计为可以运行在配置为最低 512 MiB 内存的 x86_64 架构的计算机上，但如果是从安装媒介启动系统并成功安装在计算机硬盘中，则可能需要更多的内存。基本安装方式将占用小于 2 GiB 的硬盘存储空间。由于安装过程中需要从远程存储库获取软件包，计算机将需要一个有效的互联网连接。

安装前的准备

获取安装映像

打开官方映像下载页面 https://archlinux.org/download/ ，并根据需要的引导方式，获取 ISO 文件或 netboot 映像以及相应的 PGP 签名。

验证签名

建议使用前先验证所下载文件的签名，特别是从 **HTTP 镜像源/非官方镜像仓库**下载的文件，因为 HTTP 连接一般来说容易遭到拦截而提供恶意文件，而非官方镜像仓库可能会对镜像的文件进行恶意修改。验证签名也能及时发现文件损坏。

验证哈希校验和

在 Arch Linux 网站的下载页面的 HTTP Direct Downloads 一节提供了安装映像的校验和，包括 SHA256 和 BLAKE2b 两种算法，您可以使用任何支持其中一个算法的检验工具来检查文件校验和是否正确。例如，7-Zip 具有此功能。

如果您正在使用的系统上安装了 `sha256sum` 命令，则可以下载网站提供的 *sha256sums.txt* 文件，并将它放在前面下载的安装映像的同一个目录下，然后运行以下命令来验证：

```
$ sha256sum -c sha256sums.txt --ignore-missing
```

验证 PGP 签名

从下载页面 下载 **PGP 签名**到 ISO 文件所在的路径，并根据该页面提供的指引来校验 ISO 文件。

另外，在一台已经安装 Arch Linux 的计算机上可以通过以下方式验证：

```
$ pacman-key -v archlinux-版本-x86_64.iso.sig
```

注意：

- 如果安装映像是从镜像站点下载，而不是从 ArchLinux.org 下载的话，其签名有被伪造的风险。在这种情况下，请您确保用来解码签名的公钥是被另一个可信的密钥签署的。`gpg` 命令将会输出公钥的指纹。
- 另一种验证签名的方法是确保公钥的指纹等于其中一位签署了 ISO 文件 Arch Linux 开发者的指纹。请您参阅维基百科获取更多关于公钥加密的信息。

提示：在 Windows 上，您可尝试Gpg4win，注意：您所下载、安装和使用的 Gpg4win 程序需要您自行甄别其真伪。

准备安装介质

Arch Linux 的ISO文件可以被制作成多种类型安装介质，如 U 盘、光盘和带有 PXE 的网络安装映像。请按照合适的文章与教程，使用ISO文件为自己准备安装介质。

下载archiso文件时请考虑使用镜像站，比如中国境内的镜像站。可以考虑使用 Ventoy 制作安装介质。

对于 Netboot 映像，请按照网络引导#从U盘启动准备用于 UEFI 引导的 USB 闪存驱动器。

启动到 live 环境

注意：Arch Linux 安装映像不支持 UEFI 安全启动（Secure Boot）功能。如果要引导安装媒介，需要禁用安全启动。如果需要，可在完成安装后重新配置。

1. 选择从带有 Arch 安装文件的媒介启动，通常是需要在计算机启动加电自检时快速按下某个热键（比如部分主板会是 `F12` 键或 `F8` 键）。启动时的画面也可能会有提示，详情请参考自己的计算机的说明书或主板说明书。
2. 当引导加载程序菜单出现时，
3. 如果你使用了 ISO 映像，选择 *Arch Linux install medium* 并按 `Enter` 进入安装环境。
4. 如果您使用了 Netboot 映像，请从 *Mirror* 菜单中选择一个地理位置接近的镜像，然后选择 *Boot Arch Linux* 并按 `Enter`。
5. 您将会以 root 身份登录进入一个虚拟控制台，默认的 Shell 是 Zsh。

提示：

- 安装映像在 UEFI 模式下使用 systemd-boot 引导，在 BIOS 模式下使用 Syslinux 引导。分别使用 `e` 或 `Tab` 来输入引导参数。Netboot 映像使用 iPXE，可以在*启动选项*菜单中指定启动参数。请参阅 /usr/share/doc/mkinitcpio-archiso/README.bootparams 获取引导参数列表。
- 手动定义启动参数的一个常见例子是改变系统显示在超高分辨率（HiDPI）屏幕的字体的大小。为使系统在 HiDPI 屏幕上显示的字体有更好的可读性——当 Live 系统启动时屏幕还没有被识别为 HiDPI 的时候——使用 `fbcon=font:TER16x32` 会有帮助。参见 HiDPI#Linux 控制台的详细解释。

如果要使用其他控制台（例如在安装时使用 Lynx 查看本指南），可以使用 `Alt+*`箭头`*` 键盘快捷键切换不同的控制台。要编辑配置文件，可以使用 mcedit(1)、nano 和 vim 等文本编辑软件。请参阅 packages.x86_64 获取 Live 环境中包含的软件包列表。

关于 archinstall 的提示

如果您觉得用命令行安装过于繁琐，可以试试官方的安装脚本 `archinstall`。详情可参见 archinstall。

但**不建议**初学者用 archinstall 走捷径，因为手动安装 Arch Linux 本身就是 Arch Linux 教学实操中**重要的一环**，少了这个环节，会让初学者面对一些基础问题时束手无策，为自己和社区带来不必要的麻烦。

```
# archinstall
```

提示：使用 archinstall 安装基础系统以后，可能需要进行一些配置，才能使用户体验得以优化提升。参见#安装后的工作与建议阅读等页面。

配置控制台键盘布局和字体

系统启动后，控制台键盘布局默认为 `us`（美式键盘）。如果您需要列出所有可用的键盘布局，可以使用以下命令：

```
# localectl list-keymaps
```

如果您想要更改键盘布局，可以使用 loadkeys(1)。比如，要添加德语键盘布局：

```
# loadkeys de-latin1
```

注意：中文环境使用者请勿照抄以上设置，默认键盘布局（us/C）已经可以正常地使用。

控制台字体位于 `/usr/share/kbd/consolefonts/` 目录中，设置方式请参考 setfont(8)。例如，要使用适合 HiDPI 屏幕)的最大字体之一，请运行以下命令：

```
# setfont ter-132b
```

验证引导模式

要验证系统目前的引导模式，请检查 UEFI 固件位数：

```
# cat /sys/firmware/efi/fw_platform_size
```

- 如果命令结果为 `64`，则系统是以 UEFI 模式引导且使用 64 位 x64 UEFI。
- 如果命令结果为 `32`，则系统是以 UEFI 模式引导且使用 32 位 IA32 UEFI，虽然其受支持，但引导加载程序只能使用 systemd-boot 和 GRUB。
- 如果命令结果为`No such file or directory`，则系统可能是以 BIOS 模式（或 CSM 模式，这两种模式通常出现在老旧的电脑或**未经配置的虚拟机**上）引导。

如果系统没有以您想要的模式（UEFI 或 BIOS）引导启动，请您参考自己的计算机或主板说明书。

连接到互联网

要在 Live 环境中配置网络连接，请遵循以下步骤：

1. 确保系统已经列出并启用了网络接口，用 ip-link(8) 检查：

```
    # ip link
```

1. 对于无线局域网（Wi-Fi）和无线广域网（WWAN），请确保网卡未被 rfkill 禁用。

2. 连接到网络：
3. 有线以太网——连接网线。
4. WiFi——使用 iwctl 认证无线网络。
5. 移动宽带调制解调器（移动网卡） - 使用 mmcli（英语：mmcli） 连接到移动网络。
6. 配置网络连接：
7. DHCP：对于有线以太网、无线局域网（WLAN）和无线广域网（WWAN）网络接口来说，动态 IP 地址和 DNS 服务器分配（由 systemd-networkd 和 systemd-resolved 提供功能）能够开箱即用。
8. 静态 IP 地址：按照网络配置#静态 IP 地址进行操作。
9. 用 ping 检查网络连接：

```
# ping ping.archlinux.org
```

注意：默认情况下，安装映像在启动时已经预先配置好并启用了 systemd-networkd、systemd-resolved、iwd 和 ModemManager（英语：ModemManager）。但在已经安装完成了的系统之中并非如此。

更新系统时间

为确保软件包签名校验成功以及防止 TLS 证书错误，Live 系统需要准确的时间，为此 systemd-timesyncd 默认启用，也就是说当系统已经建立互联网连接后，系统时间将自动同步。

使用 timedatectl(1) 确保系统时间是同步的：

```
# timedatectl
```

建立硬盘分区

系统如果识别到计算机的内置硬盘、U盘或者移动硬盘等类型磁盘，就会将其分配为一个块设备，如 `/dev/sda`、`/dev/nvme0n1` 或 `/dev/mmcblk0`。可以使用 lsblk 或者 fdisk 等工具查看：

```
# fdisk -l    #此处为小写字母l
```

结果中以 `rom`、`loop` 或者 `airootfs` 结尾的设备可以被忽略。结果中以 `rpmb`、`boot0` 或者 `boot1` 结尾的 `mmcblk*` 设备也可以被忽略。

注意：如果您想要的磁盘没有显示出来，请确保您的磁盘控制器未处于 RAID 模式。

提示：在分区之前，请您检查 NVMe 驱动器和先进格式化硬盘是否使用了最佳逻辑扇区大小。需要注意的是，更改逻辑扇区大小后，可能会导致在 Windows 系统中出现兼容性问题。

对于一个选定的设备，以下分区是**必需**的：

- 一个根分区（挂载在根目录）`/`；

- 要在 UEFI 模式中启动，还需要一个 EFI 系统分区。

请使用分区工具（fdisk 、parted、cfdisk 等等）修改分区表。例如：

```
# cfdisk /dev/要被分区的磁盘
```

注意：

- 花时间规划一个长期的分区方案，以避免将来需要有风险和复杂的转换或重新分区。
- 如果您想为 LVM、系统加密 或 RAID 创建任何堆叠块设备，请立即执行。
- 如果要从中启动的磁盘已经有一个 EFI 分区，请不要创建另一个 EFI 分区，而是使用现有分区。
- Swap 交换空间可以在 Swap#交换文件 上为支持它的文件系统设置。或者，你也可以在安装完毕后配置基于 zram 的 swap 以避免使用基于磁盘的 swap。

分区方案示例

<table class="wikitable"><caption>对于 UEFI 与 GPT 分区表的磁盘分区方案</caption><tbody><tr><th>已安装系统上的挂载点</th><th>分区</th><th>分区类型</th><th>注释</th><th>建议大小</th></tr><tr><td><code>/boot</code>¹</td><td><code>/dev/efi_system_partition</code></td><td>EFI System</td><td>EFI 系统分区</td><td>1 GiB</td></tr><tr><td><code>[SWAP]</code></td><td><code>/dev/swap_partition</code></td><td>Linux swap</td><td>交换空间</td><td>至少 4 GiB</td></tr><tr><td><code>/</code></td><td><code>/dev/root_partition</code></td><td>Linux root (x86-64)</td><td>根目录</td><td>设备剩余空间，至少 23-32 GiB</td></tr></tbody></table>

1. 如果使用的引导加载程序能够从根磁盘卷中加载内核和 initramfs 映像，则可以使用其他挂载点（例如 `/efi`）。请您参阅引导加载程序中的警告部分。

<table class="wikitable"><caption>对于传统 BIOS 与 MBR 分区表的磁盘分区方案</caption><tbody><tr><th>已安装系统上的挂载点</th><th>分区</th><th>分区类型</th><th>建议大小</th></tr><tr><td><code>[SWAP]</code></td><td><code>/dev/swap_partition</code></td><td>Linux swap (交换空间)</td><td>至少 4GiB</td></tr><tr><td><code>/</code></td><td><code>/dev/

root_partition</code></td><td>Linux</td><td>设备剩余空间，至少 23-32GiB</td></tr></tbody></table>

另请参阅布局示例。

格式化分区

创建分区后，必须使用合适的文件系统对每个新创建的分区进行格式化。详情请参阅文件系统#创建文件系统。

注意：对于堆叠式块设备（stacked block devices）请使用恰当的块设备路径替换下文中的 `/dev/**_partition*` 处。

例如，要在根分区 `/dev/*root_partition*` 上创建一个 Ext4 文件系统，请运行：

```
# mkfs.ext4 /dev/root_partition（根分区）
```

如果你要创建一个 EFI 系统分区，使用 mkfs.fat(8) 将其格式化为 Fat32。

警告：只有在分区步骤中创建 EFI 系统分区时才需要格式化。如果这个磁盘上已经有一个 EFI 系统分区了，将它重新格式化会破坏其他已安装操作系统的引导加载程序。

```
# mkfs.fat -F 32 /dev/efi_system_partition（EFI 系统分区）
```

如果创建了交换分区，请使用 mkswap(8) 将其初始化：

```
# mkswap /dev/swap_partition（交换空间分区）
```

挂载分区

将根磁盘卷挂载到 `/mnt`，例如：

```
# mount /dev/root_partition（根分区） /mnt
```

然后使用 mkdir(1) 在 `/mnt` 下创建任何剩余的挂载点（例如，为`/boot` 而创建`/mnt/boot`），并按相应的层级顺序挂载相应的磁盘卷。

提示：使用 `--mkdir` 选项运行 mount(8) 来创建指定的挂载点。或者，先使用 mkdir(1) 创建挂载点再挂载。

注意：挂载分区一定要遵循顺序，先挂载根（root）分区（到 `/mnt`），再挂载引导（boot）分区（到 `/mnt/boot` 或 `/mnt/efi`，如果单独分出来了的话），最后再挂载其他分区。否则您可能遇到安装完成后无法启动系统的问题。参见 en:Talk:Installation guide#Clarify root mount。

对于 UEFI 系统，挂载 EFI 系统分区，例如：

```
# mount --mkdir /dev/efi_system_partition /mnt/boot
```

如果创建了交换空间卷，请使用 swapon(8) 启用它：

```
# swapon /dev/swap_partition（交换空间分区）
```

稍后 genfstab 将自动检测挂载的文件系统和交换空间。

开始安装系统

选择镜像站

需安装的软件包会从文件 `/etc/pacman.d/mirrorlist` 中所列的镜像站下载。下载软件包时，列表中越靠前的镜像站会拥有越高的优先级。

在从 LiveCD 启动的系统中，所有未注释的 HTTPS 镜像站都会启用，位于顶端的全球镜像站对大多用户来说应当足够快，但仍**请您检查 `/etc/pacman.d/mirrorlist` 文件**，看看列出的镜像站的顺序是否合适。如果不合适，可以手动编辑文件，将离您所处地理位置最近的镜像移到文件的头部，同时也应该考虑一些其他的评判标准。此外，您可使用 reflector 根据几种不同的评判标准来创建新的 mirrorlist 文件。

如果 `/etc/pacman.d/mirrorlist` 文件中没有合适的镜像站，可以手动从 archlinux 官方网站的镜像站列表下载一份。这需要您所在的网络能够正常访问 Arch Linux 官方网站。例如，使用 curl(1) 下载位于中国大陆的 HTTPS 镜像站：

```
# curl -L 'https://archlinux.org/mirrorlist/?country=CN&protocol=https' -
o /etc/pacman.d/mirrorlist
```

或者，也可以通过安装 pacman-mirrorlist包 来获取按国家分列的原始镜像列表。这么做的缺点是，pacman 仍然会使用当前配置中的镜像站来下载软件包数据库和 pacman-mirrorlist包，这可能会很慢。在挑选了能用的镜像之后，可以执行

```
# pacman -Sy pacman-mirrorlist
```

再将 `/etc/pacman.d/mirrorlist.pacnew` 复制到 `/etc/pacman.d/mirrorlist` 并进行编辑。

或者，如果您记得想要使用的镜像站的 URL，那么可以手动编辑 `/etc/pacman.d/mirrorlist` 文件并手动输入 URL。

这个文件接下来还会被 *pacstrap* 拷贝到新系统里，所以请您确保设置正确。

安装必需的软件包

除 `/etc/pacman.d/mirrorlist` 之外的配置不会从 Live 环境传递到安装的系统中。因此，若您在 Live 环境的`/etc/pacman.conf` 里启用了非官方用户仓库（如 archlinuxcn 仓库），请在新安装的系统手动启用之。base包 是唯一的必需软件包，其不包含来自 Live 系统的所有工具，因此需要安装其他的软件包。

请着重关注并按需选择下述软件包：

- 您选择的内核包。

- CPU 微码更新——amd-ucode包 或 intel-ucode包——用于硬件错误和安全修复
- 未包含在 linux-firmware包 中的额外必需固件（如用于内建音频的 sof-firmware包、用于 Marvell 无线的 linux-firmware-marvell包 以及博通无线网卡的固件包)。
- 文件系统的用户空间工具（比如 XFS 和 Btrfs 文件系统对应的管理工具），可用于创建和管理文件系统以及 fsck。
- 访问和管理 RAID 或 LVM 分区的工具。
- 联网所需要的程序（例如网络管理器或 DHCP 客户端，Wi-Fi 认证软件和移动宽带连接所需的 ModemManager（英语：ModemManager），以及部分 USB 无线网卡需要的 usb_modeswitch包）。
- GPG 密钥环（如 archlinuxcn-keyringCNRepo，若启用了某些非官方用户仓库。）
- 控制台文本编辑器（如 nano、vim 和 emacs包）以便从控制台编辑配置文件（如点文件）。
- 访问 man 和 info 文档的工具：man-db包, man-pages包 和 texinfo包。
- 引导加载程序。请根据系统引导模式，选择合适的引导程序（如 GRUB 或 Systemd-boot），稍后参考后文的#安装引导程序将其安装到新系统中。

文件 pkglist.x86_64.txt 中包含 Live 系统安装的软件包列表。

要安装您选择的其他软件包或软件包组，请将它们的名字追加到下面的 pacstrap(8) 命令后（以空格分隔），或者也可以在 Chroot 进入新系统后使用 pacman 手动安装。

例如，一个包含 Linux 内核和常见硬件固件的基本安装：

```
# pacstrap -K /mnt base linux linux-firmware
```

注意：有时，由于安装映像中的软件包版本过旧，会导致在运行 *pacstrap* 的时候无法正常安装软件包（提示为文件签名损坏）。 请参阅 Pacman/软件包签名#手动更新密钥环 中提到的解决方案。 如果您启用了中文社区仓库或其他非官方用户仓库，也不要忘了一同更新其密钥环。

提示：

- 使用 pacstrap 安装的软件包只需保证系统能够启动即可，其他软件包可以之后再行安装或替换。
- 在虚拟机或容器)中安装时，可以不需要微码和固件包。容器中还可以不需要内核包。
- 参见 Pacman 以增强您的 pacman 使用体验。

配置系统

生成 fstab 文件

为使需要的文件系统（如启动目录 `/boot`）在启动时被自动挂载，请使用 genfstab(8) 生成一个具有持久化块设备命名的 fstab 文件。例如，使用 `-U` 选项来通过 UUID 指定文件系统：

```
# genfstab -U /mnt > /mnt/etc/fstab
```

强烈建议在执行完以上命令后，检查一下生成的 `/mnt/etc/fstab` 文件是否正确。如果有问题，最好在现在手动修改。对于熟练者，可以自行编辑 `/mnt/etc/fstab` 文件。

可以手动修改 `/mnt/etc/fstab` 添加挂载选项（参见 mount(8) § FILESYSTEM-INDEPENDENT_MOUNT_OPTIONS 和 ext4(5) § MOUNT_OPTIONS）。例如，给固态硬盘的 ext4 文件系统添加 `discard` 选项以启用持续性 TRIM。

警告：在启用 TRIM 功能前，请确保 SSD 支持 TRIM，否则可能导致数据丢失！

chroot 到新安装的系统

接下来的步骤需要像启动到新安装的系统一样直接与其环境、工具和配置进行交互，请 chroot 到新安装的系统：

```
# arch-chroot -S /mnt
```

注意：由于部分 systemd 工具如 hostnamectl(1)、localectl(1) 和 timedatectl(1) 需要有效的 dbus 连接，因此无法在 chroot(1) 内使用这些工具。

提示：此处使用的是[arch-chroot](../../02_系统管理与内核/系统恢复/Chroot.md#使用_arch-chroot)而不是直接使用[chroot](../../02_系统管理与内核/系统恢复/Chroot.md#使用_chroot)，注意不要输错了。

设置时间和时区

为顺应人类习惯（如显示正确的当地时间及处理夏令时），请设置时区：

```
# ln -sf /usr/share/zoneinfo/地区名/城市名 /etc/localtime
```

提示：例如，在中国大陆需要将时区设置为北京时间，那么请运行 `# ln -sf /usr/share/zoneinfo/Asia/Shanghai /etc/localtime`。时区名称是上海而非北京，是因为上海是该时区内人口最多的城市。

然后运行 hwclock(8) 以生成 `/etc/adjtime`：

```
# hwclock --systohc
```

这个命令假定已设置硬件时间为 UTC 时间。详细信息请查看系统时间#时间标准。

为了防止时钟漂移并确保时间准确，请手动启用 NTP（网络时间协议，Network Time Protocol）客户端（例如 systemd-timesyncd）设置时间同步。

区域和本地化设置

程序和库如果需要本地化文本，都依赖区域设置，后者明确规定了地域、货币、时区日期的格式、字符排列方式和其他本地化标准。

需要设置这两个文件：`locale.gen` 与 `locale.conf`。

编辑 `/etc/locale.gen`，然后取消掉 `en_US.UTF-8 UTF-8` 和其他需要的 UTF-8 区域设置前的注释（**#**）。

接着执行 `locale-gen` 以生成 locale 信息：

```
# locale-gen
```

然后创建 locale.conf(5) 文件，并编辑设定 LANG 变量，比如：

```
/etc/locale.conf
LANG=en_US.UTF-8
```

另外对于中文用户：

提示：

- 用户可以设置自己的 locale，详情请参阅 在用户会话中覆盖系统区域设置 或 设置当前区域；
- 将系统 locale 设置为 `en_US.UTF-8`，系统的日志就会用英文显示，这样更容易判断和处理问题；
- 也可以设置为 `en_GB.UTF-8`［英语（英国）］或 `en_SG.UTF-8`［英语（新加坡）］，附带以下特点：
- 进入桌面环境后以 24 小时制显示时间；
- LibreOffice 等办公软件的纸张尺寸会默认为 `A4` 而非 `Letter(US)`；
- 可尽量避免不必要且可能造成处理麻烦的英制单位。
- 设置的 LANG 变量需与 locale 设置一致，否则会出现以下错误：

```
::Cannot set LC_CTYPE to default locale: No such file or directory
```

注意：如果在此处设置为中文 locale，这可能会导致 tty 上中文显示为方块（因为 TTY 下没有 CJK 字体）。如果您主要使用桌面环境而非 TTY，则不用在意这个提醒。您也可以参照简体中文本地化#终端中文支持

如果需要修改#控制台键盘布局和字体，可编辑 vconsole.conf(5) 使其长期生效，以德语键盘（qwertz）为例：

```
/etc/vconsole.conf
KEYMAP=de-latin1
```

注意：如果键盘为标准美式键盘（中国最常见的键盘布局），请勿照抄如上设置。

网络配置

为您的系统设置一个固定且易于辨识的名称（在联网环境中尤其有用），请创建 hostname 文件：

```
/etc/hostname
```

主机名

提示：请参阅 RFC 1178 以获取一些关于为计算机取名的建议。如 hostname(7) 所述，其必须包含 1-63 个字符，仅使用小写的 a-z、0-9 以及 -，但不得以 - 开头。

请接着完成新安装的环境的网络配置，配置过程中可能需要安装合适的网络管理软件。

警告：请按上述网络配置指引正确配置好网络后再重新启动，否则系统重新启动后可能无法连接网络（不过可以用 LiveCD 重新进入 arch-chroot 进行配置）。例如在虚拟机软件 VirtualBox 安装并使用桥接模式时就需要配置 DHCP 。

关于 initramfs

通常不需要自己创建新的 *initramfs*，因为在执行 *pacstrap* 时已经安装 linux包，这时已经运行过 mkinitcpio 了。

如果是 LVM（英语：Install Arch Linux on LVM#Adding mkinitcpio hooks）、系统加密或 RAID 等分区配置，请修改 mkinitcpio.conf 并用以下命令重新创建 Initramfs。此外，如果您更改了默认的控制台键盘映射，只需要重新创建 initramfs 即可：

```
# mkinitcpio -P
```

设置 root 密码

使用以下命令为 root 用户设置一个安全的密码以便进行管理操作：

```
# passwd
```

提示：建议为 root 用户选择一个强密码并保护其安全。

安装引导程序

需要正确安装 Linux 引导加载程序，才能在安装后启动系统。您应当已在前文安装了一个引导加载程序软件包，请参阅其 wiki 页面以配置它。

警告：这是安装的最后一步也是关键的一步，请点击上述链接并按指引正确安装好引导加载程序后再重新启动。否则计算机重新启动后将无法正常进入 Arch Linux 系统。

重新启动计算机

输入 `exit` 或按 `Ctrl+d` 退出 chroot 环境。

可选用 `umount -R /mnt` 手动卸载被挂载的分区：这有助于发现任何“繁忙”的分区，并通过 fuser(1) 查找原因。

最后，通过执行 `reboot` 重启系统，*systemd* 将自动卸载仍然挂载的任何分区。这时候不要忘记移除安装介质，然后使用 root 账户登录到新系统。

提示：若系统出现问题，可尝试重新进入Chroot 环境以补充、完善安装和配置，或查看系统日志以定位、修复问题。

安装后的工作

创建非特权账户、图形用户界面的安装、声音管理、触摸板支持等系统管理教程和后期工作参见建议阅读及其子页面。

感兴趣的各类程序，请参见应用程序列表及其子页面。

还建议新用户阅读给新用户的关于如何不去弄坏 Arch Linux 系统的建议一文。

3.2 Ventoy

难度 入门 | **篇幅** 3.6 KB | **所属** 安装过程

本节将讲到

- 安装
- 使用方法
- 锦囊妙计
- 参见

翻译状态：

- 本文（或部分内容）译自 Ventoy，最近一次同步于 2024-07-16，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

Ventoy 是一个免费且开源的引导工具，您可以在驱动器分区上复写 EFI，IMG，ISO，VHD(x) 或者 WIM 文件，而后选择、引导。

支持绝大多数的操作系统（Windows/WinPE/Linux/ChromeOS/Unix/VMware/Xen 等）。参见全表。

安装

安装 ventoy-binAUR 或 ventoyAUR 软件包。

使用方法

警告：Ventoy 安装时会擦除驱动器上所有数据。

安装媒介有三个实用工具：

1. `/opt/ventoy/Ventoy2Disk.sh`：从命令行运行的 shell 脚本。
2. `/opt/ventoy/VentoyGUI.x86_64`：图形化程序。若程序不能以 root 权限启动，需要 `xauth` 或类似程序以提升权限。
3. 用网络浏览器打开 `file:///opt/ventoy/WebUI/index.html`。

这些实用工具同样可以用来升级驱动器上的 Ventoy 安装。

Ventoy 在驱动器上创建两个分区。默认名 Ventoy 和 VTOYEFI。Ventoy 分区用于储存可引导镜像（iso 文件），以及其他数据。VTOYEFI 存储 Ventoy 的二进制文件。

欲添加要引导的镜像，挂载 Ventoy 分区，将镜像复制过去。

提示：您可以将镜像复制到 Ventoy 分区任意目录下，它们没有必要放在根目录下或放一起。

无安装的使用

如果您只是想要创建一个多引导的驱动器，您仅需在 Github releases 页面下载预构建的可执行文件。您需要 dosfstools包 以安装，因为 vfat 分区是由 *mkfs.vfat* 创建的。核对 sha256 校验和并解压后，您可以运行不带参数的 shell 脚本来显示附带的 CLI（终端用户接口） 帮助页面：

```
# ./ventoy-VERSION/Ventoy2Disk.sh
```

然后，您可以执行相似过程以将驱动器升级到未来版本。下载新版文件，校验、解压并执行：

```
# ./ventoy-NEWER-VERSION/Ventoy2Disk.sh
```

遵从提供的帮助信息。

对此处提到的 shell 脚本有一个替代品，包括一个 GTK/qt图形化用户接口，以及一个 web 用户接口。

锦囊妙计

如何查看已安装的版本?

用上文提及的 `Ventoy2Disk.sh` 实用工具，启动更新过程 (`-u`)，但别确认更新：确认更新前，它会告诉你当前版本。

校验文件

如上游文档所述，有一个内置的校验实用工具可以校验文件。

参见

- Ventoy 文档
- Wikipedia:Ventoy

3.3 Archinstall

难度 入门 | **篇幅** 3.7 KB | **所属** 安装过程

本节将讲到

- 运行安装程序

- 参见

archinstall 是一个用于自动化 Arch Linux 安装过程的帮助库。它和其它的预配置安装程序一起打包，作为“向导”安装程序。

本文不讨论将 archinstall 作为 Python 库使用的情况。这种情况请参看官方文档。

警告：

- *archinstall* 以纯文本形式存储所有用户和（辅助）磁盘加密的密码。
- *archinstall* 的默认配置与安装指南不同。如使用 archinstall 安装系统出现问题的话，请在反馈中注明，并提供 `/var/log/archinstall/install.log`。
- 如果选择单独的 `/home` 分区，*archinstall* 创建的根分区会小于推荐的 23-32 GiB。

运行安装程序

首先，按照安装指南#安装前的准备中的启动到 Live 环境操作。archinstall包 包是 live 镜像的一部分，因此可以直接运行：

```
# archinstall
```

向导安装程序将执行多个步骤，并要求用户输入，参见官方文档。

警告：将 root 密码留空会禁用 root 账户，使用 sudo 来提升权限。这可能会导致你将自己锁定在系统之外，因此可能不需要这样做。请参阅Sudo#禁止 root 登录。

注意：本安装程序可使用 systemd-networkd 配置目标系统的有线网络，或把ISO中的网络配置复制到目标系统中。也就是说，如果你使用 iwd#iwctl 配置无线网络的话，包括网络登录密码在内的配置信息会被复制到目标系统中。与此同时，安装程序也会将安装介质里的有线网络配置复制到目标系统中。

附加软件包也可以通过在 `Write additional packages to install` 选项后输入软件包名称安装。建议在此处或安装完成后的 chroot 环境中安装必需的驱动软件包，以免安装成功后无法正常进入系统。

安装完成后会提示是否进入 chroot 环境，通过 chroot 可以在进入系统前进行额外的配置。

配置文件

archinstall 包含一些配置文件，也就是在基本系统安装完成后，安装的一系列软件包，以及一些预配置文件。

警告：配置文件是专门为 *archinstall* 编写的，不受软件包维护者支持。在使用前请仔细查看每个配置文件的细节。

参见

- python-archinstall: guided

3.4 Genfstab

难度 入门 | **篇幅** 3.9 KB | **所属** 安装过程

本节将讲到

- 安装
- 用法
- 参见

本文或本章节可能需要合并到fstab#Tips and tricks。

附注： There is not enough content for a separate article. The genfstab(8) man page provides everything besides the warnings and examples.（在 Talk:Genfstab 中讨论）

翻译状态：

- 本文（或部分内容）译自 Genfstab，最近一次同步于 2024-7-30，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

genfstab 是一个 Bash 脚本，用于自动检测给定挂载点下的所有挂载，然后将其输出重定向到一个文件中，通常是 `/etc/fstab`。

安装

Arch 安装介质中默认包含该程序，也可作为 arch-install-scripts包 软件包的一部分安装到已安装的系统中。

该工具还有一个独立的分支，可以在其他发行版上使用，你可以在此处找到它。

用法

您可以使用以下命令获取当前的挂载的列表：

```
$ genfstab /
```

脚本支持通过内核描述符、设备/分区标签或设备/分区 UUID 查找挂载。默认情况下，它将输出 `kernel descriptor`（kernel descriptor 为 `/dev/*xxx*`），你可以使用 `-L`, `-t PARTLABEL`, `-U` 或 `-t PARTUID` 分别表示文件系统标签、GPT 分区标签、文件系统 UUID 或 GPT 分区 UUID。

警告：块设备的内核名称描述符不是持久的，每次启动都会更改，因此不应在配置文件（包括 `/etc/fstab`）中使用。

更常见的使用情况是为 chroot 获取一个 fstab，为此你需要执行类似下面的操作：

```
# mount /dev/sda3 /mnt
```

```
# mount --mkdir /dev/sda1 /mnt/efi
$ genfstab -U /mnt
# /dev/sda3
UUID=185aebd2-ce76-47dd-baf4-5ad0a80fa963       /               ext4
rw,noatime      0 1

# /dev/sda1
UUID=E5C7-6DD7          /efi        vfat
rw,relatime,fmask=0077,dmask=0077,codepage=437,iocharset=ascii,shortname=mixed,utf8,err
ors=remount-ro   0 2
```

在这种情况下，`genfstab` 会显示 `/mnt` 下的挂载，并按照设备 `UUID` 列出。

提示：请注意，我们将设备 `sda3` 挂载到了 `/mnt` 中，但 genfstab 却将其显示为主根挂载点 `/`，这是因为它将给定的挂载点视为根挂载点。

通常情况下，您希望将输出重定向到一个文件，这可以通过以下方法实现：

```
# genfstab -U /mnt >> /mnt/etc/fstab
```

警告：

- 确保在覆盖现有 fstab 之前创建了一个备份。
- 注意保存 fstab 文件的位置，例如，如果要为 chroot 创建该文件，则不要覆盖主安装上的文件。

参见

- genfstab(8)

3.5 无盘系统

难度 进阶 | **篇幅** 12.1 KB | **所属** 安装过程

本节将讲到

- 服务端配置
- 无盘系统配置
- 配置无盘系统
- 无盘机启动
- 参阅

引用自 Gentoo Wiki:无盘节点

: *一台无盘机器是一台没有任何诸如硬盘、软盘驱动器或CD-ROM等常见引导设备的PC。无盘节点引导自网络，并需要一台服务器来提供当作本地硬盘来使用的存储空间。*

服务端配置

首先我们应当安装如下组件:

- 用于向无盘节点分配IP地址的 DHCP 服务端。
- 用于分发启动镜像的 TFTP 服务端 （所有PXE镜像必需）。

- 选择 (NFS 或 NBD) 中的一种作为存放和传输无盘节点的 Arch 安装副本的网络存储服务。

注意： dnsmasq包 能够同时兼任 DHCP 和 TFTP 服务器。更多信息请见 Dnsmasq。

DHCP

安装并配置 ISC dhcp包:

```
/etc/dhcpd.conf
allow booting;
allow bootp;

authoritative;

option domain-name-servers 10.0.0.1;

option architecture code 93 = unsigned integer 16;

group {
    next-server 10.0.0.1;

    if option architecture = 00:07 {
        filename "/grub/x86_64-efi/core.efi";
    } else {
        filename "/grub/i386-pc/core.0";
    }

    subnet 10.0.0.0 netmask 255.255.255.0 {
        option routers 10.0.0.1;
        range 10.0.0.128 10.0.0.254;
    }
}
```

注意：`next-server` 应为 TFTP 服务端监听的IP地址; 你需要酌情更改配置中的地址以匹配你已有的网络环境。

RFC 4578 定义了"客户机架构类型（Client System Architecture Type）" 的 dhcp 参数。在如上配置中，如果 PXE 客户机（暨无盘系统）请求了 x86_64-efi 镜像 (类型 0x7) 服务端将返回对应的文件, 否则自动回落到 legacy 镜像。这样，同一网络中的 UEFI 和 legacy BIOS 均能正常启动。

然后启动 ISC DHCP systemd 服务。

TFTP

TFTP 用于向无盘系统传输 启动器、内核以及 initramfs。

将 TFTP 根目录设置为 `/srv/arch/boot`。参阅 Tftpd server 配置安装并配置。

网络存储

NFS 与 NBD 的主要区别是，尽管两者均允许你为多个无盘系统使用同一份安装完毕的 Arch Linux，在使用 NBD 时 (由于其直接与文件系统交互的本质) 你将必需使用 `CopyOnWrite` 的模式, 并将导致在无盘系统断开连接时丢弃所有改动。然而在一些场景中，这是一项不可多得的优势。

NFS

在服务端安装 nfs-utils包。

你需要将 Arch 安装副本的根目录加入 NFS 的共享（“导出”）列表:

```
/etc/exports
/srv        *(rw,fsid=0,no_root_squash,no_subtree_check)
/srv/arch   *(rw,no_root_squash,no_subtree_check)
```

然后启动 NFS 服务: `rpc-idmapd rpc-mountd`.

NBD

安装并配置 nbd包。

```
/etc/nbd-server/config
[generic]
    user = nbd
    group = nbd
[arch]
    exportname = /srv/arch.img
    copyonwrite = false
```

注意：

- 如果你需要多台无盘系统同时启动改镜像，请将 `copyonwrite` 设为 `true`；详情请参阅 nbd-server(5)。
- 可能需要使用 chown 将 exportname 目录的所有权赋予 `nbd` 用户。

启动 `nbd` systemd 服务。

无盘系统配置

接下来我们将在服务器端的子目录中创建一个完整的 Arch Linux 安装副本。启动无盘机时，其将从 DHCP 服务器获取一个 IP 地址，而后通过 PXE 启动并挂载这份安装副本作为根目录。

安装目录创建

创建一个至少 1GB 的 sparse file 并格式化为 btrfs。 (你当然也可以使用其他块设备如实体硬盘或 LVM)。

注意：对于 NBD，创建一个单独的文件系统是必须的；但对于 NFS 这是可选的，故可酌情忽略或跳过。

```
# truncate -s 1G /srv/arch.img
# mkfs.btrfs /srv/arch.img
# export root=/srv/arch
# mkdir -p "$root"
# mount -o loop,compress=lzo /srv/arch.img "$root"
```

Bootstrap 安装

安装 devtools包 和 arch-install-scripts包, 然后运行 `pacstrap` 以为无盘系统安装必须的软件包:

```
# pacstrap -d "$root" base linux linux-firmware mkinitcpio-nfs-utils nfs-utils
```

注意：在任何配置中 mkinitcpio-nfs-utils包 都必须安装：它将提供早期启动过程中所需的 `ipconfig`。

现在应当生成好 initramfs。

NFS

要使 NFSv4 挂载正常工作，需要轻微改动 `net` hook (这不被默认的`net` hook：`nfsmount` 所支持)。

```
# sed s/nfsmount/mount.nfs4/ "$root/usr/lib/initcpio/hooks/net" >
"$root/usr/lib/initcpio/hooks/netnfs4"
# cp $root/usr/lib/initcpio/install/net{,nfs4}
```

此时需要备份 `net` 以防止其在 mkinitcpio-nfs-utils包 升级时被覆盖.

Edit `$root/etc/mkinitcpio.conf` and add `nfsv4` to `MODULES`, `netnfs4` to `HOOKS`, and `/usr/bin/mount.nfs4` to `BINARIES`.

然后 chroot 到安装副本的根目录并执行 *mkinitcpio*:

```
# arch-chroot "$root" mkinitcpio -p linux
```

NBD

在无盘系统的 Arch 安装副本中安装 mkinitcpio-nbdAUR。使用 makepkg 构建并安装它：

```
# pacman --root "$root" --dbpath "$root/var/lib/pacman" -U mkinitcpio-nbd-0.4-1-
any.pkg.tar.xz
```

然后你需要将 `HOOKS` array 中 `net` 之后加入 `nbd`； `net` 将负责配置网络, 但只要不在 kernel 行中指定，它不会尝试 NFS 挂载。

配置无盘系统

在上述步骤完成后，你仍需要时情况完成主机名，时区，区域设置，键盘布局等其他 Installation guide 提及的配置步骤。

启动器

GRUB

尽管没有详细文档, GRUB 支持从 PXE 加载：

```
# pacman --root "$root" --dbpath "$root/var/lib/pacman" -S grub
```

使用 `grub-mknetdir` 在目标系统（无盘系统的 Arch 安装副本）中创建一个 grub prefix：

```
# arch-chroot "$root" grub-mknetdir --net-directory=/boot --subdir=grub
```

幸运的是， `grub-mknetdir` 为所有已编译/安装的目标创建 prefix, 且 grub包 的维护者在同一包中提供了两种架构, 所以 `grub-mknetdir` 仅需执行一次。

现在我们创建一个 GRUB 配置:

```
# vim "$root/boot/grub/grub.cfg"
menuentry "Arch Linux" {
    linux /vmlinuz-linux quiet add_efi_memmap ip=:::::eth0:dhcp nfsroot=10.0.0.1:/arch
    initrd /initramfs-linux.img
```

```
}
```

GRUB 黑魔法将会自动设置 `set root=(tftp,10.0.0.1)`，这样内核和 initramfs 将自动通过 TFTP 传输过来，不过当你有其他非 TFTP 的启动项时，你仍希望显式声明这些配置。

注意：根据需要依照PXELINUX修改你的于NBD相关的内核行配置。

PXELINUX

PXELINUX 由 syslinux包 提供，详见 PXELINUX。

额外的挂载点

NBD 根

late boot 时你需要将根分区的文件系统切换为可读写（`rw`）并且启动 `compress=lzo` 以获得相较于 NFS 可观的性能提升。

```
# vim "$root/etc/fstab"
/dev/nbd0   /   btrfs   rw,noatime,compress=lzo   0 0
```

程序状态目录

**本文或本章节的事实准确性存在争议。**

原因： 当 /var/log/journal 为 tmpfs 或不存在时，systemd 默认不会进行持久日志记录。（在 Talk:无盘系统 中讨论）

你可以将 `/var/log` 挂载为例如 tmpfs 的文件系统，这样多台主机的日志将不会意外混杂在一起。对于 `/var/spool/cups` 亦可实行此操作，这样 20 个使用相同 spool 的 CUPS 不会互相打架然后执行进行 1498 份打印工作并在一夜之间吃掉整摞纸（或更糟：墨盒）。

```
# vim "$root/etc/fstab"
tmpfs   /var/log        tmpfs     nodev,nosuid    0 0
tmpfs   /var/spool/cups tmpfs     nodev,nosuid    0 0
```

对于有某种唯一状态或数据库的软件，最好的配置是为每台无盘主机配置唯一的储存目录。例，如果你想要运行 puppet 你可以简单地在 puppet 的 systemd unit file 中使用 `%H` 标识符:

```
# vim "$root/etc/systemd/system/puppetagent.service"
[Unit]
Description=Puppet agent
Wants=basic.target
After=basic.target network.target

[Service]
Type=forking
PIDFile=/run/puppet/agent.pid
ExecStartPre=/usr/bin/install -d -o puppet -m 755 /run/puppet
ExecStart=/usr/bin/puppet agent --vardir=/var/lib/puppet-%H --ssldir=/etc/puppet/ssl-%H

[Install]
WantedBy=multi-user.target
```

Puppet-agent 将会在 vardir 或 ssldir 不存在时自动创建之。

如果上述方法都不适用，你还可以选择使用 systemd generator 来针对当前主机创建一个 mount unit，不幸的是 *%H* 标识符无法使用。

无盘机启动

NBD

**本文或本章节的事实准确性存在争议。**

原因： 在服务端启动了CopyOnWrite特性时，所有无盘系统应当均有效地只读挂载了根分区所在的文件系统，所以理论上在NBD服务器上读写挂载是安全的。（在 Talk:无盘系统 中讨论）

当使用 NBD 作为网络存储服务时，你需要无盘系统启动前确保卸载（`umount`）安装副本所在的块设备。

这一特性使得内核更新的过程十分有趣：当客户机开机时你不能挂载它所使用的文件系统, 这意味着你需要用一个单独的内核来执行内核更新。

首先从客户端安装副本复制 `$root/boot` 到你的 tftp 根目录 (通常即 `/srv/boot`).

```
# cp -r "$root/boot" /srv/boot
```

然后在客户机启动前卸载 `$root`。

```
# umount "$root"
```

注意：如上配置中如需升级内核,你需要在开始内核更新前于无盘机中使用 NFS 在 fstab 中挂载 `/srv/boot`，或是在客户机于NBD断开后手动挂载客户机文件系统。

参阅

- kernel.org: Mounting the root filesystem via NFS (nfsroot)
- syslinux.org: pxelinux FAQ

3.6 在可移动设备上安装 Arch Linux

难度 进阶 | **篇幅** 12.2 KB | **所属** 安装过程

本节将讲到

- 安装
- 引导加载程序配置
- 提示和技巧
- 另请参阅

翻译状态：

- 本文（或部分内容）译自 Install Arch Linux on a removable medium，最近一次同步于 2025-04-25，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

本文介绍如何在可移动磁盘（如 U 盘）上进行常规的 Arch 安装，结果是与正常安装到硬盘一样的持久化安装。与此相对的是制作 LiveUSB（U 盘安装介质）。

安装

注意：请阅读分区#分区方案以决定需要的磁盘空间。

取决于您的操作系统，可通过多种方式在可移动磁盘上安装 Arch：

- 若您有另一台 Linux 计算机（不必是 Arch），您可按照从现有 Linux 发行版安装 Arch Linux 安装。
- Arch Linux CD/USB 可用于将 Arch 安装到可移动磁盘，只需从 CD/USB 启动并按照安装指南进行安装。如果从 Live USB 启动，则无法安装到同一个可移动磁盘。
- 若您运行 Windows 或 macOS，则可以下载 VirtualBox，安装 VirtualBox 扩展，将您的可移动磁盘连接到 Linux 虚拟机（已安装好或从 live ISO 启动），然后按照安装指南安装到连接的磁盘。
- 可以在您启动安装环境的 USB 设备上安装 Arch Linux，但是您不能在安装过程中重启。若如此做，您可能需要再次制作安装介质。

安装时的微调

- 在创建初始 RAM disk 前，修改 `/etc/mkinitcpio.conf`，将 `block` 和 `keyboard` 钩子移动到 `autodetect` 钩子之前。这样才能在分别需要早期用户空间中不同模块的系统上启动。
- 如果您希望能在其他操作系统中使用安装完的磁盘，可以创建使用适当文件系统的分区（如 NTFS 或 exFAT）。注意，数据分区可能需要是设备的第一个分区，因为 Windows 会假定可移动磁盘仅有一个分区，并将自动挂载 EFI 系统分区。记得安装 dosfstools包 和 ntfs-3g包。网上的一些工具可能可以翻转可移动磁盘的可移动媒体位（RMB）。这将使得操作系统把它看作外置硬盘，这样您就可以随意使用您选择的分区方式。
- 如果您的 Arch 所在的可移动磁盘需要有两家制造商的微码，请安装 amd-ucode包 和 intel-ucode包。见 Microcode#加载微码。
- 确保在生成的 `/etc/fstab` 文件内删除了主机上的文件系统的相关条目。启动时，若相关条目的文件系统不存在，Arch 会等待该文件系统较长时间。

警告：不是所有的 U 盘都可以翻转可移动媒体位（RMB），并且使用不兼容您的设备的软件可能会造成损坏。**不建议**尝试翻转RMB。

引导加载程序配置

注意：Systemd 会自动探测根文件系统 `/`，因此不必在 `/etc/fstab` 中为其建立条目。

GRUB

按照 GRUB#BIOS 系统和 GRUB#UEFI 系统中的说明为 BIOS 和 UEFI 启动安装 GRUB：

```
# grub-install --target=i386-pc /dev/sdX --recheck
# grub-install --target=x86_64-efi --efi-directory=esp --removable --recheck
```

注意： 此处安装grub时必须使用--removable参数，该参数可以使efi被安装于uefi固件的默认检索位置，对于多设备启动至关重要，缺失几乎必然导致其他设备下无法检索引导程序。 再次检查 device map，即使 `/boot/grub/device.map` 已经存在。每当您添加/删除计算机中的磁盘时都应使用这一选项。

Syslinux

使用UUID:

```
LABEL Arch
        MENU LABEL Arch Linux
        LINUX ../vmlinuz-linux
        APPEND root=UUID=3a9f8929-627b-4667-9db4-388c4eaaf9fa rw
        INITRD ../initramfs-linux.img
```

rEFInd

```
menuentry "Arch Linux" {
   icon     /EFI/BOOT/icons/os_arch.png
   volume   5028fa50-0079-4c40-b240-abfaf28693ea
   loader   /path/to/vmlinux_image
   initrd   /path/to/initramfs
   options  "root=PARTUUID=5028fa50-0079-4c40-b240-abfaf28693ea rw"
}
```

请参阅 rEFInd#手动启动项以获取更多有关创建手动启动项的信息。

安装 rEFInd 时，您还必须使用 `--usedefault /dev/sd*XY*` 参数。

提示和技巧

在多个机器上使用您的设备

显卡驱动

**本文或本章节的事实准确性存在争议。**

原因： 唯一剩下的专有驱动 NVIDIA 运行良好，而 nouveau 自从阻止 reclocking 后就一直在退步。（在 Talk:在可移动设备上安装 Arch Linux 中讨论）

注意：

- 对于这种安装，不建议使用专有的显卡驱动。
- 在决定安装 xf86-video-intel包 和 xf86-video-nouveau包 前请先阅读 Intel 图形处理器#安装 和 Nouveau#安装 中分别关于此二者的注释。

为支持最常见的 GPU，请安装 xf86-video-vesa包、xf86-video-ati包、xf86-video-amdgpu包 和 xf86-video-fbdev包。

声卡驱动

为支持最常见的声卡，请安装 sof-firmware包 和 alsa-firmware包。请参阅 ALSA 以获取更多关于配置音频设备的信息。

块设备持久化命名

建议在 fstab 和启动管理器配置中使用 UUID。详情请参阅块设备持久化命名。

或者，您可以自行创建 udev 规则为您的磁盘创建符号链接，然后在 fstab 和启动管理器配置中使用。详情请参阅 udev#设置静态设备名。

内核参数

**本文或本章节的事实准确性存在争议。**

原因： 该问题是否依然存在？未链接到错误报告。（在 Talk:在可移动设备上安装 Arch Linux 中讨论）

您可能由于各种各样的原因希望禁用 KMS，例如在使用 Intel 显卡时遇到空白屏幕、显示器“无信号”错误等。要禁用 KMS，添加 `nomodeset` 内核参数。详情请参阅内核参数。

警告：KMS 禁用时某些 Xorg 驱动将无法工作。请在您的驱动对应的 wiki 页面上查找详细信息。特别是 Nouveau，它需要 KMS 才能判断正确的分辨率。如果您已添加 `nomodeset`，那么您可能需要在使用 Nvidia 显卡的设备上手动调整分辨率。详情请参阅 Xrandr。

兼容性

使用 fallback 镜像可获得最大的兼容性。

最小化磁盘访问

当安装到总写入量有限的设备（如 U 盘、SD 卡等）上时，减少写入量可以延长设备的寿命。这也可以减小写入慢对性能的影响。

- 强烈建议在选择文件系统前阅读性能优化#减少磁盘读写。概括起来，对于基于闪存的磁盘（如 U 盘、SD 卡等），无日志的 ext4 应该就可以了，它可通过 `mkfs.ext4 -O "^has_journal" /dev/sdXX` 创建。使用无日志的文件系统的明显的缺点就是在 ungraceful dismount 时会丢失数据。不过要意识到闪存的总写入量是有限的，而日志会占用其中的一部分。由于同样的原因，最好也不要想着 swap 分区了。注意这并不影响安装到可移动硬盘。
- 您可能想将 systemd 日志配置为在内存中存储日志。为此，可以创建一个配置文件：

```
/etc/systemd/journald.conf.d/usbstick.conf
[Journal]
Storage=volatile
RuntimeMaxUse=30M
```

- 要在网页浏览器和其他不写入关键数据的应用程序中禁用 `fsync` 和相关的系统调用，可以使用来自 libeatmydata包 的 `eatmydata` 命令：

```
$ eatmydata firefox
```

UI 响应速度

高 I/O 负载可能会导致 UI 冻结，特别是在慢磁盘上。更改 I/O 调度器，或者切换到默认使用不同调度器的内核，可以极大地改进 UI 的响应速度。例如 BFQ 可以改进 UI 的响应速度（它在 linux-zen包 和 linux-ckAUR 上默认启用）。

详见性能优化#调度算法。

另请参阅

- Arch Linux USB\[失效链接 2022-09-18 ⓘ\] - c-magyar 关于创建持久化 Live USB 安装的精彩内容。
- archuseriso - 用于构建 Arch Linux Live ISO 镜像的配置文件。具有持久存储和加密功能的可启动 USB 驱动器创建工具。

3.7 多引导 USB 设备

难度 进阶 | **篇幅** 13.9 KB | **所属** 安装过程

本节将讲到

- GRUB引导器及loopback devices实现
- Syslinux和memdisk实现
- 自动制作工具实现(开源软件)
- 更多参考

多引导USB启动盘可以从单个设备引导多个ISO镜像文件。ISO镜像文件可以不需要解压复制到U盘上就可以启动。以下有多种可用的方案，但不一定兼容所有ISO镜像。

GRUB引导器及loopback devices实现

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：multiple style issues（在Talk:多引导 USB 设备讨论）

优点:

- 仅需要一个分区
- 所有ISO文件镜像可在同一个文件夹下被检测到
- 易于添加或移除ISO镜像文件

缺点:

- 并非所有镜像兼容
- ISO文件原本的启动菜单不会显示
- 找到可用的启动项可能并不容易

准备

在USB设备上创建至少一个分区和一个GRUB支持的文件系统。分区的大小取决与要存储在其中的ISO文件大小和引导程序所计划需要的空间。

安装 GRUB引导器

简单安装

挂载U盘:

```
# mount /dev/sdXY /mnt
```

U盘上创建 /boot文件夹:

```
# mkdir /mnt/boot
```

安装GRUB到U盘:

```
# grub-install --target=i386-pc --recheck --boot-directory=/mnt/boot /dev/sdX
```

如果以UEFI启动镜像, 您需要按UEFI方式安装GRUB:

```
# grub-install --target=x86_64-efi --removable --boot-directory=/mnt/boot --efi-directory=/mnt
```

对于UEFI, 引导分区必须是FAT32格式, 且启动分区必须是MBR分区表中的第一个分区.

混合 UEFI GPT + BIOS GPT/MBR 启动

以下有用的配置可以帮助您创建一份可以在任意地方启动的通用U盘。首先，您必须在您的设备上创建一份 GPT 分区。您至少需要以下三个分区：

1. 一个BIOS启动分区（gdisk代号编码 `EF02`），该分区至少需要1 MiB的大小。
2. 一个EFI系统分区（gdisk代号编码`EF00`且它使用FAT32文件系统），该分区至少需要50 MiB的大小。
3. 您自己的数据分区（使用GRUB支持的文件系统)，该分区的大小是由您硬盘的剩余空间容量来决定的。

接下来您必须创建一个MBR混合分区表。如果没有它，一个仅BIOS MBR引导启动的电脑将无法启动。因为它无法寻找到硬盘里的实例以启动。

以下是一个使用gdisk创建MBR混合分区表的例子：

```
# gdisk /dev/sdX
```

```
Command (? for help): r
Recovery/transformation command (? for help): h

WARNING! Hybrid MBRs are flaky and dangerous! If you decide not to use one,
just hit the Enter key at the below prompt and your MBR partition table will
be untouched.

Type from one to three GPT partition numbers, separated by spaces, to be added to the
hybrid MBR, in sequence: 1 2 3
Place EFI GPT (0xEE) partition first in MBR (good for GRUB)? (Y/N): N

Creating entry for GPT partition #1 (MBR partition #1)
Enter an MBR hex code (default EF):
Set the bootable flag? (Y/N): N

Creating entry for GPT partition #2 (MBR partition #2)
Enter an MBR hex code (default EF):
Set the bootable flag? (Y/N): N

Creating entry for GPT partition #3 (MBR partition #3)
Enter an MBR hex code (default 83):
Set the bootable flag? (Y/N): Y

Recovery/transformation command (? for help): x
Expert command (? for help): h
Expert command (? for help): w

Final checks complete. About to write GPT data. THIS WILL OVERWRITE EXISTING
PARTITIONS!!

Do you want to proceed? (Y/N): Y
```

注意 Hybird MBR 只可以输入三个分区。

不要忘了格式化这些分区：

```
# mkfs.fat -F32 /dev/sdX2
# mkfs.ext4 /dev/sdX3
```

您现在可以安装GRUB以同时支持EFI + GPT引导以及BIOS + GPT/MBR引导。

GRUB配置(--boot-directory）将会被保持在同一个地方。

首先，您需要挂载您USB驱动器上的EFI系统分区和数据分区。

以下是如何挂载的例子：

```
# mount /dev/sdX3 /mnt
# mkdir -p /mnt/boot/EFI
# mount /dev/sdX2 /mnt/boot/EFI
```

然后，您就可以在UEFI上安装GRUB：

在大多数案例中，`EFI_MOUNTPOINT`将会对应至您所挂载的U盘上的`/mnt/boot/EFI`这个子目录中。`DATA_MOUNTPOINT`是数据分区的挂载目录。在本例中就是 sdX3 的挂载目录，即`/mnt`。

```
# grub-install --target=x86_64-efi --recheck --removable --efi-
directory=/EFI_MOUNTPOINT --boot-directory=/DATA_MOUNTPOINT/boot
```

在BIOS上安装GRUB：

```
# grub-install --target=i386-pc --recheck --boot-
directory=/DATA_MOUNTPOINT/boot /dev/sdX
```

作为后备处理，您也可以把GRUB安装进您的MBR启动分区里面。

```
# grub-install --target=i386-pc --recheck --boot-
directory=/DATA_MOUNTPOINT/boot /dev/sdX3
```

配置 GRUB

参考模板

有些git项目中提供了一些预先存在的GRUB配置文件，一个优秀且通用的 grub.cfg 文件可以用来按需加载其他引导项，只有在指定的ISO文件（或包含它们的文件夹）存在于驱动器上时才会显示它们。

Multiboot USB: https://github.com/hackerncoder/multibootusb

GLIM (GRUB2 Live ISO Multiboot): https://github.com/thias/glim

手动配置

通过手动编辑`grub.cfg`以代替自动生成它可以更容易的达到制作多启动USB驱动盘的目的。或者，在`/etc/grub.d/40_custom`或`/mnt/boot/grub/custom.cfg`中做出如下改动并通过grub-mkconfig生成`/mnt/boot/grub/grub.cfg`。

As it is recommend to use a persistent name instead of `/dev/sd*xY*` to identify the partition on the USB drive where the image files are located, define a variable for convenience to hold the value. If the ISO images are on the same partition as GRUB, use the following to read the UUID at boot time:

```
/mnt/boot/grub/grub.cfg
# path to the partition holding ISO images (using UUID)
probe -u $root --set=rootuuid
set imgdevpath="/dev/disk/by-uuid/$rootuuid"
```

Or specify the UUID explicitly:

```
/mnt/boot/grub/grub.cfg
# path to the partition holding ISO images (using UUID)
set imgdevpath="/dev/disk/by-uuid/UUID_value"
```

Alternatively, use the device label instead of UUID:

```
/mnt/boot/grub/grub.cfg
# path to the partition holding ISO images (using labels)
set imgdevpath="/dev/disk/by-label/label_value"
```

The necessary UUID or label can be found using `lsblk -f`. Do not use the same label as the Arch ISO for the USB device, otherwise the boot process will fail.

To complete the configuration, a boot entry for each ISO image has to be added below this header, see the next section for examples.

启动项

It is assumed that the ISO images are stored in the `boot/iso/` directory on the same filesystem where GRUB is installed. Otherwise it would be necessary to prefix the path to ISO file with device identification when using the `loopback` command,

for example `loopback loop **(hd1,2)**$isofile`. As this identification of devices is not persistent, it is not used in the examples in this section.

One can use persistent block device naming like so. Replace the UUID according to your ISO filesystem UUID.

```
# define globally (i.e outside any menuentry)
insmod search_fs_uuid
search --no-floppy --set=isopart --fs-uuid 123-456
# later use inside each menuentry instead
loopback loop ($isopart)$isofile
```

提示： For a list of kernel parameters, see the kernel's command-line parameter documentation. For more examples of boot entries, see the GRUB upstream documentation or the documentation for the distribution you wish to boot.

Arch Linux monthly release

Also see archiso.

```
menuentry '[loopback]archlinux-2020.10.01-x86_64.iso' {
	set isofile='/boot/iso/archlinux-2020.10.01-x86_64.iso'
	loopback loop $isofile
	linux (loop)/arch/boot/x86_64/vmlinuz-linux img_dev=$imgdevpath
img_loop=$isofile earlymodules=loop
	initrd (loop)/arch/boot/intel-ucode.img (loop)/arch/boot/amd-ucode.img
(loop)/arch/boot/x86_64/initramfs-linux.img
}
```

See README.bootparams for archiso options supported in kernel command line.

Memtest86+

Memtest86+\[损坏的链接：无效的章节\]包含在每月构建的 ISO中。

```
menuentry '[loopback]archlinux-2020.10.01-x86_64.iso' {
	set isofile='/boot/iso/archlinux-2020.10.01-x86_64.iso'
	loopback loop $isofile
	linux16 (loop)/arch/boot/memtest
}
```

archboot

参见 Archboot Homepage。

```
menuentry '[loopback]archlinux-2014.11-1-archboot' {
	set isofile='/boot/iso/archlinux-2014.11-1-archboot.iso'
	loopback loop $isofile
	linux (loop)/boot/vmlinuz_x86_64 iso_loop_dev=$imgdevpath iso_loop_path=$isofile
	initrd (loop)/boot/initramfs_x86_64.img
}
```

Syslinux和memdisk实现

Using the memdisk module, the ISO image is loaded into memory, and its bootloader is loaded. Make sure that the system that will boot this USB drive has sufficient amount of memory for the image file and running operating system.

准备

Make sure that the USB drive is properly partitioned and that there is a partition with file system supported by Syslinux, for example fat32 or ext4. Then install Syslinux to this partition, see Syslinux#Installation on BIOS.

安装 memdisk 模块

The memdisk module was not installed during Syslinux installation, it has to be installed manually. Mount the partition where Syslinux is installed to `/mnt/` and copy the memdisk module to the same directory where Syslinux is installed:

```
# cp /usr/lib/syslinux/bios/memdisk /mnt/boot/syslinux/
```

配置

在将ISO文件复制至USB驱动盘之后，编辑Syslinux配置文件并为ISO镜像创立菜单条目，最基础的条目看起来像这样：

```
boot/syslinux/syslinux.cfg
LABEL some_label
    LINUX memdisk
    INITRD /path/to/image.iso
    APPEND iso
```

参见memdisk on Syslinux wiki 以了解更多调试选项。

自动制作工具实现(开源软件)

- **GRUB2 Live ISO Multiboot (GLIM)** — A set of GRUB configuration files to turn a VFAT formatted USB memory stick with GNU/Linux distribution ISO images into a multiboot USB drive.

: https://github.com/thias/glim || 未被打包？在 AUR 里搜索

- **liveusb-builder** — 一个为GNU/Linux发行版创建多启动U盘的脚本套件

: https://github.com/mytbk/liveusb-builder || liveusb-builder-gitAUR

- **MultiBootUSB** — 一个具有CLI和GUI界面的跨平台Python软件，可添加或删除多个Linux LIVE映像环境到U盘。

: https://github.com/mbusb/multibootusb || multibootusbAUR

- **MultiSystem** — 一个图形化工具，可添加或删除多个Linux LIVE映像环境到U盘。

: http://liveusb.info/dotclear/ || multisystemAUR

- **Ventoy** — 一个开源工具，为ISO/WIM/IMG/VHD(x)/EFI文件创建可引导的USB驱动器。不需要一遍又一遍格式化磁盘，只需要复制文件到U盘即可启动。

: https://www.ventoy.net/ || ventoy-binAUR

更多参考

- GRUB:
- https://help.ubuntu.com/community/Grub2/ISOBoot/Examples
- https://help.ubuntu.com/community/Grub2/ISOBoot

- GRUB Live ISO Multiboot - GRUB configurations for booting ISO images
- Syslinux:
- Boot an ISO image

3.8 从现有 Linux 发行版安装 Arch Linux

难度 进阶 | **篇幅** 14.0 KB | **所属** 安装过程

本节将讲到

- 备份和准备
- 从一个正在运行 Arch Linux 的主机
- 从一个运行另一个 Linux 发行版的主机
- 重启前要检查的
- 不使用 LiveCD 替换当前系统

本指南给出了从当前 Linux 发行版安装 Arch Linux 所需的准备步骤。 准备完成后的安装参考 安装指南。

从当前 Linux 发行版安装 Arch Linux 对以下情形有所帮助：

- 远程安装 Arch Linux，如一台（虚拟的）根服务器
- 无需 LiveCD 替换当前 Linux 发行版（参见#不使用 LiveCD 替换当前系统）
- 创建基于 Arch Linux 的新 Linux 发行版或 LiveCD
- 创建 Arch Linux 的 chroot 环境，如可为 Docker 基础容器创建
- 为无盘机器准备 rootfs-over-NFS

这些准备步骤的目的在于为搭建一个 arch-install-scripts包（如 pacstrap(8) 和 arch-chroot(8)）可运行的环境。

如果当前系统是 Arch Linux，这个目的可通过在当前系统安装 arch-install-scripts包 达成。如果当前系统不是 Arch Linux，可以构建基于 Arch Linux 的 chroot 环境。

注意：本指南要求当前系统能够运行目标 Arch Linux 构架的程序。这意味着当前系统必须是 x86_64 架构。

警告：在进行每一步前，确保你理解你在干什么。这些操作很容易毁坏你的系统或造成数据丢失！

备份和准备

备份你的所有数据，包括邮件、网页服务器等，把所有数据都放在手边。记录下你的服务器配置、hostname 等数据。

你可能会用到的数据：

- IP 地址

- hostname(s)，（注意：根服务器可能是域名提供的一部份，在你删除 `/etc/hosts` 前先检查并备份）
- DNS 服务器（检查 `/etc/resolv.conf`）
- SSH 密钥（如果其它人也要用你的服务器，如果你删除了 SSH 密钥，他们就需要新的密钥了。这些密钥可能来自：Apache、邮件服务器、SSH 服务器和其它服务）
- 硬件信息（比如网卡，参考你的 `/etc/modules.conf`）
- 引导加载程序（Boot Loader）的配置文件。

总之，在本地备份一份原本的 `/etc` 总不会错。

从一个正在运行 Arch Linux 的主机

安装 arch-install-scripts包。

参考 Installation guide#挂载分区来挂载用来安装的文件系统根目录，以及其它必要的挂载点。如果 `/mnt` 文件夹已经被占用，只要新建一个文件夹，比如 `/mnt/install` 用来替代即可。

目前，Arch Linux 可以从头安装，或者作为宿主系统的镜像，这两种选项会在下面说明。

进行全新的 Arch Linux 安装

参考 安装指南#开始安装系统。

你可以跳过 Installation guide#选择镜像，因为主机中应该已经有了合适的镜像列表。

警告：

pacstrap 会使用 **来自宿主机的** `/etc/pacman.conf` 的配置文件以及 pacman 钩子。参见 archlinux/arch-install-scripts#60。 如果你的设置偏离了标准（例如你在你的 pacman 配置中有 `Noextract=`），这也会影响到新安装的系统。

- 如果需要，下载默认的 pacman.conf 并通过 *pacstrap* 的 `-C` 选项来指定这个文件。

提示：

- 要避免重新下载所有软件包，可以参考 pacman/Tips and tricks#在网络上共享pacman缓存。或者使用 *pacstrap* 的 `-c` 选项，来使用宿主系统上的软件包缓存。

创建已有的 Arch 的备份

复制宿主系统的文件系统到新分区，再做一些必要的调整，就可以备份已有的 Arch Linux。

第一步是拷贝宿主文件到新的分区里，可以考虑使用 Rsync#全盘系统备份里展示的方法。

然后，参考 Installation guide#配置系统里的步骤，注意以下几点：

- 可以跳过 Installation guide#时区、Installation guide#本地化和 Installation guide#Root 密码
- 如果改变了文件系统，比如从 ext4 改为 Btrfs，必须进行 Installation guide#Initramfs
- 参考 Installation guide#安装引导程序，务必重新安装引导程序
- 删除 `/etc/machine-id`，并使用 systemd-machine-id-setup(1) 来创建一个新的。

如果镜像 Arch 必须要使用与宿主系统不同的配置，或者要安装到不同的硬件上，考虑进行以下操作：

- 在进行 Installation guide#安装引导程序时，使用目标系统 CPU 的微码
- 如果宿主系统上有于目标系统上不兼容的 Xorg#配置，参考 Moving an existing install into (or out of) a virtual machine#Disable any Xorg-related files
- 其它针对目标系统的配置诸如网络和音频。

从一个运行另一个 Linux 发行版的主机

下列是多个可以自动处理大量步骤的工具。具体方法可以参考他们各自主页的相关说明。

- archstrap (Bash)
- digitalocean-debian-to-arch (需要重新分区，针对 DigitalOcean; **未经过PGP 签名验证**)
- image-bootstrap (Python; **未经过PGP 签名验证**)
- vps2arch (Bash; **未经过PGP 签名验证**)

以下是介绍手动处理的办法，具体思路要么是直接在宿主系统上运行 pacman，要么是在宿主系统里运行一个 Arch 系统，这个嵌套系统位于 chroot 中。

在宿主系统上运行 pacman

Pacman 可以在大部分 Linux 发行版上编译运行，可以直接用来在宿主系统上创建 Arch Linux。最近的发行版上 arch-install-scripts 应该可以顺利运行。

一些发行版官方提供 *pacman* 和 / 或者 *arch-install-scripts* 的软件包。截止 2020 六月，Void Linux 提供了 *pacman* 软件包，Apline Linux 和 Fedora 提供了 *pacman* 和 *arch-install-scripts* 的软件包。

创建 chroot

从镜像站下载 bootstrap 镜像到 `/tmp`。 从下载页下载 bootstrap 的签名文件并把它放在与镜像相同的文件夹下。不要从镜像站下载它。 使用 GnuPG 验证。

解压 tarball：

```
# tar xf /path-to-bootstrap-image/archlinux-bootstrap-x86_64.tar.zst --numeric-owner
```

留意最后的 `--numeric-owner` 选项。你当前的 Linux 系统使用的 UID 和 GID 可能与 Arch Linux 不同，这个选项可以保留解压出文件的 UID 和 GID。

然后编辑 `/tmp/root.x86_64/etc/pacman.d/mirrorlist` 来选择软件仓库服务器。

进入 chroot：

- 若安装了4或更高版本的 bash，并且 *unshare* 支持 `--fork --pid` 选项 (util-linux 2.24 or later):

```
# /tmp/root.x86_64/bin/arch-chroot /tmp/root.x86_64/
```

- 若无，执行：

:```

```
### 使用 chroot 环境

初始环境非常基础（没有 [nano](https://archlinux.org/packages/?name=nano)包 或者 [lvm2](https://archlinux.org/packages/?name=lvm2)包），因此，我们得设置好 pacman 来安装必要的软件包。

#### 初始化 pacman 密钥

开始安装前，需要设置 pacman 密钥。执行以下命令：
更多请参见 [pacman/软件包签名#Initializing the keyring](../包管理器/Pacman_软件包签名.md#Initializing_the_keyring)。

#### 下载基本工具

[刷新软件包列表](../../00_关于与社区/关于%20Arch/镜像源.md#强制_pacman_刷新软件包列表)并安装你需要的软件包：[base](https://archlinux.org/packages/?name=base)包、[base-devel](https://archlinux.org/groups/x86_64/base-devel/)包组、[parted](https://archlinux.org/packages/?name=parted)包 等等。

> [!](https://wiki.archlinuxcn.org/wzh/images/8/87/Tango-edit-clear.png)](https://wiki.archlinuxcn.org/wzh/File:Tango-edit-clear.png)**本文或本章节的语言、语法或风格需要改进。参考：[帮助:风格](https://wiki.archlinuxcn.org/wzh/Help:%E9%A3%8E%E6%A0%BC)**
>
```

> **原因：**This error is explained within [arch-chroot(8) § DESCRIPTION](https://man.archlinux.org/man/arch-chroot.8#DESCRIPTION), it should be linked there.（在[Talk:从现有 Linux 发行版安装 Arch Linux](https://wiki.archlinuxcn.org/wiki/Talk:%E4%BB%8E%E7%8E%B0%E6%9C%89_Linux_%E5%8F%91%E8%A1%8C%E7%89%88%E5%AE%89%E8%A3%85_Arch_Linux)讨论）

> **注意：**你在用 pacman 安装软件包时，可能会遇到错误：`error: could not determine cachedir mount point /var/cache/pacman/pkg`。解决办法是在 chroot 前运行：
>
> ```
> # mount --bind directory-to-livecd-or-bootstrap directory-to-livecd-or-bootstrap
> ```
>
> 参考 [FS#46169](https://bugs.archlinux.org/task/46169)。

安装提示

请按照[安装指南](安装指南.md)中的[挂载分区](安装指南.md#挂载分区)和[安装必须的软件](安装指南.md#安装必需的软件包)小节进行安装。

一些宿主系统或配置可能需要额外的步骤，参考下面的章节。

基于 Debian 的当前系统

/dev/shm

在基于 Debian 的当前系统上，`pacstrap` 会发生以下错误：
==> Creating install root at /mnt mount: mount point /mnt/dev/shm is a symbolic link to nowhere ==> ERROR: failed to setup API filesystems in new root

Debian 中，`/dev/shm` 指向 `/run/shm`。而在基于 Arch 的 chroot 中，`/run/shm` 并不存在，因而链接失效。创建 /run/shm 目录可修复此错误：
基于Fedora的当前系统

在基于 Fedora 的宿主系统和 live USBs 上时，当你用 *genfstab* 生成你的 [fstab](../启动过程/Fstab.md) 你可能会遇到问题。移除重复的入口和 `seclabel` 选项（如果有），因为它们是针对 Fedora 的。

重启前要检查的

为得到一个成功的安装，重启前请再次检查一些你的安装的细节。 首先 chroot 到你的新系统然后：

- [创建一个用户并设置密码](../../08_安全与加密/安全/用户和用户组.md#User_management)， 使得你可以通过 *ssh* 登录。这很重要因为从 OpenSSH-7.1p2 开始登录 root 被默认禁用。
- [设置 root 用户密码](安装指南.md#Root_password) 以便之后你可以通过 *su* 切换至 root。
- 安装一个 [ssh](../../03_网络与服务/SSH/安全外壳协议.md) 实现并启用服务端实例使得它开机自动启动。
- 设置[网络配置](../../03_网络与服务/网络/网络配置.md)使得你在启动后能使用网络。
- 设置 [boot loader](../启动过程/Arch%20的启动流程.md) 并配置它使用你之前所挪用的交换分区做为根分区 。 你可能还想让你的 boot loader 能够启动到你的旧系统；对此可复用已存在的 `/boot` 分区来达成这个目的。

不使用 LiveCD 替换当前系统

在硬盘上划分出 ~700 MiB 的空闲空间，如分割交换分区。你可以禁用交换分区并用省下的空间建立新系统。

把旧的交换分区设为新的根分区

检查 `cfdisk`、`/proc/swaps` 或者 `/etc/fstab` 来找到你的交换分区。假设你的硬盘位于 `sda*X*`（`*X*` 是数字）。

执行下面的操作：

禁用交换分区：
在上面建立新的文件系统
(设置 /dev/sdaX ID 为 "Linux" - Hex 83)

创建一个新目录来挂载它
最后，挂载新目录并安装过渡系统。
安装

[安装必要软件包](安装指南.md#Install_essential_packages)和其它让系统联网和在临时文件系统下运行的必需软件包。注意 ~700 MB 的空间限制。当使用 *pacstrap* 安装时，考虑加上 `-c` 选项来避免占满宝贵的空间。

一旦完成安装，修复引导器配置，然后重启进入到新系统并[rsync 整个系统](../../02_系统管理与内核/同步/Rsync.md#全盘系统备份)至主分区。

3.9 Install Arch Linux with Fake RAID

难度 进阶 | **篇幅** 14.1 KB | **所属** 安装过程

本节将讲到

- 什么是"fake RAID"
- 历史
- 备份
- 提纲
- 准备

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

本指南的目的是使用由板上BIOS的RAID控制器所生成的RAID，从而使得GRUB可以从RAID内的Linux和Windows的分区启动。当使用所谓的"fake RAID"或"host RAID"时,硬盘是`/dev/mapper/chipsetName_randomName`而不是`/dev/sdX`.

什么是"fake RAID"

维基:

: *基于操作系统的RAID并不总是保护引导过程,对Windows桌面版本一般的不实用。硬件RAID昂贵且是专有的。为了填补这一缺口，引入了价格便宜的"RAID控制器"，它不包含RAID控制器芯片，但只是一个带特殊的固件和控制器芯片的标准磁盘驱动器。在启动初期，RAID通过固件实现。当加载了保护模式操作系统内核如Linux或现代微软Windows系统，驱动程序接管RAID。*

: *这些控制器被制造商描述为RAID控制器，但很少清楚地告诉购买者，RAID处理的开销是由主机的CPU承担,而不是RAID控制器本身,硬件RAID不存在这样的开销。固件控制器往往只能使用特定的几种硬盘（例如：Intel的Matrix RAID使用SATA硬盘，现代Intel ICH南桥不支持PATA和SCSI；但主板厂商在一些主板的南桥之外实现了RAID控制器）。因为在此之前，“RAID控制器“已经实现了--控制器做了处理，所以这种新技术被技术知识界称为“fake RAID“，即使RAID本身是正确实施。 Adaptec的称他们为“host RAID“。*wikipedia:RAID

参考 Wikipedia:RAID or FakeRaidHowto @ Community Ubuntu Documentation以获得更多的信息。

不考虑术语，通过dmraid包建立的"fake RAID"软RAID是健壮的，提供了一个通过多个磁盘实现的坚实的镜像或条带的数据系统，并只有可以忽略不计开销。 dmraid的对比于mdraid（纯粹的Linux软RAID）提供了如下好处：当出错时，能够在重启之前完全重建一个硬盘。

历史

在Linux 2.4中， ATARAID kernel framework提供了对fake RAID (由BIOS协助的软RAID)的支持. Linux 2.6中，device-mapper framework ，包括其它的如LVM和EVMS, 可以做ATARAID在2.4中做的事.虽然新的代码中处理RAID的I/O仍然在内核中运行时，device-mapper通常是由一个用户空间应用程序配置。很明显，当使用RAID的device-mapper，检测会在用户空间。

Heinz Maulshagen开发了dmraid工具来检测RAID和创建它们的映射.支持的硬件是带BIOS功能的fake RAID IDE/SATA. 常见的如: Promise FastTrak controllers; HighPoint HPT37x; Intel Matrix RAID; Silicon Image Medley; 和 NVIDIA nForce.

备份

警告：使用RAID前请备份您的数据。条带式RAID对硬盘错误是很第敏感的。可以考虑常规的备份或镜像式RAID。**警告!**

提纲

- 准备
- 从安装盘启动
- 加载dmraid
- 执行传统安装
- 安装GRUB

准备

- 在其它机器上打开需要的指南(如 Installation guide)。如果没有其它的机器，打印出来。
- 下载最新的Arch Linux安装镜像.
- 备份所有重要的文件，因为目标分区中的所有文件将被破坏。

配置RAID

警告：如果您的硬盘没有配置RAID并安装了Windows, 切换到"RAID"可能造成Windows启动时的BSOD错误。\[失效链接 2021-05-13 ⓘ\]

- 进入BIOS设置，激活RAID控制器。
- BIOS可能包含配置SATA硬盘的选项如"IDE","AHCI"或者"RAID"; 确认选择了"RAID"。
- 保存并退出BIOS设置。启动时进入RAID设置工具。
- RAID设置工具通常可以通过启动菜单(通常是F8, F10或CTRL+I)或RAID控制器启动时进入。
- 使用RAID设置工具来建立所选的条带或镜像的集。

提示：详情查看主板文档。具体型号可能不同。

从安装盘启动

详情参见Installation guide#Pre-installation。

提示：如果您的显示器支持，考虑在启动时加`vga=795`选项以获得更高的framebuffer分辨率。

加载dmraid

加载device-mapper并寻找RAID:

```
# modprobe dm_mod
# dmraid -ay
# ls -la /dev/mapper/
```

输出例子:

```
/dev/mapper/control             <- 由device-mapper生成;如果存在, device-mapper是工作的。
/dev/mapper/sil_aiageicechah    <- Silicon镜像SATA RAID控制器下的一个RAID集
/dev/mapper/sil_aiageicechah1   <- 该RAID的第一个分区
```

如果只有(`/dev/mapper/control`), 用`lsmod`检查您是否加载了控制芯片的模块。如果加载了,那么dmraid不支持该控制芯片或系统没有RAID集(再检查一下BIOS中的RAID设置)。如果还是正确的，那您只能用software RAID (也就是说您不能用双启动的RAID系统)。

如果你的芯片模块没有加载，加载它，如:

```
# modprobe sata_sil
```

可用的驱动参见`/lib/modules/uname -r/kernel/drivers/ata/`。

测试RAID:

```
# dmraid -tay
```

执行传统安装

切换到**tty2**开始安装:

```
# /arch/setup
```

RAID分区

- 在**Prepare Hard Drive**选择**Manually partition hard drives**因为**Auto-prepare**选项找不到您的RAID。
- 选择OTHER，输入您的RAID路径(如`/dev/mapper/sil_aiageicechah`)。切换到**tty1**检查您的输入。
- 按常规方法分区。

提示：如果计划双启动，这时可以安装其它系统。如安装Windows XP到"C:"，则所有Windows分区前的分区应该改为类型\1B\来在安装windows时隐藏分区.当安装完后再将它们改回为类型\[83\] (Linux)。当然上述过程需重启。

加载文件系统

如果在**Manually configure block devices, filesystems and mountpoints**没有找到新的分区--很可能是这种情况:

- 切换到**tty1**.
- 去除所有device-mapper节点:

```
# dmsetup remove_all
```

- 重新激活新建立的RAID节点:

```
# dmraid -ay
# ls -la /dev/mapper
```

- 切换到**tty2**,重新进入**Manually configure block devices, filesystems and mountpoints**菜单，分区应该就可用了。

安装和配置Archlinux

提示：使用3个终端:一个使用GUI来配置系统,一个使用chroot来安装GRUB,一全使用cfdisk来参考因为RAID盘的名字很奇怪。

- **tty1:** chroot和安装grub
- **tty2:** /arch/setup
- **tty3:** 用cfdisk做输入参考，分区

让程序一直运行，使用时切换。

切换到安装程序(**tty2**)继续:

- 选择包
- 确认标记了安装**dmraid**
- 配置系统
- 在`mkinitcpio.conf`中的MODULES行加入**dm_mod**。如果使用镜像阵列还要加入**dm_mirror**
- 如果需要还要在MODULES行加入芯片驱动模块**chipset_module_driver**
- 在`mkinitcpio.conf`中HOOKS行加入**dmraid**;可加在**sata** but before **filesystems**之后

安装 GRUB

警告：You can normally specify **default saved** instead of a number in `menu.lst` so that the default entry is the entry saved with the command **savedefault**. If you are using dmraid do not use **savedefault** or your array will de-sync and will not let you boot your system.

Please read GRUB for more information about configuring GRUB. Installation is begun by selecting **Install Bootloader** from the Arch installer.

注意：For an unknown reason, the default `menu.lst` will likely be incorrectly populated when installing via fake RAID. Double-check the **root** lines (e.g. `root (hd0,0)`). Additionally, if you did **not** create a separate `/boot` partition, ensure the kernel/initrd paths are correct (e.g. `/boot/vmlinuz-linux` and `/boot/initramfs-linux.img` instead of `/vmlinuz-linux` and `/initramfs-linux.img`.

For example, if you created logical partitions (creating the equivalent of sda5, sda6, sda7, etc.) that were mapped as:

```
  /dev/mapper      |    Linux    GRUB Partition
                   |  Partition      Number
nvidia_fffadgic    |
nvidia_fffadgic5   |    /              4
nvidia_fffadgic6   |    /boot          5
nvidia_fffadgic7   |    /home          6
```

The correct root designation would be **(hd0,5)** in this example.

注意：If you use more than one set of dmraid arrays or multiple Linux distributions installed on different dmraid arrays (for example 2 disks in nvidia_fdaacfde and 2 disks in nvidia_fffadgic and you are installing to the second dmraid array (nvidia_fffadgic)), you will need designate the second array's `/boot` partition as the GRUB root. In the example above, if nvidia_fffadgic was the second dmraid array you were installing to, your root designation would be root **(hd1,5)**.

After saving the configuration file, the GRUB installer will **FAIL**. However it will still copy files to `/boot`. **DO NOT GIVE UP AND REBOOT** -- just follow the directions below:

- Switch to **tty1** and chroot into our installed system:

```
# mount -o bind /dev /mnt/dev
# mount -t proc none /mnt/proc
# mount -t sysfs none /mnt/sys
```

```
# chroot /mnt /bin/bash
```

- Switch to **tty3** and look up the geometry of the RAID set. In order for cfdisk to find the array and provide the proper C H S information, you may need to start cfdisk providing your raid set as the first argument. (i.e. cfdisk /dev/mapper/nvidia_fffadgic):
- The number of **C**ylinders, **H**eads and **S**ectors on the RAID set should be written at the top of the screen inside cfdisk. **Note:** cfdisk shows the information in **H S C** order, but grub requires you to enter the geometry information in **C H S** order.

: Example: `18079 255 63` for a RAID stripe of two 74GB Raptor discs. : Example: `38914 255 63` for a RAID stripe of two 160GB laptop discs.

- GRUB will fail to properly read the drives; the **geometry** command must be used to manually direct GRUB:
- Switch to **tty1**, the chrooted environment.
- Install GRUB on `/dev/mapper/raidSet`:

```
# dmsetup mknodes
# grub --device-map=/dev/null

grub> device (hd0) /dev/mapper/raidSet
grub> geometry (hd0) C H S
```

Exchange **C H S** above with the proper numbers (be aware: they are **not** entered in the same order as they are read from cfdisk).

If geometry is entered properly, GRUB will list partitions found on this RAID set. You can confirm that grub is using the correct geometry and verify the proper grub root device to boot from by using the grub find command. If you have created a separate boot partition, then search for /grub/stage1 with find. If you have no separate boot partition, then search /boot/grub/stage1 with find. Examples:

```
grub> find /grub/stage1       # use when you have a separate boot partition
grub> find /boot/grub/stage1  # use when you have no separate boot partition
```

Grub will report the proper device to designate as the grub root below (i.e. (hd0,0), (hd0,4), etc...) Then, continue to install the bootloader into the Master Boot Record, changing "hd0" to "hd1" if required.

```
grub> root (hd0,0)
grub> setup (hd0)
grub> quit
```

注意：With dmraid >= 1.0.0.rc15-8, partitions are labeled "raidSet**p1**, raidSet**p2**, etc. instead of raidSet**1**, raidSet**2**, etc. If the setup command fails with "error 22: No such partition", temporary symlinks must be created.

The problem is that GRUB still uses an older detection algorithm, and is looking for `/dev/mapper/raidSet1` instead of `/dev/mapper/raidSetp1`.

The solution is to create a symlink from `/dev/mapper/raidSetp1` to `/dev/mapper/raidSet1` (changing the partition number as needed). The simplest way to accomplish this is to:

```
# cd /dev/mapper
# for i in raidSetp*; do ln -s $i ${i/p/}; done
```

Lastly, if you have multiple dmraid devices with multiple sets of arrays set up (say: nvidia_fdaacfde and nvidia_fffadgic), then create the `/boot/grub/device.map` file to help GRUB retain its sanity when working with the arrays. All the file does is map the dmraid device to a traditional hd#. Using these dmraid devices, your device.map file will look like this:

```
(hd0) /dev/mapper/nvidia_fdaacfde
(hd1) /dev/mapper/nvidia_fffadgic
```

And now you are finished with the installation!

```
# reboot
```

Troubleshooting

Booting with degraded array

One drawback of the fake RAID approach on GNU/Linux is that dmraid is currently unable to handle degraded arrays, and will refuse to activate. In this scenario, one must resolve the problem from within another OS (e.g. Windows) or via the BIOS/chipset RAID utility.

Alternatively, if using a mirrored (RAID 1) array, users may temporarily bypass dmraid during the boot process and boot from a single drive:

1. Edit the **kernel** line from the GRUB menu
2. Remove references to dmraid devices (e.g. change `/dev/mapper/raidSet1` to `/dev/sda1`)
3. Append `disablehooks=dmraid` to prevent a kernel panic when dmraid discovers the degraded array
4. Boot the system

3.10 U 盘安装介质

难度 高级 | **篇幅** 27.9 KB | **所属** 安装过程

本节将讲到

- 直接使用 ISO（BIOS 和 UEFI）
- 手动方法
- 使用多引导 USB 设备
- 不推荐的方法
- 故障排除

USB 驱动器，也被称为闪存驱动器、USB记忆棒、U盘等。可以从 ISO 制作 Arch LiveUSB 系统，从 UEFI 或 BIOS 系统上直接启动 Arch Linux 安装环境。由于 / 根目录使用了 Overlayfs，关机后所有的更改都会丢失。这样的系统可用于 Arch Linux 安装、系统维护或者系统恢复。

要在 U 盘上运行完整的 Arch Linux（即能保留设置），请阅读在可移动设备上安装 Arch Linux。如果想将 Arch Linux U 盘当作救援 USB 来用，参见 chroot。

在进行下列任何操作前，先从 https://archlinux.org/download/ 下载 ISO 并验证其完整性。

直接使用 ISO（BIOS 和 UEFI）

警告：这一操作将不可恢复地删除 USB 驱动器上的所有数据，所以在操作前先确保驱动器上没有存放任何重要文件。

注意：如果你想将 ISO 写入到机械硬盘或是固态硬盘上，而不是使用 U 盘或是 SD 卡，请确保硬盘的逻辑扇区大小不大于 2048 字节（ISO 9660 扇区大小），并按此数值进行对齐。这也意味着不能用此方法将 ISO 写入到 4Kn 先进格式化硬盘中。

GNU/Linux

使用基本命令行工具

推荐该方法是因为它简单，且由于这些工具是 coreutils包（由 base包 元软件包引入）的一部分，其通用性也很广。

使用 `ls -l /dev/disk/by-id/usb-*` 找到 U 盘的名字，然后使用 `lsblk` 检查确保其没有被挂载。

将下列命令中的 `/dev/disk/by-id/usb-*My_flash_drive*` 替换为你的 U 盘（例如`/dev/disk/by-id/usb-Kingston_DataTraveler_2.0_408D5C1654FDB471E98BED5C-0:0`）。(不要加上分区编号，也就是不要输入类似 `/dev/disk/by-id/usb-Kingston_DataTraveler_2.0_408D5C1654FDB471E98BED5C-0:0**-part1**` 或者是 `/dev/sdb**1**` 的东西）：

- 使用 cat(1)：

```
# cat path/to/archlinux-version-x86_64.iso > /dev/disk/by-id/usb-My_flash_drive
```

- 使用 cp(1)：

```
# cp path/to/archlinux-version-x86_64.iso /dev/disk/by-id/usb-My_flash_drive
```

- 使用 dd：

```
# dd bs=4M if=path/to/archlinux-version-x86_64.iso of=/dev/disk/by-id/usb-My_flash_drive conv=fsync oflag=direct status=progress
```

- 使用 tee：

```
# tee < path/to/archlinux-version-x86_64.iso > /dev/disk/by-id/usb-My_flash_drive
```

- 使用 pv包：

```
# pv path/to/archlinux-version-x86_64.iso > /dev/disk/by-id/usb-My_flash_drive
```

可参考 和 来对比这些工具并了解它们的大致用法，以及为何 *dd* 可能是最不适用的方法。

提示：

- 执行完命令后，使用根权限执行 `sync` 来确保在移除硬盘前缓冲区数据已完全写入。
- If the UEFI version of the USB's Arch ISO hangs or is unable to load, try repeating the medium creation process on the same USB drive one or more times. If this does not work, you may also try updating your motherboard's firmware.

注意：如果在使用完 Arch ISO 镜像后要 U 盘恢复为空的可用 USB 存储设备，须在重分区和重新格式化 U 盘前使用根权限运行 `wipefs --all /dev/disk/by-id/usb-*My_flash_drive*` 将 ISO 9660 文件系统签名移除。

使用 GNOME Disk Utility

Linux distributions running GNOME can easily make a live CD through nautilus包 and gnome-disk-utility包. Simply right-click on the *.iso* file, and select *Open With Disk Image Writer*. When GNOME Disk Utility opens, specify the flash drive from the *Destination* drop-down menu and click *Start Restoring*.

使用 MultiWriter

gnome-multi-writer包 是一个基于 GTK3 的图形化工具，用于将 ISO 文件同时写入到单个或多个 U 盘中。

使用 Kindd

Kindd 是基于 Qt 的 dd 图形化前端，可通过 kinddAUR 获取。

使用 Popsicle

Popsicle 是由 PopOS 开发团队制作的工具，用于同时向多个 U 盘写入 ISO 文件。它使用 Rust 和 GTK 编写，可通过 popsicleAUR 获取。

使用 SUSE Studio ImageWriter

SUSE Studio ImageWriter 是由 OpenSUSE 开发团队基于 Qt 制作的 Qt，可通过 imagewriterAUR 获取。

使用 xorriso-dd-target

xorriso-dd-target（来自 libisoburn包）是一个 shell 脚本，尝试降低覆盖到错误存储设备的风险。它最安全的模式名为 `-plug_test`。以可以通过 sudo 提升到 root 权限的普通用户为例：

```
$ xorriso-dd-target -with_sudo -plug_test -DO_WRITE -image_file archlinux-version-x86_64.iso
```

更多信息可参考 xorriso-dd-target(1)。

使用 USBImager

USBImager 是一个跨平台的图形化程序，用于将压缩的磁盘镜像写入到 U 盘中并进行验证，并可以创建备份。可通过 usbimagerAUR 获取。

Windows

使用 win32diskimager

win32diskimager 是另一个用于在 Windows 上写入镜像到 USB 或者 SD/CF 卡上的工具。只需选择 ISO 映像和目标 USB 盘符（可能需要先格式化 USB 驱动器后才能分配驱动器号），然后点击“Write”即可。

使用 USBwriter

这一方法和 Linux 下的 `dd` 一样简单，只要下载 Arch Linux 安装镜像，并通过本地管理员权限使用 USBwriter 写入 U 盘即可。

使用 USBImager

USBImager 是一个跨平台的图形化程序，用于将压缩的磁盘镜像写入到 U 盘中并进行验证，并可以创建备份。可通过 usbimagerAUR 获取。

使用 Rufus

Rufus 是一款多用途的 USB ISO 写入工具。它提供了图形化用户界面，并不关心驱动器是否被正确地格式化。

只要简单地选择 Arch Linux ISO、要创建可启动 Arch Linux 的 USB 驱动器，然后点击*开始*即可。

注意：如果USB设备使用默认USB镜像模式无法正确引导，请选用""DD Image 模式""。如果要切换到这个模式，请在“分区方案”下拉菜单中选择“GPT”，点击“开始”后你会看到选项提示框，最后选择“DD Image 模式”

提示：要添加 一个用于持久存储的额外分区 ，请使用滑块选择持久分区的大小。在使用持久分区功能时，请确保在 “分区方案 ”下拉菜单中选择 “MBR”，并在 “目标系统 ”中选择 “BIOS 或 UEFI”，否则硬盘将无法同时用于 BIOS 和 UEFI 启动。

使用 Cygwin

先确保你的 Cygwin 环境包含了 `dd` 包。

将您的镜像文件放在您的 home 目录中：

```
C:\cygwin\home\User\
```

以管理员身份运行 cygwin（以允许 cygwin 访问硬件）。用下列命令写入到您的 USB 设备：

```
dd if=archlinux-version-x86_64.iso of=\\.\x: bs=4M
```

其中 `archlinux-*version*-x86_64.iso` 是 `cygwin` 目录下的 ISO 镜像路径，而 `\\.\*x*`：是你的 U 盘，里面的 `*x*` 是 Windows 为您的 USB 设备指定的盘符，例如 `\\.\d:`。

Cygwin 6.0 版本中可以这样找到正确的分区：

```
cat /proc/partitions
```

然后根据输出信息写入ISO映像，例如：

```
dd if=archlinux-version-x86_64.iso of=/dev/sdb bs=4M
```

使用 dd for Windows

http://www.chrysocome.net/dd 上有基于 GPL 协议的 Windows 版 dd。相比于 Cygwin，它的优势在于需下载量较小。使用方法参考上面的 Cygwin 部分。

首先下载最新版本的 dd for Windows，并把内容解压到 `Downloads` 目录或者别的什么地方。

现在以管理员身份运行*命令提示符*。然后切换目录（`cd`）到 `Downloads` 目录。

如果你的 Arch Linux ISO 在别的地方，你可能需要列出完整路径。为了方便起见，你可能会想把它放在 *dd* 程序的同一目录中。命令的基本格式像这样：

```
# dd if=archlinux-version-x86_64.iso od=\\.\x: bs=4M
```

注意：Windows 的驱动器盘符链接到的是分区。为了选择完整的硬盘，上述命令使用了 *dd for Windows* 提供的 `od` 参数。需要注意，这一参数仅适用于 *dd for Windows*，并不适用于其它 *dd* 实现。

使用 flashnul

flashnul 是一款用于验证和维护闪存（USB 闪存、IDE 闪存、SecureDigital、MMC、MemoryStick、SmartMedia、XD、CompactFlash 等）功能的实用程序。

在命令提示符下，用 `-p` 调用 flashnul，并确定 USB 驱动器的设备索引，例如

```
C:\>flashnul -p
Avaible physical drives:
Avaible logical disks:
C:\
D:\
E:\
```

确定了正确的设备后，您就可以将镜像写入硬盘，通过用`-L`设备索引和镜像的路径来调用 flashnul，例如：

```
C:\>flashnul E: -L path\to\archlinux-version-x86_64.iso
```

只要你已经确定要开始写入数据，就输入 yes ，稍等片刻等待写入完成。如果出现了拒绝访问的错误，请关闭所有打开的资源管理器窗口。

注意：请以管理员身份打开命令提示符，否则 flashnul 将无法将镜像作为块设备打开，只能通过 windows 提供的驱动器句柄写入。

macOS

使用 macOS dd

首先确认设备，打开 `/Applications/Utilities/Terminal` 然后通过下面命令查看所有存储设备：

```
$ diskutil list
```

USB 设备应该类似于 `/dev/disk2 (external, physical)`。通过名称和大小确认设备是否正确，在后面的命令中用其替换掉 `/dev/disk*X*`。

macOS 通常会自动挂载 USB 设备，在使用 `dd` 前需要将其卸载（不是弹出）。打开 *Terminal*，然后输入：

```
$ diskutil unmountDisk /dev/diskX
```

然后将 ISO 镜像复制到设备上：

{{Note|

- BSD 的 `dd` 衍生版本（包括 macOS 预装的）使用小写 `m` 后缀。这与本文其它地方使用的 GNU 版本 `dd` 不同。
- `disk` 前的 `r}` 意味着使用 `raw` 模式，可以大幅提升传输速率。

`- 在新版 macOS 中，dd 命令支持使用 status=progress 来显示进度。你也可以通过下方描述手动显示进度。`

```
# dd if=path/to/archlinux-version-x86_64.iso of=/dev/rdiskX bs=1m
```

这一命令将静默执行。如果要显示进度，可以按下 {{ic|Ctrl+t} 来传递 SIGINFO 信号。注意，这里的 `disk*X*` 不应包含 `s1` 后缀，否则 U 盘将仅支持 UEFI 模式启动。完成后，macOS 可能抱怨 *The disk you inserted was not readable by this computer*，选择忽略就行。启动盘就做好了。

使用 USBImager

USBImager 是一个跨平台的图形化程序，用于将压缩的磁盘镜像写入到 U 盘中并进行验证，并可以创建备份。可通过 usbimagerAUR 获取。

Android

使用 EtchDroid

EtchDroid 是一个安卓平台上的系统镜像烧录器，从安卓 5 开始不再需要 root 权限。如果遇到任何问题，可查看 GitHub 上游。

要创建 Arch Linux 安装盘，需先将 ISO 镜像下载到安卓设备上。将 U 盘插入到设备，有需要的话可以使用 USB-OTG 转接器。接下来打开 EtchDroid，选择 *Flash raw image*，选择 Arch ISO 文件，然后选择 U 盘。最后授予 USB API 权限并确认。

刷入镜像时请将手机放在桌子上：很多 USB-OTG 转接器的连接都不太稳，很容易不小心被拔掉。

手动方法

BIOS 和 UEFI

GNU/Linux

这种方法比通过 `dd` 直接写入镜像复杂，但能够保持 U 盘可用于数据存储（ISO 被装在分了区的设备的特定分区里而无需改变其他分区）。

注意：接下来会用 `/dev/disk/by-id/usb-*My_flash_drive*-part*n*` 指代目标分区，请根据实际情况修改命令中的 `*My_flash_drive*` 和 `*n*`。

- 预先在 `/dev/disk/by-id/usb-*My_flash_drive*` 创建分区表。
- 预先在设备上创建一个分区，`/dev/disk/by-id/usb-*My_flash_drive*-part*n*` 分区必须格式化为 FAT32。
- 挂载 U 盘上的 FAT32 文件系统，并将 ISO 镜像中的内容解压到其中。

```
# mount /dev/disk/by-id/usb-My_flash_drive-partn /mnt
# bsdtar -x -f archlinux-version-x86_64.iso -C /mnt
```

在 BIOS 平台上启动时需指定文件所在的卷，默认情况下会使用 UUID `*YYYY-mm-dd-HH-MM-SS*-00`（即 UTC 时区的镜像发布时间）。可以将 `/mnt/boot/syslinux/archiso_sys-linux.cfg` 中的 `archisodevice=` 参数替换为对应设备的持久化设备识别符，例如：`archisodevice=UUID=*YOUR-UUID*`

注意：错误的 `archisolabel` 或 `archisodevice` 将导致无法从创建好的介质中启动。

用于 BIOS 的 Syslinux 文件已被拷入到 `/mnt/boot/syslinux/`。卸载 FAT 文件系统，安装 syslinux包 和 mtools包 包，并运行以下命令来使分区变为可启动：

```
# umount /mnt
# syslinux --directory boot/syslinux --install /dev/disk/by-id/usb-My_flash_drive-partn
# dd bs=440 count=1 conv=notrunc if=/usr/lib/syslinux/bios/mbr.bin of=/dev/disk/by-id/usb-My_flash_drive
```

注意：

- 如果 `/dev/disk/by-id/usb-*My_flash_drive*` 使用 GUID 分区表，需将 `mbr.bin` 替换为 `gptmbr.bin`。详细信息可参考 Syslinux#手动安装。
- 对于 MBR 分区表，需要设定“boot”标记。详细信息可参考 Syslinux#MBR_分区表。

Windows

注意：

- 对于手动方法而言，不要使用任何**可启动 USB 创建工具**制作 UEFI 启动 U 盘，也不要用 *dd for Windows* 将 ISO 镜像写入 U 盘。

- 下列命令假定 U 盘盘符为 `*X*:`。
- Windows 使用反斜杠 `\` 作为路径分隔符，这一规则同样用于下列命令中。
- 所有命令都应该在**以管理员身份运行**的命令提示符内执行。
- `>` 表示 Windows 命令提示符。
- 使用 Rufus USB partitioner对 U 盘进行分区和格式化。在分区类型一项选择 **MBR for BIOS and UEFI**，文件系统选择 **FAT32**。反选“Create a bootable disk using ISO image”和“创建扩展的标签和图标文件”两项。
- 如果用的是官方 ISO 镜像（Archiso），需修改 U 盘的**卷标**到你想要的值。这一步骤也可以在“分区并格式化”这一步中通过 Rufus 来完成。
- 使用 7-Zip 将 ISO 中的文件解压（与解压 ZIP 压缩包类似）到 U 盘中。
- 编辑 `*X*:\boot\syslinux\archiso_sys-linux.cfg`，将所有 `archisodevice=UUID=*YYYY-mm-dd-HH-MM-SS*-00` 替换为 `archisodevice=LABEl=*YOUR_LABEL*`，其中 `*YOUR_LABEL*` 为上一步设定的卷标。
- 从 https://www.kernel.org/pub/linux/utils/boot/syslinux/ 下载官方 Syslinux 6.xx 二进制文件（ZIP 打包文件）并解压。Syslinux 的版本应与 ISO 镜像所使用的版本一致。
- 在命令提示符内，以管理员身份运行以下命令：

```
> cd bios\
> for /r %Y in (*.c32) do copy "%Y" "X:\boot\syslinux\" /y
> copy mbr\*.bin X:\boot\syslinux\ /y
```

- 通过以下命令将 Syslinux 安装到 U 盘（64 位系统则使用 `win64\syslinux64.exe`）：

```
> cd bios\
> win32\syslinux.exe -d /boot/syslinux -i -a -m X:
```

注意：

- 仅复制文件并不能使得可以从 U 盘启动；最后一条命令将 Syslinux 的文件安装到了 U 盘的 VBR 分区，在 MBR 分区表中将该分区设置为“活跃/启动”，并将 MBR 启动代码写入到了 U 盘中。
- `-d` 参数需要使用正斜杠作为路径分隔符，与类 unix 系统上类似。

仅 BIOS

GNU/Linux

制作 USB-ZIP 盘

某些旧的 BIOS 设备仅支持从 USB-ZIP 盘启动。该方法让你可以继续从 USB 硬盘上启动。

- 安装 syslinux包 和 mtools包。
- 通过 `ls /dev/disk/by-id/usb-*` 命令找到你的 U 盘。

- 输入 `mkdiskimage -4 /dev/disk/by-id/usb-*My_flash_drive* 0 64 32`，完成需要一段时间。

接下来继续按手动方法进行操作。由于 ZIP 硬盘的工作方式，分区将使用 `/dev/disk/by-id/usb-*My_flash_drive*-part4`。

注意：不要将盘格式化为 FAT32 格式，必须使用 FAT16。

仅 UEFI

如果仅使用 UEFI 启动，只需将 ISO 中文件解压到 FAT 格式的 U 盘中即可。

由于所有 UEFI 系统都可以 U 盘的任何 FAT 卷中启动，不需要单独创建 EFI 系统分区。兼容性最强的配置方法为使用 MBR 分区表，并创建单个活动（可启动）主分区，类型为 `0c`“W95 FAT32 (LBA)”。

提示：See Secure Boot#Sign the official ISO with custom keys to understand which files need to be signed if Secure Boot is configured with custom keys.

GNU/Linux

This method extracts files from the ISO image to a USB flash drive.

1. If not done yet, create a partition table on `/dev/disk/by-id/usb-*My_flash_drive*` and a partition (`/dev/disk/by-id/usb-*My_flash_drive*-part*n*`) on the device.
2. If not done yet, format the partition to FAT32:

```
# mkfs.fat -F 32 /dev/disk/by-id/usb-My_flash_drive-partn
```

1. Mount the file system:

```
# mount /dev/disk/by-id/usb-My_flash_drive-partn /mnt
```

1. Extract the ISO image to the mounted file system:

```
# bsdtar -x -f archlinux-version-x86_64.iso -C /mnt
```

1. Unmount the file system.

Windows

This method copies files from the ISO image to a USB flash drive.

1. Partition the USB flash drive and format it to FAT32.
2. Right click on `archlinux-*version*-x86_64.iso` and select *Mount*.
3. Navigate to the newly created DVD drive and copy all files and folders to the USB flash drive.
4. When done copying, right click on the DVD drive and select *Eject*.
5. Eject the USB flash drive.

macOS

Neither *DiskImageMounter* nor *Disk Utility* can mount isohybrid ISOs, but since macOS ships with *libarchive*, the ISO can simply be extracted onto the flash drive using *bsdtar*.

1. If not done yet, partition the USB flash drive and format the partition to FAT32 using *Disk Utility*.
2. Mount the volume.
3. Open the *Terminal* application and use *bsdtar* to extract the ISO image to the mounted file system:

```
$ bsdtar -x -f archlinux-version-x86_64.iso -C /Volumes/your-flash-drive
```

1. When done, unmount and eject the USB flash drive.

使用多引导 USB 设备

**本文或本章节可能需要合并到多引导USB设备。**

附注： For Multiboot USB drive#Using Syslinux and memdisk, this is the same method, only Syslinux is installed from Windows. This whole section should be merged there, since Ventoy is already mentioned over there too.（在 Talk:U 盘安装介质 中讨论）

该方法允许从单个 U 盘启动包括 archiso 在内的多个 ISO。将现有 U 盘升级到新版 ISO 的流程比其余多数方法都简单，详情可参考多引导USB设备。

使用 ventoy

Ventoy 是一个适用于 ISO/WIM/IMG/VHD(x)/EFI 文件的开源可启动 U 盘制作工具。通过使用 ventoy，你不需要反复格式化硬盘，而只需将 ISO/WIM/IMG/VHD(x)EFI 文件复制到 U 盘，然后直接启动即可。你可以一次性复制多个文件，ventoy 会显示一个启动清单让你进行选择。ventoy 可通过 ventoy-binAUR 获取。

Windows

从内存加载安装介质

这个方法使用 Syslinux 和一个 Ramdisk（MEMDISK）来把整个 Arch Linux ISO 镜像加载到内存中。由于它将完全运行于系统内存中，您要确保使用这种方法安装的设备上有足够的内存。至少 500MB 到 1G 内存就足以在 MEMDISK 上安装 Arch Linux。

Arch Linux 和 MEMDISK 系统要求参见安装指南和这里，也可以参考之前的论坛帖。

提示：一旦安装程序加载完毕，您就可以移除 U 盘，甚至再在另一台机器上用它开始执行整个安装步骤。使用 MEMDISK 还允许你在同一个 U 盘中引导并安装 Arch Linux 到它上面。

准备 U 盘

首先将 U 盘格式化为 **FAT32** 文件系统。然后在格式化后的磁盘中创建以下目录：

- `Boot`
- `Boot/ISOs`
- `Boot/Settings`

复制需要的文件到 U 盘

然后，把要引导的 ISO 复制到 `Boot/ISOs` 目录。接着，从这里下载最新版的 syslinux包，并从中提取以下文件并复制到下列目录：

- `./win32/syslinux.exe` 到桌面或者系统中的 Downloads 目录。
- `./memdisk/memdisk` 到 U 盘中的 `Settings` 目录。

创建配置文件

复制完所需文件后，在 U 盘的 /Boot/Settings 目录中创建文件 `syslinux.cfg`。

注意：在 `INITRD` 一行中，确保使用了复制到 `ISOs` 文件夹中的 ISO 文件名。

```
/Boot/Settings/syslinux.cfg
DEFAULT arch_iso

LABEL arch_iso
        MENU LABEL Arch Setup
        LINUX memdisk
        INITRD /Boot/ISOs/archlinux-version-x86_64.iso
        APPEND iso
```

更多信息可参考 Syslinux。

最后一步

最后，在 `syslinux.exe` 所在目录创建一个 `*.bat` 文件并运行（如果是在 Vista 或者 Windows 7 下，需要“以管理员身份运行”）：

```
C:\Documents and Settings\username\Desktop\install.bat
@echo off
syslinux.exe -m -a -d /Boot/Settings X:
```

不推荐的方法

警告：不推荐使用下列方法。

使用 etcher

etcher 包含分析数据收集和第一方广告。可参考， 和 。

使用 Universal USB Installer

- 下载页包含可能带毒的假 *下载*按钮。
- Universal USB Installer 不会按原样写入 ISO 数据，会由于 syslinux 版本的差异导致 BIOS 环境无法启动，可参考 。

- Arch Linux 安装镜像的引导加载器期望 ISO 数据位于卷标为 `ARCH_*YYYYXX*` 的卷上。Universal USB Installer 不会更新卷标，也不会修正引导加载器的配置。

使用 UNetbootin

- UNetbootin 不会按原样写入 ISO 数据，会由于 syslinux 版本的差异导致 BIOS 环境无法启动。
- Arch Linux 安装镜像的引导加载器期望 ISO 数据位于卷标为 `ARCH_*YYYYXX*` 的卷上。UNetbootin 不会更新卷标，也不会修正引导加载器的配置。

故障排除

未显示设备

如果由于 `/dev/disk/by-label/ARCH_*YYYYMM*` 未挂载而出现 `device did not show up after 30 seconds` 报错，可以尝试将 U 盘重命名为 `ARCH_*YYYYMM*` 来让 Arch 进行自动搜寻（例如 ISO 名为 `archlinux-2021.02.01-x86_64.iso`，设备名应为 `ARCH_202102`）。

Failed to set up loop devices: No such file or directory

如果你看到了 `losetup: /run/archiso/bootmnt/arch/x86_64/airootfs.sfs: failed to set up loop devices: No such file or directory` 报错，请尝试换用 USB 2.0 接口，有时 USB 集线器的部分 USB 3.0 接口会失效。

其它报错

对于其它报错，请尝试换一个 U 盘，通常情况下可以解决所有问题。

参见

- Gentoo:LiveUSB/Guide
- Fedora wiki - How to create and use Live USB
- openSUSE wiki - SDB:Live USB stick

3.11 在 ZFS 上安装 Arch Linux

难度 高级 | **篇幅** 29.0 KB | **所属** 安装过程

本节将讲到

- 创建自定义 ISO
- 分区
- 设置 ZFS 文件系统
- 安装并配置 Arch Linux

- 设置 hostid

翻译状态：

- 本文（或部分内容）译自 Install Arch Linux on ZFS，最近一次同步于 2026-08-05，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

这篇文章详细描述了将 Arch Linux 安装在 ZFS 文件系统上所需的步骤。

创建自定义 ISO

要在 ZFS 上安装 Arch Linux，您需要使用带有 ZFS 内核模块的安装介质。

请按照 archiso#安装 的步骤操作。

接下来，依照 archiso#准备自定义配置文件 来准备一个自定义的配置文件（下文中以 `archlive` 为例），另请参阅 archiso#ArchZFS_实例。

下一步，编辑软件包列表 `packages.x86_64` 并在其中加入 linux-lts包 内核与下列 ZFS 软件包：

**本文或本章节的事实准确性存在争议。**

原因： 为何需要 libunwind包？如果有任何 ZFS 软件包需要它，则它应该已经是相关软件包的依赖项。（在 Talk:在 ZFS 上安装 Arch Linux 中讨论）

```
packages.x86_64
...
linux-lts
linux-lts-headers
libunwind
zfs-utils
zfs-dkms
```

请确保从列表中移除 `linux` 与 `broadcom-wl` (后者会拉取 linux包 包)，并将 `airootfs/etc/mkinitcpio.d/linux.preset` 删除。

您还需要编辑 `pacman.conf` 并在其中加入：

**本文或本章节可能需要合并到非官方用户仓库#archzfs。**

附注： 重复关于 `pacman.conf` 的相关教程没有意义，写明指向非官方用户仓库#archzfs 的链接即可，否则用户可能重复更改 `Server` 部分。（在 Talk:在 ZFS 上安装 Arch Linux 中讨论）

```
...
[archzfs]
SigLevel = TrustAll Optional
Server = https://github.com/archzfs/archzfs/releases/download/experimental
```

您还需要编辑引导加载器的配置文件，以将 `vmlinuz-linux` 与 `initramfs-linux.img` 的条目分别改为 `vmlinuz-linux-lts` 和 `initramfs-linux-lts.img`:

```
$ sed -i -E 's/(vmlinuz|initramfs)-linux/&-lts/g' archlive/efiboot/loader/entries/*.conf archlive/syslinux/*.cfg archlive/grub/*.cfg
```

接下来，开始镜像的构建过程：

```
$ mkarchiso -v -r -w /tmp/archiso-tmp -o isobuild ~/archlive
```

现在，您的 `archlive/isobuild` 目录下应该已经出现了`archlinux-*YYYY*.*MM*.*DD*.x86_64.iso`。

请将该文件烧录至您选择使用的安装介质。

注意：

- 如果在系统更新后，`zpool status -v` 的输出显示您的存储池需要升级以支持新的特性，您可能需要重新制作上述安装介质。
- 在实际安装时，您应该使用 linux-lts包 与 linux-lts-headers包 来保证最大兼容性。
- 您应从非官方的 archzfs 仓库而非 AUR 获取 *zfs-utils* 与 *zfs-dkms* 的更新。如果您选择使用 AUR 中的版本，则可能出现当前的主线或 zen 内核不被 ZFS 支持的情况，从而导致您的系统无法启动。因此，在实际操作时，建议您使用 linux-lts 内核。

请确保先在虚拟机中测试您的 ISO。只需运行：

```
# modprobe zfs
# zpool status
```

如果命令执行失败，则您的 ZFS 模块并未成功构建，您需要重新尝试。

警告：请勿分发您的自定义 ISO 镜像，否则您将违反 GNU 通用公共许可证 (GPL) 与通用开发和发行许可证 (CDDL)。

分区

ZFS 支持 GUID 分区图与主引导记录分区表。要决定使用哪种分区图，请参考分区#选择_GPT_还是_MBR。

提示：

- 虽然您可以为旧设备创建基于主引导记录的可引导 ZFS 根分区，但更建议您使用 GUID 分区图。
- 推荐您使用单独的 `/boot` 分区以避免有关引导加载程序的问题，同时保证最好的兼容性。

ZFS 会管理自己的分区，所以只需创建一个最简单的分区类型即可。创建 ZFS 文件系统的分区类型应为类型 `bf00` ，或“ Solaris 根目录分区”：

```
Part     Size   Type
----     ----   ------------------------
   1       1G   EFI system partition (ef00)
   2     XXXG   Solaris Root (bf00)
```

在 BIOS（或一台以 Legacy 模式启动的）设备上使用 GUID 分区图与 GRUB 启动引导器的示例：

```
Part     Size    Type
----     ----    ------------------------
   1       2M    BIOS boot partition (ef02)
   2       1G    Linux Partition (8300)
   3     XXXG    Solaris Root (bf00)
```

您可以选择创建独立的 linux 交换分区或将 zvol 用作交换空间，后者详见 ZFS#交换卷。

如果您想要创建一个传统的交换分区，详见 分区#布局示例。

若您创建了单独的 boot 分区以及其他非 ZFS 分区，请将其格式化。

设置 ZFS 文件系统

首先，确认 ZFS 内核模块已经被加载，

```
# modprobe zfs
```

创建根存储池

创建存储池并设置好数据集的默认选项。存储池上新创建的任何数据集都会保留这个存储池创建时使用 `-O` 设定的选项。默认选项在在 ZFS 上安装 Debian Buster # 第二步: 磁盘格式化中有详细说明。

注意：物理扇区大小为512字节的磁盘驱动器应使用 `-o ashift=9` 参数，而物理扇区大小为4096字节的磁盘驱动器应使用 `-o ashift=12` 参数。要获得每个 SCSI/SATA 磁盘驱动器的物理扇区大小，可以运行 `lsblk -S -o NAME,PHY-SEC`。如果想查看所有设备的物理扇区大小，可从命令中删去 `-S`。若使用 NVMe 驱动器，使用 `nvme id-ns /dev/nvmeXnY -H | grep "LBA Format"` 来获取正在使用的逻辑块地址。大部分 NVMe 驱动器使用512字节的逻辑块大小，见 OpenZFS: NVMe low level formatting 以将其大小改为4096字节。

警告：应始终注意现代的大部分设备的物理扇区大小为4096字节，但有些设备会将自己的物理扇区大小报告为512字节，尤其是固态驱动器（SSD）。即使在报告扇区大小为512字节、实际物理扇区大小为4096字节的设备上选择 `ashift=9` 也**一定会**导致严重的性能下降。在物理扇区大小为512字节的设备上选择 `ashift=12` 不会导致性能下降，但可能使磁盘实际可用的空间减少。如果您不确定的话，对于现代设备，应使用 `ashift=12`，或者您可以搜索您设备对应的正确值。对于有关的讨论，参见 OpenZFS issue #967 ；对设置较高的 ashift 值可能出现的问题，见 OpenZFS issue #2497 。

警告： 接下来创建存储池时，如果使用例如 */dev/nvme0n1p2* 的一般设备名而不是 */dev/disk/by-id/<device number>* 来创建存储池，则有可能会使存储池的导入出现问题。若要在下一部分开始前知晓您设备的正确 by-id 名称，请运行：

```
ls -lh /dev/disk/by-id/
```

并记录您将要使用的正确 *by-id* 名称，而非标准设备名，如：

```
lrwxrwxrwx 1 root root 9 Aug 12 16:26 ata-ST3000DM001-9YN166_S1F0JKRR -
> ../../sdc
```

使用压缩与 TRIM

```
# zpool create -f -o ashift=12          \
             -o autotrim=on             \
             -O acltype=posixacl        \
             -O relatime=on             \
             -O xattr=sa                \
             -O dnodesize=auto        \
             -O normalization=formD     \
             -O mountpoint=none         \
             -O canmount=off            \
             -O devices=off             \
             -R /mnt                    \
             -O compression=lz4         \
             zroot /dev/disk/by-id/id-to-partition-partx
```

使用压缩, TRIM 与原生加密

以下命令创建的存储池会在所有数据集上默认启用压缩与原生加密：

```
# zpool create -f -o ashift=12          \
             -o autotrim=on             \
             -O acltype=posixacl        \
             -O relatime=on             \
             -O xattr=sa                \
             -O dnodesize=auto        \
             -O normalization=formD     \
             -O mountpoint=none         \
             -O canmount=off            \
             -O devices=off             \
             -R /mnt                    \
             -O compression=lz4         \
             -O encryption=aes-256-gcm  \
             -O keyformat=passphrase    \
             -O keylocation=prompt      \
             zroot /dev/disk/by-id/id-to-partition-partx
```

注意：

前述所有选项对 GNU/Linux 系统都有意义，而且它们中的大部分在根文件系统上必须被使用，尤其是 `canmount` 与 `mountpoint`，这点与通常的存储设备设置方式不同。`xattr`, `dnodesize` 与 `normalization` 为可选项。

加密与压缩为可选项，但本教程推荐您启用压缩。为了在数据无损压缩存储的情况下避免性能损失，建议使用 lz4 算法。较为均衡的 zstd 算法是默认压缩选项。

您可以在手册页面 zfsprops(7)找到关于这些选项的详细解释。

`-O` 之后的参数将控制 ZFS 的行为。 您可以在 zfsprops(7) 手册页面中找到关于这些参数的详细解释。

警告：由于在不使用 spl hostid 与 zpool.cache 文件的情况下导入 ZFS 存储池存在较多问题，且可能会导致存储池损坏，本教程不使用 canmount=on 选项。如果您未设置 spl hostid 并使用 zfs-zed 加载您的存储池，您在跨系统使用存储池的情况下将面临存储池损坏的风险。

警告：

- 使用 ZFS 时应始终使用设备的 id 名称，否则导入存储池时会发生错误。

- 除使用 by-id 名称外，也可以考虑使用 by-partuuid 或 by-uuid 名称，因为即使一个内置磁盘驱动器被移入USB移动硬盘盒，这些名称的值也不会发生改变，反之亦然（这仅当 ZFS 在磁盘上的某个分区中才有效，若 ZFS 占据整个磁盘则无效）。

创建您的数据集

ZFS 使用数据集的概念来管理您的存储设备，而非使用传统的磁盘分区。与磁盘分区不同，数据集没有固定的大小，每个数据集也可以有各自不同的属性，例如压缩。普通的 ZFS 数据集由 ZFS 自动挂载，而传统类型的数据集则需由 fstab 或 使用 mount 命令挂载。

ZFS 最实用的功能之一便是启动环境。启动环境使您可以创建系统的可引导快照，您也可以通过简单地重启到某启动环境来将整个系统回滚到那个快照。这使得系统更新变得更加安全，对软件开发与测试来讲也十分有用。要使用如 beadm, zectlAUR (用于 systemd-boot), or zedenvAUR (用于 GRUB) 等的启动环境管理器来管理启动环境，您的数据集必须有正确的配置。其关键是将您存放数据的目录 (如 `/home`) 与系统数据分别放在相互独立的不同数据集中，且不要在存储池根目录中存放数据，因为放在存储池根目录的数据以后将不能被移动。

您总是至少应该为您的根目录创建一个数据集，且多数情况下您也会想要把 `/home` 存放在一个单独的数据集中。您也可以自行选择是否要无视启动环境而始终保留完整的日志文件。如果您使用的某些软件会在 `/home` 之外存放数据 (如数据库服务器)，您应整理数据集的结构，使得这些软件的数据目录与根目录数据集分离开来。

以下的示例命令会创建一个只分根目录数据集与 `/home` 数据集的最基本可用于启动环境使用的配置。数据集使用其所在的存储池在创建时设定的默认选项。

```
# zfs create -o mountpoint=none zroot/data
# zfs create -o mountpoint=none zroot/ROOT
# zfs create -o mountpoint=/ -o canmount=noauto zroot/ROOT/default
# zfs create -o mountpoint=/home zroot/data/home
```

创建根目录数据集时您也可以不将挂载点指定为 / ，毕竟无论如何 GRUB 启动引导器会将其挂载至 / 。这也使得您可以通过克隆旧版数据集并将其放入 GRUB 启动菜单来直接从旧版的根目录启动。这种情况下，您可以使用以下命令来创建您的根目录数据集：

```
# zfs create -o mountpoint=/roots/default zroot/ROOT/default
```

您可以将 `/root` 存储在您的 `zroot/data/home` 数据集中。

```
# zfs create -o mountpoint=/root zroot/data/home/root
```

系统数据集

为系统目录创建数据集时，请使用`canmount=off`选项。

示例请参见 在 ZFS 上安装 Debian Buster # 第三步：安装系统。

注意：若要将如 `zroot/var/log` 的数据集挂载至 `/var/log`，应考虑使用 ***zfs-mount-generator*** 而非 `zfs-mount.service`。这会修复文件系统挂载顺序，在 这里 有详细介绍。

```
# zfs create -o mountpoint=/var -o canmount=off                 zroot/var
# zfs create                                                    zroot/var/log
# zfs create -o mountpoint=/var/log/journal -o acltype=posixacl zroot/var/log/journal
# zfs create -o mountpoint=/var/lib -o canmount=off             zroot/var/lib
# zfs create                                                    zroot/var/lib/libvirt
# zfs create                                                    zroot/var/lib/docker
```

注意：systemd-journald 需要您为 /var/log/journal/ 启用 POSIX 访问控制列表，即为该目录所在的数据集设置 acltype=posix，否则 systemd-journald.service 将无法启动。详见 systemd#systemd-tmpfiles-setup.service在系统启动时失败。

导出并导入您的存储池

要验证您的设置，将您所有的 ZFS 存储池先导出后再重新导入。

警告：不要跳过这一步，否则您再导入存储池时必须使用 `-f` 参数。这会使导入的存储池卸载。

注意：如果您使用交换分区的话，这一步可能会失败。您需要先使用 ***swapoff*** 命令将其关闭。

```
# zpool export zroot
# zpool import -d /dev/disk/by-id -R /mnt zroot -N
```

注意：`-d` 并不是设备的实际 ID，而是包含着符号链接的 `/dev/by-id` 目录。

如果这个命令执行失败并且您被要求使用数字 ID 来导入某个存储池，运行 `zpool import` 来找到您存储池的 ID，然后使用类似下方的命令导入存储池：

```
# zpool import 9876543212345678910 -R /mnt zroot
```

如果您启用了原生加密选项，请加载 ZFS 密钥。

```
# zfs load-key zroot
```

由于根目录数据集使用 `canmount=noauto` 参数，您需要先将其手动挂载，然后再挂载其他数据集。

```
# zfs mount zroot/ROOT/default
# zfs mount -a
```

现在，您的 ZFS 文件系统已准备就绪。

配置根目录文件系统

如果您使用了传统类型的数据集，则您需要将其写入 `/etc/fstab`。

为根目录所在的子文件系统设置 bootfs（启动文件系统）属性，以便引导加载程序找到操作系统。

```
# zpool set bootfs=zroot/ROOT/default zroot
```

如果您还没有 `/etc/zfs/zpool.cache`，请手动创建：

```
# zpool set cachefile=/etc/zfs/zpool.cache zroot
```

切记要将 `zpool.cache` 文件放入您的新系统中。稍后 ZFS 守护进程启动时需要这个文件。

```
# mkdir -p /mnt/etc/zfs
# cp /etc/zfs/zpool.cache /mnt/etc/zfs/zpool.cache
```

安装并配置 Arch Linux

按照安装指南安装系统。若有涉及 ZFSonLinux 所需的特殊操作，将会在此列出。

- 首先使用 mount 命令挂载所有的传统类型数据集以及非 ZFS 的引导或系统分区。
- 安装基本系统。

注意：您可能想要在 `pacstrap` 操作时使用 linux-lts包 作为您的内核。同时请确保使用 pacstrap 安装 libunwind包 从而为 zfs-utilsAUR 解析依赖项目。

- 安装指南#生成 fstab 文件中所描述的方式对 ZFS 来说并非必要。通常 ZFS 会自行挂载自己的分区，所以除非用户为系统目录使用了传统类型数据集，`fstab` 文件中不需要任何有关 ZFS 的部分。 要为文件系统生成 `fstab`，运行：

```
# genfstab -U -p /mnt >> /mnt/etc/fstab
```

- 依照安装指南#chroot 到新安装的系统中的方法将根目录切换至新安装的系统内。

```
# arch-chroot /mnt
```

- 编辑 `/etc/fstab`：

注意：

- 如果您选择为系统目录创建传统类型数据集，则请在 `fstab` 中保留这些数据集。
- 将除交换空间与 EFI 系统分区以外的所有非传统类型数据集注释掉。传统做法是，可以使用较简单的 `/dev/zvol/zroot/swap` 来取代交换空间的 UUID。
- 在更新 ramdisk 并启用 ZFS 支持前，您需要在 `/etc/pacman.conf` 中加入 archzfs 仓库，将其签名并在 arch-chroot 环境下安装 zfs-dkmsAUR 和 zfs-utilsAUR。

注意：出于简洁性以及兼容性考虑，本指南建议您使用 zfs-dkmsAUR 与 linux-lts包。

- 创建初始 ramdisk 前，请先编辑 `/etc/mkinitcpio.conf`，并将 `zfs` 加至 `MODULES`：

```
MODULES=(zfs)
```

接下来在 `HOOKS` 中将 `zfs` 加至 filesystems 前，并将 `systemd` 切换为 `udev`。您的 `HOOKS` 一行看起来应类似如下示例：

```
HOOKS=(base udev ... block zfs filesystems)
```

警告：

- 默认情况下，根文件系统为 ZFS 的情况下，不支持在 initramfs 中使用 systemd。使用 systemd 会导致 root 文件系统无法挂载，从而导致系统无法启动。要解决这个问题，请将 mkinitcpio 使用的所有 systemd HOOK 更换为 mkinitcpio#常用钩子中描述的基于 Busybox 的 hook。如果您仍然想要在 initramfs 中使用 systemd，请阅读下文中关于 mkinitcpio-sd-zfsAUR 的信息。

注意：

- zfs-utilsAUR 提供了一个开箱即用且适用于基于 systemd 的 initramfs 的 dracut hook。
- 由 zfs-utilsAUR 提供的 mkinitcpio 的 ZFS hook 仅兼容基于 busybox 的 initramfs 镜像。
- 对于使用了 ZFS 加密的用户，建议您将 `keyboard` 移至 `autodetect` 前，以防止在输入密码时出现问题。
- mkinitcpio-sd-zfsAUR 提供了与 systemd 兼容的 mkinitcpio initramfs 的 ZFS hook。请注意，"sd-zfs" hook 未经过完整测试，并且可能不兼容启用了加密的文件系统。
- 重新生成 initramfs。
- 将 ZFS 添加至您的内核参数。

现在，请设置您的 引导加载程序。要使得 ZFS 可以启动，您还需要以下的内核参数：

```
root=ZFS=zroot/ROOT/default rw
```

设置 hostid

为了使您的系统能够顺利启动，您还需要设置 hostid。

注意：

- 本章节的内容假设您仍处在 `arch-chroot` 环境中。
- 虽然 zpool.cache 正待废弃，但zfs-utils 仍需要 zpool.cache 来获得可以启动的系统，故现在这仍是取得可启动的系统的默认以及建议方式。\详见: [https://github.com/openzfs/zfs/issues/1035\]

对每个您想自动挂载的存储池运行以下命令：

```
# zpool set cachefile=/etc/zfs/zpool.cache pool
```

当根目录文件系统为 ZFS 时，本机的 hostid 在挂载时将不可见。解决方案有两种。您可以在引导加载程序中将您的 spl hostid 写入 内核参数中，例如 `spl.spl_hostid=0x00bab10c`。要确定您的 hostid，请运行 `hostid`。

另一种，同时也是推荐的解决方案，是保证 `/etc/hostid` 中写入了 hostid，并重新生成 initramfs 以将其拷贝至 initramfs 镜像内。要安全地写入 hostid 文件，请使用 `zgenhostid`。

要使用 libc 生成的 hostid（建议的操作）：

```
# zgenhostid $(hostid)
```

要使用自定义的 hostid，运行以下命令（注意，hostid 必须是8个字符的十六进制数）：

```
# zgenhostid deadbeef
```

完成后请勿忘记重新生成 initramfs。

配置 ZFS 挂载 （适用于 systemd）

注意：

这一章节仅适用于 systemd，即 Arch Linux 默认使用的 init 系统。

要自动挂载存储池，您需要启用 `zfs.target`。

启用 `zfs.target`

要在系统启动时自动挂载 ZFS 存储池，您需要启用 `zfs-import-cache.service`、`zfs-mount.service` 与 `zfs-import.target`。

配置 ZFS 挂载 （适用于 OpenRC）

警告：

本章节适用于 OpenRC init 与服务管理系统的用户。OpenRC 仅在 AUR 可用，Arch Linux 对其没有官方支持。根据 OpenRC 页面，当向他人求助时，您应声明自己在使用 OpenRC。

注意：

虽然 Arch Linux 的默认 init 系统是 systemd，本章节仍出于文档完整性的目的提供。

您需要将 `zfs-import` 与 `zfs-mount` init 脚本添加至 boot runlevel。

```
# rc-update add zfs-import boot
# rc-update add zfs-mount boot
```

注意：

如果您启用了 ZFS 加密并且希望在启动时解锁您的 ZFS 分区，还应将 zfs-load-key 添加至 boot runlevel。

```
# rc-update add zfs-load-key boot
```

卸载文件系统并重新启动

我们离成功不远了！如果您使用传统类型的启动引导分区，先运行：

```
# umount /mnt/boot
```

如果未使用传统的单独引导分区，直接运行：

```
# zfs umount -a
# zfs umount zroot/ROOT/default
# zpool export zroot
```

现在，重新启动系统。

警告：如果您没有正确将存储池导出，重新启动后 ZFS 将拒绝在 ramdisk 环境中导入存储池，从而使您卡在 busybox 终端界面。

从 USB 存储设备加载密钥

可以将密钥存储在 USB 存储设备上并在启动时加载：

在 USB 存储介质的初始字节处存储密钥：

```
# dd if=your_password_file (您的密钥文件) bs=32 count=1 of=/dev/disk/by-id/usb_stick (USB 存储设备)
```

要创建 ZFS 分区，您可以使用上文所述的输入密钥的方式，或直接使用 dd 命令配合管道写入 USB 存储设备中存储的密钥：

```
# dd if=/dev/disk/by-id/usb_stick bs=32 count=1 | zfs create -o encryption=on -o keyformat=passphrase zroot/ROOT
```

下一步就要更改 zfs hook。zfs 默认会询问密钥。您需要将获取密钥的方式改为通过从您存放着密钥的 USB 设备中 dd 来获取。要达成这个目的，

将 `/usr/lib/initcpio/hooks/zfs` 中的以下行：

```
# ! eval zfs load-key "${encryptionroot}"; do
```

改为：

```
# ! eval dd if=/dev/disk/by-id/usb_stick bs=32 count=1 | zfs load-key "${encryptionroot}"; do
```

您刚刚更改了您的 zfs hook，所以不要忘记重新生成 initramfs。现在 zfs 应该能在启动时从您的 USB 设备中加载密钥。

故障排除

系统因 "无法导入 zroot：存储池不存在（cannot import zroot: no such pool available）" 而无法启动

您可以尝试以下步骤，看看是否有帮助。

- 使用 archzfs 仓库中提供的内核模块，不要使用 dkms 版本。您可以在成功启动后再改为使用 dkms 变种。
- 移除 `/etc/zfs/zpool.cache` 并运行：

```
# zpool set cachefile=none zroot
```

- 移除 `/etc/hostid`。
- 重新生成 initramfs。

zpool 因为忙碌而拒绝导出

Arch-chroot 会在系统里挂在特定的内核空间文件系统。如果没有卸载这些文件系统，存储池可能会拒绝导出。此时，请重新挂载 ZFS 分区，并运行 `findmnt -R /mnt`

然后运行对于每个仍然挂载的文件系统，运行 `umount -f */path/to/partition*`。

之后您应当能够正常导出您的存储池。

另请参阅

- A reference guide to ZFS on Arch Linux
- Aaron Toponce Install ZFS on Debian GNU/Linux
- Arch Linux On ZFS
- Arch Linux Root on ZFS
- FreeBSD ZFS Handbook
- Funtoo:ZFS Install Guide
- HOWTO install Ubuntu to a Native ZFS Root
- Open-ZFS Bootcamp
- OpenZFS wiki
- ZFS cheatsheet
- ZFSOnLinux wiki
- Gentoo ZFS Wiki

启动加载程序

能否开机，取决于引导加载程序是否装对。本章讲清 GRUB 与 systemd-boot 两套主流方案，以及 UEFI 与 BIOS 两条固件路径。建议先读《GRUB》，走通两种固件下的安装与配置；再读《Systemd-boot》，它更轻，与统一内核映像配合方便；随后读《统一内核映像》，处理安全启动签名。《EFISTUB》与《REFInd》是不装引导器的替代路线，《GRUB/技巧和窍门》等能开机后再查，《GRUB Legacy》《Syslinux》《Clover》面向旧固件或特定场景，可先跳过。

最容易踩的坑是把 UEFI 与 BIOS 的步骤混着照抄，两者的安装与配置并不通用；另一处误解是以为非装一个引导器不可，《EFISTUB》与《REFInd》说明固件本身就能直接启动内核，而一旦启用安全启动，未签名的统一内核映像又会在签名这一步卡住。

4.1　Clover

难度 进阶 | **篇幅** 8.4 KB | **所属** 启动加载程序

本节将讲到

- 支持的文件系统类型
- 安装
- 配置
- 参见

翻译状态：

- 本文（或部分内容）译自 Clover，最近一次同步于 2026-06-26，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Clover EFI 是一个用于在传统引导模式或 UEFI 引导模式下启动 macOS（Hackintoshes）、Windows 和 Linux 的 引导加载程序。

Clover 的主要优势有：

- 在传统 BIOS 系统上模拟 UEFI。
- 启动支持 EFI boot stub 的 Linux 内核。
- 在如今常见的宽屏上支持原生分辨率的图形界面。
- 简单易用
- 易于配置和自定义

支持的文件系统类型

Clover 继承了固件所支持的文件系统（例如，至少支持 FAT12、FAT16 和 FAT32），除此以外还会加载位于安装目录中的 `drivers` 子目录下的所有 UEFI 驱动（目录位置示例：`*esp*/EFI/clover/drivers`）。

此外 Clover 也有多个EFI 文件系统驱动。

安装

由于 Clover 在 BIOS 系统上模拟 UEFI 环境，其在不同系统上的安装流程是类似的。

UEFI

若直接从 UEFI 启动到一个 EFI boot stub 内核中，推荐将 EFI 系统分区挂载到 `/boot`。

然后使用 mkinitcpio 生成早期用户空间环境。

从 Clover 发布页面 上下载可引导的 Clover ISO 文件。

解压已下载的压缩文件，然后找到文件名为 `Clover-*-X64.iso` 的镜像文件，将其挂载到如 `/mnt/iso` 之类的目录下。需要注意的是，所有文件或目录名称在 Linux 中都会显示为小写字母，这和 Windows 与 Mac OS 上是不同的。

如果已按上述流程将镜像文件挂载到 `/mnt/iso` 下，请将整个 `/mnt/iso/efi` 目录复制到 EFI 系统分区中。`/boot` 目录树应该像下面这样：

```
/boot/EFI
/boot/EFI/BOOT
/boot/EFI/CLOVER
/boot/initramfs-linux-fallback.img
/boot/initramfs-linux.img
/boot/vmlinuz-linux
```

如果当前有其他的操作系统（例如 Windows）接管了你的 EFI 分区，不要忘记备份 EFI 分区中 `BOOT` 文件夹下的 bootx64.efi 文件。

BIOS

注意：传统 BIOS 下依然需要使用 UEFI/GPT 分区模式，唯一的区别是需要在 EFI 系统分区上设置“Legacy BIOS Bootable”标识。

从 Clover 发布页面 上下载可引导的 Clover ISO 文件。

解压已下载的压缩文件，然后找到文件名为 `Clover-*-X64.iso` 的镜像文件，将其挂载到如 `/mnt/iso` 之类的目录下。

此时必须将 Clover 与主分区记录（MBR）和启动分区记录（PBR）合并。 以下是示例，此示例中有一个块设备 `/dev/sda`，ESP 位于其第一个分区 `/dev/sda1` 上：

```
# dd if=/dev/sda1 of=/tmp/original_PBR bs=512 count=1 conv=notrunc
# cp /mnt/iso/usr/standalone/i386/boot1f32 /tmp/new_PBR
# dd if=/tmp/original_PBR of=/tmp/new_PBR skip=3 seek=3 bs=1 count=87 conv=notrunc
# dd if=/tmp/new_PBR of=/dev/sda1 bs=512 count=1 conv=notrunc
# dd if=/mnt/iso/usr/standalone/i386/boot0ss of=/dev/sda bs=440 count=1 conv=notrunc
```

然后将 EFI 系统分区挂载到 `/boot`。

将整个 `/mnt/iso/efi` 目录复制到 EFI 系统分区。

将传统引导加载程序复制到 EFI 系统分区中：

```
# cp /mnt/iso/usr/standalone/i386/x64/boot6 /boot/boot
```

配置

Clover 的配置是通过编辑 EFI 系统分区中 `EFI/CLOVER` 目录下的 XML 文件 `config.plist` 来完成的。

各配置参数的详情请参见 Clover 的 wiki。`Volume` 应是 EFI 系统分区的 UUID 且必须大写。`Arguments` 中的 最小 initramfs `initramfs-linux.img` 和 `Path` 中的 Linux 内核可执行文件 `vmlinuz-linux` 路径都是基于 EFI 系统分区的相对路径。反斜杠应根据 EFI 标准使用。对于 `Arguments` 中的其他可用参数，请参见 EFI boot stub 和内核参数#参数列表。

下面的例子中，initramfs 和内核文件都位于 EFI 系统分区的根目录下，即 `efi` 的同一级目录，并且 EFI 系统分区挂载于 `/boot`：

```
/boot/EFI/CLOVER/config.plist
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"https://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>GUI</key>
  <dict>
    <key>Custom</key>
    <dict>
      <key>Entries</key>
      <array>
        <dict>
          <key>Arguments</key>
          <string>initrd=\initramfs-linux.img root=PARTUUID=d5d0fceb-c58e-c745-a7ff-
f2f1b2824e66 rw add_efi_memmap</string>
          <key>Path</key>
          <string>\vmlinuz-linux</string>
          <key>Title</key>
          <string>Arch Linux</string>
          <key>Type</key>
          <string>Linux</string>
          <key>Volume</key>
          <string>33BA6C23-4772-294D-9053-72A49FCAEF39</string>
          <key>VolumeType</key>
          <string>Internal</string>
        </dict>
```

```
        <dict>
          <key>Arguments</key>
          <string>initrd=\initramfs-linux-fallback.img root=PARTUUID=d5d0fceb-c58e-
c745-a7ff-f2f1b2824e66 rw debug add_efi_memmap</string>
          <key>Path</key>
          <string>\vmlinuz-linux</string>
          <key>Title</key>
          <string>Arch Linux Rescue Mode</string>
          <key>Type</key>
          <string>Linux</string>
          <key>Volume</key>
          <string>33BA6C23-4772-294D-9053-72A49FCAEF39</string>
          <key>VolumeType</key>
          <string>Internal</string>
        </dict>
      </array>
    </dict>
  </dict>
</dict>
</plist>
```

链式加载 systemd-boot

若需要一个遵守 The Boot Loader Specification 的引导加载程序，systemd-boot 可以完成这项功能。以下是一份令 Clover 链式加载 systemd-boot 的配置文件：

```
/boot/EFI/CLOVER/config.plist
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"https://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
       <key>Boot</key>
       <dict>
              <key>DefaultVolume</key>
              <string>EFI</string>
              <key>DefaultLoader</key>
              <string>\EFI\systemd\systemd-bootx64.efi</string>
              <key>Fast</key>
              <true/>
       </dict>
       <key>GUI</key>
       <dict>
              <key>Custom</key>
              <dict>
                     <key>Entries</key>
                     <array>
                            <dict>
                                   <key>Hidden</key>
                                   <false/>
                                   <key>Disabled</key>
                                   <false/>
                                   <key>Image</key>
                                   <string>os_arch</string>
                                   <key>Volume</key>
                                   <string>EFI</string>
                                   <key>Path</key>
                                   <string>\EFI\systemd\systemd-bootx64.efi</string>
                                   <key>Title</key>
                                   <string>Arch Linux</string>
                                   <key>Type</key>
                                   <string>Linux</string>
                            </dict>
                     </array>
              </dict>
       </dict>
</dict>
</plist>
```

参见

- 项目主页
- Clover Wiki

4.2 EFISTUB

难度 进阶 | **篇幅** 13.1 KB | **所属** 启动加载程序

本节将讲到

- 另选方案：统一内核镜像
- 引导 EFI boot stub
- 一些小提示和技巧
- 故障排除
- 参见

EFI boot stub (也称为 **EFI stub**)是本身就为 EFI 可执行程序的内核，也就是说，它可以直接被 UEFI 引导启动。

此前，本文和 Debian Wiki 曾将该术语拼写为单个单词 (EFISTUB 或 EFIStub).

默认情况下，Arch Linux提供的内核都是 EFI boot stubs。如果自行编译内核，需将 `CONFIG_EFI_STUB` 配置为 `y` 以启用它，详见The EFI Boot Stub。

在继续之前，需要一个 EFI 系统分区（ESP），并且选择其挂载方式。

提示：

- 如果将ESP分区挂载到`/boot`，pacman 更新的内核就能直接使 UEFI 读取。
- EFI boot stub也可以使用引导加载程序间接启动:
- 一些 UEFI 启动管理器可以为 UEFI 引导提供额外选项或简化过程
- 这对于测试内核参数或有多个内核/操作系统且主板 UEFI 启动菜单不易使用的情况尤为有用
- 如果启动管理器拥有内核和 initramfs 所在分区的文件系统的驱动（例如rEFInd），则无需将他们保存在 ESP 中。

另选方案：统一内核镜像

统一内核镜像 (UKI) 本身就是一个 EFI stub 内核,同时含有 initramfs 和启动选项在单一文件中。下表列出了在它们被 UEFI 引导时，UKI 与非 UKI 的 EFI stub 的特性对比。

	UKI	非 UKI 的 EFI stub
initramfs	包含在内	在单一文件中，存储在 ESP 中
启动选项	自身包包含在内含	存储在 UEFI 变量中

当没有问题时，UKI 更简单，因为它只需要一个文件，并容纳了所有启动内核所必需的一切。然而，非 UKI 内核可以在不同的 initramfs 文件之间切换和尝试不同的启动选项。对于 UKI 来说，每次变动都会导致生成一个单独的 UKI。

引导 EFI boot stub

注意：传递给内核 EFI boot stub 的 initramfs 路径应当相对于 ESP 的根目录，并使用反斜杠分隔路径（遵循 EFI 标准）。例如，如果initramfs 在 `*esp*/EFI/arch/initramfs-linux.img`，则对应的 UEFI 格式行应为 `initrd=\EFI\arch\initramfs-linux.img`。在接下来的示例中将默认所有的文件都位于 `*esp*/` 目录中.

直接使用 UEFI

UEFI 旨在摆脱对像 GRUB 这样的中间引导加载程序的依赖. 如果你的主板有良好的 UEFI 实现，那么可能直接将内核参数写入 UEFI 引导项目以让主板直接引导 Arch。可以使用 efibootmgr 或 UEFI Shell v2 以修改主板引导项目。

注意：

- 过时的UEFI 实现可能跟 Linux 内核有兼容性问题。如果设备的 UEFI 有新的修复了问题的版本，可以考虑使用主板厂商推荐的工具刷入。
- 此项可能已过时：一些固件 (尤其是 **Lenovo** 和 **Dell** 笔记本电脑) 不会将 NVRAM 中的引导项命令行参数传给 EFI 可执行程序(详见 Arch BBS 2014年的帖子)这种情况下，可以使用 UKI。

efibootmgr

使用 *efibootmgr* 添加加载内核的引导项

```
# efibootmgr --create --disk /dev/block_device_name --part X --label "Arch Linux" --loader /vmlinuz-linux --unicode 'root=block_device_identifier rw initrd=\initramfs-linux.img'
```

其中，*`block_device_name`* 是块设备名，*`X`* 是分区编号，`root=` 后的 *`block_device_identifier`* 是实际的 Linux 根目录分区标识符。

注意：

- 有关支持的设备名称格式，请参见内核参数。
- 有关如何获取对应的值，请参见块设备持久化命名。
- 有关具体示例，请参见块设备持久化命名#内核参数。

注意：`-u`/`--unicode` 后的字符串是内核参数，可能还需要根据实际情况添加其它参数（如用于休眠到磁盘的参数）。

基于 LTS Linux 内核、NVMe 存储、带有特定子卷的 BTRFS 文件系统以及 Swap 分区休眠的示例：

```
# efibootmgr --create \
 --disk /dev/nvme0n1 --part 1 \
 --label "EFISTUB Arch" \
 --loader /vmlinuz-linux-lts \
```

```
 --unicode 'root=UUID=01a40dd8-28f0-4636-be1e-aeed60c98095
resume=UUID=2d877d5d-4ca1-4d46-a3d6-b6ee94cbbd78 rw rootflags=subvol=@ loglevel=3 quiet
initrd=\initramfs-linux-lts.img'
```

要获取启动项列表、设置引导顺序或删除启动项，请参见 efibootmgr。

提示：

- kesboot-gitAUR 包可以简易且自动地实现此过程，同时他还含有 pacman hook 可在执行软件包操作时添加和移除 EFI 变量。
- https://github.com/de-arl/auto-UEFI-entry 是一个生成上述指令的 bash 辅助工具。
- 将创建引导项的命令保存到 Shell 脚本将非常方便，能让后续修改（例如更改内核参数时）。在这个过程中，可以考虑将删除旧启动项自动化, 因为 *efibootmgr* 当前 不支持编辑现有引导项.
- 题为 The Linux kernel with built-in boot loader 的论坛帖子或许具有参考价值

bcfg

一些 UEFI的实现方式使得使用 efibootmgr 来成功修改 NVRAM 变得非常困难。如果 efibootmgr 无法成功创建相应的条目，那么可以使用 UEFI Shell v2 中的bcfg命令来进行操作。 (即从Arch Linux的Live ISO镜像中执行该命令)。

首先，需要找到您的 ESP分区 所使用的设备编号：

```
 Shell> map
```

在示例中1被用作设备编号。要列出 ESP分区 的内容，请执行以下操作：

```
 Shell> ls FS1:
```

要查看当前的启动项，请执行以下操作：

```
Shell> bcfg boot dump
```

要为你的内核添加一条记录，请使用以下格式：

```
Shell> bcfg boot add N FS1:\vmlinuz-linux "Arch Linux"
```

N是会在启动菜单中添加该条目所在的位置。0 是第一个菜单项。已经存在的菜单项会被移动进去，而不会被删除。

你可以直接添加内核选项：

```
 Shell> bcfg boot -opt N "root=/dev/sda2 initrd=\initramfs-linux.img"
```

或者通过在你的 ESP 分区上创建一个文件来实现：

```
Shell> edit FS1:\options.txt
```

在文件中添加引导行。例如：

```
root=/dev/sda2 rw initrd=\initramfs-linux.img
```

注意：在文件的每一行的开头需要添加额外的空格。在行首有一个字节顺序标记，它会覆盖该位置上的任何字符，这会导致在启动时出现错误。

按 F2 保存,然后按 F3 退出程序。

在之前的条目中添加这些选项：

```
Shell> bcfg boot -opt N FS1:\options.txt
```

对于其他所有条目，都重复进行这个步骤。

若要删除之前添加的项目，请执行以下操作：

```
Shell> bcfg boot rm N
```

使用 UEFI Shell

如果你不想创建永久的启动项，那么可以从 UEFI Shell中启动内核，因为 UEFI Shell 本身就是一个普通的 UEFI 应用程序。

```
> FS0:
> \vmlinuz-linux root=PARTUUID=3518bb68-d01e-45c9-b973-0b5d918aae96 rw
initrd=\initramfs-linux.img
```

在这种情况下，内核参数将会被作为普通参数传递给 EFI 启动脚本内核。

为了避免每次都需要记住所有的内核参数，你可以将可执行命令保存到一个 shell 脚本中，例如保存在 EFI 系统分区中的`archlinux.nsh`脚本，然后运行它：

```
> FS0:
> archlinux
```

使用 startup.nsh 脚本

一些 UEFI 实现在冷启动过程中不会保留 EFI 变量（例如，在 6.1版本之前的 VirtualBox ），因此通过 UEFI 设置的所有选项都会在关机时丢失。

UEFI Shell 规范 2.0规定，ESP 分区根目录下的 `startup.nsh` 脚本始终会被执行并且可以包含任意指令。在这些指令中，你可以设置用于引导加载的语句。请确保将 ESP 分区挂载在 `/boot`并创建一个包含内核引导语句的 `startup.nsh` 脚本，例如：

```
vmlinuz-linux rw root=/dev/sdX [rootfs=myfs] [rootflags=myrootflags] \
 [kernel.flag=foo] [mymodule.flag=bar] \
 initrd=\initramfs-linux.img
```

这种方法适用于几乎所有你在实际硬件中可能会遇到的 UEFI 版本。你可以将其作为最后的解决方案来使用。该脚本应该只包含一个长行代码。括号中的内容是可选的，仅作为指导之用。类似 Shell 风格的换行符只是为了便于阅读而设计的。FAT 文件系统使用反斜杠作为路径分隔符，在这种情况下，反斜杠表示 initramfs 位于 ESP 分区的根目录下。（对于其他设备，sd"X"代表的ESP分区设备或者编号都可能不同）

一些小提示和技巧

带有启动内存盘（ramdisk）的后备启动项

如果没有引导管理器，那么内核命令行在启动时是无法被修改的。为了至少提供一些后备方案，例如可以使用`initramfs-linux-fallback.img` 或者在不使用英特尔微码的情况下启动系统，只需使用 efibootmgr 创建一个额外的引导条目即可。这个引导条目的标签可以命名例如“Arch Linux Fallback”，并设置所需的备用选项。

在ESP分区上的Archiso

可以将 Arch Linux 的 ISO 镜像放置在 ESP 分区中，从而拥有一个恢复系统。截至 2026 年 7 月 1 日的版本发布时，镜像的文件大小为 1.5G。

首先，下载 archlinux-YYYY.MM.DD-x86_64.iso。

接下来，在你的 ESP 目录下创建一个用于 archiso 的目录：

```
# mkdir -p esp/EFI/archiso
```

提取 arch 目录中的所有内容：

```
# bsdtar -v -x --no-same-permissions --strip-components 1 -f archlinux-YYYY.MM.DD-x86_64.iso -C esp/EFI/archiso arch
```

在最后一步中，使用 `efibootmgr` 创建一个启动项。请替换以下内容：

diskpath 是设备的路径（例如：/dev/sda)）

`diskpart` 是分区编号

```
# efibootmgr --create --disk diskpath \
--part diskpart \
--label "Arch (rescue system)" \
--loader '\EFI\archiso\boot\x86_64\vmlinuz-linux' \
--unicode 'archisobasedir=/EFI/archiso
archisosearchfilename=/EFI/archiso/boot/x86_64/vmlinuz-linux
initrd=\EFI\archiso\boot\x86_64\initramfs-linux.img'
```

现在你可以从 UEFI 启动加载程序中选择恢复系统了。

故障排除

一些主板，比如 Haswell 架构时代的华硕（ASUS）主板 (如在法国论坛上遇到的那样), 在系统启动时不会注意到引导条目发生的变化，除非是以另一个已存在的引导条目来启动系统。

参见

- Linux 内核文档——EFI 启动脚本（The EFI Boot Stub）
- Linux 内核提交代码 - x86 架构，efi：支持 EFI 启动脚本功能（EFI Boot Stub 支持）
- Rod·Smith的做法——使用内核的 EFI 加载器

4.3 统一内核映像

难度 进阶 | **篇幅** 18.1 KB | **所属** 启动加载程序

本节将讲到

- 准备统一内核映像
- 为安全启动对 UKI 进行签名

- 启动
- 参阅

统一内核映像（Unified Kernel Image，UKI）是一个可执行文件，可直接从 UEFI 固件进行启动，也可以无需或通过简单的配置由引导加载器自动生成。它将一个 UEFI boot stub 程序（例如 systemd-stub(7)）、一个 Linux 内核映像、一个 initrd 以及其它资源打包到了单个 UEFI PE 文件中。

该文件，也就是包括其下的所有元件都可以很方便地进行签名，以便与安全启动一起使用。

注意：在该文中，`*esp*` 指代 EFI 系统分区的挂载点。

准备统一内核映像

有多种方法可以生成 UKI 映像，并将其安装到正确位置下（*esp/EFI/Linux* 目录）。有多种工具都能完成该操作，可根据你的需要和喜好进行选择。

注意：

- 你只需要完成下面任一种选项。
- 可以将 UKI 放置到默认启动路径 `*esp*/EFI/BOOT/BOOTx64.EFI`（对于 32 位 IA32 UEFI 固件为 `BOOTIA32.EFI`）。使用默认启动路径无需在 NVRAM 显式创建启动项。

mkinitcpio

除非安装了 systemd-ukify包，否则 mkinitcpio 会自行构建 UKI。在安装了 systemd-ukify包 的情况下，除非指定了 `--no-ukify` 选项，否则构建 UKI 的工作会被下放给 *ukify*。

内核命令行参数

mkinitcpio 支持从 `/etc/cmdline.d` 文件夹下的命令行参数文件读取内核参数。Mkinitcpio 会将该文件夹下的所有文件内容集中到一个 *.conf* 文件下，并使用其生成内核命令行参数。命令行文件内以 `#` 开头的行都将被视为注释，并被 mkinitcpio 忽略掉。请**移除掉**指向微码和 initramfs 的启动项。

示例：

```
/etc/cmdline.d/root.conf
root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw
```

提示：

**这一章节正在考虑移除。**

原因: 与内核参数#参数列表重复。 (在 Talk:统一内核映像 讨论)

- 如果你的根文件系统位于非默认的 Btrfs 子卷下，请确保在 `rootflags` 中设置了必要的挂载参数。举个例子，如果系统的子卷 ID 是 `256`，那么就要将 `rootflags=subvolid=256` 添加到内核命令行中。具体请参考 Btrfs#挂载子卷为根挂载点。
- `rootflags` 仅用于启动阶段，因此不需要将 `/etc/fstab` 中的全部挂载选项都复制过来。systemd-remount-fs.service(8) 会在启动后自动读取 `/etc/fstab`，重新挂载并应用所有挂载选项。

```
/etc/cmdline.d/security.conf
# enable apparmor
lsm=landlock,lockdown,yama,integrity,apparmor,bpf audit=1 audit_backlog_limit=256
```

也可以使用 `/etc/kernel/cmdline` 配置内核命令行选项，例如：

```
/etc/kernel/cmdline
root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet bgrt_disable
```

提示：

- 如果根分区由 systemd 自动挂载，可以省略 `root=` 参数。
- `bgrt_disable` 参数可以让 Linux 在加载 ACPI 表后不要显示 OEM 图标。

.preset 文件

下一步，参考如下编辑 `/etc/mkinitcpio.d/linux.preset` 或是你在使用的预设文件，并指向到 EFI 系统分区的挂载点：

- 将 `PRESETS=` 中每一项的 `*PRESET*_uki=` 参数取消注释（即移除 `#`），
- 可以选择注释掉 `*PRESET*_image=`，以避免存储多余的 `initramfs-*.img` 文件，
- 对于想要添加启动画面的项，可以在每行 `*PRESET*_options=` 中添加或取消注释掉 `--splash` 参数。

以下为一个可用 `linux.preset` 示例，用于 linux包 内核并包含 Arch 启动画面：

```
/etc/mkinitcpio.d/linux.preset
# mkinitcpio preset file for the 'linux' package

#ALL_config="/etc/mkinitcpio.conf"
ALL_kver="/boot/vmlinuz-linux"

PRESETS=('default' 'fallback')

#default_config="/etc/mkinitcpio.conf"
#default_image="/boot/initramfs-linux.img"
default_uki="esp/EFI/Linux/arch-linux.efi"
default_options="--splash=/usr/share/systemd/bootctl/splash-arch.bmp"

#fallback_config="/etc/mkinitcpio.conf"
#fallback_image="/boot/initramfs-linux-fallback.img"
fallback_uki="esp/EFI/Linux/arch-linux-fallback.efi"
fallback_options="-S autodetect"
```

提示：

- 如果你只是想从统一内核映像启动，可以将 ESP 挂载到 `/efi`，并只将必要文件放置到 ESP 分区。

- 可以通过添加 `--cmdline /etc/kernel/*fallback_cmdline*` 到 `fallback_options` 以对后备映像使用不同的 cmdline（例如不带 `quiet` 参数）。
- 要省略嵌入内核命令行，请在 `*PRESET*_options=` 中添加 `--no-cmdline` 参数。内核参数将需要通过引导加载器传入。

注意：`*PRESET*_uki` 选项之前被称作 `*PRESET*_efi_image`，于 2022 年 11 月变更（参见 archlinux/mkinitcpio/mkinitcpio#134）；旧选项已被废弃，但现在尚能正常使用。

pacman 钩子

更新 systemd-stub（systemd包 的一部分），微码（包括 intel-ucode包 和 amd-ucode包）及 linux包 内核会自动重新构建 UKI，但你可能会想检查下 `/etc/pacman.d/hooks/` 下如用于 NVIDIA 驱动的其它 pacman 钩子。

构建 UKI

最后，确保用于 UKI 的目录存在，并重新生成 initramfs。例如，对于 *linux*预设：

```
# mkdir -p esp/EFI/Linux
# mkinitcpio -p linux
```

另外，可以移除 `/boot` 或 `/efi` 下多余的 `initramfs-*.img`。

kernel-install

Kernel-install 是 systemd包 提供的另一替代品，并依赖于 systemd-ukify包 构建统一内核映像。首先确保已正确配置了 kernel-install。

要生成 UKI，需要安装 systemd-ukify包，然后将 `kernel-install` 的 *layout* 设为 `uki`：

```
/etc/kernel/install.conf
layout=uki
```

注意：mkinitcpio 会负责创建 initrd，然后d #ukify 会进一步生成 UKI。

为 #ukify 进行的任何配置都必须在 `/etc/kernel/uki.conf` 中完成，才能被 kernel-install 使用，例如：

```
/etc/kernel/uki.conf
[UKI]
Splash=/usr/share/systemd/bootctl/splash-arch.bmp
```

注意：你也可以将 `/usr/lib/kernel/uki.conf` 模板复制到 `/etc/kernel/uki.conf`，然后将需要的部分取消注释。**不要**在该文件中设置命令行参数，否则设定的参数将被忽略。具体信息请参考 Kernel-install#Kernel command line。

也可以通过配置默认 `uki_generator` 来使用 mkinitcpio 生成 UKI：

```
/etc/kernel/install.conf
layout=uki
uki_generator=mkinitcpio
```

在这种情况下可以不使用 systemd-ukify包。你也可以使用其它 `initrd_generator`，具体信息请参考 kernel-install(8)。

为了使配置生效，需要重新安装当前使用的内核软件包。

dracut

请参考 dracut#统一内核映像和 dracut#升级内核时生成新 initramfs。

ukify

安装 systemd-ukify包。如需使用自动签名功能，请一并安装 sbsigntools包。由于 *ukify* 不能自行生成 initramfs，因此如有需求，需要单独通过 dracut，mkinitcpio 或 booster 进行生成。

最小可用示例如下：

```
# ukify build --linux=/boot/vmlinuz-linux \
              --initrd=/boot/initramfs-linux.img \
              --cmdline="quiet rw"
```

注意：如果使用了外置 initramfs 微码映像，必须将 `/boot/amd-ucode.img` 或 `/boot/intel-ucode.img` 放置到**第一位**，放在主 initramfs 映像之前，例如：`--initrd=/boot/intel-ucode.img --initrd=/boot/initramfs-linux.img`。

提示：

- 要跳过将 EFI 执行文件复制到 EFI 系统分区的步骤，可对 *ukify* 使用 `--output=*esp*/EFI/Linux/*filename*.efi` 命令行选项。
- 在使用 `--cmdline` 选项时，可在文件名前使用 `@` 符号来指定内核参数来源文件（以 `/etc/kernel/cmdline` 为例，使用 `--cmdline=@*/path/to/cmdline*`）

更多信息请参考 ukify(1)。

注意：要在内核/微码/initramfs 改变时自动更新 UKI，只需将 ukify 搭配如 kernel-install 或 mkinitcpio 使用。具体细节请参考之前的 #kernel-install 和 #mkinitcpio 章节。

手动构建

Put the kernel command line you want to use in a file, and create the bundle file using objcopy(1).

For microcode, first concatenate the microcode file and your initrd, as follows:

```
$ cat esp/cpu_manufacturer-ucode.img esp/initramfs-linux.img > /tmp/combined_initrd.img
```

When building the unified kernel image, pass in `/tmp/combined_initrd.img` as the initrd. This file can be removed afterwards.

注意：For IA32 UEFI, replace `/usr/lib/systemd/boot/efi/linuxx64.efi.stub` with `/usr/lib/systemd/boot/efi/linuxia32.efi.stub` in the following commands.

```
$ align="$(objdump -p /usr/lib/systemd/boot/efi/linuxx64.efi.stub | awk '{ if ($1 == "SectionAlignment"){print $2} }')"
$ align=$((16#$align))
$ osrel_offs="$(objdump -h "/usr/lib/systemd/boot/efi/linuxx64.efi.stub" | awk 'NF==7 {size=strtonum("0x"$3); offset=strtonum("0x"$4)} END {print size + offset}')"
$ osrel_offs=$((osrel_offs + "$align" - osrel_offs % "$align"))
$ cmdline_offs=$((osrel_offs + $(stat -Lc%s "/usr/lib/os-release")))
$ cmdline_offs=$((cmdline_offs + "$align" - cmdline_offs % "$align"))
```

```
$ splash_offs=$((cmdline_offs + $(stat -Lc%s "/etc/kernel/cmdline")))
$ splash_offs=$((splash_offs + "$align" - splash_offs % "$align"))
$ initrd_offs=$((splash_offs + $(stat -Lc%s "/usr/share/systemd/bootctl/splash-
arch.bmp")))
$ initrd_offs=$((initrd_offs + "$align" - initrd_offs % "$align"))
$ linux_offs=$((initrd_offs + $(stat -Lc%s "initrd-file")))
$ linux_offs=$((linux_offs + "$align" - linux_offs % "$align"))

$ objcopy \
    --add-section .osrel="/usr/lib/os-release" --change-section-vma .osrel=$(printf
0x%x $osrel_offs) \
    --add-section .cmdline="/etc/kernel/cmdline" \
    --change-section-vma .cmdline=$(printf 0x%x $cmdline_offs) \
    --add-section .splash="/usr/share/systemd/bootctl/splash-arch.bmp" \
    --change-section-vma .splash=$(printf 0x%x $splash_offs) \
    --add-section .initrd="initrd-file" \
    --change-section-vma .initrd=$(printf 0x%x $initrd_offs) \
    --add-section .linux="vmlinuz-file" \
    --change-section-vma .linux=$(printf 0x%x $linux_offs) \
    "/usr/lib/systemd/boot/efi/linuxx64.efi.stub" "linux.efi"
```

A few things to note:

- The offsets are dynamically calculated so no sections overlap, as recommended in .
- The sections are aligned to what the `SectionAlignment` field of the PE stub indicates (usually 0x1000).
- The kernel image must be in the last section, to prevent in-place decompression from overwriting the sections that follow, as stated in .

After creating the image, copy it to the EFI system partition:

```
# cp linux.efi esp/EFI/Linux/
```

为安全启动对 UKI 进行签名

sbctl

sbctl包 provides a kernel-install script, a mkinitcpio post-hook, and pacman hooks to sign updated binaries.

mkinitcpio

By using a mkinitcpio post hook, the generated unified kernel images can be signed for Secure Boot. Create the following file and make it executable:

```
/etc/initcpio/post/uki-sbsign
#!/usr/bin/env bash

uki="$3"
[[ -n "$uki" ]] || exit 0

keypairs=(/path/to/db.key /path/to/db.crt)

for (( i=0; i<${#keypairs[@]}; i+=2 )); do
    key="${keypairs[$i]}" cert="${keypairs[(( i + 1 ))]}"
    if ! sbverify --cert "$cert" "$uki" &>/dev/null; then
        sbsign --key "$key" --cert "$cert" --output "$uki" "$uki"
    fi
done
```

Replace `*/path/to/*db.key` and `*/path/to/*db.crt` with the paths to the key pair you want to use for signing the image.

ukify

安装 sbsigntools包，并在 `/etc/kernel/uki.conf` 中指定 `--secureboot-private-key` 和 `--secureboot-certificate`。

启动

注意：当启用安全启动时，内嵌 `.cmdline` 的统一内核映像会忽略所有传入的命令行参数（无论是通过启动项还是交互式传入）。在不启用安全启动时，通过命令行传入的选项会绕过掉内嵌的 `.cmdline`。

Limine

Limine 不会自动检测 UKI，但可以通过编辑 `limine.conf` 手动进行加载。

示例 1：从默认 EFI 系统分区启动 UKI

如果 UKI 位于 `*esp*/EFI/Linux/` 下，可以在 `limine.conf` 中添加如下配置：

```
limine.conf
/Arch Linux
  protocol: efi
  path: boot():/EFI/Linux/arch-linux.efi
```

示例 2：从其它分区启动 UKI

如果 UKI 文件位于不同的 FAT32 分区下，可以通过 `guid(*PARTUUID*)` 使用 PARTUUID 进行指定：

```
limine.conf
/Arch Linux
  protocol: efi
  path: guid(xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx):/EFI/Linux/arch-linux.efi
```

关于支持的路径和配置选项等更多信息，请参考 Limine Paths 文档。

systemd-boot

systemd-boot 会在 `*esp*/EFI/Linux/` 路径下搜索统一内核映像，无需额外配置。具体细节请参考 sd-boot(7) § FILES。

rEFInd

rEFInd 会自动检测到 EFI 系统分区下的 UKI，并可以进行加载。也可以在 `refind.conf` 中手动进行指定，参考如下：

```
esp/EFI/refind/refind.conf
menuentry "Arch Linux" {
    icon \EFI\refind\icons\os_arch.png
    ostype Linux
    loader \EFI\Linux\arch-linux.efi
}
```

注意，以该种方式启动时，是不会传入 `*esp*/EFI/refind_linux.conf` 中的内核参数的。如果 UKI 生成时没有使用 `.cmdline`，需要在启动项添加 `options` 一行指定内核参数。

GRUB

GRUB 可以连锁加载 EFI UKI，具体细节请参考 GRUB#Chainload 一个统一的内核镜像。

直接从 UEFI 启动

efibootmgr 可以为 *.efi* 文件创建 UEFI 启动项：

```
# efibootmgr --create --disk /dev/sdX --part partition_number --label "Arch Linux" --loader '\EFI\Linux\arch-linux.efi' --unicode
```

选项具体作用请参考 efibootmgr(8)。

注意：UEFI 标准使用反斜杠\ 作为路径分隔符，但 *efibootmgr* 可以自动转换 UNIX 风格的 / 路径分隔符。

参阅

- All Systems Go! talk on UKIs
- Unified kernel image specification
- mkinitcpio v31 and UEFI stubs

4.4 REFInd

难度 高级 | **篇幅** 28.2 KB | **所属** 启动加载程序

本节将讲到

- 安装
- 安装 rEFInd 启动管理器
- 配置
- 在已有的 Windows UEFI 安装中使用 rEFInd
- 工具

rEFInd 是一个 UEFI 能够启动 EFISTUB 内核的启动管理器。它是 rEFIt （不再维护）的一个分支并且针对非 Mac 硬件修复了若干问题。它被设计为平台无关，可启动多个操作系统。

注意：整篇文章中 `*esp*` 代表 EFI system partition 的挂载点。

安装

安装 refind包 包。

安装 rEFInd 启动管理器

rEFInd 附带了实现了对 ReiserFS, Ext2, Ext4, Btrfs, ISO-9660 and HFS+ **只读**支持的 UEFI drivers。而且 rEFInd 能够访问任何 UEFI 本身能够存取的文件系统，包括 FAT (由 UEFI 规范强制要求), 在 Mac 上的 HFS+ 和一些系统上的 ISO-9660。

额外的驱动可以参考 The rEFInd Boot Manager: Using EFI Drivers: Finding Additional EFI Drivers。

为了使用 rEFInd, 务必将它安装到 EFI 分区， 使用 rEFInd 安装脚本或者手动复制文件并设置启动入口.

警告：你的内核和 initramfs 必须在一个 rEFInd 能够读取的文件系统上。

以 rEFInd 安装脚本安装

rEFInd 包含有一个 *refind-install* 脚本来简化将你的 rEFInd 设置为默认 EFI 启动项的过程。这个脚本有几个选项用于处理不同的设置和 UEFI 实现。想要了解不同选项的含义，请参见 refind-install(8) 或者读安装脚本里面的注释。

对于许多系统来说，运行下面的命令就足够了：

```
# refind-install
```

此操作会尝试找到并挂载您的 ESP，将 rEFInd 的文件复制到 `*esp*/EFI/refind/`，并使用 efibootmgr 将 rEFInd 设置为默认的 UEFI 启动项。

或者您可以将 rEFInd 安装到默认/回退（fallback）启动路径 `*esp*/EFI/BOOT/bootx64.efi`。这对于可启动的 USB 驱动器或者在那些 *efibootmgr* 做的 NVRAM 更改有问题的系统上会有帮助：

```
# refind-install --usedefault /dev/sdXY
```

其中 `*/dev/sdXY*` 是你的 EFI 分区 (块设备, 不是挂载点).

提示：默认情况下 `refind-install` 只会为内核所在的文件系统安装驱动。 其他的文件系统需要手动安装，通过将 `/usr/share/refind/drivers_x64/` 复制到 `*esp*/EFI/refind/drivers_x64/`, 或者你可以以 `--alldrivers` 选项安装。 这对于可启动的 USB 驱动器有帮助。

将 rEFInd 的文件安装到 ESP 之后， 验证 rEFInd 已经在你的内核所在文件夹创建了包含内核参数的 `refind_linux.conf` 文件。如果你用了 `--usedefault` 选项，该文件不会被创建，请以 root 身份运行 `mkrlconf` 来创建它。

警告：当 `refind-install` 运行在chroot环境下 (例如：安装Arch Linux时的live环境) `/boot/refind_linux.conf` 内将会添加live系统的内核选项，而不是安装它的系统。

编辑 `/boot/refind_linux.conf` 并确保其中的 内核参数 对于你的系统是正确的，否则下次启动可能会出现内核错误。

查看示例：rEFInd#refind_linux.conf。

默认情况下，rEFInd 会扫描你的所有的驱动器（它有驱动的那些）并为它找到的每一个 EFI bootloader 添加一个启动入口，其中就包含你的内核（因为 Arch 默认启用了 EFISTUB ）。因此你在这时候就可能有一个可以启动的系统。

Secure Boot

要在 rEFInd 中使用 Secure Boot，请参考 Managing Secure Boot。

使用 PreLoader

要获取经过签名的 `PreLoader.efi` 和 `HashTool.efi` 二进制文件，请参考 Secure Boot#Set up PreLoader。

以 `--preloader */path/to/preloader*` 选项执行 `refind-install`。

```
# refind-install --preloader /usr/share/preloader-signed/PreLoader.efi
```

当你下次以 Secure Boot 开启状态启动时，HashTool 会启动并让你注册 rEFInd (`loader.efi`)、rEFInd 的驱动器 (如 `ext4_x64.efi`)和内核 (如 `vmlinuz-linux`) 的 hash。

更多信息参见 refind-install(8)。

提示：经过签名的 HashTool 只能够访问启动它的分区。 这意味着如果你的内核不在 EPS 上, 你就不能从 HashTool 注册它的 hash. 你可以使用 #KeyTool 来替代这种情况, 因为它可以将一个 hash 注册到 MokList 并且不被限制在一个分区。在使用之前，请记住要先注册 KeyTool 的 hash。

使用 shim

安装 shim-signedAUR。阅读 Secure Boot#shim，但是跳过所有的文件复制。

使用 hash

要只使用带有 hash 的 shim，请以 `--shim */path/to/shim*` 参数执行 `refind-install`：

```
# refind-install --shim /usr/share/shim-signed/shimx64.efi
```

当你下次以 Secure Boot 开启状态启动时, MokManager 会启动并让你注册 rEFInd (`loader.efi`)、rEFInd 的驱动器 (如 `ext4_x64.efi`)和内核 (如 `vmlinuz-linux`) 的 hash。

使用机器所有者密钥（Machine Owner Key）

要使用计算机所有者密钥（MOK）对 rEFInd 进行签名，请安装 sbsigntools包。

提示：如果您已经创建了一个计算机所有者密钥，则将文件命名为 `refind_local.key`（PEM 格式私钥）、`refind_local.crt`（PEM 格式证书）和 `refind_local.cer`（DER 格式证书），并放置在`/etc/refind.d/keys`目录中。

以 `--shim */path/to/shim*` 和 `--localkeys` 选项执行 `refind-install`：

```
# refind-install --shim /usr/share/shim-signed/shimx64.efi --localkeys
```

refind-install 会为你创建密钥并为它和它的驱动器签名。你需要用同一个密钥为内核签名，如：

```
# sbsign --key /etc/refind.d/keys/refind_local.key --
cert /etc/refind.d/keys/refind_local.crt --output /boot/vmlinuz-linux /boot/vmlinuz-
linux
```

提示：内核签名可以由 pacman 钩子自动化, 参见 Secure Boot#Signing the kernel with a pacman hook。

进入 MokManager 之后，将 `refind_local.cer` 添加到 MoKList。在 rEFInd 的安装目录下，如 `*esp*/EFI/refind/keys/refind_local.cer`，一个叫做 `keys` 的文件夹里面可以找到 `refind_local.cer`。

更多信息参见 refind-install(8)。

使用你自己的密钥

跟随 Secure Boot#Using your own keys 来创建密钥。

创建目录 `/etc/refind.d/keys` 并将 Signature Database (**db**) 密钥和证书放在其中。 将文件命名为： `refind_local.key` (PEM 格式私钥), `refind_local.crt` (PEM 格式证书) 和 `refind_local.cer` (DER 格式证书)。

在运行安装脚本时添加 `--localkeys` 选项，如：

```
# refind-install --localkeys
```

rEFInd EFI 二进制文件将会被用提供的密钥和证书签名。

手动安装

提示： rEFInd 能够以多种方式启动 Linux。参见 The rEFInd Boot Manager: Methods of Booting Linux。本节将阐述使用 EFISTUB 的过程。

注意：对于 32 位 EFI，请将 **x64** 替换为 **ia32**。

如果 `refind-install` 脚本没有正常工作，您可以手动设置 rEFInd。

首先，将可执行文件复制到 ESP:

```
# mkdir -p esp/EFI/refind
# cp /usr/share/refind/refind_x64.efi esp/EFI/refind/
```

如果想要将 rEFInd 安装到默认/回退（fallback) 启动路径，请在下面将 `*esp*/EFI/refind/` 替换为 `*esp*/EFI/BOOT/`，并将 rEFInd EFI 可执行文件复制到 `*esp*/EFI/BOOT/bootx64.efi`:

```
# mkdir -p esp/EFI/BOOT
# cp /usr/share/refind/refind_x64.efi esp/EFI/BOOT/bootx64.efi
```

然后使用 efibootmgr 来在 UEFI NVRAM 中创建一个启动入口，其中 `*/dev/sdX*` 和 `*Y*` 是你的 EFI 分区的设备和分区号。如果你要将 rEFI 安装到默认/回退（fallback) 启动路径 `*esp*/EFI/BOOT/bootx64.efi`，你可以跳过这一步。

```
# efibootmgr --create --disk /dev/sdX --part Y --loader /EFI/refind/refind_x64.efi --
label "rEFInd Boot Manager" --verbose
```

此时，你应该能重启到 rEFInd，但是它无法引导你的内核。如果你的内核不在你的 ESP 上，那么 rEFInd 可以挂载你的分区来查找它，只要它具有正确的驱动程序即可。

rEFInd 会从其安装目录中的子目录 `drivers` 和 `drivers_*arch*`（例如 `drivers_x64`）自动加载所有驱动程序。

```
# mkdir esp/EFI/refind/drivers_x64
# cp /usr/share/refind/drivers_x64/drivername_x64.efi esp/EFI/refind/drivers_x64/
```

现在，rEFInd应该为你的内核提供了一个启动项，但是不会传递正确的内核参数。设置#传递内核参数。现在，你应该可以使用 rEFInd 引导内核了。如果仍然无法启动或想要调整 rEFInd 的设置，则可以使用配置文件来更改许多选项：

```
# cp /usr/share/refind/refind.conf-sample esp/EFI/refind/refind.conf
```

该示例配置文件具有很好的注释并且不言自明。

除非你在配置文件中设置了`textonly`，否则你应该复制 rEFInd 的图标以摆脱难看的占位符：

```
# cp -r /usr/share/refind/icons esp/EFI/refind/
```

你可以尝试通过复制不同的字体并更改 `refind.conf` 中的 `font` 设置来进行尝试：

```
# cp -r /usr/share/refind/fonts esp/EFI/refind/
```

提示：在 rEFInd 中按 `F10`将会保存屏幕截图到 ESP 分区的根目录。

更新 rEFInd

Pacman 只更新在 `/usr/share/refind` 中的文件，不会将新文件复制到 ESP。 如果最初 `refind-install` 成功安装了 rEFInd，则可以再次运行以更新文件。 新的配置文件会被复制为 `refind.conf-sample`， 你可以使用一个 diff 工具选择、改变合并到你的配置文件。如果你的 rEFInd 需要#手动安装，你需要自己找出需要复制哪些文件。

Pacman 钩子

你可以使用一个 pacman 钩子来自动化更新过程:

```
/etc/pacman.d/hooks/refind.hook
[Trigger]
Operation=Upgrade
Type=Package
Target=refind

[Action]
Description = Updating rEFInd on ESP
When=PostTransaction
Exec=/usr/bin/refind-install
```

你可能需要为你的设置将 `Exec=`更改为正确的更新命令。如果您进行了#手动安装，则可以创建自己的更新脚本以使用钩子进行调用。

提示：如果你使用 #Secure Boot 设置 rEFInd，则可能需要在 `refind-install` 命令中额外添加选项 `--yes`。如果在禁用 Secure Boot 后执行该命令，它将防止该命令失败。更多信息参见 refind-install(8)。

注意：如果 ESP 未挂载到 `/boot`，而你依靠自动挂载来挂载它，请确保按照 EFI system partition#Alternative mount points 来预加载 `vfat` 模块。否则，如果 refind包与内核一起升级，ESP 将变得不可访问。

配置

rEFInd 的配置文件 `refind.conf` 存放在 rEFInd EFI 应用的同一目录下（通常为 `*esp*/EFI/refind` 或 `*esp*/EFI/BOOT`）。默认的配置文件包含了解释了所有选项的扩展注释，更详细的解释参考 Configuring the Boot Manager。

传递内核参数

有两种方法可以设置 rEFInd 传递给内核的内核参数。

对于 rEFInd 自动检测到的内核

对于自动检测到的内核，你可以在 `/boot/refind_linux.conf` 中显式指定内核参数，也可以依靠 rEFInd 识别 root 分区和内核参数的能力。更多信息参见Methods of Booting Linux: For Those With Foresight or Luck: The Easiest Method。

提示：

- 当 `/etc/os-release` 与内核位于同一分区时，rEFInd 会自动为启动项选择 Arch Linux图标 (`os_arch.png`)。如果你的 `/boot` 是一个单独的分区，参考 Configuring the Boot Manager: Setting OS Icons。
- rEFInd 不支持检测统一内核映像的分布。要为统一内核映像提供图标，请将 `/usr/share/refind/icons/os_arch.png` 复制到 `*esp*/EFI/Linux/` 并确保文件名匹配。例如，如果你有 `*esp*/EFI/Linux/Arch-linux.efi`，则将对应的图标命名为 `*esp*/EFI/Linux/Arch-linux.png`。

为了使 rEFInd 支持 Arch Linux 内核的命名方案并因此使其与各自的 initramfs 镜像相匹配，必须取消注释并编辑 `refind.conf` 中的 `extra_kernel_version_strings`选项。例如：

```
esp/EFI/refind/refind.conf
...
extra_kernel_version_strings linux-hardened,linux-zen,linux-lts,linux
...
```

注意：

- rEFInd 仅支持为每个内核检测一个 initramfs 镜像，这意味着它将不检测 fallback initramfs 或微码镜像，必须手动指定它们。
- 如果没有上述 `extra_kernel_version_strings` 行，`refind_linux.conf` 中的 `%v` 变量将不适用于 Arch Linux 内核。

refind_linux.conf

如果 rEFInd 自动检测到你的内核，你可以将包含内核参数的 `refind_linux.conf` 文件放置在与内核相同的目录中。你可以使用 `/usr/share/refind/refind_linux.conf-sample` 作为起点。`refind_linux.conf` 未注释的第一行将是内核的默认参数。随后的行将在子菜单中创建可使用 `+`, `F2`, 或 `Insert` 访问的条目。

或者，尝试以 root 身份运行 `mkrlconf`。它将会尝试在 `/boot` 中找到你的内核，并自动生成 `refind_linux.conf`。该脚本只会设置最基本的内核参数，因此请确保检查其创建的文件的正确性。

如果未指定 `initrd=` 参数，rEFInd 将通过在与内核相同的目录中搜索常用RAM磁盘文件名来自动添加它。如果需要多个 `initrd=` 参数，则必须在 `refind_linux.conf` 中手动指定它们。例如，一个在 initramfs 之前传递的微码：

```
/boot/refind_linux.conf
"Boot using default options"     "root=PARTUUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX rw add_efi_memmap initrd=boot\intel-ucode.img initrd=boot\amd-ucode.img initrd=boot\initramfs-%v.img"
"Boot using fallback initramfs"  "root=PARTUUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX rw add_efi_memmap initrd=boot\intel-ucode.img initrd=boot\amd-ucode.img initrd=boot\initramfs-%v-fallback.img"
"Boot to terminal"               "root=PARTUUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX rw add_efi_memmap initrd=boot\intel-ucode.img initrd=boot\amd-ucode.img initrd=boot\initramfs-%v.img systemd.unit=multi-user.target"
```

警告：

- `initrd` 路径是相对于内核所在文件系统的根目录。这可能是 `initrd=\boot\initramfs-%v.img` 或者 `initrd = initramfs-%v.img`（如果 `/boot` 是一个单独的分区，例如 ESP）。
- 在参数 `initrd` 中使用反斜杠 (`\`) 作为路径分隔符，否则内核可能无法找到 initramfs 镜像：`EFI stub: ERROR: Failed to open file: /boot/intel-ucode.img`。

注意： rEFInd用内核的版本替换 `refind_linux.conf` 中的 `%v}`（通过从文件名中提取if）。为了使 rEFInd 支持Arch Linux内核，必须按照 #对于 rEFInd 自动检测到的内核中的说明编辑 `esp/EFI/refind/refind.conf` 中的 `extra_kernel_version_strings`。

无需配置的情况

如果你仅将 rEFInd 安装到 ESP 上并毫不费力地启动它（例如通过 UEFI Shell 或 KeyTool，或直接从固件），您仍然可以通过自动检测从启动菜单，而无需进行任何配置。

这之所以可行，是因为 rEFInd 具有回退机制，可以：

- 通过 Discoverable Partitions Specification 或者 `/etc/fstab` 分辨 root 分区（对于 `root=` 参数）。

- 从 GPT partition attributes 检测内核选项 (`ro` 或者 `rw`)。（使用属性 `60` "read-only") 或者 `/etc/fstab`。

注意：rEFInd 不支持转义码 (如 `/etc/fstab` 中的 路径名空格)。

手动启动项

如果未自动检测到内核，或者你只想对菜单项的选项进行更多控制，则可以使用 `refind.conf` 中的 stanzas 来手动创建引导项。确保 `scanfor` 包含 `manual`，否则这些条目将不会出现在 rEFInd 的菜单中。内核参数使用 `options` 关键字设置。rEFInd 将使用 stanza 中的 `initrd` 关键字指定的文件附加 `initrd=` 参数。如果你需要其他 initrd（例如微码），则可以在 `options` 中指定它们（并且 `initrd` 关键字指定的会添加到末尾）。

手动启动项在 Creating Manual Boot Stanzas 中有详细解释。

```
esp/EFI/refind/refind.conf
...

menuentry "Arch Linux" {
	icon     /EFI/refind/icons/os_arch.png
	volume   "Arch Linux"
	loader   /boot/vmlinuz-linux
	initrd   /boot/initramfs-linux.img
	options  "root=PARTUUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX rw add_efi_memmap
initrd=boot\intel-ucode.img initrd=boot\amd-ucode.img"
	submenuentry "Boot using fallback initramfs" {
		initrd /boot/initramfs-linux-fallback.img
	}
	submenuentry "Boot to terminal" {
		add_options "systemd.unit=multi-user.target"
	}
}
```

您可能需要更改 `volume`以匹配文件系统的标签、PARTLABEL 或内核镜像所在分区的 PARTUUID。PARTUUID 必须大写的。有关分配卷标的示例，请参见 Persistent block device naming#通过标签。如果未指定 `volume`，则默认为启动 rEFInd 的卷（通常是EFI系统分区）。

警告：

- `loader` 和 `initrd` 的路径是相对于 `volume` 的根目录的相对路径。如果 `/boot` 是一个单独的分区（例如 ESP ），那么加载程序和 initrd 路径分别是 `/vmlinuz-linux` 和 `/initramfs-linux.img`。
- 在所有引用的 `initrd` 参数中使用反斜杠 (`\`) 作为路径分隔符，否则内核可能无法找到 initramfs 镜像：`EFI stub: ERROR: Failed to open file: /boot/initramfs-linux.img`。

在已有的 Windows UEFI 安装中使用 rEFInd

注意：在页面 Windows and Arch Dual Boot 中查看通常的注意事项。

rEFInd 兼容 UEFI Windows 安装时创建的 EFI 系统分区，因此没有必要创建或格式化另一个 FAT32 分区。只需挂载 Windows 的 ESP 并像往常一样安装 rEFInd。默认情况下，rEFInd 的自动检测功能应该识别任何现有的 Windows 引导程序。

注意：在某些情况下，Windows 的行为不同（低分辨率启动屏幕、OEM logo 被 Windows logo 替换、启动屏幕后出现黑屏、伪影）。如果您遇到这样的问题，请尝试在 `*esp*/EFI/refind/refind.conf` 中设置 `use_graphics_for +,windows` 或者将 `graphics on` 添加到Windows 引导项。

工具

rEFInd支持运行各种第三方工具。工具需要单独安装。在 `refind.conf` 中编辑 `showtools` 文件选择要显示的内容。

```
esp/EFI/refind/refind.conf
...
showtools shell, memtest, mok_tool, gdisk, fwupdate ...
...
```

UEFI shell

参见 Unified Extensible Firmware Interface#UEFI Shell。

复制 `shellx64.efi` 到 EFI 分区的根目录。

Memtest86

安装 memtest86-efiAUR 并将它复制到 `*esp*/EFI/tools/`。

```
# cp /usr/share/memtest86-efi/bootx64.efi esp/EFI/tools/memtest86.efi
```

密钥管理工具

rEFInd 能够检测 Secure Boot 密钥管理工具，如果他们被放在 EPS 上的 rEFInd 的文件夹内，如 `*esp*/` 或 `*esp*/EFI/tools/`。

HashTool

参照 #使用 PreLoader， `HashTool.efi` 将被放在 rEFInd 的文件夹内。

MokManager

参照 #使用 shim，MokManager 将被放在 rEFInd 的文件夹内。

KeyTool

安装 efitools包。

将 KeyTool EFI 二进制文件放在 `*esp*/` 或者 `*esp*/EFI/tools/` 中，并命名为 `KeyTool.efi` 或 `KeyTool-signed.efi`。

关于签名 `KeyTool.efi` 的指导，请参考 Secure Boot#Using KeyTool。

GPT fdisk (gdisk)

下载 gdisk EFI 应用并将 `gdisk_x64.efi` 复制到 `*esp*/EFI/tools/`。

fwupdate

安装并设置好 fwupd。

将 `fwupx64.efi` 和固件复制到 `*esp*/EFI/tools/`：

```
# cp /usr/lib/fwupd/efi/fwupdx64.efi esp/EFI/tools/
```

关机或重启

据报道，rEFInd 内置了关机和重启菜单项。由于这个工具列表是这个 wiki 中的同类工具中最广泛的，因此 UEFI shell 或其他 UEFI 引导管理器（如 systemd-boot ）的用户可能会对 poweroffreboot.efiAUR 感兴趣。

提示与小技巧

在 UEFI shell 中使用驱动

要在 UEFI shell 中使用 rEFInd 的驱动，请使用`load` 命令来加载它们，并使用 `map -r` 来刷新驱动器映射。

```
Shell> load FS0:\EFI\refind\drivers\ext4_x64.efi
Shell> map -r
```

现在你可以从 UEFI shell 访问你的文件系统了。

设置 efifb 分辨率

如果 `refind.conf` 中的分辨率被设置为一个不正确的值，在除了 Apple Mac 的所有系统中， rEFInd 都会展示一个支持的分辨率列表。对于 Apple Mac, 它会静默使用默认分辨率。

要确定 efifb 支持的帧缓冲区分辨率，请 gnu-efi包 将 `/usr/share/gnu-efi/apps/x86_64/modelist.efi` 复制到 ESP 的根目录。进入 UEFI shell 并运行 `modelist.efi`。

```
Shell> FS0:\modelist.efi
GOP reports MaxMode 3
 0: 640x480 BGRR pitch 640
*1: 800x600 BGRR pitch 800
 2: 1024x768 BGRR pitch 1024
```

在 `refind.conf` 中设置一个。重启并用 `dmesg | grep efifb` 来检查设置是否已被应用。

Btrfs 子卷支持

提示：确保安装了 `btrfs_x64.efi` 驱动程序, 可以将 `/usr/share/refind/drivers_x64/btrfs_x64.efi` 复制到 `*esp*/EFI/refind/drivers_x64/btrfs_x64.efi`来手动安装, 或者你也可以以 `refind-install /dev/sdx --alldrivers` 选项安装所有驱动程序。

警告：`btrfs_x64.efi` 不支持 `raid1c3/4`。

自动检测

要允许在 Btrfs 子卷上进行内核自动检测，请取消注释并编辑 `refind.conf` 中的 `also_scan_dirs`。

```
esp/EFI/refind/refind.conf
...
also_scan_dirs +,subvolume/boot
...
```

然后在 `refind_linux.conf` 中将 `subvol=*subvolume*` 添加到 `rootflags`，之后将 `*subvolume*` 添加到 initrd 路径之前。

```
/boot/refind_linux.conf
"Boot using standard options"  "root=PARTUUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX rw
rootflags=subvol=subvolume initrd=subvolume\boot\initramfs-%v.img"
```

手动启动项

如果将一个 btrfs 子卷作为根目录引导，请将子卷的路径预先添加到 loader 和 initrd 路径之前，并用 `rootflags=subvol=*root_subvolume*` 修改 `options` 行。在下面的示例中，root 已作为名为“ROOT”的btrfs子卷挂载（例如 `mount -o subvol=ROOT /dev/sdxY /mnt`）：

```
esp/EFI/refind/refind.conf
...
menuentry "Arch Linux" {
        icon     /EFI/refind/icons/os_arch.png
        volume   "[bootdevice]"
        loader   /ROOT/boot/vmlinuz-linux
        initrd   /ROOT/boot/initramfs-linux.img
        options  "root=PARTUUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX rw
rootflags=subvol=ROOT"
...
}
```

如果失败将导致以下错误信息：`ERROR: Root device mounted successfully, but /sbin/init does not exist.`

LoaderDevicePartUUID

从 0.13.1 版本开始，rEFInd 支持设置 UEFI 变量 LoaderDevicePartUUID。启用此选项允许 systemd-gpt-auto-generator(8) 自动挂载 EFI 系统分区，而无需在 `/etc/fstab` 中指定它。请参阅 systemd#GPT partition automounting。

对于 rEFInd， 要设置 `LoaderDevicePartUUID`, 请编辑 `refind.conf` 并取消注释 `write_systemd_vars true`:

```
esp/EFI/refind/refind.conf
...
write_systemd_vars true
...
```

您可以通过使用 `cat /sys/firmware/efi/efivars/LoaderDevicePartUUID-4a67b082-0a`

`4c-41cf-b6c7-440b29bb8c4f` 检查它的值，或者通过查看 `bootclt` 输出中 “Boot loader sets ESP partition information” 的状态来验证它是否已设置。

疑难解答

Apple Macs

AUR 上的 mactel-bootAUR 是 "bless" 工具的实验性替代品。如果它不能正常工作，在 OS X 中使用 "bless" 来将 rEFInd 设置为默认启动项。

```
 # bless --setBoot --folder esp/EFI/refind --file esp/EFI/refind/refind_x64.efi
```

VirtualBox

6.1 版本之前的 VirtualBox 将只引导默认的 `*esp*/EFI/BOOT/bootx64.efi` 路径，因此 `refind-install` 至少需要与 `--usedefault` 选项一起使用。有关详细信息，请参阅 VirtualBox/Install Arch Linux as a guest#Installation in EFI mode on VirtualBox < 6.1。

参见

- The rEFInd Boot Manager by Roderick W. Smith.
- Wikipedia:rEFInd
- `/usr/share/refind/docs/README.txt`
- rEFInd discussion forum on Sourceforge

4.5 Systemd-boot

难度 高级 | **篇幅** 28.5 KB | **所属** 启动加载程序

本节将讲到

- 支持的文件系统
- 安装
- 配置
- 小提示
- 排除问题

systemd-boot(7)（有时被称为 *sd-boot*），曾用名 **gummiboot** （德语里“橡皮筏”的意思），是一款易于配置的 UEFI 引导加载程序。它提供了一个用于选择启动项的文本菜单，以及一个用于配置内核命令行的编辑器。

注意，*systemd-boot* 只能从安装到的 EFI 系统分区或同一硬盘上的扩展引导加载程序分区（XBOOTLDR 分区）启动 EFI 可执行程序（例如 Linux 内核 EFI boot stub，UEFI shell，GRUB 或者 Windows Boot Manager）。

注意：本文将使用 `*esp*` 指代 EFI 系统分区的挂载点，以及使用 `*boot*` 指代可选 XBOOTLDR 分区的挂载点。下文将假设你已经 chroot 到了系统挂载点下。

支持的文件系统

systemd-boot 从固件继承了文件系统兼容性（例如至少支持 FAT12，FAT16 和 FAT32），还可以加载 `*esp*/EFI/systemd/drivers/` 目录下的 UEFI 驱动。

安装

systemd-boot 随 systemd包 包一同安装，其为 base包 元软件包的依赖，因此无需手动安装额外软件包。

安装 UEFI 启动管理器

要安装 *systemd-boot*，首先确保启动方式是 UEFI 模式，可以访问 UEFI 变量。用 `efivar --list` 命令进行检查，如果没有安装 efivar包，使用 `ls /sys/firmware/efi/efivars`（如果目录存在，则表明系统是以 UEFI 模式启动的）。

使用 bootctl(1) 将 *systemd-boot* 安装到 ESP：

```
# bootctl install
```

这将把 *systemd-boot* UEFI 启动管理器复制到 ESP，同时为其创建一项 UEFI 启动入口，并将其设置为 UEFI 启动顺序的第一项。

- 在 x64 UEFI 环境中，`/usr/lib/systemd/boot/efi/systemd-bootx64.efi` 将被复制到 `*esp*/EFI/systemd/systemd-bootx64.efi` 和 `*esp*/EFI/BOOT/BOOTX64.EFI`。
- 在 IA32 UEFI 环境中，`/usr/lib/systemd/boot/efi/systemd-bootia32.efi` 将被复制到 `*esp*/EFI/systemd/systemd-bootia32.efi` 和 `*esp*/EFI/BOOT/BOOTIA32.EFI`。
- 原有的旧版 `systemd-bootx64.efi` 将保留作为备用，并被复制为 `systemd-boot-fallbackx64.efi`，同时系统会自动创建一个名为“Fallback Linux Boot Manager”的 UEFI 启动项。

UEFI 启动选项将被命名为“Linux Boot Manager”，根据 UEFI 位数不同，启动选项将指向到 ESP 的 `\EFI\systemd\systemd-bootx64.efi` 或 `\EFI\systemd\systemd-bootia32.efi` 位置下。

注意：

- 在运行 `bootctl install` 时，*systemd-boot* 会尝试在 `/efi`，`/boot` 和 `/boot/efi` 目录下寻找 ESP。可以通过 `--esp-path=*esp*` 参数指定 `*esp*` 目录（详细信息请参考 bootctl(1) § OPTIONS）。

- 安装 *systemd-boot* 将覆盖现有的 `*esp*/EFI/BOOT/BOOTX64.EFI`（或是 IA32 UEFI 下的 `*esp*/EFI/BOOT/BOOTIA32.EFI`），例如 Microsoft 版本的文件。
- 当在 pid 命名空间中运行时（例如使用 arch-chroot(8) 的非 systemd 模式），*bootctl*会禁止操作 UEFI 变量或启动项。若要在 chroot 环境中创建启动项，请改用 `arch-chroot -S`.

要完成安装，请配置 *systemd-boot*。

通过 XBOOTLDR 安装

**此页面或章节适合移动到 分区#多分区。**

附注： 所有分区相关的信息都需要移到“分区”页，仅保留与 systemd-boot 相关的步骤。（在 Talk:Systemd-boot 讨论）

可以单独创建一个“Linux extended boot”（XBOOTLDR）类型的 /boot 分区将内核和 initramfs 从 ESP 中分离出来，有助于在现有 ESP 过小的情况下配置 Arch + Windows 双系统。这亦可用于 Grml 或 Archiso等场景。

跟随通常步骤配置 ESP，然后在同一物理硬盘上为 XBOOTLDR 创建另一分区。XBOOTLDR 分区的类型 GUID 必须是 `bc13c2ff-59e6-4262-a352-b275fd6f7172`（gdisk 下是 `ea00`，fdisk 下是 `xbootldr`）。XBOOTLDR 的容量必须至少为要安装的所有内核的总大小。

注意：

- *systemd-boot* 不会像 ESP 那样检查 XBOOTLDR 的文件系统，因此可以使用你的 UEFI 实现可读取的任意文件系统类型。
- 在启用“快速启动”时，UEFI 可能会跳过加载除 ESP 外的所有分区，可能会导致 *systemd-boot*无法在 XBOOTLDR 分区上找到启动项。在这种情况下，请禁用“快速启动”。
- XBOOTLDR 分区必须与 ESP 位于同一物理硬盘，否则 *systemd-boot* 将无法识别到该分区。

在安装时，将 ESP 挂载到 `/mnt/efi`，将 XBOOTLDR 分区挂载到 `/mnt/boot`。

chroot 后，执行：

```
# bootctl --esp-path=/efi --boot-path=/boot install
```

最后配置 *systemd-boot*。

更新 EFI 启动管理器

每当 *systemd-boot* 有新版本时，用户可以选择重新安装启动管理器。该操作可以手动或自动进行，具体方式将在下文中描述。

注意：UEFI 启动管理器是一个独立 EFI 可执行文件，任意版本都可以被用于启动系统（由于 pacman 仅会安装 *systemd-boot* 安装器而不是 *systemd-boot* 自身，因此该项不适用于部分升级）。但是，更新版本可能会加入新功能或错误修正，因此建议持续更新 *systemd-boot*。

警告：如果你启用了 安全启动，你需要在更新引导加载程序后为其签名。请查看下方#为安全启动进行签名一节的相关说明。

手动更新

使用*bootctl* 更新 *systemd-boot*：

```
# bootctl update
```

注意：与 `bootctl install` 类似，*systemd-boot* 会尝试在 `/efi`，`/boot` 和 `/boot/efi` 三个位置下寻找 ESP。可以用 `--esp-path=*esp*` 参数指定 `*esp*` 位置。

自动更新

如果你需要自动更新 *systemd-boot*，你可以尝试使用 systemd 服务或 Pacman 钩子，下方介绍了这两种方法。

systemd 服务

在版本 250 后， systemd包 添加了 `systemd-boot-update.service`。启用这个服务后将在每次启动系统时执行以下命令：

```
# bootctl --variables=no --graceful update
```

与手动更新类似，它会在 `/efi`、`/boot` 和 `/boot/efi` 目录下寻找 ESP。该命令将在 `/usr/lib/systemd/boot/efi/` 目录下存在新版本时更新 ESP 中安装的所有 systemd-boot。它会先寻找以 `.efi.signed` 结尾的 systemd-boot 文件，以允许用户为安全启动对映像进行签名。

pacman 钩子

软件包 systemd-boot-pacman-hookAUR 提供了一个 Pacman 钩子，将在每次更新 systemd包 后自动执行。该钩子与 systemd 服务方式不同，它会在 systemd包 被更新时立即尝试更新引导管理器。

如果不想安装 AUR 包，可以在 `/etc/pacman.d/hooks/` 目录下手动添加以下文件：

```
/etc/pacman.d/hooks/95-systemd-boot.hook
[Trigger]
Type = Package
Operation = Upgrade
Target = systemd

[Action]
Description = Gracefully upgrading systemd-boot...
When = PostTransaction
Exec = /usr/bin/systemctl restart systemd-boot-update.service
```

为安全启动进行签名

如果你启用了安全启动，你需要添加一个 Pacman 钩子以在更新后自动为其重新签名：

```
/etc/pacman.d/hooks/80-secureboot.hook
```

```
[Trigger]
Operation = Install
Operation = Upgrade
Type = Path
Target = usr/lib/systemd/boot/efi/systemd-boot*.efi

[Action]
Description = Signing systemd-boot EFI binary for Secure Boot
When = PostTransaction
Exec = /bin/sh -c 'while read -r f; do /usr/lib/systemd/systemd-sbsign sign --private-key /path/to/keyfile.key --certificate /path/to/certificate.crt --output "${f}.signed" "$f"; done;'
Depends = sh
Depends = sbsigntools
NeedsTargets
```

将 `*/path/to/keyfile.key*` 和 `*/path/to/certificate.crt*` 替换为你的签名密钥和证书，具体信息可参考 systemd-sbsign(1)。

已经创建的 `/usr/lib/systemd/boot/efi/systemd-boot*.efi.*signed*` 会自动地被 `bootctl install` 和 `bootctl update` 识别并使用。参见 bootctl(1) § SIGNED .EFI FILES.

另一个选择是，使用 UEFI/安全启动#sbctl.

配置

提示：

- 修改配置后，可以不带参数执行 `bootctl` 来确保 systemd-boot 能够正常读取配置内容。
- 可使用 kernel-install 工具自动生成与 systemd-boot 兼容的配置文件。本节说明如何进行手动配置。

启动选单配置

配置文件保存于 `*esp*/loader/loader.conf`，具体信息可参考 loader.conf(5) § OPTIONS 。

以下是一个简单的示例：

```
esp/loader/loader.conf
default  arch.conf
timeout  4
console-mode max
editor   no
```

提示：

- `default` 和 `timeout` 可在启动选单中修改，变更将覆盖保存到 `LoaderEntryDefault` 和 `LoaderConfigTimeout` 这两个 UEFI 变量中。
- `bootctl set-default ""` 和 `bootctl set-timeout ""` 可分别用于清除覆盖了 `default` and `timeout` 选项的 UEFI 变量。
- 如果你设置了 `timeout 0`，可以通过按下空格键来访问启动菜单。
- 基本配置文件示例位于 `/usr/share/systemd/bootctl/loader.conf`。

- 如果在选择启动项页面时菜单显示异常或分辨率不对，可以尝试将 `console-mode` 设置为 `auto`（启发式选择最佳分辨率），`keep`（保持固件提供的分辨率）或 `2`（尝试使用第一个非 UEFI 标准的分辨率）。

记住上一次的启动项

可以将默认启动项设为 `@saved` 来记住上次使用的启动项。该选项对 Windows 双系统或 Windows 更新自动启动到了 Linux 的情况非常有用。

```
esp/loader/loader.conf
default @saved
...
```

更多信息请参考 loader.conf(5)。

增加启动选项

systemd-boot 会在它启动的 EFI 系统分区下的 `/loader/entries/` 目录和同硬盘下的 XBOOTLDR 分区中寻找 *.conf* 文件。

注意：

- `*esp*/loader/entries/*.conf` 下的启动项只能调用 `*esp*` 下的文件（例如内核，initramfs，映像等），`*boot*/loader/entries/*.conf` 下的启动项也一样只能调用 `*boot*` 下的文件。
- 文件路径参数是相对于 EFI 系统分区或 XBOOTLDR 分区的根的。例如，如果你的 EFI 系统分区或 XBOOTLDR 挂载到了 `/boot` 目录，那么就必须在 `linux` 参数中将 `/boot/vmlinuz-linux` 指定为 `/vmlinuz-linux`。
- 启用安全启动后，内嵌 `.cmdline` 的统一内核映像（UKI）将忽略所有传入的命令行选项（无论是使用 `options` 传入启动选项还是交互式传入的）。当未启用安全启动时，通过命令行传入的选项会覆盖掉 `.cmdline` 内置的选项。

以下为从卷启动 Arch 的启动选项文件示例，其中卷的 UUID 为 `*xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx*`：

```
esp/loader/entries/arch.conf
title   Arch Linux
linux   /vmlinuz-linux
initrd  /initramfs-linux.img
options root=UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx rw
esp/loader/entries/arch-fallback.conf
title   Arch Linux (fallback initramfs)
linux   /vmlinuz-linux
initrd  /initramfs-linux-fallback.img
options root=UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx rw
```

所有配置选项可参考引导加载器规范。

systemd-boot 会在启动时自动搜索位于 `/EFI/Microsoft/Boot/Bootmgfw.efi` 的 **Windows Boot Manager**，固件中的 **Apple macOS Boot Manager**，`/shellx64.efi`（UEFI shell）和 `/EFI/BOOT/bootx64.efi`（**EFI Default Loader**），同时也会在 `/EFI/Linux/` 内查找内核文件。在检测到后，会自

动生成名称分别为 `auto-windows`，`auto-osx`，`auto-efi-shell` 和 `auto-efi-default` 的启动选项，因此这些选项不需要手动配置引导器。但和 rEFInd 不同，不会为其它 EFI 应用程序创建启动选项，所以这些还需要进行进一步设置。

提示：

- 可以用 `bootctl list` 列出所有可用启动选项。
- 启动选项配置示例位于 `/usr/share/systemd/bootctl/arch.conf` 。
- 适用于如 LVM，LUKS，dm-crypt 或 Btrfs 的内核参数可在对应的页面中获取。

注意：如果使用了外置微码 initramfs 映像（如使用了 Booster 作为 initramfs 生成器），那么必须在单独的 `initrd` 中指定 `/boot/amd-ucode.img` 或`/boot/intel-ucode.img`，并将其放置到主 initramfs 之前的**首位**。

UEFI Shells 或其他 EFI 应用程序

如果你通过 edk2-shell包 安装了 UEFI shell，那么在对应 EFI 文件放置到了 `*esp*/shellx64.efi` 的情况下 *systemd-boot* 会自动检测到并为其创建新启动选项。 要启用自动检测，可以在安装软件包后执行如下命令：

```
# cp /usr/share/edk2-shell/x64/Shell.efi /boot/shellx64.efi
```

另外如果你安装了其他 EFI 应用程序到 ESP，也可以像这样进行加载：

注意：`efi` 参数的文件路径是相对于你的 EFI 系统分区的。如果你的 EFI 系统分区挂载到了 `/boot`，且你的 EFI 二进制文件位于 `/boot/EFI/xx.efi` 和 `/boot/yy.efi`，那么你需要指定对应参数分别为 `efi /EFI/xx.efi` 和 `efi /yy.efi`。

```
esp/loader/entries/fwupd.conf
title  Firmware updater
efi     /EFI/tools/fwupdx64.efi
esp/loader/entries/gdisk.conf
title  GPT fdisk (gdisk)
efi     /EFI/tools/gdisk_x64.efi
```

Memtest86+

首先需要安装 memtest86+-efi包。如果使用了安全启动，需要同时对 EFI 二进制文件进行签名。

```
esp/loader/entries/memtest.conf
title Memtest86+
efi /memtest86+/memtest.efi
```

网络引导

systemd-boot 可以串联加载网络引导。下载 `ipxe-arch.efi` EFI 二进制文件和签名，验证并将其放置到如 `*esp*/EFI/arch_netboot/arch_netboot.efi` 的位置下：

```
esp/loader/entries/arch_netboot.conf
title Arch Linux Netboot
efi /EFI/arch_netboot/arch_netboot.efi
```

GRUB

systemd-boot 可以串联加载 GRUB。`grubx64.efi` 二进制文件的位置与安装 GRUB 到 ESP 时使用的 `--bootloader-id=` 参数一致。

```
esp/loader/entries/grub.conf
title GRUB
efi /EFI/GRUB/grubx64.efi
```

从其它硬盘启动

systemd-boot 不能从它启动的 ESP 或 XBOOTLDR 分区所在硬盘外的分区中启动 EFI 二进制文件，但可以引导其它 UEFI shell 进行这一操作。

首先，按照上面的步骤安装 edk2-shell包。接着，在 UEFI shell 环境下，使用 *map* 命令获取带有对应 PARTUUID 的分区的 **FS alias**（例如 HD0a66666a2、HD0b、FS1 或 BLK7）并记录下来。

下一步，使用 `exit` 命令启动回到 LInux 环境，然后创建一条新启动选项来通过 UEFI shell 启动目标 EFI 应用：

```
esp/loader/entries/windows.conf
title   Windows
efi     /shellx64.efi
options -nointerrupt -nomap -noversion HD0b:EFI\Microsoft\Boot\Bootmgfw.efi
```

确保 `efi` 路径和复制到 *esp* 路径下的 `shellx64.efi` 位置一致。顺带一提，可以将 `shellx64.efi` EFI 文件移动到其它位置来防止 *systemd-boot* 自动创建启动选项。

将 `HD0b` 替换为之前记录的 *FS alias*。

- `-nointerrupt` 选项可以避免通过 `Ctrl+c` 选项终端目标 EFI 程序允许。
- `-nomap -noversion` 选项会隐藏掉默认 UEFI shell 欢迎信息。
- 如需让 UEFI shell 在目标 EFI 程序退出后（如出现错误等原因）自动回到启动引导器，可以添加 `-exit` 选项。
- 如果 UEFI shell 还会出现无用输出，可以添加 `-noconsoleout` 选项。

启动到 UEFI 固件设置

如果你设备的固件支持从操作系统重启到固件设置，那 systemd-boot 会自动检测到并添加启动到 UEFI 固件设置的选项。

为内核参数编辑器加上密码保护

你也可以安装 systemd-boot-passwordAUR，它支持 `password` 基本配置选项。使用 `sbpctl generate` 可以为该选项生成值。

使用如下命令安装 *systemd-boot-password*：

```
# sbpctl install esp
```

启用编辑器后，系统会提示你输入密码，然后才能编辑内核参数。

小提示

启动选单中的按键操作

在启动选单中，你可以使用 `t` 和 `T` 调整超时时间，使用 `e` 编辑当前启动项的内核参数。按下 `h` 可以看到一个简略的快捷键列表，完整的启动选单内可用快捷键列表可参考 systemd-boot(7) § KEY BINDINGS 。

选择下一次启动选项

启动管理器与 systemctl 命令集成，允许你选择重启后的启动选项。举个例子，假设你构建了一个自定义内核，并创建了一个启动项文件 `*esp*/loader/entries/arch-custom.conf` 来启动它，只需执行：

```
$ systemctl reboot --boot-loader-entry=arch-custom.conf
```

然后系统就会重启到对应的启动项，同时系统后续启动将保留现有设定不变。所有可用启动项清单可通过 `--boot-loader-entry=help` 选项查看。

可以通过以下命令直接启动到主板固件：

```
$ systemctl reboot --firmware-setup
```

统一内核映像

位于 `*esp*/EFI/Linux/` 的统一内核映像会自动地被 systemd-boot 识别，无需在 `*esp*/loader/entries` 中添加条目。需要注意的是，拥有 `.efi` 扩展名的统一内核镜像才会被 systemd-boot 识别。

提示：如果在 `*esp*/loader/loader.conf` 中未设置 `default`，则会优先启动位于 `*esp*/loader/entries/` 的文件。请移除这些条目，或使用完整档名来设置预设项目，例如 `default arch-linux.efi`

Grml on ESP

注意：以下指令并非仅限于 Grml，稍作调整后，也能用来安装其他系统，例如：SystemRescueCD

提示：有可用 AUR ：grml-systemd-bootAUR。

Grml 是精简的 Live 系统，包含一系列用于系统管理和救援的软件。

如需将 Grml 安装到 ESP，只需将 iso 镜像中的内核文件 `vmlinuz`、initramfs 镜像 `initrd.img` 和压缩镜像 `grml64-small.squashfs` 复制到 ESP.

首先，下载 grml64-small.iso 并挂载（下文中称挂载点为 mnt）；内核和 initramfs 位于 `*mnt*/boot/grml64small/`，压缩镜像位于 `*mnt*/live/grml64-small/`。

然后，在你的 ESP（EFI 系统分区）中新建一个 Grml 目录。

```
# mkdir -p esp/grml
```

将上文提到的文件复制到目录中：

```
# cp mnt/boot/grml64small/vmlinuz esp/grml
# cp mnt/boot/grml64small/initrd.img esp/grml
# cp mnt/live/grml64-small/grml64-small.squashfs esp/grml
```

最后在 `*esp*/loader/entries` 中创建一个 `grml.conf` 文件，以在 systemd-boot 中创建一个启动项：

```
esp/loader/entries/grml.conf
title   Grml Live Linux
linux   /grml/vmlinuz
initrd  /grml/initrd.img
options apm=power-off boot=live live-media-path=/grml/ nomce net.ifnames=0
```

要查看所有可用引导选项，请参考 cheatcode for Grml。

Archiso on ESP

提示：有可用的 AUR：archiso-systemd-bootAUR。

与 Grml 相同，也可以使用 Arch Linux 的 ISO 建立启动环境。为此，需要将 ISO 文件中的以下内容复制到 EFI 系统分区（ESP）：

内核文件：`vmlinuz-linux`

初始内存盘：`initramfs-linux.img`

压缩的根文件系统映像：`airootfs.sfs`

这些文件是系统启动所需的基本组件。后续步骤将说明如何挂载 ISO 并执行文件复制操作。

第一步，下载 archlinux-YYYY.MM.DD-x86_64.iso

然后，在 EFI 系统分区（ESP）中创建一个名为 archiso 的目录，用于存放这些文件：

```
# mkdir -p esp/EFI/archiso
```

将 `arch` 目录中所有内容解压或复制到此文件夹中：

```
# bsdtar -v -x --no-same-permissions --strip-components 1 -f archlinux-YYYY.MM.DD-x86_64.iso -C esp/EFI/archiso arch
```

最后在 `*esp*/loader/entries` 中创建一个 `arch-rescue.conf` 文件，以便在 systemd-boot 中添加一个启动项：

```
esp/loader/entries/arch-rescue.conf
title   Arch Linux (rescue system)
linux   /EFI/archiso/boot/x86_64/vmlinuz-linux
initrd  /EFI/archiso/boot/x86_64/initramfs-linux.img
options archisobasedir=/EFI/archiso
archisosearchfilename=/EFI/archiso/boot/x86_64/vmlinuz-linux
```

如需了解可用的开机选项，参见 README.bootparams for mkinitcpio-archiso。

在启用了安全启动的系统制作 Arch Linux 恢复环境

官方的 Arch ISO 目前不支持安全启动。所以，进入 ISO 进行恢复或者维护前必须禁用安全启动。这也会破坏系统的安全性，并不是个好办法。

一个可能的选择是使用 mkosi 创建签名的统一内核映像，前提是系统上的安全启动已经正确配置并可以运行。这可以让你引导进一个签名的 Arch 恢复环境而无需禁用安全启动或随身携带一个 ISO USB 驱动器。

https://swsnr.de/archlinux-rescue-image-with-mkosi/ 描述了配置兼容安全启动的 Arch 恢复镜像的方法。开箱即用的 mkosi 配置可在 https://codeberg.org/swsnr/rescue-image 获取，可以自行添加软件包。

在 BIOS 系统上使用 systemd-boot

如果需要一个符合 The Boot Loader Specification 的 BIOS 系统启动加载器，也可以使用 systemd-boot. Clover 支持在 BIOS 系统中启动并模拟一个 UEFI 环境（以便使用 systemd-boot）.

排除问题

systemd-boot 无法显示我的启动项

该问题可能是由配置文件问题（如内核路径错误）导致的。可以执行以下命令进行检查：

```
# bootctl
```

在传统启动（BIOS 模式）下安装

注意：不建议进行该操作!

如果你以 BIOS 模式启动电脑，你还是可以正常安装 *systemd-boot*，但需要在安装后手动向你的固件提供如何启动 *systemd-boot* EFI 文件的相关信息,为此你需要下列工具之一：

- 一个 UEFI Shell
- 你的 UEFI 固件设置中提供了更改启动选项的选项.
- 如果 UEFI 没有其它启动项，某些固件会直接使用 `*esp*/EFI/BOOT/BOOTX64.EFI` 。

满足条件后，进入你的 UEFI Shell 或是 UEFI 固件设置，修改你的默认 EFI 启动加载器为 `*esp*/EFI/systemd/systemd-bootx64.efi` 。

注意：在某些 Dell Latitude 计算机上，UEFI 固件设置界面提供了设置 UEFI 启动所需的工具,而 EFI Shell 无法修改那些设置.

通过 efibootmgr 手动添加启动选项

如果运行`bootctl install`命令失败,你可以通过 efibootmgr包手动增加选项:

```
# efibootmgr --create --disk /dev/sdX --part Y --loader '\EFI\systemd\systemd-bootx64.efi' --label "Linux Boot Manager" --unicode
```

用 EFI 系统分区的设备名称替换 `/dev/sd*XY*` 。

在 Windows 上通过 bcdedit 添加启动选项

如需从 Windows 添加 UEFI 启动入口，你可以用管理员权限执行这些指令：

```
> bcdedit /copy "{bootmgr}" /d "Linux Boot Manager"
> bcdedit /set "{guid}" path \EFI\systemd\systemd-bootx64.efi
```

用第一条指令返回的 id 替换 `*guid*`. 你也可以使用以下指令将它设置为默认入口：

```
> bcdedit /default "{guid}"
```

在 Windows 升级后看不到启动菜单

参阅 UEFI#Windows 改变了启动次序。

添加 Windows BitLocker TPM 解锁支持

在 *loader.conf* 中添加以下内容以阻止 BitLocker 请求恢复密钥：

```
esp/loader/loader.conf
reboot-for-bitlocker yes
```

这一步会设定 *BootNext* UEFI 变量，从而无需使 BitLocker 请求恢复密钥就能加载 *Windows Boot Manager*。该操作只需进行一次，且 *systemd-boot* 仍是默认引导加载程序。如果已自动检测到 Windows，则无需将其指定为条目。

注意，这是实验性功能，使用前请查阅 loader.conf(5)。

参阅

- https://systemd.io/BOOT/
- https://bbs.archlinux.org/viewtopic.php?id=254374
- https://uapi-group.org/specifications/specs/boot_loader_specification/

4.6 GRUB Legacy

难度 高级 | **篇幅** 30.2 KB | **所属** 启动加载程序

本节将讲到

- 安装
- 升级到 GRUB2
- 配置
- 安装启动器

- 小技巧

GRUB Legacy 是一个原本由 GNU Project 维护的多启动引导器，其前身为 GRUB (GRand Unified Bootloader), 最初由 Erich Stefan Boleyn 设计和实现。

简单的说，**启动引导程序**是计算机启动时运行的第一个程序。它可用于选择操作系统分区上的不同内核，也可用于向这些内核传递启动参数。然后内核逐个初始化操作系统的其余部分。

警告：上游已停止维护 GRUB Legacy 并且 Arch 官方也停止支持 (见这条新闻). 我们推荐用户使用 GRUB(2) 或 Syslinux 来替代。见 升级到 GRUB2.

注意：如果你在你的系统里把 grub-legacy 安装为包 grub-0.97, 那么在软件包升级时会升级为包 grub包 \-2.xx (GRUB2). 如果你想继续使用 grub-legacy, 从 AUR 安装包 grub-legacyAUR 以避免冲突。升级过程中只改变了 `/usr/lib/grub/` 里的文件，grub-legacy 安装到 `/boot/grub` 的文件和 MBR 并未被移除。只需重命名 `/boot/grub/menu.lst.pacsave` 为 `/boot/grub/menu.lst` 即可启动进入 grub-legacy.

安装

GRUB Legacy 已从官方源移除以支持 GRUB version 2.x. 但依然可以通过 grub-legacyAUR 安装。

另外，若想 GRUB 以引导器工作它必须被安装到设备或分区的引导扇区。这在安装启动器部分有所提及。

升级到 GRUB2

必须升级吗?

不是必须的。老 GRUB 不会从系统删除，照旧工作。

然而，和其它不被支持的软件包一样，后续的 bug 将无人修复。所以建议所有的用户抽时间升级到 GRUB version 2.x, 或其它支持的 Boot loader.

GRUB legacy 不支持 GPT 硬盘，Btrfs 文件系统和 UEFI 固件。

如何升级

升级 GRUB Legacy 到 GRUB version 2.x 和在已有 Arch Linux 上安装 GRUB 一样。详细教程见这里。

区别

- GRUB Legacy 和 GRUB 的命令之间有许多区别。执行命令之前请学习 GRUB commands (例如 "find" 被替换为 "search").

- GRUB 现在是*模块化*的并再也不需要 "stage 1.5". 因此，引导器本身受到限制 -- 若想扩展功能必须加载模块 (例如为了添加 LVM 或 RAID 支持).
- GRUB Legacy 和 GRUB 显示的设备名不同。分区从 1 而不是 0 开始计数，但驱动器依然从 0 开始，并添加了分区表前缀。例如，`/dev/sda1` 会显示为 `(hd0,msdos1)` (对于 MBR) 或 `(hd0,gpt1)` (对于 GPT).
- GRUB 显著比 GRUB legacy 大 (占用 `/boot` ~13 MB). 如果你从单独的 `/boot` 分区启动并且它小于 32 MB, 你就会碰到硬盘空间问题，pacman 不会安装新的内核。

备份重要数据

尽管 GRUB 会平滑地完成安装，我们依然强烈建议在升级到 GRUB v2 之前备份 GRUB Legacy 的文件。

```
# mv /boot/grub /boot/grub-legacy
```

备份含有引导代码和分区表的 MBR (把 `/dev/sd*X*` 替换为你的真实路径):

```
# dd if=/dev/sdX of=/path/to/backup/mbr_backup bs=512 count=1
```

MBR 只有 446B 包含引导代码，接下来的 64B 包含分区表。如果你不想在恢复时覆盖分区表，我们强烈建议只备份 MBR 引导代码:

```
# dd if=/dev/sdX of=/path/to/backup/bootcode_backup bs=446 count=1
```

如果无法正常安装 GRUB2 , 见恢复 GRUB Legacy.

把 GRUB Legacy 的配置文件转换为新格式

如果 `grub-mkconfig` 失败，如下转换你的 `/boot/grub/menu.lst` 为 `/boot/grub/grub.cfg`:

```
# grub-menulst2cfg /boot/grub/menu.lst /boot/grub/grub.cfg
```

注意：该选项仅在 BIOS 系统有效，UEFI 系统则不。

例如:

```
/boot/grub/menu.lst
default=0
timeout=5

title  Arch Linux Stock Kernel
root   (hd0,0)
kernel /vmlinuz-linux root=/dev/sda2 ro
initrd /initramfs-linux.img

title  Arch Linux Stock Kernel Fallback
root   (hd0,0)
kernel /vmlinuz-linux root=/dev/sda2 ro
initrd /initramfs-linux-fallback.img
/boot/grub/grub.cfg
set default='0'; if [ x"$default" = xsaved ]; then load_env; set
default="$saved_entry"; fi
set timeout=5

menuentry 'Arch Linux Stock Kernel' {
```

```
  set root='(hd0,1)'; set legacy_hdbias='0'
  legacy_kernel   '/vmlinuz-linux' '/vmlinuz-linux' 'root=/dev/sda2' 'ro'
  legacy_initrd '/initramfs-linux.img' '/initramfs-linux.img'
}

menuentry 'Arch Linux Stock Kernel Fallback' {
  set root='(hd0,1)'; set legacy_hdbias='0'
  legacy_kernel   '/vmlinuz-linux' '/vmlinuz-linux' 'root=/dev/sda2' 'ro'
  legacy_initrd '/initramfs-linux-fallback.img' '/initramfs-linux-fallback.img'
}
```

如果你忘了创建 GRUB `/boot/grub/grub.cfg` 配置文件然后启动进了 GRUB Command Shell, 输入:

```
sh:grub> insmod legacycfg
sh:grub> legacy_configfile ${prefix}/menu.lst
```

启动进 Arch 重新创建 GRUB `/boot/grub/grub.cfg` 配置文件。

恢复 GRUB Legacy

- 移除 GRUB v2 的文件:

```
# mv /boot/grub /boot/grub.nonfunctional
```

- 复制 GRUB Legacy 到 `/boot`:

```
# cp -af /boot/grub-legacy /boot/grub
```

- 从备份恢复 sda 的 MBR 以及接下来的 62 扇区

警告：该命令也恢复了分区表，所以以旧版覆盖修改过的分区表时请小心，它**将会**导致系统混乱。

```
# dd if=/path/to/backup/first-sectors of=/dev/sdX bs=512 count=1
```

以下是一个更安全的，只恢复 MBR 引导代码的方法:

```
# dd if=/path/to/backup/mbr-boot-code of=/dev/sdX bs=446 count=1
```

配置

配置文件位于 `/boot/grub/menu.lst`。按照你的要求编辑这个文件。

- `timeout #` -- 默认操作系统被自动加载前的等待时间(单位是秒)。
- `default #` -- 超时时默认的启动项。

一个范例配置文件(`/boot` 位于独立分区中):

```
/boot/grub/menu.lst
# Config file for GRUB - The GNU GRand Unified Bootloader
# /boot/grub/menu.lst

# DEVICE NAME CONVERSIONS
#
#  Linux           GRUB
# -------------------------
#  /dev/fd0        (fd0)
#  /dev/sda        (hd0)
#  /dev/sdb2       (hd1,1)
#  /dev/sda3       (hd0,2)
#

#  FRAMEBUFFER RESOLUTION SETTINGS
#     +-------------------------------------------------+
```

```
#          | 640x480    800x600    1024x768   1280x1024
#       ----+-------------------------------------------
#       256 | 0x301=769  0x303=771  0x305=773   0x307=775
#       32K | 0x310=784  0x313=787  0x316=790   0x319=793
#       64K | 0x311=785  0x314=788  0x317=791   0x31A=794
#       16M | 0x312=786  0x315=789  0x318=792   0x31B=795
#      +-------------------------------------------------+
#   for more details and different resolutions see
#   https://wiki.archlinux.org/index.php/GRUB#Framebuffer_Resolution

# general configuration:
timeout   5
default   0
color light-blue/black light-cyan/blue

# boot sections follow
# each is implicitly numbered from 0 in the order of appearance below
#
# TIP: If you want a 1024x768 framebuffer, add "vga=773" to your kernel line.
#
#-*

# (0) Arch Linux
title  Arch Linux
root   (hd0,0)
kernel /vmlinuz-linux root=/dev/sda3 ro
initrd /initramfs-linux.img

# (1) Windows
#title Windows
#rootnoverify (hd0,0)
#makeactive
#chainloader +1
```

寻找 GRUB 的根分区

GRUB 必须被告知它的文件位于文件系统的位置。因为可能有多个系统存在 (也就是在多启动环境中)。 GRUB 文件总是位于`/boot`，它有可能在一个独立的分区中。

注意：GRUB 定义的存储设备的方式与传统的内核命名方式不同。

- 硬盘定义为 **(hdX)**；这也可以用来指定任何 USB 存储设备。
- 设备和分区的编号从0开始。例如，BIOS 识别出的第一个硬盘被定为 **(hd0)**。第二个设备被叫做 **(hd1)**。这也应用到分区。因此第一个硬盘上的第二个分区叫做 **(hd0,1)**.

如果你不知道 `/boot` 的位置，使用 GRUB shell `find` 命令来定位 GRUB 文件。以 root 身份进入 GRUB shell:

```
# grub
```

下面的例子适用于*没有*独立 `/boot` 分区的系统。此时 `/boot` 只是 `/` 下的一个目录:

```
grub> find /boot/grub/stage1
```

下面的例子适用于*有*独立的 `/boot` 分区的系统:

```
grub> find /grub/stage1
```

GRUB 会找到这个文件，然后输出 `stage1` 文件的位置。例如:

```
grub> find /grub/stage1
 (hd0,0)
```

这个返回值需要输入到配置文件的 `root` 行。输入 `quit` 退出这个 shell.

双启动到 Windows

添加以下内容到你的 `/boot/grub/menu.lst` 末尾 (假设你的 Windows 分区是第一个设备的第一个分区):

```
/boot/grub/menu.lst
 title Windows
 rootnoverify (hd0,0)
 makeactive
 chainloader +1
```

注意：

- 如果你尝试双启动到 Windows 7，你需要注释掉 `makeactive` 行。
- Windows 2000 和之后的版本**不需要**在第一个分区就可以启动(与通常认为的不同)。如果 Windows 分区改变了 (也就是说，如果你在 Windows 分区的前面添加了一个分区)，你需要编辑 Windows 的 `boot.ini` 文件来应对改变。(参见本文获取更多细节。

如果 Windows 位于另一块硬盘，需要使用 map 命令。这会欺骗你的 Windows 安装程序，让它以为自己在主硬盘上。假设你的 Windows 分区是在第二块硬盘上的第一个分区:

```
/boot/grub/menu.lst
 title Windows
 map (hd0) (hd1)
 map (hd1) (hd0)
 rootnoverify (hd1,0)
 makeactive
 chainloader +1
```

注意：如果你尝试双启动到 Windows 7，你需要注释掉 `makeactive` 行。

双启动到 GNU/Linux

和Arch linux的加载方式一样。例如:

```
/boot/grub/menu.lst
 title Other Linux
 root (hd0,2)
 kernel /path/to/kernel root=/dev/sda3 ro
 initrd /path/to/initrd
```

注意：也许需要额外的选项，同时也可能不需要使用初始化内存盘。检查一下另一个发行版的 `/boot/grub/menu.lst` 来得到启动选项，或者参考 #chainloader 和 configfile (推荐)。

chainloader 和 configfile

为了方便系统维护，应该使用 `chainloader` 和 `configfile` 命令以启动提供了”自动化“引导机制的其他 Linux 操作系统(例如： Debian, Ubuntu, openSUSE),这样就允许每个发行版自行维护各自的 `menu.lst` 启动选项。

- `chainloader`命令能装载其他的启动引导器（而不是内核镜像)； 当其他的启动引导器安装在一个分区的启动扇区(例如：GRUB)时就有用了. 这样就可以实现在 MBR 安装一个“主”GRUB 和在每个分区的引导记录区 (PBR) 安装一些分散的 GRUB.

- {ic|configfile}}命令能指导 GRUB 装载确切的配置文件. 这样就可以引导其他的操作系统的 `menu.lst` 而不需要在引导的操作系统上安装 GRUB. 这种方法一大缺陷就是其他操作系统的 `menu.lst` 可能和安装的 GRUB 版本不兼容; 许多版本的操作系统高度依赖于 GRUB 的版本。

例如, GRUB 即将安装在 MBR (主引导记录区)上，但是其他的启动引导器(可能是 GRUB 或者 LILO)已经安装到了 `(hd0,2)` 启动扇区。

```
-----------------------------------------------
|   |           |           |   %           |
| M |           |           | B %           |
| B |  (hd0,0)  |  (hd0,1)  | L %  (hd0,2)  |
| R |           |           |   %           |
|   |           |           |   %           |
<br>-----------------------------------------------<br>|                                ^<br>|
chainloading        |<br>----------------------------<br>```
```

你可以在 `menu.lst` 下添加:
title Other Linux root (hd0,2) chainloader +1

或者，当 `(hd0,2)` 上的启动引导器是 GRUB 时:
title Other Linux root (hd0,2) configfile /boot/grub/menu.lst

`chainloader` 命令也可以用来装载第二个驱动器的 MBR:
title Other drive rootnoverify (hd1) chainloader +1

双启动 GNU/Linux (GRUB2)

如果其他 Linux 发行版用 GRUB2 (如: Ubuntu 9.10或者更高版本)，而且你在它的/分区安装了启动装载程序，你可以添加如下条目到你的 `/boot/grub/menu.lst`:
/boot/grub/menu.lst

title Ubuntu root (hd0,2) kernel /boot/grub/core.img

在启动时选择这个条目就可以引导其他发行版的 GRUB2 菜单 (假设安装在 `/dev/sda3`)。

安装启动器

手动恢复 GRUB 库

`*stage*` 文件在 `/boot/grub` 目录下，但是在如下情况下没有：启动装载程序在系统安装时没有安装或者/分区的文件系统损坏，意外删除等。

使用如下的命令手动复制 GRUB 的库文件:
{{注意|如果单独分区的话，不要忘记挂载系统的 {{ic|/boot}} 分区!(上文假设 `/boot} 分区既可以挂载在根分区也可以挂载在根分区的/boot目录下!)`

安装 GRUB 的常识

GRUB 有可以一个单独的介质安装(例如:一张 LiveCD),或者直接从运行着的 Arch 中安装。GRUB 启动引导器*很少*需要重新安装，当遇到如下情况时*不*需要安装:

- 更新了配置文件。
- 升级了软件包 [grub-legacy](https://aur.archlinux.org/packages/grub-legacy/)AUR.

遇到如下请况时*需要*安装：

- 引导器还没有安装。
- 其他的操作系统覆盖了 Linux 的引导器。
- 引导器因为一些未知的原因未能成功引导系统。

在重装或安装之前，请注意以下几点：

- 在进行操作之前确保你的GRUB配置文件是正确的 (`/boot/grub/menu.lst`)。请参考[#寻找 GRUB 的根分区](#寻找_GRUB_的根分区)以确认你的设备被正确的定义。
- GRUB 必须安装在 [MBR](../../02_系统管理与内核/系统恢复/分区.md) (硬盘的第一扇区)，或者在能被大多数 BIOS 识别的第一个存储设备的第一个分区。为了满足个人的多系统引导需求，GRUB 的多重设置就起作用了，请参考[#chainloader 和 configfile](#chainloader_和_configfile).
- 在安装 GRUB 启动引导器时可能需要在 `chroot` 过的环境下完成 (例如：通过安装光盘 chroot 安装的系统)，比如你编辑RAID的配置文件或者你忘记安装或者破坏了你的 GRUB，你都需要通过一张 LiveCD 或者其他的 Linux 操作系统 [Change root](../../02_系统管理与内核/系统恢复/Chroot.md) (即更改root目录).

首先，进入 GRUB shell:
使用 `root` 命令配合 `find` 的输出 (见 [#寻找 GRUB 的根分区](#寻找_GRUB_的根分区))，以告诉 GRUB 哪个分区包含了 stage1 (同理还有 `/boot`):

```
grub> root (hd1,0)
```

> **提示：**GRUB Shell 支持 Tab 补全，如果你键入'root (hd'然后连续敲击 `Tab` 两次你就可以看见可用的存储分区设备，在分区下也适用，Tab 补全在 GRUB 引导菜单上也适用，如果你的配置文件有错误你可以在引导菜单里编辑并使用 Tab 补全来协助寻找驱动器和分区。请参考[#在引导菜单中编辑 GRUB 条目](#在引导菜单中编辑_GRUB_条目).

安装到 MBR

下面的例子中把 GRUB 安装到第一个驱动器的 [MBR](../../02_系统管理与内核/系统恢复/分区.md):

```
grub> setup (hd0)
```

安装到分区

下面的例子中把 GRUB 安装到第一个驱动器的第一个分区:

```
grub> setup (hd0,0)
```

在运行 `setup` 完成之后，键入 `quit` 退出 shell. 如果你chroot 过，[退出 chroot 并卸载分区](../../02_系统管理与内核/系统恢复/Chroot.md)。现在重启测试一下。

备选方案 (grub-install)

> **注意：**这个方法不怎么可靠，推荐使用 GRUB shell.

使用 `grub-install` +你安装启动装载程序的分区。例如将 GRUB 安装到第一个驱动器的 MBR:
GRUB 会提示它是否安装成功. 如果没有安装成功, 你得用 GRUB Shell.

小技巧

额外的配置笔记。

Framebuffer 分辨率

我们可以用 `menu.lst` 里给出的分辨率，但也许你想使用 LCD 宽屏的最大原生分辨率。如下操作：

在 [Wikipedia](https://en.wikipedia.org/wiki/VESA_BIOS_Extensions#Linux_video_mode_numbers)上，列出了

framebuffer 分辨率（即排除了 VBE 标准中的）. 但是，比如我想用 1440x900 (`vga=867`) 却用不了。这是因为显卡制造商可以随便设置并非 VBE 3 标准的数字，这就导致了卡与卡之间编码不同（就算是同一个制造商）。

所以除了使用表格以外，你可以使用如下方法得到正确的编码：

GRUB 可识别的值

只使用 GRUB 的话有一个简单的方法来获得正确的编码。

在 kernel 行，让内核询问该使用什么模式。

kernel /vmlinuz-linux root=/dev/sda1 ro vga=ask

重启。GRUB 现在会展示一系列可用的编码以及一个扫描更多的选项。

你可以选择你想要的编码（别把它忘了，下一步还要用）并在启动时使用。

在 kernel 行把 `ask` 替换为你选择的正确编码。

例如，`1680x1050x32` 的 kernel 行看起来如下：

kernel /vmlinuz-linux root=/dev/sda1 ro vga=0x369

hwinfo

1. 从官方源安装 [hwinfo](https://archlinux.org/packages/?name=hwinfo)包.
2. 以 root 身份运行 `hwinfo --framebuffer`.
3. 选择代表了你想要的分辨率的编码。
4. 在该6位数字编码前加上 0x 前缀，并把它加入到 `menu.lst` kernel 行的 `vga=` 选项后。或是转换为十进制以避免使用 0x 前缀。

hwinfo 输出样例：

Mode 0x0364: 1440x900 (+1440), 8 bits Mode 0x0365: 1440x900 (+5760), 24 bits

以及 kernel 行：

kernel /vmlinuz-linux root=/dev/sda1 ro vga=0x0365

给分区命名

按盘符

如果你偶尔会调整（或准备去调整）分区大小你可以考虑按盘符给你的分区/驱动器命名。如下命名 ext2, ext3, ext4 分区：

e2label /dev/drive|partition label

名字可以长达16个字符但不能有空格。在你的 `menu.lst` 里设置：

kernel /boot/vmlinuz-linux root=/dev/disk/by-label/Arch_Linux ro

按 UUID

可使用 `blkid` 或 `ls -l /dev/disk/by-uuid` 来获取分区的 [UUID](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md) (Universally Unique IDentifier, 通用唯一标识符). 在 `menu.lst` 里如下设置：

kernel /boot/vmlinuz-linux root=/dev/disk/by-uuid/uuid_number

或：

kernel /boot/vmlinuz-linux root=UUID=uuid_number

都可。

以 root 身份启动 (单用户模式)

在引导器界面，选择一个条目并编辑 (按 `e`). 加上如下内核参数:
[...] single init=/bin/bash

然后会以单用户模式 (init 1) 启动，即没有输入密码前会停止在 root 提示。 这对设置 root 密码等恢复操作有用。 然而，如果你没有为 GRUB 设置[#密码保护](#密码保护)的话这是一个巨大的安全漏洞。

密码保护

你可以在 GRUB 配置文件里为你想要保护的操作系统设置密码保护。如果 BIOS 没有此类功能而你又需要额外安保措施的话你可能需要引导器密码保护。

首先选择一个你记得住的密码并加密:
Password: Retype password: 1ZOGor$GABXUQ/hnzns/d5JYqqjw

然后把它加到 GRUB 配置文件 `/boot/grub/menu.lst` 的开头 (必须加到开头以便 GRUB 识别):
timeout 5 default 0 color light-blue/black light-cyan/blue

password --md5 1ZOGor$GABXUQ/hnzns/d5JYqqjw

> **注意：**GRUB 使用 QWERTY 键盘布局输入。

然后为每个你要保护的操作系统加上 `lock` 命令:
title Arch Linux lock root (hd0,1) kernel /boot/vmlinuz-linux root=/dev/disk/by-label/Arch_Linux ro initrd /boot/initramfs-linux.img

> **警告：**如果你在 BIOS 禁用了从其他设备启动 (比如 CD 驱动器) 那么密码保护会保护你所有的操作系统，并且如果密码忘掉的话很难重新进入系统了。

在主板设置跳线来重置 BIOS 永远都是可能的 (参见你主板的手册，每个型号都有所不同). 所以别人能够操作硬件的情况下基本上没法阻止他们突破启动密码。

启动命名过的引导选项

如果你意识到你经常需要切换到其他一些非默认的操作系统 (如 Windows) 中并需要重新启动，等待GRUB菜单出现是乏味的。 GRUB 提供了一种方法在重新启动时不用等待菜单，而是使用时指定一个临时的新的的默认选项，该选项用于记录您的操作系统选择。

假设 `menu.lst` 配置如下:
/boot/grub/menu.lst

timeout 10 default 0 color light-blue/black light-cyan/blue

title Arch Linux root (hd0,1) kernel /boot/vmlinuz-linux root=/dev/disk/by-label/ARCH ro initrd /boot/initramfs-linux.img

title Windows XP rootnoverify (hd0,0) makeactive chainloader +1

Arch 是 default (0). 现在我们要重启进入 Windows. 改变 `default 0` 为 `default saved` -- 不管 **savedefault** 命令有没有使用，这将把当前的默认选项记录到 GRUB目录下的 `default` 文件。把 `savedefault 0` 添加到 Windows 条目的底部。每当 Windows 启动时，将会重置默认为 Arch, 如此来临时指定默认为 Windows.

现在我们需要的是一种能轻松改变默认的方式。可通过命令 `grub-set-default` 完成。因此为启动进入 Windows, 输入如下命令:

然后重启。

易用起见，你可能希望实现 "[Allow users to shutdown](../../02_系统管理与内核/电源管理/电源管理.md) 补丁" （包括`/sbin/grub-set-default` 中不需要密码来执行的命令）.

LILO 和 GRUB 相互配合

如果 LILO 软件包已经安装，删除它。

由于有些任务 （例如使用 `make all` 编译内核） 会调用 LILO，并且 LILO 会覆盖安装 GRUB. LILO 可能已包含于你的基本系统中，这取决于你的安装介质版本以及你在软件安装阶段是否选择了它。

> **注意：**如果被安装到 MBR 的话，`pacman -R lilo` 不会把 LILO 从 MBR 移除而仅仅删除 [lilo](https://aur.archlinux.org/packages/lilo/)AUR 软件包。安装到 MBR 的 LILO 引导器会当 GRUB （或其他引导器） 安装在 MBR 上时被覆盖。

GRUB 启动盘

首先，你需要格式化一张软盘：
然后挂载它：
在上面安装 GRUB：
复制你的 `menu.lst` 文件到磁盘上：
现在卸载你的软盘：
这样就完成了！现在你可以用这张磁盘重启你的计算机，它应该会用GRUB启动。当然还需先确认在你的BIOS里启动顺序中已经将软盘设为比硬盘更高优先级。

另见: [Super GRUB Disk](https://www.supergrubdisk.org/).

隐藏 GRUB 菜单

`hiddenmenu` 选项能默认隐藏菜单. 这样在启动时没有菜单显示但默认的选项会在启动延时后自动选择.你仍然可以按 `Esc` 让菜单显示出来. 要用这个选项， 只要在 `/boot/grub/menu.lst`文件中添加：
hiddenmenu

进一步排除 bug

见 [article](../../02_系统管理与内核/系统恢复/常规故障排除.md).

疑难问题

GRUB Error 17

> **注意：**以下解决方案对 GRUB Error 15 也有效。

第一步是拔下所有外部设备，看起来显而易见但是经常被忽略 ;)

如果你的分区表混乱了，下一次开机时就只会看到"GRUB error 17". 有许多导致分区表混乱的原因。通常是使用 [GParted](../../02_系统管理与内核/系统恢复/Parted.md) 操作硬盘 -- 特别是对逻辑分区 -- 造成分区顺序改变。例如，你删除了 `/dev/sda6` 并调整 `/dev/sda7` 的大小，最后在原 `/dev/sda6` 处创建分区，它会显示到列表底部，比如 `/dev/sda9`. 尽管物理上它们的顺序没有改变，它们被识别的顺序改变了。

修复分区表很简单。从你的 Arch CD/DVD/USB，启动，以 root 身份登录，运行：
进入硬盘， e\[x\]tra/expert 模式，\[f\]ix 分区顺序，然后 \[w\]rite 分区表并退出。

你可通过 `fdisk -l` 来检查确实已经修复好了。现在只需修复 GRUB. 见[引导器的安装](#安装启动器)部分。

基本上只要告诉 GRUB 正确的 `/boot` 然后重新写入 GRUB 到硬盘上的 [MBR](../../02_系统管理与内核/系统恢复/分区.md).

例如:

grub> root (hd0,6) grub> setup (hd0) grub> quit

更深入的总结见[这个页面](https://stringofthoughts.wordpress.com/2009/05/24/grub-error-17-debianubuntu)。

/boot/grub/stage1 not read correctly

如果在设置 GRUB 时碰到这个问题并且分区表不是很干净时，值得一查。
将会显示 `/dev/sda` 的分区表。在此检验你的分区的"Id"值是否正确。 "System"栏目会给出"Id"值的解释。

比如你的启动分区被标记为"HPFS/NTFS", 你得把它改成"Linux". 为此，转到 fdisk,
用 `t` 改变分区的系统 id, 选择新的分区类型和 id (Linux = 83). 也可用 `L` 列出所有的系统 id.

如果你改变了分区系统 id, 你需要 \[v\]erify 分区表然后 \[w\]rite.

现在尝试重新设置 GRUB.

[这里](https://bbs.archlinux.org/viewtopic.php?pid=799930)是报告该问题的论坛帖子。

意外安装到了 Windows 分区

如果你意外地把 GRUB 安装到了 Windows 分区，GRUB 会覆盖该分区的引导扇区并擦除到 Windows 引导器的引用。 (NTLDR 及更早的确证有此类问题，更新的版本不确定).

你要使用 Windows Recovery Console 来修复此问题。由于许多制造商在他们的产品里不包含这个 (更多的是使用恢复分区), Microsoft 开放了下载。如果你使用 XP, 参见[这个页面](https://web.archive.org/web/20120723080002/http://tips.vlaurie.com/2006/05/recovery-console-for-those-without-an-xp-disk/)来把软盘做成 Recovery CD. 启动 Recovery CD (或启用 Windows 恢复模式) 并运行 `fixboot` 来修复引导扇区。之后你需要重新安装 GRUB --- 这次安装到 MBR, 不是 Windows 分区---来引导 Linux.

更深入的讨论见[这里](../启动过程/Arch%20+%20Windows%20双系统.md#Restoring_a_Windows_boot_record)。

在引导菜单中编辑 GRUB 条目

你选中了引导菜单中的条目后就可以按 `e` 来编辑。如果要找设备的话使用 Tab 补全，然后 `Esc` 退出。之后按 `b` 来引导。

> **注意：**这些设置**不会被保存**。

device.map error

如果安装或引导时错误信息提及 `/boot/grub/device.map`, 运行:
以强制 GRUB 再次检查设备映射，即使之前有过一次。这在调整分区和添加/移除驱动器之后是必需的。

KDE 重启后下拉菜单失效

如果你已经打开了一个含有 GRUB 配置的所有操作系统的列表子菜单，在其中选择一个，并且在重新启动时，你仍然启动了默认的操作系统，那么你可能要检查你在 `/boot/grub/menu.lst` 里是否有下面这行:

default saved

GRUB 无法找到或安装到 virtio */dev/vd** 或其他非 BIOS 设备

我曾经在把 virtio 设备作为硬件设备的 KVM 虚拟机里的 Arch Linux 上安装 GRUB 时碰到了麻烦。为了安装 GRUB，尝试了以下方案：按 `Ctrl+Alt+F2` 或其他按键进入虚拟终端。 假设你的根文件系统挂载到 `/mnt`，boot 挂载或存储到 `/mnt/boot`.

1. 通过如下命令确认 GRUB 文件存在于 boot 目录里（假设挂载到 `/mnt/boot`）：
2. 如果 `/mnt/boot/grub` 已包含所有需要的文件，跳至第 3 步。否则，运行如下命令（把 `/mnt`，`your_kernel` 和 `your_initrd` 替换为真实路径和名称）。你的 `menu.lst` 也应在这里：
3. 通过如下命令启动 GRUB shell：
4. 键入以下命令。把 `/dev/vda` 和 `(hd0,0)` 替换为你的设备和分区：
device (hd0) /dev/vda root (hd0,0) setup (hd0) quit

5. 如果 GRUB 没有错误提示，那么就差不多完成了。你还要往 ramdisk 加入合适的模块。更多信息参见 [QEMU#Preparing an Arch Linux guest](../../02_系统管理与内核/虚拟机管理/QEMU.md#Preparing_an_Arch_Linux_guest).

参阅

- [GNU GRUB](https://www.gnu.org/software/grub/)
- [GRUB Grotto](http://www.troubleshooters.com/linux/grub/index.htm)
- [Boot debugging](../../02_系统管理与内核/系统恢复/常规故障排除.md) - Debugging with GRUB, set module values
- [Boot with GRUB](https://www.linuxjournal.com/article/4622) legacy

4.7 GRUB/技巧和窍门

难度 高级 | **篇幅** 32.1 KB | **所属** 启动加载程序

本节将讲到

- 其它安装方式
- 图形化配置工具
- 视觉配置
- 通过 GRUB 直接启动 ISO9660 映像文件
- 用密码保护 GRUB 菜单

其它安装方式

安装到 U 盘

BIOS

假设 U 盘的第一个分区是 FAT32格式，其分区是/dev/sdy1

```
# mkdir -p /mnt/usb
# mount /dev/sdy1 /mnt/usb
# grub-install --target=i386-pc --debug --boot-directory=/mnt/usb/boot /dev/sdy
# grub-mkconfig -o /mnt/usb/boot/grub/grub.cfg
```

可以选择将配置备份到 `grub.cfg`：

```
# mkdir -p /mnt/usb/etc/default
# cp /etc/default/grub /mnt/usb/etc/default
# cp -a /etc/grub.d /mnt/usb/etc
# sync; umount /mnt/usb
```

EFI

在你运行 `grub-install` 的时候使用 `--removable` 选项，就可以让 GRUB 把它的 EFI 映像写入到 `/boot/efi/EFI/BOOT/BOOTX64.efi`，这样启动固件就可以在没有 UEFI 启动条目的的情况下找到。

安装到分区上或者无分区磁盘上

警告：GRUB 极不推荐 将其安装到分区启动扇区或者无分区磁盘上（Grub Legacy 和 syslinux 相反）。这种安装方式不安全，当升级时可能会损坏。Arch 开发人员也 **不支持** 这种方式.

下面的命令将会将 GRUB 安装到分区扇区或者无分区磁盘（也称作超级软盘），或者安装到软盘上。下面例子中以 `/dev/sdaX` 作为 `/boot` 分区。

```
# chattr -i /boot/grub/i386-pc/core.img
# grub-install --target=i386-pc --debug --force /dev/sdaX
# chattr +i /boot/grub/i386-pc/core.img
```

注意：

- `/dev/sdaX` 仅用作示例。
- `--target=i386-pc` 令 `grub-install` 仅为 BIOS 系统安装。建议总是使用这个选项来排除 grub-install 命令中的模糊性。

你应该使用 `--force` 选项来启用对 blocklists（块列表）的支持，而不应该使用 `--grub-setup=/bin/true`，后者类似于单纯地生成 `core.img`。

`grub-install` 会生成以下警告，来提醒你哪里有可能出现问题。

```
/sbin/grub-setup: warn: Attempting to install GRUB to a partitionless disk or to a partition. This is a BAD idea.
/sbin/grub-setup: warn: Embedding is not possible. GRUB can only be installed in this setup by using blocklists.
                        However, blocklists are UNRELIABLE and their use is discouraged.
```

不使用 `--force` 选项则可能会出现以下错误，并且 `grub-setup` 不会将启动代码安装到启动扇区上：

```
/sbin/grub-setup: error: will not proceed with blocklists
```

而指定了 `--force`，会出现：

```
Installation finished. No error reported.
```

`grub-setup` 默认不启用这个选项，是因为在分区或者无分区磁盘上，`grub` 依赖于嵌入分区引导扇区的块列表 (blocklists) 来定位 `/boot/grub/i386-pc/core.img` 和前缀目录 `/boot/grub`。而 `core.img` 在分区上的扇区位置很有可能随着分区文件系统的更改而变化（复制文件，删除文件等）。详情请参考 https://bugzilla.redhat.com/show_bug.cgi?id=728742 和 https://bugzilla.redhat.com/show_bug.cgi?id=730915.

临时解决方案是给 `/boot/grub/i386-pc/core.img` 文件加“不可变”(immutable)标志（按照上面提过的，使用 `chattr` 命令）。这样 `core.img` 文件的位置就不会变。只有当将 GRUB 安装到分区启动扇区或者无分区磁盘上时才需要给 `/boot/grub/i386-pc/core.img` 加上“不可变”标志，在安装到 MBR 或者单纯地生成 `core.img` 而不嵌入到其他引导扇区里的时候，就不用添加（正如上面所述）。

然而即使没有报错，生成的 `grub.cfg` 文件也不会包含正确的 UUID。参考 https://bbs.archlinux.org/viewtopic.php?pid=1294604#p1294604 。 要解决这个问题，使用如下命令：

```
# mount /dev/sdxY /mnt         #Your root partition.
# mount /dev/sdxZ /mnt/boot    #Your boot partition (if you have one).
# arch-chroot /mnt
```

现在，安装linux包，然后：

```
# grub-mkconfig -o /boot/grub/grub.cfg
```

只生成 core.img

通过添加 `--grub-setup=/bin/true` 选项，`grub-install` 命令会填充 `/boot/grub` 文件夹并生成 `/boot/grub/i386-pc/core.img`，但是不会将 GRUB 启动引导代码嵌入到 MBR、MBR 后部区域或者分区引导扇区中。

```
# grub-install --target=i386-pc --grub-setup=/bin/true --debug /dev/sda
```

注意：

- `/dev/sda` 仅是示例。
- `--target=i386-pc` 令 `grub-install` 仅为 BIOS 系统安装。建议总是使用这个选项来排除 grub-install 命令中的模糊性。

生成后，Grub Legacy 或者 syslinux 就可以通过链式加载 GRUB 的 `core.img` 来间接加载 Linux 内核或者多启动内核了。参阅Syslinux#链式加载。

图形化配置工具

- **grub-customizer** — GRUB 或者 BURG 的 GTK 定制器

: https://launchpad.net/grub-customizer || grub-customizer包

视觉配置

GRUB 默认支持改变菜单外观。如果你没有初始化，请务必以视频模式 gfxmode 初始化 GRUB 图形终端 gfxterm。此视频模式由 GRUB 通过 gfxpayload 传递给 Linux 内核，任何视觉配置都需要这种模式才能够有效果。

设置帧缓冲分辨率

GRUB 既可以为自己，也可以为内核设定帧缓冲。现在已经不使用老的 `vga=` 配置了，推荐方法是在 `/etc/default/grub` 文件中按照下面的例子编辑，来设定宽度（像素）x高度（像素）x颜色深度：

```
GRUB_GFXMODE=1024x768x32
GRUB_GFXPAYLOAD_LINUX=keep
```

可以指定多种分辨率，包括默认的 `auto`，所以建议你编辑成这个样：`GRUB_GFXMODE=*心仪的分辨率,*后备分辨率比如 1024x768*,auto*`。*更多信息请见 GRUB gfxmode 文档。 gfxpayload 属性可以确保内核也保持该分辨率。*

注意：

- 只有显卡通过 VESA BIOS 拓展支持的模式才能用。要查看所支持的模式列表，安装 hwinfo包 然后以 root 权限运行 `hwinfo --framebuffer`。或者进入 GRUB 命令行运行 `videoinfo` 命令。
- 早期版本的 NVIDIA 专有驱动（使用显卡 GeForce GTX 970，驱动 nvidia 370 进行了测试）支持的 `GRUB_GFXMODE` 格式为 `*宽*x*高*-*色深*`（例如 `1920x1200-24`，而不是 `1920x1200x24`）。这应该不会影响新的显卡和驱动。使用比较新的驱动的 Pascal 显卡（以 GeForce GTX 1060 显卡和 nvidia 381.22 驱动测试）没法使用上文建议的格式，而且会引发严重的问题，包括但不限于系统崩溃以及 hard lock。当前的驱动和显卡最好使用标准的 `*宽*x*高*x*色深*` 格式来配置 `GRUB_GFXMODE`。
- 确保在修改完后运行 `grub-mkconfig -o /boot/grub/grub.cfg`。

这种方法不管用的话，可以试试老的 `vga=` 方法。将它添加到 `/etc/default/grub` 文件中的 `"GRUB_CMDLINE_LINUX_DEFAULT="` 一行就行了。比如 `"GRUB_CMDLINE_LINUX_DEFAULT="quiet splash vga=792"` 可以将系统的分辨率设定为 `1024x768`。

915resolution 破解

有些时候,Intel 显卡无法通过 `# hwinfo --framebuffer` 或 `vbeinfo` 显示你需要的分辨率。这种情况下，你可以使用 `915resolution` 破解。这种破解会临时性的修改显卡 BIOS 来添加所需的分辨率。详情请参考915resolution 主页\[失效链接 2022-09-18 ⓘ\]。这个包可以在这里找到：915resolutionAUR

首先，找一个你想要修改的视频模式。为此需要在 GRUB 命令行模式下运行：

```
sh:grub> 915resolution -l
Intel 800/900 Series VBIOS Hack : version 0.5.3
[...]
Mode 30 : 640x480, 8 bits/pixel
[...]
```

然后，使用 `1440x900` 分辨率覆盖 `Mode 30`：

```
/etc/grub.d/00_header
[...]
```

```
915resolution 30 1440 900  # Inserted line
set gfxmode=${GRUB_GFXMODE}
[...]
```

最后按照之前描述的方式设置 GRUB_GFXMODE，再重新生成 GRUB 配置文件，重启并测试是否生效。

背景图像和点阵字体

GRUB 原生支持设置背景图像和 pf2 格式的点阵字体。grub包 包中包含了 unifont 字体，名为unicode.pf2，（也有可能只包含名为 ascii.pf2 的ASCII字符字体），运行pacman -Ql grub | grep pf2来得到对应的文件路径。

在载入正确的模块之后，GRUB 支持的图像格式有 tga, png 和 jpeg。所支持的最大图像分辨率跟硬件有关。

请确保你已经设定了合适的帧缓冲分辨率。

按如下方式编辑 /etc/default/grub：

```
GRUB_BACKGROUND="/boot/grub/myimage"
#GRUB_THEME="/path/to/gfxtheme"
GRUB_FONT="/path/to/font.pf2"
```

注意：如果你将 GRUB 安装在单独的分区上，grub.cfg 文件中会自动将 /boot/grub/myimage 改成 /grub/myimage。

需要重新生成 grub.cfg 来让修改起作用。如果成功添加了背景图片，用户会在运行命令的终端中看到 "Found background image..."。如果句话没有出现，你设定的图像信息可能没有成功添加进 grub.cfg 文件。

如果图像没有正确显示，执行如下检查：

- 在 /etc/default/grub 文件中，图像的路径和名字要正确
- 图像的大小和格式要合适 (tga, png, 8-bit jpg)
- 图像需要以 RGB 模式存储，而且没有索引
- /etc/default/grub 文件里面开启了 console 模式
- 需要执行 grub-mkconfig 命令以将图像信息写入 /boot/grub/grub.cfg 文件
- grub-mkconfig 脚本不会对 grub.cfg 文件中所写的文件名添加引号，所以要确保这些名字里没有空格

主题

下面的例子将展示如何使用 GRUB 包里面的 Starfield 主题：

编辑 /etc/default/grub

```
GRUB_THEME="/usr/share/grub/themes/starfield/theme.txt"
```

需要重新生成 `grub.cfg` 来让修改起作用。如果配置成功的话，在重生成配置过程中，会在终端里出现 `Found theme: /usr/share/grub/themes/starfield/theme.txt`。

一旦使用了主题，你设置的背景图像通常就不起作用了。

菜单颜色

GRUB 支持设置菜单颜色。可使用的颜色能从 GRUB 手册里面找到。示例如下：

编辑 `/etc/default/grub`：

```
GRUB_COLOR_NORMAL="light-blue/black"
GRUB_COLOR_HIGHLIGHT="light-cyan/blue"
```

隐藏菜单

GRUB 特性之一就是支持隐藏/跳过菜单，可以在需要的时候按 `Esc` 来取消隐藏/跳过。同时还支持设置是否显示 timeout 计时器。

按照你的想法来编辑 `/etc/default/grub`。添加下面的几行可以启动这个功能，其中计时器被设定为 5 秒钟，而且可以被用户看到：

```
GRUB_TIMEOUT=5
GRUB_TIMEOUT_STYLE='countdown'
```

`GRUB_TIMEOUT` 是设置显示菜单前等待几秒。

禁用 framebuffer

使用 NVIDIA 私有驱动的用户可能希望禁用 GRUB 的 framebuffer，因为它会导致驱动错误。

要禁用 framebuffer，只要在 `/etc/default/grub` 中取消下面这行的注释：

```
GRUB_TERMINAL_OUTPUT=console
```

如果你想保留 GRUB 的 framebuffer，解决方法是在 GRUB 载入内核前进入文字模式。可以通过在 `/etc/default/grub` 中进行如下修改：

```
GRUB_GFXPAYLOAD_LINUX=text
```

通过 GRUB 直接启动 ISO9660 映像文件

GRUB 支持通过 loopback 设备直接从 ISO 映像启动，例如参考 Multiboot USB drive#Using GRUB and loopback devices。

用密码保护 GRUB 菜单

警告：如果有人能对你的机器进行物理上的访问，而且可以使用 Live USB 或磁盘启动（例如 BIOS 允许从外接磁盘启动），而 `/boot` 又处于一个没有加密的分区上面，那他就可以非常简单地通过编辑 GRUB 设置文件来绕过下面的这些。参考GRUB#/boot 加密和Security#磁盘加密。

如果你想禁止其他人改变启动参数或者使用 GRUB 命令行，可以给 GRUB 的配置文件关联一个用户名/密码。只需 运行 `grub-mkpasswd-pbkdf2`，输入密码然后确认：

```
grub-mkpasswd-pbkdf2
[...]
Your PBKDF2 is
grub.pbkdf2.sha512.10000.C8ABD3E93C4DFC83138B0C7A3D719BC650E6234310DA069E6FDB0DD4156313
DA3D0D9BFFC2846C21D5A2DDA515114CF6378F8A064C94198D0618E70D23717E82.509BFA8A4217EAD0B33C
87432524C0B6B64B34FBAD22D3E6E6874D9B101996C5F98AB1746FE7C7199147ECF4ABD8661C222EEEDB7D1
4A843261FFF2C07B1269A
```

修改`/etc/grub.d/40_custom`的权限以确保只有root能够查看该文件的内容，然后将下面的内容添加到此文件：

```
/etc/grub.d/40_custom
set superusers="用户名"
password_pbkdf2 用户名 密码
```

这里的 `*密码*` 是由 `grub-mkpasswd_pbkdf2` 所生成的那个字符串。

然后重新生成主配置文件，现在你的 GRUB 命令行、启动参数和所有的启动条目都得到保护了。

可以参考 GRUB 手册中的 "Security" 部分来将设置放宽或者针对多用户进行更复杂的定制。

只针对编辑 GRUB 和控制台选项进行密码保护

对一个菜单条目添加 `--unrestricted` 选项将会允许所有的用户启动这个操作系统，但却不能修改这个条目，也不能进入 GRUB 命令行控制台。 只有超级用户或者由 `--user` 开关指定的用户才能修改这个菜单条目。

```
/boot/grub/grub.cfg
menuentry 'Arch Linux' --unrestricted --class arch --class gnu-linux --class os ...
```

**本文或本章节的事实准确性存在争议。**

原因： `/etc/grub.d/10_linux` 文件不应该被手工编辑，每次升级 grub包 包都会自动把它覆盖掉。（在 Talk:GRUB/技巧和窍门 中讨论）

要给 Linux 启动条目添加 `--unrestricted`，可以修改 `/etc/grub.d/10_linux` 文件开头的 `CLASS` 变量。

```
/etc/grub.d/10_linux
CLASS="--class gnu-linux --class gnu --class os --unrestricted"
```

在没有按着 SHIFT 键时隐藏 GRUB 界面

为了获取更快的启动速度，而不用等 GRUB 倒计时，可以命令 GRUB 在启动时隐藏目录，仅在 `Shift` 被按住的时候才显示。

将如下行添加到`/etc/default/grub`来启动这个功能：

```
GRUB_FORCE_HIDDEN_MENU="true"
```

然后创建`/etc/grub.d/31_hold_shift`文件并写入以下链接中的内容：，给它可执行权限，然后重新生成主配置文件：

```
# chmod a+x /etc/grub.d/31_hold_shift
# grub-mkconfig -o /boot/grub/grub.cfg
```

注意：这个设置使用 keystatus 来检测按键事件，所以可能在某些机器上不能用。

将 UUID 和基础脚本结合使用

如果你想要使用 UUID 来避免不可靠的 BIOS 设备命名或者正在研究 GRUB 语法，这里有个使用了 UUID 的启动菜单项，和一个让 GRUB 使用你系统中正确的磁盘分区的小脚本。如果你想要将其移植到自己的系统上，只需要把 UUID 改成你的系统上的就行了。这个例子假设系统的 boot 和 root 文件系统是在不同的分区上，如果你还有其他分区，请相应地修改 GRUB 设置。

```
menuentry "Arch Linux 64" {
    # Set the UUIDs for your boot and root partition respectively
    set the_boot_uuid=ece0448f-bb08-486d-9864-ac3271bd8d07
    set the_root_uuid=c55da16f-e2af-4603-9e0b-03f5f565ec4a

    # (Note: This may be the same as your boot partition)

    # Get the boot/root devices and set them in the root and grub_boot variables
    search --fs-uuid $the_root_uuid --set=root
    search --fs-uuid $the_boot_uuid --set=grub_boot

    # Check to see if boot and root are equal.
    # If they are, then append /boot to $grub_boot (Since $grub_boot is actually the
root partition)
    if [ $the_boot_uuid == $the_root_uuid ] ; then
        set grub_boot=($grub_boot)/boot
    else
        set grub_boot=($grub_boot)
    fi

    # $grub_boot now points to the correct location, so the following will properly
find the kernel and initrd
    linux $grub_boot/vmlinuz-linux root=/dev/disk/by-uuid/$the_root_uuid ro
    initrd $grub_boot/initramfs-linux.img
}
```

多个启动条目

取消子菜单

如果你安装了多个内核，比如说 linux 和 linux-lts，那么`grub-mkconfig`默认会把他们分成一组建立一个子菜单。如果你不想这样，可以在`/etc/default/grub`文件里添加下面这行来回到仅有一个菜单的情形：

```
GRUB_DISABLE_SUBMENU=y
```

调用之前的启动条目

GRUB 能够记住你最近一次使用的启动项，并且在下次启动时将其作为默认项。当你使用多个内核或操作系统时（比如当前的 Arch 和一个用来做后备选项的 LTS 内

核），这个特性很有用。要开启这个功能，编辑 `/etc/default/grub` 中的 `GRUB_DEFAULT` 选项：

```
GRUB_DEFAULT=saved
```

上面的命令会告诉 GRUB 使用记住的启动项为默认启动项。要想让 GRUB 记住当前选择的启动项，将下面的行添加到 `/etc/default/grub`：

```
GRUB_SAVEDEFAULT=true
```

仅当 /boot 不是 btrfs 文件系统的时候才能用，因为 GRUB 没法对 btrfs 进行写入操作。但它会生成一个容易误导人的错误信息："sparse file not allowed. Press any key to continue."

注意：手动添加启动项到 `/etc/grub.d/40_custom` 或 `/boot/grub/custom.cfg`中，比如添加 Windows 启动项，需要先添加`savedefault`选项。

修改默认菜单条目

可以通过修改 `/etc/default/grub` 中的 `GRUB_DEFAULT` 值来改变默认启动项：

使用菜单标题：

```
GRUB_DEFAULT='Advanced options for Arch Linux>Arch Linux, with Linux linux'
```

注意：如果你使用的是非英语 locale，生成的菜单标题可能和上述例子不一致。可以考虑在 `GRUB_DEFAULT` 之上添加 `LANG=C`(会导致 GRUB 主菜单使用英文) 或者根据实际生成的菜单标题修改 `GRUB_DEFAULT`，如 `GRUB_DEFAULT='Arch Linux 的高级选项>Arch Linux，使用 Linux linux'`。

使用数字编号：

```
 GRUB_DEFAULT="1>2"
```

GRUB 启动项序号从 `0` 开始计数，`0` 代表第一个启动项，也是上述选项的默认值，`1` 表示第二个启动项，以此类推。主菜单和子菜单项之间用 `>` 隔开。

上面的例子启动的是主菜单项 'Advanced options for Arch Linux' 下子菜单的第三项。

自动启动非默认启动条目（仅一次）

如果你想在下一次启动的时候启动一个非默认的启动项，命令 `grub-reboot` 非常有用。当系统下一次启动时，GRUB 会自动载入这个命令后的第一个参数所指的那个启动项，而以后再次启动时，GRUB 会回到正常状态加载默认条目。这样就不用修改配置文件或者在启动时进行手动选择了。

注意：这个功能需要在 `/etc/default/grub` 中设定 `GRUB_DEFAULT=saved`，然后重新生成 `grub.cfg`；或者在手动生成的 `grub.cfg` 中，使用 `set default="${saved_entry}"`。

演奏一曲

通过修改 GRUB_INIT_TUNE 变量，你可以在启动时（在引导菜单显示之前）让扬声器播放音乐。比如要演奏柏辽兹《幻想交响曲》“妖魔夜宴”乐章片段（大管部分），你可以添加下面的设置：

```
GRUB_INIT_TUNE="312 262 3 247 3 262 3 220 3 247 3 196 3 220 3 220 3 262 3 262 3 294 3 262 3 247 3 220 3 196 3 247 3 262 3 247 5 220 1 220 5"
```

你可以通过创建以下文件，然后重新运行grub-mkconfig，添加一个菜单项来播放这些常见的GRUB_INIT_TUNE。

```
/etc/grub.d/91_tune_demo
#!/bin/sh
exec tail -n +3 $0
menuentry "INIT_TUNE Demos" {
   echo "Close Encounters/5 Tone"
   play 480 900 2 1000 2 800 2 400 2 600 3
   echo "Fur Elise (note long)"
   play 480 420 1 400 1 420 1 400 1 420 1 315 1 370 1 335 1 282 3 180 1 215 1 282 1 315 3 213 1 262 1 315 1 335 3 213 1 420 1 400 1 420 1 400 1 420 1 315 1 370 1 335 1 282 3 180 1 215 1 282 1 315 3 213 1 330 1 315 1 282 3
   echo "Berlioz's extract from Sabbath Night of Symphonie Fantastique"
   play 312 262 3 247 3 262 3 220 3 247 3 196 3 220 3 220 3 262 3 262 3 294 3 262 3 247 3 220 3 196 3 247 3 262 3 247 5 220 1 220 5
   echo "Oldskool Batman tune"
   play 380 120 1 140 1 160 1 200 8 190 4
   echo "Legend of Zelda tune"
   play 12000 440 100 0 1 329 150 440 50 0 1 440 25 493 25 523 25 587 25 659 200
   echo "Super Mario"
   play 1000 334 1 334 1 0 1 334 1 0 1 261 1 334 1 0 1 392 2 0 4 196 2
   echo "Super Mario Alternate"
   play 480 165 2 165 2 165 3 554 1 587 1 554 2 370 1 554 1 523 2 349 1 523 1 494 3 165 2 165 2 165 2
   echo "Mario Mushroom"
   play 1750 523 1 392 1 523 1 659 1 784 1 1047 1 784 1 415 1 523 1 622 1 831 1 622 1 831 1 1046 1 1244 1 1661 1 1244 1 466 1 587 1 698 1 932 1 1175 1 1397 1 1865 1 1397 1
   echo "Star Wars Imperial Death March"
   play 480 440 4 440 4 440 4 349 3 523 1 440 4 349 3 523 1 440 8 659 4 659 4 659 4 698 3 523 1 415 4 349 3 523 1 440 8
   echo "My Little Pony"
   play 2400 587 18 554 4 587 8 659 12 587 16 0 10 587 4 659 4 740 8 587 4 784 12 740 8 659 8 587 4 740 20 587 40
   echo "Wolfenstein 3D"
   play 300 131 1 196 1 196 1 196 1 294 1 196 1 294 1 196 1 131 1
   echo "Mall (Nothing Special)"
   play 180 440 1 554 1 659 1
   echo "Final Countdown"
   play 480 554 1 494 1 554 4 370 6 10 3 587 1 554 1 587 2 554 2 494 6
   echo "Xie-Jelei's Tune"
   play 2000 400 4 0 1 500 4 0 1 600 4 0 1 800 6
   echo "Random tune"
   play 480 220 1 277 1 330 1 440 1 185 1 220 1 277 1 370 1 294 1 370 1 440 1 587 1 330 1 415 1 494 1 659 1
   echo "Fleetwood Mac: The Chain Bass Riff (about 5 seconds)"
   play 304 55 5 0 1 55 1 62 1 65 2 62 1 55 1 49 1 55 1 62 2 41 8
   echo "Fleetwood Mac: The Chain Bass Riff Extended Version (WARNING, 25 SECONDS)"
   play 9120 55 150 0 30 55 30 62 30 65 60 62 30 55 30 49 30 55 30 62 60 41 360 0 120 55 150 0 30 55 30 62 30 65 60 62 30 55 30 49 30 55 30 62 60 41 360 0 60 41 12 42 3 43 3 44 3 45 3 46 6 47 3 48 3 49 6 50 3 51 3 52 6 53 3 54 3 55 150 0 30 55 30 62 30 65 60 62 30 55 30 49 30 55 30 62 60 41 240 82 20 0 10 82 26 0 4 82 26 0 4 82 26 0 4 82 56 0 4 82 60 55 150 0 30 55 30 62 30 65 60 62 30 55 30 49 30 55 30 62 60 41 360 0 120
}
```

更多相关信息可以查看 info grub -n play。

为早期启动手动配置核心映像

如果你需要用一个特殊的键盘布局，或者要让 GRUB 环境访问 `/boot` 的配置步骤太过复杂，导致 GRUB 没法自动配置，你可以自己生成一个核心映像。在 UEFI 系统上，核心映像就是在启动时需要被固件载入的 `grubx64.efi` 文件。自己建立一个核心映像，就可以嵌入在早期启动过程中需要用到的所有模块，以及用来引导 GRUB 配置脚本。

首先，假设在一个 UEFI 系统中，我们需要在早期启动过程中载入 `dvorak` 键盘布局，以输入 `/boot` 加密分区的密码。

从生成的 `/boot/grub/grub.cfg` 文件中查看需要使用哪些模块来挂载加密的 `/boot` 分区。例如在你的 `menuentry` 一项中你应该看到类似下面的内容：

```
insmod diskfilter cryptodisk luks gcry_rijndael gcry_rijndael gcry_sha256
insmod ext2
cryptomount -u 1234abcdef1234abcdef1234abcdef
set root='cryptouuid/1234abcdef1234abcdef1234abcdef'
```

把这些模块都记下来，他们都应该被包含到核心映像里面。现在把你的键盘布局打包，它应该作为一个 memdisk 来嵌入到核心映像里：

```
# grub-kbdcomp -o dvorak.gkb dvorak
# tar cf memdisk.tar dvorak.gkb
```

现在创建需要在 GRUB 核心映像中使用的配置文件。这和正常的 GRUB 配置文件的格式是一样的，但只需要简单几行来载入 `/boot` 分区里的主配置文件就可以了：

```
early-grub.cfg
set root=(memdisk)
set prefix=($root)/

terminal_input at_keyboard
keymap /dvorak.gkb

cryptomount -u 1234abcdef1234abcdef1234abcdef
set root='cryptouuid/1234abcdef1234abcdef1234abcdef'
set prefix=($root)/grub

configfile grub.cfg
```

最后，生成核心映像，列出在 `grub.cfg` 文件中所需要的所有模块，再加上 `early-grub.cfg` 脚本中所使用的模块。上例中需要的模块有 `memdisk`, `tar`, `at_keyboard`, `keylayouts` 和 `configfile`。

```
# grub-mkimage -c early-grub.cfg -o grubx64.efi -O x86_64-efi -m memdisk.tar diskfilter
cryptodisk luks gcry_rijndael gcry_sha256 ext2 memdisk tar at_keyboard keylayouts
configfile
```

生成的 EFI 核心映像可以和由 `grub-install` 所自动生成的一样使用，把它放在你的 EFI 分区，然后使用 `efibootmgr` 来启用它，或者依照你的系统固件来适当地进行配置。

另请参阅 Debian cryptsetup docs.

UEFI 延伸阅读

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：本节还可以更加简明有序。（在Talk:GRUB/技巧和窍门讨论）

下面是关于通过 UEFI 安装 Arch 的其他相关信息。

另一种安装方法

通常，不管 EFI 系统分区挂载到哪，GRUB 都会将所有文件包括配置文件放到 `/boot`。

如果你想将所有的文件放在 EFI 系统分区中，请给 grub-install 命令添加 `--boot-directory=*esp*` 选项：

```
# grub-install --target=x86_64-efi --efi-directory=esp --bootloader-id=grub --boot-directory=esp --debug
```

这个命令会将 GRUB 文件放在 `*esp*/grub` 而不是 `/boot/grub` 中。使用这个方法，请确保 grub-mkconfig 将配置文件放在与上面一样的目录里：

```
# grub-mkconfig -o esp/grub/grub.cfg
```

其他的配置是完全一样的。

UEFI 固件解决方案

请参阅 GRUB#缺省/后备启动路径

在固件启动管理器中创建 GRUB 条目

`grub-install` 会自动尝试在启动管理器中创建 GRUB 条目。如果没有成功，请参考 Unified Extensible Firmware Interface#efibootmgr，里面有如何使用 `efibootmgr` 创建启动目录条目的介绍。一般来说，这个问题都是因为你没有以 UEFI 模式启动 CD/USB 造成的。请参考 Unified Extensible Firmware Interface#从 ISO 创建 UEFI 可启动 USB。

独立的 GRUB

本节假设你要为 x86_64 系统 (x86_64-efi) 创建一个独立的 GRUB。至于 32 位 (IA32) EFI 系统，适当地将 `x86_64-efi` 修改成 `i386-efi` 就可以了。

你可以建立一个 `grubx64_standalone.efi` 应用，这个 UEFI 应用将所有的模组嵌入应用内部的一个 tar 包中，所以就不需要使用单独的目录来存放 GRUB UEFI 模组和其他相关文件了。使用 grub包 包里的 `grub-mkstandalone` 命令按照下面的方法操作来实现这个功能：

```
# echo 'configfile ${cmdpath}/grub.cfg' > /tmp/grub.cfg
```

```
# grub-mkstandalone -d /usr/lib/grub/x86_64-efi/ -O x86_64-efi --modules="part_gpt part_msdos" --locales="en@quot" --themes="" -o "esp/EFI/grub/grubx64_standalone.efi" "boot/grub/grub.cfg=/tmp/grub.cfg" -v
```

然后把 GRUB 配置文件复制到 `*esp*/EFI/grub/grub.cfg`，再使用 efibootmgr 为 `*esp*/EFI/grub/grubx64_standalone.efi` 创建一个 UEFI 启动管理器条目。

注意：要让 `${cmdpath}` 功能正常工作，`--modules="part_gpt part_msdos"` 选项（包含引号）是必要的。

警告：如果你在填写 `${cmdpath}` 的时候漏掉一个斜杠（比如把 `(hd1,msdos2)/EFI/Boot` 写成 `(hd1,msdos2)EFI/Boot`），你就会发现 `grub.cfg` 文件没法读取，然后被迫进入 GRUB 命令行中。如果出现这样的问题，检查 `${cmdpath}` 设置成了什么 (`echo ${cmdpath}`)，然后手工载入配置文件（例如 `configfile (hd1,msdos2)/EFI/Boot/grub.cfg`）。

技术信息

GRUB EFI 文件总是要配置文件放在 `${prefix}/grub.cfg`。当然在独立的 GRUB EFI 文件中，`${prefix}` 位于一个 tar 包的内部，嵌入在 GRUB EFI 文件自身之中（在 GRUB 环境中，这被称作 `(memdisk)`）。这个 tar 包包含了所有在正常的 GRUB EFI 下存储在 `/boot/grub` 中的文件。

因为将 `/boot/grub` 中的内容嵌入到了独立映像的内部，它就不再对任何实际的（外部的） `/boot/grub` 有任何依赖了。因此在独立 GRUB EFI 文件的情形下 `${prefix}==(memdisk)/boot/grub`，而且独立的 GRUB EFI 文件认为其配置文件存放于 `${prefix}/grub.cfg==(memdisk)/boot/grub/grub.cfg`。

因此要让独立的 GRUB EFI 文件读取存储在与 EFI 文件相同目录（在 GRUB 环境中用 `${cmdpath}` 表示）下的外部 `grub.cfg` 时，我们创建一个简单的 `/tmp/grub.cfg` 来让 GRUB 使用 `${cmdpath}/grub.cfg` 来做其配置文件。（在 `(memdisk)/boot/grub/grub.cfg` 中使用 `configfile ${cmdpath}/grub.cfg` 命令。）然后我们给 grub-mkstandalone 添加 `"boot/grub/grub.cfg=/tmp/grub.cfg"` 选项，把这个 `/tmp/grub.cfg` 文件复制到 `${prefix}/grub.cfg`（事实上就是 `(memdisk)/boot/grub/grub.cfg`）。

这样，独立的 GRUB EFI 文件和这个 `grub.cfg` 可以存放在 EFI 系统分区中的任何一个目录里，只有它们处在同一个目录中就可以。这样它们就变得可移植了。

加快GRUB中的LUKS解密速度

警告：在阅读本节之前，请确保你了解高熵密码的重要性。关于如何生成安全密码的信息可以在 Wikipedia:Password strength找到。

在启动时，GRUB在某些情况下可能需要很长的时间来验证密码。这可能是由于PBKDF的迭代次数太多，你可以按以下方法检查:

```
# cryptsetup luksDump /dev/sda3
```

问题是，给定的密钥槽的迭代次数是在添加钥匙时产生的，这个迭代次数需要确保既足够高来防止暴力攻击，也要使你的计算机尽可能快的推导出密钥。然而，当GRUB启动时，它可能没有同样的计算资源在手，因此推导速度会大大降低。

如果你的密码本身能提供足够的熵来对抗常见的攻击，你可以降低迭代次数。

```
# cryptsetup luksChangeKey --pbkdf-force-iterations 1000 /dev/sda3
```

根据RFC 2898，建议至少1000次迭代，但如果可以的话，你应该尽可能使用更高的数值（攻击者的成本以及密钥推导的时间是线性扩展的）。

提示：GRUB依次尝试启用密钥槽。在添加密钥时，可以使用`--key-slot`选项来明确指定一个密钥槽。

参考资料

- GRUB 2主题教程

4.8 Syslinux

难度 高级 | **篇幅** 42.3 KB | **所属** 启动加载程序

本节将讲到

- BIOS 系统
- UEFI 系统
- 配置
- 常见问题
- 更多信息

Syslinux是一个启动加载器集合，可以从硬盘、光盘或通过 PXE 的网络引导启动系统。支持的文件系统包括 FAT， ext2， ext3， ext4 和非压缩单设备 Btrfs 文件系统。

警告：在 Syslinux 6.03 版本中，启动加载器可能不支持某些文件系统的部分功能。详情请参考 。

注意：Syslinux 本身只能访问其所在分区上的数据，参阅 #链式加载 来了解如何绕过这个限制。

BIOS 系统

启动流程

1. **第一阶段 - 第一部分 - 加载MBR** 电脑启动时，BIOS 会先加载磁盘开始的 440 字节 MBR\[损坏的链接：无效的章节\] 启动代码 (`/usr/lib/syslinux/bios/mbr.bin` 或 `/usr/lib/syslinux/bios/gptmbr.bin`)。

2. **第一阶段 - 第二部分 - 寻找活动分区**第一阶段的MBR启动代码会寻找活动分区（设置了可启动标记的 MBR 分区），此处我们假设是 `/boot` 分区。
3. **第二阶段 - 第一部分 - 执行卷启动记录程序** MBR 启动代码会执行上面找到的 `/boot` 分区的卷启动记录程序（VBR，volume boot record）。对于 Syslinux 来说，VBR 就是由 `extlinux --install` 命令创建的 `/boot/syslinux/ldlinux.sys` 位于开始扇区的部分。请注意 `ldlinux.sys` 和 `ldlinux.c32` 是不同的。
4. **第二阶段 - 第二部分 - 执行 `/boot/syslinux/ldlinux.sys`** VBR 会加载 `ldlinux.sys` 剩余的部分。`ldlinux.sys` 所处在的扇区位置不可更改，否则 syslinux 无法启动。

> **注意**：对于 Btrfs 来说，因为文件不断移动导致`ldlinux.sys`扇区的位置不断变化，上述的方法将不会工作。因此在 Btrfs 中整个 `ldlinux.sys` 文件会直接紧接着嵌入卷启动记录程序，而不是像其他文件系统那样保存在 `/boot/syslinux/ldlinux.sys` 处。

1. **第三阶段 - 加载 `/boot/syslinux/ldlinux.c32`** `ldlinux.sys` 加载剩下的 syslinux 的核心部分 `/boot/syslinux/ldlinux.c32`（这部分是因为文件大小限制无法放入 `ldlinux.sys` 中的核心模块）。`ldlinux.c32` 文件应该在每一个装有 syslinux 的实例中出现，并且与分区中的 `ldlinux.sys` 版本相匹配，否则 Syslinux 将无法启动。更多资料请参阅 。
2. **第四阶段 - 查找并加载配置文件**当 syslinux 完全加载完毕，它将自动查找配置文件 `/boot/syslinux/syslinux.cfg` (或某些情况下的 `/boot/syslinux/extlinux.conf`)，如果找到即加载。否则会进入 Syslinux `boot:` 的命令提示符。这一步和剩下的非核心 Syslinux 部分(除 `lib*.c32` 和 `ldlinux.c32`的`/boot/syslinux/*.c32` 模块) 需要提供 `/boot/syslinux/lib*.c32` (库)模块。同样，{ic|lib*.c32}} 库模块和非核心的 `*.c32` 模块需要与分区中的 `ldlinux.sys` 版本相匹配。

在 BIOS 上安装

安装软件包 syslinux包。

注意：

- GPT 支持需要安装软件包 gptfdisk包。请参考#自动安装部分。
- FAT 支持需要安装软件包 mtools包。

安装软件包并不是安装启动加载器。在安装完相关的包后，还需要安装启动加载器代码（到适合的位置，一般是 VBR）才能启动系统；接下来的部分对您的特定系统的特性提供了替代的指令。

自动安装

注意：脚本 `syslinux-install_update` 是 Arch Linux 特有的，并不被 Syslinux 的上游提供/支持。请不要向上游，而是直接向 Arch Bug Tracker 提交关于这个特定脚本的 bug 报告。

- 执行完 `syslinux-install_update` 脚本后，请不要忘记按照#配置和#内核参数两节编辑 `/boot/syslinux/syslinux.cfg`。

警告：`syslinux-install_update` 脚本很有可能会设置一个与您的特定系统不同的默认根分区。将 `/boot/syslinux/syslinux.cfg` 文件中的根分区修改正确对于成功启动是至关重要的。参见#内核参数。

syslinux-install_update脚本将自动安装启动加载器代码（一般到 VBR）、复制 `*.c32` 模块到`/boot/syslinux`、设置分区启动标记并将启动代码安装到 MBR。它可以处理软 RAID、MBR\[损坏的链接：无效的章节\] 和 GPT 磁盘。

如果您使用分开的启动分区，请用 `lsblk` 命令确保它们都已被挂载。如果您没有看到 `/boot` 挂载点，请在继续之前将其挂载。

- `syslinux-install_update` 的参数：`-i` (安装文件)，`-a` (将活动分区标上启动标记)，`-m` (安装 MBR 启动代码): 如果

```
# syslinux-install_update -i -a -m
```

命令失败并报错*Syslinux BIOS install failed*，问题可能出在 `extlinux` 可执行文件不能找到包含 `/boot` 的分区：

```
# extlinux --install /boot/syslinux/
extlinux: cannot find device for path /boot/syslinux
extlinux: cannot open device (null)
```

例如，这可能发生在从 LILO 升级时，后者在引导当前的自定义内核时，将诸如 `root=/dev/sda1` 的内核命令行参数转写成了等效的数字参数 `root=801`。这可以从 `/proc/cmdline` 和 `mount` 等命令的输出确认。 可以通过向 `extlinux` 手动指定 `--device=/dev/sda1` 来解决下文手动安装时的问题，或者先重启到现有的Arch Linux内核，这样会使用 initramfs 来避免此问题。

注意：

- 如果您现在重新启动系统，则会出现 Syslinux 的提示符。如果想要实现系统自动引导或显示引导目录，您需要创建（或编辑）配置文件。
- 如果您位于另一个根目录（例如，从安装磁盘上），则可以通过 chroot，重定向后安装：

```
# syslinux-install_update -i -a -m -c /mnt
```

- 现在您需要按照接下来的#配置和#内核参数两节编辑 `/boot/syslinux/syslinux.cfg`文件。

手动安装

注意：如果您尝试使用Live CD拯救已安装的系统，请确保在执行这些命令之前已经 chroot 到目标系统。否则您必须在所有文件（除了`/dev/`）路径之前添加挂载点。

您计划安装Syslinux的启动分区必须包含 FAT，ext2，ext3，ext4 或 Btrfs 文件系统。您不必把它安装在文件系统的根目录上，比如把磁盘 `/dev/sda1` 挂在到 `/boot`。 例如，可以在`syslinux`子目录里安装 Syslinux:

```
 # mkdir /boot/syslinux
```

如果您希望使用除基本引导提示之外的任何目录或配置，请把 `/usr/lib/syslinux/bios/` 里的所有的 `.c32` 文件复制到 `/boot/syslinux/` 中。请不要使用符号链接。

```
 # cp /usr/lib/syslinux/bios/*.c32 /boot/syslinux/
```

现在安装启动加载器。对 挂载后的 FAT，ext2/3/4，或 btrfs 启动分区使用 *extlinux* 安装:

```
# extlinux --install /boot/syslinux
```

替代的，对一个**没有被挂载**的 FAT 启动分区使用*syslinux* 安装:

```
 # syslinux --directory syslinux --install /dev/sda1
```

在此之后，继续安装适用于对应分区表的 Syslinux 引导代码：

- #MBR 分区表上会安装 `mbr.bin`
- #GPT 分区表上会安装 `gptmbr.bin`

这些会在下面的内容中详细描述。

更多信息，请参考 MBR 分区表\[损坏的链接：无效的章节\]。

提示：如果您不确定您的分区表类型，可以通过 `blkid -s PTTYPE -o value /dev/sda` 来检查。

注意：对于一个无盘安装实例，不需要向MBR中安装Syslinux 引导代码。您可以跳过这一段，直接跳到#配置部分。更多信息请参考。

MBR 分区表

对于一个安装在 MBR 分区表\[损坏的链接：无效的章节\]上的实例，请确保您的启动分区已经被标记为活动分区（有启动标志）。您可以用 fdisk、parted 等工具确认。它应该看起来像这个样子：

```
# fdisk -l /dev/sda
[...]
  Device Boot      Start         End      Blocks   Id  System
/dev/sda1   *        2048      104447       51200   83  Linux
/dev/sda2          104448   625142447   312519000   83  Linux
```

安装 MBR 启动代码：

```
# dd bs=440 count=1 conv=notrunc if=/usr/lib/syslinux/bios/mbr.bin of=/dev/sda
```

Syslinux 提供一个替代的分区表 `altmbr.bin`。这个分区表*不会*扫描可启动分区，而是从 MBR 的最后一个字节中的值读取用于启动的分区。下面的操作会将这个分区表写入。

```
# printf '\x5' | cat /usr/lib/syslinux/bios/altmbr.bin - | dd bs=440 count=1 iflag=fullblock of=/dev/sda
```

在这个例子中，一个数值为5（十六进制的）的单个字节会被添加到 `altmbr.bin` 的内容后，并向 `sda` 的 MBR 分区表中写入440字节的内容。Syslinux 会被安装到磁盘的第一个逻辑分区(`/dev/sda5`)。

GPT 分区表

对于一个安装在 GPT 分区表上的实例, 请确保您的启动分区 `/boot` 被设置上了 2 号属性"传统BIOS可启动"（legacy BIOS bootable）。Parted 可以使用"legacy_boot"参数实现，而 sgdisk 则需要输入下面的命令：

```
# sgdisk /dev/sda --attributes=1:set:2
```

这会在 `/dev/sda` 的第一个分区设置"传统BIOS可启动"属性。检查命令：

```
# sgdisk /dev/sda --attributes=1:show
1:2:1 (legacy BIOS bootable)
```

安装 MBR 启动代码：

```
# dd bs=440 count=1 conv=notrunc if=/usr/lib/syslinux/bios/gptmbr.bin of=/dev/sda
```

UEFI 系统

注意：

- `efi64` 对应 x86_64 UEFI 系统。如果是 IA32（32位） EFI 您需要在下面的命令中将 `efi64` 替换为 `efi32`。
- 因为 Syslinux（目前）没有办法访问它自己所在以外的分区，因此内核与 initramfs 文件需要位于 Syslinux 同在的 EFI 系统分区（也叫ESP）。因此，这里推荐将ESP分区挂载在 `/boot`。
- `/usr/bin/syslinux-install_update` 自动安装脚本不支持 UEFI 安装。
- `syslinux.cfg` 文件的配置与 BIOS 的配置方法相同。

UEFI Syslinux 的局限性

- UEFI Syslinux 程序 `syslinux.efi` 不能通过 `sbsign`(sbsigntools包 软件包提供)签名，导致无法进行 UEFI 安全启动。查看关于此的 Bug 报告：
- 在 UEFI Syslinux 目录中编辑内核参数时使用 TAB 缩进会导致显示错误（重叠显示）。查看关于此的 Bug 报告：
- UEFI Syslinux 程序不支持链式加载其他如 `UEFI Shell`、`Windows Boot Manager` 等 EFI 应用程序。查看关于此的增强请求：
- 在某些情况下，UEFI Syslinux 可能不能在通过 QEMU/OVMF、VirtualBox 和 VMware 的一些特定产品或版本构建的虚拟机的中启动。一些如 DUET 等

UEFI 模拟环境中也是如此。Syslinux 的贡献者已经确认，在 VMware Workstation 10.0.2 版本、Syslinux 6.02 版本以后不会发生这个问题。查看关于此的 Bug 报告：, 和

- Memdisk 不被 UEFI 支持。查看关于此的增强请求：

在 UEFI 上安装

注意：在与 UEFI 有关的命令中，`*esp*` 指的是 EFI 系统分区(ESP)所挂载的位置。

- 安装来自官方软件源的 syslinux包 和 efibootmgr包。然后按如下步骤安装 Syslinux 到 EFI 系统分区(ESP)：
- 复制 Syslinux 文件到 EFI 系统分区:

```
# mkdir -p esp/EFI/syslinux
# cp -r /usr/lib/syslinux/efi64/* esp/EFI/syslinux
```

- 使用 efibootmgr 安装引导记录:

```
# efibootmgr --create --disk /dev/sdX --part Y --loader /EFI/syslinux/syslinux.efi --label "Syslinux" --verbose
```

其中，`/dev/sdXY`是包含启动加载器的分区。

- 按照#配置创建或编辑`*esp*/EFI/syslinux/syslinux.cfg`文件。

注意：

- UEFI 的配置文件是 `*esp*/EFI/syslinux/syslinux.cfg` 而非 `/boot/syslinux/syslinux.cfg`。在 `/boot/syslinux/` 中的文件是 BIOS 启动专用的，和 UEFI Syslinux 无关。

在BIOS模式下启动时， efibootmgr包 将无法为 `/EFI/syslinux/syslinux.efi` 设置 EFI 存储于非易失性存储器（nvram）的条目。欲实现此，请将资源放在默认 EFI 位置：`*esp*/EFI/syslinux/* -> *esp*/EFI/BOOT/*` 和 `*esp*/EFI/syslinux/syslinux.efi -> *esp*/EFI/BOOT/bootx64.efi` 处。

配置

Syslinux 的配置文件 `syslinux.cfg` 必须和 Syslinux 放在同一个目录下。在我们的示例中，BIOS 系统放在 `/boot/syslinux/` 处，UEFI 系统放在 `*esp*/EFI/syslinux/` 处。

启动器将自动寻找 `syslinux.cfg` (优先)和 `extlinux.conf`这两个配置文件之一。

提示：

- 除了 `LINUX`，您也可以使用 `KERNEL` 关键字。`KERNEL` 关键字会试图去检测内核文件的类型，而 `LINUX` 总是会将其按照 Linux 内核处理。
- `TIMEOUT` 的单位是 **0.1 秒**，也就是说 50 代表 5 秒。

示例

注意：

- 示例这一节中的所有配置文件都需要编辑并设定合适的内核参数。参见#内核参数节。
- 请一定仔细检查路径。这个实例可能不能够直接照搬到您的安装实例，尤其是 UEFI 系统。
- 接下来的例子假设内核与 initrd 文件都位于 `syslinux.cfg` 的父目录。(准确的说，是工作目录的父目录)。

启动提示符

这是一个非常简单的配置文件，会显示 `boot:` 提示符并在 5 秒后自动启动。如果您不希望看见提示符，请将 `PROMPT` 设置为 `0`。

配置文件：

```
* BIOS: /boot/syslinux/syslinux.cfg
* UEFI: esp/EFI/syslinux/syslinux.cfg
PROMPT 1
TIMEOUT 50
DEFAULT arch

LABEL arch
       LINUX ../vmlinuz-linux
       APPEND root=/dev/sda2 rw
       INITRD ../initramfs-linux.img

LABEL archfallback
       LINUX ../vmlinuz-linux
       APPEND root=/dev/sda2 rw
       INITRD ../initramfs-linux-fallback.img
```

文本启动目录

Syslinux 允许您使用一个文本启动目录来选择。要做到这一点，请将 `menu` 和 `libutil` 模块复制到您的 Syslinux 目录：

```
# cp /usr/lib/syslinux/bios/{menu,libutil}.c32 /boot/syslinux/
```

在 5.00 版本之后，其他的 `lib*.c32` 库模块也会频繁地被调用。参考 Syslinux 维基来查看依赖树。

配置文件：

```
* BIOS: /boot/syslinux/syslinux.cfg
* UEFI: esp/EFI/syslinux/syslinux.cfg
UI menu.c32
PROMPT 0

MENU TITLE Boot Menu
TIMEOUT 50
DEFAULT arch

LABEL arch
       MENU LABEL Arch Linux
       LINUX ../vmlinuz-linux
       APPEND root=/dev/sda2 rw
       INITRD ../initramfs-linux.img

LABEL archfallback
       MENU LABEL Arch Linux Fallback
```

```
       LINUX ../vmlinuz-linux
       APPEND root=/dev/sda2 rw
       INITRD ../initramfs-linux-fallback.img
```

更多关于启动目录的信息，请参考 Syslinux 维基。

图形化启动目录

Syslinux 允许您使用一个文本启动目录来选择。要做到这一点，请将 `vesamenu` COM32 模块复制到您的 Syslinux 目录：

```
# cp /usr/lib/syslinux/bios/vesamenu.c32 /boot/syslinux/
```

在 5.00 版本之后，其他的 `lib*.c32` 库模块也会频繁地被调用。参考 Syslinux 维基来查看依赖树。

注意：如果您使用的是 UEFI 系统，请一定从 `/usr/lib/syslinux/efi64/` 目录(或者 IA32 (32位) EFI 的 `efi32`) 中复制。否则电脑会黑屏。如果出现这个情况，请从 Live CD 中启动并使用 chroot 来进行正确的修改。

这个配置文件使用了和 Arch Linux 安装光盘相同的目录设计，您可以在 gitlab.archlinux.org 上找到。Arch Linux 启动背景图像也可以在那儿下载。请把图像复制到 `/boot/syslinux/splash.png` 处。

配置文件：

```
* BIOS: /boot/syslinux/syslinux.cfg
* UEFI: esp/EFI/syslinux/syslinux.cfg
UI vesamenu.c32
DEFAULT arch
PROMPT 0
MENU TITLE Boot Menu
MENU BACKGROUND splash.png
TIMEOUT 50

MENU WIDTH 78
MENU MARGIN 4
MENU ROWS 5
MENU VSHIFT 10
MENU TIMEOUTROW 13
MENU TABMSGROW 11
MENU CMDLINEROW 11
MENU HELPMSGROW 16
MENU HELPMSGENDROW 29

# Refer to https://wiki.syslinux.org/wiki/index.php/Comboot/menu.c32

MENU COLOR border       30;44   #40ffffff #a0000000 std
MENU COLOR title        1;36;44 #9033ccff #a0000000 std
MENU COLOR sel          7;37;40 #e0ffffff #20ffffff all
MENU COLOR unsel        37;44   #50ffffff #a0000000 std
MENU COLOR help         37;40   #c0ffffff #a0000000 std
MENU COLOR timeout_msg  37;40   #80ffffff #00000000 std
MENU COLOR timeout      1;37;40 #c0ffffff #00000000 std
MENU COLOR msg07        37;40   #90ffffff #a0000000 std
MENU COLOR tabmsg       31;40   #30ffffff #00000000 std

LABEL arch
       MENU LABEL Arch Linux
       LINUX ../vmlinuz-linux
       APPEND root=/dev/sda2 rw
       INITRD ../initramfs-linux.img
```

```
LABEL archfallback
       MENU LABEL Arch Linux Fallback
       LINUX ../vmlinuz-linux
       APPEND root=/dev/sda2 rw
       INITRD ../initramfs-linux-fallback.img
```

从 Syslinux 3.84 版本开始，`vesamenu.c32` 支持 `MENU RESOLUTION $WIDTH $HEIGHT` 指令。 使用这个指令，请将 `MENU RESOLUTION 1440 900` 插入您的配置文件，这样可以将分辨率设置为 1440x900。 但是，背景图像需要和分辨率完全一致，否则 Syslinux 将拒绝加载目录。

如果想要将目录居中并调整分辨率，请调节 `MENU RESOLUTION`、`MENU HSHIFT $N` 和 `MENU VSHIFT $N` 三个参数。将 `$N` 设置为非负值会将其从左上角开始移动。默认值都是 `0`，因此目录会显示在显示器的左上角。反过来，如果这个数值为负，则会从反方向开始移动。(比如说 `VHSHIFT -4`是从底部向上移动四行)。

如果想要使目录居中，请添加或改为这些值：

```
* BIOS: /boot/syslinux/syslinux.cfg
* UEFI: esp/EFI/syslinux/syslinux.cfg
MENU RESOLUTION 800 600 # or whatever your screen resolution is
MENU WIDTH 78           # width of the menu also required to bring the menu box to size
MENU VSHIFT 10          # moves menu down
MENU HSHIFT 10          # moves menu right
```

VESA 标准通常最大允许 25 行 80 列，因此如果将其设置得过大可能会将目录移出屏幕外，可能需要使用救援介质才能改回来。

内核参数

在 `syslinux.cfg` 中，内核参数的设定使用 `APPEND` 指令： 对每一个 `LABEL` 记录， APPEND 命令只能出现在一行内(就是说，把命令拆分成多行是无效的)。

推荐将下列的设置同时作为回滚条目。

最简单的情形，`root` 参数中的分区名称需要被修改。 将 `/dev/sda2` 改为正确的启动分区。

```
APPEND root=/dev/sda2
```

如果您想使用 **UUID\[损坏的链接：无效的章节\]**，通过块设备持久化命名制定启动分区，请如下改变 `APPEND` 行。这里我们用`1234`来代表您需要修改的`UUID`：

```
APPEND root=UUID=1234 rw
```

如果您想使用 **dm-crypt 加密设备**请如下改变 `APPEND` 行。

```
APPEND root=/dev/mapper/name cryptdevice=/dev/sda2:name rw
```

如果您使用 **mdadm 构建的软 RAID** , 请如下改变 `APPEND` 行来适配您的 RAID 阵列。下面是一个适配 3 RAID-1 阵列、并将一个合适的分区设为根分区的示例。

```
APPEND root=/dev/md1 rw md=0,/dev/sda2,/dev/sdb2 md=1,/dev/sda3,/dev/sdb3
md=2,/dev/sda4,/dev/sdb4
```

如果在内核设备节点模式下通过软 RAID 启动失败，下面是一个替代的更可靠的方法：使用分区标签：

```
APPEND root=LABEL=THEROOTPARTITIONLABEL rw
```

如果从一个 **btrfs 子卷启动**，向 `APPEND` 添加 `rootflags=subvol=<root subvolume>` 这一行。举个例子，我们称 `/dev/sda2` 被挂载到一个叫 'ROOT' 的 btrfs 子卷。(例如 `mount -o noatime,subvol=ROOT /dev/sda2 /mnt`)，这时应该将 `APPEND` 行改为这样：

```
APPEND root=/dev/sda2 rw rootflags=subvol=ROOT
```

如果此处错误，会收到 `ERROR: Root device mounted successfully, but /sbin/init does not exist.`错误信息。

自动启动

If you do not want to see the Syslinux menu at all, use the 如果您根本不想看见 Syslinux 的目录，请参考#启动提示符一节，将 `PROMPT` 设置为 `0` 并将任何 `UI` 的目录记录注释掉。将 `TIMEOUT` 设置为 `0` 也可以。确保您在 `syslinux.cfg` 设置的有默认回滚 `DEFAULT` 选项。按下 `Shift` 或者 `Alt`，或者设置为按下 `Caps Lock` 或 `Scroll Lock`， 在启动时可以允许更多除了默认选项以外的操作。更多可以设置的替代键，请参考上游的维基。

安全措施

Syslinux 有两级启动加载器安全措施，可以在 `syslinux.cfg` 中配置目录主密码，和分别针对每个条目的密码。使用

```
MENU MASTER PASSWD passwd
```

来配置目录主密码，在 `LABEL` 的块中使用

```
MENU PASSWD passwd
```

来设定针对这个条目的密码。

密码可以是明文，也可以是哈希值。更多信息，请参考官方文档。

链式加载

在 BIOS 上配置的 Syslinux 不能直接链式加载其他分区上的文件，但是 `chain.c32` 模块可以启动其他启动分区上的卷启动记录程序(VBR)，或是另一个磁盘上的 MBR 启动代码。

链式加载卷启动记录程序

如果您想要链式加载另一个操作系统（比如 Windows）或启动加载器，请将 `chain.c32` 模块复制到 Syslinux 目录(还需要复制其他需要的 `lib*.c32` 库模块。更多信息请参考前面几节)，然后在配置文件中建立下面这一节：

```
/boot/syslinux/syslinux.cfg
...
LABEL windows
       MENU LABEL Windows
       COM32 chain.c32
       APPEND hd0 3
...
```

`hd0 3` 是第一个 BIOS 控制驱动器上的第三个分区。请注意，驱动器计数从 0 开始，而分区计数则是从 1 开始。

注意：对于引导 Windows，这样会跳过 Windows 自己的启动管理器环节 (`bootmgr`)，而一些系统更新（例子）会需要这个组件才能完成。如果更新需要，在这种情况下建议临时将 MBR 的启动分区标记设在 Windows 所在分区(可以使用 GParted)并完成更新安装，再将启动分区标记改回 Syslinux 所在的分区。(可以使用 Windows自带的 DiskPart)。

链式加载 MBR 启动代码

如果您不知道 BIOS 会怎么安排驱动器的编号，您也可以使用 MBR 标记，或者说 GPT 的文件系统标签。使用 MBR 标记请如下文所示设置：

```
# fdisk -l /dev/sdb
Disk /dev/sdb: 128.0 GB, 128035676160 bytes
255 heads, 63 sectors/track, 15566 cylinders, total 250069680 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0xf00f1fd3

Device Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048     4196351     2097152    7  HPFS/NTFS/exFAT
/dev/sdb2         4196352   250066943   122935296    7  HPFS/NTFS/exFAT
```

将 `/dev/sdb` 改成您想要链式加载的驱动器。这里需要使用您磁盘的十六进制标识符，这个例子中是 `0xf00f1fd3`。`syslinux.cfg` 需要改成这个样子：

```
/boot/syslinux/syslinux.cfg
...
LABEL windows
       MENU LABEL Windows
       COM32 chain.c32
       APPEND mbr:0xf00f1fd3
...
```

更多关于链式启动的信息，请参阅 Syslinux 维基。

链式加载另一个启动加载器

如果您在同一个分区安装了 GRUB 引导程序，您可以像这样来链式加载它：

```
/boot/syslinux/syslinux.cfg
...
LABEL grub2
       MENU LABEL Grub2
       COM32 chain.c32
       APPEND file=../grub/boot.img
```

```
...
```

作为替代，也可以将 GRUB 作为一个 linux 内核来引导。只需要用 `lnxboot.img` 来替代 `core.img`。`lnxboot.img` 文件是 `core/grub` 提供的，您可以在 `/usr/lib/grub/i386-pc` 处找到它。

```
/boot/syslinux/syslinux.cfg
...
LABEL grub2lnx
       MENU LABEL Grub2 (lnxboot)
       LINUX ../grub/i386-pc/lnxboot.img
       INITRD ../grub/i386-pc/core.img
...
```

对于需要从 ISO 镜像引导的系统，这可能会被用到。

使用内存测试 memtest

安装来自官方软件源的 memtest86+包。

用这个 `LABEL` 片段来启动内存测试：

```
/boot/syslinux/syslinux.cfg
...
LABEL memtest
       MENU LABEL Memtest86+
       LINUX ../memtest86+/memtest.bin
...
```

注意：如果您使用 PXELINUX，请将 `memtest.bin` 重命名为 `memtest`。因为 PXELINUX 会把有 *.bin* 后缀名的文件当作启动扇区，这样就只会读取文件的前 2KB。

硬件检测工具

硬件检测工具(HDT)) 可以显示硬件信息。和前文所述的一样，`.c32` 模块需要从 `/boot/syslinux/` 中复制，其他需要的 `lib*.c32` 库模块也需要一并复制。 如果想要显示 PCI 总线信息请将 `/usr/share/hwdata/pci.ids` 复制到 `/boot/syslinux/pci.ids`，并在您的配置文件中添加这些行：

```
/boot/syslinux/syslinux.cfg
LABEL hdt
       MENU LABEL Hardware Info
       COM32 hdt.c32
```

重启和关机

注意：对于 Syslinux 6.03 版本，`poweroff.c32` 只能在高级电源管理（APM）下使用，而不适配高级配置与电源接口（ACPI）。您可以参考acpioff: 一种关闭使用 ACPI 控制机器的 COM32 模块，这是一种可能的解决办法。

使用下面的片段来重启或关机：

```
/boot/syslinux/syslinux.cfg
LABEL reboot
       MENU LABEL Reboot
       COM32 reboot.c32

LABEL poweroff
       MENU LABEL Power Off
       COM32 poweroff.c32
```

清空屏幕

当退出时清空屏幕，在配置文件中加入下面这一行：

```
/boot/syslinux/syslinux.cfg
MENU CLEAR
```

键盘布局

如果您需要经常修改 Syslinux 启动提示符下的参数，您可能想要匹配您的键盘布局。这样您就可以在一个非美式英语键盘上方便地输入"="、"/"等字符了。

注意：syslinux包 软件包下的 `keytab-lilo` 是一个调用*键位加载*的 perl 脚本。

生成一个适配的键盘布局映射表，（比如说，德语键盘），运行：

```
# keytab-
lilo /usr/share/kbd/keymaps/i386/qwerty/us.map.gz /usr/share/kbd/keymaps/i386/qwertz/de
.map.gz > /boot/syslinux/de.ktl
```

现在在 `syslinux.cfg` 文件中加入

```
/boot/syslinux/syslinux.cfg
KBDMAP de.ktl
```

更多细节，请参考 Syslinux 维基。

隐藏目录

在配置文件中使用

```
/boot/syslinux/syslinux.cfg
MENU HIDDEN
```

参数来隐藏目录并只显示超时倒计时。按下任意按键来显示目录。

PXELINUX

注意：对于 UEFI 设备，Syslinux 使用相同的二进制文件来从磁盘和网络启动。从 TFTP 或其他网络协议中启动要求 Syslinux 的网络启动。

PXELINUX 和 syslinux包 软件包一起提供。

对于一个 BIOS 客户机，将 `{l,}pxelinux.0` 启动加载器复制到客户机的启动目录。对于 5.00 版本及以上的 Syslinux，还需要将同一个软件包中的 `ldlinux.c32` 复制过去。

```
# cp /usr/lib/syslinux/bios/pxelinux.0 "TFTP_root/boot/"
# cp /usr/lib/syslinux/bios/ldlinux.c32 "TFTP_root/boot/"
# mkdir "TFTP_root/boot/pxelinux.cfg"
```

这时，我们同时也创建了 `pxelinux.cfg` 目录。PXELINUX 会默认在这个目录中寻找配置文件。我们不想对每一个不同的主机 MAC 地址都作出单独的设置，我们需要创建 `default` 作为默认配置。

```
TFTP_root/boot/pxelinux.cfg/default
DEFAULT linux

LABEL linux
```

```
        KERNEL vmlinuz-linux
        APPEND initrd=initramfs-linux.img quiet ip=:::::eth0:dhcp nfsroot=10.0.0.1:/arch
```

如果您使用 NBD 网络存储启动，添加下面这行：

```
append ro initrd=initramfs-linux.img ip=:::::eth0:dhcp nbd_host=10.0.0.1 nbd_port=10809 nbd_name=arch root=/dev/nbd0
```

注意：您需要对应修改 `nbd_host` 和/或 `nfsroot` 来与您的网络配置（NFS/NBD 服务器的地址）相匹配。

PXELINUX 的配置和 SYSLINUX 使用相同的语法。参考上游的文档来获取更多信息。

内核与 initramfs 会通过 TFTP 传输，因此需要设置对 TFTP 根目录的相对路径。否则，根文件系统需要挂载在 NFS 挂载点上。

若要加载 PXELINUX, 请将 `/etc/dhcpd.conf` 文件中的 `filename "/grub/i386-pc/core.0";` 替换为 `filename "/pxelinux.0"` (或 `filename "/lpxelinux.0"`)。

通过 memdisk 启动 ISO9660 镜像

Syslinux 支持通过 memdisk 模块从 ISO 镜像中启动。参考Multiboot USB drive#Using Syslinux and memdisk中的示例。

串行控制台

参考 Working with the serial console#Syslinux。

单次引导到另一个操作系统

可以临时改变 Syslinux 的行为，让其仅在下一次启动时引导进入另一个操作系统。下面的命令展示了怎样临时引导 `archfallback`。

```
# extlinux -o archfallback /boot/syslinux
```

在下一次启动的过程中，上面制定的启动标签会在没有任何 Syslinux 启动提示符的情况下直接启动。默认的启动行为会在下一次重启的时候复原。

常见问题

无法加载 ldlinux

启动时出现 "Failed to load ldlinux.c32" 错误提示可能会有很多原因。 其中的一种可能是文件系统工具或者文件系统结构遭到了更改。

警告：在 Syslinux 6.03 版本中，启动加载器可能不支持某些文件系统的部分功能。详情请参考 。

注意：出现 `Failed to load ldlinux.c32` 并不一定与文件系统出现问题直接相关：

- 文件系统可能出现的其他症状，除了这条信息，还可能指出另一些与文件系统有关的问题。

- 这条信息并不说明问题处在了文件系统。还有其他的一些问题会导致这条信息出现。

您也可以参考 。 (这一整页都和常见问题有关)。

使用 Syslinux 启动提示符

您可以输入(在您的 `syslinux.cfg` 配置过的) `LABEL` 的名称来启动对应的系统。如果您是按照上述的例子进行的配置，您只需要输入：

```
boot: arch
```

如果您收到了配置文件无法加载的错误信息，您可以向启动提示符中直接输入您的启动参数。比如说：

```
boot: ../vmlinuz-linux root=/dev/sda2 rw initrd=../initramfs-linux.img
```

如果在ramfs中您没有 `boot:` 的访问权限，并且您暂时无法启动内核，

: 1\. 创建一个临时的目录，来加载您的根分区 (如果不存在的话)：

```
 # mkdir -p /new_root
```

: 2\. 将 `/` 挂载到 `/new_root` (我们假设 `/boot/` 也在同一个分区，否则您需要将这些分区全部挂载):

注意：如果 `/boot` 在自己的 ext2 分区，Busybox 将无法挂载。

```
 # mount /dev/sd[a-z][1-9] /new_root
```

: 3\. 使用 `vim` 来重新将 `syslinux.cfg` 编辑正确。保存文件。 : 4\. 重启。

根分区文件系统检测失败

如果根分区被严重损坏（分区日志受损），在ramfs的应急 shell 中，加载根分区文件系统：

```
# mount /dev/root partition /new_root
```

将二进制程序 tune2fs 从根分区中复制出来（这个不包含在 Syslinux 中）。

```
# cp /new_root/sbin/tune2fs /sbin/
```

按照下面的指示ext2fs: 日志损坏重建根分区日志。

没有找到默认回滚和界面设置

有些主板厂家对 USB 设备启动的支持会差一些。此时一个在通常的现代电脑上都能启动的 ext4 格式的 U 盘，一些要求内核与 initrd 文件必须存储在 FAT16 分区的电脑可能无法正常运作。为了避免这种老设备无法读取 `syslinux.cfg` 而去加载 `ldlinux`，用 dosfstools包 工具建立一个 FAT16 的小分区(≤ 2 GB)：

```
# mkfs.fat -F 16 /dev/sda1
```

再来安装配置 Syslinux。

找不到操作系统

- 确保您在#MBR 分区表上安装的是 `mbr.bin`，在#GPT 分区表上安装的是 `gptmbr.bin`。如果安装错误，`mbr.bin` 会显示 "Missing operating system" 错误信息，而 `gptmbr.bin` 会显示 "Missing OS" 信息。
- 检查包含 `/boot` 的分区是否有可启动标记。
- 检查第一个分区是否是从 1 扇区，而不是 63 扇区或 2048 扇区开始的。您可以使用 `fdisk -l` 命令确认。如果是从 1 扇区开始的，您可以通过救援介质上的 `gparted` 修改。如果您有另一个启动分区，您可以将 `/boot` 备份：

```
# cp -a /boot /boot.bak
```

然后使用 Arch 安装磁盘启动。接下来，使用 `cfdisk` 删掉 `/boot` 分区，再重建它。现在这个分区应该从正确的 **63** 扇区开始。现在挂载您的分区，并按照安装指南中写的那样 `chroot` 进入待修复系统。使用以下指令恢复 `/boot`。

```
# cp -a /boot.bak/ /boot/
```

检查 `/etc/fstab` 是否正确：

```
# syslinux-install_update -iam
```

然后重启。

如果您试图从用一个太新以至于 Syslinux 无法识别其元数据的工具创建的 md RAID\-1 阵列启动，您也会得到这个错误信息。在 2013 年 8 月 mdadm 会创建使用 1.2 版本的元数据的阵列，但是 Syslinux 不能够识别高于 1.0 版本的元数据。如果是因此而出错，您需要重建您的序列，并在使用 mdadm 时添加 `--metadata=1.0` 参数。

启动了 Windows，Syslinux 被忽略掉了

解决办法：确保包含 `/boot` 的分区上有可启动标记。同时，确保 Windows 分区上没有这个标记。请参考上面的#链式加载卷启动记录程序部分。

Syslinux 所在的 MBR 会找到第一个有可启动标记的活动分区。Windows 分区很可能最先被发现。如果您想要修改，您可以使用 Windows 或 MS-DOS 上的 `fdisk`。

目录项失效

您选择了一个目录项，但是什么都没有发生，只是"*刷新*"了一下目录。这通常意味着您的 `syslinux.cfg` 文件出了问题。按下 `Tab` 键来修改您的启动参数。或者您也可以按下 `Esc` 键，并输入您的启动项的 `LABEL`。（比如说，*arch*）。另一种可能导致这个情况的原因可能是您没有安装内核。想个办法进入您的文件系统（比如说，用live CD），检查 `/mount/vmlinuz-linux` 文件存在并且非空。如果是这里出错，请重新安装您的内核。

无法删除 ldlinux.sys

`ldlinux.sys` 文件设置了不可变属性，禁止您删除或覆盖它。这样保证了除了重装 Syslinux，没有别的办法来修改这个文件。若要删除这个文件，请运行下面的指令：

```
# chattr -i /boot/syslinux/ldlinux.sys
# rm /boot/syslinux/ldlinux.sys
```

vesamenu的左上角有白色块

问题： *从 linux-3.0 开始，模式设置驱动程序会在更改分辨率后尝试保留屏幕的当前内容（至少在我的 Intel 设备上，使用 Syslinux 的文本模式的情况下如此）。而 Syslinux 中的 vesamenu 模块会与这个特性冲突。（白色方框实际上是驱动程序试图保留 Syslinux 目录，但是驱动程序无法从 vesa 图形模式下捕获图片）。*

如果您的分辨率是自定义的，而 `vesamenu` 在很早就进行了模式设置，请尝试在 `syslinux.cfg` 添加这一行来消除白色块：

```
APPEND root=/dev/sda6 rw 5 vga=current quiet splash
```

当 Windows 安装在另一张磁盘上时，链式启动失效

如果 Windows 安装在在另一张磁盘上，您又不能通过链式启动来引导 Windows， 请尝试下列配置：

```
LABEL Windows
       MENU LABEL Windows
       COM32 chain.c32
       APPEND mbr:0xdfc1ba9e swap
```

将 MBR 代码换成您的 Windows 驱动器的值(详见上文)， 并将 `swap` 加入选项。

查看启动加载器日志

在一些情况下（比如启动加载器无法启动内核），我们很想得到启动的更多信息。Syslinux 会显示错误信息，但是文字一闪而过，看不清。为了保留日志信息，请禁用 `syslinux.cfg` 中的 `UI menu`，并使用默认的命令提示符。这意味着

- 不使用 `UI` 命令
- 不使用 `ONTIMEOUT`
- 不使用 `ONERROR`
- 不使用 `MENU CLEAR`
- 将 `TIMEOUT` 设为更高的值
- 使用 `PROMPT 1`
- 使用 `DEFAULT *problematic_label*`

若要得到更详细的调试日志，To get more detailed debug log, 添加下面的 C 编译标志并重新编译 syslinux包 软件包：

```
-DDEBUG_STDIO=1 -DCORE_DEBUG=1
```

Btrfs 压缩

从压缩 Btrfs 文件系统启动不受支持。 会显示下列错误信息：

```
btrfs: found compressed data, cannot continue!
invalid or corrupt kernel image.
```

多设备 Btrfs

从多设备 Btrfs 文件系统启动不受支持。 (21-Jul-2016 版本，main.c 文件第 1246 行 validate_device_btrfs()会对此进行检查) 这个令人抓狂的问题会显示下列错误信息：(假设您将 Syslinux 装在 sda1):

```
/boot/syslinux is device /dev/sda1
extlinux: path /boot/syslinux doesn't match device /dev/sda1
```

更多信息

- 官方网站

4.9 GRUB

难度 高级 | **篇幅** 63.2 KB | **所属** 启动加载程序

本节将讲到

- 受支持的文件系统
- UEFI 系统
- BIOS 系统
- 配置
- 使用 GRUB 命令行

GRUB（GRand Unified Bootloader，大一统启动加载器），是一个引导加载程序。当前的 GRUB 也被称作 **GRUB 2**，而原始 GRUB（GRUB Legacy）表示 0.9x 版本。本页只描述 GRUB 2。

注意：在本文中，`*esp*` 表示EFI 系统分区（ESP）的挂载点。

受支持的文件系统

GRUB捆绑了由自己支持的多种文件系统，尤其是FAT32，ext4，Btrfs和XFS。有关一些注意事项，参见#不支持的文件系统

警告：文件系统可能会获得一些GRUB还未支持的新特性，除非关闭这些不兼容的特性，否则这些特性无法适用于`/boot`。通常规避这个问题的方式是使用一个单独的/boot分区，并格式化为通用的文件系统，如FAT32。

UEFI 系统

注意：

- 建议阅读并理解统一可扩展固件接口 (UEFI)，分区#GUID 分区表 和 Arch 的启动流程#UEFI这几个页面。
- 使用UEFI安装时，一定要让安装介质以UEFI模式启动，否则 *efibootmgr* 将无法添加 GRUB UEFI 启动项。 但即使在 BIOS 模式工作时，安装到后备启动路径仍然可行，因为这一过程用不到 NVRAM。
- 要从一个磁盘上使用 UEFI 模式启动，磁盘上必须要先有一个 EFI 分区。按照 EFI 系统分区#检查现有的分区 上说的来查看你是否已经有一个 EFI 分区，如若没有，就创建一个。
- 本页所有内容假定GRUB2是能够通过`insmod`加载额外模块的。#Shim-lock中讨论了相关问题，这种情况下UEFI系统无法启用安全启动（Secure Boot）功能。在一个启用了安全启动的系统上，如果你想要使用任何不包含在标准GRUB EFI `grubx64.efi`文件中的额外GRUB模块，你必须通过`grub-mkstandalone`或者使用`grub-install`重新安装GRUB，以重新生成包含所需模块的`grubx64.efi`文件。

安装

注意：

- 不同硬件厂商的 UEFI 实现方式不一样，下面描述的步骤应该可以在大部分 UEFI 系统上面正常应用。对于用了下面的方法却遇到问题的用户，请将在特定的硬件上所遇到的问题的细节，以及可能的解决办法分享出来。这些案例可以添加到页面 GRUB/EFI examples 上面。
- 本节假设您正在 x64（64位）UEFI系统上安装 GRUB。对于 IA32 （32 位） UEFI 系统（不要和 32 位 CPU 相混淆）， 将`x86_64-efi`替换成`i386-efi`。根据UEFI#检查系统位数中的说明来确定你的UEFI系统位数。

警告：自grub包 2:2.06.r566.g857af0e17-1起，无法从IA32 UEFI(`i386-efi`)系统启动。参见 FS#79098。

首先安装软件包 grub包 和 efibootmgr包。其中“GRUB”是启动引导器，“efibootmgr”被 GRUB 脚本用来将启动项写入 NVRAM。

然后按照下列步骤将GRUB安装到你的硬盘上：

1. 挂载 EFI 系统分区。需要注意的是在本节的内容里，把 `*esp*` 替换成ESP分区挂载点；
2. 选择一个bootloader-id，在本例被称为 `GRUB`。这将在 `*esp*/EFI/` 中创建一个与标识同名的目录来储存 EFI 二进制文件，而且这个名字还会用于在 UEFI 启动菜单中区分 GRUB 启动项；
3. 执行下面的命令来将 GRUB EFI 应用 `grubx64.efi` 安装到 `*esp*/EFI/GRUB/`，并将其模块安装到 `/boot/grub/x86_64-efi/`。

: : > **注意：**

- 确保你安装 GRUB 软件包和运行命令的系统是你想用 GRUB 引导的系统。也就是说如果你是通过安装介质引导的，你需要在 chroot 之后再运行 `grub-install`。如果因为某些原因不得不在

安装的系统之外运行 grub-install，在后面加上 --boot-directory= 选项来指定挂载 /boot 目录的路径，例如 --boot-directory=/mnt/boot。

- 某些主板无法处理包含空格的 bootloader-id 。

:```

上述安装完成后， GRUB 的主目录将位于 `/boot/grub/`。注意上述例子中，`grub-install` 还将[在固件启动管理器中创建一个条目](GRUB_技巧和窍门.md#在固件启动管理器中创建_GRUB_条目)，名叫 `GRUB`。如果你的启动项已满，这个命令会执行失败。你需要使用 [efibootmgr](../启动过程/UEFI.md#efibootmgr) 来删除不必要的条目。

在配置完成后，记得[#生成主配置文件](#生成主配置文件)。

> **提示：**如果你使用了 `--removable` 选项，那 GRUB 将被安装到 `*esp*/EFI/BOOT/BOOTX64.EFI`（当使用 `i386-efi` 时是 `*esp*/EFI/BOOT/BOOTIA32.EFI`），此时即使 EFI 变量被重设或者你把这个驱动器接到其他电脑上，你仍可从这个驱动器上启动。通常来说，你只要像操作 BIOS 设备一样在启动时选择这个驱动器就可以了。如果设备是同时安装了Windows的多引导启动，注意 Windows 通常会在这里安装一个 EFI 可执行程序，该程序的目的是仅重建Windows的UEFI启动项。如果你想在[Mac](../../04_硬件与外设/Apple/Mac.md)上安装GRUB，那你必须要使用该选项。某些台式机主板只会在此位置寻找 EFI 可执行文件，因此该选项是必需的，尤其是微星（MSI）主板。 如果你更新了UEFI，启动项可能会在更新后丢失。因此可以创建一个"removable"启动项作为后备。

> **注意：**
>
> - `--efi-directory` 和 `--bootloader-id` 是 GRUB UEFI 特有的。`--efi-directory` 替代了已经废弃的 `--root-directory`。
> - 您可能注意到在 `grub-install` 命令中没有 *device_path* 选项（例如 `/dev/sda`）。事实上即使提供了 *device_path*，也会被 GRUB UEFI 安装脚本忽略，因为 UEFI 启动加载器不使用 MBR 启动代码或启动扇区。

如果遇到问题，查看 [UEFI 故障排查](#UEFI_异常)。参见[GRUB/技巧和窍门#UEFI 延伸阅读](GRUB_技巧和窍门.md#UEFI_延伸阅读)。

启用安全启动

GRUB支持使用CA密钥或shim进行安全启动，安装命令受你选择的方法而不同。

> **警告：**
>
> - 错误配置[安全启动](../启动过程/UEFI_安全启动.md)会造成你的系统无法启动。如果启用安全启动后系统因某种原因无法启动，你应该在固件设置中关闭它并重启系统。
> - 启动引导器加载不必要的模块会带来潜在安全风险，没有必要不要使用这些命令。

CA密钥

下列命令使用CA密钥：
Shim-lock

> **注意：**进行下面的操作前，你应该确保已经按照[UEFI/安全启动#shim](../启动过程/UEFI_安全启动.md#shim)中的指示完成设置，并已经安装了[sbsigntools](https://archlinux.org/packages/?name=sbsigntools)包用于接收密钥。

当使用Shim-lock时，只有当GRUB二进制文件包含了全部用于读取含有[vmlinuz](../../02_系统管理与内核/内核/内核.md)和[initramfs](../启动过程/Arch%20的启动流程.md)镜像的文件系统的必要模块时，才可以成功进入安全启动模式。

自GRUB版本`2.06.r261.g2f4430cc0`起，无法在安全启动模式下通过`insmod`加载模块，因为这违背了不允许侧加载任意代码的要求。如果GRUB模块没有嵌入在EFI二进制文件中，而GRUB又通过`insmod`侧加载它们，GRUB会产生启动失败的信息：

error: prohibited by secure boot policy

根据Ubuntu的[官方构建脚本](https://git.launchpad.net/~ubuntu-core-dev/grub/+git/ubuntu/tree/debian/build-efi-images?h=debian/2.06-2ubuntu12)，整合下列模块到已签名的GRUB EFI二进制文件`grubx64.efi`：

- ["基础"模块](https://git.launchpad.net/~ubuntu-core-dev/grub/+git/ubuntu/tree/debian/build-efi-images?h=debian/2.06-2ubuntu12#n87)，用于从CD或简单分区硬盘启动： `all_video`, `boot`, `btrfs`, `cat`, `chain`, `configfile`, `echo`, `efifwsetup`, `efinet`, `ext2`, `fat`, `font`, `gettext`, `gfxmenu`, `gfxterm`, `gfxterm_background`, `gzio`, `halt`, `help`, `hfsplus`, `iso9660`, `jpeg`, `keystatus`, `loadenv`, `loopback`, `linux`, `ls`, `lsefi`, `lsefimmap`, `lsefisystab`, `lssal`, `memdisk`, `minicmd`, `normal`, `ntfs`, `part_apple`, `part_msdos`, `part_gpt`, `password_pbkdf2`, `png`, `probe`, `reboot`, `regexp`, `search`, `search_fs_uuid`, `search_fs_file`, `search_label`, `sleep`, `smbios`, `squash4`, `test`, `true`, `video`, `xfs`, `zfs`, `zfscrypt`, `zfsinfo`

- ["平台特定"的模块](https://git.launchpad.net/~ubuntu-core-dev/grub/+git/ubuntu/tree/debian/build-efi-images?h=debian/2.06-2ubuntu12#n147)用于x86_64-efi架构，如：
    - `play`：启动时播放声音
    - `cpuid`：启动时检查CPU功能
    - `tpm`：提供可度量启动（Measured Boot） / [可信平台模块](../../04_硬件与外设/硬件/可信平台模块.md)支持
- ["高级"模块](https://git.launchpad.net/~ubuntu-core-dev/grub/+git/ubuntu/tree/debian/build-efi-images?h=debian/2.06-2ubuntu12#n159)，包含以下模块：
    - `cryptodisk`：从[Dm-crypt](../../08_安全与加密/静态数据加密/Dm-crypt.md)加密磁盘启动
    - `gcry_*algorithm*`：支持特定散列和加密算法
    - `luks`：从[LUKS](../../08_安全与加密/静态数据加密/Dm-crypt.md)加密的磁盘启动
    - {{ic|lvm}}：从[LVM](../../02_系统管理与内核/存储虚拟化/LVM.md)逻辑卷磁盘启动
    - `mdraid09`, `mdraid1x`, `raid5rec`, `raid6rec`：从[RAID](../../02_系统管理与内核/存储虚拟化/RAID.md)虚拟磁盘启动

你必须通过设置shell变量的方式构造GRUB模块列表，我们将变量设置为`GRUB_MODULES`。你也可以使用[最新的Ubuntu脚本](https://git.launchpad.net/~ubuntu-core-dev/grub/+git/ubuntu/tree/debian/build-efi-images)作为出发点，然后去除你系统不需要的模块。去除模块会让启动速度相对更快，并节省一些EFI系统分区的空间。

如果GRUB从UEFI shim 加载器启动，你还需要一个[Secure Boot Advanced Targeting (SBAT)](https://github.com/rhboot/shim/blob/main/SBAT.md) 文件/部分包含在EFI二进制文件中，来提高安全性。这个SBAT文件/部分包含GRUB二进制文件的元数据（版本，维护者，开发者，上游URL），能够让shim更容易确认存在安全漏洞的GRUB版本，并拒绝加载，如[UEFI shim 启动加载器安全启动生命周期改进](https://github.com/rhboot/shim/blob/main/SBAT.md)文档所述。

如果`grubx64.efi`缺少SBAT部分，第一阶段UEFI启动加载器shim会拒绝加载启动`grubx64.efi`。

GRUB安装后，会提供一个简单的SBAT*.csv*文件在`/usr/share/grub/sbat.csv`。

重新安装GRUB，使用提供的SBAT文件和包含所有需要模块的`GRUB_MODULES`，并签名：
重启，选择*MokManager*中的密钥，安全启动应该就可以工作了。

使用安全启动

安装完成后，参考[安全启动#实施安全启动](../启动过程/UEFI_安全启动.md#实施安全启动)来启用安全启动。

如果使用CA密钥，则可以使用 [sbctl](https://archlinux.org/packages/?name=sbctl)包 自动执行密钥管理、注册和文件签名，有关详细信息，请参阅[安全启动#sbctl](../启动过程/UEFI_安全启动.md#sbctl)。

BIOS 系统

GUID 分区表 (GPT) 特殊操作

使用BIOS引导[GPT](../../02_系统管理与内核/系统恢复/分区.md)的分区情况（BIOS/[GPT](../../02_系统管理与内核/系统恢复/分区.md)）下，必须使用 [BIOS 启动分区](https://www.gnu.org/software/grub/manual/grub/html_node/BIOS-installation.html#BIOS-installation)。GRUB将`core.img`嵌入到这个分区。

> **注意：**
>
> - 在尝试这种方法之前请记住不是所有的系统都支持这种分区方案，请参阅 [GUID 分区表](../../02_系统管理与内核/系统恢复/分区.md#GUID_Partition_Table)。
> - 此额外分区只由 GRUB 在 BIOS/GPT 分区方式中使用。对于 BIOS/MBR 分区方式，GRUB 会把`core.img`放到 [MBR 后间隙(post-MBR gap)](../启动过程/Arch%20的启动流程.md#BIOS)中去，而在 GPT 中并不能保证在第一个分区之前有可以这样使用的空间。
> - [UEFI](../启动过程/UEFI.md) 系统也不需要这额外分区，因为它不需要嵌入启动扇区。UEFI 系统需要有 [EFI 系统分区](../启动过程/EFI%20系统分区.md)。

安装 GRUB 前，在一个没有文件系统的磁盘上，创建一个1兆字节（使用 [fdisk](../../02_系统管理与内核/系统恢复/Fdisk.md) 或 [gdisk](../../02_系统管理与内核/系统恢复/GPT%20fdisk.md) 和参数`+1M`）的分区，将分区类型设置为 GUID `21686148-6449-6E6F-744E-656564454649`。

- 对于 [fdisk](../../02_系统管理与内核/系统恢复/Fdisk.md)，选择分区类型 `BIOS boot`。
- 对于 [gdisk](../../02_系统管理与内核/系统恢复/GPT%20fdisk.md)，选择分区类型代码 `ef02`。
- 对于 [parted](../../02_系统管理与内核/系统恢复/Parted.md)， 在新创建的分区上设置/激活 `bios_grub` 标记。

这个分区可以处于磁盘的前 2TB 空间中的任意位置，但需要在安装 GRUB 之前创建好。分区建立好后，按下面的命令安装启动管理器。

第一个分区之前的空间也可以用作 BIOS 启动分区，但是这会违反 GPT 对齐规范。因为这个分区不会经常访问，所以性能的影响很小，只不过有些分区工具会发出警告。可以在 [fdisk](../../02_系统管理与内核/系统恢复/Fdisk.md) 或 [gdisk](../../02_系统管理与内核/系统恢复/GPT%20fdisk.md) 中创建一个从 34 扇区开始，一直到 2047扇区的分区，然后按照上述方式设置类型。为了让其它分区对齐，可以最后再创建此分区。

主引导记录 (MBR) 特殊操作

一般来说，如果使用兼容 DOS 的分区对齐模式，[主引导记录](../../02_系统管理与内核/系统恢复/分区.md#主引导记录) 512 字节结束位置和第一个分区之间都有 31KB 的空闲空间。不过，为了提供足够的空间嵌入 GRUB 的`core.img`文件，建议将这个空间设置为 1 到 2 MB ([FS#24103](https://bugs.archlinux.org/task/24103))。 建议使用支持 1 MB [分区对齐](../../02_系统管理与内核/系统恢复/分区.md#Partition_alignment)的分区软件来分区， 因为这样也能满足非 512 字节扇区磁盘分区的需求（这一点就与嵌入`core.img`没有关系了）。

安装

（在 [archiso](../../00_关于与社区/Live%20Arch%20系统/Archiso.md) 环境下，比如[安装系统](../安装过程/安装指南.md)时，请记得在 arch-chroot 环境中操作。）安装软件包 [grub](https://archlinux.org/packages/?name=grub)包。如果之前安装过 [grub-legacy](https://aur.archlinux.org/packages/grub-legacy/)AUR，安装完成后 [grub](https://archlinux.org/packages/?name=grub)包 会取代它。然后运行

> **注意：**这里的`i386-pc`是有意为之，与你机器的实际架构无关， 其中 `*/dev/sdX*` 是要安装 GRUB 的**磁盘**(**不是分区**)，比如磁盘 `/dev/sda`、`/dev/nvme0n1`或者`/dev/mmcblk0`，而**不是**分

区 `/dev/sda1`，你可以查看[Device file#Block device names](../../02_系统管理与内核/系统管理/设备文件.md#Block_device_names)来获得块设备命名方案的详细描述。

现在你需要[生成主配置文件](#生成主配置文件)。

如果你的 `/boot` 使用了 [LVM](../../02_系统管理与内核/存储虚拟化/LVM.md)（逻辑分卷管理器），GRUB 可以安装到多个物理磁盘上。

> **提示：**对于安装 GRUB 的其他方式，例如安装到一个U盘上，可以参考 [其它安装方式](GRUB_技巧和窍门.md#其它安装方式)。

`grub-install` 命令的详细信息请参考 [grub-install(8)](https://man.archlinux.org/man/grub-install.8) 和 [GRUB 手册](https://www.gnu.org/software/grub/manual/grub/html_node/BIOS-installation.html#BIOS-installation)。

配置

完成安装之后，GRUB 在每次启动的时候加载配置文件 `/boot/grub/grub.cfg`。你可以使用工具来[生成 grub.cfg](#生成_grub.cfg)，或者可以手动[定制 grub.cfg](#定制_grub.cfg)。

生成 grub.cfg

本节只讲述如何编辑配置文件 `/etc/default/grub`。更多信息请见 [GRUB/技巧和窍门](GRUB_技巧和窍门.md)。

> **注意：**请记住，每当修改 `/etc/default/grub` 或者 `/etc/grub.d/` 中的文件之后，都需要再次[生成主配置文件](#生成主配置文件)。

> **警告：**如果新版本的GRUB改变了配置文件的句法规则，比如新版本的配置文件可能会使用现有GRUB二进制文件不支持的功能，错误的配置文件可能会导致系统无法引导，请更新或重新安装引导器（见[GRUB#UEFI 系统](#UEFI_系统)或[GRUB#BIOS 系统](#BIOS_系统)）

生成主配置文件

安装后,需要生成主配置文件 `/boot/grub/grub.cfg`。配置文件的生成过程受到 `/etc/default/grub` 中的选项和 `/etc/grub.d/` 下脚本的影响。对于`/boot/grub/grub.cfg`中的选项，GNU的[文档](https://www.gnu.org/software/grub/manual/grub/html_node/Simple-configuration.html)有各个选项的简明描述。

如果你没有进行额外配置，自动生成程序会在当前启动的系统的根文件系统中侦测配置文件。所以请确保系统已经启动或者已经通过 chroot 进入。

> **注意：**
>
> - 默认的文件路径是 `/boot/grub/grub.cfg`，而非 `/boot/grub/i386-pc/grub.cfg`。
>
> - 如果你是在 chroot 或者 systemd-nspawn 容器中运行 grub-mkconfig，可能会报 grub-probe 无法获取 "canonical path of /dev/sdaX" 错误而无法正常执行。此时可以尝试使用 arch-chroot，参见 [BBS post](https://bbs.archlinux.org/viewtopic.php?pid=1225067#p1225067)。

使用 grub-mkconfig 工具来生成 `/boot/grub/grub.cfg`：
自动生成脚本默认将在生成的配置文件中为所有已安装的 Arch Linux [内核](../../02_系统管理与内核/内核/内核.md)添加条目。

> **提示：**
>

> - 每次安装或者移除一个[内核](../../02_系统管理与内核/内核/内核.md)后，你都需要重新运行一次 grub-mkconfig 命令。
> - 若要管理多个 GRUB 条目，比如既使用 [linux](https://archlinux.org/packages/?name=linux)包又使用 [linux-lts](https://archlinux.org/packages/?name=linux-lts)包 内核，相关的提示可以参见 [GRUB/技巧和窍门#多个启动条目](GRUB_技巧和窍门.md#多个启动条目)。

如果想要自动为其他操作系统添加条目，请见[#探测其他操作系统](#探测其他操作系统)。

如果想要添加自定义条目，你可以编辑 `/etc/grub.d/40_custom` 文件，然后重新生成 `/boot/grub/grub.cfg`。或者你可以创建 `/boot/grub/custom.cfg` 文件然后把条目添加进这里面。修改 `/boot/grub/custom.cfg` 文件后不用再运行 grub-mkconfig 程序，因为 `/etc/grub.d/41_custom` 文件已经在生成的主配置文件中添加了相关的 `source` 语句来引用 `/boot/grub/custom.cfg`。

> **提示：**`/etc/grub.d/40_custom` 可以用做创建 `/etc/grub.d/*nn*_custom` 文件的模板，其中 `*nn*` 为优先级，规定脚本文件的执行顺序。而脚本文件的执行顺序决定了其所添加的条目在 GRUB 启动菜单中的位置。`*nn*` 应当比 `06` 大，以此保证重要的脚本能够优先执行。

如要参考自定义菜单条目的例子，请看[#启动菜单条目示例](#启动菜单条目示例)。

探测其他操作系统

想要让 grub-mkconfig 探测其他已经安装的系统并自动把他们添加到启动菜单中，安装软件包 [os-prober](https://archlinux.org/packages/?name=os-prober)包 并[挂载](../../02_系统管理与内核/文件系统/文件系统.md#挂载文件系统)包含其它系统引导程序的磁盘分区。然后重新运行 grub-mkconfig。如果你得到以下输出：`Warning: os-prober will not be executed to detect other bootable partitions`，你需要编辑`/etc/default/grub`并取消下面这一行的注释，如果没有相应注释的话就在文件末尾添加上：

```
GRUB_DISABLE_OS_PROBER=false
```

然后运行 grub-mkconfig 再试一次。

> **注意：**
>
> - 分区挂载点并不重要，*os-prober*读取`mtab`信息来确认并搜索引导程序的位置。
> - 记得每次运行 grub-mkconfig 之前都把包含其他操作系统引导程序的分区挂载上，以免这些操作系统的启动项丢失。
> - *os-prober*在chroot中可能无法正常运作。如果遇到这种情况，重启并引导进入系统后再次尝试。

> **提示：**你可能想让grub记住你上次选择的启动项，参见[GRUB/技巧和窍门#调用之前的启动条目](GRUB_技巧和窍门.md#调用之前的启动条目)

Windows

对于以 UEFI 模式安装的Windows，确保含有 Windows Boot Manager (`bootmgfw.efi`，Windows EFI 引导程序) 的分区被挂载，以root身份运行[os-prober](https://archlinux.org/packages/?name=os-prober)包来检测并为其生成启动菜单项。

对于以 BIOS 模式安装的 Windows，挂载 Windows 的*系统分区*（其[文件系统标签](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md#通过标签)应该为`System Reserved`或`SYSTEM`），以root身份运行[os-prober](https://archlinux.org/packages/?name=os-prober)包来检测并为其生成启动菜单项。

> **注意：**
>
> - `os-prober` 会尝试使用 `grub-mount` 来挂载并检测某个分区中是否存在所必需的 `.efi` 文件。可能需要安装 [fuse3](https://archlinux.org/packages/?name=fuse3)包 以确保 `grub-mount` 正常工作，否则将无法检测到 Windows 系统。

> - 通过默认 Linux 驱动挂载的 NTFS 分区可能无法被检测到。如果 GRUB 无法检测到，尝试安装[NTFS-3G](../../02_系统管理与内核/FUSE/NTFS-3G.md)并重新挂载分区。

额外的参数

如想为 Linux 镜像添加额外的参数，你可以在 `/etc/default/grub` 中设置 `GRUB_CMDLINE_LINUX` 和 `GRUB_CMDLINE_LINUX_DEFAULT` 变量。生成普通启动项时，这两个参数的值会合并在一起传给内核。生成 recovery 启动项时，仅使用 `GRUB_CMDLINE_LINUX` 参数。

两个参数不是一定要一起用。例如要系统支持[休眠](../../02_系统管理与内核/电源管理/电源管理_挂起与休眠.md#休眠)后恢复,可以使用 `GRUB_CMDLINE_LINUX_DEFAULT="resume=UUID=*uuid-of-swap-partition* quiet"`，其中 `*uuid-of-swap-partition*` 是你的交换分区的 [UUID](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md#通过_uuid)。这样在生成 recovery 启动项时，将不会启用 resume 功能，也不会有 `quiet` 参数来省略启动时的内核信息。而其他的普通启动项会包含它们。

grub-mkconfig 默认使用 [UUID](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md#通过_uuid) 定位根文件系统，要禁用此设置，取消 `GRUB_DISABLE_LINUX_UUID=true` 的注释。

要生成 GRUB recovery 启动项，需要确保在 `/etc/default/grub` 中 `GRUB_DISABLE_RECOVERY` 没有设置为 `true`。

更多信息请参考[内核参数](../../02_系统管理与内核/内核/内核参数.md)。

设置启动菜单的顶层启动项

默认情况下*grub-mkconfig*使用`sort -V`对内核进行排序，并把第一个内核作为顶层启动项。也就是说，如果你同时安装了[linux-lts](https://archlinux.org/packages/?name=linux-lts)包和[linux](https://archlinux.org/packages/?name=linux)包 ，因为`/boot/vmlinuz-linux-lts`排在`/boot/vmlinuz-linux`前面，LTS内核会作为顶层启动项，这可能是不希望的结果。在`/etc/default/grub`中指定`GRUB_TOP_LEVEL=*内核路径*`可以覆盖自动排序结果。要让普通内核作为顶层启动项，设置为`GRUB_TOP_LEVEL="/boot/vmlinuz-linux"`。

LVM

> **警告：**GRUB 不支持 thin-provisioned 逻辑卷。

如果你的 `/boot` 或者 `/` 分区使用了 [LVM](../../02_系统管理与内核/存储虚拟化/LVM.md)，确保 `lvm` 模块已经预先加载好。

/etc/default/grub

GRUB_PRELOAD_MODULES="... lvm"

独立磁盘冗余阵列（RAID）

GRUB 可以很方便地操作 [RAID](../../02_系统管理与内核/存储虚拟化/RAID.md) 卷，你只需加载 GRUB 模块 `mdraid09` 或者 `mdraid1x` 就可以像其他卷一样进行操作了。

/etc/default/grub

GRUB_PRELOAD_MODULES="... mdraid09 mdraid1x"

例如 `/dev/md0` 写成：

set root=(md/0)

而 RAID 卷上的分区（如 `/dev/md0p1`）则是：

set root=(md/0,1)

如要在 `/boot` 分区使用 RAID1 时（或者 `/boot` 位于使用了 RAID1 的根分区之中）安装 GRUB，对于 BIOS 系统，直接在各个驱动器上运行 grub-install 即可，就像这样：
上例中 `/boot` 所在的 RAID 1 序列位于 `/dev/sda` 和 `/dev/sdb` 上。

> **注意：**GRUB 支持从 [Btrfs](../../02_系统管理与内核/文件系统/Btrfs.md) RAID 0/1/10 启动，但 **不支持** RAID 5/6。对 RAID 5/6，你可以使用 [mdadm](../../02_系统管理与内核/存储虚拟化/RAID.md#安装)，这个 GRUB 是支持的。

加密的/boot

GRUB 还专门支持从加密的 `/boot` 启动。这需要解锁一个 [LUKS](../../08_安全与加密/静态数据加密/Dm-crypt.md) 块设备，来读取配置文件以及加载 [initramfs](../启动过程/Arch%20的启动流程.md#initramfs) 和[内核](../../02_系统管理与内核/内核/内核.md)。这个选项试图解决[未加密的 boot 分区](../../08_安全与加密/静态数据加密/Dm-crypt_特殊应用.md#保护未加密的boot分区)问题。

> **提示：**`/boot` **不需要** 专门放到一个单独的分区，它也可以就留在系统的根目录 `/` 下面。

> **警告：**GRUB 2.12rc1 仅对 LUKS2 提供了有限的支持，详情请见[#LUKS2](#LUKS2)节。

要启用这个功能，使用 [LUKS](../../08_安全与加密/静态数据加密/Dm-crypt.md) 将 `/boot` 所在的分区加密，然后在 `/etc/default/grub` 中添加如下选项：
/etc/default/grub

GRUB_ENABLE_CRYPTODISK=y

grub-install 使用这个选项来生成 `core.img`，所以在修改这个选项或加密分区之后要重新[安装 grub](#安装)。

如果没有进一步的修改，你需要两次输入一个密码：第一次是为了让 GRUB 在启动伊始解锁 `/boot` 的挂载点，第二次是在 initramfs 的要求下解锁根文件系统。你可以用 [keyfile](../../08_安全与加密/静态数据加密/Dm-crypt_设备加密.md#With_a_keyfile_embedded_in_the_initramfs) 来避免密码输入过程。

> **警告：**
>
> - 如果你想要 [生成主配置文件](#生成主配置文件)，确保 `/boot` 已经挂载好了。
> - 为了进行与 `/boot` 的挂载点有关的系统更新，确保在进行更新之前已经对加密的 `/boot` 进行了解锁和挂载。如果使用了独立的 `/boot` 分区，这个可以通过使用 [crypttab](../../08_安全与加密/静态数据加密/Dm-crypt_系统配置.md) 和一个 [keyfile](../../08_安全与加密/静态数据加密/Dm-crypt_设备加密.md#With_a_keyfile_embedded_in_the_initramfs) 在启动的时候自动完成。

> **注意：**
>
> - 如果你使用了特别的键盘映射，默认安装的 GRUB 是不知道的。这关系到如何输入密码来解锁 LUKS 块设备。请查阅 [GRUB/技巧和窍门#为早期启动手动配置核心映像](GRUB_技巧和窍门.md#为早期启动手动配置核心映像)
> - 如果你遇到问题没法显示输入密码的界面（与 cryptouuid, cryptodisk相关的错误，或者 "device not found"），可以试着重新安装 GRUB，并在 `grub-install` 命令的尾部加上 `--modules="part_gpt part_msdos"`。

> **提示：**你可以使用 [pacman hooks](https://bbs.archlinux.org/viewtopic.php?id=234607) 来在升级时涉及到 `/boot` 中的文件的时候自动挂载它。

LUKS2

根据[#安装](#安装)一节的内容，使用`grub-install`创建支持LUKS的可引导GRUB镜像。请注意下面的事项：

- GRUB 2.06 添加了对 LUKS2 的初步支持，GRUB 2.12rc1 也只是部分解决了一些限制，参见 [GRUB bug #55093](https://savannah.gnu.org/bugs/?55093) 。

- GRUB 2.12rc1 版本的`grub-install`可以创建一个解锁LUKS2的镜像。但是仅支持PBKDF2，不支持Argon2。
- Argon2id (*cryptsetup* 默认) 和 Argon2i PBKDFs是不支持的，仅支持 PBKDF2 。

> **提示：**你可以使用 [grub-improved-luks2-git](https://aur.archlinux.org/packages/grub-improved-luks2-git/)AUR ，该包为LUKS2和Argon支持打了补丁。但请注意该包的Argon支持需要UEFI系统。

> **注意：**早于 GRUB 2.12rc1 的版本需要配合一个定制GRUB配置文件，使用`grub-mkimage`手动创建一个EFI二进制文件。例如，通过`/boot/grub/grub-pre.cfg`调用`cryptomount`，`insmod normal`和`normal`。这在新版里不再需要，`grub-install` 已经足够了。但是从2.06版升级后，你可能需要至少运行`grub-mkconfig -o /boot/grub/grub.cfg`一次。

如果在启动过程中遇到密码无效（invalid passphrase）的提示并最后进入GRUB救急模式，尝试运行命令：`cryptomount -a`来挂载所有加密分区，或者使用`cryptomount -u $crypto_uuid`挂载指定分区，接下来和平常一样，运行`insmod normal` 和 `normal`。

如果确定输入的密码正确，但密码输入后立刻返回密码无效（invalid passphrase）的错误，检查是否指定了正确的加密模块。使用`cryptsetup luksDump */dev/nvme0n1p2*`检查 hash function（SHA-256，SHA-512）是否匹配镜像安装的模块（`gcry_sha256`， `gcry_sha512`），PBKDF算法是否为pbkdf2。可以使用 `cryptsetup luksConvertKey --hash *sha256* --pbkdf pbkdf2 */dev/nvme0n1p2*`更改加密分区的 hash 和 PBDKDF 算法。正常情况下，输入密码后需要花费几秒时间处理，而不是立刻返回结果。

定制 grub.cfg

这一节讲述如何在 `/boot/grub/grub.cfg` 中手工创建 GRUB 启动条目，而非使用 grub-mkconfig。

基础的 GRUB 配置文件使用如下的设置：

- `(hd*X*,*Y*)` 为磁盘 *X* 上的分区 *Y*，分区编号从 1 开始，磁盘编号从 0 开始。
- `set default=*N*` 为在用户选择时间内没有进行选择时的默认启动条目。
- `set timeout=*M*` 即在使用默认条目启动前，等待用户自行选择的时间为 *M* 秒。
- `menuentry "title" {entry options}` 为一个标题为 `title` 的启动条目。
- `set root=(hd*X*,*Y*)` 设置 `\boot` 分区，即内核和 GRUB 模块存储的位置。（`\boot` 不一定要位于一个独立的分区，可能是根分区(`/`) 下面的一个目录。）

LoaderDevicePartUUID

[GPT分区自动挂载](../初始化/Systemd.md#GPT分区自动挂载)使用的[systemd-gpt-auto-generator(8)](https://man.archlinux.org/man/systemd-gpt-auto-generator.8)需要`LoaderDevicePartUUID`UEFI变量以正常工作。要使GRUB设置该变量，在`grub.cfg`中加载`bli`模块：

```
if [ "$grub_platform" = "efi" ]; then insmod bli fi
```

启动菜单条目示例

> **提示：**在使用由 grub-mkconfig 所生成的 `/boot/grub/grub.cfg` 时，这些启动条目仍然是可以用的。将它们添加到 `/etc/grub.d/40_custom` 中然后[重新生成主配置文件](#生成主配置文件)，或者将它们添加到 `/boot/grub/custom.cfg`中。

若要管理多个 GRUB 条目，比如既使用 [linux](https://archlinux.org/packages/?name=linux)包 又使用 [linux-lts](https://archlinux.org/packages/?name=linux-lts)包 内核，相关的提示可以参见 [GRUB/技巧和窍门#多个启动条目](GRUB_技巧和窍门.md#多个启动条目)。

对于 [Archiso](../../00_关于与社区/Live%20Arch%20系统/Archiso.md) 和 [Archboot](https://archboot.com) 启动菜单条目，参见 [Multiboot USB drive#Boot entries](../安装过程/多引导%20USB%20设备.md#Boot_entries).

GRUB 命令

"关机" 菜单项
menuentry "System shutdown" { echo "System shutting down..." halt }

"重启" 菜单项
menuentry "System restart" { echo "System rebooting..." reboot }

"UEFI固件设置" 菜单项
if [${grub_platform} == "efi"]; then menuentry 'UEFI Firmware Settings' --id 'uefi-firmware' { fwsetup } fi

EFI 可执行文件

在启用了 UEFI 模式时，GRUB 可以 chainload 其它 EFI 可执行文件。

> **提示：**如要让这些启动条目仅在 GRUB 处于 UEFI 模式的时候显示，只需把它们放到下面的 `if` 语句中：
>
> ```
> if [ ${grub_platform} == "efi" ]; then
> 放入仅 UEFI 显示的启动条目
> fi
> ```

UEFI Shell

要启动 [UEFI Shell](../启动过程/UEFI.md#UEFI_Shell)，你可以将它放在 [EFI 系统分区](../启动过程/EFI%20系统分区.md)的根目录里，然后添加如下菜单条目：
menuentry "UEFI Shell" { insmod fat insmod chain search --no-floppy --set=root --file /shellx64.efi chainloader /shellx64.efi }

gdisk

下载 [gdisk EFI application](../../02_系统管理与内核/系统恢复/GPT%20fdisk.md#gdisk_EFI_application) 然后复制 `gdisk_x64.efi` 到 `*esp*/EFI/tools/`。
menuentry "gdisk" { insmod fat insmod chain search --no-floppy --set=root --file /EFI/tools/gdisk_x64.efi chainloader /EFI/tools/gdisk_x64.efi }

Chainload 一个统一的内核镜像

如果你有一个按照 [Secure Boot](../启动过程/UEFI_安全启动.md) 或者其他方法生成的 *.efi* 文件，你可以把它添加到启动菜单里。例如：
menuentry "Arch Linux " { insmod fat insmod chain search --no-floppy --set=root --fs-uuid FILESYSTEM_UUID chainloader /EFI/Linux/arch-linux.efi }

多系统启动

GNU/Linux

假设另一个发行版位于 `sda2`：
menuentry "Other Linux" { set root=(hd0,2) linux /boot/vmlinuz (add other options here as required) initrd /boot/initrd.img (if the other kernel uses/needs one) }

或者让 GRUB 根据 UUID 或文件系统标签查找正确的分区：
menuentry "Other Linux" {

search --no-floppy --set=root --fs-uuid 763A-9CB6

linux /boot/vmlinuz （按需求在这里添加其他的选项，例如：root=UUID=763A-9CB6 ） initrd /boot/initrd.img （如果其他的内核需要的话） }

如果其他发行版已经有一个有效的`/boot`文件夹，并安装了GRUB、`grub.cfg`、kernel和initramfs，可以指示GRUB在启动时即时加载这些其他`grub.cfg`文件。例如，对于`hd0`和第四个GPT分区：

menuentry "configfile hd0,gpt4" { insmod part_gpt insmod btrfs insmod ext2 set root='hd0,gpt4' configfile /boot/grub/grub.cfg }

选择此条目时，GRUB会从另一卷加载`grub.cfg`文件并显示该菜单。在`configfile`返回后，文件中的命令所做的任何环境变量改变将不会被保留。按`Esc`返回到第一个GRUB菜单。

UEFI/GPT 模式下安装的 Windows

这个模式寻找 Windows 的启动加载器的位置，然后当用户选择了相应的菜单条目的时候，通过链式加载的方法在 GRUB 之后加载它。这里主要的任务是找到 EFI 系统分区然后从上面运行启动加载器。

> **注意：**这个启动项仅在 UEFI 模式下才起作用，而且 Windows 和 UEFI 的位数必须相同。如果 GRUB 是 BIOS 模式，这个方法无效。参考 [Arch + Windows 双系统#Windows 在 UEFI 和 BIOS 上的限制](../启动过程/Arch%20+%20Windows%20双系统.md#Windows_在_UEFI_和_BIOS_上的限制) 和 [Arch + Windows 双系统#UEFI 和 BIOS 引导加载程序的限制](../启动过程/Arch%20+%20Windows%20双系统.md#UEFI_和_BIOS_引导加载程序的限制)。

if ["${grub_platform}" == "efi"]; then menuentry "Microsoft Windows Vista/7/8/8.1 UEFI/GPT" { insmod part_gpt insmod fat insmod chain search --no-floppy --fs-uuid --set=root $hints_string $fs_uuid chainloader /EFI/Microsoft/Boot/bootmgfw.efi } fi

其中 `$hints_string` 和 `$fs_uuid` 由下述两个命令得到。

`$fs_uuid` 命令检测 EFI 系统分区的 UUID：

1ce5-7f28

或者你可以运行 `blkid --fs` 然后从结果中找到 EFI 系统分区的 UUID。

`$hints_string` 命令可以确定 EFI 系统分区的位置，在当前的例子中是 harddrive 0：

--hint-bios=hd0,gpt1 --hint-efi=hd0,gpt1 --hint-baremetal=ahci0,gpt1

这两个命令都是假设 Windows 使用的 ESP 是挂载在`$esp`上的。当然，Windows的 EFI 文件路径可能有变,因为这就是Windows....

BIOS/MBR 模式下安装的 Windows

> **注意：**GRUB 支持直接启动 `bootmgr`，如今启动 BIOS/MBR 模式下安装的 Windows 时不再需要[链式加载](https://www.gnu.org/software/grub/manual/grub.html#Chain_002dloading)分区启动扇区了。

> **警告：**`bootmgr` 位于**系统分区**(**system partition**)，而不是 Windows 系统所在的分区（通常为 `C:` 盘）。系统分区的[文件系统标签](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md#通过标签) 是 `System Reserved` 或者 `SYSTEM` 而且这个分区的容量只有大概 100 到 549 MB。详情参考 [Wikipedia:System partition and boot partition](https://en.wikipedia.org/wiki/System_partition_and_boot_partition)。

本节假设你的 Windows 分区是 `/dev/sda1`。如果分区不同，需要对每一处 `hd0,msdos1` 进行修改。

> **注意：**这些菜单条目仅在 BIOS 启动模式下可用，不能在 UEFI 模式下安装的 GRUB 上使用。参考 [Arch + Windows 双系统#Windows 在 UEFI 和 BIOS 上的限制](../启动过程/Arch%20+%20Windows%20双系

统.md#Windows_在_UEFI_和_BIOS_上的限制）和 [Arch + Windows 双系统#UEFI 和 BIOS 引导加载程序的限制](../启动过程/Arch%20+%20Windows%20双系统.md#UEFI_和_BIOS_引导加载程序的限制)。

在所有例子里，`*XXXX-XXXX*`是指文件系统的 UUID，可以通过 `lsblk --fs` 命令得到。

对于 Windows Vista/7/8/8.1/10：

```
if [ "${grub_platform}" == "pc" ]; then menuentry "Microsoft Windows Vista/7/8/8.1/10 BIOS/MBR" { insmod part_msdos insmod ntfs insmod ntldr search --no-floppy --fs-uuid --set=root --hint-bios=hd0,msdos1 --hint-efi=hd0,msdos1 --hint-baremetal=ahci0,msdos1 XXXX-XXXX ntldr /bootmgr } fi
```

对于 Windows XP：

```
if [ "${grub_platform}" == "pc" ]; then menuentry "Microsoft Windows XP" { insmod part_msdos insmod ntfs insmod ntldr search --no-floppy --fs-uuid --set=root --hint-bios=hd0,msdos1 --hint-efi=hd0,msdos1 --hint-baremetal=ahci0,msdos1 XXXX-XXXX ntldr /ntldr } fi
```

> **注意：**在某些情况下，可能在安装 Windows 8 之前就已经安装了GRUB。启动 Windows 时可能会出现 `\boot\bcd` 报错（错误代码为 `0xc000000f`）。要修复这个问题，可以进入 Windows Recovery Console（安装磁盘中的 `cmd.exe`）然后运行：
>
> ```
> X:\> bootrec.exe /fixboot
> X:\> bootrec.exe /RebuildBcd
> ```
>
> **不要** 使用 `bootrec.exe /Fixmbr`，因为那会将 GRUB 清除掉。或者你可以用 Troubleshooting 菜单里的 Boot Repair 函数，它不会清除 GRUB 而且可以修复大部分错误。而且最好是保证连接电脑的介质 **只有** 目标硬盘和你的可启动介质，如果你连接了其他的设备，Windows 很可能会没法修复启动信息。

使用标签

通过使用`search`的`--label`选项，可以使用人类易于阅读的文件系统标签。首先请[确保你的文件系统有唯一标签](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md#by-label)；

然后你就可以使用标签来添加一个条目：

```
menuentry "Arch Linux, session texte" { search --label --set=root archroot linux /boot/vmlinuz-linux root=/dev/disk/by-label/archroot ro initrd /boot/initramfs-linux.img }
```

使用 GRUB 命令行

MBR 太小，不足以存储所有的 GRUB 模块，所以 MBR 里面只有启动目录和一些很基本的命令。GRUB 的主要功能通过 `/boot/grub` 里的模块实现，按需加载。出现错误时，GRUB 可能不能引导启动（比如磁盘分区发生了变化）。这时候，一般会出现命令行界面。

GRUB 不止提供一个 shell，如果 GRUB 不能读取到启动目录配置，但是能找到磁盘，你很可能会进入 "正常" shell：

```
grub>
```

如果有更严重的问题（比如 GRUB 找不到必须的文件了），GRUB 就可能会让你进入 "救急" shell：

```
grub rescue>
```

救急模式下的 shell 是正常 shell 的一个严格的子集，其支持的功能更少。如果不幸进入了救急模式的 shell 里，首先尝试加载 normal 模块，然后启动正常 shell：

grub rescue> set prefix=(hdX,Y)/boot/grub grub rescue> insmod (hdX,Y)/boot/grub/i386-pc/normal.mod rescue:grub> normal

分页支持

GRUB 支持对长输出进行分页（比如运行 `help` 的输出）。不过只能在正常 shell 中支持，在救急 shell 中则不支持。开启分页支持需要在 GRUB 命令行中键入：

sh:grub> set pager=1

使用命令行引导操作系统

grub>

可以使用 GRUB 命令行引导操作系统，一个典型的应用场景是通过"chainloading"来引导储存在一个驱动器或者分区中的 Windows 或 Linux 系统。

ChainLoading 的意思是用当前的启动加载器去加载另一个启动加载器，所以叫做**链式**加载。

要被加载的另一个启动加载器可能嵌入在一个有分区表的磁盘的头部 (MBR)，或在一个未分区磁盘或者一个分区的头部 (VBR)，也可能在使用 UEFI 的情形下是一个 EFI 可执行文件。

链式加载一个分区的 VBR

set root=(hdX,Y) chainloader +1 boot

X=0,1,2... Y=1,2,3...

比如链式加载一个位于首磁盘,首分区上的 Windows：

set root=(hd0,1) chainloader +1 boot

同样也可以使用 GRUB 链式加载另一个分区引导扇区上的 GRUB。

链式加载磁盘的 MBR 或未分区磁盘的 VBR

set root=hdX chainloader +1 boot

链式加载 UEFI 模式下安装的 Windows/Linux

insmod fat set root=(hd0,gpt4) chainloader (${root})/EFI/Microsoft/Boot/bootmgfw.efi boot

`insmod fat` 用来加载 FAT 文件系统模块，以访问 EFI 系统分区上的 Windows 启动加载器。`(hd0,gpt4)` 或 `/dev/sda4` 是该示例中的 EFI 系统分区。 `chainloader` 一行中的条目用来指定需要被链式加载的 .efi 文件。

正常加载

请参考[#使用救急控制台](#使用救急控制台)中的例子。

使用救急控制台

请先阅读[#使用 GRUB 命令行](#使用_GRUB_命令行)。如果无法进入正常的命令行，请尝试使用 Live CD 或者其他救急磁盘引导，然后修正配置错误，重新安装 GRUB。不过有些时候我们手上没有此类救急磁盘，这时救急控制台就可以派上用场了。

GRUB 应急控制台里可用的命令有 `insmod`，`ls`，`set` 和 `unset`。这个例子里用了 `set` 和 `insmod`。`set` 用来修改变量，`insmod` 用来加载模块以添加功能。

首先，用户必须知道启动分区 (`/boot`) 所在位置（是一个独立的分区或者是根目录下的子目录），然后设置：

grub rescue> set prefix=(hdX,Y)/boot/grub

其中 X 是物理驱动器的编号，而 Y 是分区的编号。

> **注意：**如果启动分区是个独立的分区，要在路径中省略 `/boot`（例如键入 `set prefix=(hdX,Y)/grub`）。

通过加载 `linux` 模块来扩展命令行的功能：

grub rescue> insmod i386-pc/linux.mod

或者直接

grub rescue> insmod linux

这个模块会启动对我们熟悉的 `linux` 和 `initrd` 命令的支持。

比如要启动 Arch Linux：

set root=(hd0,5) linux /boot/vmlinuz-linux root=/dev/sda5 initrd /boot/initramfs-linux.img boot

如果 `/boot` 在单独分区上（例如在用 UEFI 的时候），适当地进行修改：

> **注意：**因为 boot 是一个单独的分区而不是根分区的一部分，你得手动把它的地址写清楚，格式和前面的 prefix 一样。

set root=(hd0,5) linux (hdX,Y)/vmlinuz-linux root=/dev/sda6 initrd (hdX,Y)/initramfs-linux.img boot

> **注意：**如果你在执行 `linux` 命令的时候遇到了 `error: premature end of file /YOUR_KERNEL_NAME`，你可以尝试用 `linux16` 来替代。

成功启动 Arch Linux 后，用户可以修正 `grub.cfg` 然后重新安装 GRUB。

为了完全修正错误和重新安装 GRUB，可能需要修改 `/dev/sda`。详情请参考[#安装](#安装)（BIOS）或者[#安装 2](#安装_2)（UEFI）章节。

移除 GRUB

UEFI系统

删除*grub*前，确保安装了其它启动加载器并配置其接管启动引导工作。

$ efibootmgr

BootOrder: 0003,0001,0000,0002 Boot0000 *Windows Boot Manager HD(2,GPT,4dabbedf-191b-4432-bc09-8bcbd1d7dabf,0x109000,0x32000)/File(\EFI\Microsoft\Boot\bootmgfw.efi) Boot0001* GRUB HD(2,GPT,4dabbedf-191b-4432-bc09-8bcbd1d7dabf,0x109000,0x32000)/File(\EFI\GRUB\grubx64.efi) Boot0002 *Linux-Firmware-Updater HD(2,GPT,5dabbedf-191b-4432-bc09-8bcbd1d7dabf,0x109000,0x32000)/File(\EFI\arch\fwupdx64.efi) Boot0003* Linux Boot Manager HD(2,GPT,4dabbedf-191b-4432-bc09-8bcbd1d7dabf,0x109000,0x32000)/File(\EFI\systemd\systemd-bootx64.efi)

如果`BootOrder`第一项是*grub*，安装其它启动加载器并放在其前面（比如上面的[systemd-boot](Systemd-boot.md)），然后*grub*可以通过使用*bootnum*删除：
再删除`*esp*/EFI/grub` 和 `/boot/grub` 目录。

BIOS系统

要将*grub*替换为其它BIOS启动加载器，只需要安装该加载器，[MBR启动代码](../../02_系统管理与内核/系统恢复/分区.md#MBR（引导代码）)将会被覆盖。

`grub-install`还创建了`/boot/grub`目录，需要手动删除。如果你还想再次安装*grub*，可以保留该目录。

在迁移到UEFI/GPT后，使用dd移除MBR启动代码（英语：[dd#Remove bootloader](https://wiki.archlinux.org/title/dd#Remove_bootloader)）。

疑难解答

os-prober 段错误

> **提示：**此问题已于 2:2.12.r260.gaae2ea61-1 被修复。

- [archlinux/packaging/packages/grub#11](https://gitlab.archlinux.org/archlinux/packaging/packages/grub/-/issues/11)
- [论坛主题](https://bbs.archlinux.org/viewtopic.php?pid=2228660#p2228660)

在将 [grub](https://archlinux.org/packages/?name=grub)包 升级到 *grub 2:2.12.r212.g4dc616657-2* 后，运行 `grub-mkconfig` 时 [os-prober](https://archlinux.org/packages/?name=os-prober)包 会出现段错误。

`grub-mkconfig` 的输出类似于：

Generating grub configuration file ... Warning: os-prober will be executed to detect other bootable partitions. Its output will be used to detect bootable binaries on them and create new boot entries. /usr/lib/os-probes/50mounted-tests: line 72: 164878 Segmentation fault grub-mount "$partition" "$tmpmnt" 2> /dev/null Adding boot menu entry for UEFI Firmware Settings ... done

此时尝试进入 Windows 系统，会显示：

Setting partition type to 0x83 error: invalid signature.

Press any key to continue..._

这似乎是上游问题，同样的问题影响了 [archlinux/packaging/packages/grub#12](https://gitlab.archlinux.org/archlinux/packaging/packages/grub/-/issues/12)。已有部分用户提供了解决方案：

降级 grub

将 [grub](https://archlinux.org/packages/?name=grub)包 [降级](../包管理/降级软件包.md)至 *2:2.12-3*。

自行编写启动条目

参照[#定制 grub.cfg](#定制_grub.cfg) 一节自行编写启动条目。

对于 BIOS/MBR 模式下安装的 Windows，使用[上文](#BIOS/MBR_模式下安装的_Windows)提供的示例配置（使用 *search* 关键字）很可能不奏效，对此，请使用以下配置：

menuentry "Microsoft Windows Vista/7/8/8.1/10 BIOS/MBR" { insmod part_msdos insmod ntfs set root=(hdX,msdosY) chainloader +1 }

不支持的文件系统

如果根目录分区是一个GRUB不支持的文件系统，那就需要为 `/boot` 单独分区，并使用一个支持的文件系统。某些情况下，GRUB 的开发版 [grub-git](https://aur.archlinux.org/packages/grub-git/)AUR 可能已经支持了那个文件系统。

如果将 GRUB 和一个不支持的文件系统一起用，它将无法提取到你的驱动器的 [UUID](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md#通过_uuid)，只能使用传统的名称 `/dev/*sdXx*` 来代替，而这个名称是可能会变化的。此时你可能需要手动编辑 `/boot/grub/grub.cfg`，将 `root=/dev/*sdXx*` 改为 `root=UUID=*XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX*`。你可以使用 `blkid` 命令来获取你的设备的 UUID ，参见 [Persistent block device naming](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md)。

尽管 GRUB 自2.0.4起支持 [F2FS](../../02_系统管理与内核/文件系统/F2FS.md) 文件系统，GRUB还是不能从启用`extra_attr` flag 的 F2FS 分区正确读取引导文件。

启用调试信息

> **注意：**每当你[生成主配置文件](#生成主配置文件)的时候，这个设定就会被覆盖掉。

在 `grub.cfg` 中添加：
set pager=1 set debug=all

出现 “msdos-style” 错误消息
grub-setup: warn: This msdos-style partition label has no post-MBR gap; embedding will not be possible! grub-setup: warn: Embedding is not possible. GRUB can only be installed in this setup by using blocklists. However, blocklists are UNRELIABLE and its use is discouraged. grub-setup: error: If you really want blocklists, use --force.

这个错误可能当你在 VMware 容器里面安装 GRUB 的时候出现。请阅读[相关链接](https://bbs.archlinux.org/viewtopic.php?pid=581760#p581760)。这种情况是因为首分区直接从 MBR 后开始（即第 64 个扇区），而不是和正常的那样在 MBR 后面有 1MB（2048个扇区）的间隙。请参阅[主引导记录 (MBR) 特殊操作](#主引导记录_(MBR)_特殊操作)。

UEFI 异常

常见安装错误

- 一些UEFI设备会遇到`Could not prepare Boot variable: Read-only file system`错误。请通过重新挂载`/sys/firmware/efi/efivars`开启读写权限，参见[Gentoo Wiki](https://wiki.gentoo.org/wiki/Handbook:AMD64/Installation/Bootloader#Install)。
- 如果你在将 *sysfs* 或者 *procfs* 与 *grub-install* 一起使用的时候，遇到要求你必须运行 `modprobe efivarfs` 的问题，尝试[使用上文的命令挂载efivarfs](../启动过程/UEFI.md#挂载_efivarfs)
- 如果不用 `--target` 或者 `--directory` 选项，grub-install 不知道应该安装哪一个固件。此时 `grub-install` 会输出 `source_dir does not exist. Please specify --target or --directory`。
- 如果在运行 grub-install 以后，被告知`error: *esp* doesn't look like an EFI partition`，那这个分区很可能不是[FAT32](../../02_系统管理与内核/文件系统/FAT.md)格式。

在固件启动管理器中创建一个GRUB条目

`grub-install`会自动尝试在启动管理器中创建一个菜单项。如果没有，请参见[UEFI#efibootmgr](../启动过程/UEFI.md#efibootmgr)，了解使用`efibootmgr`来创建菜单项的说明。问题也可能在于你没有以UEFI模式启动CD/USB，参见[UEFI#Create UEFI bootable USB from ISO](../启动过程/UEFI.md#Create_UEFI_bootable_USB_from_ISO)。

另一种在固件引导管理器中创建GRUB条目的方法是使用`efibootmgr -c`，这个命令默认 `/dev/sda1` 是EFI系统分区，并且挂载到了`/boot/efi`，这也是`efibootmgr`的默认行为。该命令在引导选项表顶部创建了一个名为"Linux"的启动项目，传递其它参数可调整默认行为。生成的OS引导器位于`\EFI\arch\grub.efi`。

启动时进入了救急控制台

如果 GRUB 直接就启动到了救急控制台下，而且没报错，这可能是因为如下两种原因：

- 可能是因为 `grub.cfg` 丢失或者位置不对。如果 GRUB UEFI 安装时设定了 `--boot-directory` 参数，而 `grub.cfg` 文件却不在那里，就会发生这样的问题。
- 如果启动分区的分区号发生了变化（这个分区号会被直接编码到 `grubx64.efi` 文件中），也会出现这个问题。

GRUB UEFI 无法加载

下面是一个正常的 UEFI 的示例：

BootCurrent: 0000 Timeout: 3 seconds BootOrder: 0000,0001,0002 Boot0000 *GRUB HD(1,800,32000,23532fbb-1bfa-4e46-851a-b494bfe9478c)File(\EFI\GRUB\grubx64.efi) Boot0001* Shell HD(1,800,32000,23532fbb-1bfa-4e46-851a-b494bfe9478c)File(\shellx64.efi) Boot0002* Festplatte BIOS(2,0,00)P0: SAMSUNG HD204UI

如果启动后，屏幕直接变黑，几秒后就跳到了下一个启动项，根据[相关链接](https://bbs.archlinux.org/viewtopic.php?pid=981560#p981560)的说法是，将 GRUB 移动到 root 分区上可能会解决这个问题。必须先删除启动项，然后在移动 GRUB 后重建。操作之后上述命令的输出中，GRUB 条目应该像这样：

Boot0000* GRUB HD(1,800,32000,23532fbb-1bfa-4e46-851a-b494bfe9478c)File(\grubx64.efi)

缺省/后备启动路径

一些 UEFI 固件在显示 UEFI NVRAM 启动条目之前，需要在一个已知的位置上有一个可启动文件。如果是这种情况， `grub-install` 会说明 `efibootmgr` 添加了一个启动 GRUB 的条目，但这个条目不会在 VisualBIOS 启动顺序选择器中显示。解决方法是把 GRUB 安装到缺省/后备启动路径当中：
或者你可以把已经安装好的 GRUB EFI 执行文件移动到缺省/后备路径中：
"Invalid signature"（无效签名错误）

如果在启动 Windows 时出现了 "invalid signature" 错误（比如在重新分区或者添加了新硬盘后），删除 GRUB 的设备配置，然后让它重建一个：
`grub-mkconfig` 此时就应该生成了新的启动项了，包括 Windows。确认能启动成功后，再将备份文件 `/boot/grub/device.map-old` 删除。

引导过程卡死

如果在 GRUB 加载内核并初始化 ramdisk 后引导过程卡死了，又没有错误信息的话，请尝试移除 `add_efi_memmap` 这个内核参数。

其他系统不能自动发现 Arch Linux

有人发现有些发行版不能使用 `os-prober` 自动发现 Arch Linux。据说如果 `/etc/lsb-release` 文件存在的话，可以提高探测能力。这个文件和和更新工具可以在 [lsb-release](https://archlinux.org/packages/?name=lsb-release)包 包中找到。

在 chroot 环境下安装时遇到警告

当位于 chroot 环境里，要在 LVM 系统上安装 GRUB 的时候（比如在安装系统的时候），你可能会收到一个这样的警告：

/run/lvm/lvmetad.socket: connect failed: No such file or directory

或者

WARNING: failed to connect to lvmetad: No such file or directory. Falling back to internal scanning.

这是因为在 chroot 环境里面 `/run` 是不可用的。只要每个步骤都做对了，这些警告不会影响系统启动，你可以放心继续进行下一步的系统安装。

GRUB 加载非常慢

当磁盘空间很小的时候 GRUB 的加载时间可能会很长。如果你遇到了这样的问题，检查一下你的 `/boot` 或者 `/` 分区是不是有足够的剩余空间。

error: unknown filesystem（未知文件系统错误）

因某些原因，GRUB 可能会不能启动并输出 `error: unknown filesystem`。如果你确定所有的 [UUID](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md#通过_uuid) 都对了而且所有的文件系统都是有效而且被 GRUB 支持的，问题的原因可能是你的 [BIOS 启动分区](#GUID_分区表_(GPT)_特殊操作)不在驱动器的前 2 TB 空间里 。 选择一个分区工具调整这个分区，让它完全在前 2 TB 空间中，然后重新安装和配置 GRUB。

这个错误也可能是因为一个 [ext4](../../02_系统管理与内核/文件系统/Ext4.md) 文件系统拥有 GRUB 不支持的特性：

- `large_dir` - 不被支持。
- `metadata_csum_seed` - GRUB 2.11 将会支持([commit](https://git.savannah.gnu.org/cgit/grub.git/commit/?id=7fd5feff97c4b1f446f8fcf6d37aca0c64e7c763))。

> **警告：**当你在`/boot`中启用文件系统特性时，请确保grub支持你的 [文件系统](../../02_系统管理与内核/文件系统/文件系统.md) 特性。

grub-reboot 不能重新设定

GRUB 好像不能写入 Btrfs 格式的根分区。如果你使用 grub-reboot 来启动到另一个启动条目，就会没法更新其 on-disk 环境。要么换一个启动条目来运行 grub-reboot（例如在不同发行版之间切换的时候），要么考虑换个文件系统。你可以通过运行 `grub-editenv create` 来重设一个 "sticky" 条目，然后在 `/etc/default/grub` 中设置 `GRUB_DEFAULT=0`（不要忘了运行 `grub-mkconfig -o /boot/grub/grub.cfg`）。

不能在旧的 Btrfs 上进行安装

如果一个驱动器在没有创建分区表的情形下被格式化成 BTRFS（比如 /dev/sdx），之后又创建了一个分区表，那么会有部分 Btrfs 格式保留下来。大部分功能和操作系统都不会注意这个，但是 GRUB 则无法安装，即使使用 --force 参数也不行。
你可以把整个驱动器置零来解决问题，但还有一个办法既简单又能保留你的数据，那就是用 `wipefs -o 0x10040 /dev/sdx` 命令来擦掉 Btrfs superblock。

未找到 Windows 8/10

Windows 8/10 如果启用了 "Hiberboot", "Hybrid Boot" 或 "Fast Boot"，可能会导致 Windows 分区无法被挂载。所以 `grub-mkconfig` 无法找到安装的 Windows。在 Windows 里禁用 Hiberboot，然后它就可以被添加到 GRUB 菜单了。

GRUB 救急与加密启动

在使用加密启动，而你没法键入正确的密码的时候，就会进入 GRUB 救急命令行。

这个救急命令行只有有限的功能，可以使用下面的命令来完成启动：
grub rescue> cryptomount <分区> grub rescue> insmod normal grub rescue> normal

更好的介绍参见[这个帖子](https://blog.stigok.com/2017/12/30/decrypt-and-mount-luks-disk-from-grub-rescue-mode.html)。

GRUB 已安装但引导过程没有显示菜单

检查`/etc/default/grub`中的`GRUB_TIMEOUT` 是否设置成了`0`，设置为一个正数来调整GRUB条目加载前的等待时间，按秒计时。另外要检查`GRUB_TIMEOUT_STYLE`是否为`hidden`，设置为`menu`确保菜单显示。[重新生成主配置文件](#生成主配置文件)后，重启检查设置是否生效。

如果仍然没有解决问题，可能存在图形终端的兼容问题。设置`/etc/default/grub`中的`GRUB_TERMINAL_OUTPUT`为`console`关闭GRUB图形终端。

在旧的 Lenovo 设备上安装好 GRUB 后依然有“ERROR CODE 1962 - No operating system found”的错误消息

参见[这个链接](https://www.reddit.com/r/ManjaroLinux/comments/e682d6/fixing_lenovos_error_code_1962_by_spoofing_the/)。

参阅

- [Wikipedia:GNU GRUB](https://en.wikipedia.org/wiki/GNU_GRUB)
- [官方 GRUB 手册](https://www.gnu.org/software/grub/manual/grub.html)
- [GRUB 的 Ubuntu wiki 页面](https://help.ubuntu.com/community/Grub2)
- [描述为 UEFI 系统编译的 GRUB wiki 页面](https://help.ubuntu.com/community/UEFIBooting)
- [Wikipedia:BIOS Boot partition](https://en.wikipedia.org/wiki/BIOS_Boot_partition)
- [怎样配置 GRUB](https://web.archive.org/web/20160424042444/http://members.iinet.net/~herman546/p20/GRUB2%20Configuration%20File%20Commands.html#Editing_etcgrub.d05_debian_theme)

第 5 章　共 8 节　80 KB

包开发

自己写软件包，难点不在背下 PKGBUILD 的字段，而在把构建、调试与验证串成一条可靠的链路。本章先讲清 PKGBUILD 中的软件包名称、版本、一般变量、依赖关系与包关系，再说明 .SRCINFO 如何生成、语法与要求。

阅读上，PKGBUILD 与 .SRCINFO 两节必读，是其余内容的地基。其余按需取用：Clang 讲如何用它构建软件包，含 Qt、Rust 与静态分析工具；Ccache 用于缩短重复构建；Debuginfod 与调试/获取跟踪信息用于出问题后定位；OCaml 与 Eclipse 插件打包准则只服务特定生态，不涉及即可跳过。最易犯的错是把 .SRCINFO 当成手写文件、任其与 PKGBUILD 脱节，或以为装好 Ccache、Debuginfod 便自动生效——两者都要配置，缓存另有注意事项。

5.1　Clang

难度 入门 | **篇幅** 4.2 KB | **所属** 包开发

本节将讲到

- 安装
- 用 Clang 构建软件包
- Qt 软件包
- Rust 软件包
- 使用静态分析工具

Clang 是基于 LLVM 的 C/C++/Objective C/CUDA 编译器。最新版本依据 “带有 LLVM 例外条款的 Apache 2.0 许可证” 分发。

安装

安装 clang包 软件包。

用 Clang 构建软件包

**本文或本章节可能需要合并到makepkg 。**

附注： 用于 makepkg 的编译器标志（在 Talk:Clang 中讨论）

通用配置

要更改构建软件包时使用的默认编译器，请 编辑 以下文件：

```
/etc/makepkg.conf
...
export CC=clang
export CXX=clang++
```

若要使用 `libc++` 替代 GCC 的 `libstdc++` 作为 C++ 标准库：需先安装 libc++包 软件包，再在 `/etc/makepkg.conf` 的 `CXXFLAGS` 中添加 `-stdlib=libc++`。

如需支持链接时优化（LTO）：安装 lld包 软件包，再在 `/etc/makepkg.conf` 的 `LDFLAGS` 中添加 `-fuse-ld=lld`。

若以 `debug` 模式构建，还需从 `DEBUG_CFLAGS` 和 `DEBUG_CXXFLAGS` 中移除 `-fvar-tracking-assignments`，因为 Clang 不支持该选项。

注意： 对于指定了 GCC 专属构建选项的软件包，可能会出现构建错误，此时需编辑源软件包、PKGBUILD 文件，或注释掉 `makepkg.conf` 中与 Clang 相关的配置行。

Qt 软件包

Qt 软件包可能需要额外配置。Qt 有预定义的构建配置（称为 “mkspecs”），在 Linux 上默认使用 GCC。

部分情况下，mkspec 会根据 `CC`/`CXX` 变量自动设置为 `linux-clang`；但在其他情况下（如直接调用 `qmake` 的软件包）则不会，因此需手动显式设置：

```
/etc/makepkg.conf
export QMAKESPEC=linux-clang
```

注意： 部分软件包需要安装 llvm包 软件包，因为 `linux-clang` 配置会使用 `llvm-ar` 等工具。

Rust 软件包

当 Clang 被设为系统默认编译器时，需配置 Rust 使用 Clang 作为链接器 ——Rust 应用构建过程中，常会编译 C 代码作为依赖。

配置方法：在 `/etc/makepkg.conf.d/rust.conf` 中指定 Clang（可选指定 lld）。示例（使用 Clang 和 lld）：

```
/etc/makepkg.conf.d/rust.conf
RUSTFLAGS="-Cforce-frame-pointers=yes -Clinker=clang -Clink-arg=-fuse-ld=lld"
```

使用静态分析工具

要分析项目，只需在构建命令的前面放置 `scan-build`。 例如：

```
$ scan-build make
```

提示：如果您的项目已经被编译， `scan-build`将不会重建，也不会对其进行分析。如需强制重新编译和分析，请使用 `-B`参数：

```
$ scan-build make -B
```

也可针对特定文件进行分析：

```
$ scan-build gcc -c t1.c t2.c
```

参见

- Wikipedia: Clang
- scan-build：从命令行运行分析器

- 使用 Clang 编译 CUDA 代码

5.2 .SRCINFO

难度 入门 | **篇幅** 4.5 KB | **所属** 包开发

本节将讲到

- 生成
- 语法
- 要求

.SRCINFO 文件（旧称 .AURINFO）用一个简单直接的格式记录软件包的元数据，这样像 devtools包 的工具和 AUR 网站后台或 AUR 助手可以不用解析 PKGBUILD 就获得需要的信息。直接解析 shell 脚本的潜在问题可参考 FS#25210，FS#15043 和 FS#16394。

警告：对 PKGBUILD 元数据的修改例如 pkgver() 更新，不会自动同步到 .SRCINFO. 如果修改了 PKGBUILD 但是没更新 .SRCINFO 会导致 AUR 使用错误的数据。

生成

可以通过 makepkg 自动生成 .SRCINFO 文件：

```
$ makepkg --printsrcinfo > .SRCINFO
```

语法

.SRCINFO 按行解析，每一行由像 key = value 一样的键值对组成。

每一个键都是一个字段，键值与 PKGBUILD 变量对应，具体请参考 PKGBUILD(5) § OPTIONS AND DIRECTIVES。键和数值都不应该加引号。等号两边一定要有一个空格（即使值为空）而且值之间不能包含逗号。有些值在 PKGBUILD 中是数组，需要变为同样的值多次指定，例如：

```
PKGBUILD
arch=(i686 x86_64)
.SRCINFO
arch = i686
arch = x86_64
```

文件以 pkgbase 声明开头，此段落包含适用于整个软件包的数据。标准的 PKGBUILD 描述了一个单独的软件包，那么这就是唯一的主段落，后面跟着包含软件包的名称的 pkgname 声明，与前面的 pkgbase 对应：

```
pkgbase = foo
        pkgdesc = An example package.
        ...
        md5sums = SKIP

pkgname = foo
```

如果 PKGBUILD 包含多个软件包，那么每个段落都以 pkgname 开头，然后记录针对那个软件包的数据。

要求

下面字段只能在 .SRCINFO 文件的 pkgbase 段落出现一次：

- pkgver, pkgrel, epoch

下面字段可以在任何字段出现，每段落最多一次：

- pkgdesc, url, install, changelog

下面字段可以在一个段落里多次出现：

- arch
- groups
- license
- noextract
- options
- backup
- validpgpkeys（只能在 pkgbase 出现）

下列的字段可以重复出现，以多个体系结构为例：

```
source_x86_64 = https://foo.bar/file.tar.gz
source_i686 = https://foo.bar/file_i686_patch.tar.gz
```

- source
- depends, checkdepends, makedepends, optdepends
- provides, conflicts, replaces
- md5sums, sha1sums, sha224sums, sha256sums, sha384sums, sha512sums

其它字段名称、换行和以 # 开头的行会被忽略。

5.3 Debuginfod

难度 入门 | **篇幅** 4.5 KB | **所属** 包开发

本节将讲到

- 安装
- 使用
- 调试器支持
- 缓存
- 参见

翻译状态：

- 本文（或部分内容）译自 Debuginfod，最近一次同步于 2024-11-24，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Debuginfod 是一种通过 HTTP API 提供 debug 信息的服务。

安装

gdb包 会自动尝试从 `DEBUGINFOD_URLS` 环境变量中指定的服务器下载调试文件（debug files），该变量是一串以空格分隔的 URL。

libelf包 是 gdb包 的依赖项，附带了 `/etc/profile.d/debuginfod.sh` 和 `/etc/profile.d/debuginfod.csh` 脚本，这些脚本在登录时设置变量，因此无需安装额外的软件包。这些脚本解析 `/etc/debuginfod/` 中的 *.urls* 文件，并将 `DEBUGINFOD_URLS` 环境变量默认设置为 `https://debuginfod.archlinux.org`。

提示：还可以使用 `https://debuginfod.elfutils.org/` 作为联合服务器并查询所有可用的 *debuginfod* 服务器。

您可以选择性安装提供 debuginfod-find(1) 实用程序的 debuginfod包 软件包，此包是 delve包 中的 debuginfod 支持所必需的。

使用

提示：关于 *debuginfod* 配合 *gdb* 一起使用，参考 Debugging/Getting traces#Getting the trace。

手动下载

如果要手动检索 zstd包 中某些源文件的调试符号，可以使用 debuginfod-find包:

```
$ debuginfod-find debuginfo /usr/bin/zstd
/home/user/.cache/debuginfod_client/70e1b456c5813658df6436a3deb71812e75a0267/debuginfo
$ debuginfod-find source /usr/bin/zstd /usr/src/debug/zstd-1.5.2/programs/fileio.c
/home/user/.cache/debuginfod_client/70e1b456c5813658df6436a3deb71812e75a0267/source##usr##src##debug##zstd-1.5.2##programs##fileio.c
```

禁用

可以通过清除 `DEBUGINFOD_URLS` 来禁用 *debuginfod* 客户端服务：

```
$ unset DEBUGINFOD_URLS
```

如果想使用本地缓存，而不尝试连接任何服务器，可以将 `DEBUGINFOD_URLS` 设置为非空字符串，例如 `/dev/null`。

调试器支持

多种调试器支持使用 debuginfod 查找调试符号和源代码列表。

软件包	状况	注意
gdb包	支持	
delve包	支持	
KDE Crash Report	支持	
valgrind包	支持	

缓存

debuginfod 的缓存存储在 `$DEBUGINFOD_CACHE_PATH` （如果已设置）指定的位置。否则，如果设置了 `$XDG_CACHE_HOME`，它将使用 `$HOME/.cache/debuginfod_client/` 或 `$XDG_CACHE_HOME/debuginfod_client/`。缓存的大小可能增长得非常快，具体取决于您拥有的调试会话数量以及涉及的软件包。

有 3 个参数可以配置缓存行为，如 debuginfod(8) § CACHE 手册中的描述:

- `cache_clean_interval_s`: 每次自动清理之间的间隔（默认为 86400，即 1 天）
- `max_unused_age_s`: 未使用的数据保留多长时间（默认为 604800，即 1 周）
- `cache_miss_s`: 记住失败的查询多长时间（默认为 600，即 10 分钟）

每个参数都由缓存文件夹中同名文件中的数字定义。

如果你很少使用 *debuginfod*，可以手动删除缓存中的所有目录（保留参数文件）或完整的缓存目录。

参见

- 主页
- The elfutils debuginfod server, FOSDEM 2020

5.4 Ccache

难度 入门 | **篇幅** 5.2 KB | **所属** 包开发

本节将讲到

- 安装
- 配置
- Misc
- 注意事项

- 参阅

Ccache 是一个编译器的包装，它将已编译的二进制文件存储在指定位置，并将其提供给用户，以加快同一代码的重新编译。虽然第一次编译程序可能要时间长些，但随后的编译会快得多，因为不需要进行重复编译，只需要对之前存储的二进制文件进行查找。ccache 与 GCC 和 Clang 兼容。

安装

安装位于官方软件仓库的 ccache包 软件包。

配置

可以用配置文件修改默认行为，优先级由高到低为:

1. 环境变量
2. 单个 Cache 的配置文件 (`$HOME/.config/ccache/ccache.conf`)
3. 系统配置文件 (`/etc/ccache.conf`)

详见 ccache(1)。

为 makepkg 启用 ccache

要在 makepkg 启用 *ccache*，请编辑 `/etc/makepkg.conf`. 在 `BUILDENV` 中删除 ccache 前的感叹号：

```
BUILDENV=(fakeroot !distcc color ccache !xdelta)
```

在命令行中启用

如果要从中命令行编译而不是生成软件包，同样可以使用 *ccache* 提高速度。

为此，你可以在每个编译命令前加上 `ccache`。

```
$ ccache cc hello_world.c
```

或者修改你的 `$PATH`，在编译器的路径之前加上 *ccache* 的二进制文件目录。

```
export PATH="/usr/lib/ccache/bin/:$PATH"
```

你他可以把它设置到你的环境变量里，方便多次使用。

注意：如果用这个方法，将同时对 *makepkg* 启用 *ccache*。

启用 colorgcc 支持

由于 colorgcc 也是一个编译器的包装，所以需要确保包装的调用顺序是正确的。

```
export PATH="/usr/lib/colorgcc/bin/:$PATH"     # 按照正常安装 colorgcc 的情况保持不变（不要添加 ccache）
export CCACHE_PATH="/usr/bin"                  # 告诉 ccache 只使用这里的编译器
```

colorgcc 需要调用 *ccache* 而不是真正的编译器。编辑 `/etc/colorgcc/colorgccrc` 修改所有 `/usr/bin` 路径为 `/usr/lib/ccache/bin`：

```
/etc/colorgcc/colorgccrc
g++: /usr/lib/ccache/bin/g++
gcc: /usr/lib/ccache/bin/gcc
c++: /usr/lib/ccache/bin/g++
cc: /usr/lib/ccache/bin/gcc
g77:/usr/bin/g77
f77:/usr/bin/g77
gcj:/usr/bin/gcj
```

新版本的 *ccache* 在设置 `GCC_COLORS` 时，将始终为GCC启用颜色支持，同时 Clang 默认启动多色输出。如果输出对象不是 TTY，*ccache* 会让编译器生成颜色，将它们保存在缓存中，但从输出中剥离。在统一 \-fdiagnostics-color 方面仍然存在一些问题。

Misc

注意：此章节未进行翻译。

修改缓存目录

可以将缓存目录 `~/.cache/.ccache` 配置到其它地方，例如 SSD 或 ramdisk:

要在修改当前 shell 的缓存目录：

```
$ export CCACHE_DIR=/ramdisk/ccache
```

要修改默认缓存目录：

```
~/.config/.ccache/ccache.conf
cache_dir = /ramdisk/ccache
```

设置最大缓存大小

默认值是 5G，可以通过配置修改：

```
/home/<user>/.ccache/ccache.conf
max_size = 2.0G
```

通过环境变量禁用缓存

如果你想只在当前的shell中禁用 *ccache*:

```
 $ export CCACHE_DISABLE=1
```

CLI

此外可以使用 ‘’ccache‘’ 命令行工具。

显示统计数据:

```
$ ccache -s
```

清空缓存:

```
$ ccache -C
```

makechrootpkg

devtools包 中 makechrootpkg 也可以使用 ccache，要在清理 chroot 后保留缓存，可以使用 makechrootpkg 的 -d 选项将 cache 目录从普通系统绑定到 chroot:

```
$ mkdir /path/of/chroot/ccache
$ makechrootpkg -d /path/to/cache/:/ccache -r /path/of/chroot -- CCACHE_DIR=/ccache
```

这样 chroot 中就可以和正常系统中一样配置和使用 *ccache*.

注意事项

ccache 只有在编译*完全相同*的源代码时才有效。(或者说是预处理过的源码)

在 Gentoo Linux 社区，一个基于源代码的发行版，*ccache* 因其安慰剂效应、编译失败（由于不受欢迎的遗留对象）等而臭名昭著。Gentoo要求在报告编译失败之前关闭 *ccache*。参见 Gentoo:Handbook:Parts/Working/Features#Caching compilation objects 和 the blog post 题为 "Debunking ccache myths"，作者是 Diego Pettenò，一位前 Gentoo 开发者。

参阅

- ccache manual

5.5 OCaml 打包准则

难度 进阶 | **篇幅** 7.6 KB | **所属** 包开发

本节将讲到

- 命名
- 文件的位置
- OCaml bytecode and levels
- 使用 Dune 的 PKGBUILD 实例
- 使用 findlib 的 PKGBUILD 实例

为使用 OCaml 编写的软件创建 PKGBUILD。

命名

若是库则采用 `ocaml-*modulename*` 的格式，若是程序则采用程序名。不论如何，名字都应当是全小写的。

文件的位置

库

OCaml 库应当安装到 `/usr/lib/ocaml`。到 `/usr/lib/ocaml/site-lib` 已经弃用。

OCaml 库应当使用 ocaml-findlib包 来安装。`ocaml-findlib` 在包种包含了元信息来使库管理更加简单。这是一种惯例，现在很多 OCaml 都需要它。

`ocaml-findlib` 从应被包含在源码包中的 `META` 的文件中取出必要的数据。If this file is not included, one should either be obtained from the corresponding Debian, Ubuntu, or Fedora package, or created for the package by the maintainer. A request to include the file should also be made to the upstream developers of the package.

The `OCAMLFIND_DESTDIR` variable should be used when installing packages with `ocaml-findlib`. See the example PKGBUILD below for details.

OASIS

OCaml packages that install executables using OASIS ignore `DESTDIR`. This is a known limitation of OASIS (issue #493). One way to enable `DESTDIR`\-like functionality is to run the `configure` script with the `--destdir` argument, like so:

```
build() {
    cd "${srcname}-${pkgver}"
    ./configure --prefix /usr --destdir "$pkgdir"

    # build commands
}
```

OCaml bytecode and levels

OCaml can run code on multiple "levels", the top level interprets OCaml Code without compiling, the bytecode level creates machine independent bytecode and the native level creates machine code binaries (just like C/C++).

When building OCaml Packages you need to be aware if the build process is compiling native machine code, bytecode, or as in many cases both. This creates a number of situations which can cause problems with package options and the right dependencies.

If bytecode is produced at all then the PKGBUILD must contain the following to protect the bytecode:

```
options=('!strip')
```

If the package does not contain bytecode and only distributes a binary, then `ocaml` is not needed as a dependency, but it of course is required as a makedepends since the `ocaml` package provides the OCaml compiler. If the package contains both native code and bytecode then `ocaml` should be a dependency and a makedepends.

OCaml code is rarely (if ever) distributed as bytecode only and will almost always include native code: the only case where using *any* as the *arch* is advisable is when only un-compiled source code is distributed, usually with a library, though many libraries still distribute native code.

The moral of the story here is to be aware of what it is you are distributing, chances are your package contains both native machine code and bytecode.

使用 Dune 的 PKGBUILD 实例

Dune 是一种新的构建系统，现在已经越来越多的被用于 OCaml 项目中。

One thing to be aware is that a single project can build several "packages" in the OPAM/findlib sense, each with its own directory in `/usr/lib/ocaml/`. See ocaml-cairo包 for an example. For release builds, all "packages" have to be explicitly listed.

```
# Contributor: Your Name <youremail@domain.com>

pkgname=ocaml-<package name>
pkgver=4.2
pkgrel=1
license=()
arch=('x86_64')
pkgdesc="An OCaml Package"
url=""
depends=('ocaml')
makedepends=('dune')
source=()
options=('!strip')
sha256sums=()

build() {
  cd "${pkgname}-${pkgver}"
  # The "-p" flag is necessary for release builds, see the Dune manpage. Dune will
complain if you forget some packages.
  dune build -p package1,package1-extension,package2
}

package() {
   cd "${pkgname}-${pkgver}"

   DESTDIR="${pkgdir}" dune install --prefix "/usr" --libdir "/usr/lib/ocaml" --docdir
"/usr/share/doc"
```

使用 findlib 的 PKGBUILD 实例

```
# Contributor: Your Name <youremail@domain.com>

pkgname=ocaml-<package name>
pkgver=4.2
pkgrel=1
license=()
arch=('x86_64')
pkgdesc="An OCaml Package"
url=""
depends=('ocaml')
makedepends=('ocaml-findlib')
source=()
options=('!strip')
md5sums=()

OCAMLFIND_DESTDIR="${pkgdir}$(ocamlfind printconf destdir)"

build() {
  cd "${pkgname}-${pkgver}"
  mkdir -p "$OCAMLFIND_DESTDIR"
  ./configure --prefix=/usr
  make
}
```

```
package() {
  cd "${pkgname}-${pkgver}"
  env DESTDIR="${pkgdir}" \
    OCAMLFIND_DESTDIR="$OCAMLFIND_DESTDIR" \
    make install
}
```

需要留意的是许多 OCaml 包都需要给 `make` 与 `make install` 一些额外信息。同时，请去除 `!strip` 选项，并在包不编译为字节码时指定架构。

5.6 Eclipse 插件打包准则

难度 进阶 | **篇幅** 7.8 KB | **所属** 包开发

本节将讲到

- Eclipse 插件结构以及安装
- 打包

有许多安装有效 Eclipse 插件的方法。尤其是在 Eclipse 3.4 里引入 *dropins* 文件夹之后。但其中一些比较凌乱，而且具有标准化和一致的封装方法对于一个干净的系统结构是非常重要的。然而，如果没有一个对 Eclipse 插件如何工作一清二楚的打包者的话，要实现这一切并不简单。本文试图定义一个标准且精简的 Eclipse 插件 PKGBUILD 的结构，如此来在不用麻烦打包者全部重新打包的情况下保持一个干净的文件系统结构。

Eclipse 插件结构以及安装

一般的 Eclipse 插件包含两个目录，`features` 和 `plugins`, 自从 Eclipse 3.3 起它们只能放在 `/usr/share/eclipse/`. 这两个文件夹的内容可以与其他插件的混合，如此会导致混乱并且结构难以管理，也很难来一目了然地区分哪个软件包包含哪些文件。

这种安装方法依然被 Eclipse 3.4 支持，然而我们更推荐使用 `/usr/share/eclipse/dropins/` 目录。该文件夹内可包含不限数目的子目录，每个子目录包含自己的 `features` 和 `plugins` 子目录。这也能保持一个干净而精简的系统结构，并应当成为打包的标准。

打包

PKGBUILD 样例

这里是一个样例，稍后我们将介绍如何自定义修改。

```
PKGBUILD-eclipse.proto
pkgname=eclipse-mylyn
pkgver=3.0.3
pkgrel=1
pkgdesc="A task-focused interface for Eclipse"
```

```
arch=('i686' 'x86_64')
url="http://www.eclipse.org/mylyn/"
license=('EPL')
depends=('eclipse')
optdepends=('bugzilla: ticketing support')
source=(http://download.eclipse.org/tools/mylyn/update/mylyn-${pkgver}-e3.4.zip)
md5sums=('e98cd7ab3c5d5aeb7c32845844f85c22')

prepare() {
  # remove features and plug-ins containing sources
  rm features/*.source_*
  rm plugins/*.source_*
  # remove gz files
  rm plugins/*.pack.gz
}

package() {
  _dest=${pkgdir}/usr/share/eclipse/dropins/${pkgname/eclipse-}/eclipse

  # Features
  find features -type f | while read _feature ; do
    if [[ ${_feature} =~ (.*\.jar$) ]] ; then
      install -dm755 ${_dest}/${_feature%*.jar}
      cd ${_dest}/${_feature/.jar}
      # extract features (otherwise they are not visible in about dialog)
      jar xf ${srcdir}/${_feature} || return 1
    else
      install -Dm644 ${_feature} ${_dest}/${_feature}
    fi
  done

  # Plugins
  find plugins -type f | while read _plugin ; do
    install -Dm644 "${_plugin}" "${_dest}/${_plugin}"
  done
}
```

如何自定义

需要修改的，最主要的变量就是 `pkgname`. 如果你要打包一个一般的插件，那么你只需要做这些: 大部分插件以 zip 文件发布，其中只含有 `features` 和 `plugins` 子目录。所以，如果你要打包 `foo` 插件并且源文件只包含 `features` 和 `plugins`, 只需把 `pkgname` 改成 `eclipse-foo`, 这样就完成了。

请仔细阅读，获取 PKGBUILD 的内部结构，这能帮助你理解在其他情况下如何设置编译。

深入分析 PKGBUILD

包的命名

包应该命名为 `eclipse-*pluginname*`, 如此能被识别为 Eclipse 相关软件包并且能轻松使用 shell 替代来提取出如 `${pkgname/eclipse-}` 的插件名称，而不用使用不需要的 `${_realname}` 变量。插件名对于清理安装时临时文件并避免冲突是必需的。

文件

解压

一些插件需要的功能要从 jar 文件中提取。已包含于 JRE 的 `jar` 工具,就是用来干这个的。然而，`jar` 并不能解压到当前目录以外的地方: 这意味着，创建每个目录之后，在解压之前需要 `cd` 进入该目录。`${_dest}` 变量被用在这样的背景下，以提高文本可读性和PKGBUILD整洁性。

地址

如同我们所说，源文档提供两个目录 `features` 和 `plugins`, 每个打包为 jar 文件。更好的嵌入式结构应该如下:

```
/usr/share/eclipse/dropins/pluginname/eclipse/features/feature1/...
/usr/share/eclipse/dropins/pluginname/eclipse/features/feature2/...
/usr/share/eclipse/dropins/pluginname/eclipse/plugins/plugin1.jar
/usr/share/eclipse/dropins/pluginname/eclipse/plugins/plugin2.jar
```

这种结构支持把不同插件需要的不同版本的库混起来，同时很清楚它是属于那个包的。这也能避免当不同插件提供了相同库时的冲突。唯一的选择是把每个包和它的库一起它所需的一切乱七八糟的东西分离，并且你也别想这样就能起效，因为有些包需要旧版的库才能工作。 必须要从 jar 里解压出来因为 Eclipse 不会自动检测它们，不然的话整个安装的插件都不会工作。这归因于 Eclipse 认为更新站点和本地安装是不同的 (别问为什么，它就这样).

build() 函数

首先要注意的就是 `cd ${srcdir}` 命令。通常源压缩文档直接在 `${srcdir}` 下解压出 `features` 和 `plugins` 文件夹，但并不是所有的都这样。总之，对大多数*(事实上)*非标准的插件这是需要改变的唯一路线。

某些特性也随源码发布。对一个普通的释出版来说源码并不需要，可以删除。更多的特性包含 `*.pack.gz` 文件，与 jar 文档相比内容相同。所以这些文件也能删除。

接下来是 `features` 部分。它为每个 jar 文件创建必要的目录并解压到对应的目录去。同样，`plugins` 部分把 jar 安装到文件夹里去。一个 while 循环用于防止生成名字奇怪的文件。

5.7 调试/获取跟踪信息

难度 进阶 | **篇幅** 15.6 KB | **所属** 包开发

本节将讲到

- 获取追踪文件
- Manually getting debug info
- 参见

通常，为了让人们方便调试，会让可执行文件尽可能小。在没有调试信息的情况下获取跟踪信息，会大大降低其实用性。例如，在没有调试信息的情况下，gdb包 会话的跟踪信息可能如下所示：

```
[...]
Backtrace was generated from '/usr/bin/epiphany'

(no debugging symbols found)
Using host libthread_db library "/lib/libthread_db.so.1".
(no debugging symbols found)
[Thread debugging using libthread_db enabled]
[New Thread -1241265952 (LWP 12630)]
(no debugging symbols found)
0xb7f25410 in __kernel_vsyscall ()
#0  0xb7f25410 in __kernel_vsyscall ()
#1  0xb741b45b in ?? () from /lib/libpthread.so.0
[...]
```

`??` 会显示调试信息缺失的位置，以及调用该函数的库或可执行文件的名称。同样，当出现 `(no debugging symbols found)` 时，应查找所述文件名。

要获得对程序开发人员有用的正确跟踪，请遵循以下章节。大多数 Arch 官方软件包都有单独的调试文件，可以通过 Debuginfod 下载（参见 #获取跟踪）。 如果没有在可执行文件中添加enhanced debugging information，则必须启用调试符号重建软件包。

使用完整的堆栈跟踪来告知开发人员您之前发现的错误。

获取追踪文件

实际的回溯（或堆栈跟踪）可以通过 gdb包 （GNU 调试器）获取。根据是否应启动程序的新实例、是否应附加到现有进程、或是否应检查之前的崩溃，可以以多种方式使用调试器。

启动程序的新实例

Start *gdb* with an executable program that can be found in `$PATH` (or a full path):

```
$ gdb application
```

gdb automatically tries to download debug information and symbols for packages in the official repositories. When *gdb* asks whether Debuginfod should be enabled in the debugging session, answer `y`:

```
This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.archlinux.org>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for /usr/bin/application
Reading symbols
from /home/user/.cache/debuginfod_client/fbaee841e2ed2c11ecbbda26f39eeec1da23d6c3/debug
info...
```

Then, within *gdb*, type `run` followed by any arguments you wish the program to start with:

```
(gdb) run arguments
```

提示：Alternatively, you can pass arguments when starting *gdb* using `gdb --args *application arguments...*` and then use only `run` without arguments within *gdb*. For example, to debug an application written in Python, run `gdb --args /usr/bin/python */path/to/application*`.

Now do whatever is necessary to evoke the bug. *gdb* will automatically halt the application when it crashes and prompt for commands. In case of freezes or similar issues, press `Ctrl+c` and you will be returned to the command prompt, too.

Then enable logging to a file:

```
(gdb) set logging enabled on
```

提示：The default file name is `gdb.txt`. An alternate file name can be specified with `set logging file *trace.log*` within *gdb*.

And finally have the backtrace written to the specified file in the current working directory:

```
(gdb) thread apply all backtrace full
```

Attaching to an existing process

If the program you want to debug is already running, you need to first find its process ID. Tools such as pidof(1) or pgrep(1) are available. For example:

```
$ pidof python3
907171 491909
```

When the output does not give a unique ID, you can try more filtering or look at the output of `ps aux` or `pstree --show-pids`.

Attaching as regular user does not work by default due to restricted ptrace scope. The restriction can be lowered temporarily with `echo 0 > /proc/sys/kernel/yama/ptrace_scope` or you can run *gdb* as a privileged user, e.g. using sudo.

Start *gdb* attaching it to the process:

```
$ gdb --pid=PID
```

gdb will ask if Debuginfod should be enabled in this debugging session, to which you should answer `y`.

Note that attaching to a process has stopped it and it needs to be explicitly continued. This replaces the `run` command from the workflow in the #Starting a new instance of a program section:

```
(gdb) continue
```

Now do whatever is necessary to evoke the bug in the attached process. Then proceed with enabling logging and obtaining the trace same as in #Starting a new instance of a program.

Examining a previous crash

To debug an application that has already crashed, you will want to invoke *gdb* on its *core dump*. See Core dump#Analyzing a core dump for details.

If debugging information for the crashed program is not available and a proper backtrace was not obtained, you may need to rebuild the package and reproduce the crash again.

Manually getting debug info

The first thing to do is to obtain the names of the packages which require rebuilding or the install of a debug package.

```
[...]
Backtrace was generated from '/usr/bin/epiphany'

(no debugging symbols found)
Using host libthread_db library "/lib/libthread_db.so.1".
(no debugging symbols found)
[...]
```

For example for the above extract from a trace, the package name for the associated package can be obtained with pacman:

```
$ pacman -Qo /lib/libthread_db.so.1
/lib/libthread_db.so.1 is owned by glibc 2.5-8
```

The package is called glibc包 in version 2.5-8. Repeat this step for every package that needs debugging information.

Installing debug packages

注意：Debug packages are not archived by Arch Linux on Arch Linux Archive.

A few mirrors currently distribute debug packages in accessible repositories. These are sponsored mirrors controlled by Arch Linux and are given access to the debug repositories.

- https://geo.mirror.pkgbuild.com (GeoDNS mirror)

To install a package you can install it directly from the repository. For example:

```
# pacman -U https://geo.mirror.pkgbuild.com/core-debug/os/x86_64/zstd-debug-1.5.2-2-x86_64.pkg.tar.zst
```

警告：Debug packages from one mirror are not compatible with regular packages from another mirror, if both mirrors are not in sync, and thus have mismatching builds. In this case, avoid mixing packages from different mirrors (this would result in a partial upgrade), but point all repositories to the `debug` mirror.

本文或本章节可能需要合并到Official repositories。

附注： Official repositories have a dedicated page（在 Talk:调试/获取跟踪信息#mirrors distributing debug packages. 中讨论）

Another option is to add the repositories to your pacman configuration.

```
/etc/pacman.conf
# Testing Repositories

[core-testing-debug]
Include = /etc/pacman.d/mirrorlist

[extra-testing-debug]
```

```
Include = /etc/pacman.d/mirrorlist

[multilib-testing-debug]
Include = /etc/pacman.d/mirrorlist

# Stable repositories

[core-debug]
Include = /etc/pacman.d/mirrorlist

[extra-debug]
Include = /etc/pacman.d/mirrorlist

[multilib-debug]
Include = /etc/pacman.d/mirrorlist
```

Place a mirror with debug packages as the first one in the mirrorlist file:

```
/etc/pacman.d/mirrorlist
Server = https://geo.mirror.pkgbuild.com/$repo/os/$arch
...
```

Rebuilding packages

If debug information is not exposed through *debuginfod* (for example, when the package originates from the AUR), then it can be rebuilt from source. See ABS for packages in the official repositories, or AUR#Acquire build files for packages in the AUR.

To set the required #Compilation options, you can modify the makepkg configuration if you will only use *makepkg* for debug purposes. In other cases, you should modify package's `PKGBUILD` file only for each package you would like to rebuild.

Compilation options

As of pacman 4.1, makepkg.conf(5) has debug compilation flags in `DEBUG_CFLAGS` and `DEBUG_CXXFLAGS`. To use them, enable the `debug` *makepkg* option, and disable `strip`.

These settings will force compilation with debug symbols and will disable their stripping from executables.

```
/etc/makepkg.conf
OPTIONS+=(debug !strip)
```

To apply this setting to a single package, modify the `PKGBUILD`:

```
PKGBUILD
options=(debug !strip)
```

Alternatively you can put the debug information in a separate package by enabling both `debug` and `strip`, debug symbols will then be stripped from the main package and placed, together with source files to aid in stepping through the debugger, in a separate `*pkgbase*-debug` package. This is advantageous if the package contains very large binaries (e.g. over a GB with debug symbols included) as it might cause freezing and other strange, unwanted behavior occurring.

注意：It is insufficient to simply install the newly compiled debug package, because the debugger will check that the file containing the debug symbols is from the same build as the associated library and executable. You must install both of the recompiled packages. In Arch, the debug symbol files are

installed under `/usr/lib/debug/`, and source files are installed under `/usr/src/debug`. See the GDB documentation for more information about debug packages.

glibc

Certain packages such as *glibc* are stripped regardless. Check the `PKGBUILD` for sections such as:

```
strip $STRIP_BINARIES usr/bin/{gencat,getconf,getent,iconv,iconvconfig} \
                      usr/bin/{ldconfig,locale,localedef,makedb} \
                      usr/bin/{pcprofiledump,pldd,rpcgen,sln,sprof} \
                      usr/lib/getconf/*

strip $STRIP_STATIC usr/lib/*.a

strip $STRIP_SHARED usr/lib/{libanl,libBrokenLocale,libcidn,libcrypt}-*.so \
                    usr/lib/libnss_{compat,db,dns,files,hesiod,nis,nisplus}-*.so \
                    usr/lib/{libdl,libm,libnsl,libresolv,librt,libutil}-*.so \
                    usr/lib/{libmemusage,libpcprofile,libSegFault}.so \
                    usr/lib/{audit,gconv}/*.so
```

And remove them where appropriate.

Clang

Packages using Clang as the compiler will not build with the `debug` option due to the debug flag `-fvar-tracking-assignments'` not being handled (e.g. the previous js78 PKGBUILD).

Add the following at the top of the `build()` function to only remove the flag for the affected package:

```
build() {
  CFLAGS=${CFLAGS/-fvar-tracking-assignments}
  CXXFLAGS=${CXXFLAGS/-fvar-tracking-assignments}
[...]
```

LTO

Using Link-time optimization (LTO) will, both during compiling and in a debugger, use more memory. Depending on the application, especially if it is a large one like Firefox or Qt, it might exceed the available memory. Build the application without LTO if this happens.

All packages in the official repositories are generally built with LTO.

Building and installing the package

Build the package from source using `makepkg` while in the `PKGBUILD`'s directory. This could take some time:

```
$ makepkg
```

Then install the built package:

```
# pacman -U glibc-2.26-1-x86_64.pkg.tar.gz
```

参见

- Debian:HowToGetABacktrace
- Gentoo:Project:Quality Assurance/Backtraces
- Fedora:StackTraces

- GNOME - Getting Stack Traces
- gdb mini intro

5.8 PKGBUILD

难度 高级 | **篇幅** 30.3 KB | **所属** 包开发

本节将讲到

- 软件包名称
- 版本
- 一般变量
- 依赖关系
- 包关系

本页面讨论 PKGBUILD 中可由维护者定义的变量。若要获取关于 PKGBUILD 中的函数和创建软件包的基本信息，请参考创建软件包和 PKGBUILD(5)。

PKGBUILD 是一个 Bash 脚本，包含在构建 Arch Linux 软件包时需要的信息。

Arch Linux 使用 makepkg 构建软件包。当 *makepkg* 运行时，它会在当前目录寻找 PKGBUILD 文件，并依照其中的指令编译或获取所需的文件，并生成 *pkgname*.pkg.tar.zst 软件包。生成的包内有二进制文件和安装指令，可以使用 pacman 进行安装。

必须定义的变量有 pkgname，pkgver，pkgrel 和 arch。在构建软件包时不强制要求定义 license，但若要分享 PKGBUILD 文件给其他人，推荐加上该变量，否则 *makepkg* 会提示相关报错。

一般来说，建议但不强制要求按照下面的顺序在 PKGBUILD 文件中定义这些变量。

提示：

- 使用 namcap 来检查 PKGBUILD 文件中的常见打包问题。
- 可以使用 shellcheck(1) 来检查 PKGBUILD 中的常见语法错误，并排除 SC2034 和 SC2154：

```
: shellcheck --shell=bash --exclude=SC2034,SC2154 PKGBUILD
```

- termux-language-serverAUR 为 PKGBUILD 和 makepkg.conf 等提供了一个语言服务器。

软件包名称

pkgbase

在构建常规的软件包时，不应该在 PKGBUILD 文件中显式定义该变量，这个值默认会与 #pkgname 的值相同。

构建拆分包时，这个变量可用于在 *makepkg* 的输出和纯源代码包中指定软件包组。此变量不允许以下划线开头。若该变量没有明确定义，则会默认对应到 `pkgname` 数组的第一个元素。

拆分软件包中的所有选项和指令都默认使用 `PKGBUILD` 中全局定义的值。以下选项可以在拆分包的打包函数中覆写：#pkgdesc、#arch、#url、#license、#groups、#depends、#optdepends、#provides、#conflicts、#replaces、#backup、#options、#install 和 #changelog。

pkgname

对于常规的软件包，这个变量设定软件包的名称，例如 `pkgname='foo'`。对于拆分包则是一个名称的序列，例如 `pkgname=('foo' 'bar')`。名称只能由由小写字母、数字和 `@` `.` `_` `+` `-` (at 符号、英文句点、下划线、加号、连字符)构成，且不能以连字符或英文句点开头。为了保证一致性，`pkgname` 应该与软件的源代码文件相匹配。比如：源文件包名为 `foobar-2.5.tar.gz`，那么应该使用 `pkgname=foobar`。

版本

pkgver

软件包的版本号，应该与软件上游发布的版本号一致。变量的值可以由字母、数字和英文句点 `.`，下划线 `_` 组成，但不能包含连字符（`-`）。如果上游版本号中使用了连字符，则应该用下划线 `_` 来替代。在之后的 `PKGBUILD` 指令中`pkgver` 中的下划线可以用下面这个方法替换回连字符：`source=("${pkgname}-${pkgver//_/-}.tar.gz")`。

注意：如果上游使用时间戳格式的版本号，例如 `30102014`，请修改为年份在前的格式 `20141030`（ISO 8601 格式）。否则新版本可能无法被系统识别出来是新的。

提示：

- 不常用变量的顺序可以通过 pacman 软件包提供的 vercmp(8) 进行测试。
- 在 `PKGBUILD` 中定义 `pkgver()` 函数，makepkg 就可以自动更新此变量。详情参阅 VCS 软件打包准则#pkgver() 函数_函数)。

pkgrel

软件的发布号。这通常是一个正整数，用来区分同一版本软件的多次构建。当软件包的补丁和附加功能被添加进入 `PKGBUILD`，从而导致生成的软件包发生变化时，`pkgrel` 应该增加 1。而当这个软件包发布一个新版本时，发布号重置为 1。在个别情况下，也会有其他的发布号形式。比如*主版本号.次要版本号*。

epoch

警告：除了特别、绝对、显式要求需要这样做，否则不允许使用 `epoch` 变量。

用于强制升级软件包。在判定逻辑中，不论版本号如何，只要 `epoch` 值较大，就会被视为更新的软件包。这个值应为非负整数，且默认值为 0。通常当一个软件的版本编号方式改变（或者使用某些字母-数字混编的版本符号），导致正常的版本比较逻辑无法进行时，会使用这个变量来控制升级。比如：

```
pkgver=5.13
pkgrel=2
epoch=1
1:5.13-2
```

更多关于版本比较的信息，参见 pacman(8)。

一般变量

pkgdesc

软件包描述。建议最多 80 个字符，并不要自引软件包的名字，除非软件包名和程序名不相同。比如：`pkgdesc='Nedit is a text editor for X11'` 应该被写成 `pkgdesc='Text editor for X11'`。

合理地在描述中使用关键字，这样可以使软件包在相关的搜索中出现的可能性增加。

arch

一个描述能够生成并运行该软件包的架构的数组。Arch 官方仅支持 `x86_64`, 但是其它项目提供了其它架构支持。比如说，Arch Linux 32 提供了 `i686` 和 `pentium4` 支持；Arch Linux ARM 项目提供 `armv7h` (带有硬件浮点运算模块的 armv7）和 `aarch64` (64 位 armv8) 的支持。

这个数组有两种形式：

- `arch=('any')` 表明软件包可以在任意架构上生成，并且一旦编译完成，编译时的架构就不再有影响了（例如 shell 脚本、字体、主题、各种扩展、Java 程序等）。
- `arch=(...)` 包含一个或更多架构（`any`除外），表明这个软件包可以在上述的任意架构上生成，但生成产物只能在编译它的架构上运行。对于这些软件包，您必须在 `PKGBUILD` 文件中指定所有官方支持的架构。对于官方软件源和 AUR 软件包，这指的是 `arch=('x86_64')`。不过，AUR 软件包可以选择添加一些已知的兼容架构的支持。

在生成过程中可以通过 `$CARCH` 变量来获知目标架构。

url

待打包软件官方站点的网址。

license

**本文或本章节可能需要合并到Arch 打包准则#授权协议。**

附注： PKGBUILD 格式并没有强制执行某种打包标准。（在 Talk:PKGBUILD 中讨论）

软件分发所使用的许可证。Arch Linux 使用 SPDX 许可证标识符，所有许可证都需在 `/usr/share/licenses/` 下有对应项。

licenses包 包含了常见许可证（例如 `GPL-3.0-or-later`）的对应文件，默认已作为 base包 元软件包的依赖之一安装在 `/usr/share/licenses/spdx/` 目录下，使用时只需引用 SPDX 标识符清单中的标识符。

类似 BSD 或 MIT 的许可证族严格来说不是特定的单个许可证，每个实例都需要独立的许可证文件。对于这些许可证，需要在 `license` 中使用通用 SPDX 标识符（例如 `BSD-3-Clause` 或 `MIT`），并像自定义许可证一样提供对应的许可证文件。

对于自定义许可证，如果其不属于常见许可证族，就需要使用 `LicenseRef-*license-name*` 或 `custom:*license-name*` 作为标识符。对应的许可证文件必须放置在 `/usr/share/licenses/*pkgname*` 目录下。可以在 `package()` 中使用如下代码安装许可证文件：

```
install -Dm644 LICENSE "${pkgdir}/usr/share/licenses/${pkgname}/LICENSE"
```

注意：`pkgdir` 变量由 *makepkg* 定义，具体信息请参考 PKGBUILD(5) § PACKAGING FUNCTIONS。

如果要结合多个许可证或添加例外，需要遵循 SPDX 语法。例如，如果软件包发布于 GNU/GPL 2.0 *或* GNU/LGPL 2.1，需要使用 `'GPL-2.0-or-later OR LGPL-2.1-or-later'`；如果软件包发布于 Apache 2.0 并带有 LLVM 例外，需要使用 `'Apache-2.0 WITH LLVM-exception'`；如果软件包部分发布于 BSD 3 clause，其它部分发布于 GNU/LGPL 2，还有一部分发布于 GNU/GPL 2.0，就需要使用 `'BSD-3-Clause AND LGPL-2.1-or-later AND GPL-2.0-or-later'`。注意，这需要是单个字符串，因此需要将整个表达式用引号括起来。截至 2023 年 11 月为止，SPDX 例外列表还非常有限，因此你通常还是需要遵循添加自定义许可证的方式。

如果在 SPDX 标识符方面遇到问题，可以在过渡期间使用旧标识符（`/usr/share/licenses/common` 中的目录名称）。

另请参考 Nonfree applications package guidelines。

有关自由和开放源码软件许可证的更多信息和观点，请参阅以下网页：

- Wikipedia:Free software licence
- zhwp:自由及开放源代码软件许可协议比较
- 开源和自由软件项目法律问题入门指南
- GNU 项目 - 各类许可证及其评论
- Debian - 许可证信息
- 开放源代码促进会 - 许可证清单

groups

软件包所在的软件包组。例如：当你安装 plasma包组，它会安装组里的所有包。

依赖关系

注意：架构相关的额外依赖可以通过在名称后添加下划线和架构的方式指定，例如 `optdepends_x86_64=()`。

depends

为了生成和运行软件，而必须安装的包列表。如果是只在运行时才依赖的包，请在 `package()` 函数中定义。

可以使用比较运算符来描述版本限制。例如 `depends=('foobar>=1.8.0')`。如有多重限制，你可以添加多条，例如 `depends=('foobar>=1.8.0' 'foobar<2.0.0')`。

**本文或本章节可能需要合并到Arch 打包准则。**

附注： PKGBUILD 格式并没有强制执行某种打包标准。（在 Talk:PKGBUILD 中讨论）

`depends` 应该列出所有的直接依赖，即使已由某个依赖项间接引入时也应该如此。如果不这样做，那可能会发生以下情况：如果一个软件包 *foo* 依赖 *bar* 和 *baz* 两个软件包，而 *bar* 软件包也依赖于 *baz* 软件包，当 **bar** 软件包不再依赖于 **baz** 就极有可能会导致意外行为——pacman 不会在安装 **foo** 软件包时安装 **baz** 软件包，也会在清除孤儿软件包时把 **baz** 清理掉。由于 **baz** 软件包被依赖却没有被安装，**foo** 可能崩溃或者发生运行错误。

但在一些情况下，上面所述的某些依赖是没有必要或者不应该被列出的。比如说 glibc 包，每个操作系统都需要 C 运行库，因此它是不能够被卸载的。又比如，当软件包已经依赖于一个以 *python-* 开头的模块，就不需要再单独依赖 python包——因为 *python-* 开头的模块必定依赖于 python包 软件包，而且不允许从依赖列表中删除。

通常的依赖应该包括生成所有可选功能所需的依赖。否则，对于任何依赖于额外软件包的可选功能，应显式通过配置选项将其禁用。如果不这样做，会给软件包添加了所谓“自动魔法依赖”：一些生成时可选功能因为生成软件包的机器上安装的一些传递依赖或者不相关的软件被意外启用了，但是没有表现在包的依赖中。

如果依赖的名称写成了库的名称（例如 `depends=('libfoobar.so')`），*makepkg* 会在编译完成的包中尝试寻找依赖这个库的二进制文件，并添加二进制文件所需的 soname 版本号到依赖列表中。可以自己加上版本号来停用自动检测，比如说 `depends=('libfoobar.so=2')`。

makedepends

仅在软件生成时需要的软件包列表。可以像 `depends` 序列一样指定依赖的版本限制。`depends` 序列里面的软件包默认也是生成时需要的，此处不应该重复。

注意：

- 在使用 *makepkg* 构建软件包时，默认 base-devel包 已安装。该包的依赖项**不应该**出现在 `makedepends` 列表中。
- 如果使用 VCS 源，记得要把对应的 VCS 工具（git包，subversion包，cvs包 等等）加进去。

提示：用以下命令查看某个依赖项是否已经被 base-devel包 直接依赖：`pactree -rsud1 *package* | grep base-devel`（需要安装 pacman-contrib包）。

checkdepends

运行软件的测试组件时需要，而运行时不需要的包列表。该列表中的包遵循 `depends` 相同的格式。这些依赖只在 check()) 函数存在，且被 *makepkg* 执行时会被处理。

注意：在使用 *makepkg* 构建软件包时，默认 base-devel包 已安装。该包的依赖项**不应该**出现在 `checkdepends` 列表中。

optdepends

可选软件包序列。这些可选软件包不影响软件主要功能，但能提供额外特性。这通常意味着除非安装了对应的可选软件包，该软件包所提供的个别可执行文件可能无法正常使用。如果软件有一些替代依赖，您可以将其在此处全部列出，而不是 `depends` 列表中列出。

应该简要说明每个包所能提供的额外功能，例如：

```
optdepends=('cups: printing support'
            'sane: scanners support'
            'libgphoto2: digital cameras support'
            'alsa-lib: sound support'
            'giflib: GIF images support'
            'libjpeg: JPEG images support'
            'libpng: PNG images support')
```

包关系

注意：架构相关的附加列表可以通过在变量名称后添加下划线和架构名字的方式指定，例如 `conflicts_x86_64=()`。

provides

该列表说明的是本软件包能够提供列表中的包（或者像 *cron*、*sh* 这样的虚包和所有外部共享库）所提供的功能。只要 `provides` 列表中的包都没有声明 `conflicts`，提供相同功能的软件包就可以同时安装。

注意：

- 该变量中的软件包应当加上版本号（`pkgver`，可能的话还有`pkgrel`），特别是当依赖该软件包的其他软件对版本号特别敏感的时候。就是说，如果一个修改过的 *qt* 包其版本号为 3.3.8，命名为 *qt-foobar*，那么 `provides` 应该写成 `provides=('qt=3.3.8')`。如果忽略了版本号，会导致所有依赖于 *qt* 的某个特定版本的包编译失败。
- 不要把 `pkgname` 加入 `provides` 序列。这个操作会自动进行。

conflicts

这个列表描述的是与当前软件包冲突的包，或者与该包同时存在会产生问题的包。安装此软件时，这个列表中的所有软件包和提供这个功能的软件包都会被删除。可以像 `depends` 那样指定冲突包的版本号。

需要注意的是，冲突检查的对象是 `pkgname` 以及 `provides` 中的名字。因此，如果你的包 `provides *foo*` 功能，在 `conflicts` 中指定 `*foo*` 就会导致你的包和所有其他在 `provides` 中包含了 `*foo*` 的包冲突（也就是说，你不需要一个一个地指定所有冲突的包的名字）。例子是：

- netbeans包 提供 `netbeans`（因为 `pkgname` 就是如此）。
- netbeans-cppAUR 提供 `netbeans` 功能，并和 `netbeans` 冲突。
- netbeans-phpAUR 提供 `netbeans` 功能，而且和 `netbeans` 冲突，但是不需要再说明它与 netbeans-cppAUR 冲突，因为提供相同功能的包隐含了冲突关系。

当不同的包通过在 `provides` 列表中声明的方式提供相同的功能，那么是否特意在 `conflicts` 列表中添加可替换的包会产生不同的结果。特意说明的话，两个包就会被认为是可相互替代的；如果 `conflicts` 列表不存在，那么两个包会被认为是“可能可以同时存在”。在决定是否需要声明 `conflicts` 列表的时候，请忽略 `provides` 列表的内容，独立决策。

replaces

会因安装当前包而取代的过时的包的列表。比如：wireshark-qt包 中的 `replaces=('wireshark')`。在**同步**软件数据库后，*pacman* 会立刻用软件库中的另一个包替换掉 `replaces` 中已安装的包。如果你只是提供已存在包的一个替代品，或者上传到 AUR, 请不要使用 `replace`，而是使用 `conflicts` 和 `provides` 两个变量——它们仅在安装冲突软件包时被检查。

其它

backup

当包被升级或卸载时，应当保留的文件（的路径）序列。这些文件一般是用户会更改的文件，如主要放置在 `/etc` 中的配置文件。如果在安装软件包后这些文件没有被修改过，那么它们会随升级和移除软件包时一同被替换或删除。

列表中的文件应该使用**相对**路径，即不是以斜杠（/）开头的路径（如 `etc/pacman.conf` 而不是 `/etc/pacman.conf`）。`backup` 数组不支持空目录和类似"*"的通配符。

在升级时，新版本会被命名为 `*file*.pacnew` 以避免覆盖原来的被用户修改过的文件。当卸载包时，用户修改过的文件会以 `*file*.pacsave` 为名而保留下来——除非用 `pacman -Rn` 命令卸载。

参见 pacnew 和 pacsave 文件获取更多信息。

options

参见 PKGBUILD(5) § OPTIONS AND DIRECTIVES 以获取所有可用选项。

install

需要包含在包中的 *.install* 脚本的名称。

pacman 可以在安装、卸载或升级一个软件包时存储及执行一些特定的脚本。脚本包含了下面几个函数，并且在特定时刻执行它们：

- `pre_install` - 提取包中文件前运行的脚本。会传递一个参数：新版本号。
- `post_install` - 提取包中文件后运行的脚本。安装软件包后输出的提示信息需放置于该函数内。会传递一个参数：新版本号。
- `pre_upgrade` - 提取包中文件前运行的脚本。两个参数会按以下顺序传递：新版本号，旧版本号。
- `post_upgrade` - 提取包中文件后运行的脚本。两个参数会按以下顺序传递：新版本号，旧版本号。

- `pre_remove` - 文件被删除前运行的脚本，会传递一个参数：旧版本号。
- `post_remove` - 文件被删除后运行的脚本，会传递一个参数：旧版本号。

每一个函数都是 chroot 到 *pacman* 安装目录下运行的。参见这个帖子.

提示：

- 可以在 /usr/share/pacman/proto.install 这里找到一个 `.install` 文件的模板（原型）。
- Pacman 钩子也提供相似的功能。

注意：脚本不要以 `exit` 结束，否则包含该脚本的函数无法执行。

changelog

软件包的更新日志的文件名。要查看安装软件的更新日志（如果有）：

```
$ pacman -Qc pkgname
```

源码

source

构建软件包时需要的文件列表。它必须包含软件源的位置，大多数情况下是一个完整的 HTTP 或 FTP 地址。您可以在此处调用前面提到的变量 `pkgname` 和 `pkgver`，来实现高效的命名（如 `source=("https://example.com/${pkgname}-${pkgver}.tar.gz")`）。

文件也可以放到与 `PKGBUILD` 文件相同目录，并将文件名添加到这个列表。在实际的编译过程开始之前，所有该列表中引用的文件都会被下载或检查是否存在，如果有文件丢失 *makepkg* 就不会继续。

.install 文件会被 *makepkg* 自动识别，而不应该被包含在这个列表中。*makepkg* 会自动把 `source` 中 *.sig*，*.sign* 或 *.asc* 结尾的文件当成 PGP 签名，并自动验证对应的源文件的完整性。

警告：因为所有包的 SRCDEST 的值可能相同，所以下载的文件的文件名需要唯一。比如说，如果一个项目只用版本号来命名，可能会与其他有相同版本号的不同项目冲突。在这种情况下，你可以为下载的文件指定不同的文件名：`source=('*unique_package_name***::***file_uri*')`。例如 `source=("${pkgname}-${pkgver}.tar.gz::https://github.com/coder/program/archive/v${pkgver}.tar.gz")`。

注意：

- 架构相关的文件可以通过在名称后添加下划线和架构的方式指定，例如 `source_x86_64=()`。必须提供对应的完整性校验和序列，例如 `sha256sums_x86_64=()`。
- 有些服务器通过 *User-Agent* 或其它方式来筛选并限制下载。这个限制可以通过 DLAGENTS 规避。

- 可以使用 `file://` URL 来指向本地目录或文件，例如指定本地 Git 仓库：`"${pkgname}::git+file://*/path/to/repository*"`。
- Magnet link support can be added using transmission-dlagentAUR as `DLAGENT` and using the `magnet://` URI prefix instead of the canonical `magnet:?`.
- See PKGBUILD(5) § USING VCS SOURCES and VCS 软件打包准则#VCS 源代码 for details on VCS specific options, such as targeting a specific Git branch or commit.

noextract

一个在 `source` 中列出，但不应该由 makepkg 解包的文件列表。这通常包括那些不能被 `/usr/bin/bsdtar` 处理的压缩文件，或者本来就不需要解压、按照原样提供的文件。对于前者，需要将额外的解包工具（如 `unzip`，`p7zip`，lrzip包 等）加入 `makedepends` 序列，且 prepare()) 函数的第一行需进行手动解压。例如：

```
prepare() {
  lrzip -d source.tar.lrz
}
```

注意当 `source` 是一些 URL 时，`noextract` **仅仅**取文件名部分：

```
source=("http://foo.org/bar/foobar.tar.xz")
noextract=('foobar.tar.xz')
```

不提取任何东西时，可以像这样：

- 如果 `source` 只包含了纯 URL，而没有自定义的文件名时，将内容从最后一个斜杠之前像这样从 `source` 序列中提出来：

: ``` noextract=("${source[@]##*/}")

- 如果 `source` 只包含了带自定义文件名的项，将内容从分隔符 `::` 之前像这样从 `source` 序列中提出来（这是从旧版 [firefox-i18n 的 PKGBUILD 中找到的](https://gitlab.archlinux.org/archlinux/packaging/packages/firefox-i18n/-/blob/46492498ef720353cbb84de5096102de694faf90/PKGBUILD#L132)）：

: ```
noextract=("${source[@]%%::*}")
```

### *validpgpkeys*

PGP 指纹列表。如果使用，*makepkg* 仅接受这里定义的签名，并且忽略密钥环中的值。如果源代码用子密钥签名，*makepkg* 仍然会使用主密钥进行比较。

此处仅接受完整的指纹。它们必须是大写字母而且不能有空白字符。

**注意**：可以使用 `gpg --list-keys --fingerprint *KEYID*` 查找密钥的指纹。

请参阅 makepkg#验证签名 了解签名验证过程的详细信息。

## 完整性检验

下面描述的这些序列中的变量是 source 序列中对应文件的校验和。可以插入 `SKIP` 跳过某个不需要检验的文件。
```

校验和的类型和数值应该始终使用上游提供的数值（比如在新版本公告中的）。当存在多种类型的时候，最好选用最强的校验类型。类型按照从强到弱的顺序如下：`b2` > `sha512` > `sha384` > `sha256` > `sha224` > `sha1` > `md5` > `ck`，这样可以最大限度地保证从上游的公告到软件包的生成整个流程中下载文件的完整性。

注意：另外，当上游具备数字签名文件时，这个签名文件应该被添加到 source 序列中，PGP 密钥添加到 validpgpkeys 序列中。这样允许在生成软件包时进行验证。

makepkg 的 `-g`/`--geninteg` 选项可以自动生成校验值，通常可以通过 `makepkg -g >> PKGBUILD` 命令写入。pacman-contrib包 提供的 updpkgsums(8) 命令也可以更新 `PKGBUILD` 中的变量。两个工具都会自动检测 PKGBUILD 中的算法, 如果没找到就回滚到 `md5sums`。

The file integrity checks to use can be set up with the `INTEGRITY_CHECK` option in `/etc/makepkg.conf`. See makepkg.conf(5).

注意：架构相关的额外完整性检验可以通过在名称后添加下划线和架构的方式指定，例如 `sha256sums_x86_64=()`。

b2sums

BLAKE2 校验和数组，大小为 512 位。

sha512sums，*sha384sums*，*sha256sums*，*sha224sums*

SHA-2 校验和数组，大小分别为 512，384，256 和 224 位。最常见的是 `sha256sums`。

sha1sums

`source` 数组中文件的 160 位 SHA-1 校验和数组。

md5sums

`source` 数组中文件的 128 位 MD5 校验和数组。

cksums

`source` 数组中文件的 CRC32（使用 UNIX 标准的 cksum）校验和数组。

包管理

本章处理装完系统后的三件维护难题：某个命令跟着哪个包走、pacman 从哪个镜像取包、把软件包退回旧版本。先读《Reflector》和《Pkgfile》，后者靠更新过的索引才能回答“Command not found”该装什么，不装钩子就形同虚设；接着必读《降级软件包》。其余《Pkgstats》《Local Mirror》《Powerpill》《软件包补丁》可跳过。

最容易踩的坑：把《元软件包和软件包组》混为一谈，移除元软件包会连带卸掉整组依赖；从 [testing] 退回来时以为降一个包就够。

6.1　Pkgstats

难度 入门 | **篇幅** 1.8 KB | **所属** 包管理

本节将讲到

- 安装
- 用法
- 结果和参考

pkgstats 将所有已安装软件包，您正在使用的体系结构和镜像的列表发送到 Arch Linux 项目。该信息是匿名的，不能用于识别用户，但是它将帮助 Arch 开发人员确定其工作的优先级（源代码）。另请参阅该项目的隐私权政策。

安装

安装 pkgstats包 软件包。

用法

pkgstats 设置为使用 systemd/Timers 每周自动运行。一旦安装，它将在下次重启后被激活。

如果不想等待重新启动周期，则可以手动启动 `pkgstats.timer`。

pkgstats 也可以手动运行：有关用法的信息，请参见 `pkgstats -h`。

结果和参考

从 https://pkgstats.archlinux.de/ 上可以找到统计信息和文档，https://pkgstats.archlinux.de/fun 上包含相关的比较。

通过以下公开的 API 可以获取 JSON 格式的统计数据：API 文档。

6.2 Pkgfile

难度 入门 | **篇幅** 2.8 KB | **所属** 包管理

本节将讲到

- 安装
- 使用
- “Command not found” 钩子
- 自动更新

工具 **pkgfile** 是检查官方软件仓库中软件包文件的工具。

提示：pacman包 有内置的类似功能。

安装

安装软件包 pkgfile包，或者安装开发版本的软件包 pkgfile-gitAUR。

然后，pkgfile 数据库可以用以下方式同步:

```
# pkgfile -u
```

使用

查找文件 “makepkg” 属于哪个软件包:

```
$ pkgfile makepkg
core/pacman
```

列出 archlinux-keyring包 包含的所有文件：

```
$ pkgfile -l archlinux-keyring
core/archlinux-keyring usr/
core/archlinux-keyring usr/share/
core/archlinux-keyring usr/share/pacman/
core/archlinux-keyring usr/share/pacman/keyrings/
core/archlinux-keyring usr/share/pacman/keyrings/archlinux-revoked
core/archlinux-keyring usr/share/pacman/keyrings/archlinux-trusted
core/archlinux-keyring usr/share/pacman/keyrings/archlinux.gpg
```

这个结果与 `pacman -Ql` 类似(参考pacman#查询包数据库)，只不过这个命令查询的是远程仓库中的软件包。

“Command not found” 钩子

详见 Bash#找不到命令、Zsh#pkgfile“未找到命令”处理程序与 Fish#"command not found" 事件函数。

自动更新

pkgfile 提供了 systemd 服务和定时器，可以自动同步 pkgfile 数据库。要自动启动，请启用 `pkgfile-update.timer`.

默认情况下 pkgfile 每天更新一次，可以通过编辑单元文件进行配置。

6.3 Reflector

难度 入门 | **篇幅** 2.9 KB | **所属** 包管理

本节将讲到

- 安装
- 用法
- 自动处理
- 参考

Reflector 是一个 Python 脚本；它可以从 Arch Linux Mirror Status 页面获取最新的镜像列表，然后筛选出最新的镜像并按速度排序，最后将结果写入到 `/etc/pacman.d/mirrorlist` 文件。

安装

安装 reflector包 软件包。

用法

警告：

- 在下面的例子中，会覆盖 `/etc/pacman.d/mirrorlist` 文件。进行操作前应该进行备份。
- 在同步或更新 pacman 前，需要确保 `/etc/pacman.d/mirrorlist` 文件没有包含你不信任的镜像。

要查看所有可用选项，运行以下命令：

```
$ reflector --help
```

示例

请见 reflector(1) § EXAMPLES。覆盖 `/etc/pacman.d/mirrorlist` 的示例需要以 root 用户运行。

提示：要想包含全球服务器,向 country 选项传递一个空的字符串。举个例子,要想包含法国,德国和全球镜像,应该使用 `--country 'France,Germany,'`。

自动处理

systemd service

Reflector 附带一个 `reflector.service`。这个服务会用 `/etc/xdg/reflector/reflector.conf` 中指定的参数运行 Reflector。此文件中的默认选项是一个很好的起点和示例。

例如，从法国和德国的镜像中筛选出 5 个最新的并且支持 HTTPS 的镜像，然后将结果覆写到 `/etc/pacman.d/mirrorlist` 文件内；使用：

```
/etc/xdg/reflector/reflector.conf
--save /etc/pacman.d/mirrorlist
--country France,Germany
--protocol https
--latest 5
```

启用 `reflector.service` 服务可以在引导时运行 Reflector 脚本。要立即运行，启动该服务。

注意：`reflector.service` 依赖于一个通过 network-online.target 配置的网络等待服务。

systemd timer

Reflector 提供一个 systemd 定时器（`reflector.timer`），这样每周都会调用 `reflector.service` 服务。可以编辑 `reflector.timer` 来调整时间表。

首先像服务一节那样编辑好配置文件。配置文件更新后，启用或启动 `reflector.timer` 定时器。

要提前刷新镜像列表，启动 `reflector.service` 服务。

pacman hook

pacman-mirrorlist包 不定期更新,只是因为一些地区添加或移除镜像调用 Reflector 与此无关。改为使用基于时间的自动化。如果你完全不想安装 `mirrorlist.pacnew`, 在 `pacman.conf` 使用 `NoExtract`。

参考

- Arch Linux 论坛上的 Reflector 帖子
- 包含变更日志的上游项目页面

6.4 元软件包和软件包组

难度 入门 | **篇幅** 4.4 KB | **所属** 包管理

本节将讲到

- 区别
- 元软件包
- 软件包组

软件打包者可以将一组相关的软件包定义为**元软件包**或**软件包组**。通过安装一个元软件包或软件包组，可以同时安装一系列的软件包。虽然软件包组不是一个软件包，但安装操作和一个真正的软件包相同。请参考 pacman#安装包组和 PKGBUILD#groups。

区别

元包和普通包的区别在于元包是空的，存在元包纯粹是为了通过依赖关系将相关包链接在一起。*元包*通常（但不总是）以“-meta”结尾，提供与包组类似的功能，因为它可以同时安装或卸载多个相关包。

每个方式都有其优缺点:

元软件包：

- 安装方法和其它软件包一样（pacman#安装指定的包）。
- 删除方法和其他软件包一样（pacman#删除软件包）。
- 所有后续加入的软件包都会在更新时自动安装。
- 无法选择仅安装元软件包依赖的部分软件。
- 只有删除了元软件包之后，才能删除其依赖的软件包。

软件包组：

- 安装软件包组时会可以选择安装哪些软件包（Pacman#安装包组）。
- 软件包组仅是一个列表，所以无法删除一个软件包组，`pacman -R *groupname*` 会尝试删除软件包组中的所有软件。
- 新加入软件包组的软件不会在更新时自动安装，但可以通过 `comm -23 <(pacman -Sg *软件包组名* | awk '{print $2}' | sort) <(pacman -Qq | sort)` 查看 *软件包组名* 中的哪些软件还没有被安装。
- 可以选择安装组中的部分软件。
- 可以选择仅删除软件包组中的部分软件，而无需删除整个包组。

元软件包

最重要的元软件包是 base包，它包含了 Arch Linux 安装时需要的最小软件集，其中有：

- 基本的软件包，如 glibc包 和 bash
- 发行版相关工具，如 pacman 和 systemd
- POSIX 工具，例如核心工具、进程、文件、压缩工具等
- 网络工具，如 iproute2包

内核是可选依赖，请参考发布说明和为什么 base 是元软件包。

另一个元软件包是 base-devel包，它包含了完整的 makepkg 编译环境。这里记录了将其从软件包组转换为元软件包的原因。

软件包组

软件包组常用来简化桌面环境的安装，请参考桌面环境#桌面环境列表。

另外一个软件包组是 pro-audio包组，其提供了官方仓库中与专业音频相关的软件。

请参考软件包组列表。

6.5 Powerpill

难度 入门 | **篇幅** 5.1 KB | **所属** 包管理

本节将讲到

- 安装
- 配置
- 使用 Reflector
- 使用 rsync
- 基本使用

Powerpill 是一个 pacman 包装，它使用并行和分段下载来加速 Pacman 的下载。内部使用 Aria2 和 Reflector 来实现这一点。Powerpill 还可以将 rsync 用于支持它的官方镜像。对于从单镜像下载时已经使用全部带宽的用户来说非常有效。配置文件也支持 Pacserve，并将在从外部镜像下载之前使用。示例：有人想要更新然后运行了 `pacman -Syu`，它返回一个包含20个包的列表用于更新，总计200兆。如果用户通过 pacman 下载，它们将一次下载一个。如果用户通过 powerpill 下载，它们将同时下载，许多情况下所需时间会下降几倍（取决于一个人的连接速度、服务器上软件包的可用性以及服务器/负载的速度等。）

在一个系统上对 pacman 和 powerpill 进行的对比测试表明，在上述场景中，pacman 的下载速度平均为 300 kB/sec，powerpill 的下载速度平均为 1.2 MB/sec，速度提高了4倍。

安装

安装 powerpillAUR 软件包。

配置

Powerpill 有一个单独配置文件 `/etc/powerpill/powerpill.json`，你可以按喜好进行编辑。参考 powerpill.json(1) 手册页获取详细信息。

使用 Reflector

默认情况下，Powerpill 配置为使用 Reflector 从 Arch Linux 服务器的 web API 检索当前镜像列表，并将其用于并行下载。这是为了确保列表中有足够的服务器以显著提高速度。

使用 rsync

一些镜像可以支持 Rsync。在启用后，因为使用单连接，数据库同步 (`pacman -Sy`) 和其它操作可能会快得多。rsync 协议本身也加速了更新检查，有时还加速了文件传输。

要找到支持 rsync 的镜像，使用 `reflector`:

```
$ reflector -p rsync
```

另外，你可以使用标志 `-f *n*` 以获得 `*n*` 个最快的服务器，和使用标志 `-l *m*` 以获得 `*m*` 个最新同步的服务器：

```
$ reflector -p rsync -f n -l m
```

选择你想使用的镜像。`-c` 选项可以按国家进行筛选（使用 `reflector --list-countries` 查看全部列表，在名称周围加引号并注意大小写！）。完成后，编辑 `/etc/powerpill/powerpill.json`，向下滚动到 `rsync` 部分，并向 servers 字段添加任意数量的服务器。

之后，所有官方数据库和软件包将尽可能从 rsync 服务器下载。

注意，需要检查数据库和包是否位于带 reflector 的官方存储库中，因此安装 reflector 是 rsync 功能运行所必需的。

基本使用

对于大多数操作，*powerpill* 与 pacman 的工作方式差不多，因为它是 *pacman* 的包装脚本。

系统更新

想要使用 powerpill 更新你的系统（同步并更新所有安装的包），和 *pacman* 一样使用菜单项 `-Syu`:

```
# powerpill -Syu
```

安装软件包

想要安装软件包和它的依赖，和 *pacman* 一样使用菜单项 `-S`:

```
# powerpill -S package
```

安装多个软件包时也和使用 *pacman* 一样：

```
# powerpill -S package1 package2 package3
```

疑难解答

如果你得到 <repo>.db.sig 文件的 \[err\]:

```
   b5d7d7|ERR |       0B/s|/var/lib/pacman/sync/extra.db.sig
   899e91|ERR |       0B/s|/var/lib/pacman/sync/multilib.db.sig
   8fcc32|ERR |       0B/s|/var/lib/pacman/sync/core.db.sig
   85eb3d|ERR |       0B/s|/var/lib/pacman/sync/community.db.sig
```

这是因为该存储库缺少签名文件，而且你还没有在 `/etc/pacman.conf` 中显式设置

```
SigLevel = PackageRequired
```

，更多说明请见这个帖子。

参考

- Powerpill - 项目官方页面
- powerpill reborn - powerpill is back :)

6.6 降级软件包

难度 入门 | **篇幅** 5.4 KB | **所属** 包管理

本节将讲到

- 如何降级软件包
- 从\[testing\]中返回

在决定降级之前，请小心考虑。如果是因为现有包有Bug，请在Bug追踪系统搜索现有的Bug报告。如果没有，请花上几分钟帮忙把它报告给Arch的Bug追踪系统或软件包的项目地址，或者在论坛中警告其他可能遇到类似问题的用户。

警告：

- 降级某个软件包可能需要降级相应的依赖包.如果依赖包数量巨大,参见Arch Linux Archive#How to restore all packages to a specific date.
- 在修改配置文件和脚本时请小心.只要我们不绕过pacman的安全措施,它将为我们处理这些问题.
- 如果软件包降级会引进 soname 变更，所有依赖包可能都需要降级或重新编译.

如何降级软件包

使用pacman的临时文件

如果一个新包刚刚被安装并且没有删除pacman cache,你可以在`/var/cache/pacman/pkg/`中找到较早版本. 安装替换现有的版本.pacman会处理依赖包但不会处理依赖库的版本冲突。如果一个其依赖库因该包降级需要降级，你需要手动降级这些包。

```
 # pacman -U /var/cache/pacman/pkg/package-old_version.pkg.tar.type
```

对老的软件包，`type` 应该是 `xz`，遵循 2020 变更的新软件包，`type` 应该是 `zst`。

当成功降级该包以后，请暂时将其加入pacman.conf的IgnorePkg section，直到您的问题被解决。

降级内核

新内核出问题时，可以通过使用 pacman 缓存降级内核。

在/var/cache/pacman/pkg 寻找旧版安装包。必须降级的有linux包， linux-headers包和任何内核模块。举例来说：

```
# pacman -U linux-4.15.8-1-x86_64.pkg.tar.xz linux-headers-4.15.8-1-x86_64.pkg.tar.xz virtualbox-host-modules-arch-5.2.8-4-x86_64.pkg.tar.xz
```

退出并重启。

如果您在升级内核后无法启动，您可以通过使用 live CD 来降级内核。方法类似Arch Linux的安装进程。启动后在/mnt挂载你的根目录文件系统，别忘了挂载其他分区如/boot或/var等等。(例子 mount /dev/sdc3 /mnt/boot)。然后chroot进入系统。

Arch Linux Archive

Arch Linux Archive是official repositories的日更快照。

*ALA*能被用来降级包或者还原整个系统到过去版本。

手动编译

如果找不到编译好的软件包，就需要自己找到 PKGBUILD 并通过 makepkg 编译。

官方软件仓库中的 PKGBUILD 可以通过 ABS 获取并修改软件版本。或者访问软件包页面搜索你需要的降级的软件包，然后点 "查看修改" 链接，选择“查看日志“。找到需要的版本并通过 *Tree* 视图下载 .tar.gz 快照。

参阅 Arch Build System#Checkout an older version of a package.

老的 AUR 软件包可以从 AUR 软件包 git 仓库提出旧版本。如果是 2015 AUR3 之前的 PKGBUILD，请参阅 Arch User Repository#Git repositories for AUR3 packages.

自动化

downgrade — 基于Bash使用本地缓存和Arch Rollback Machine。详见 downgrade(8)。

: https://github.com/pbrisbin/downgrade || downgradeAUR

从\[testing\]中返回

参见 Official repositories#Disabling testing repositories。

6.7 Local Mirror

难度 进阶 | **篇幅** 6.1 KB | **所属** 包管理

本节将讲到

- 稍等
- 本地镜像

稍等

警告：通常不赞成创建一个本地镜像，因为需要很大的带宽。其中一个替代的方法也许会满足你的要求。请查看下面的替代方法。

替代方法

- 在网络上共享 pacman 缓存

本地镜像

需要牢记的事实

- 镜像的带宽不是免费的，镜像必须为提供给你的数据付费
- 尽管你已经为你的 ISP 付费，这也是成立的。
- 有很多你可能下载了却永远用不到的包
- 镜像的管理员会更喜欢让你仅下载需要的包
- 再次请求仔细查看上面的替代方案

如果你完全确定本地镜像是唯一合适的解决方案，那么按照下面的指引操作。

服务器设置

编写 Rsync 命令

- 参照 DeveloperWiki:NewMirrors 使用 rsync 参数
- 从上文中选择一个服务器
- 在 rsync 参数中包含 `--exclude-from="/path/to/exclude.txt"` 以排除那些你不想包含的目录或文件。示例内容可能包括：

```
iso

#Exclude i686 Packages
*/os/i686
pool/*/*-i686.pkg.tar.xz
pool/*/*-i686.pkg.tar.gz

#Exclude x86_64 Packages
*/os/x86_64
pool/*/*-x86_64.pkg.tar.xz
pool/*/*-x86_64.pkg.tar.gz
```

- 所有软件包都位于 pool 目录。然后从 pool 创建符号到 to core/extra/testing/etc。
- 截至 9/21/2010 此迁移仍没有完成。
- 在 ${repo}/os/${arch} 中应该有确切的软件包，而不是符号链接
- 排除那些你不需要的顶极目录

例如： `rsync *$rsync_arguments* --exclude="/path/to/exclude.txt" *rsync://example.com/* /path/to/destination`

示例脚本

警告：除非你阅读了本文开头的警告，否则不要使用本脚本

警告：仅使用本脚本同步 Core/Extra/Community！ 如果你需要 Testing，gnome-unstable 或是任何其他软件库，另外使用 rsync --exclude！

是的，为了避免人们以复制粘贴的形式搭建他们自己的镜像，本脚本故意缺失了部分内容。如果你确实想要搭建一个镜像，错误是很容易修补的。

```
#!/bin/bash

################################################################################
##########
### It is generally frowned upon to create a local mirror due the bandwidth that is
required.
### One of the alternatives will likely fulfill your needs.
### REMEMBER:
###   * Bandwidth is not free for the mirrors. They must pay for all the data they
serve you
###       => This still applies although you pay your ISP
###       => There are many packages that will be downloaded that you will likely never
use
###       => Mirror operators will much prefer you to download only the packages you
need
###   * Really please look at the alternatives on this page:
###       https://wiki.archlinux.org/index.php?title=Local_Mirror
### If you are ABSOLUTELY CERTAIN that a local mirror is the only sensible solution,
then this
### script will get you on your way to creating it.
################################################################################
##########

# Configuration
SOURCE='rsync://mirror.example.com/archlinux'
DEST='/srv/mirrors/archlinux'
BW_LIMIT='500'
REPOS='core extra'
RSYNC_OPTS="-rtlHq --delete-after --delay-updates --copy-links --safe-links --max-
delete=1000 --bwlimit=${BW_LIMIT} --delete-excluded --exclude=.*"
LCK_FLE='/var/run/repo-sync.lck'

# Make sure only 1 instance runs
if [ -e "$LCK_FLE" ] ; then
       OTHER_PID=`/bin/cat $LCK_FLE`
       echo "Another instance already running: $OTHER_PID"
       exit 1
fi
echo $$ > "$LCK_FLE"

for REPO in $REPOS ; do
```

```
        echo "Syncing $REPO"
        echo /usr/bin/rsync $RSYNC_OPTS ${SOURCE}/${REPO} ${DEST}
done

# Cleanup
/bin/rm -f "$LCK_FLE"

exit 0
```

另外一个使用 lftp 的脚本

lftp 可以通过许多不同的协议来做镜像，如： ftp，http。它会在出错时重启，而且可以在后台运行。把如下代码放置到你的 $PATH 可以方便地使你在登出后镜像仍正常工作。

```
#!/usr/bin/lftp -f
lcd /local/path/to/your/mirror
open ftp.archlinux.org (or whatever your favorite mirror is)
# Use 'cd' to change into the proper directory on the mirror, if necessary.
mirror -cve -x '.*i686.*' core &
mirror -cve -x '.*i686.*' extra &
mirror -cve -x '.*i686.*' community &
mirror -cve -x '.*i686.*' multilib &
lcd pool
cd pool
mirror -cve -x '.*i686.*' community &
mirror -cve -x '.*i686.*' packages &
```

如果你想要查看当前镜像的状态，在终端打开 lftp ，输入 'attach <PID>'

部分镜像

由于大多数软件包集中在 `\pool/\`，仅镜像部分软件库显然不容易。 这篇博文提供了一个脚本用于完成此任务。

服务

- HTTP (LAN)
- LAMP
- Lighttpd
- FTP (LAN)
- vsftpd
- 物理媒介
- 闪存
- 移动硬盘

客户端配置

- 在 /etc/pacman.d/mirrorlist 添加合适的 Server = variable
- 对于物理媒介 (例如闪存)，可以输入如下内容： Server = file:///mnt/media/repo/$repo/os/$arch (*/mnt/media/repo 是本地镜像位于的目录*)

6.8 软件包补丁

难度 进阶 | **篇幅** 7.1 KB | **所属** 包管理

本节将讲到

- 创建补丁
- 应用补丁
- 使用 quilt
- 另见

本文涵盖了如何在 Arch 构建系统（ABS）中为软件包创建和应用补丁。

补丁)描述了一个或多个文件的行更改。补丁通常用于自动更改源代码。

创建补丁

注意：如果你只是想要更改一两行，你可能会想使用 sed。

diff 工具会逐行比较文件。将它的输出保存下来你就得到了一个补丁，如 `diff --unified --recursive --text foo bar > foobar.patch`（可以将参数简写为 `diff -ura`）。如果你传递了目录， *diff* 会比较它们包含的文件。

1. 如果你已将构建了软件包，请删除 `src` 目录。
2. 运行 `makepkg --nobuild`/`makepkg -o` 会下载并提取 `PKGBUILD` 中声明的源文件，但不会进行构建。如果你正在创建补丁的系统上没有必要的依赖，你可能需要转而使用 `makepkg --nobuild --nodeps`/`makepkg -od`。
3. 在 `src` 目录中创建两份提取的目录的副本，一个作为原始版本，一个作为你修改过的版本，下面我们会将他们称为 `package.orig` 和 `package.new`。
4. 在 `package.new` 目录中进行你的更改。
5. 运行 `diff -ura package.orig package.new --color` 并检查补丁是否看起来良好。
6. 运行 `diff -ura package.orig package.new > package.patch` 来生成补丁。
7. 进入 `package.orig` 目录并使用 `patch --strip=1 --input=../package.patch`（可以简写为 `patch -p1 -i ../package.patch`）命令来应用补丁。运行 `makepkg --noextract --install`/`makepkg -ei` 命令构建并安装更改后的软件包来验证补丁是否正常工作。

注意：你也可以使用 Git 的 `git diff` 或 `git format-patch` 命令 。

更多信息参见 diff(1) 和 git-diff(1)。

提示：大部分 git 托管平台（例如 GitHub 和 GitLab）支持为特定提交或合并生成补丁，只需在其 URL 结尾添加 `.patch`。对于 GitHub，请考虑在 URL 最后添加 `?full_index=1` 参数，以避免未来对象哈希值长度变更导致的校验值不匹配错误。

应用补丁

本节概述如何在 `PKGBUILD` 的 `prepare()` 函数中应用你创建或从网络下载的补丁。请遵循以下步骤：

1. 为补丁文件在 `PKGBUILD` 的 `source` 数组中添加一个条目，将该条目与原始源 url 之间用空格隔开。如果该文件可在网上获得，你可以提供完整的 URL ，它将被自动下载并放在 `src` 目录中。如果它是你自己创建的补丁，则应将补丁文件与 `PKGBUILD` 文件放在同一目录中，并只需将文件名添加到 `source` 数组中，以便将其复制到 `src` 目录中。如果你要分发 `PKGBUILD`，就需要将补丁与 `PKGBUILD` 一同发布。
2. 然后使用 `makepkg -g >> PKGBUILD` 或者 `updpkgsums`（来自于 pacman-contrib包）来更新 `sha512sums` 数组。或者手动添加一个条目到 `sha512sums` 数组； 你可以使用 *sha512sum* 工具来生成你的补丁的校验和。
3. 如果 `PKGBUILD` 中还没有 `prepare()` 函数，就创建一个。
4. 第一步是在 `prepare()` 函数中进入到需要被打补丁的目录，可以通过 `cd $pkgname-$pkgver` 来完成。`$pkgname-$pkgver` 通常是源码包解包后的目录名，但并不总是这样。
5. 现在你只需要在这个目录里面应用补丁。这可以通过在 `prepare()` 函数中添加 `patch -p1 -i *pkgname*.patch` 来完成。将 `*pkgname*.patch` 改成包含 diff 的文件的名字（由于其在 `PKGBUILD` 的 `source` 数组中，因此已被自动复制到 `src` 目录下）。

一个 prepare 函数示例：

```
PKGBUILD
prepare() {
    cd $pkgname-$pkgver
    patch -Np1 -i ../eject.patch
}
```

或者不先 cd，直接使用 `patch` 的 `--directory`/`-d` 参数：

```
PKGBUILD
prepare() {
    patch -d $pkgname-$pkgver -Np1 -i ../eject.patch
}
```

从终端运行 `makepkg`，如果一切顺利，补丁会被自动应用，并且你的新软件包会包含补丁中的更改。如果不是这样的话，你可能需要试验一下 patch 的 `--strip`/`-p` 选

项。在试验的时候，你会发现 `--dry-run`, `--reverse` 或者 `--verbose` 选项会有用。更多信息参见 patch(1)。

基本上它的工作原理如下：如果 diff 文件被用于将补丁应用到 `myversion/` 中的文件，那么 diff 文件将被应用于 `myversion/file`。你当前的运行目录为 `yourversion/`（即 `PKGBUILD` 中 cd 进的目录），在将补丁应用到文件时，需要将 `myversion/` 部分移除，只应用到 `file` 文件，所以需要使用 `-p1` 参数将路径中的第一个目录移除。但如果需要修补 `myfiles/myversion` 中的文件，就需要使用 `-p2` 移除两个目录。

如果你不应用一个 `-p` 选项，它将会去掉所有目录结构。如果所有文件都在基目录中的时候是可以的，但是如果补丁是在 `myversion/` 上创建的，并且有一个被编辑的文件是 `myversion/src/file`，当你从 `yourversion` 目录以不带 `-p` 参数应用补丁的时候，它会尝试为一个名为 `yourversion/file` 的文件打补丁。

大多数开发者从被打补丁的目录的父目录创建补丁，因此 `-p1` 通常是正确的。

使用 quilt

一个更简单的创建补丁的方法是使用 quilt包，它为管理大量补丁提供了更好的支持，例如应用补丁，更新补丁和将被打补丁的文件回退到原始状态。Debian 使用 quilt包来管理他们的补丁。使用 quilt 生成和应用补丁、回退被打补丁的文件的基本用法和信息可参考使用 Quilt。

另见

- http://www.kegel.com/academy/opensource.html —— 有关修补文件的有用信息

初始化

内核交出控制权后由谁启动服务、管理运行级别，是这一层要回答的问题。主线是默认的 systemd 一节，讲日常用法、单元文件与启动目标的组织；Init 一节理清整合式 init、启动脚本与服务管理器的模型；BusyBox、OpenRC、SysVinit 三节是替代方案。

Systemd 与 FAQ 必读，后者解答常见问题；Init 梳理模型，时间紧可略读；替代方案真要用时再看。最易踩的坑是把 systemd 当作开机脚本，以为别家的写法能照搬：两套体系并不通用，混用只会得到不生效的配置。

7.1　BusyBox

难度 入门 | **篇幅** 2.0 KB | **所属** 初始化

本节将讲到

- 安装
- 用法
- 参见

BusyBox 在单个小型可执行文件中为嵌入式系统提供了许多常见的 UNIX 实用程序。软件包还包含 runit; 详情请参考 runit。

注意：不要期望完全的即插即用替换和兼容性。某些实用程序可能不存在，而对于某些实用程序，则可能缺少选项。

安装

安装 busybox包 包。

Busybox 命令是指向 `/usr/bin/busybox` 的符号链接，因此仅占用很少的空间。这对于低占用空间的系统特别有用。

用法

init

Init 脚本可以与 busybox-init 一起使用，例如 minirc-gitAUR。有关详细信息，请参见 init。

getty

这些 getty 在文件 `/etc/inittab` 中定义。默认情况下，getty 在 tty 1 到 4 上启动。

为了启用/禁用 getty，您只需将这一行放在 `/etc/inittab` 中

```
tty2::respawn:/sbin/agetty -8 -s 38400 tty2 linux
```

只需将 tty2 替换为 tty，就可以开始使用 getty。 如果您想让 init 在启动 gettty 之前询问您，则将 `respawn` 替换为 `askfirst`。

mdev

请参阅 Gentoo wiki。

runit

请参考 runit。

参见

- w:ToyBox

7.2 Systemd/FAQ

难度 进阶 | **篇幅** 6.4 KB | **所属** 初始化

本节将讲到

- 常见问题

常见问题

最新的已知问题，参见：TODO。

为什么控制台上会显示日志信息?

请自行设置内核日志等级（loglevel）。以前，`/etc/rc.sysinit` 帮我们把 dmesg 的日志等级设置为 `3`，是比较合适的。内核参数中加入 `loglevel=3` 或 `quiet` 即可。

如何修改默认的 tty 控制台（getty）数量?

目前默认仅启动一个 getty，如果切换到其它 tty, 一改新的 getty 会通过 socket 激活方式启动。例如 \[Ctl\] \[Alt\] \[F2\] 会在 tty2 启动 getty。

默认情况下，最多可以自动启动 6 个 getty。\[F7\] 到 \[F12\] 不会启动 getty。要修改最大值，在 `/etc/systemd/logind.conf` 中修改 `NAutoVTs`。如果要启用所有的 \[F*x*\]，可以将其设置为 12。如果要将日志转发到tty12，可以把 NAutoVTs 设置为 11。

修改启动时启用的 tty 个数，可以采取下面方法：

添加新的 getty：

在 `/etc/systemd/system/getty.target.wants/` 添加新的软链接即可：

```
# ln -
sf /usr/lib/systemd/system/getty@.service /etc/systemd/system/getty.target.wants/getty@
tty9.service
# systemctl start getty@tty9.service
```

移除 getty：

从 /etc/systemd/system/getty.target.wants/ 删除对应的软链接即可：

```
#
rm /etc/systemd/system/getty.target.wants/getty@tty5.service /etc/systemd/system/getty.
target.wants/getty@tty6.service
# systemctl stop getty@tty5.service getty@tty6.service
```

systemd 不使用 /etc/inittab 文件。

怎样输出更详细的开机信息?

如果内核信息输出后就什么信息都不输出了，很可能是因为你在内核参数中添加了 quiet。删除即可，然后你就可以看到一列列绿色的 \[OK \] 和红色的 \[FAILED \] 了。

所有信息都记录在系统日志，可以通过 $ systemctl 查看系统状态，通过 journalctl 查看日志。

开机后控制台信息会被清空，如何避免？

创建 /etc/systemd/system/getty@.service.d 目录，创建文件 nodisallocate.conf，将 TTYVTDisallocate 设置为 no.

```
/etc/systemd/system/getty@.service.d/nodisallocate.conf
[Service]
TTYVTDisallocate=no
```

Systemd 需要哪些内核模块?

systemd 不支持 3.0 版本之前的内核。

如果是自己编译内核，需要开启一些选项。请参考/usr/share/doc/systemd/README.

自己编译 Systemd，请参考systemd git 中的版本.

怎样知道一个目标需要哪些进程服务?

例如，你可能想搞明白目标单元 multi-user.target 究竟启用了哪些服务，那么以下命令即可：

```
$ systemctl show -p "Wants" multi-user.target
Wants=rc-local.service avahi-daemon.service rpcbind.service NetworkManager.service
acpid.service dbus.service atd.service crond.service auditd.service ntpd.service
udisks.service bluetooth.service org.cups.cupsd.service wpa_supplicant.service
getty.target modem-manager.service portreserve.service abrtd.service yum-
updatesd.service upowerd.service test-first.service pcscd.service rsyslog.service
haldaemon.service remote-fs.target plymouth-quit.service systemd-update-utmp-
runlevel.service sendmail.service lvm2-monitor.service cpuspeed.service udev-
post.service mdmonitor.service iscsid.service livesys.service livesys-late.service
irqbalance.service iscsi.service
```

除了 Wants，还可以查看各种形式的依赖和被依赖信息：WantedBy、Requires、RequiredBy、Conflicts、ConflictedBy、Before、After。

电脑关闭了但电源没有断。

使用systemctl poweroff 而不是 systemctl halt.

如何在启动的时候，运行自定义的一个脚本?

在/etc/systemd/system 中新建一个文件(名称可以为 *myscript*.service) 然后在其中写入如下内容：

```
[Unit]
Description=My script

[Service]
ExecStart=/usr/bin/my-script

[Install]
WantedBy=multi-user.target
```

然后开启该守护进程

```
# systemctl enable myscript.service
```

本例是说当目标multi-usr载入的时候，会启动你这个自定义脚本。脚本需要有可执行权限。

注意：如果要启动 shell 脚本，请把 #!/bin/sh 加到脚本的第一行，下面的做法不会成功 ExecStart=/bin/sh /path/to/script.sh。

.service 状态显示绿色的 "active (exited)" (例如 iptables)

如果单次执行的程序配置了 RemainAfterExit=yes，这是正常的状态。详情请参考 systemd.service(5) § OPTIONS。 systemd-user-sessions.services、nftables.service、iptables.service 等单次执行服务均是这样。

因为软链接已经存在，无法启用服务

启用单元时，可能出现如下错误：

```
Failed to enable unit: File /etc/systemd/system/symlink already exists and is a symlink to file.
```

此错误一般发生在启用指令要创建的/etc/systemd/system/软链已经存在的时候。典型的例子是在切换显示管理器的时候(例如切换 GDM 或 SDDM需要启用 gdm.service 或 sddm.service)，这时/etc/systemd/system/display-manager.service 已经存在。

要解决此问题，先禁用原来的显示管理器或者使用 -f/--force 选项强制覆盖原有链接，请参考 systemctl(1) § OPTIONS。

7.3 OpenRC

难度 进阶 | **篇幅** 8.9 KB | **所属** 初始化

本节将讲到

- 安装
- 配置
- 使用
- 提示和技巧
- 故障排除

翻译状态：

- 本文（或部分内容）译自 OpenRC，最近一次同步于 2026-02-26，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

警告：Arch Linux只为systemd提供官方支持。求助时请在帮助请求中提及使用OpenRC。

OpenRC是一个由Gentoo开发者维护的服务管理器。OpenRC是基于依赖关系的，通常使用系统提供的init程序SysVinit。

安装

关于init组件的细节，请查阅Init。

安装openrcAUR包。在0.25版本之后，OpenRC在`/usr/bin/openrc-init`提供它自己的init。

可选地，您可以使用其他init,例如busybox包。请注意，当使用`openrc-init`时，它必须和`openrc-shutdown`成对使用。并且不要使用来自其他包的`shutdown`和`reboot`命令，否则您将会遇到错误。

一组基本的服务文件可以从openrc-arch-services-gitAUR包获得。其他包可能有在此包之外提供的服务文件，建议在AUR上搜索。

为了保持和以前的初始化脚本的兼容性，配置文件已经在`/etc/openrc/`中安装。

启动

在kernel parameters中设置`init`选项来使用OpenRC启动。

要使用OpenRC内建的init，设置`init=/usr/bin/openrc-init`。

请注意，当使用`openrc-init`时，`/etc/inittab`文件将不会被使用。

配置

/etc/openrc/conf.d目录和/etc/openrc/rc.d文件被用于配置。

关于配置OpenRC的一般信息，请查看：

- OpenRC manuals
- Gentoo:OpenRC

关于从systemd迁移的说明，请查看Init#Configuration。

服务

通过root身份运行rc-update add *service_name* *runlevel*来启动OpenRC服务。建议启动以下服务：

服务名	Runlevel	描述
udev	sysinit	设备热插拔
alsa	default	ALSA state
acpid	default	ACPI事件
dbus	default	消息总线
dcron	default	定时任务
syslog-ng	default	系统日志

警告：如果在您的kernel parameters中使用init=/usr/bin/openrc-init，您将需要手动启用getty服务，否则您将无法使用如该文档中所述的交互式TTY

如果有需要，通过创建到/etc/openrc/init.d/getty的符号链接来为每个getty创建服务。例如为/dev/tty1：

```
# ln -s /etc/openrc/init.d/agetty{,.tty1}
# rc-update add agetty.tty1 default
```

为了防止PAM在登录后尝试向systemd注册（这可能会引发问题, 可以安全地删除或注释/etc/pam.d/system-auth中提到systemd的行。

另请参阅Gentoo:Systemd#Native services。

网络

网络通过newnet 配置。

编辑/etc/openrc/conf.d/network;ip、(iproute2包)和ifconfig、(net-tools包)命令都被支持。下面是一个使用ip的例子：

```
ip_eth0="192.168.1.2/24"
defaultiproute="via 192.168.1.1"
ifup_eth0="ip link set \$int mtu 1500"
```

网络服务被默认添加到引导运行级别(boot runlevel)，所以不需要采取进一步操作。关于一般网络信息，请参阅Network configuration。

<nowiki>

注意：您也可以通过启用相应的服务来使用 NetworkManager 或者 dhcpcd 。

启动日志

要启用启动日志记录，将`/etc/openrc/rc.conf`中`rc_logger="YES"`一行去注释。在启用时，启动日志存储在`/var/log/rc.log`。

主机名

OpenRC通过`/etc/openrc/conf.d/hostname`来修改主机名。它看起来像这样：

```
#设置这台电脑的主机名
hostname="myhostname"
```

内核模块

OpenRC使用`/etc/openrc/conf.d/modules`而不是`/etc/modules-load.d`。例如：

```
/etc/openrc/conf.d/modules
#您应该查阅内核文档和配置获取模块及其选项的列表

modules="vboxdrv acpi_cpufreq"
```

本地化

键盘布局可以通过`/etc/openrc/conf.d/keymaps`和`/etc/openrc/conf.d/consolefont`来配置。您也可以通过`/etc/locale.conf`文件，其来源于`/etc/profile.d/locale.sh`。

使用

本节对systemd以及其他init进行比较。

您可以忽略`.service`和`.target`扩展，尤其是在编辑kernel parameters时。

systemd	SysVinit	OpenRC	描述
`systemctl list-units`	`rc.d list`	`rc-status`	列出所有正在运行的服务的状态
`systemctl --failed`		`rc-status --crashed`	检查所有失败的服务
`systemctl --all`		`rc-update -v show`	显示所有可用服务
`systemctl (start, stop, restart, status) daemon.service`	`rc.d (start, stop, restart) daemon`	`rc-service daemon (start, stop, restart, status)`	改变服务状态
`systemctl (enable, disable) daemon.service`	`chkconfig daemon (on, off)`	`rc-update (add, del) daemon`	启用或停用服务
`systemctl daemon-reload`	`chkconfig daemon --add`		创建或修改配置

提示和技巧

静默启动

要隐藏OpenRC的启动信息，您可以编辑`/etc/inittab`，将`--quiet`添加到每个OpenRC命令。有关更多信息，请查看`$ openrc -h`。

故障排除

卸载/tmp时出错

当关机时，您可能会得到这样的信息：

```
* Unmounting /tmp ...
* in use but fuser finds nothing [ !! ]
```

可以通过向`/etc/openrc/conf.d/localmount`添加如下内容解决：

```
no_umounts="/tmp"
```

注意：只有将/tmp安装为tmpfs时才会出现此问题。

禁用IPv6无效

一种选择是向`/etc/openrc/sysctl.d`下一个以`.conf`结尾的文件添加：

```
# 禁用IPv6
net.ipv6.conf.all.disable_ipv6 = 1
```

在关机期间以只读模式重新挂载根目录失败

如果发生上述情况，请编辑`/etc/openrc/init.d/mount-ro`，找到下面一行：

```
# Flush all pending disk writes now
sync; sync
```

在其之后放入：

```
telinit u
```

找不到/etc/sysctl.conf

默认情况下，调用`sysctl --system`来加载sysctl配置。这包括从Arch中删除的`/etc/sysctl.conf`。

要放置丢失文件的错误，请创建文件：

```
# touch /etc/sysctl.conf
```

opentmpfiles-setup无法启动

在启动OpenRC时，您可能会看到如下行：

```
* Setting up tmpfiles.d entries ...
chattr: Operation not supported while setting flags on /var/log/journal
chattr: No such file or directory while trying to stat /var/log/journal/%m
chattr: Operation not supported while setting flags on /var/log/journal/remote
[ !! ]
ERROR: opentmpfiles-setup failed to start
```

这是由于/usr/lib/tmpfiles.d/journal-nocow.conf使用的选项，只有当journal位于btrfs上时才有效。

请查阅https://github.com/OpenRC/opentmpfiles/issues/2来获取详细信息。

一种解决方法是创建一个空的/etc/tmpfiles.d/journal-nocow.conf来覆盖这些设置。

恢复到systemd

在大多数情况下，恢复到systemd应该很简单。它本质上是迁移到OpenRC的逆向，注意以下几点：

- 删除或以其他方式编辑内核命令行上的init=参数。
- 用等价的包替换OpenRC定制的或no-systemd的包。

另请参阅

- Wikipedia:OpenRC
- Gentoo:OpenRC
- Forum thread about OpenRC in Arch
- Manjaro wiki

7.4 SysVinit

难度 进阶 | **篇幅** 10.0 KB | **所属** 初始化

本节将讲到

- 迁移到 Systemd
- init、inittab 概览
- 调整运行级别
- inittab
- 另见

**本文已被弃用。**

本文所述教程或工具由于已经过时而被弃用，请使用其他替代方案： Systemd (在Talk:SysVinit讨论)

init是Linux内核加载后执行的第一个进程。Arch的默认的 init 程序是systemd-sysvcompat包提供的/sbin/init(新安装的系统已经默认使用systemd) 或sysvinitAUR.。本文中，**init**指sysvinit。

inittab文件位于/etc目录，是init的启动配置文件，其中指定了一些启动脚本、程序的路径，并指定在哪些运行级别执行它们。

注意：Arch's initscripts has been officially unsupported since 2012-11-04.

迁移到 Systemd

从2012-10-13版安装介质开始，安装程序已经默认安装systemd包 和 systemd-sysvcompat包。本部分帮助还在使用sysvinitAUR 和 initscripts 的用户迁移到 **systemd**.

注意：如果是在 VPS 中使用 Arch，请先阅读：Arch Linux VPS。

- 阅读该站，了解 systemd。
- systemd 自己有一套日志（**journal**）系统，用于代替 **syslog**。
- 虽然 systemd 可以替换 **cron**、**acpid**、**xinetd** 等的部分功能。至少目前还可以继续使用这些服务，无需立即切换。
- 交互式 initscripts 启动脚本在 systemd 中无法工作。

安装

1. 从官方软件仓库安装 systemd包 并添加内核参数 `init=/usr/lib/systemd/systemd`
2. 使用 `systemctl enable` <服务名> 启用需要的服务（大致相当于以前 `DAEMONS` 数组的作用，新的服务名称参见 Daemons List）。
3. 重启系统，执行命令 `cat /proc/1/comm`，如果返回`systemd`，表示 systemd 已经正常启动。
4. 确认主机名已经正确设置：`hostnamectl set-hostname myhostname`。
5. 删除 initscripts 和 sysvinit，并安装systemd-sysvcompat包.
6. （可选）删除`init=/usr/lib/systemd/systemd`内核参数，现在已经不需要它了。systemd-sysvcompat包 软件包提供了一个软链接，使 systemd 成为默认 init。

附加信息

- 如果内核参数中有 `quiet`，建议在一开始先去掉，以便调试。
- 使用 systemd 的时候无需将用户加入特殊用户组（如`sys`、`disk`、`lp`、`network`、`video`、`audio`、`optical`、`storage`、`scanner`、`power`等等）。加入这些组反而会有问题，例如audio组会导致程序阻塞软件混声。每个 PAM 登录都拥有一个 logind 会话，它通过POSIX ACLs，赋予本地会话以声音设备访问权限、通过udisks挂载和卸载移动设备的权限等。
- 阅读网络配置，了解如何配置网络。

init、inittab 概览

init总是Linux的1号进程，并且是一切进程的父进程。通过`pstree`可以形象看出init在所有进程中所处的地位。

```
$ pstree -Ap
init(1)-+-acpid(3432)
        |-crond(3423)
        |-dbus-daemon(3469)
        |-gpm(3485)
        |-mylogin(3536)
        |-ngetty(3535)---login(3954)---zsh(4043)---pstree(4326)
        |-polkitd(4033)---{polkitd}(4035)
        |-syslog-ng(3413)---syslog-ng(3414)
        `-udevd(643)-+-udevd(3194)
                     `-udevd(3218)
```

除了系统初始化，init还负责重启、关机、单用户恢复模式。为了支持上述操作，inittab把条目分到不同的**运行级别**(runlevel)中去。Arch使用以下运行级别：0——关机，1（又叫S）——单用户模式，3——普通的多用户模式，5——X使用，6——重启。其他发行版可能有所不同，但0、1、6级别是通用的。

运行时，init检查inittab并进行适当的操作。inittab中的启动项目格式如下：

```
id:runlevels:action:process
```

`id`是项目独一无二的标识符（但只是个名称，对init没任何作用）；`runlevels`是一串无分隔字符串，设置运行级别；当init进入了指定的`runlevels`，执行`action`；如果顺利，执行`process`。某些特殊的`action`会忽略`runlevels`，使用特殊的匹配方法。下一节有更详细的介绍。

调整运行级别

通过启动加载器

想要改变系统启动时的运行级别，只需要添加想要的运行级别 `n` 到启动加载器的内核参数。这通常的应用是 #inittab。要启动到需要的运行级别，将号码加入内核参数 (例如 `3` 则启动要运行级别 3)。

运行级别追加到最后，这样内核就知道用哪个运行级别启动。想要使用另一个 init 程序(如 systemd)，添加`init=/usr/lib/systemd/systemd`或者类似的命令到内核行。

注意：如果你使用 sysvinit 之外的 init 程序，运行级别参数可能被忽略。

启动之后

系统启动后，可以调用`telinit n`通知init切换到运行级别`n`。然后init读取inittab，并做出当前运行级别到新的运行级别需要的改变——杀死新级别中没有的进程，执行旧

级别未执行过的操作。两个级别共有的进程此时都会保留不动。杀死进程的过程有些复杂，技术信息参见init的manpage。

init不会监视inittab的改动，需要执行`telinit`应用更改。`telinit q` 命令只应用inittab而不会修改运行级别。

inittab

这一部分将探究inittab中的常见项目，之后会给出几个inittab项目的实例。叙述顺序按照Arch默认的inittab。

警告：重启之前，务必使用 `telinit q` 测试修改过的 `/etc/inittab`，任何小小的语法错误都将导致系统无法启动。

默认运行级别

默认运行级别为3。如果想要设置默认运行级别为5（通常X使用的级别），添加下面一行内容：

```
id:5:initdefault:
```

主启动脚本

下面几行描述了主启动脚本：

```
rc::sysinit:/etc/rc.sysinit
rs:S1:wait:/etc/rc.single
rm:2345:wait:/etc/rc.multi
rh:06:wait:/etc/rc.shutdown
```

单用户启动

有时，因为重要文件丢失、文件系统损坏或硬件问题，内核可能启动失败。此时init可能自动进入**单用户模式**，此模式只允许使用root登录，使用/sbin/sulogin（而非/sbin/login）控制login进程。也可以在 GRUB, LILO 或 syslinux 启动项添加S参数进入单用户模式。如果不想使用sulogin，可以在这里设置：

```
su:S:wait:/sbin/sulogin -p
```

终端初始化

该部分是初始化虚拟终端的关键。默认设置会在tty1-6开启6个getty，显示终端登录提示。另见：openvt， chvt，stty，ioctl。

```
c1:234:respawn:/sbin/agetty 9600 tty1 xterm-color
c5:5:respawn:/sbin/agetty 57600 tty2 xterm-256color
```

Ctrl-Alt-Del

以下内容定义按下`Ctrl+Alt+Del`组合键时进行的操作：

```
ca::ctrlaltdel:/sbin/shutdown -t3 -r now
```

X 程序

如果不怕麻烦，在inittab中启动各种程序都是可以的。以下内容示范了如何设置系统进入运行级别5时启动登录管理器 SLiM ：

```
x:5:respawn:/usr/bin/slim >/dev/null 2>&1
#x:5:respawn:/usr/bin/xdm -nodaemon -confi /etc/X11/xdm/archlinux/xdm-config
```

电源检测脚本

init可以根据UPS设备状态执行相应进程，示例如下：

```
pf::powerfail:/sbin/shutdown -f -h +2 "Power Failure; System Shutting Down"
pr:12345:powerokwait:/sbin/shutdown -c "Power Restored; Shutdown Cancelled"
```

自定义键盘请求

类似`Ctrl+Alt+Del`，下面的内容添加了按下特定组合键时执行命令的功能：

```
kb::kbrequest:/usr/bin/wall "Keyboard Request -- edit /etc/inittab to customize"
```

触发 kbrequest

使用root用户，可以通过向init发送WINCH信号触发**kbrequest**。对于上述例子，命令：

```
kill -WINCH 1
```

会导致`wall`命令执行，向所有用户发送信息：

```
Broadcast message from root@askapachehost (console) (Wed Oct 27 14:02:26 2010):
Keyboard Request -- edit /etc/inittab to customize
```

另见

- Automatic login to virtual console
- Disable clearing of boot messages
- 开机启动 X
- Xinitrc
- 登录管理器
- SLiM

外部链接

- 维基百科：init
- Linux Knowledge Base and Tutorial：运行级别
- Linux.com：runlevels 和 inittab 的介绍
- Linux.com：服务、运行级别、rc.d脚本的介绍

7.5 Init

难度 进阶 | **篇幅** 10.4 KB | **所属** 初始化

本节将讲到

- 整合式 init
- init
- init 脚本
- 服务管理器
- 配置

警告：Arch Linux 只提供对 systemd 的官方支持。如使用其他 init 系统，在请求支持时请注明。

Init 是系统启动时创建的第一个进程。它是一个守护进程，会一直运行到系统关闭。init 是其他所有进程的直接或间接祖先，并自动监护所有孤儿进程。内核按照硬编码的文件名启动它，如果内核不能启动它，将会导致内核崩溃。init 的进程标识符（PID）通常是 1。

在系统启动和关闭时，init 进程会启动 init 脚本（或称 rc）来保障基本功能。这包括挂载和卸载文件系统，以及启动守护进程。进一步，有一个服务管理器提供对已启动进程的主动控制，称为进程监控。例如监测崩溃的进程并适时重启。

这些元素加起来就成了 init **系统**。某些 init 将服务管理器包含在 init 进程中，或是有紧密联系的 init 脚本。在下面，这类 init 将被称为整合式的。其他的分类下的条目可能会相互依赖。

整合式 init

- **anopa** — 围绕 s6 监视套件构建的 init 系统。

: https://jjacky.com/anopa/ || anopaAUR

- **GNU Shepherd** — 用 Guile 编写的 init 系统。

: https://www.gnu.org/software/shepherd/ || shepherdAUR

- **OpenRC** — 基于依赖的 init 系统。

: https://www.gentoo.org/proj/en/base/openrc/ || openrcAUR openrc-arch-services-gitAUR

- **systemd** — 基于依赖的 init 系统，具备激进的并行化，使用 cgroups 提供进程监护，及依赖于给定挂载点或 dbus 服务的能力。

: https://freedesktop.org/wiki/Software/systemd/ || systemd包

init

- **BusyBox** — 用于救援和嵌入式系统的工具。

: https://busybox.net/ || busybox包

- **sinit** — 基于 Rich Felker 所作最简 init 的简单init。

: https://core.suckless.org/sinit || sinitAUR

- **SysVinit** — 传统的 system V init。

: https://savannah.nongnu.org/projects/sysvinit || sysvinitAUR

init 脚本

- **initscripts-fork** — 另行维护的 Arch Linux 的 SysVinit 脚本分支。

: https://bitbucket.org/TZ86/initscripts-fork/overview || initscripts-forkAUR

- **minirc** — 为 BusyBox 设计的最简 init 脚本。

: https://github.com/hut/minirc/ || minirc-gitAUR

- **kisslinux-init** — KISS Linux 的 init 框架。

: https://github.com/kisslinux/init || kisslinux-initAUR

服务管理器

- **monit** — monit是 Unix 和 Linux 系统的进程管理工具。monit 支持直接从命令行或是原生的 HTTP(S) 网络服务器查看系统状态。

: https://mmonit.com/monit/ || monit包

- **perp** — 适用于 UNIX 的持久进程（服务）监管器和管理框架。

: http://b0llix.net/perp/ || perpAUR

- **runit** — UNIX init 框架，用于替代 SysVinit 和其他 init 框架。

: http://smarden.org/runit/ || busybox包

- **s6** — 一小组 UNIX 程序, 设计来允许代替 daemontools 和 runit 的服务监管。

: https://skarnet.org/software/s6/ || s6AUR

- **Supervisor** — 一个允许用户在类 UNIX 系统上监控进程的系统。

: https://supervisord.org/ || supervisor包

配置

迁移正在运行的服务

为了在新 init 下运行守护进程，首先要保存正在运行的守护进程清单：

```
$ systemctl list-units --state=running "*.service" > daemons.list
```

然后相应配置 #init 脚本。参看。

注意：systemd-tmpfiles(8)、内核模块和 sysctl可能也需要配置。

logind

logind 要求 init 进程是 systemd。因此，本地会话和其他功能不可用。

提示：有一个单独版本的 logind。elogind-gitAUR

设备权限

将用户添加到相应的用户组以允许设备访问和重启。应当先用 `id *user*` 检查当前所属的用户组。 以下命令允许用户访问大多数设备：

```
# usermod -a -G video,audio,power,disk,storage,optical,lp,scanner,input user
```

另可参见 systemd 之前的群组。

要创建 polkit 使用的组规则，参见跳过口令提示。

没有 root 权限的 X

因为 `Xorg.wrap` 不检查 logind 是否活跃，Xorg 的根权限需要手动启用：

```
/etc/X11/Xwrapper.config
needs_root_rights = yes
```

电源管理

参见 pm-utilsAUR 和 acpid 以替换用 systemd 进行的电源管理。

定时任务

Arch 默认使用 timer 而非 cron。可在 archlinux-cronjobs 参看基本的 cron 任务。

D-Bus

dbus-daemon 的用户实例由 systemd 用户服务启动。在需要桌面应用间通讯时，恢复`30-dbus.sh`：

```
/etc/X11/xinit/xinitrc.d/30-dbus.sh
#!/bin/bash

# launches a session dbus instance
if [ -z "${DBUS_SESSION_BUS_ADDRESS-}" ] && type dbus-launch >/dev/null; then
  eval $(dbus-launch --sh-syntax --exit-with-session)
fi
```

提示与技巧

systemd-nspawn

systemd-nspawn 是 systemd 系统的工具。从 Linux 2.6.19 起，可以在非 systemd 系统上用 PID 名称空间运行 systemd。这需要内核配置了 `CONFIG_PID_NS` 和 `CONFIG_NAMESPACES`。

PID 名称空间创建一个新的进程架构，从 PID 1 开始。另外，systemd 需要一个已经 chroot 的文件系统才能挂载。因此，你必须至少做一个绑定挂载，否则某些服务可能会报这样的错：

```
"Failed at step NAMESPACE spawning" due to "Invalid operation"
```

这是因为 systemd 尝试以 `private` 选项重新挂载根目录。

可以用 jchroot 设置一个有新 PID 名称空间的 chroot。

确保不要在 chroot 前挂载新根目录的 `/proc`，否则 systemd 会检测到 chroot 环境。在 systemd 运行以后可以挂载。

替换 udev

警告：替换 udev 并非必要，因为即使 systemd 不是 PID 1，systemd-udev 也能工作。某些替换比如 eudev 无法与 systemd包 共存，请确保在安装之前启动了另外的 init。

- **eudev** — eudev 是 Gentoo 项目开发的 udev 分支项目。需要与 OpenRC 配合设计和测试。

: Gentoo:eudev || eudevAUR

- **mdev** — 用于嵌入式系统的设备管理器。

: https://git.busybox.net/busybox/plain/docs/mdev.txt || busybox包

- **smdev** — smdev 是一个简单的管理设备节点的程序。其基本与 mdev 兼容，但没有后者的全部特性。

: https://git.suckless.org/smdev/ || smdevAUR

参见

7.6 Systemd

难度 高级 | **篇幅** 47.1 KB | **所属** 初始化

本节将讲到

- systemctl 基本用法
- 编写单元文件
- 目标（target）
- systemd组件
- 小技巧

摘自项目主页：

: *systemd* 是一个 Linux 系统基础组件的集合，提供了一个系统和服务管理器，运行为 PID 1 并负责启动其它程序。功能包括：支持并行化任务；同时采用 socket 式与 D-Bus 总线式启用服务；按需启动守护进程（daemon）；利用 Linux 的 cgroups 监视进程；支持快照和系统恢复；维护挂载点和自动挂载点；各服务间基于依赖关系进行精密控制。*systemd* 支持 SysV 和 LSB 初始脚本，可以替代 sysvinit。除此之外，功能还包括日志进程、控制基础系统配置，维护登录用户列表以及系统账户、运行时目录和设置，可以运行容器和虚拟机，可以简单的管理网络配置、网络时间同步、日志转发和名称解析等。

在历史上，systemd中的“服务”（service）被称作守护进程（daemon），它们在后台运行（即没有UI、不与终端交互），等待特定事件的发生并提供服务。例如，Web服务器会等待一个请求以提供相应的页面，SSH服务器会等待登录请求。除了这种提供完整功能的，还有一些守护进程的工作是隐藏在幕后的，如负责向日志文件写入消息的`syslog`、`metalog`，确保系统时间准确的ntpd。更多信息详见daemon(7)。

注意：Arch Linux 论坛的这篇帖子 详细地解释了 Arch Linux 迁移到 systemd 的原因。

systemctl 基本用法

监视和控制 *systemd* 的主要命令是 *systemctl*。其用途包括查看系统状态以及管理系统和服务。详见 systemctl(1)。

提示：

- 在 *systemctl* 参数中添加 `-H *用户名*@*主机名*` 可以远程控制其他机器。该功能使用 SSH 连接到远程的 *systemd* 实例。
- Plasma 用户可以安装 *systemctl* 图形前端 systemdgenie包。安装后可以在*系统管理*下找到。

使用单元

单元（unit）通常包括但不限于：服务（*.service*）、挂载点（*.mount*）、设备（*.device*）和套接字（*.socket*）。

使用 *systemctl* 时，通常需要使用单元文件的全名，包括扩展名（例如 `sshd.socket`）。不过在以下 *systemctl* 命令中可以使用简写：

- 如果无扩展名，systemctl 会假定扩展名为 *.service*。例如，`netctl` 和 `netctl.service` 是等价的。
- 挂载点会自动转化为相应的 *.mount* 单元。例如，`/home` 等价于 `home.mount`。
- 与挂载点类似，设备会自动转化为相应的 *.device* 单元，因此 `/dev/sda2` 等价于 `dev-sda2.device`。

详见 systemd.unit(5)。

注意：有一些单元的名称包含一个 `@` 符号（例如：`name@*string*.service`），这意味着它是模板单元 `name@.service` 的一个 实例，模板单元的实际文件名中不包括 `*string*` 部分（如 `name@.service`）。`*string*` 被称作实例标识符，在 *systemctl* 调用模板单元时，会将其当作一个参数传给模板单元，模板单元会使用这个传入的参数代替模板中的 `%i` 指示符。在启动单元时，尝试从模板单元实例化之前，*systemd* 会先检查 `name@string.suffix` 文件是否存在。如果存在，就直接使用这个文件，而不是模板实例化（不过，这种"碰撞"非常少见）。大多数情况下，包含 `@` 标记都意味着这个文件是模板。如果一个模板单元被调用时没有指定实例标识符，该调用通常会失败，除非是在某些特殊的 *systemctl*命令（如`cat`）中使用。

因为调用*systemctl*时默认了 `--system` 参数，下列命令默认对**系统单元**进行操作。若要对（**调用用户**的）**用户单元**进行操作，则需在非root身份下执行systemctl --user。参看systemd/用户#基础设置以为**所有用户**启用或禁用单元。

提示：

- 下面的大部分命令都可以跟多个单元名，详细信息参见 systemctl(1)。
- `--now` 选项可与 `enable`、`disable` 和 `mask` 同时使用，可使这些动作**立即**生效，而非重启后生效。
- 一个软件包可能会为不同的功能提供多个不同的单元。如果你刚安装了软件包，可以通过 `pacman -Qql *package* | grep -Fe .service -e .socket` 命令检查这个软件包提供了哪些单元。

动作	**命令**	**注释**
分析系统状态		
显示系统状态	`systemctl status`	
列出正在运行的单元	`systemctl` 或 `systemctl list-units`	
列出失败的单元	`systemctl --failed`	
列出已安装的单元1	`systemctl list-unit-files`	
显示特定PID对应进程的状态	`systemctl status *pid*`	cgroup slice，内存占用，父进程
检查单元状态		
显示单元的**手册页**	`systemctl help *单元*`	如果单元支持
显示单元的**状态**	`systemctl status *单元*`	包括其是否在运行
检查单元是否配置为自动启动（enabled）	`systemctl is-enabled *单元*`	
启动（start）、重新启动和重新加载单元		
立即**启动**单元	以root身份执行`systemctl start *单元*`	
立即**停止**单元	以root身份执行`systemctl stop *单元*`	
重新启动单元	以root身份执行`systemctl restart *单元*`	
重新加载单元及其配置	以root身份执行`systemctl reload *单元*`	
重新加载 systemd 配置2		扫描单元的变动

动作	命令	注释
	以root身份执行`systemctl daemon-reload`	
启用（enable）单元		
启用单元：开机时自动**启动**该单元	以root身份执行`systemctl enable *单元*`	
启用单元，并立即**启动**它	以root身份执行`systemctl enable --now *单元*`	
取消开机自动启动单元	以root身份执行`systemctl disable *单元*`	
重新启用单元3	以root身份执行`systemctl reenable *单元*`	先取消启用，再启用
屏蔽单元		
屏蔽单元，使其无法启动4	以root身份执行`systemctl mask *单元*`	
取消屏蔽单元	以root身份执行`systemctl unmask *单元*`	

1. systemd.unit(5) § UNIT FILE LOAD PATH 中描述了查找单元文件的路径。
2. 这不会要求已改变的单元重新加载自己的配置（见**重新加载**动作）。
3. 可在单元的`[Install]`节发生变更（在上一次启用后）时使用。
4. 既不能手动启动，也无法作为依赖启动。因此屏蔽单元是危险的。 查看已屏蔽的单元：

```
# systemctl list-unit-files --state=masked
```

电源管理

安装 polkit 后才能以普通用户身份使用电源管理。如果你正登录在一个本地的 *systemd-logind* 用户会话，且当前没有其它活动的会话，那么以下命令无需 root 权限即可执行。否则（例如，当前有另一个用户登录在某个 tty），*systemd* 将会自动请求输入 root 密码。

动作	命令
重启	`systemctl reboot`
关机	`systemctl poweroff`
待机	`systemctl suspend`
休眠（将内存写入硬盘）	`systemctl hibernate`
进入混合休眠模式（同时休眠到硬盘并待机）	`systemctl hybrid-sleep`
先待机，在一个特定时间唤醒后进入休眠模式	`systemctl suspend-then-hibernate`
使用#软重启来仅重启用户空间	`systemctl soft-reboot`

软重启

软重启是一种不涉及内核，仅重启用户空间的重启操作。这个操作由 systemd-soft-reboot.service(8) 实现，并且可以通过 `systemctl soft-reboot` 来进行一次软重启。正如 kexec 一样，这个操作跳过了重新初始化固件，也不会进行内核初始化和初始化 RAM 文件系统。同时，未锁定的 dm-crypt 设备也将保持解锁状态。

当 `/run/nextroot` 包含了一个合法的 rootfs 结构（比如说挂载了另一个发行版的根分区或者另一个快照），*软重启操作*将会切换系统根分区到其中。切换之后并不会丢失一些由内核管理的状态，比如说网络连接。

提示：`/run/nextroot/`不一定是一个挂载点或者存在于物理设备上。例如，它可以在 `/run/` tmpfs 中。*systemd* 将会在软重启时自动将`/run/nextroot/` 转化为一个挂载点。

注意：不要在涉及内核和 initramfs 的更新后执行 `systemctl soft-reboot`。

编写单元文件

`systemd` 单元文件的语法来源于XDG 桌面项配置的.desktop文件，最初的源头则是Microsoft Windows的.ini文件。单元文件可以从多个地方加载，`systemctl show --property=UnitPath` 可以显示加载目录。主要的加载目录为（按优先级从低到高排列）：

- `/usr/lib/systemd/system/`：软件包安装的单元
- `/etc/systemd/system/`：系统管理员安装的单元

注意：

- 当 `systemd` 运行在用户模式下时，使用的加载路径是完全不同的。
- systemd 单元名仅能包含 ASCII 字符，下划线和点号以及有特殊意义的字符('@', '-')。其它字符需要用C风格的"\\x2d" 替换或使用对应的预定义语法（“@”，“-”）。参阅 systemd.unit(5) 和 systemd-escape(1) 。

单元文件的语法，可以参考系统已经安装的单元，也可以参考 systemd.service(5) § EXAMPLES。

提示：在单元文件中以#开头的行将被视作注释，但#仅能在一行开头使用。不要在 *systemd* 的参数后面使用行末注释，否则该单元将不能正常启动。

systemd-analyze(1) 可以帮助你检查错误。请参见那一章节的 `systemd-analyze verify FILE...` 部分。

处理依赖关系

使用 systemd 时，可通过正确编写单元配置文件来解决单元间的依赖关系。典型的情况是，**单元A**要求**单元B**在**A**启动之前运行。在此情况下，可向**单元A**配置文件中的 `[Unit]` 段添加 `Requires=*B*` 和 `After=*B*` 即可。若此依赖关系是可选的，可添加 `Wants=*B*` 和 `After=*B*`。请注意 `Wants=` 和 `Requires=` 并不暗含 `After=`，即如果 `After=`选项没有指定，这两个单元将被并行启动。

依赖关系通常被用在服务（service）而不是目标（target）上。例如，`network.target` 一般会被某个配置网络接口的服务引入作为依赖。所以，若要让自定义单元在系统到达`network.target`后再启动， 将自定义的单元排在该类配置网络接口的服务之后即可，因为该类服务启动时 `network.target` 一定已经启动。

服务（service）类型

编写自定义的服务（service）文件时，可以选择几种不同的服务启动方式。启动方式可通过配置文件 `[Service]` 段中的 `Type=` 参数进行设置。

- `Type=simple`：（默认值） systemd认为该服务将立即启动且服务进程不会fork。如果其他服务需要在此服务之后启动，请不要使用此类型，除非它是通过套接字激活的。
- `Type=forking`：systemd认为当该服务进程fork，且父进程退出后服务启动成功。对于典型的守护进程，除非你确定此启动方式无法满足需求，使用此类型启动即可。使用此启动类型应同时指定 `PIDFile=`，以便systemd能够跟踪服务的主进程。
- `Type=oneshot`：这一选项适用于只执行一项任务、随后立即退出的服务。可能需要同时设置 `RemainAfterExit=yes`使得systemd在服务进程退出之后仍然认为服务处于启用状态，这对于改变系统状态（如挂载分区）的单元尤其适用。对于 simple 和 oneshot 类型的区别，请参阅 。
- `Type=notify`：与 `Type=simple` 相同，但约定服务会在就绪后向systemd发送一个信号以通知systemd。这一通知的参考实现由 `libsystemd-daemon.so` 提供。
- `Type=dbus`：若以此方式启动，当指定的 `BusName` 出现在DBus系统总线上时，systemd认为服务就绪。
- `Type=idle`：`systemd`会等待所有活动任务都完成后再执行服务进程。其他行为与 `Type=simple` 类似。

`type` 的更多解释可以参考 systemd.service(5) § OPTIONS。

修改现存单元文件

为了避免和 pacman 冲突，不应该直接编辑软件包提供的文件。有两种方法可以不改动原始文件就做到修改单元文件：创建一个优先级更高的本地单元文件以覆盖原有的单元文件或创建一个片段（drop-in snippets），应用到原始单元文件之上。两种方法都需要在修改后重新加载单元：用 `systemctl edit` 编辑单元(会自动重载单元)或通过下面命令重新加载单元：

```
# systemctl daemon-reload
```

提示：

- `systemd-delta` 命令用来查看哪些单元文件被覆盖、扩增，哪些被修改。
- 使用 `systemctl cat *unit*` 可以查看单元的内容和所有相关的片段。

替换单元文件

要替换 `/usr/lib/systemd/system/*unit*`, 创建文件 `/etc/systemd/system/*unit*` 并重新启用单元以更新符号链接：

```
# systemctl reenable unit
```

或者运行：

```
# systemctl edit --full unit
```

这将会在默认编辑器中打开 `/etc/systemd/system/*unit*`。如果文件不存在，可以将软件包安装的版本复制到这里，在编辑完成之后，会自动加载新版本。

注意：即使 Pacman 更新了新的单元文件，软件包中的版本也不会被使用，所以这个方式会增加系统维护的难度，推荐使用下面一种方法。

附加配置片段

要为单元文件`/usr/lib/systemd/system/*unit*`附加配置片段，先创建名为 `/etc/systemd/system/*unit*.d/` 的目录，然后放入 `*.conf` 文件，其中可以添加或覆盖参数。这里设置的参数优先级高于原来的单元文件。*systemd*会解析这些参数并将这些文件应用到原单元文件上。

要附加配置片段，最简单的方法是执行：

```
# systemctl edit unit --drop-in=drop_in_name
```

这将会在编辑器中打开文件 `/etc/systemd/system/*unit*.d/*drop_in_name*.conf`，编辑完成之后自动加载。若省略`--drop-in=`选项，*systemd*将使用默认文件名：`override.conf`。

注意：

- 附加配置片段中的键（配置项）仍必须置于相应的节（section）下。
- 并不是所有参数都会被子配置文件覆盖。例如要修改 `Conflicts=` 就必须替换原始文件。
- 你可以使用顶级附加配置文件来影响所有相同类型的单元。例如，在 `/etc/systemd/system/service.d/` 中的附加配置文件会影响所有 *.service* 单元。你可以在#Notifying_about_failed_services中看到一个示例。

重置到软件包的版本

要回退使用`systemctl edit`对单元进行的所有变更，执行:

```
# systemctl revert unit
```

示例

如果想添加一个额外的依赖，创建如下文件即可：

```
/etc/systemd/system/<unit>.d/customdependency.conf
[Unit]
Requires=<新依赖>
After=<新依赖>
```

要修改一个单元的 `ExecStart` 命令，创建下面文件:

```
/etc/systemd/system/unit.d/customexec.conf
[Service]
ExecStart=
ExecStart=<新命令>
```

修改 `ExecStart` 前必须将其置空，参见 。所有可能多次赋值的变量都需要这个操作，例如定时器的 `OnCalendar` 。

下面是自动重启服务的一个例子：

```
/etc/systemd/system/unit.d/restart.conf
[Service]
Restart=always
RestartSec=30
```

服务日志级别

对于直接向 journald 或 syslog 发送日志的服务，你可以通过在 `[Service]` 部分设置 `LogLevelMax=` 参数来控制其详细程度，该参数的值可以是 0 到 6 之间的数字。例如：

```
/etc/systemd/system/unit.d/override.conf
[Service]
LogLevelMax=3
```

标准的日志级别与用于过滤 journal 的级别相同。设置较低的数字会排除所有较高且不太重要的日志消息。

抑制服务的标准输出

如果服务正在回显 stdout 和/或 stderr 输出，默认情况下这些输出也会进入 journal。可以通过在 `[Service]` 部分设置 `StandardOutput=null` 和/或 `StandardError=null` 来抑制此行为。除了 `null` 之外，还可以使用其他值来进一步调整此行为。参见 systemd.exec(5) § LOGGING_AND_STANDARD_INPUT/OUTPUT。

目标（target）

*systemd*使用**目标**来通过依赖关系将多个单元组织起来。**目标**还是系统的标准化同步点（确保系统处于特定状态）。**目标**与runlevels的设计目的相似，但两者也有些许不同。每一个**目标**都以名称而不是数字标识，用以达成特定的目的。多个**目标**可以同时

被激活。有的**目标**继承另一个**目标**的所有服务，同时向其中增添一些服务。有些*systemd***目标**模仿了常见的SystemVinit runlevels。

获取当前目标

使用*systemd*的命令而非`runlevel`：

```
$ systemctl list-units --type=target
```

创建自定义目标

在sysvinit中有明确定义的运行级别（如：0、1、3、5、6）与 *systemd* 中特定的**目标**存在一一对应的关系。然而，对于用户自定义运行级别（2、4）却没有。如需要同样功能，建议你以原有运行级别所对应的 systemd 目标为基础，新建一个`/etc/systemd/system/<目标名>.target`（可参考`/usr/lib/systemd/system/graphical.target`），然后创建目录`/etc/systemd/system/<目标名>.wants`，并向其中加入需启用的服务链接（指向`/usr/lib/systemd/system/`中的对应文件）。

“SysV运行级别”与“systemd目标”对照表

SysV 运行级别	Systemd 目标	注释
0	poweroff.target	关闭系统（halt）
1, s, single	rescue.target	单用户模式
2, 4	multi-user.target	用户自定义运行级别，默认与级别3相同。
3	multi-user.target	多用户，无图形界面。用户可以通过终端或网络登录。
5	graphical.target	多用户，图形界面。继承级别3的服务，并启动图形界面服务。
6	reboot.target	重启
emergency	emergency.target	急救模式（Emergency shell）

切换当前运行目标

systemd中，目标通过“目标单元”访问。通过如下命令切换：

```
# systemctl isolate graphical.target
```

该命令仅更改当前运行目标，对下次启动无影响。这等价于sysvinit中的`telinit 3`或`telinit 5`命令。

更改开机默认启动目标

开机启动的目标是`default.target`，默认符号链接到`graphical.target`（大致相当于原来的运行级别5）。

用 *systemctl* 检查当前的默认启动目标：

```
# systemctl get-default
```

用*systemctl*修改`default.target`符号链接以变更开机默认启动目标：

```
# systemctl set-default multi-user.target
Removed /etc/systemd/system/default.target.
Created symlink /etc/systemd/system/default.target -> /usr/lib/systemd/system/multi-user.target.
```

另一方法是向bootloader添加内核参数：

- `systemd.unit=multi-user.target` （大致相当于运行级别3）
- `systemd.unit=rescue.target` （大致相当于运行级别1）

默认目标顺序

Systemd 根据下面顺序选择 `default.target`：

1. 上面的内核参数
2. `/etc/systemd/system/default.target` 软链接
3. `/usr/lib/systemd/system/default.target` 软链接

systemd组件

*systemd*的部分组件如下：

- kernel-install——一个用以自动将内核及对应的initramfs镜像移动到boot分区的脚本；
- systemd-analyze(1)——可用于确定启动性能、统计信息并检索其他状态和跟踪信息，以及验证单元文件的正确性。它还用于访问高级调试所需的特殊功能。
- systemd-boot——简单的UEFI启动管理器；
- systemd-creds——安全地存储并获取systemd单元使用的credentials；
- systemd-cryptenroll——Enroll PKCS#11, FIDO2, TPM2 token/devices to LUKS2 encrypted volumes；
- systemd-firstboot——负责在系统第一次启动前的设置初始化；
- systemd-homed——portable human-user accounts；
- systemd-logind(8)——会话管理；
- systemd-networkd——网络配置管理；
- systemd-nspawn——轻量namespace容器；
- systemd-repart——创建分区表，添加或扩大分区；
- systemd-resolved——网络域名解析；
- systemd-run(1) / run0(1)——以交互式的方式临时的提升或获得不同的权限。
- systemd-stub(7)——用于创建统一内核镜像的UEFI boot stub；
- systemd-sysusers(8)——在安装软件包或系统启动时创建系统用户和组、将用户加入组；

- systemd-timesyncd——通过网络同步系统时间；
- systemd/Journal——系统日志；
- systemd/Timers——用于控制*.service*文件或事件的间隔或实时计时器，可替代cron。

systemd.mount - 挂载

*systemd*负责挂载`/etc/fstab`中指定的分区和文件系统。systemd-fstab-generator(8)将`/etc/fstab`中的所有条目翻译为*systemd*单元。该过程将在系统启动或系统管理器的配置被重新加载时执行。

*systemd*扩展了通常fstab的用法并提供了更多挂载选项。这些挂载选项可影响挂载点单元的依赖关系。例如，它们可以确保某一挂载操作只会在连接网络或另一分区挂载后进行。*systemd*的特定挂载选项（大多以`x-systemd.`开头）的完整列表见systemd.mount(5) § FSTAB。

*automounting*是这些挂载选项的一个例子。它意味着资源只会在请求时挂载，而非在系统启动时自动挂载，详见fstab#通过 systemd 自动挂载。

GPT分区自动挂载

在UEFI启动的系统上，GPT分区如`root`、`home`、`swap`等可以根据Discoverable Partitions Specification自动挂载。因此，可在fstab中省略自动挂载的分区。此外，若根分区已被自动挂载，可省去内核命令行中的`root=`。详见systemd-gpt-auto-generator(8)。

使用GPT分区自动挂载的先决条件有：

- 使用mkinitcpio时，需要systemd hook。
- 所有自动挂载的分区必须与ESP分区在同一块物理硬盘上。
- 必须设置正确的GPT分区类型。详见Partitioning#Partition scheme。
- 引导加载程序必须设置LoaderDevicePartUUIDEFI变量，以便识别使用的EFI系统分区。systemd-boot，systemd-stub(7)，GRUB (使用*grub-mkconfig*生成`grub.cfg`)，rEFInd（默认未启用）支持该特性。若使用自定义的`grub.cfg`，则需要加载bli模块。 可通过执行`bootctl`并检查`Boot loader sets ESP information`的状态进行确认，或在用统一内核镜像启动时检查`Stub sets ESP information`的状态。

*udev*会创建一个指向根卷块设备的`/dev/gpt-auto-root`符号链接。如果根分区使用LUKS加密，`/dev/gpt-auto-root`将指向解锁/映射的卷，而`/dev/gpt-auto-root-luks`将指向加密分区。

提示：某个分区的自动挂载可通过如下两种方式关闭：修改分区的类型GUID或设置分区属性的63"do not automount"位，详见gdisk#Prevent GPT partition automounting。

/var

要自动挂载`/var`分区，分区的PARTUUID必须与以machine ID为密钥，分区类型UUID为消息的SHA256 HMAC哈希计算结果相同。可通过如下命令得到符合要求的PARTUUID：

```
$ systemd-id128 -u var-partition-uuid
```

注意：systemd-id128(1)从`/etc/machine-id`读取machine ID，因此必须在系统安装后才能得到符合要求的PARTUUID。

systemd-sysvcompat

systemd-sysvcompat包（由base包需要）的主要工作是提供传统的Linux init可执行文件。在由systemd控制的系统上，`init`只是一个到`systemd`可执行文件的符号链接。

此外，该包还提供了SysVinit用户可能需要的功能的4个快捷方式：halt(8), poweroff(8), reboot(8)和shutdown(8)。这4个命令都是到`systemctl`的符号链接，其行为受*systemd*控制。因此，在#电源管理中提到的规则对它们同样适用。

在由systemd控制的系统上，可通过`init=`内核启动参数取消对System V的兼容性支持。详见/bin/init is in systemd-sysvcompat ?。

systemd-tmpfiles - 临时文件

*systemd-tmpfiles*创建，删除并清理临时文件/文件夹。*systemd-tmpfiles*读取`/etc/tmpfiles.d/`、`/usr/lib/tmpfiles.d/`中的配置文件（前者比后者优先级高）以确定执行什么操作。

这些配置文件通常与服务文件一同提供，并以`/usr/lib/tmpfiles.d/<程序>.conf`风格命名。例如，Samba守护进程需要`/run/samba`目录存在且权限设置正确，因此samba包中附带了如下配置：

```
/usr/lib/tmpfiles.d/samba.conf
D /run/samba 0755 root root
```

配置文件也可能用于在启动时向特定文件中写入值。例如，要禁止系统从USB设备唤醒。在之前可使用`/etc/rc.local`在启动时执行`echo USBE > /proc/acpi/wakeup`，而现在可以这么做：

```
/etc/tmpfiles.d/disable-usb-wake.conf
#    Path                 Mode UID  GID  Age Argument
```

```
w    /proc/acpi/wakeup      -    -    -    -   USBE
```

可以通过在参数中使用 `\n` 或者在多行配置的情况下用 `w+` 类型来进行*追加*（包括第一行）。

```
/etc/tmpfiles.d/disable-usb-wake.conf
#    Path                   Mode UID  GID  Age Argument
w+   /proc/acpi/wakeup      -    -    -    -   USBE
w+   /proc/acpi/wakeup      -    -    -    -   LID0
```

详见systemd-tmpfiles(8)、tmpfiles.d(5)。

注意：若要向`/sys`中的配置文件写入值，可能不能使用该方法。因为*systemd-tmpfiles-setup*服务可能在相关模块加载前运行。在这种情况下，可先通过`modinfo <模块名>`检查该模块是否可通过设定模块参数来修改相应的选项。若有对应的模块参数，可通过/etc/modprobe.d目录中的配置文件设定相应参数以达到修改选项的目的。否则，将只能通过udev 规则来在设备被识别时设定相应属性。

Drop-in 配置文件

**本文或本章节的事实准确性存在争议。**

原因： 本页面是关于PID 1（init）的，单元文件的drop-in配置已经提到。Drop-in是一个通用概念，加上其他systemd组件有专门的wiki页面。因此本节似乎不属于这里。（在 Talk:Systemd#YHNdnzj : Configuration files in conf.d / drop-in snippets: misplaced? 中讨论）

为了避免与pacman更新冲突，不应直接编辑由软件包提供的配置文件。为此，许多（但不是所有）systemd软件包提供了一种修改配置的方式，但不直接修改原始文件，而是通过创建drop-in片段来实现。请检查软件包手册以查看是否支持drop-in配置文件。

要为单元文件`/etc/systemd/*unit*.conf`创建drop-in配置文件，请创建目录`/etc/systemd/*unit*.conf.d/`并在其中放置*.conf*文件以覆盖或添加新选项。*systemd*将解析并应用这些文件到原始单元之上。

检查整体配置：

```
$ systemd-analyze cat-config systemd/unit.conf
```

应用的drop-in片段文件及其内容将列在最后。重启服务以使更改生效。

小技巧

激活套接字

有一些包提供了一个 *.socket* 单元。例如，cups包 提供了一个 `cups.socket` 单元。如果 `cups.socket` 被启用（并且 `cups.service` 保持关闭），*systemd* 并不会立刻启动 CUPS，它会监听适当的套接字并且当任何一个程序尝试连接 CUPS 服务中的一个时，*systemd* 会启动 `cups.service` 并且透明地将这些端口的控制权移交给 CUPS 进程。

GUI配置工具

- **systemadm** — 用于*systemd*单元的图形化浏览器。可显示单元列表，并能按类型筛选。

: https://cgit.freedesktop.org/systemd/systemd-ui/ || systemd-ui包

- **SystemdGenie** — 基于KDE的*systemd*管理工具。

: https://invent.kde.org/system/systemdgenie || systemdgenie包

在网络已连接后再启动某服务

如果需要将某服务延迟到网络已连接后再启动, 直接在 *.service* 文件中包含以下依赖项：

```
/etc/systemd/system/<单元名>.service
[Unit]
...
Wants=network-online.target
After=network-online.target
...
```

要使`network-online.target`正确反映网络状态，必须启用所使用的网络管理器的网络等待服务。

- 若使用NetworkManager, `NetworkManager-wait-online.service`应该与`NetworkManager.service`一同启用。可通过`systemctl is-enabled NetworkManager-wait-online.service`进行检查。若服务没用启用，重新启用`NetworkManager.service`。
- 若使用netctl, 启用`netctl-wait-online.service` (除非使用了*netctl-auto*； 详见FS#75836)。
- 若使用systemd-networkd, `systemd-networkd-wait-online.service`应该与`systemd-networkd.service`一同启用。可通过`systemctl is-enabled systemd-networkd-wait-online.service`进行检查。若服务没用启用，重新启用`systemd-networkd.service`。

如果需要更为详细的解释，请查看网络配置同步点中的讨论。

若某服务需要执行DNS查询，其应该被排在`nss-lookup.target`后：

```
/etc/systemd/system/<单元名>.service
[Unit]
...
Wants=network-online.target
After=network-online.target nss-lookup.target
...
```

详见systemd.special(7) § Special Passive System Units.

要nss-lookup.target发挥作用，必须有一个单元通过Wants=nss-lookup.target引入nss-lookup.target，并通过Before=nss-lookup.target排在nss-lookup.target之前。一般情况下，上述配置由本地DNS解析器完成。

检查哪一个正在运行的单元引入了nss-lookup.target：

```
$ systemctl list-dependencies --reverse nss-lookup.target
```

默认启用新安装的单元

Arch Linux附带的/usr/lib/systemd/system-preset/99-default.preset包含disable *，这会导致默认情况下的*systemctl preset*是禁用所有新安装的的单元。因此某个新包安装后, 用户必须手动启用新单元。

若上述行为不符合预期, 创建一个从/etc/systemd/system-preset/99-default.preset到/dev/null的符号链接来覆盖该配置文件，这会导致 *systemctl preset*无视单元类型直接启用所有单元，除非*systemctl preset*的配置目录中有文件另有声明。用户单元不受影响。详见systemd.preset(5)。

注意：若某个软件包含有多个互相冲突的单元，默认启用所有单元可能会出现问题。*systemctl preset*原本旨在供发行版或系统管理员使用。在有两个冲突单元将被同时启用的情况下，应明确在*systemctl preset*的配置文件中禁用其中一个单元，详见systemd.preset(5)。

应用程序环境沙盒

可通过一个单元文件创建一个沙盒以在加固的虚拟环境中隔离应用程序及其进程。*systemd*充分利用namespaces，一系列允许/拒绝 capabilities 和 cgroups，以通过可拓展的执行环境设置（systemd.exec(5)）将进程禁锢在容器中。

将现有*systemd*单元文件进行沙盒化加固通常需要结合使用strace包，stderr和journalctl(1)等进行大量试验。因此，最好先在上游文档中搜索已完成的测试以作为试验的基础。

首先，最好通过以下命令获得特定单元可能的加固选项作为基础：

```
$ systemd-analyze security <单元>
```

关于如何使用 systemd 部署沙盒的一些示例：

- CapabilityBoundingSet定义了一个列表，以指明某个单元可以/不能使用哪些capabilities(7)。详见systemd.exec(5) § CAPABILITIES。
- 例如，对CAP_SYS_ADM capability的控制应该是安全沙盒的目标: CapabilityBoundingSet=~ CAP_SYS_ADM

通知失败的单元

**本文或本章节可能需要合并到 systemd/Timers#MAILTO。**

附注： 相同话题，不同解决方案（在 Talk:Systemd 中讨论）

要在服务故障时发出通知，可在相应服务的单元文件中添加`OnFailure=`设置，这可通过#附加配置片段完成。若要在每一个服务的单元文件中添加该设置，可使用一个顶层附加配置片段，详见systemd.unit(5)。

为服务单元创建顶层附加配置片段：

```
/etc/systemd/system/service.d/toplevel-override.conf
[Unit]
OnFailure=failure-notification@%n.service
```

这将向每个服务单元文件中添加`OnFailure=failure-notification@%n.service`。若**单元甲**失败，会启动`failure-notification@`**单元甲**`.service`来递送相关通知（或执行其它配置的动作）。

创建`failure-notification@.service`模板单元：

```
/etc/systemd/system/failure-notification@.service
[Unit]
Description=Send a notification about a failed systemd unit

[Service]
Type=oneshot
ExecStart=/path/to/failure-notification.sh %i
# 以临时用户/组运行并启用其他安全措施
DynamicUser=true
```

接下来可创建`failure-notification.sh`脚本，定义执行的操作或发送通知的方式（邮件、桌面通知、gotify、XMPP等）。`%i`将被替换为失败单元的名称并作为参数被传递给`failure-notification.sh`脚本。

为防止`failure-notification@.service`的实例启动失败时引发`failure-notification@.service`实例的递归性启动，在`failure-notification@.service`单元的配置目录下创建一个与顶层附加配置片段文件名相同的空的附加配置片段（该空的附加配置片段比全局附加配置片段优先级更高）：

```
# mkdir -p /etc/systemd/system/failure-notification@.service.d
# touch /etc/systemd/system/failure-notification@.service.d/toplevel-override.conf
```

疑难解答

寻找失败的单元

要寻找启动失败的*systemd*单元，执行：

```
$ systemctl --state=failed
```

可通过它们的日志查找失败的原因。详见systemd/Journal#过滤输出。

诊断系统启动问题

*systemd*有许多用于调试系统启动过程中的问题的选项。要捕获在*systemd*接管启动流程前的日志，参阅boot debugging。还可参阅此系统调试文档。

诊断一个服务

如果某个 *systemd* 服务的工作状况不合预期或想了解发生了什么的更多信息，将`SYSTEMD_LOG_LEVEL`环境变量的值设为`debug`。例如，要以调试模式运行*systemd-networkd*守护进程：

在此服务的配置文件中加入如下配置片段：

```
[Service]
Environment=SYSTEMD_LOG_LEVEL=debug
```

或者手动设置该环境变量以达到相同效果：

```
# SYSTEMD_LOG_LEVEL=debug /lib/systemd/systemd-networkd
```

之后，重新启动*systemd-networkd*并以`-f`/`--follow`启动journalctl观察该单元的日志。

关机/重启十分缓慢

如果关机十分缓慢（甚至跟死机了一样），很可能是某个拒不退出的服务在作怪。对于每个服务*systemd*会等待一段时间，然后再尝试杀死它。要确定是否受到该问题影响，详见 *systemd* 文档中的Shutdown completes eventually 这篇文章。

一个常见的原因是存在一个搁置的关机或挂起进程。要确认是否为这种情况，执行下列命令中的任何一个并观察有无类似的输出：

```
# systemctl poweroff
Failed to power off system via logind: There's already a shutdown or sleep operation in
progress
# systemctl list-jobs
  JOB UNIT                    TYPE  STATE
...
21593 systemd-suspend.service start running
21592 suspend.target          start waiting
..
```

该问题的解决方法是取消这些关机/挂起工作：

```
# systemctl cancel
# systemctl stop systemd-suspend.service
```

并再次尝试关机或重启。

一些短命进程似乎没有产生任何日志输出

对于短命的服务进程（启动后迅速失败），若以root身份执行`journalctl -u <`单元名>可能看不到任何输出，此时可转而使用进程的PID。例如，若`systemd-`

`modules-load.service`单元失败，通过`systemctl status systemd-modules-load`得到的对应进程PID为123，以root身份执行`journalctl -b _PID=123`或许就能得到日志输出。上述问题的原因是日志元数据（如`_SYSTEMD_UNIT`、`_COMM`）以异步方式被采集，且依赖于`/proc`中相应进程的目录。修复该问题需要修改内核以通过socket提供需要的数据（类似于`SCM_CREDENTIALS`）。简而言之，该问题是一个bug，只需记住由于*systemd*的设计，在启动后迅速失败的单元可能不会打印任何输出到系统日志中。

systemd-tmpfiles-setup.service在系统启动时失败

自*systemd* 219开始，`/usr/lib/tmpfiles.d/systemd.conf`为`/var/log/journal`下的目录指定了ACL选项。因此存放日志的文件系统必须启用ACL支持。

如何在存放`/var/log/journal`的文件系统上启用ACL的说明见Access Control Lists#启用 ACL。

在远程主机上关闭救援（emergency）模式

当救援模式被触发时，机器不会连接网络，因此对于托管于Azure或Google Cloud的远程主机，或许需要关闭救援模式。

要关闭救援模式，屏蔽`emergency.service`和`emergency.target`。

参见

- Wikipedia:systemd
- 官方网站
- systemd 优化
- systemd 常见问题解答
- systemd 技巧与窍门
- systemd(1)
- 其他发行版
- Gentoo:Systemd
- Debian:Systemd
- Fedora:Systemd
- 如何使用 Systemctl 管理 Systemd 服务和单元
- 使用 systemd-logind 进行会话管理
- Emacs 对 Systemd 文件的语法高亮
- 两篇 部分介绍文章，发表于 *The H Open* 杂志。

包管理器

本章解决使用日久才显形的问题：本地数据库损坏后如何重建，升级留下的 .pacnew 与 .pacsave 如何处置，软件包签名与密钥如何维护。

建议从《Pacman》读起，再读《pacnew 与 pacsave》和《软件包签名》，它们对应最常触发的故障；《恢复本地数据库》当手册，出事再翻；《提示和技巧》按需取用；《各软件包管理器命令对应关系》留作速查。最容易踩的坑是把 pacnew 当作可长期忽略的残留，让旧配置一直压着新默认值；把恢复本地数据库当成重装软件包；照搬别处的参数习惯做出部分升级。

8.1　Pacman/恢复本地数据库

难度 入门 | **篇幅** 5.5 KB | **所属** 包管理器

本节将讲到

- 生成软件包恢复清单
- 开始恢复

如果遇到下面的问题，很可能需要恢复 pacman 本地数据库：

- `pacman -Q` 什么都不输出，`pacman -Syu` 错误地报告系统已为最新。
- 使用 `pacman -S` 安装软件包时，很多已经安装过的依赖提示未安装。

pacman 储存本地软件包的数据库 `/var/lib/pacman/local` 很可能已经损坏甚至丢失。这是很严重的问题，请按照如下步骤修复。

首先，确认 pacman 的日志文件还在：

```
$ ls /var/log/pacman.log
```

如果日志丢失了，那就*不能*使用本方法修复，可以尝试使用Xyne的软件包检测脚本重建数据库。要是还行不通，很遗憾，最后的路就是重装系统。

生成软件包恢复清单

警告：如果出于某些原因，您的 pacman 缓存或 makepkg package destination 包含了其他架构的包，请在继续前移除它们。

安装 pacman-contrib包 以获取 *paclog-pkglist*。

创建日志过滤脚本并使其可执行：

```
pacrecover
#!/bin/bash -e

# load configuration settings from the makepkg configuration file
. /etc/makepkg.conf
```

```
# determine the cache directory from pacman configuration, defaulting
to /var/cache/pacman/pkg, remove prefix with sed
PKGCACHE=$( (grep -m 1 '^CacheDir' /etc/pacman.conf || echo 'CacheDir
= /var/cache/pacman/pkg') | sed 's/CacheDir = //')

# define directories to search for package files
pkgdirs=("$@" "$PKGDEST" "$PKGCACHE")

# read package name and version from input and construct a search pattern for package
files
while read -r -a parampart; do

        # loop through each directory to search for matching package files
        for pkgdir in "${pkgdirs[@]}"; do

                # check each file matching the pattern in the current directory
                for i in "$pkgdir"/"${parampart}"-"${parampart}"-*.pkg.tar.{xz,zst} ;
do

                        # if a file exists, print its path and stop checking further
                        [ -f "${i}" ] && { echo "${i}" ; break; };
                done

                # If no file is found, output the package name to stderr
        done || echo "${parampart}" 1>&2
done
```

运行脚本（可选择将带有包的目录作为参数）：

```
$ paclog-pkglist /var/log/pacman.log | ./pacrecover >files.list 2>pkglist.orig
```

这样做会创建两个文件：`files.list` 存储着依然在设备上的软件包的包文件;`pkglist.orig` 存储着需要下载的包。接下来的操作可能会导致设备上的旧版本包的部分文件与新版不匹配，需要手动干预。

过滤掉第二个列表中无法用软件仓库安装的包：

```
$ { cat pkglist.orig; pacman -Slq; } | sort | uniq -d > pkglist
```

注意：如果失败并返回报错 `failed to initialise alpm library`，请检查`/var/lib/pacman/local/ALPM_DB_VERSION` 是否存在。若不存在，请以 root 执行`pacman-db-upgrade`，然后执行 `pacman -Sy`，接下来重试之前的命令。

检查是否缺失一些重要的 *base* 软件包并将它们加入列表：

```
$ comm -23 <({ echo base ; expac -l '\n' '%E' base; } | sort) pkglist.orig >> pkglist
```

如果两个列表的内容均就绪，无甚缺漏，就请继续下一步，它们将被用于恢复 pacman 的已安装软件包数据库 `/var/lib/pacman/local/`。

开始恢复

为恢复工作定义一个 bash 函数。

```
 recovery-pacman() {
    pacman "$@"  \
    --log /dev/null   \
    --noscriptlet     \
    --dbonly          \
    --overwrite "*"   \
    --nodeps          \
    --needed
```

```
}
```

`--log /dev/null` 参数可以避免污染 pacman 日志；`--needed` 参数可以跳过数据库中已存在的包以节省时间；`--nodeps` 参数允许安装已缓存的包，即使已安装的包依赖于更新版本。其余选项用于使 **pacman** 在进行操作时不读写文件系统。

同步数据库：

```
# pacman -Sy
```

从 `files.list` 安装本地可用的包以开始生成数据库：

```
# recovery-pacman -U $(< files.list)
```

从 `pkglist` 安装其余包：

```
# recovery-pacman -S $(< pkglist)
```

更新本地数据库以将不被其它包依赖的包标记为明确按照并将其它包标记为依赖。您将来在移除软件包时需要格外小心，但是原始数据库已丢失，这已经是我们所能做的最好的。

```
# pacman -D --asdeps $(pacman -Qq)
# pacman -D --asexplicit $(pacman -Qtq)
```

可选检查安装的软件包是否损坏：

```
# pacman -Qk
```

可选参阅 Pacman/提示和技巧#查找不属于任何软件包的文件。

更新所有软件包：

```
# pacman -Su
```

8.2 Pacman/pacnew 与 pacsave

难度 进阶 | **篇幅** 9.8 KB | **所属** 包管理器

本节将讲到

- 为什么会创建这些文件
- 包备份文件
- 类型说明
- 定位 .pac* 文件
- 管理 .pac* 文件

在使用 pacman 移除一个带有配置文档的软件包时，pacman 通常会将配置文档复制为一个后缀名为 `.pacsave` 的备份文档。

同样的，当 pacman 升级一个软件包，而新软件包含有与与当前配置不同的新配置文件时，pacman 会将新配置写入 `pacnew` 文件。当写入这些文件时，pacman 会输出提示信息。

为什么会创建这些文件

当升级某个软件包（命令为 `pacman -Syu`, `pacman -Su` 或者`pacman -U`）时，可能会创建一个 `.pacnew` 文件，以避免覆盖一个之前被用户修改过的已存在文件。此时，pacman会输出如下信息：

```
warning: /etc/pam.d/usermod installed as /etc/pam.d/usermod.pacnew
```

当卸载某个软件包（命令为 `pacman -R`）或升级某个软件包（该软件包必须首先被卸载）时，可能会创建一个 `.pacsave` 文件。当 pacman 数据库记录了应当备份该软件包的某个文件时，pacman 会创建一个 `.pacsave` 文件。此时，pacman 会输出如下信息：

```
warning: /etc/pam.d/usermod saved as /etc/pam.d/usermod.pacsave
```

这些文件需要用户手动干预，我们推荐您在每次软件包升级或卸载之后马上处理它们。如果不处理，不当的配置可能导致软件功能出问题，甚至完全无法使用。

包备份文件

软件包的 PKGBUILD 文件指定了升级或卸载软件包时需要被保存或备份的文件。例如，PulseAudio 的 PKGBUILD 文件包含如下行：

```
backup=(etc/pulse/{daemon.conf,default.pa,system.pa})
```

安装后，可以使用 `pacman -Qii *软件包名*` 从 pacman 数据库中查询此列表。

要阻止任何软件包覆盖某文件，请阅读 Pacman#在升级时跳过文件.

类型说明

.pacnew

对于每一个 #包备份文件，在升级过程中，Pacman 会将从文件内容生成的三个 MD5 校验和进行交叉比较：一个校验和对应最初由软件包安装的版本，一个对应当前在文件系统中的版本，还有一个对应新软件包中的版本。如果当前在文件系统中的文件版本已经被修改过，那么Pacman无法知道如何将这些更改与新版本合并。因此，在升级过程中，为了避免覆盖已修改的文件，Pacman会保存新版本，并附加上 `.pacnew` 后缀，而不会对已修改的版本做任何改动。

注意：软件包的安装可能会导致其他软件包所拥有的文件发生变化（例如 FS#77404）。因此，即使您没有手动更改某些文件，也可能生成 `.pacnew` 文件。

进一步详细说明，三重 MD5 校验和对比会导致以下几种结果之一：

原始版本 = *X*, 当前版本 = *X*，新版本 = *X* : 所有三个版本的内容完全相同，所以覆盖是没有问题的。用新版本覆盖当前版本，并且不通知用户（尽管文件内容相同，

这次覆盖将会更新文件系统的有关该文件的安装、修改及访问时间的信息，并确保应用任何文件权限变更）。

原始版本 = *X*，当前版本 = *X*，新版本 = *Y* : 当前版本的内容与原始版本一致，但新版本不同。由于用户没有修改当前版本，而新版本可能包含改进或修复，所以用新版本覆盖当前版本，并且不通知用户。这是 Pacman 能够执行的唯一一种自动合并新更改的情况。

原始版本 = *X*，当前版本 = *Y*，新版本 = *X* : 原始软件包和新软件包都包含了文件的完全相同版本，但是当前文件系统中的版本已被修改。保留当前版本，并丢弃新版本而不通知用户。

原始版本 = *X*，当前版本 = *Y*，新版本 = *Y* : 新版本与当前版本完全相同。用新版本覆盖当前版本，并且不通知用户（尽管文件内容相同，这次覆盖将会更新文件系统的有关该文件的安装、修改及访问时间的信息，并确保应用任何文件权限变更）。

原始版本 = *X*，当前版本 = *Y*，新版本 = *Z* : 所有三个版本都不相同，因此保留当前版本，并以 `.pacnew` 后缀安装新版本，同时警告用户有关新版本的存在。用户需要手动合并新版本中必要的更改到当前版本中。

有时升级后的软件包罕见地包含了一个之前版本不存在的备份文件，这种情况会被正确处理为 X/Y/Y 或 X/Y/Z，其中 X 代表不存在。

.pacsave

如果用户修改了 `backup` 中指定的某个文件，那么在软件包其余部分移除后该文件将被重命名，带上 `.pacsave` 扩展名，仍然存于文件系统中。

注意：使用命令 `pacman -R` 中的 `-n` 选项会移除指定软件包中的所有文件，因此不会创建 `.pacsave` 文件。

定位 .pac* 文件

Pacman 不会自动处理 `.pacnew` 文件，需要自己维护。下面提供帮助你处理这些文件的一些工具。如果需要手动处理，首先需要找到这些文件。在升级或移除大量软件包时，可能会忽略一些更新的 .pac* *文件。可以使用以下方法之一检查存在的 .pac** 文件：

在用于存放大多数全局配置文件的 `/etc` 目录中搜索：

```
$ find /etc -regextype posix-extended -regex ".+\.pac(new|save)" 2> /dev/null
```

或者在将上述命令中的 `/etc` 替换为 `/` 以在整个磁盘上进行搜索（在这种情况下，你可能想要选择性地跳过某些目录以加快搜索速度）。

如果你安装了 locate，还可以使用它来搜索。首先需要更新索引数据库：

```
# updatedb
```

然后:

```
$ locate --existing --regex "\.pac(new|save)$"
```

或者使用 pacman 日志来找到它们：

```
$ egrep "pac(new|save)" /var/log/pacman.log
```

注意：日志不会跟踪当前在文件系统中的文件，也不会跟踪已经被删除的文件；上述命令将列出系统上曾经存在过的所有 `.pac*` 文件。为了只获取最近的10个 `.pac*` 文件，可以将结果通过管道传输给 `tail` 命令。

管理 .pac* 文件

pacdiff

pacman-contrib包 提供了一个简单的 pacdiff(8) 工具来管理 *.pac** 文件。

它会搜索 `.pacnew`、`.pacsave` 和 `.pacorig` 文件，然后提示对它们采取行动。

默认情况下，它使用 `--pacmandb` 来搜索当前已安装包的 `backup` 列表信息。如果这不够用，您可以指定 `--find` 或 `--locate` 来进行更彻底的搜索。

默认情况下，它使用 vimdiff，但您可以通过 `DIFFPROG=*your_editor* pacdiff` 指定一个不同的工具。查看应用程序列表/工具#比较，差异，合并了解其他常见的比较工具。

第三方工具

一些第三方工具提供了不同程度的自动化功能来处理这些任务：

- **dotpac** — 基本的交互式脚本，带有基于 ncurses 的文本界面和有用的向导。没有合并或自动合并功能。

: https://github.com/AladW/dotpac || dotpacAUR

- **etc-update** — *Gentoo* 的实用工具，兼容包括 Arch 在内的其他发行版。它提供了一个简单的命令行界面来查看、合并和交互式编辑更改。像注释这样的简单更改可以自动合并。

: Gentoo:Handbook:Parts/Portage/Tools#etc-update || etc-update包

- **p3wm** — 三方合并 `.pacnew` 文件。它可以自动合并简单更改。如果发生冲突，它将启动 vimdiff、meld 或 kdiff3 来解决它们。

: https://github.com/5long/p3wm || p3wmAUR

- **pacnews-git** — 一个简单的脚本，旨在找到所有的 `.pacnew` 文件，然后用 vimdiff 编辑它们。

: https://github.com/pbrisbin/scripts/blob/master/pacnews || pacnews-gitAUR

- **pacfiles-mode** — 一个用于 Emacs 管理和合并.*pacnew*文件的包。

: https://github.com/UndeadKernel/pacfiles-mode || 在 melpa 中可用

- **pacdiff-pacman-hook-git** — Pacman 钩子，自动运行 pacdiff。

: https://github.com/desbma/pacman-hooks || pacdiff-pacman-hook-gitAUR

参阅

- Arch Linux 论坛: 处理 .pacnew 文件

8.3 Pacman/软件包签名

难度 进阶 | **篇幅** 12.4 KB | **所属** 包管理器

本节将讲到

- 配置
- 管理密钥
- 提示和技巧
- 问题解决
- 参阅

为了保证软件包来自开发者, Pacman 使用信任网络中的 GnuPG 密钥进行软件包验证。这个网址记录了 Archlinux 的主密钥。其中至少三个主密匙被用来签署官方开发者和授信用户自己的密钥，而他们将用这些密钥签署自己的包。用户在配置 pacman-key 时也会生成一个唯一的密钥。所以信任网络也会把用户的密钥连接到五大主密钥上面。

信任网络示例:

- **自定义软件包**: 用户自己构建软件包并用本地密钥签名。
- **非官方软件包**: 开发者构建软并签名软件包。用户需要用自己的密钥签名开发者的密钥，将其变为可信。
- **官方软件包**: 开发者构建软件包，而开发者的密钥已经被 Arch 主密钥签名。最终用户用自己的密钥签名主密钥，这样就能信任所有官方开发者。

提示：HKP协议使用11371/tcp端口用来通信。为了从服务器得到签署的密钥（使用pacman-key），这个端口必须打开。

配置

配置 pacman

`/etc/pacman.conf` 中的 `SigLevel` 配置用 `pacman -S` 安装软件时使用的检查级别。文件的注释中列出了几个可选设置，pacman.conf(5) § PACKAGE AND DATABASE SIGNATURE CHECKING 有详细介绍。签名检查可以全局配置或针对某个仓库配置。如果 `SigLevel` 在 \[options\] 节中进行了全局设置，那么所有的包都必须签名。如果使用 `pacman.conf` 中默认的 `LocalFileSigLevel` ，所有编译和用 `pacman -U` 安装的软件包都不需要使用 `makepkg` 进行签名。

注意：尽管所有的官方软件包现在都进行了签名，但是在2018年11月的时候签名数据库还在开发。如果设置了 `Required`，那么 `DatabaseOptional` 也应该被设置。

默认的设置下，系统只安装被授信的密钥签署的软件包。

```
/etc/pacman.conf
SigLevel = Required DatabaseOptional
```

`TrustedOnly` 是 pacman 的默认设置。

默认配置的效果与下行等同：

```
SigLevel = Required DatabaseOptional TrustedOnly
```

上面这些也可以在仓库内部进行设置，比如：

```
[core]
SigLevel = PackageRequired # 'Optional' here would turn off a global 'Required' for this repository
Include = /etc/pacman.d/mirrorlist
```

软件仓库中的软件启用了签名验证，但是并不要求仓库数据库也被签名了。

警告：SigLevel `TrustAll` 设置仅仅为了测试而存在，使用它会信任未被验证的密钥。对于所有的官方软件源你应该使用 `TrustedOnly`。

初始化密钥环

要初始化 pacman 密钥:

```
# pacman-key --init
```

初始化密钥，需要收集到足够的熵。请随意移动鼠标，随机按键盘或者运行一些磁盘读写操作（比如在其他终端运行`ls -R /`或者`find / -name foo` 或者 `dd if=/dev/sda8 of=/dev/tty7`之类的）应该会收集足够的熵。如果你的系统没有足够的熵，这项工作需要好几个小时，但是如果你有，那么就会快多了。

这会在 `/etc/pacman.d/gnupg` 初始化密钥并生成系统主密钥。

注意：如果`pacman-key --init`运行时系统没有足够的熵，可能会需要很长时间。请在目标机器上安装 haveged 或 rng-tools。然后在用 root 权限执行`pacman-key --init` 前启动 `haveged.service`。

管理密钥

验证主密钥

通过以下命令进行配置：

```
# pacman-key --populate
```

该命令对 Master Signing Keys 进行验证，when prompted as these are used to co-sign (and therefore trust) all other packager's keys.

PGP 通常很长(2048 位或更长)，不太容易使用，所以通常创建一个40位十六进制指纹，最后八位被称为密钥 ID，是密钥的名字。长签名可以用来检测两个密钥是否相同。

官方开发者密钥

官方开发者和 TU 的密钥已经被主密钥签名认证，所以不需要用 pacman-key 认证它们。pacman 遇到不认识的签名时，它将会询问是否从密钥服务器（设置在`/etc/pacman.d/gnupg/gpg.conf`文件中，或在命令行中使用`--keyserver`选项）下载。

提示：维基百科提供了一份公钥服务器列表)。

下载开发者密钥后，以后都不需要下载。以后会用它验证所有这个开发者构建的软件包。

注意：

如果开发者和 TU 的密钥是较早之前导入，它们的签名可能还不存在于本地数据库，用下面命令更新：

```
# pacman-key --refresh-keys
```

当使用`--refresh-keys` 时，本地签名也会被远程查找，并收到未找到的消息，这是正常的。

导入非官方密钥

可以通过此方法在pacman密钥环中添加你自己的密钥，或者启用已签名的非官方软件仓库。

首先从密钥持有者手中拿到密钥 ID(`*keyid*`)，然后把密钥加入密钥环：

- 如果密钥位于密钥服务器，通过下面命令导入：

```
# pacman-key -r keyid
```

- 如果提供了地址，先下载，然后用下面密钥导入：

```
# pacman-key --add /path/to/downloaded/keyfile
```

对于所有要签名的密钥，都通过指纹进行验证：

```
$ pacman-key -f keyid
```

最后，本地签名导入的密钥：

```
# pacman-key --lsign-key keyid
```

现在可以用这个密钥签名软件包了。

用GPG调试

如果需要，可以直接使用GPG调试pacman钥匙环，例如：

```
# gpg --homedir /etc/pacman.d/gnupg --list-keys
```

提示和技巧

定期升级系统

定期使用 pacman#升级软件包更新系统可以防止绝大部分签名错误。

手动更新密钥环

如果有时不可避免地在一段时间里没有更新系统，或者在安装系统时使用的安装映像版本较旧，可能会遇到密钥环过旧而导致的签名错误，提示为“文件签名损坏”。这时你可以在更新系统之前手动同步包数据库并升级 archlinux-keyring包：

```
# pacman -Sy archlinux-keyring
```

此命令成功以后，应立即升级所有软件包以避免部分升级：

```
# pacman -Su
```

如果你是在安装系统的过程中，可以跳过此 `pacman -Su` 命令，因为接下来将要运行的 `pacstrap` 命令会在新安装的系统中安装最新的软件包。

这是**唯一**可以接受的使用 `-Sy` 升级软件包而不被视为部分升级的情况。其余情况下你总是应该使用 `pacman -Syu` 来升级系统。

Update system time regularly

When the 系统时间 is faulty, signing keys could be considered expired (or invalid) and signature checks on packages will fail. Synchronize the system clock regularly by using the Network Time Protocol daemon.

问题解决

error: signature from xxx is invalid

Pacman-key 依赖于系统时间。如果系统时间是错误的，将会获得一个报错：

```
error: PackageName: signature from "User <email@archlinux.org>" is invalid
error: failed to commit transaction (invalid or corrupted package (PGP signature))
Errors occured, no packages were upgraded.
```

如果是使用 ntpd, (root权限) 执行 `ntpd -qg`，然后执行 `hwclock -w`.

如果使用的是其他 NTP 客户端，请参考系统时间#时钟同步。

如果问题依然存在，请尝试下面方法：

删除报错的包

如果相同的包持续失败，并且您确定 pacman-key 的相关操作是正确的，可以尝试移除软件包 `rm /var/cache/pacman/pkg/*packagename*` 或者清空所有缓存，然后可以重新下载软件包。

重置所有密钥

如果要删除或重置系统，删除 `/etc/pacman.d/gnupg` 目录并重新运行 `pacman-key --init`。通过 `pacman-key --populate` 重新添加默认密钥。

禁用签名检查

警告：小心使用，禁用签名检查，pacman 会自动安装不信任的软件包。

如果不在意软件包签名，可以完全禁用 PGP 签名检查，编辑 `/etc/pacman.conf` 并取消注释 \[options\] 下的如下行:

```
SigLevel = Never
 #LocalFileSigLevel = Optional
#RemoteFileSigLevel = Required
```

需要同时注释掉单独软件源的 SigLevel 设置，因为他们会覆盖全局设置。

这样就不会进行任何签名检查，和 pacman 4 之前一样。如果这样，就不需要用 pacman-key 建立密钥环。

密钥导入失败

有三种可能的情况导致这个问题：

- 过期的archlinux-keyring包 包。
- 不正确的系统时间。
- 你的ISP屏蔽了用于导入 PGP keys 的端口。
- *pacman* 缓存中包含之前的未签名软件包
- 未正确设置 `dirmngr`

过期的 archlinux-keyring包 包可能会导致这个问题，根据情况采用下面的解决方法

切换公钥服务器

如果这样没有起作用，并且系统时间是正确的，你可以尝试切换到 Ubuntu 提供的公钥服务器(keyserver)。编辑 `/etc/pacman.d/gnupg/gpg.conf` 将 `keyserver` 行替换为

```
keyserver hkp://keyserver.ubuntu.com
```

清理软件包缓存

如果上面方法都不起作用，pacman 缓存 `/var/cache/pacman/pkg/` 可以包含之前下载的未签名软件包，手动清空缓存：

```
# pacman -Sc
```

```
=== Signature is unknown trust ===
```

在执行 `pacman -Syu` 时可能出现下面错误：

```
error: package-name: signature from "packager" is unknown trust
```

这是因为 `*package-name*` 软件包使用的 `*packager*` 的密钥不被本地的 pacman-key gpg 数据库信任。可能是密钥在加入密钥链之后失效了，可以通过下面方法解决：

- 用 `pacman-key --refresh-keys` 刷新密钥
- 本地手动签署密钥
- 刷新所有密钥
- 将 `SigLevel` 设置成 `TrustAll` (不推荐)

后面两个选项将会破坏信任连，所以请谨慎使用。

通过代理更新密钥

要使用代理更新密钥，必须同时在 `/etc/gnupg/dirmngr.conf` 和 `/etc/pacman.d/gnupg/dirmngr.conf` 中设置 `honor-http-proxy` 选项。更多信息请阅读 GnuPG#Use a keyserver.

注意：如果 *pacman-key* 没有用 `honor-http-proxy` 选项，执行失败，重启可能能够解决问题。

参阅

- DeveloperWiki:Package Signing Proposal for Pacman
- Pacman Package Signing – 1: Makepkg and Repo-add
- Pacman Package Signing – 2: Pacman-key
- Pacman Package Signing – 3: Pacman
- Pacman Package Signing – 4: Arch Linux

8.4 Pacman/各软件包管理器命令对应关系

难度 高级 | **篇幅** 20.7 KB | **所属** 包管理器

本节将讲到

- 基本操作
- 查询特定的软件包
- 查询软件包列表
- 查询软件包依赖

- 软件源管理

这个页面用表格展示一些流行的 Linux 发行版包管理器命令的对应关系。这是受到 openSUSE's Software Management Command Line Comparison 的启发而成的。

提示：Arch 用户在临时处理其他发行版时可以用 pacapt，它是对其它包管理器的简单包装。

基本操作

行为	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
搜索软件包（们）。默认情况下搜索的确切字段因每个工具而异。大多数选项都会带来同等水平的工具。	`pacman -Ss`	`dnf search`	`apt search`	`zypper search` 或 `zypper se [-s]`	`emerge --search` (`-s`) 或 `emerge --searchdesc` (`-S`)

查询特定的软件包

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
显示关于一个软件包的全部或者大多数信息。针对默认命令，每个根据的信息详细程度各不相同。但通过选项，各个工具可以达到大致相同的详细程度。	`pacman -Si` 或`pacman -Qi`	`dnf list` 或 `dnf info`	`apt show` 或 `apt-cache policy`	`zypper info`或 `zypper if`	`emerge -S`, `emerge -pv` 或`eix`
显示本地软件包信息：名称、版本、描述等等	`pacman -Qi`	`rpm -qi` / `dnf info installed`	`dpkg -s` 或 `aptitude show`	`zypper --no-remote info`或 `rpm -qi`	`emerge -pv` 或`emerge -S`
显示远端软件包信息：名称、版本、描述等等	`pacman -Si`	`dnf info`	`apt-cache show` 或 `aptitude show`	`zypper info`	`emerge -pv` and `emerge -S`或`equery meta`
显示由本地软件包提供的文件	`pacman -Ql`	`rpm -ql`	`dpkg -L`	`rpm -ql`	`equery files`或 `qlist`
显示由远端软件包提供的文件	`pacman -Fl`	`dnf repoquery -l`或`repoquery -l` (from package yum-utils)	`apt-file list`		`pfl`
	`pacman -Qo`	`rpm -qf` (installed only)			

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
查询拥有某个文件的软件包		或`dnf provides` (everything)或`repoquery -f` (from package yum-utils)	`dpkg -S`或`dlocate`	`rpm -qf` (installed only)或`zypper search -f` (everything)	`equery belongs` 或`qfile`
列出软件包所提供的文件. 同样,这个功能可以被其他更复杂的命令实现	`pacman -Ql`或`pacman -Fl`	`dnf repoquery -l`	`dpkg-query -L`	`rpm -ql`	`equery files` 或`qlist`
显示提供给定表达式的包，也称为反向提供。主要是搜索特定字段的快捷方式。其他工具可能通过搜索命令提供此功能。	`pacman -F`	`dnf provides`	`apt-file search`	`zypper what-provides`或`zypper wp` (exact match)或`zypper se --provides` (fuzzy match)	`equery belongs` (only installed packages)或`pfl`
搜索所有软件包以找到持有某个特定文件的包	`pacman -F`	`dnf provides`	`apt-file search`或`auto-apt` is using this functionality.	`zypper search -f`	`equery belongs` 或`qfile`
显示软件包的变更日志	`pacman -Qc`	`dnf changelog`	`apt-get changelog`	`rpm -q --changelog`	`equery changes -f`

查询软件包列表

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
通过软件包的名称、描述、简介来检索软件包。不同工具检索的字段可能有所不同，	`pacman -Ss`	`dnf search`	`apt search`	`zypper search` 或`zypper se -s`	`emerge -S` 或`eix`

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
大多数选项可以使这些工具的功能趋于一致					
列出有可用更新的软件包。注意：有些工具提供特殊命令来限制输出到某些安装源，而有些工具则使用选项来实现相同的功能。	`pacman -Qu`	`dnf list updates` 或 `dnf check-update`	`apt list --upgradable`	`zypper list-updates`或 `zypper patch-check` (just for patches)	`emerge -uDNp @world`
显示由软件包管理器处理的所有安装源中的软件包列	`pacman -Sl`	`dnf list available`	`apt-cache dumpavail`或 `apt-cache dump` (Cache only)或`apt-cache pkgnames`	`zypper packages`	`portageq all_best_visible /`

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo	
表。一些工具提供选项或额外的命令来限制输出到特定的安装源。						
生成一个已安装软件包的列表	`pacman -Q`	`dnf list installed`	`dpkg --list	grep ^i`	`zypper packages --installed-only`	`qlist -IC`
列出已安装但不(再)属于任何软件源的软件包	`pacman -Qm`	`dnf list extras`	`apt --installed list	grep ,local`	`zypper packages --installed-only --orphaned`	`eix-test-obsolete`
列出最近添加到其中一个安装源的软件包，即其中一个安装源中新增的软件包。		`dnf list recent`	`aptitude search '~N'`或 `aptitude forget-new`		`eix-diff`	
	`pacman -Q`	`rpm -qa`			`qlist -ICv`	

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo	
列出本地已安装的软件包及版本			`dpkg -l`或`apt list --installed`	`zypper search -si`或`rpm -qa`		
搜索本地已安装的软件包名或描述	`pacman -Qs`	`rpm -qa '*<str>*'`	`aptitude search '~i(~n $name`	~d $description)``	`zypper search --installed-only --search-descriptions`	`eix -S -I`
列出不再被任何软件包所需要的依赖	`pacman -Qtt`	`dnf leaves`或`package-cleanup --leaves --all`	`deborphan -anp1`	`zypper packages --unneeded`	`emerge -pc`	
列出显式安装的软件包(不作为任何其他软件包的依赖而安装)	`pacman -Qe`	`dnf history userinstalled`	`apt-mark showmanual`	`zypper search *	grep -E '^i\+'` (workaround)*	`emerge -pvO @selected`或`eix --selected`
列出(作为依赖)自动安装的软件包	`pacman -Qd`	`zypper search *	grep -E '^i[^+]'` (workaround)*	`apt-mark showauto`		

查询软件包依赖

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
显示需要安装X的包，也称	`pacman -Sii`或	`dnf repoquery --alldeps --`	`apt-cache rdepends`或`aptitude search ~D$pattern`	`zypper search --requires`	`equery depends`

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
为显示反向依赖关系	pacman -Qii	whatrequires或repoquery --whatrequires			
显示与给定表达式（通常是软件包）冲突的软件包。也可以使用搜索功能来模仿此功能。	pacman -Si或pacman -Qi	dnf repoquery --conflicts	aptitude search '~C$pattern'	zypper search --conflicts	
列出所有为给定软件包所需的软件包，也就是显示依赖项	pacman -Si或pacman -Qi	dnf repoquery --requires或repoquery -R	apt-cache depends或apt-cache show	zypper info --requires	emerge -ep
列出当前软件包提供的内容	pacman -Sii或pacman -Qii	dnf repoquery --provides	dpkg -s或aptitude show	zypper info --provides	equery files或qlist
列出所有需要特定软件包的软件包	pacman -Sii	dnf repoquery --installed --alldeps --whatrequires	aptitude search ~D{depends,recommends,suggests}:$pattern或aptitude why或apt-cache rdepends	zypper search --requires	equery depends -a
Display all packages that the specified packages obsoletes.	pacman -Si或pacman -Qi	dnf list obsoletes	apt-cache show	zypper info --obsoletes	
Generates an output suitable for processing with dotty for the given package(s).			apt-cache dotty		

软件源管理

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
软件源管理	编辑 /etc/pacman.conf	编辑 /etc/yum.repos.d/${REPO}.repo	编辑 /etc/apt/sources.list或者apt edit-source	编辑 /etc/zypp/repos.d/${REPO}.repo	layman或eselect repository
将安装源添加到系统中。一些工具为某些源提供了额外的命令，而另一些工具则允许添加命令使用所有类型的源URI。还有一些，比如apt强制编辑源列表。apt-cdrom是一个特殊的命令，为CD/DVD作为源提供	edit /etc/pacman.conf	dnf config-manager	apt-cdrom add	zypper ar <URL or .repo file>	layman或overlays

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
了选项。					
刷新所有或指定安装源的信息	`pacman -Sy` (always upgrade the whole system afterwards)	`dnf clean expire-cache` and then `dnf check-update`	`apt-get update`	`zypper refresh`或`zypper ref [-s]`	`emerge --sync`或`layman -S`
打印所有安装源的列表,包括URI、别名等重要信息。	`cat /etc/pacman.d/mirrorlist`	`cat /etc/yum.repos.d/*`	`apt-cache policy`	`zypper repos`或`zypper lr --uri --alias`	`layman -l`或`eselect repository list`
列出某一个仓库中的所有软件包	`paclist <repo>`			`zypper packages -r <repo>` or `zypper pa -r <repo>`	`eix --in-overlay`
Disable an installation source for an operation		`dnf --disablerepo=`			`emerge package::repo-to-use`
Download packages from a different version of the distribution than the one installed.	`pacman -S *repo_name*/*package*`	`dnf --releasever=`	`apt-get install -t release package`或`apt-get install package/release` (dependencies not covered)	`zypper install -r <repo> package`	`echo "category/package ~amd64" >> /etc/portage/package.keywords` and then `emerge package`

Overrides

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
Add a package lock rule to keep its current state from being changed	edit `/etc/pacman.conf` modifying IgnorePkg array	edit `dnf.conf` adding/amending the `exclude` option	`apt-mark hold pkg`	`zypper al` 或put package name in `/etc/zypp/locks`	`/etc/portage/package.mask`
Delete a package lock rule	edit `/etc/pacman.conf` removing package from IgnorePkg line		`apt-mark unhold pkg`	`zypper rl` 或remove package name from `/etc/zypp/locks`	`/etc/portage/package.mask` (or `package.unmask`)
Show a listing of all lock rules	`cat /etc/pacman.conf`		`/etc/apt/preferences`	`zypper ll` 或 view `/etc/zypp/locks`	`cat /etc/portage/package.mask`
Set the priority of the given package to avoid upgrade, force downgrade或 to overwrite any default behavior. Can also be used to prefer a package version from a certain installation	edit `/etc/pacman.conf` modifying HoldPkg and/or IgnorePkg arrays		`/etc/apt/preferences`, `apt-cache policy`	`zypper mr -p`	edit `/etc/portage/package.accept_keywords` adding a line with `=category/package-version`

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
source .					
Remove a previously set priority			`/etc/apt/preferences`	`zypper mr -p`	edit `/etc/portage/package.accept_keywords` removing offending line
Show a list of set priorities			`apt-cache policy` 或`/etc/apt/preferences`	`zypper lr -p`	`grep -r . /etc/portage/package.accept_keywords`

验证和修复

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
验证单个软件包	`pacman -Qk` (can add another `k`)	`rpm -V`	`debsums`	`rpm -V`	`equery check`
验证所有软件包	`pacman -Qk` (can add another `k`)	`rpm -Va`	`debsums`	`rpm -Va`	`equery check`
重新安装给定的软件包。这将重新安装给定的包，而不会产生依赖性问题。	`pacman -S`	`dnf reinstall`	`apt install --reinstall`	`zypper install --force`	`emerge -1O`
验证整个系统的依赖关系; 安装过程被强制终止时使用	`pacman -Dk`	`dnf repoquery --requires`	`apt-get check`	`zypper verify`	`emerge -uDN @world`
~使用一些魔法来修理损坏的依赖关系~	for *pacman* dependency level, use `pacman -Dk`; for shared library level, use findbrokenpkgsAUR 或`lddd` (from devtools包)	`dnf repoquery --unsatisfied`	`apt-get --fix-broken` and then `aptitude install`	`zypper verify`	`revdep-rebuild`
为软件包系统添加一个检查点，以备后续回滚。		(unnecessary, it is done on every transaction)		n/a	
从系统中移除一个检查点。	n/a	n/a		n/a	
提供所有系统检查点的列表。	n/a	`dnf history list`		n/a	
将整个软件包回滚到特	n/a	`dnf history rollback`		n/a	

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
定日期或检查点					
撤销单个指定的事务	n/a	`dnf history undo`		n/a	

Using package files and building packages

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo		
查询命令行上提供的包，而不是包管理数据库中的条目	`pacman -Qp`	`rpm -qp`	`dpkg -I`				
List the contents of a package file	`pacman -Qpl`	`rpmls rpm -qpl`	`dpkg -c`	`rpm -qpl`			
Install local package file, e.g. app.rpm and uses the installation sources to resolve dependencies	`pacman -U`	`dnf install`	`dpkg -i`	`zypper in`	`emerge`		
Updates package(s) with local packages and uses the installation sources to resolve dependencies	`pacman -U`	`dnf upgrade`	`debi`		`emerge`		
Add a local package to the local package cache mostly for debugging purposes.	`cp *package-filename* /var/cache/pacman/pkg/`		`apt-cache add *package-filename*`	n/a	`cp *package-filename* /usr/portage/distfiles`		
解包一个软件包	`tar -xvf`	`rpm2cpio`	cpio -vid`	`dpkg-deb -x`	`rpm2cpio`	`cpio -vid`	`tar -jxvf`
					`emerge -o`		

Action	Arch	Red Hat/Fedora	Debian/Ubuntu	SLES/openSUSE	Gentoo
Install/Remove packages to satisfy build-dependencies. Uses information in the source package	Use ABS and `makepkg -seoc`	`dnf builddep`	`apt-get build-dep`	`zypper si -d`	
Display the source package to the given package name(s)		`dnf repoquery -s`	`apt-cache showsrc`	n/a	
Download the corresponding source package(s) to the given package name(s)	Use ABS and `makepkg -o`	`dnf download --source`	`apt-get source`或`debcheckout`	`zypper source-install`	`emerge --fetchonly`
构建一个软件包	`makepkg -s`	`rpmbuild -ba` (normal)或 *mock* (in chroot)	`debuild`	`rpmbuild -ba`, then build, and then `osc build`	`ebuild`或`quickpkg`
检查可能的打包问题	*namcap* (requires namcap包)	*rpmlint*	*lintian*	*rpmlint*	*repoman*

Log file rotation

By default, Arch Linux does not rotate `pacman.log`. See, for example, FS#11272 and FS#20428#comment66480. This is in contrast to the default policy of most other Linux distributions. Some distributions, notably Gentoo, hardly write log files by default.

另见

- Changes in DNF CLI compared to Yum

8.5 Pacman

难度 高级 | **篇幅** 40.3 KB | **所属** 包管理器

本节将讲到

- 用法

- 配置
- 疑难解答
- 另请参见

Pacman软件包管理器是 Arch Linux 发行版的一大亮点。它将一个简单的二进制包格式和易用的构建系统结合了起来。*Pacman*的目标是简化对软件包的管理，无论软件包是来自官方软件仓库还是用户自己创建的软件包。

Pacman 通过和主服务器同步软件包列表来保持系统是最新的。这种服务器/客户端模式可使得用户使用简单的命令，就能下载或安装软件包，并包含其所有必需的依赖包。

Pacman 用 C 语言编写，并使用 bsdtar(1) tar) 作为打包格式。

提示：pacman包 软件包提供了如 makepkg 和 vercmp(8) 等工具。其它有用的工具如 pactree、checkupdates 等则在 pacman-contrib包 软件包中（它曾是 *pacman* 的一部分）。运行 `pacman -Ql pacman | grep -E 'bin/.+'` 可查看完整的工具列表。

用法

以下只是 *pacman* 可以执行的操作的一小部分示例。要阅览更多示例可参考 pacman(8)。

提示：对于以前使用过其他 Linux 发行版的用户，Pacman Rosetta 这篇对比文章将大有帮助。

安装软件包

一个软件包就是一个归档包，其中包含：

- 该软件所有（编译好的）文件，并按照每个文件的安装位置储存在一个与根目录结构相同的目录中
- 该软件的元数据，例如软件名称、版本和依赖等
- 供 pacman 使用的其它描述性文件
- （可选）在安装、升级、卸载时执行的安装脚本

Arch 的软件包管理器 *pacman* 可以安装、更新和删除这些软件包。使用软件包而不是自己编译和安装程序有很多好处：

- 轻松升级：*pacman* 会在更新可用时立即更新已安装的软件包
- 依赖检查：*pacman* 会为你处理依赖问题，只需要指明程序（名），*pacman* 就会将它和它所需的所有其他程序都一起安装。
- 干净卸载：*pacman* 持有软件包包含的所有文件的列表。这样一来，当你决定移除软件包时，不会无意留下任何文件。

注意：

- 软件包通常有可选依赖，它们为软件提供额外的功能，但并不是软件运行所必需的。安装软件时，*pacman* 会列出软件包的可选依赖，但这份列表不会记载在 `pacman.log` 中。请使用#查询包数据库命令查询软件包的可选依赖。
- 当你安装软件包以作为一些其他的软件包的（可选）依赖时（即没有明确要求安装时），推荐使用 `--asdeps` 选项。请参阅#安装原因一节了解详细内容。

警告：在 Arch 上安装软件包时，请避免在还没有更新系统前刷新同步软件包列表（例如，当官方软件仓库不再提供某个软件包时）。实际操作上，请使用 `pacman -Sy**u** *软件包名*`，而**不要**使用 `pacman -Sy *软件包名*`，因为后者可能会导致依赖问题。参见 系统维护#不支持部分升级一文和 BBS#89328 论坛讨论。

安装指定的包

要安装单个或者一系列软件包（包含软件包的依赖），请使用如下命令：

```
# pacman -S 包名_1 包名_2 ...
```

要通过正则表达式安装一系列软件包（参见这个论坛帖子）：

```
# pacman -S $(pacman -Ssq 包正则表达式)
```

有时软件包有多个版本，放在不同的仓库内（例如 *extra* 和 *testing*）。在以下示例中，要安装 *extra* 仓库的版本，需要在包名称前定义仓库名：

```
# pacman -S extra/包名
```

要安装多个含有相似名称的软件包，可以使用花括号扩展。例如：

```
# pacman -S plasma-{desktop,mediacenter,nm}
```

可以多层扩展到需要的层次：

```
# pacman -S plasma-{workspace{,-wallpapers},pa}
```

虚包

虚拟软件包是一个特殊的软件包，它本身并不存在，但由一或多个其它软件包提供。虚拟软件包允许其它软件包不以某一个特定的包为依赖，以应对有多个候选的情况。虚包不能用它们的名称安装，相反它们会在你安装*提供*虚包的软件包时被安装到你的系统中。一个例子是dbus-units软件包。

提示：当存在多个候选的情况下，可选包列表先按照`pacman.conf`中仓库出现的顺序排序，同一个仓库有多个可选再按字母顺序排序。

安装包组

一些包属于一个可以同时安装的软件包组。例如，运行下面的命令

```
$ pacman -S gnome
```

会提醒用户选择 `gnome` 内需要安装的包。

有的包组包含大量的软件包，有时用户只需其中几个。除了逐一键入序号外，pacman 还支持选择或排除某个区间内的的软件包：

```
Enter a selection (default=all): 1-10 15
```

这将选中序号 1 至 10 和 15 的软件包。而

```
Enter a selection (default=all): ^5-8 ^2
```

将会选中除了序号 5 至 8 和 2 之外的所有软件包。

想要查看哪些包属于 gnome 组，运行：

```
# pacman -Sg gnome
```

也可以访问 https://archlinux.org/groups/ 查看可用的包组。

注意：如果列表中的包已经安装在系统中，它会被重新安装，即使它已经是最新的。可以用 `--needed` 选项覆盖这种行为。

删除软件包

删除单个软件包，保留其全部已经安装的依赖关系

```
# pacman -R package_name
```

删除指定软件包，及其所有没有被其他已安装软件包使用的依赖关系：

```
# pacman -Rs package_name
```

警告：删除类似 *gnome* 这样的软件包组时，将会忽略组中软件包的安装原因，因为实际操作上执行的是逐一删除软件组的每一个软件，依赖软件包的安装原因不会被忽略。

上面这条命令在移除包含其他所需包的组时有时候会拒绝运行。这种情况下可以尝试

```
# pacman -Rsu package_name
```

要删除软件包和所有依赖这个软件包的程序:

警告：此操作是递归的，请小心检查，可能会一次删除大量的软件包。

```
# pacman -Rsc package_name
```

要删除一个被其他软件包依赖的软件包，但是不删除依赖这个软件包的其他软件包：

警告：此操作有破坏系统的能力，应该尽量避免使用。详情请看 避免某些 Pacman 命令。

```
# pacman -Rdd package_name
```

pacman 删除某些程序时会备份重要配置文件，在其后面加上*.pacsave扩展名。-n 选项可以避免备份这些文件：

```
pacman -Rn package_name
```

注意：*pacman* 不会删除软件自己创建的文件（例如主目录中的“点文件”不会被删除。）

升级软件包

警告：

- 建议用户遵守系统维护#更新系统的指导，定期更新系统，并不盲目地执行这些命令。
- Arch 只支持系统完整升级，详细参见系统维护#不支持部分升级和#安装软件包。

一个 *pacman* 命令就可以升级整个系统。花费的时间取决于系统有多老。这个命令会同步非本地(local)软件仓库并升级系统的软件包：

```
# pacman -Syu
```

查询包数据库

pacman 使用 `-Q` 参数查询本地软件包数据库， `-S` 查询同步数据库，以及 `-F`查询文件数据库。要了解每个参数的子选项，分别参见 `pacman -Q --help`, `pacman -S --help`和`pacman -F --help`。

pacman 可以在包数据库中查询软件包，查询位置包含了软件包的名字和描述：

```
$ pacman -Ss string1 string2 ...
```

有时，`-s`的内置正则会匹配很多不需要的结果，所以应当指定仅搜索包名，而非描述或其他子段:

```
$ pacman -Ss '^vim-'
```

要查询已安装的软件包：

```
$ pacman -Qs string1 string2 ...
```

按文件名查找软件库：

```
$ pacman -F string1 string2 ...
```

显示软件包的详尽的信息：

```
$ pacman -Si package_name
```

查询本地安装包的详细信息：

```
$ pacman -Qi package_name
```

使用两个 `-i` 将同时显示备份文件和修改状态：

```
$ pacman -Qii package_name
```

获取已安装软件包所包含文件路径（比如用来查看软件包提供了什么可执行文件）：

```
$ pacman -Ql package_name
```

查询远程库中软件包包含的文件：

```
$ pacman -Fl package_name
```

检查软件包安装的文件是否都存在：

```
$ pacman -Qk package_name
```

两个参数`k`将会执行一次更彻底的检查。

查询数据库获取某个文件属于哪个软件包：

```
$ pacman -Qo /path/to/file_name
```

查询文件属于远程数据库中的哪个软件包：

```
$ pacman -F /path/to/file_name
```

要罗列所有不再作为依赖的软件包(孤立orphans)：

```
$ pacman -Qdt
```

要罗列所有明确安装而且不被其它包依赖的软件包：

```
$ pacman -Qet
```

更多例子查看pacman tips。

Pactree

注意：pactree(8) 不再是 pacman包 的一部分。它现在在 pacman-contrib包 中。

要显示软件包的依赖树：

```
$ pactree package_name
```

要检查一个*已安装*的软件包被哪些包依赖，可以将递归标识 `-r` 传递给 *pactree*，或者使用 pkgtoolsAUR 中的 *whoneeds*

数据库结构

pacman 数据库通常位于 `/var/lib/pacman/sync`。对于每一个在 `/etc/pacman.conf` 中指定的软件仓库， 这里都有一个一致的数据库。数据库文件夹里每个 tar.gz 文件都包含着一个仓库的软件包信息。例如 which包 包:

```
$ tree which-2.21-5
which-2.21-5
|-- desc<br>```
```

这个 `depends` 项列出了该软件的依赖包， 而 `desc` 有该包的介绍，例如文件大小和MD5值 。

清理软件包缓存

pacman 将下载的软件包保存在 `/var/cache/pacman/pkg/` 并且不会自动移除旧的和未安装版本的软件包。这样做有一些好处：

1. 这样允许[降级](../包管理/降级软件包.md)软件包而不需要通过其他方式提取旧版本，例如 [Arch Linux Archive](../../00_关于与社区/Arch%20项目/Arch%20Linux%20Archive.md).
2. 被卸载的软件包可以轻易地直接从缓存文件夹重新安装，不需要重新从软件仓库下载。

然而，需要定期手动清理缓存来避免该文件夹无限制增大。

[pacman-contrib](https://archlinux.org/packages/?name=pacman-contrib)包 提供的 [paccache(8)](https://man.archlinux.org/man/paccache.8) 脚本默认会删除所有缓存的版本和已卸载的软件包，除了最近的3个会被保留：
启用和启动 `paccache.timer`来每周删除不使用的包。

> **提示：**可以使用[#钩子](#钩子)自动执行清理，安装 [paccache-hook](https://aur.archlinux.org/packages/paccache-hook/)AUR 并参考。

也可以自己设置保留最近几个版本：
添加`-u`/`--uninstalled`开关来限制*paccache*的行为只作用于卸载的包。例如清理所有卸载的包的缓存版本，可以用以下命令：
更多参数参见`paccache -h`。

*pacman*也有一些内建参数用于清除缓存和那些不再在`/etc/pacman.conf`配置文件中列出的软件仓库残留数据库文件。然而*pacman*并不提供保留一定数量的过去版本的功能，因此它比*paccache*的默认选项更加激进。

要删除目前没有安装的所有缓存的包，和没有被使用的同步数据库，执行：

要删除缓存中的全部文件，使用两次`-c`开关。这是最为激进的方式，将会清空缓存文件夹：
> **警告：**应当避免从缓存中删除所有过去版本和卸载的包，除非需要更多磁盘空间。这样会导致无法降级或重新安装软件包时需要再次下载。

[pkgcacheclean](https://aur.archlinux.org/packages/pkgcacheclean/)AUR以及[pacleaner](https://aur.archlinux.org/packages/pacleaner/)AUR是两个进一步清理缓存的替代工具

其它命令

升级系统时安装其他软件包：
下载包而不安装它：
安装一个**本地**包(不从源里下载)：
要将本地包保存至缓存，可执行：
安装一个**远程**包（不在 *pacman* 配置的源里面）：
查看软件变动

pacman 会列出需要安装和删除的软件，并在执行动作前要求需要的权限。

要抑制`-S`，`-U` 和 `-R`，的动作并列出涉及该动作相关软件的列表，可以使用`--print`或缩写`-p`。

可以添加`--print-format` 参数将输出格式化，例如使用`--print-format %n`将返回不带有版本号的列表。

安装原因

pacman 数据库将软件包按照安装的原因分为两类：

- **显式安装**：那些真正地被传递给通用*pacman*`-S`和`-U`命令的包；
- **依赖**：那些虽然（一般）从未被传递给*pacman*安装命令，但由于被其它显式安装的包需要从而被隐式安装的包

当安装软件包时，可以把安装原因强制设为**依赖**：
该命令使用的原因通常是显式安装的包可能会提供[可选安装包](../包开发/PKGBUILD.md#optdepends)，这些包提供了非必须功能，可供用户自由选择。

> **提示：**用`--asdeps`安装可选依赖将确保如果你[移除孤立包](Pacman_提示和技巧.md#删除未使用的软件包（孤立软件包）)，*pacman*将会一同移除按照上述方法指定安装的可选依赖。

但是重新安装该软件包，当前安装原因默认会被保留。

显式安装的软件包列表可用`pacman -Qe`获取，设置为依赖的软件包可用`pacman -Qd`获取。

改变某个已安装软件包的安装原因，可以执行：
反过来的对应参数`--asexplicit`

> **注意：**在升级时使用`--asdeps`和`--asexplicit`选项，例如`pacman -Syu *package_name* --asdeps`，是不被推荐的。这会导致不仅改变要被安装的软件包的安装原因，也会改变被升级的软件包的安装原因。

查询一个包含具体文件的包名

同步文件数据库：
查询包含某个文件的包名，比如：
core/pacman 5.0.1-4 usr/bin/pacman usr/share/bash-completion/completions/pacman extra/xscreensaver 5.36-1 usr/lib/xscreensaver/pacman

> **提示：** 可以[启用/启动](https://wiki.archlinuxcn.org/wzh/Help:%E9%98%85%E8%AF%BB#控制_systemd_单元) `pacman-filesdb-refresh.timer`（该定时器在[pacman-contrib](https://archlinux.org/packages/?name=pacman-contrib)包中）来每周同步文件信息数据库。

如果需要高级功能请安装 [pkgfile](../包管理/Pkgfile.md)，它使用一个单独的数据库来保存文件和它们所关联的软件包的信息。

安装/更新/删除 软件包时发生了什么

动作成功执行时，事务包含五大步骤和事务钩子程序：

1. 如果数据库未被锁定，初始化事务
2. 确认事务中要添加或删除的软件包
3. 准备事务，根据参数，对同步数据库、软件包和依赖关系进行校验
4. 提交事务：
    1. 如果需要，下载软件包 (`_alpm_sync_load`)
    2. 如果有可以执行的 *pacman* `PreTransaction` 钩子，执行这些钩子.
    3. 被替换、出现冲突或被指定删除的软件包将会被删除
    4. 如果需要添加软件包，每个软件包都会执行提交
        1. 如果有安装脚本，执行 `pre_install` 函数(更新时执行 `pre_upgrade`，删除时执行 `pre_remove`).
        2. *pacman* 删除软件包之前版本的所有文件 (在更新或删除软件包时)。配置文件将会被保留 (请参阅 [Pacman/Pacnew and Pacsave](Pacman_pacnew%20与%20pacsave.md)).
        3. *pacman* 将软件包文件解压到系统(安装或升级软件包时)。会覆盖旧的已被手动修改的配置文件时，配置文件会命名为 (.pacnew).
        4. 如果软件包有安装脚本，执行 `post_install` 函数(更新时执行 `post_upgrade`，删除时执行 `post_remove`).
    5. 如果*pacman* `PostTransaction` 钩子可执行，执行这些钩子.
5. 释放事务及事务资源 (例如，数据库锁)

配置

pacman 的配置文件位于`/etc/pacman.conf`。 [man pacman.conf](https://archlinux.org/pacman/pacman.conf.5.html) 可以查看配置文件的进一步信息。

通用选项

通用选项都在`[options]`段。阅读 man 手册或者查看默认的 pacman.conf 可以获得有关信息和用法。

升级前对比版本

要查看旧版和新版的有效安装包，请取消`/etc/pacman.conf`中"VerbosePkgLists"的注释。修改后的`pacman -Syu`输出如下：

Package (6) Old Version New Version Net Change Download Size

extra/libmariadbclient 10.1.9-4 10.1.10-1 0.03 MiB 4.35 MiB extra/libpng 1.6.19-1 1.6.20-1 0.00 MiB 0.23 MiB extra/mariadb 10.1.9-4 10.1.10-1 0.26 MiB 13.80 MiB

启用并行下载

Pacman 6.0 增加了并行下载选项，将 `/etc/pacman.conf` 中 `[options]` 选项 `ParallelDownloads` 设置成正整数，例如 `5`，将会同时下载 5 个软件包，如果未设置此选项，软件包将会被依次下载。

在升级时跳过软件包

> **警告：**在跳过软件包时要小心，因为[部分升级不受支持](../../02_系统管理与内核/系统管理/系统维护.md#不支持部分升级)

要想在[升级](#升级软件包)系统时跳过特定的软件包，用像如下的命令指明：
IgnorePkg=linux

多软件包可以用空格隔开，或者用另外的`IgnorePkg`行。也可使用 [glob](https://en.wikipedia.org/wiki/glob_(programming)) 模式。如果只打算忽略一次升级，可以使用 `--ignore` 选项，这时使用逗号隔开的列表。

忽略了的软件包依然可通过 `pacman -S` 升级。这种情况下*pacman*会提醒你这些软件包已经被包含在`IgnorePkg`声明中。

在升级时跳过软件包组

> **警告：**在跳过软件包时要小心，因为[部分升级不受支持](../../02_系统管理与内核/系统管理/系统维护.md#不支持部分升级)

和软件包一样，也可以不升级某个软件包组：
IgnoreGroup = gnome

在升级时跳过文件

所有在`NoUpgrade`指令中列出的文件都会在软件包被安装/升级时不会被更改，并且新文件会以带有*.pacnew*后缀名的形式安装
NoUpgrade=path/to/file

存在多个跳过文件也可以像这样指定：

NoUpgrade=path/to/file1 path/to/file2

> **注意：**这个路径指软件包中的文件，所以不要包括开头的斜线。

在安装时跳过文件

要总是跳过某些文件夹的安装，可以将它们放到 `NoExtract` 中，例如不想安装 [systemd](../初始化/Systemd.md) 模块：
NoExtract=usr/lib/systemd/system/*

后面的规则覆盖前面的规则，加上 `!` 可以取消跳过效果。

> **提示：***pacman*会在更新locales已经被*localepurge*或者*bleachbit*清除的包时发出警告。注释掉`pacman.conf`中的`CheckSpace`参数能够抑制这种警告，但是要注意space-check功能将会对所有软件包禁用。

保留多个配置文件

如果你有多个配置文件（比如，主配置和启用了[测试仓库](../../00_关于与社区/关于%20Arch/官方仓库.md#testing_仓库)的配置文件），需要共享一些设置，你可以在配置文件中使用`Include`选项，例如：
Include = /path/to/common/settings

`*/path/to/common/settings*`文件中是两个配置文件共享的相同配置。

钩子

*pacman*可以在处理前后，运行`/usr/share/libalpm/hooks/`文件夹下的hooks，更多的hooks文件夹可以通过`HookDir`选项在`pacman.conf`中指明，默认`/etc/pacman.d/hooks`。Hook文件必须以*.hook*为后缀。Pacman hooks不是交互式的。

pacman hooks用于，比如说，和`systemd-sysusers`和`systemd-tmpfiles`结合来在安装包时自动创建系统用户和文件。例如，[tomcat8](https://archlinux.org/packages/?name=tomcat8)包指明它想要创建一个叫做`tomcat8`的系统用户和一些属于该用户的文件。当*pacman*确定[tomcat8](https://archlinux.org/packages/?name=tomcat8)包包含文件指明用户和临时文件时，*pacman* hooks `systemd-sysusers.hook`和`systemd-tmpfiles.hook`唤起`systemd-sysusers`和`systemd-tmpfiles`。

有关 alpm hooks 的更多信息，参见[alpm-hooks(5)](https://man.archlinux.org/man/alpm-hooks.5)。

软件仓库

除了特殊的[通用选项](#通用选项)节，每个`pacman.conf`中的`[section]`都定义了一个使用的软件包仓库，*仓库*是多个软件包的*逻辑*上的集合，他们*物理*上存储在一个或多个服务器：这也是为什么每一个服务器都叫做这个仓库的*镜像*。

仓库区分为[官方仓库](../../00_关于与社区/关于%20Arch/官方仓库.md)与[非官方仓库](../非官方用户仓库/非官方用户仓库.md)。配置文件中仓库的顺序十分重要；当几个仓库出现同名安装包，不管版本号如何，*pacman*将使用配置文件中排前的仓库。要在添加后使用某个仓库，你需要先[升级](#升级软件包)整个系统

每个仓库节都可以直接指定镜像列表或者`Include`引用其他的文件：例如，官方镜像引用了`/etc/pacman.d/mirrorlist/`。具体查看[Mirrors](../../00_关于与社区/关于%20Arch/镜像源.md)。

软件包的缓存目录

在`pacman.conf`[通用选项](#通用选项)节，`CacheDir`指定了Pacman下载的安装包缓存位置(如果未设置，默认位置为 `/var/cache/pacman/pkg/`)。

即使只保留最新版本的安装包，缓存目录大小仍然有可能随着时间增长。

如果要将缓存目录移动到更方便的地方，只需要完成下列方法中的任意一个：

- 在`pacman.conf`[通用选项](#通用选项)节，设置`CacheDir`为要使用的缓存目录，并确保目录以"/"结尾。**推荐使用该方法**.
- 将一个专用分区挂载到默认缓存点`/var/cache/pacman/pkg/`，例如使用[Btrfs 子卷](../../02_系统管理与内核/文件系统/Btrfs.md#子卷)。
- 将一个目录与 `/var/cache/pacman/pkg/`绑定挂载。

> **警告：****不要使用符号链接** 将 `/var/cache/pacman/pkg/` 目录指向其它位置。 **这会导致pacman出现异常**，特别是当pacman尝试升级自己时。

软件包的安全性

pacman 支持软件包签名，会为软件包提供额外的安全性。默认配置，`SigLevel = Required DatabaseOptional` 将启用全局签名验证，但会被每个软件仓库的 `SigLevel` 行所覆盖。有关软件包签名和签名验证的更多细节，参见 [pacman-key](Pacman_软件包签名.md)。

疑难解答

"Failed to commit transaction (conflicting files)" 错误

如果碰到如下错误：

error: could not prepare transaction error: failed to commit transaction (conflicting files) package: /path/to/file exists in filesystem Errors occurred, no packages were upgraded.

这是因为 *pacman* 检测到文件冲突，而且按照设计，*pacman* 不会覆盖文件。这是设计功能，不是缺陷。

这个问题通常很容易解决（当然你首先应该尝试搞清楚这些文件是怎么变成这样的）。一个安全的方式是，首先检查是否有另一个软件包提供了这个文件 (`pacman -Qo */path/to/file*`)。如果文件被另一个软件包所有，请[报告问题](../../00_关于与社区/Arch%20开发/漏洞报告准则.md)。如果不是其它软件包提供，将 'exists in filesystem' 的文件重命名并重新运行update命令。如果一切顺利，可以删掉这个文件。

如果你手动安装了一个软件，没有通过*pacman*，例如用`make install`，你必须将软件连同它的全部文件一起移除/卸载。参见[Pacman/提示和技巧#查找不属于任何软件包的文件](Pacman_提示和技巧.md#查找不属于任何软件包的文件)。

每一个安装的软件包都会提供一个 `/var/lib/pacman/local/*$package-$version*/files` 文件，包含此软件包的元数据。如果文件损坏或者丢失，将会导致升级时出现`file exists in filesystem` 错误。此错误通常只会影响一个软件包，除了手动删除或移动所有的问题文件，你也可以显式地使用`pacman -S --overwrite *glob package*`让 *pacman* 强制覆盖匹配`*glob*`的文件。

> **警告：**尽量避免使用`--overwrite`开关。参见[System maintenance#避免某些 pacman 命令](../../02_系统管理与内核/系统管理/系统维护.md#避免某些_pacman_命令)

"Failed to commit transaction (invalid or corrupted package)" 错误

看看`/var/cache/pacman/pkg`中是否有`*.part`结尾的文件，它们是没有完全下载的文件，删除它们并重新执行更新。这些程序一般是自定义的`XferCommand` 下载命令造成的。
同样的错误也可能是因为*archlinux-keyring*过期所致，阻止了*pacman*验证签名。详情见[Pacman/Package signing#Upgrade system regularly](Pacman_软件包签名.md#Upgrade_system_regularly)以修复并避免在未来遇到同样问题。

"Failed to init transaction (unable to lock database)" 错误

pacman 在改变软件包数据库前，比如安装软件包时，会创建一个文件锁 `/var/lib/pacman/db.lck`。这会阻止其他 *pacman* 实例在同一时间修改软件包数据库。

如果 *pacman* 在更新数据库时被打断，旧锁可能保留下来。如果确认没有 *pacman* 实例在运行，那么删掉文件锁：
> **提示：**以root身份运行`fuser /var/lib/pacman/db.lck`可以确认是否有其它进程仍在使用它。

安装时无法获取软件包

错误内容包含：`Not found in sync db`, `Target not found` 或 `Failed retrieving file`.

首先确认软件包确实存在（并注意错别字）。如果确认软件包存在，可能本地数据库过时了或者软件仓库没有配置好，试试 `pacman -Syyu` 强制数据库更新和升级。

也有可能包含该软件包的软件仓库没有启动。例如，该软件包可能在 *multilib* 仓库里，但该仓库没有在 *pacman.conf* 中启用。

参阅[常见问题#Q) 在官方仓库中，为什么只为每个共享链接库提供一个版本？](../../00_关于与社区/关于%20Arch/常见问题.md#Q)_在官方仓库中，为什么只为每个共享链接库提供一个版本？).

pacman 更新时崩溃

如果 *pacman* 在删除、重新安装或更新软件包时 "数据库写入" 出错：

1. 从 Arch 安装媒体启动，最好用最新的安装媒体
2. 挂载根文件系统，例如以 root 身份`mount /dev/sdaX /mnt`，通过 `df -h` 确认根文件目录包含足够的空间

3. 同样挂载 proc, sys 和 dev 文件系统：`mount -t proc proc /mnt/proc; mount --rbind /sys /mnt/sys; mount --rbind /dev /mnt/dev`
4. 如果系统使用默认的数据库目录位置，可以通过 root 用户执行下面命令`pacman --root=/mnt --cachedir=/mnt/var/cache/pacman/pkg -Syu`来更新系统的 *pacman* 数据库。
5. 如果无法更新，可以[重新安装所有软件包](Pacman_提示和技巧.md#重新安装所有软件包)。
6. 更新之后，可以通过下面命令确认是否还存在损坏的包：`find /mnt/usr/lib -size 0`
7. 通过`pacman --root /mnt --cachedir=/mnt/var/cache/pacman/pkg -S *package*`重新安装依然损坏的软件包

pacman: command not found

如果`/var/cache/pacman/pkg`是一个符号链接，*pacman*在升级自身时会创建该目录并移除符号链接，这会导致升级失败，并产生`/usr/bin/pacman` 与其它 [pacman](https://archlinux.org/packages/?name=pacman)包包中内容丢失的后果。

永远不要为`/var/cache/pacman/pkg`创建软连接，因为该目录受*pacman*控制。使用`CacheDir`选项或通过绑定挂载。参见[#软件包的缓存目录](#软件包的缓存目录)。

如果你已经遇到这个问题，你可以手动提取安装包中`/usr`的内容恢复，并重新安装*pacman*。详情见[FS#73306](https://bugs.archlinux.org/task/73306)和[论坛相关话题](https://bbs.archlinux.org/viewtopic.php?id=241213)

手动重新安装 pacman

使用 pacman-static

[pacman-static](https://aur.archlinux.org/packages/pacman-static/)AUR是*pacman*的静态编译版本，因此系统中的库文件失效时依然可以运行它。当[部分升级](../../02_系统管理与内核/系统管理/系统维护.md#不支持部分升级)后*pacman*无法运行，静态编译版本就能够派上用场。

置顶评论和PKGBUILD提供了直接下载二进制文件的方法，可以用于重新安装*pacman*或在部分升级情况下升级整个系统。

使用外部 pacman

如果`pacman-static`也无法使用，考虑使用外部*pacman*。 最简单的办法是使用[archiso](../../00_关于与社区/Live%20Arch%20系统/Archiso.md)并指定`--sysroot`或`--root`参数到系统操作的挂载点。参见[Chroot#使用 arch-chroot](../../02_系统管理与内核/系统恢复/Chroot.md#使用_arch-chroot)来挂载`--sysroot`参数需要的文件系统。

手动提取

> **警告：**这种方式极其容易破坏你的系统，把它变得更糟。这个方式只是作为最后的措施，以防万一[#pacman更新时崩溃](#pacman_更新时崩溃)不起作用

即使 *pacman* 严重损坏，你仍可以手动修复它，这需要下载最新的包并把它们解压到正确的目录。大致步骤如下：

1. 确定要安装的[pacman](https://archlinux.org/packages/?name=pacman)包依赖项
2. 从你选择的mirror下载每个包
3. 把每个包解压到 root
4. 使用`pacman -S --overwrite`重新安装这些包来更新相应的软件包数据库
5. 进行完整系统更新

如果你拥有一个健康的 Arch 系统，你可以这样查看依赖项的完整列表：

```
$ pacman -Q $(pactree -u pacman)
```

但是依据你遇到的问题，你可能只需要更新其中的几个。解压一个包的示例是：

注意`w`参数表示交互模式。以非交互方式运行是非常冒险的，因为你可能就此覆盖了一个重要文件。你也要注意以正确的顺序解压软件包（例如先解压依赖）。 [这个帖子](https://bbs.archlinux.org/viewtopic.php?id=95007)包含了一些这个过程的例子，这些例子中只有少量*pacman*依赖项损坏。

升级系统重启后，出现"unable to find root device"错误，无法登录

很有可能 [initramfs](../启动过程/Arch%20的启动流程.md) 在[内核](../../02_系统管理与内核/内核/内核.md)升级时损坏（例如由错误地使用 *pacman* 的 `--overwrite` 选项导致）。有两个选择；首先尝试 *Fallback*启动项。

> **提示：**万一你删除了 *Fallback* 启动项，你总可以在启动加载器菜单显示出来的时候按`Tab`键（对于 Syslinux）或者按`e`键（对于 GRUB 或者 systemd-boot），将它重命名为`initramfs-linux-fallback.img`然后按`Enter`或者`b`（取决于你的启动加载器以新参数启动的方式）

一旦系统启动，从控制台或终端（为备用[linux](https://archlinux.org/packages/?name=linux)包内核）运行这条命令来重建 initramfs 映像：
如果上面方法不行，从一个最新的 Arch 发行版本（CD/DVD 或者 U盘），分别[挂载](../../02_系统管理与内核/文件系统/文件系统.md)你的根目录和启动分区到`/mnt`和`/mnt/boot`，然后用 *arch-chroot* [chroot](../../02_系统管理与内核/系统恢复/Chroot.md)：
> **注意：**
>
> - 如果你没有一个当前的发行或者你只有某些其他的 "live" Linux 发行版，你可以用老式的方法[chroot](../../02_系统管理与内核/系统恢复/Chroot.md)。显然，这种方式比简单地运行`arch-chroot`脚本需要更多的输入
> - 如果 *pacman* 伴随`Could not resolve host`运行失败，请[检查你的网络连接](../../03_网络与服务/网络/网络配置.md#检查连接)
> - 如果无法进入 arch-chroot 或 chroot 环境，但是需要重新安装软件包，可以使用 `pacman --sysroot /mnt -Syu foo bar` 从而在你的根分区下使用 *pacman*

重新安装内核([linux](https://archlinux.org/packages/?name=linux)包 软件包)将会自动运行 `mkinitcpio -p linux` 重新生成 initramfs 镜像，不需要单独生成。

之后建议执行 `exit`, `umount /mnt/{boot,}` 然后 `reboot`.

'warning: current locale is invalid; using default "C" locale' 错误

错误信息已经很明确了，locale 设置不正确，请阅读[Locale](../../00_关于与社区/本地化/Locale.md)进行设置。

Pacman 不使用我的代理设置

正确设置环境变量（`$http_proxy`, `$ftp_proxy` 等）。如果使用 [sudo](../../08_安全与加密/安全/Sudo.md)，需要让 sudo [将这些变量传递给 pacman](../../08_安全与加密/安全/Sudo.md#环境变量_variables)。另外，在更新密钥前确保`/etc/pacman.d/gnupg/dirmngr.conf`路径的 [dirmngr](../../03_网络与服务/邮件/GnuPG.md#公钥服务器)配置文件`honor-http-proxy`变量设置正确。

如何重装所有包并保留安装和依赖信息?

重装所有软件包：`pacman -S $(pacman -Qnq)`（`-S` 选项会保留安装缘由）。

接着需要重装外来包（不在官方仓库里的软件包）。外来包可通过`pacman -Qmq`查看。

"Cannot open shared object file" 错误

有可能是前一次 *pacman* 事务中删除或者损坏了 pacman 自身需要的一些共享库。

要修复这种情况，你需要手动解压那些共享库文件到文件系统中合适的地方。首先请查明哪个包中包含有所需的库文件，然后在 *pacman* 缓存中 (`/var/cache/pacman/pkg/`) 找到对应的包。解压出需要的库文件到文件系统的对应位置，如此可以继续使用 *pacman* 。

之后需要重新安装损坏的软件包。注意你可能需要使用 `--overwrite` 选项覆盖掉你刚刚解压出的系统文件，因为 *pacman* 可能没有跟踪到对应的文件信息。 *pacman* 随后会使用包中的文件正确替换掉共享库文件。如此能修好您的 pacman ，然后请继续更新系统中剩下的软件包。

软件包下载停滞

一些网络问题可能会报告阻止了 *pacman* 更新和同步仓库。 在实体机中安装 Arch Linux 时，这些问题已经通过替换默认的 pacman 文件下载器得到了解决 (更多细节请参阅 [Pacman/提示和技巧#性能](Pacman_提示和技巧.md#性能)) 。当在 VirtualBox 中安装 Arch Linux 作为客户操作系统时，这个问题也通过在主机属性中使用主机界面而不是 *NAT* 来解决。

无法从镜像服务器获取 'core.db'

如果从正确的[镜像服务器](../../00_关于与社区/关于%20Arch/镜像源.md)收到此错误，请换一个[域名解析服务器](../../03_网络与服务/DNS/域名解析.md)。

error: 'local-package.pkg.tar': permission denied

如果你想要在一个 sshfs 上使用`pacman -U`安装软件包并遇到了这个错误，将包移动到本地目录，再试一次。

error: could not determine cachedir mount point /var/cache/pacman/pkg

在 chroot 环境中执行 `pacman -Syu` 可能发生此错误:
error: could not determine cachedir mount point /var/cache/pacman/pkg error: failed to commit transaction (not enough free disk space)

通常是因为进入 chroot 时，进入的目录不是挂载点。解决方法请参考[从现有 Linux 发行版安装 Arch Linux#下载基本工具](../安装过程/从现有%20Linux%20发行版安装%20Arch%20Linux.md#下载基本工具)， [arch-chroot(8)](https://man.archlinux.org/man/arch-chroot.8) 解释问题的原因，并提供了一个通过 bind mount 解决问题的方法。

error: GPGME error: No data

如果你无法更新包，并遇到了该问题，尝试

`rm -r /var/lib/pacman/sync/`

后再次更新。

如果删除sync中的同步文件不能解决问题，请通过`file /var/lib/pacman/sync/*`检查同步文件是否为`gzip compressed data`，使用路由或代理可能会损坏下载的文件。

如果同步文件是正确的类型，那么问题可能出在镜像源上。使用`pacman-conf -r core`和`pacman-conf -r extra`查询使用的镜像源，将命令输出的第一个URL粘贴到浏览器访问，检查返回的文件。如果返回有问题，在{ic|/etc/pacman.d/mirrorlist}}里修改为其他可用镜像源。

另请参见

- [Pacman 主页](https://pacman.archlinux.page/)
- [libalpm(3)](https://man.archlinux.org/man/libalpm.3)
- [pacman(8)](https://man.archlinux.org/man/pacman.8)
- [pacman.conf(5)](https://man.archlinux.org/man/pacman.conf.5)
- [repo-add(8)](https://man.archlinux.org/man/repo-add.8)
- [AUR](../../00_关于与社区/关于%20Arch/Arch%20用户软件仓库.md)

8.6 Pacman/提示和技巧

难度 高级 | **篇幅** 49.6 KB | **所属** 包管理器

本节将讲到

- 维护
- 安装和修复
- 性能
- 工具

通用优化请参考核心工具和 Bash。

维护

注意：您也可以使用 `grep -Fxf` 或 `grep -Fxvf`，而不是像下面的章节中一样使用 *comm*（需要排序输入和 *sort*）。

另见系统维护。

查询软件包

未使用的软件库

默认情况下，`pacman.conf` 中列出的软件库用于同步、搜索、安装和升级。可以通过更改配置来增加灵活性，例如仅使用某些软件库进行搜索：

```
/etc/pacman.conf
...
[multilib]
Usage = Sync Search
...
```

参见 pacman.conf(5) § REPOSITORY SECTIONS。

带有版本

您可能希望获得已安装的软件包及其版本的列表，这在报告错误或讨论已安装的软件包时非常有用。

- 所有明确安装的软件包：`pacman -Qe`。
- 列出 `*group*` 软件包组中的所有软件包：`pacman -Sg *group*`。
- 所有外部软件包 (通常是手动下载安装，或者已经从数据库中删除)：`pacman -Qm`。
- 列出所有从数据库中安装的软件包：`pacman -Qn`。
- 所有明确安装的，存在于数据库而且不是直接或可选依赖的软件包：`pacman -Qent`。
- 按正则表达式查询软件包：`pacman -Qs *regex*`。

- 按正则表达式查询软件包，自定义输出格式（需要 expac包）：`expac -s "%-30n %v" *regex*`。
- 从数据库查询软件包，（如如需iwd包）：`pacman -Ss iwd`

带有大小

在尝试释放硬盘上的空间时，弄清楚哪些软件包最大可能非常有用。这里有两个选项：获取单个包的大小，或者获取包的大小及其依赖关系。

单个包

用下面命令查看单个软件包的大小:

```
$ LC_ALL=C.UTF-8 pacman -Qi | awk '/^Name/{name=$3} /^Installed Size/{print $4$5, name}' | LC_ALL=C.UTF-8 sort -h
```

软件包和依赖项

要列出软件包及其依赖项，

- 安装 expac包 并运行 `expac -H M '%m\t%n' | sort -h`。
- 运行 pacgraphAUR 带 `-c` 选项。

查询软件包的下载大小（不加 `*packages*` 将查询所有软件包）：

```
$ expac -S -H M '%k\t%n' packages
```

要列出不属于元软件包 base包 或软件包组 xorg包组 中的明确安装的包，带大小和描述：

```
$ expac -H M "%011m\t%-20n\t%10d" $(comm -23 <(pacman -Qqen | sort) <({ pacman -Qqg xorg; expac -l '\n' '%E' base; } | sort -u)) | sort -n
```

要列出标记为可升级的软件包及其下载大小：

```
$ expac -S -H M '%k\t%n' $(pacman -Qqu) | sort -sh
```

要仅列出可选依赖：

```
$ expac -S "%o" package
```

按日期查询

用 expac包 查询最近安装的 20 个软件包：

```
$ expac --timefmt='%Y-%m-%d %T' '%l\t%n' | sort | tail -n 20
```

或者，用从 epoch(1970-01-01 UTC) 开始的秒数：

```
$ expac --timefmt=%s '%l\t%n' | sort -n | tail -n 20
```

不在指定的包组、软件库或元包中

注意：要获取作为依赖项安装但已没有任何包依赖它的包的列表，见#删除未使用的软件包（孤立软件包）.

列出不在 base包 元软件包中的明确安装的软件包：

```
$ comm -23 <(pacman -Qqe | sort) <(expac -l '\n' '%E' base | sort)
```

列出不在 base包 元软件包或 xorg包组 软件包组中的明确安装的软件包：

```
$ comm -23 <(pacman -Qqe | sort) <({ pacman -Qqg xorg; expac -l '\n' '%E' base; } | sort -u)
```

列出所有没有任何软件包依赖它的已安装软件包，并且这些软件包不在 base包 元软件包或 xorg包组 软件包组中：

```
$ comm -23 <(pacman -Qqt | sort) <({ pacman -Qqg xorg; echo base; } | sort -u)
```

同上，但带描述：

```
$ expac -H M '%-20n\t%10d' $(comm -23 <(pacman -Qqt | sort) <({ pacman -Qqg xorg; echo base; } | sort -u))
```

列出不在指定软件库 *repo_name* 中的所有已安装软件包（可以同时检查多个软件库）：

```
$ comm -23 <(pacman -Qq | sort) <(pacman -Sql repo_name | sort)
```

列出在指定软件库 *repo_name* 中的所有已安装软件包（可以同时检查多个软件库）：

```
$ comm -12 <(pacman -Qq | sort) <(pacman -Sql repo_name | sort)
```

列出 Arch Linux ISO 上不在 base包 元软件包中的所有包：

```
$ comm -23 <(curl https://gitlab.archlinux.org/archlinux/archiso/-/raw/master/configs/releng/packages.x86_64) <(expac -l '\n' '%E' base | sort)
```

提示：或者，使用 moreutils包 包中的 `combine`（而不是 `comm`），该命令的语法更容易记住。见 combine(1)。

开发版软件包

要列出所有开发版/不稳定软件包，请运行：

```
$ pacman -Qq | grep -Ee '-(bzr|cvs|darcs|git|hg|svn)$'
```

软件包的依赖项

要获得包的依赖项列表，最简单的解决方案是读取以下命令的输出：

```
$ pacman -Qi package
```

要实现自动化，而不是分析 pacman 输出这种容易出错的方法，使用 expac包：

```
$ expac -S '%D' package
```

带有可选依赖

要列出明确安装的软件包及其可选依赖，请运行：

```
$ LC_ALL=C.UTF-8 pacman -Qei | sed '/^[^NO ]/d;/None$/d' | awk 'BEGIN{RS=ORS="\n\n";FS=OFS="\n\\S"} /Optional Deps/ {print $1"\nO"$2}'
```

或者，使用 expac包：

```
$ expac -d '\n\n' -l '\n\t' -Q '%n\n\t%O' $(pacman -Qeq)
```

要列出它们并忽略已安装的可选依赖，请运行：

```
$ LC_ALL=C.UTF-8 pacman -Qei | sed '/^[^NO ]/d;/None$/d' | awk 'BEGIN{RS=ORS="\n\n";FS=OFS="\n\\S"} /Optional Deps/ {print $1"\nO"$2}' | sed 's/^Optional Deps   :///;/\[installed\]$/d;s/\s\+/ /'
```

查看特定打包者打包的所有软件包

这里用到了 gawk包。

```
$ pacman -Qi | awk -F': ' '/^名字/ {name=$2} /^打包者/ && $2 ~ /特定打包者/ {print name}'
```

浏览软件包

要浏览所有已安装软件包并即时预览每个软件包：

```
$ pacman -Qq | fzf --preview 'pacman -Qil {}' --layout=reverse --bind
'enter:execute(pacman -Qil {} | less)'
```

这使用 fzf（英语：fzf） 来显示一个两窗格视图，其中列出了所有软件包，包信息显示在右侧。

输入字母以过滤包列表；使用箭头键（或 `Ctrl-j`/`Ctrl-k`）浏览；按 `Enter` 在 *less* 下查看包信息。

要使用 fzf 以类似的方式浏览当前已知的所有 *pacman* 软件包（已安装和尚未安装），请使用：

```
$ pacman -Slq | fzf --preview 'pacman -Si {}' --layout=reverse
```

浏览的按键绑定是相同的，不过 `Enter` 不能同样工作。

列出软件包拥有的文件并带大小

如果您发现某个特定的包使用了大量空间，并且您想知道哪些文件占了其中的大部分，那么这个命令可能会很有用。

```
$ pacman -Qlq package | grep -v '/$' | xargs -r du -h | sort -h
```

查找不属于任何软件包的文件

如果您的系统中有不属于任何软件包的零散文件（通常是由于您没有使用包管理器安装软件），您可能需要查找这些文件以清理它们。

一种方法是列出所有感兴趣的文件，并对照 *pacman* 进行检查：

```
# (export LC_ALL=C.UTF-8; comm -13 <(pacman -Qlq | sed 's,/$,,' | sort)
<(find /etc /usr /opt -path /usr/lib/modules -prune -o -print | sort))
```

提示：lostfiles包 脚本执行类似的步骤，但还包括一个广泛的黑名单，以从输出中删除常见的误报。

跟踪软件包创建的无主文件

多数系统将通过通常的操作过程慢慢的收集一些 ghost 文件，如状态文件、日志、索引等。

pacutils包 中的 `pacreport` 可用于通过 `/etc/pacreport.conf` 跟踪这些文件及其关联（见 pacreport(1) § FILES）。

一个例子可能看起来像这样（摘要）：

```
/etc/pacreport.conf
[Options]
IgnoreUnowned = usr/share/applications/mimeinfo.cache

[PkgIgnoreUnowned]
alsa-utils = var/lib/alsa/asound.state
bluez = var/lib/bluetooth
ca-certificates = etc/ca-certificates/trust-source/*
dbus = var/lib/dbus/machine-id
glibc = etc/ld.so.cache
grub = boot/grub/*
linux = boot/initramfs-linux.img
pacman = var/lib/pacman/local
update-mime-database = usr/share/mime/magic
```

然后，当由 root 用户使用 `pacreport --unowned-files` 时，如果不再安装关联的软件包（或者如果已创建任何新文件），则会列出所有无主文件。

此外，aconfmgr（aconfmgr-gitAUR）允许使用配置脚本跟踪修改的和孤立的文件。

删除未使用的软件包（孤立软件包）

孤立软件包是作为依赖项安装的包，任何软件包都不再依赖它。

由于使用 `pacman -R *package*` 而不是 `pacman -Rs *package*` 卸载软件包，将软件包安装为 makedepends，或者软件包的较新版本中删除了依赖项，它们可能会随着时间的推移在您的系统上累积。

递归删除孤立软件包及其配置文件：

```
# pacman -Qdtq | pacman -Rns -
```

如果没有孤立软件包，将显示错误 错误：有参数 `'-'` 但标准输入流为空。这个是正常的，因为 `pacman -Rns` 没有收到任何参数。可以通过在第二个命令前面加上来自 moreutils包 包的 ifne(1) 来避免该错误。

如果列出的软件包中有您不想删除的，可以通过将其标记为显式安装来将其从孤立软件包列表中排除：

```
# pacman -D --asexplicit package
```

注意：`-Qt` 仅显示真的孤立包，要包含可选依赖，请使用 `-Qtt`.

提示：将 `pacman -Qdt` 添加到 pacman 的一个处理后钩子用于在如果有处理产生孤立包后获得提示。这在当有软件包被仓库放弃时是有用的，因为任何被抛弃的包都会成为本地的一个孤立包（除非它是被显式安装的）。要避免在没有找到孤立包时“执行命令失败”的错误，在你的钩子中为 `Exec` 使用如下的指令: `/usr/bin/bash -c "/usr/bin/pacman -Qdt || /usr/bin/echo '=> 未找到。'"` 软件包 pacman-log-orphans-hookAUR 使用类似的钩子提供了更详细的指示。

检测更多不需要的软件包

在某些情况下，上述方法不会检测到所有可能不需要的包。例如，依赖环（也称为“循环依赖”）、额外依赖（多次满足）、一些非显式的可选项等。

要检测此类包：

```
$ pacman -Qqd | pacman -Rsu --print -
```

如果要一次删除列表中的所有包，请运行不带 `--print` 参数的命令。

删除必需的软件包以外的所有软件包

如果需要删除必需的软件包以外的所有软件包，一种方法是将非必需包的安装原因设置为依赖项，然后删除所有不必要的依赖项。

首先，对于所有“显式安装”的包，将其安装原因更改为“作为依赖项安装”：

```
# pacman -D --asdeps $(pacman -Qqe)
```

然后，将您不想删除的必需软件包安装原因更改为“显式安装”，以避免将其作为目标：

```
# pacman -D --asexplicit base linux linux-firmware
```

注意：

- 可以向上述命令中添加其他包，以避免被删除。请参阅安装指南#安装必需的软件包，以了解功能完备的基本系统可能需要的其他软件包的更多信息。
- 这还将引导加载程序选择为要删除的包。系统应该仍然可以启动，但如果没有它，启动参数可能无法更改。

最后，按照#删除未使用的软件包（孤立软件包）中的说明删除“作为依赖项安装”的所有包。

获取若干软件包的依赖项列表

依赖项按字母顺序排序，并删除重复项。

注意：要仅显示本地已安装软件包的树，使用 `pacman -Qi`。

```
$ LC_ALL=C.UTF-8 pacman -Si packages | awk -F'[:<=>]' '/^Depends/ {print $2}' | xargs -n1 | sort -u
```

或者，用 expac包：

```
$ expac -l '\n' %E -S packages | sort -u
```

列出已更改的备份文件

要列出被 *pacman* 跟踪的易被用户更改（即列在PKGBUILD backup 数组中的文件）且已被用户更改的文件，请使用以下命令：

```
# pacman -Qii | awk '/\[modified\]/ {print $(NF - 1)}'
```

使用 root 权限运行此命令将确保输出中包含只能由 root 读取的文件（例如 `/etc/sudoers`）。

这可以用在执行选择性系统备份或尝试将系统配置从一台计算机复制到另一台计算机时。

提示：

- 见#列出软件包中所有被更改的文件以列出 *pacman* 知道的所有更改过的文件，而不仅仅是备份文件。
- 见#查找不属于任何软件包的文件以列出系统中未被 *pacman* 跟踪的所有文件。

备份 pacman 数据库

以下命令可用于备份本地 *pacman* 数据库：

```
$ tar -cjf pacman_database.tar.bz2 /var/lib/pacman/local
```

将备份的 *pacman* 数据库文件存储在一个或多个脱机媒体上，如 U 盘、外部硬盘或 CD-R。

可以通过将 `pacman_database.tar.bz2` 文件移动到 `/` 目录并执行以下命令来恢复数据库：

```
# tar -xjvf pacman_database.tar.bz2
```

注意：如果 *pacman* 数据库文件已损坏，并且没有可用的备份文件，此时重建 *pacman* 数据库还是有希望的。查阅#恢复 pacman 的本地数据库。

提示：pakbak-gitAUR 包提供了一个脚本和一个 systemd 服务来自动化任务。可以在 `/etc/pakbak.conf` 中进行配置。

简单的检查更改日志

当维护者更新软件包时，提交通常会以有用的方式进行注释。用户可以通过安装 pacologAUR 从命令行快速检查这些。通过 `pacolog *package*` 使用该实用程序列出来自官方软件库或 AUR 的包的最近提交消息。

安装和修复

获取和修复软件包的额外方法。

从 CD/DVD 或 U 盘安装软件包

本文或本章节可能需要合并到#自建本地仓库。

附注： Use as an example and avoid duplication（在 Talk:Pacman/提示和技巧 中讨论）

要下载软件包，或软件包组：

```
# cd ~/Packages
# pacman -Syw --cachedir . base base-devel grub-bios xorg gimp
# repo-add ./custom.db.tar.zst ./*.pkg.tar.zst
```

Pacman 默认情况下将引用主机安装，但无法正确解析和下载现有的依赖项。在需要所有包和依赖项的情况下，建议创建一个临时空白数据库，并使用 `--dbpath` 引用它：

```
# mkdir /tmp/blankdb
```

```
# pacman -Syw --cachedir . --dbpath /tmp/blankdb base base-devel grub-bios xorg gimp
# repo-add ./custom.db.tar.zst ./*.pkg.tar.zst
```

然后，您可以将“Packages”目录刻录到光盘（如 CD、DVD）或将其传输到U盘、外部硬盘等。

1. 挂载媒体：

对于光盘驱动器：

```
# mount --mkdir /dev/sr0 /mnt/repo
```

对于 USB 闪存驱动器，硬盘驱动器等：

```
# mount --mkdir /dev/sdxY /mnt/repo
```

2. 编辑 `pacman.conf` 并将此存储库添加到其他存储库（例如，extra、core 等）之**前**。这很重要。不要只取消对底部的注释。这样可以确保 CD/DVD/USB 中的文件优先于标准存储库中的文件：

```
/etc/pacman.conf
[custom]
SigLevel = PackageRequired
Server = file:///mnt/repo/Packages
```

3. 最后，同步 *pacman* 数据库，以便能够使用新的存储库：

```
# pacman -Syu
```

自建本地仓库

使用 *pacman* 中包含的 *repo-add* 脚本为个人软件仓库生成数据库。请使用 `repo-add --help` 获取有关其用法的详细信息。 软件包数据库是一个可以压缩的 tar 文件。有效的扩展名为 *.db* 或 *.files*，后跟归档扩展名*.tar*, *.tar.gz*, *.tar.bz2*, *.tar.xz*, *.tar.zst*, 或 *.tar.Z*。文件不需要存在，但所有父目录都必须存在。

要向数据库中添加新包，或替换数据库中现有包的旧版本，请运行：

```
$ repo-add /path/to/repo.db.tar.zst /path/to/package-1.0-1-x86_64.pkg.tar.zst
```

当使用 *repo-add* 时，数据库和包不需要位于同一目录中，但请记住，当和该数据库一起使用 *pacman* 时，它们应该在一起。将要加入仓库中的所有建好的包存储在一个目录中，还允许使用 shell glob 扩展一次添加或更新多个包：

```
$ repo-add /path/to/repo.db.tar.zst /path/to/*.pkg.tar.zst
```

警告：*repo-add* 以与命令行上传递的顺序相同的顺序将条目添加到数据库中。如果涉及同一软件包的多个版本，必须小心确保最后添加正确的版本。特别注意，shell 使用的词法顺序取决于语言环境，并且不同于 *pacman* 使用的 vercmp(8) 顺序。

如果您希望支持多个架构，则应采取预防措施以防止发生错误。每个架构都应有自己的目录树：

```
$ tree ~/customrepo/ | sed "s/$(uname -m)/arch/g"
/home/archie/customrepo/
```

```
└── arch
    ├── customrepo.db -> customrepo.db.tar.zst
    ├── customrepo.db.tar.zst
    ├── customrepo.files -> customrepo.files.tar.zst
    ├── customrepo.files.tar.zst
    └── personal-website-git-b99cce0-1-arch.pkg.tar.zst

1 directory, 5 files
```

repo-add 可执行文件检查包软件是否适合。如果不是这样，您将遇到类似于以下内容的错误消息：

```
==> 错误： '/home/archie/customrepo/arch/foo-arch.pkg.tar.zst' 未包含有效的数据库存档扩展名。
```

repo-remove 用于从软件包数据库中删除包，命令行中只指定包名。

```
$ repo-remove /path/to/repo.db.tar.zst pkgname
```

创建本地仓库数据库后，将仓库添加到要使用该仓库的每个系统的 `pacman.conf` 中。自定义仓库的一个示例在 `pacman.conf` 中。仓库的名称是省略了文件扩展名的数据库文件名。在上面的示例中，仓库名仅为 *repo*。使用 `file://` URL 或通过 HTTP 使用 `http://localhost/path/to/directory` 引用仓库的位置。

如果愿意，可以将自定义仓库添加到非官方用户仓库列表中，这样社区就可以从中受益。

在网络上共享 pacman 缓存

**本文或本章节可能需要合并到 Package_Proxy_Cache。**

附注： Same topic（在 Talk:Pacman/提示和技巧 中讨论）

如果你碰巧在你的局域网上运行了几个 Arch boxes，你可以共享软件包，这样可以大大减少下载时间。请记住，你不应在不同的体系结构（即 i686 和 x86_64）之间共享，否则会遇到问题。

只读缓存

Pacman 6.1.0 直接支持缓存服务器。缓存服务器将在任何非缓存服务器之前尝试，不会因为 HTTP 404 下载错误而从服务器池中移除，并且不会用于数据库文件。

如果您正在寻找快速解决方案，只需运行基本临时网络服务器，其他计算机可以将其用作缓存服务器。

使用 Python http.server 模块开始提供此目录：

```
$ python -m http.server -d /var/cache/pacman/pkg/
```

提示：默认情况下，Python `http.server` 会监听任何接口的端口 `8000`。要使用其他端口或仅绑定到特定地址，只需添加一个参数和另一个参数：

```
$ python -m http.server -d /var/cache/pacman/pkg/ --bind 127.0.0.1 8080
```

然后在每台客户机上编辑 `/etc/pacman.d/mirrorlist`，将此服务器添加为缓存服务器：

```
/etc/pacman.d/mirrorlist
CacheServer = http://server-ip:port
```

警告：不要像其他条目一样将 `/repos/$repo/os/$arch` 追加到该自定义服务器，因为该层次结构不存在，因此查询将失败。

如果要寻找更独立的解决方案，darkhttpd包 提供了一个最小的网络服务器。将以前的 `python` 命令替换为如：

```
[http]$ darkhttpd /var/cache/pacman/pkg --no-server-id
```

为了方便，您还可以将 darkhttpd 作为 *systemd* 服务运行：见 systemd#编写单元文件。

miniserve包，一个用 Rust 编写的小型网络服务器，也可以使用：

```
$ miniserve /var/cache/pacman/pkg
```

然后像上面一样编辑 `/etc/pacman.d/mirrorlist`，让 url 位于第一位。

如果您已经在为其他用途运行网络服务器，您可能希望将其重新用作本地仓库服务器。例如，如果您已经使用 nginx 服务于某个站点，则可以在端口8080上添加 *nginx* server 块侦听：

```
/etc/nginx/nginx.conf
server {
    listen 8080;
    root /var/cache/pacman/pkg;
    server_name myarchrepo.localdomain;
    try_files $uri $uri/;
}
```

进行此更改后，记得重启 `nginx.service`。

提示：无论您使用哪种网络服务器，请确保防火墙配置（若有）允许所需流量访问所配置的端口，并且不允许任何不希望的流量。见 安全#网络与防火墙。

层叠挂载只读缓存

通过层叠挂载（英语：Overlay filesystem）其 `/var/cache/pacman/pkg` 目录，可以将本地网络上的一台机器用作只读包缓存。如果该服务器已在其上安装了其他 boxes 也使用的合理全面的最新软件包，则这种配置是有利的。这对于在低带宽上游的连接末端维护多台机器非常有用。

例如，要使用此方法：

```
# mkdir /tmp/remote_pkg /mnt/workdir_pkg /tmp/pacman_pkg
# sshfs remote_username@remote_pkgcache_addr:/var/cache/pacman/pkg /tmp/remote_pkg -C
# mount -t overlay overlay -o
lowerdir=/tmp/remote_pkg,upperdir=/var/cache/pacman/pkg,workdir=/mnt/workdir_pkg /tmp/p
acman_pkg
```

注意：工作目录必须是与上级目录位于同一挂载设备上的空目录。见 层叠文件系统#使用（英语：Overlay filesystem#Usage）。

提示：如果列出 `/tmp/pacman_pkg` 层叠目录会出现错误，例如“Stale file handle”，请尝试使用选项 `-o redirect_dir=off -o index=off` 进行层叠挂载。

在之后，使用选项 `--cachedir /tmp/pacman_pkg` 运行 *pacman*，例如：

```
# pacman -Syu --cachedir /tmp/pacman_pkg
```

分布式只读缓存

有一些特定于 Arch 的工具可以自动发现网络上的其他计算机，并提供包缓存。尝试 pacredir包, pacserve（英语：pacserve）, pkgdistcacheAUR, 或 paclanAUR。pkgdistcache 使用 Avahi 而不是普通 UDP，这可能在某些家庭网络中更好地工作，这些家庭网络在 Wi-Fi 和以太网之间路由而不是桥接。

从历史上看，有 PkgD 和 multipkg，但它们都已不再维护。

读写缓存

为了在多台计算机之间共享软件包，只需使用任何基于网络的挂载协议共享 `/var/cache/pacman/`。本章节介绍如何使用 SSHFS 在同一本地网络上的多台计算机之间共享包缓存和相关库目录。请记住，网络共享缓存可能会很慢，这取决于文件系统的选择以及其他因素。

首先，安装任何基于网络的文件系统软件包：sshfs包, curlftpfs包, samba包 或 nfs-utils包。

提示：

- 要使用 *sshfs*，请考虑阅读使用 SSH 密钥。
- 默认情况下，*smbfs* 不提供包含冒号的文件名，这会导致客户机重新下载有问题的包。若要防止这种情况，请在客户端上使用 `mapchars` 挂载选项。

然后，要共享实际的软件包，请在每台客户机上从服务器挂载 `/var/cache/pacman/pkg` 到 `/var/cache/pacman/pkg`。

警告：不要将 `/var/cache/pacman/pkg` 或其任何祖先（例如，`/var`）设置为符号链接。Pacman 希望这些是目录。当 *pacman* 重新安装或升级自己时，它将删除符号链接并创建空目录。然而，在事务 *pacman* 期间，依赖于驻留在那里的一些文件，因此中断更新过程。更多信息请查阅 FS#50298。

使用 rsync 或 FTP 双向同步

本地环境中的另一种方法是 rsync。选择用于缓存的服务器并启用 rsync 守护程序。在客户端上，通过 rsync 协议与此共享进行双向同步。包含冒号的文件名对于 rsync 协议来说没有问题。

为客户端起草示例，在共享名称中使用 `uname -m` 可确保依赖于架构的同步：

```
# rsync ... rsync://server/share_$(uname -m)/ /var/cache/pacman/pkg/
# pacman -Syu
# paccache --remove --keep 3
# rsync --delete ... /var/cache/pacman/pkg/ rsync://server/share_$(uname -m)/
```

如果不依赖未加密的 rsync 守护程序，更安全的选择是通过 ssh 使用 rsync，Rsync# 使用 SSH 自动备份提供了概述。

如果 rsync 在您的本地环境中不可用，简单的 ftp 服务也适合双向同步。lftp包 提供了 `--mirror` 和 `--delete` 选项来同步本地和远程存储。

使用 nginx 的动态反向代理缓存

nginx 可用于将软件包请求代理到官方上游镜像，并将结果缓存到本地磁盘。该软件包的所有后续请求将直接从本地缓存提供，从而最大限度地减少更新大量计算机所需的互联网流量。

在本例中，缓存服务器将在 `http://cache.domain.example:8080/` 上运行并将软件包存储在 `/srv/http/pacman-cache/` 中。

在将托管缓存的计算机上安装 nginx。为缓存创建目录并调整权限，以便 nginx 可以向其中写入文件：

```
# mkdir /srv/http/pacman-cache
# chown http:http /srv/http/pacman-cache
```

使用 nginx pacman 缓存配置作为 `/etc/nginx/nginx.conf` 的起点。检查 `resolver` 指令是否适合您的需要。在上游 server 块中，使用官方镜像的地址配置 `proxy_pass` 指令，请参阅配置文件中有关期待的格式的示例。一旦您对配置文件感到满意就启动并启用 nginx。

为了使用缓存，每个 Arch Linux 计算机（包括托管缓存的计算机）必须在 `mirrorlist` 文件的顶部具有以下行：

```
/etc/pacman.d/mirrorlist
Server = http://cache.domain.example:8080/$repo/os/$arch
...
```

注意：您需要创建一个方法来清除旧包，因为缓存目录会随着时间的推移而不断增长。`paccache`（由 pacman-contrib包 提供）可用于使用您选择的保留标准自动执行此操作。例如，`find /srv/http/pacman-cache/ -type d -exec paccache -v -r -k 2 -c {} \;` 将在缓存目录中保留最后两个版本的包。

Pacoloco 代理缓存服务器

Pacoloco 是一个易于使用的 *pacman* 仓库代理缓存服务器。它还允许自动预取缓存的包。

安装它为 pacoloco包。打开配置文件并添加 *pacman* 镜像：

```
/etc/pacoloco.yaml
port: 9129
repos:
  mycopy:
    urls:
      - http://mirror.lty.me/archlinux
      - http://mirrors.kernel.org/archlinux
```

重启 `pacoloco.service`，代理仓库将在 `http://*myserver*:9129/repo/mycopy`。

Flexo 代理缓存服务器

Flexo 是 *pacman* 仓库的另一个代理缓存服务器。Flexo 由 flexo-gitAUR 提供。安装后，启动 `flexo.service` 单元。

默认情况下，Flexo 在端口 `7878` 上运行。在 `/etc/pacman.d/mirrorlist` 的顶部加入 `Server = http://*myserver*:7878/$repo/os/$arch`，以便 *pacman* 通过 Flexo 下载软件包。

使用同步程序同步 pacman 软件包缓存

使用 Syncthing（英语：Syncthing） 或 Resilio Sync（英语：Resilio Sync） 以同步 *pacman* 缓存目录（即 `/var/cache/pacman/pkg`）。

避免过度清理缓存

执行 `pacman -Sc` 清理软件包缓存时，会删除所有当前机器上未安装的软件包。由于 *pacman* 无法判断软件包是否在其他机器上安装，这会导致移除某些还需要的软件包。

要在清理缓存时只删除*过时*的 tarballs：

```
/etc/pacman.conf
[options]
CleanMethod = KeepCurrent
```

从文件系统重新创建软件包

要从文件系统重新创建软件包，请使用 fakepkgAUR。系统中的文件按原样保存，因此任何修改都将出现在装配的软件包中。因此，不鼓励分发重新创建的包；有关替代方案，请参见 Arch 构建系统和 Arch Linux Archive。

已安装软件包的列表

保留所有明确安装的软件包的列表对于备份系统或加快新的安装非常有用：

```
$ pacman -Qqe > pkglist.txt
```

注意：

- 加上选项 `-t`，其他明确安装的软件包依赖的软件包不会被提及。如果从此列表中重新安装，它们将仅作为依赖项被安装。
- 加上选项 `-n`，将在列表中省略外部包（例如来自Arch 用户软件仓库 (AUR)）。
- 使用 `comm -13 <(pacman -Qqdt | sort) <(pacman -Qqdtt | sort) > optdeplist.txt` 还可以创建一个已安装的可选依赖项列表，这些依赖项可以使用 `--asdeps` 重新安装。
- 使用 `pacman -Qqem > foreignpkglist.txt` 创建已明确安装的 AUR 和其他外部包的列表。

要保持明确安装的软件包的最新列表（例如与版本化的 `/etc/` 结合），可以设置钩子。例子：

```
[Trigger]
Operation = Install
Operation = Remove
Type = Package
Target = *

[Action]
When = PostTransaction
Exec = /bin/sh -c '/usr/bin/pacman -Qqe > /etc/pkglist.txt'
```

从列表安装软件包

要从以前保存的软件包列表中安装软件包，而不重新安装已经安装的已更新的软件包，请运行：

```
# pacman -S --needed - < pkglist.txt
```

然而，列表中可能存在来自 AUR 或本地安装的外部软件包。要从列表中筛选出外部包，可以如下丰富前面的命令行：

```
# pacman -S --needed $(comm -12 <(pacman -Slq | sort) <(sort pkglist.txt))
```

最后，要确保系统中安装的软件包与列表匹配，并删除列表中未提及的所有软件包：

```
# pacman -Rsu $(comm -23 <(pacman -Qq | sort) <(sort pkglist.txt))
```

提示：这些任务可以自动化。有关示例，请参见 bacpacAUR, packupAUR, pacmanityAUR, 和 pugAUR。

列出软件包中所有被更改的文件

如果您怀疑文件损坏（例如，由于软件/硬件故障），但不确定文件是否损坏，您可能需要与包中的哈希值进行比较。这可以通过 pacutils包：

```
# paccheck --sha256sum --quiet
```

有关数据库的恢复，请见#恢复 pacman 的本地数据库。`mtree` 文件也可以从相应的软件包文件中提取为 C1。

注意：当怀疑存在恶意更改时，**不要**按原样使用！在这种情况下，建议采取安全防范措施，例如使用 live 介质和独立的哈希值源。

重新安装所有软件包

要重新安装所有本地软件包，使用：

```
# pacman -Qqn | pacman -S -
```

外来（AUR）软件包必须单独重新安装；您可以使用 `pacman -Qqm` 列出它们。

Pacman 默认保留安装原因。

警告：要强制覆盖所有包，请使用 `--overwrite=*`，尽管这应该是绝对的最后手段。请见系统维护#避免某些 pacman 命令。

恢复 pacman 的本地数据库

见 pacman/Restore local database。

从已有安装修复 USB 系统

如果你有一个 Arch 安装在U盘上，不小心搞坏了（比如，写入U盘时断电了），可以使用主机上的 Arch 修复U盘上的 Arch（假设U盘挂载在 `/newarch`）：

```
# pacman -S $(pacman -Qq --dbpath /newarch/var/lib/pacman) --root /newarch --dbpath /newarch/var/lib/pacman
```

查看 .pkg 文件中的单个文件

例如，如果您想查看 systemd包 包中提供的 `/etc/systemd/logind.conf` 的内容：

```
$ bsdtar -xOf /var/cache/pacman/pkg/systemd-250.4-2-x86_64.pkg.tar.zst etc/systemd/logind.conf
```

或者您可以使用 vim包 浏览归档：

```
$ vim /var/cache/pacman/pkg/systemd-250.4-2-x86_64.pkg.tar.zst
```

查找使用旧软件包中的库的应用程序

运行中的进程不会自动注意到由更新引起的更改。相反，它们继续使用旧的库版本。由于与安全漏洞或其他 bug 相关的潜在问题以及版本不兼容性，这可能是不可取的。

依赖于更新的库的进程可以使用 htop包 来找到，htop包 会突出显示受影响程序的名称，或者使用基于 lsof包 的代码片段，该代码片段还会打印库的名称：

```
# lsof +c 0 | grep -w DEL | awk '1 { print $1 ": " $NF }' | sort -u
```

此解决方案一般将仅检测由运行中进程打开的文件，这基本上将其限制为共享库（`.so` 文件）。它可能会丢失一些依赖关系，例如 Java 或 Python 应用程序的依赖关系。

仅安装所需语言的内容

许多软件包会安装多种语言的文档和翻译。有些程序旨在删除这些不必要的文件，例如 localepurgeAUR，它在安装包后运行以删除不需要的区域设置文件。更直接的方法

是通过 `/etc/pacman.conf` 中的 `NoExtract` 指令提供的，该指令可以防止这些文件被安装。

注意：如 Pacman#跳过文件安装到系统 中所述，“后面的规则会覆盖前面的规则，并且可以通过在规则前添加 ！ 来否定规则。”

要防止安装所有帮助文件的翻译，除了 C 区域设置外，请添加：

```
NoExtract = usr/share/help/* !usr/share/help/C/*
```

要防止安装所有 HTML 文档，请添加：

```
NoExtract = usr/share/gtk-doc/html/*
NoExtract = usr/share/doc/HTML/*
```

警告：一些用户注意到，删除**所有**区域设置会导致 dmenu, Steam, 甚至 Xorg 的意外结果。以下示例已进行调整，以避免此类问题，仅安装英文（US）文件和所需的 C 区域设置。

要防止安装各种 locale，除了所需的区域设置外，请添加：

```
NoExtract = usr/share/locale/* usr/share/X11/locale/*/* usr/share/i18n/locales/*
opt/google/chrome/locales/* !usr/share/X11/locale/C/*
NoExtract = !usr/share/X11/locale/compose.dir !usr/share/X11/locale/iso8859-1/*
NoExtract = !*locale*/en*/* !usr/share/*locale*/locale.*
NoExtract = !usr/share/*locales/en_?? !usr/share/*locales/i18n* !usr/share/
*locales/iso*
NoExtract = usr/share/i18n/charmaps/* !usr/share/i18n/charmaps/UTF-8.gz !
usr/share/i18n/charmaps/ANSI_X3.4-1968.gz
NoExtract = !usr/share/*locales/trans*
NoExtract = !usr/share/*locales/C !usr/share/*locales/POSIX
```

要防止安装翻译的 man page，请添加：

```
NoExtract = usr/share/man/* !usr/share/man/man*
```

要防止安装 vim-runtime包 中的语言文件，请添加：

```
NoExtract = usr/share/vim/vim*/lang/*
```

要防止安装 Qt 应用程序中除英文内容外的所有内容，请添加：

```
NoExtract = usr/share/*/translations/*.qm usr/share/*/nls/*.qm
usr/share/qt/phrasebooks/*.qph usr/share/qt/translations/*.pak !*/en-US.pak
```

要防止安装 Electron 应用程序中除英文内容外的所有内容，请添加：

```
NoExtract = usr/share/*/locales/*.pak opt/*/locales/*.pak usr/lib/*/locales/*.pak !
*/en-US.pak
```

要防止安装 LibreOffice 中的英文帮助文件，请添加：

```
NoExtract = usr/lib/libreoffice/help/en-US/*
```

要防止安装 OnlyOffice 中除英文内容外的所有内容，请添加：

```
NoExtract = opt/onlyoffice/desktopeditors/dictionaries/* !
opt/onlyoffice/desktopeditors/dictionaries/en_US/*
NoExtract = opt/onlyoffice/desktopeditors/editors/web-apps/apps/*/main/locale/* !
*/en.json
NoExtract = opt/onlyoffice/desktopeditors/editors/web-apps/apps/*/main/resources/help/
*/* !*/help/en/*
NoExtract = opt/onlyoffice/desktopeditors/editors/web-apps/apps/
*/main/resources/symboltable/* !*/en.json
NoExtract = opt/onlyoffice/desktopeditors/editors/web-
apps/apps/documenteditor/forms/locale/* !*/en.json
NoExtract = opt/onlyoffice/desktopeditors/editors/web-
apps/apps/spreadsheeteditor/main/resources/formula-lang/* !*/en.json !*/en_desc.json
```

```
NoExtract = opt/onlyoffice/desktopeditors/converter/empty/*/* !
opt/onlyoffice/desktopeditors/converter/empty/en-US/*
NoExtract = opt/onlyoffice/desktopeditors/converter/templates/*/* !
opt/onlyoffice/desktopeditors/converter/templates/EN/*
```

要防止安装除英文 iBus 表情符号字典外的所有内容，请添加：

```
NoExtract = usr/share/ibus/dicts/emoji-*.dict !usr/share/ibus/dicts/emoji-en.dict
```

在不良连接上安装软件包

当试图从不良连接（例如在火车使用手机网络）安装软件包时，请使用 `--disable-download-timeout` 选项来减少接收错误的可能性，例如：

```
error: failed retrieving file [...] Operation too slow. Less than 1 bytes/sec transferred the last 10 seconds
```

或

```
error: failed retrieving file [...] Operation timed out after 10014 milliseconds with 0 out of 0 bytes received
```

性能

下载速度

下载软件包时 *pacman* 按照 `/etc/pacman.d/mirrorlist` 的顺序使用镜像。默认情况下位于列表顶部的镜像可能不是最快的。要选择更快的镜像，请见镜像源。

Pacman 下载软件包的速度也可以通过启用并行下载来提高，或者通过使用不同的应用程序来下载软件包，而不是使用 *pacman* 内建的文件下载器。

不论怎样，在做任何修改前，你必须确定拥有了最新版的 *pacman*。

```
# pacman -Syu
```

Powerpill

Powerpill 是 *pacman* 的包裹程序，增加了平行下载和分段下载功能，加速下载过程。

wget

参见 wget#pacman 集成。

对于需要更强大代理设置的用户来说，用 wget 比用 *pacman* 内建的下载方式更加方便。

要使用 `wget`，首先安装 wget包 软件包，然后修改 `/etc/pacman.conf` 并在其中的 `[options]` 区段将下面内容去掉注释：

```
XferCommand = /usr/bin/wget --passive-ftp --show-progress -c -q -N %u
```

除了将 `wget` 参数放在 `/etc/pacman.conf` 里，你也可以直接修改 `wget` 配置文件（全局文件是 `/etc/wgetrc`，各个用户的文件是 `$HOME/.wgetrc`）。

aria2

aria2 是一个具有断点续传和分块下载功能的轻量级下载软件，支持 HTTP/HTTPS/FTP 协议。aria2 可以多线程通过 HTTP/HTTPS 和 FTP 协议连接镜像服务器，显著提高下载速度。

注意：在 *pacman* 的 XferCommand 使用 aria2c **不会**导致并行下载多个包。因为 Pacman 调用 XferCommand 时是一次一个包调用的，等待下载完成才会启动下一个。想要并行下载多个包，参见 Powerpill。

安装 aria2包，修改 `/etc/pacman.conf`，在 `[options]` 段添加下列一行：

```
XferCommand = /usr/bin/aria2c --allow-overwrite=true --continue=true --file-allocation=none --log-level=error --max-tries=2 --max-connection-per-server=2 --max-file-not-found=5 --min-split-size=5M --no-conf --remote-time=true --summary-interval=60 --timeout=5 --dir=/ --out %o %u
```

提示：使用 pacman 和 aria2 的另一种配置\]试图简化配置并添加更多配置选项。

关于使用的 aria2c 选项，见 aria2c(1) § OPTIONS。

- `-d, --dir`由 *pacman* 设定的文件下载目录。
- `-o, --out`: 输出的下载文件的文件名。
- `%o`: 代表 *pacman* 指定的本地文件名的变量。
- `%u`: 代表 *pacman* 指定的下载 URL 的变量。

其它程序

这里还有一些可以和 *pacman* 协同工作的下载软件。下面列举了它们对应的 XferCommand 命令写法：

- `snarf`: `XferCommand = /usr/bin/snarf -N %u`
- `lftp`: `XferCommand = /usr/bin/lftp -c pget %u`
- `axel`: `XferCommand = /usr/bin/axel -n 2 -v -a -o %o %u`
- `hget`: `XferCommand = /usr/bin/hget %u -n 2 -skip-tls false`（请阅读 Github 项目页面上的文档获取更多信息）
- `saldl`: `XferCommand = /usr/bin/saldl -c6 -l4 -s2m -o %o %u`（请阅读项目页面上的文档获取更多信息）

工具

- **isfree** — 一个 Bash 脚本，用于列出非自由软件包。基于 Parabola 的黑名单。

: https://github.com/leo-arch/isfree || isfreeAUR

- **Lostfiles** — 标识不属于任何包的文件的脚本。

: https://github.com/graysky2/lostfiles || lostfiles包

- **pacutils** — 基于 libalpm 的程序的帮手库。

: https://github.com/andrewgregory/pacutils || pacutils包

- **pkgfile** — 查找包拥有文件的工具。

: https://github.com/falconindy/pkgfile || pkgfile包

- **pkgtop** — 为 GNU/Linux 设计的交互式包管理器和资源监视器。

: https://github.com/orhun/pkgtop || pkgtop-gitAUR

- **Powerpill** — 通过 aria2 和 Reflector 使用并行和分段下载来尝试加快 *pacman* 的下载速度。

: https://xyne.dev/projects/powerpill/ || powerpillAUR

- **repoctl** — 帮助管理本地仓库的工具。

: https://github.com/cassava/repoctl || repoctlAUR

- **repose** — Arch Linux 仓库构建工具。

: https://github.com/vodik/repose || repose包

- **snap-pac** — 使 *pacman* 自动使用 snapper 创建前/后快照，类似 openSUSE 的 YaST。

: https://github.com/wesbarnett/snap-pac || snap-pac包

- **vrms-arch** — 一个虚拟的 Richard M. Stallman 告诉你安装了哪些非自由软件包。

: https://github.com/orospakr/vrms-arch || vrms-arch-gitAUR

Initramfs

本章解决内核到根文件系统之间的那段空白：内核要靠早期用户空间装载模块、挂载根分区，弄错一步就停在半路。必读《Mkinitcpio》，从安装、创建映像到排错都在其中；再读《Mkinitcpio/极简 initramfs》，弄清 udev 要求、.preset 作用与模块筛选。《Booster》《Dracut》是替代路线，不换方案可以跳过。

最常见的坑是改了 mkinitcpio.conf 却不重新生成映像，改动不会自动生效；只编辑 .preset 而不看实际映像也会误判。更险的是随手删模块，挂不上根分区系统就起不来。

9.1　Booster

难度 进阶 | **篇幅** 7.6 KB | **所属** Initramfs

本节将讲到

- 安装
- 配置
- 引导加载程序配置
- 问题解决
- 参见

翻译状态：

- 本文（或部分内容）译自 Booster，最近一次同步于 2024-6-9，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 需要翻译。（在 Talk:Booster# 中讨论）

Booster 是一个快速的initramfs生成器，类似于mkinitcpio和dracut。 Booster 的灵感来源于"distri" 项目，旨在创建小而快的启动映像。

Booster 提供了 `/usr/bin/booster` 用户空间工具，以生成 initramfs 映像。生成的映像默认位于 `/boot/`。

安装

安装 booster包。软件包安装钩子会生成 initramfs 镜像，每个已安装的内核生成一个镜像（例如 linux包, {linux-lts包）。镜像文件位于 `/boot` 目录下：

```
$ ls -lh /boot/booster*
-rwxr-xr-x 1 root root 4.0M Dec 16 16:20 /boot/booster-linux.img
```

也可以手动创建镜像：

```
$ booster build mybooster.img
```

配置

Booster 配置文件位于 `/etc/booster.yaml`。如果文件为空，那么booster将会使用默认配置 (host-specific images, no network)。配置详见 booster(1) § CONFIG FILE。

早期加载内核模块

有时，一些内核模块需要在 initramfs 阶段加载。

例如，当你需要加载 `nvidia` 模块时，可以在配置文件中采取以下设置：

```
/etc/booster.yaml
modules_force_load: nvidia
```

然后#重新生成 booster 镜像。

加密

Booster supports LUKS based full-disk encryption out-of-the-box like Clevis. The generator does not need any extra configuration. Yet, for the initramfs you need to append information about the LUKS partition where the root resides. This is done with either `rd.luks.uuid=*LUKSUUID*` or `rd.luks.name=*LUKSUUID*=*LUKSNAME*` kernel parameter that need to be specified in the boot loader configuration file. `*LUKSUUID*` specifies the UUID of the encrypted LUKS partition that needs to be unlocked by Booster. The booster(1) § UUID parameters manual recommends that the UUID does not contain any quotes. `*LUKSNAME*` specifies name of the unlocked partition (as in `/dev/mapper/*LUKSNAME*`). See booster(1) § BOOT TIME KERNEL PARAMETERS for related options.

No image rebuild is required. Once the boot loader configuration is done, reboot the computer. After that you will see a `Enter passphrase for *YOURROOT*:` prompt at boot time asking for a password for the encrypted root partition.

systemd style binding

Booster also supports partitions bound with systemd such as `systemd-fido2` and `systemd-tpm2`.

If you use \`systemd-fido2\` then please install libfido2包 package and add fido2-assert to the image using following configuration:

```
/etc/booster.yaml
extra_files: fido2-assert
```

Regenerate the booster images. Booster will detect this configuration during boot and use the present YubiKey to unlock the drive.

注意：Before commit 1b65577, it has been reported Booster may not load some necessary kernel modules early enough to unlock encrypted partitions with a FIDO2 key. In this case, add the following modules to be forcibly loaded:

```
/etc/booster.yaml
modules_force_load: usbhid,hid_sensor_hub
extra_files: fido2-assert
```

重新生成 booster 镜像

完成 Booster 配置后，需要更新 `/boot` 中的 Booster 镜像。可以使用 `booster build booster-foo.img` 手动生成镜像，也可以使用便捷脚本 `/usr/lib/booster/regenerate_images` 遍历所有已安装的内核，为每个内核生成 Booster 镜像。

引导加载程序配置

生成映像后，就该配置引导加载程序了。

rEFInd

如果配置已经依赖于自动检测\]，则无需额外更改配置。rEFInd 支持 initrd 文件命名为 booster*。

如果在refind.conf或 手动启动项中手动指定 initramfs 路径，请确保使用正确的文件名。例如，`**booster**-linux.img` 而不是 `**initramfs**-linux.img`。

systemd-boot

要使用 systemd-boot 启用新的 initramfs 映像，只需像这样创建一个新的引导加载器条目：

```
/boot/loader/entries/booster.conf
title Arch Linux with booster
linux /vmlinuz-linux
initrd /booster-linux.img
options root=UUID=08f83949-bcbb-47bb-bc17-089aaa59e17e rw
```

根文件系统在 `UUID=08f83949-bcbb-47bb-bc17-089aaa59e17e`。运行 `blkid /dev/*ROOTDEVICE*` 查找您的根设备 UUID。

问题解决

调试

如果 Booster 出现问题，不能按预期运行，请启用调试信息输出，以提供更多信息说明发生了什么：

- 对于生成器，有一个 `-debug` 命令行参数： `booster -debug`。
- 对于 init，有一个 `boost.debug` 内核参数。

如果您认为这是 Booster 本身的问题，那么请在 GitHub 上提交ticket。

Booster生成器出现 "too many open files"错误

如果启用了 `strip` 和 `universal`，并出现类似 `/usr/lib/modules/glue_helper.ko: pipe2: too many open files` 的错误，则需要增加每个进程的打开文件限制。请参见Limits.conf#nofile。

参见

- 上游 README

9.2 Mkinitcpio/极简 initramfs

难度 进阶 | **篇幅** 8.6 KB | **所属** Initramfs

本节将讲到

- Udev 要求
- 编辑 .preset 文件
- 找出需要的模块
- 初步编辑 mkinitcpio.conf
- 初步测试

本文介绍如何为具有特定、已知和静态硬件配置的系统创建精简、最小的 initramfs。Falconindy (Dave Reisner) 的 Optimizing Bootup With mkinitcpio 详细阐述了该过程。

Udev 要求

创建自己的 initramfs 映像的一大优点是可以移除 `udev` 钩子。这个钩子在 initramfs 中占了相当大一块空间（使用 LZ4 和 LZOP 压缩时约为 700-800 KiB，其他算法较小）。因此它不仅导致启动时加载耗时更长（需要解压更多的数据），而且初始化其自身时也要额外的时间。**但是，仍有部分操作需要** `udev`，包括解析 UUID、LABEL、PARTUUID 和 PARTLABEL 标识符（workaround hook without-udev）以及包含 `root` 分区的 LVM 和 mdadm 设备。 若您不确定是否需要 `udev`，请继续按照本页上的说明操作，直到 #初步测试章节。如果在没有 `udev` 的时候出了问题，请重新启用此钩子，再试一次。

另请注意，虽然大部分键盘（AT，PS/2，USB）不需要 `udev` 钩子也能正常工作，但使用罗技统一接收器（Logitech Unified Receiver）的 USB 设备需要。此时，您可以在所有映像中包含 `udev` 或依赖 `fallback` 映像正常工作。

如果您确实需要 `udev`，最小化映像的努力可能就白费了。映像大小也许能减小约 600 KiB，但是启动时间不会有显著改变。在此情况继续下去，不失为一次学习机会。

编辑 .preset 文件

在 Falconidy 的教程中，他编辑 `/etc/mkinitcpio.conf` 并运行 `mkinitcpio -g` 来创建测试 initramfs 映像，使系统上已知正常启动的 initramfs 映像保持不变。 但是，如果您事后盲目运行 `mkinitcpio -P`，甚至 `fallback` 映像也会被删除。

准备自行创建 initramfs 的更安全方法是修改 `/etc/mkinitcpio.d` 中的 `.preset` 文件。以下的示例設定將以最小的 initramfs 映像取代`default`，並依照 The Arch Way 的方式建立一個新的正常映像文件。 如果出现问题，您仍可以使用 `normal` 或 `fallback` 映像。 完成后，您可以从配置中删除 `normal_*` 行并删除 `initramfs-linux*-normal.img` 文件。

```
...

PRESETS=('default' 'normal' 'fallback')
...

default_options="-S udev,block,mdadm_udev,filesystems,keyboard,fsck,consolefont"
...

#normal_config="/etc/mkinitcpio.conf"
normal_image="/boot/initramfs-linux-normal.img"
#normal_options=""
...
```

`mdadm_udev` 和 `consolefont`钩子不在 Arch Linux 默认的配置中，在 `*_options` 行的 `-S` 参数中包含无关的钩子不会导致错误。

找出需要的模块

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 需要翻译（在 Talk:Mkinitcpio/极简 initramfs# 中讨论）

找出机器需要的模块的最快方法是重启电脑，使用 `fallback` 映像并通过引导加载程序（boot loader）添加 `break=postmount` 内核参数。于是在根文件系统挂载好后便可进入命令行界面。

重启电脑后，通过以下命令来获取需要的模块：

```
lsmod | awk 'NF==3{print $1}'
```

注意：`awk` 命令會返回每一行的第一個欄位（使用 `{print $1}`），但只限於那些剛好有三個欄位的行，這是通過 `NF==3` 來強制篩選的。 模組依賴關係會包含第四個欄位，用來顯示是由哪個模組引入這個依賴，因此這些行會因為有第四個欄位而被過濾掉。Arch 的 `mkinitcpio` 會處理那些正當包含在 `MODULES=()`、`FILES=()` 和 `BINARIES=()` 陣列中的依賴項目。

保存已加载模块的清单，输入 `exit` 继续启动机器。

另一种方式是安装 hwdetect包 来帮助确定所需的模块。虽然此软件包已停止维护，它仍可提供有价值的信息。另外，请参阅内核模块以开始使用本机工具。

初步编辑 mkinitcpio.conf

编辑 `/etc/mkinitcpio.conf` 修改 `MODULES=` 数组。值得注意 `/etc/mkinitcpio.conf` 作为脚本调用，可以使用 bash 脚本构建 MODULES 数组。

```
MODULES=()    # filesystems
MODULES+=()   # storage
MODULES+=()   # keyboard
MODULES+=()   # miscellaneous
```

把模块添加到配置中最后的 `miscellaneous` 行中，整理时再将模块移至合适的行去。

若需要对 `root` 分区设备和在 `/etc/fstab` 的**任何其他挂载点**进行文件系统检查操作，请按下文配置：

- 对于 ext\[2|3|4\] 设备：

```
BINARIES=(fsck fsck.ext[2|3|4] e2fsck)
```

- 对于 vfat (UEFI boot) 设备：

```
BINARIES=(fsck fsck.vfat dosfsck)
```

- 对于 btrfs 单盘设备：

```
BINARIES=(fsck fsck.btrfs btrfsck)
```

- 对于 btrfs 多盘设备：

```
BINARIES=(fsck fsck.btrfs btrfs btrfsck)
```

- 对于 xfs 设备：

```
BINARIES=(fsck fsck.xfs xfs_repair)
```

注意：

- 上述示例中的第三个选项都是可选的，但除去它们将导致无法修复损坏的文件系统，此时需要从另一个 initramfs 启动。
- 鼓励添加其他文件系统的条目。

初步测试

编辑 `/etc/mkinitcpio.conf` 并运行 `mkinitcpio -P` 重建所有 initramfs 镜像。然后重启。

如果不需要 **udev**，第一次启动应该会成功。 如果出现故障（例如，Arch 找不到根分区或键盘失灵），则需要返回并从`default_options`行的`-S`参数中删除`udev`，然后

再试一次。 如果需要使用 `udev`，请注意启动时间不会有明显改善，继续尝试只能作为学习经验。

整理模块

现在您已经有了一个已知正常的可启动 initramfs，是时候进一步精简 initramfs 了。通常是一次删除几个模块，重建 initramfs 映像，然后重新启动以查看一切是否仍然正常。若有功能异常，请使用 `fallback` initramfs 映像重启并重新添加已删除的模块，直到再次恢复正常。多次重复，直到只剩下所需的模块。 这可能是相对乏味的过程，因此提供了以下列表，以便减少前期工作。

注意：以下仅作示例参考，并不意味着适合每台设备。

文件系统模块

注意：只需要对应的模块：启动时需要检查文件系统的 `root` 设备和在 `/etc/fstab` 中的其他设备

- `ext[2,3,4]`
- `xfs`
- `jfs`
- `reiserfs`

存储设备模块

- `sd_mod` 用于所有 SCSI、SATA 和 PATA（IDE）设备
- `ahci` 用于现代 AHCI 控制器上的 SATA 设备
- `nvme` 和 `nvme_core` 用于 NVMe（M.2、PCI-E）设备
- `sata_*` 用于 IDE 模式控制器上的 SATA 设备
- `pata_*` 用于 PATA（IDE）设备
- `ehci_pci` 和 `usb_storage` 用于 USB 存储设备
- `virtio_blk` 和 `virtio_pci` 用于使用 VirtIO 进行存储的 QEMU/KVM 虚拟机

键盘模块

- `atkbd` 用于 AT 和 PS/2 键盘，以及 QEMU/KVM 中的模拟键盘。
- `hid_generic`、`ohci_pci`和 `usbhid` 用于普通 USB 键盘。
- `hid_logitech_dj`、`uhci_hcd` 和 `usbhid` 适用于使用罗技统一接收器的罗技 USB 键盘（**需要 {ic|udev}} 钩子**）。

收尾工作

将 initramfs 压缩到最小后，移除（或注释掉）`.preset` 文件中的 `normal_*` 行，并移除 `/boot` 中的 `initramfs-linux*-normal.img` 文件。

9.3 Dracut

难度 进阶 | **篇幅** 13.6 KB | **所属** Initramfs

本节将讲到

- 安装
- 用法
- 进阶配置
- 小提示
- 排障

dracut 会为内核创建初始映像，用于预载访问根文件系统所需的块设备模块（如 IDE、SCSI 或 RAID）。安装 linux包 时，可以在 mkinitcpio 和 *dracut* 之间进行选择。Fedora、RHEL、Gentoo 和 Debian 等系统都使用 *dracut*，而Arch 默认使用 mkinitcpio。

你可以在此查看 *dracut* 的完整项目文档。

安装

安装 dracut包，你也可以安装 dracut-gitAUR 使用最新的开发版本。

提示：如果 *dracut* **测试后功能一切正常**，你可以卸载 mkinitcpio包。

用法

`dracut` 的用法非常简单，而且通常就算在非标准环境下也不需要用户进行配置（如在 LUKS 上配置 LVM）。

使用如下命令为当前运行的内核生成 initramfs：

```
# dracut --hostonly --no-hostonly-cmdline --add-confdir no-network /boot/initramfs-linux.img
```

为了永久启用仅主机（hostonly）模式以便无需在命令行中进行指定，你可以在 dracut 的配置中添加：

```
/etc/dracut.conf.d/hostonly.conf
hostonly="yes"
hostonly_cmdline="no"
```

dracut-gitAUR 已默认启用仅主机模式。

注意：在某些情况下，特别是在首次安装系统的时候，无法使用上述命令。在这种情况下可以改用：

```
# dracut -f --regenerate-all
```

使用如下命令生成后备 initramfs：

```
# dracut /boot/initramfs-linux-fallback.img
```

`/boot/initramfs-linux.img` 指代输出的镜像文件。如果你使用了其它内核，请考虑更改文件名。例如，对于 linux-lts包 内核，输出文件应使用 `/boot/initramfs-linux-lts.img`。不过，只要引导加载程序的配置使用了相同的文件名，就可以随意命名这些文件。

注意：创建出的文件内嵌所有已安装的微码映像。

额外选项

`--force` 选项会覆写现有映像文件。

`--kver` 选项用于指定要使用的内核。该选项的输入需要匹配 `/usr/lib/modules` 目录下文件夹之一的名称。

其它选项可参考 dracut(8)。

进阶配置

要注意的是，initial ramdisk 阶段有两种执行各种任务的不同方式：

***基于 Shell（bash/busybox/dash）*的 initial ramdisk：该方式会启动一个初始化脚本，然后扫描 initial ramdisk 的文件系统来查找要执行的 dracut 脚本。**

***基于 systemd（默认）*的 initial ramdisk：systemd 在 initial ramdisk 阶段时就已启动。具体要执行的任务由标准的 systemd 单元文件指定，相关信息可参考 systemd 启动流程。**

这两种方式的主要区别在于 systemd dracut 模块的存在与否。详细信息请参考 #dracut 模块。

`dracut` 可通过直接传入命令行参数进行配置（参考 dracut(8) § OPTIONS）。如果你希望执行 `dracut` 命令时始终带上特定参数，可以在 `/etc/dracut.conf.d/` 目录下的 `.conf` 文件中进行指定。例如：

```
/etc/dracut.conf.d/myflags.conf
hostonly="yes"
compress="lz4"
add_drivers+=" i915 "
omit_dracutmodules+=" systemd network "
```

更多配置选项可参考 dracut.conf(5)。各个选项的完整说明可参考 dracut(8)。在下文中会对部分常用选项进行说明。

dracut 模块

dracut 使用模块化流程构建 initramfs（参考 dracut.modules(7)）。*dracut* 的所有内置模块位于 `/lib/dracut/modules.d`，可通过 `dracut --list-modules` 命令列

出。更多模块可通过其它软件包提供（如 dracut-sshd-gitAUR）。遗憾的是，*dracut* 的内置模块缺乏文档说明，尽管它们的名称通常不言自明。

部分模块默认被激活/禁用，可通过 `--add`/`--omit` 选项分别激活或禁用，也可以在配置文件中使用 `add_dracutmodules+=""` 或 `omit_dracutmodules+=""` 持久化更改：

```
/etc/dracut.conf.d/myflags.conf
# ...
add_dracutmodules+=" dracut modules to activate "
omit_dracutmodules+=" dracut modules to deactivate "
# ...
```

dracut 模块文档请参考上游文档。

大多数 dracut 模块都依赖于其它 dracut 模块。例如，蓝牙 dracut 模块就依赖于 dbus dracut 模块。

TPM2

要使用 *systemd* 通过 systemd-cryptenroll 调用 TPM2 解锁 *luks2* 加密卷的功能，需要安装 tpm2-tools包 并启用 `tpm2-tss` *dracut* 模块。

早期内核模块加载

可以通过 `--force_drivers` 命令行选项或 `force_drivers+=""` 配置项启用 dracut 早期加载功能（在 initramfs 阶段通过 `modprobe` 加载）。例如：

```
/etc/dracut.conf.d/myflags.conf
# ...
force_drivers+=" nvidia nvidia_modeset nvidia_uvm nvidia_drm "
# ...
```

内核命令行参数

内核命令行选项可放置在 `/etc/dracut.conf.d/` 的 *.conf* 文件内，并通过 `kernel_cmdline=` 选项进行配置。Dracut 会自动读取配置，然后创建并写入到 initramfs `/etc/cmdline.d/` 目录下的 `01-default.conf` 文件中。举个例子，你的内核命令行选项文件内容可能如下：

```
/etc/dracut.conf.d/cmdline.conf
kernel_cmdline="rd.luks.uuid=luks-f6c738f3-ee64-4633-b6b0-eceddb1bb010
rd.lvm.lv=arch/root rd.lvm.lv=arch/swap root=/dev/arch/root rootfstype=ext4
rootflags=rw,relatime"
```

其它说明

不需要为 `dracut` 指定根块设备。参考 dracut.cmdline(7)：

: 内核使用的根设备从来都是在启动配置文件的内核命令行选项中指定的。

不过，提前设置一些参数可能会很有用，而且还可以启用一些其它功能，如提示输入额外命令行参数。所有选项请参见 dracut.cmdline(7)。下面是一些配置选项示例：

- 从交换分区恢复：`resume=UUID=80895b78-7312-45bc-afe5-58eb4b579422`
- 提示输入额外内核命令行参数：{ic|1=rd.cmdline=ask}}
- 在设定了 `quiet` 的前提下输出更多信息：`rd.info`

统一内核映像

dracut 可以通过 `--uefi` 命令行参数或 `uefi="yes"` 配置项生成统一内核映像。

小提示

查看生成映像的信息

你可以查看生成的映像的信息，并输出到单页上：

```
# lsinitrd /path/to/initramfs_or_uefi_image | less
```

该命令会列出生成映像时传入到 `dracut` 的参数、包含的 `dracut` 模块以及包含的所有文件。

修改压缩软件

要减少压缩生成映像所消耗的时间，可以更换使用的压缩软件。

警告：请确保你的内核编译了所选软件对应的解压支持，否则系统将无法启动。你还需要安装所选压缩程序的软件包。

只需添加下列任意一行（不能多选）到 dracut 配置文件中：

```
compress="cat"
compress="gzip"
compress="bzip2"
compress="lzma"
compress="xz"
compress="lzo"
compress="lz4"
compress="zstd"
```

默认使用的是 gzip包。选择 `compress="cat"` 将不会压缩 initramfs。

你也可以使用非官方支持的压缩软件：

```
compress="program"
```

性能考虑

有些方法可以优化启动和生成 initramfs 的性能：

- 理解并配置最快的压缩方式。如果内核模块已经被压缩过，可能在生成 initramfs 时就不需要再次进行压缩。
- 理解在 initramfs 中添加 systemd 可能造成的影响。如果它会降低性能，就将其移除；如果会提升性能，就纳入进来。

- 在使用写时复制文件系统时，考虑使用 dracut-cpio。具体适用性请参考 `--enhanced-cpio` 选项。
- 减少 initramfs 中嵌入的内核模块和 dracut 模块的数量。例如：如果安装了 nfs-utils包，但不依赖其进行启动，就需要显式移除 nfs dracut 模块，否则在默认配置下生成的 initramfs 会启用网络启动 - 详细信息请参考 https://github.com/dracut-ng/dracut-ng/pull/297 。
- 考虑使用 busybox包 dracut 模块替换掉 bash。
- 考虑使用 `hostonly` 和 `hostonly_mode=strict`。

升级内核时生成新 initramfs

linux包 和 dracut-gitAUR 包自带了用于在内核更新后自动生成新 initramfs 映像的 pacman 钩子。

提示：

- dracut-ukifyAUR 软件包是使用 systemd-ukify包 生成统一内核映像的现代方法。与以下方法不同，你可以对整个内核映像签名，包括 initramfs。需要在 dracut 配置中使用 `uefi_secureboot_cert` 和 `uefi_secureboot_key` 选项（dracut.conf(5)）。
- 另外，如果你希望 initramfs 映像也是 EFI 可执行文件（即 `*esp*/EFI/Linux/linux-*kernel*-*machine_id*-*build_id*.efi`），也可以使用 dracut-uefi-hookAUR。在该目录下的 EFI 二进制文件会被 systemd-boot 自动检测到，因此不需要在 `/boot/loader/loader.conf` 中额外创建条目。

你还需要通过卸载 mkinitcpio包 或使用以下命令来阻止 mkinitcpio 创建和移除 initramfs 映像：

```
# ln -sf /dev/null /etc/pacman.d/hooks/90-mkinitcpio-install.hook
# ln -sf /dev/null /etc/pacman.d/hooks/60-mkinitcpio-remove.hook
```

蓝牙键盘支持

如果检测到了蓝牙键盘，dracut 会自动启用蓝牙模块。但 dracut 需要处于 hostonly 模式才能自动发现蓝牙键盘。

Limine boot loader support

The limine-dracut-supportAUR package utilizes limine-entry-tool with dracut and pacman hooks to automate the installation and removal of kernels and boot entries in the Limine boot loader. See Limine#Boot entry automation for more information.

排障

休眠

如果从休眠中恢复无效，你可能需要配置 `dracut` 以包含 `resume` 模块。添加一个配置文件：

```
/etc/dracut.conf.d/resume-from-hibernate.conf
add_dracutmodules+=" resume "
```

如果适用于你的系统，你可能也需要看下从加密交换分区恢复指南以及 dracut 特定指南。

LVM / 软 RAID / LUKS

如果内核无法自动发现并挂载 LVM / 软 RAID / LUKS 块设备，你可以加上以下内核命令行选项重新生成 initramfs：

```
rd.auto rd.lvm=1 rd.dm=1 rd.md=1 rd.luks=1
```

A stop job is running for "brltty"

If you have issues booting or very long shutdown processes while the system waits for `brltty`, add the following to the dracut configuration line:

```
omit_dracutmodules+=" brltty "
```

Alternatively, uninstall brltty包 if it is not needed.

No usable keyslot is available

```
Cannot use whirlpool hash for keyslot encryption.
Keyslot open failed.
No usable keyslot is available.
```

A failure to boot with a message similar to the above typically will only require the user to include the `crypt` module via `add_dracutmodules`.

参考

- Wikipedia:dracut (software))
- Gentoo:Dracut

9.4 Mkinitcpio

难度 高级 | **篇幅** 39.1 KB | **所属** Initramfs

本节将讲到

- 安装
- 创建和启用映像
- 配置
- 运行时配置
- 提示与技巧

mkinitcpio 是用于创建 initramfs 映像的 bash 脚本。有关对 initramfs 的一般性介绍，请参阅 Arch 的启动流程#initramfs。

需要注意的是，存在两种截然的不同方式用于执行早期用户空间阶段的各项任务：

基于 systemd 的 initramfs（版本 40 起默认）

: systemd 在早期用户空间阶段开始时即被启动。需要执行的任务由常规的 systemd 单元文件决定。参见 systemd 启动流程。

: **优点**：

- 与 systemd 生态系统的其余部分紧密集成，带来更一致、流畅的启动流程。
- 能更有效地并行化某些启动任务，在某些场景下可能缩短整体启动时间。
- 提供更全面的功能集，例如 systemd-cryptsetup-generator 与 GPT 分区自动挂载功能。

: **缺点**：

- 依赖较多，生成映像较大。由于包含 systemd 二进制文件及其依赖项，通常会导致 initramfs 增大，可能会稍稍延长启动时间。

基于 BusyBox 的 initramfs

: 启动一个 init 脚本，该脚本随后扫描 initramfs 查找需要执行的脚本（即本文中的“运行时钩子”）。

: **优点**：

- 轻量级，依赖较少，生成映像较小。

具体采用哪种方案取决于 `/etc/mkinitcpio.conf` 的 `HOOKS` 数组中是否包含 `systemd` 钩子。详见 #常用钩子。

mkinitcpio 由 Arch Linux 开发者及社区贡献者共同开发。参见公开 Git 仓库。

安装

安装软件包 mkinitcpio包。这个软件包是 linux包 等软件包的依赖，应该已经自动被安装了。高级用户可以从 mkinitcpio-gitAUR 获取 mkinitcpio 的最新开发版本。

注意：若要使用 git 开发版本，强烈建议同时加入 arch-projects 邮件列表！

创建和启用映像

自动匹配生成

每次安装或升级内核时，一 pacman 钩子会自动在 `/etc/mkinitcpio.d/` 生成 .preset 文件。例如，`linux.preset` 文件是官方稳定内核 linux包 生成的。.preset 文件本质上是一个创建 initramfs 映像所需信息的列表，避免手动指定各项参数及输出文件位置。

可自动创建两个映像：

- default（默认）映像，根据 #配置中的指令创建。
- fallback（后备）映像，与上述类似，但不使用 autodetect 钩子，因此包含支持大多数硬件的完整模块集。

默认仅创建 default 映像，fallback 映像必须显式启用。

创建预设文件后，该 pacman 钩子会调用 `mkinitcpio`，利用 .preset 文件中提供的信息生成映像。

注意：.preset 文件用于在内核更新后自动重新生成 initramfs，编辑时请务必小心。

手动生成

手动运行脚本，参照mkinitcpio(8) 帮助页面获得指引。尤其是当利用内核包生成（重新生成）预设时，使用`-p`/`--preset`选项按照预设生成映像。例如 为 linux包 包生成映像，使用这个命令：

```
# mkinitcpio -p linux
```

利用`-P`/`--allpresets`选项，根据所有现存的预设文件生成映像。常见的用法是当全局设置#配置变化后用此法重新生成全部的内存映像：

```
# mkinitcpio -P
```

可以按照不同的配置文件创建任意数量的内存映像。想要引导加载的映像必须在相应的引导加载程序配置中明确写出。

自定义生成

可以使用相应的配置文件产生一个映像。例如，接下来会根据`/etc/mkinitcpio-custom.conf`里指令生成初始化内存映像，并且将后者保存在 `/boot/initramfs-custom.img`。

```
# mkinitcpio --config /etc/mkinitcpio-custom.conf --generate /boot/initramfs-custom.img
```

如果为一个不是当前运行的内核创建映像，增加内核版本号到命令行。已安装的内核版本号可以在`/usr/lib/modules/`中找到，对于每个内核语法结构和`uname -r`命令输出结果一致。

```
# mkinitcpio --generate /boot/initramfs-custom2.img --kernel 5.7.12-arch1-1
```

统一内核映像

mkinitcpio 既可自行构建统一内核映像（UKI），也可通过 systemd-ukify包 构建。如果系统中未安装 systemd-ukify包 或使用 `--no-ukify` 参数明确禁用它，mkinitcpio 将使用自有实现，但此时将无法使用 ukify 提供的高级功能。

关于构建 UKI 的详细信息，请参阅统一内核映像。

配置

mkinitcpio的主配置文件是`/etc/mkinitcpio.conf`。也支持附加配置文件（Drop-in），如`/etc/mkinitcpio.conf.d/mkhooks.conf`（当使用 `-c`/`--config` 选项和/或使用了含 `ALL_config` 的预设时无效）。此外，内核软件包的预配置文件位于`/etc/mkinitcpio.d`（例如：`/etc/mkinitcpio.d/linux.preset`）。

用户可以编辑配置文件中的七个变量，参见mkinitcpio.conf(5) § VARIABLES获得更多细节：

MODULES：在钩子扩展运行前需要加载的内核模块。 **BINARIES**：内存盘映像中包含的额外的二进制文件（一般是可执行的命令）。 **FILES**：内存盘映像中包含的其他文件。 **HOOKS**：要执行的钩子扩展，它是在生成映像时执行的脚本。 **COMPRESSION**：内存盘映像的压缩方式。 **COMPRESSION_OPTIONS**：传递给压缩工具的额外参数，强烈不建议使用，*mkinitcpio* 可以自动根据压缩算法传递需要的参数(例如对 xz 传递 `--check=crc32` to xz), 手动设置了错误的参数可能导致系统无法启动。
MODULES_DECOMPRESS：解压可加载内核模块和固件文件或保持未解压原样。

注意：

- 一些系统需要的钩子像 **lvm2**, **mdadm_udev**, and **encrypt** 默认 **不是开启的** 。参见 #钩子（HOOKS） 仔细遵循指示。
- 存在多个硬盘控制器使用相同的节点名字，但是拥有不同的内核模块（例如，两个SCSI/SATA控制器或2个IDE控制器）应该使用 持久化设备命名persistent block device naming 保证正确的设备被挂载。否则在两次系统启动时根系统的位置可能改变导致内核崩溃。

模块（MODULES）

`MODULES`数组指定了启动过程最开始时就要加载的模块。

如果模块名称后加上一个`?`问号，那么即使系统无法找到该模块也不会报错。对于自己编译的内核，其中内置了某些模块，在这些模块在钩子或配置文件中显式的列出来时，该功能可能有用。

注意：

- 如果使用树外文件系统，且需要在早期用户空间挂载时（比如作为根文件系统），对应模块（如 **reiser4**）**必须**加入 `MODULES` 数组中。
- 当使用 **encrypt** 或 **sd-encrypt** 钩子时,目标系统和运行*mkinitcpio*系统不同时，键盘模块和/或启动系统期间需要解锁LUKS设备的文件系统模块加入 `MODULES` 数组。例如，如果需要使用位于一个ext2系统的钥匙文件，但是运行*mkinitcpio*的系统中ext2文件系统没有被挂载，增加`ext2`。参见 dm-crypt/System configuration#cryptkey 获得更多细节。
- 如果使用USB3接口的键盘，希望用它来解锁LUKS设备。增加`usbhid xhci_hcd`。

- 如果使用显示器连接扩展坞，可能需要加入你的显卡模块使初始化输出信息可见(例如，为大多数英特尔显卡加入 `i915`)。

二进制文件和普通文件（BINARIES、FILES）

这两个选项允许用户添加任何文件到映像中。`BINARIES`、`FILES`数组在钩子运行之前指定了要加入内存盘映像的文件，可以覆盖钩子扩展提供的文件。`BINARIES`中的二进制文件会自动放入一个标准的`PATH`路径，而且会自动加入可执行文件依赖的函数库。`FILES`中的文件则不进行上述处理，*as-is*直接原样放入映像。例如：

```
FILES=(/etc/modprobe.d/modprobe.conf)
BINARIES=(kexec)
```

注意 `BINARIES` 和 `FILES` 是 Bash数组，配置支持多个选项，用空格隔开。

钩子（HOOKS）

`HOOKS`设置是此文件中最重要的设置。钩子是一系列的小脚本，描述哪些东西需要加入到映像里面。有些钩子还包含了运行组件，可以提供额外的行特定动作，像启动服务、构建存储栈的块设备等。钩子按照配置文件中`HOOKS`项中给出的顺序执行。

默认的`HOOKS`可以满足大部分简单的单硬盘系统。对于堆栈的或多块设备像 LVM, RAID, or dm-crypt 等复杂根分区系统，请查看相应的 Wiki 页面，进行必要的进一步设置。

构建时钩子

构建时钩子位于`/usr/lib/initcpio/install`，自定义的构建时钩子可能位于`/etc/initcpio/install/`。这些文件会在运行 mkinitcpio 时被包含进脚本，应该提供如下两个函数：`build` 和 `help`。`build` 函数描述了需要加入映像的模块、文件和二进制文件。mkinitcpio(8) 中的记录的 API 帮助添加这些项目。`help` 函数在钩子执行后输出描述信息。

列出可用的钩子列表：

```
$ mkinitcpio -L
```

使用 mkinitcpio 的 `-H`/`--hookhelp` 选项输出对于某一钩子的帮助。例如：

```
$ mkinitcpio -H udev
```

运行时钩子

运行时钩子可以在`/usr/lib/initcpio/hooks`中找到自定义的运行时钩子在`/etc/initcpio/hooks/`。 对于任何运行时挂钩，都应该始终有一个同名的构建钩子，它会调用`add_runscript`将运行时挂钩添加到映像中。 这些文件是由早期用户空间中的BusyBox ash shell调用。除清理钩子以外（清理钩子是倒序执行的），它们始终将按照`HOOKS`设置中列出的顺序运行。 运行时钩子可能包含以下函数：

`run_earlyhook`: 一旦挂载了API文件系统并且已经解析了内核命令行，便会运行这个函数。 通常，从此处启动早期引导过程所需的额外守护程序（例如udev，它从早期引导过程的开始就被需要）。

`run_hook`: 这个函数在早期钩子挂接后不久便运行。 这是最常见的钩子特点，应在此处进行诸如堆叠块设备的组装之类的操作。

`run_latehook`: 此名称的函数在挂载根设备后运行。 应当谨慎地将其用于根设备的进一步设置，或用于挂载其他文件系统，例如`/usr`。

`run_cleanuphook`: 该名称的函数将尽可能晚地运行，并且以与配置文件的`HOOKS`数组中列出函数的相反顺序来运行。 这些钩子应该用于所有的最终清理操作，例如关闭由早期钩子启动的所有守护程序。

注意：运行时钩子仅仅被BusyBox的init使用。**systemd** 钩子触发一个 systemd 基础的初始化, 它不需要运行任何运行时钩子，而是使用systemd单元。

映像后处理钩子

映像后处理钩子（post hook）是位于 `/usr/lib/initcpio/post/`（软件包提供）和 `/etc/initcpio/post/`（自定义）目录下的可执行文件或 shell 脚本。这些文件会在（重新）生成映像后执行，以执行签名等附加任务。

每个可执行文件会按以下顺序接收参数：

1. 使用的**内核**（某些情况下可能为空）；
2. 生成的**initramfs 映像**位置；
3. （可选）生成的**统一内核映像**位置。

此外，还会设置以下环境变量：`KERNELVERSION` 表示完整内核版本, `KERNELDESTINATION` 表示内核要想正常启动应位于的默认位置。

常用钩子

下面是一个常见钩子和它们如何影响映像的创建以及运行时的次序。注意这个表格是不完整的，因为一些包会提供一些自定义的钩子。

BusyBox init	systemd init	构建时钩子	运行时钩子（仅 BusyBox init）
base	设置所有初始目录并安装基础工具和库。除非清楚自己在做什么，否则请始终将此钩子置于首位，因为如果没有 **systemd** 钩子，该钩子是必需的，提供关键的 BusyBox init 程序。 当使用 **systemd** 钩		

BusyBox init	systemd init	构建时钩子	运行时钩子（仅 BusyBox init）
	子时，此钩子可选，仅提供一个 BusyBox 恢复 shell。启用 **base** 后，在内核参数中临时添加 `SYSTEMD_SULOGIN_FORCE=1` 使用该 shell。		

若在此之前 **autodetect** 钩子已调用，则只会添加对应处理器的微码更新文件。

此挂钩已经取代了弃用的`--microcode`参数和在预设文件中的`microcode`选项。这使得启动配置中微码的`initrd`参数行可以删除了，因为它们已被打包进主要的 initramfs 映像中。

注意：使用不同的硬件配置启动的系统（例如笔记本电脑连接外置键盘和内置键盘或headless systems）时，此钩子需要放在**autodetect**之前才能在引导中使用键盘，例如使用`encrypt`钩子解密加密设备。

注意：启动时反复确认*keyboard*钩子放在解密设备前面。和/或当使用钥匙文件解锁时，存放钥匙文件的文件系统加入#模块（MODULES） 里。

sd-encrypt 参见 dm-crypt/系统配置#使用systemd-cryptsetup-generator。

使用这个钩子需要在内核参数加入 `rw` 参数（讨论）。参见 fsck#启动时检查。

压缩方式(COMPRESSION)

内核支持几种initramfs的压缩方式 - gzip包, bzip2包, lzma, xz包, lzo包, lz4包 and zstd包。mkinitcpio默认使用zstd压缩映像，注意zstd运行的多线程模式（使用了`-T0`选项，以侦测到内核数量作为线程数量。）

系统中的 `mkinitcpio.conf` 已经注释掉了各种 `COMPRESSION` 选项，要使用某个压缩方式，请取消前面的注释，并确认安装了相应的压缩工具包。不指定 `COMPRESSION` 选项会使用 zstd 压缩的 initramfs 文件。想要创建一个未压缩的映像，在配置中指定`COMPRESSION=cat` 或者在命令行使用 `-z cat`。

提示：lz4 在高压缩率模式(`-9`)下映像有2.5的压缩比率，以付出最慢的单线程压缩速度为代价，可以获得最快的解压缩速度。lzo使用轻度较好的压缩模式后仍然有较快的解压速度。 zstd 提供多样解决方案, 通过参数使用多线程压缩方式和大范围压缩水平，参见 zstd(1) § Operation modifiers。 xz在其高压缩预设(`-9`)中以大约5的缩减因子实现了最小的体积，但代价是解压速度要慢得多。

压缩选项(COMPRESSION_OPTIONS)

还有一些额外的标准可以传递到 `COMPRESSION` 指定的程序， 例如：

```
COMPRESSION_OPTIONS=(-9)
```

这个选项可以留空， *mkinitcpio* 会确保任何支持的压缩方法有必要的标志来产生一个可以工作的映像。

警告：使用错误的参数选项可能导致**内核不能引导**，因为错误参数导致内核不能解压生成的压缩档。

使用默认的 zstd 压缩时，`--long`选项在节省自定义内核的空间（特别是在使用 EFI 系统分区作为`/boot`时的双系统设置）非常有效。但是，内存有限的系统可能无法使用此选项解压缩 initramfs。还可能需要`-v` 选项来查看 initramfs 生成期间的详细信息。例如：

```
COMPRESSION="zstd"
COMPRESSION_OPTIONS=(-v -5 --long)
```

通过使用 xz 的 `-9e` 压缩级别并解压可加载内核模块和固件，可以实现最高（但最慢）的压缩：

```
COMPRESSION="xz"
COMPRESSION_OPTIONS=(-9e)
MODULES_DECOMPRESS="yes"
```

模块解压缩(MODULES_DECOMPRESS)

`MODULES_DECOMPRESS` 控制内核模块与固件文件是否在 initramfs 创建时解压缩。默认为 `no`。

Arch 使用 19 级 zstd 压缩内核模块和 linux-firmware包。 当为 initramfs 使用更高的压缩等级，设置`MODULES_DECOMPRESS="yes"` 可以更大幅度地减少 initramfs 的大小。这会导致在早期启动阶段更高的内存和 CPU 占用，对于内存有限、CPU 性能不足的系统有影响，因为内核将花费更多的时间来解压缩整个 initramfs 映像，而不是加载单个模块和固件时解压缩它们。

提示：在 initramfs 生成过程接近结束时，所有剩余的 *.bz2*、*.gz*、*.lz4*、*.lzma*、*.lzo*、*.xz* 和 *.zst* 文件被移动到早期未压缩的 initramfs 映像以避免双重压缩。

运行时配置

运行时配置选项可以通过内核命令行传递到 `init` 以及某些钩子。内核命令行参数通常是由启动引导器提供。下面讨论的参数可以附加到内核命令行以改变默认行为。参见 Arch 启动过程和内核参数以获取更多信息。

从基本钩子启动

`root=`：这是内核命令行中指定的最重要的参数，因为它决定了哪一个设备会被挂载为你的正确的 root 设备。mkinitcpio 允许不同的格式，详见 Persistent_block_device_naming#Kernel_parameters

注意：下面的选项会改变 `init` 在 initramfs 环境中的默认行为。参见 `/lib/initcpio/init` 获取更多信息。但当启用`systemd`钩子时，这些参数不起作用，因为来自`base`钩子的 `init` 被取代了。

`break`：如果`break` 或者 `break=premount` 被指定， `init` 会暂停启动过程（在加载钩子之后，但是在挂载根文件系统之前）然后启动一个交互式终端，一般用来排

障。如果指定为`break=postmount`，这个终端会在根目录挂载后启动。退出这个 shell 之后可以继续正常的启动流程。

`disablehooks=`：在运行时可以通过添加 `disablehooks=hook1{,hook2,...}` 禁用某些钩子。例如

```
disablehooks=resume
```

`earlymodules=`：改变模块加载的顺序。可以通过 `earlymodules=mod1{,mod2,...}` 指定一些模块提前加载。 (例如，这可以用来确保多个网络接口的正确顺序。)

参见: 常规故障排除#系统启动问题和 mkinitcpio(8)。

使用 RAID 磁盘阵列

参见 RAID#Configure mkinitcpio。

使用 net

警告：NFSv4 仍然不被支持 FS#28287。

必须的包

`net` 需要 mkinitcpio-nfs-utils包 包。

内核参数

```
内核文档包含最新的信息和清晰的说明。
```

ip=

这个参数告诉内核怎样生成设备IP地址和怎样安装路由表。它最多可选9个参数：`ip=<client-ip>:<server-ip>:<gw-ip>:<netmask>:<hostname>:<device>:<autoconf>:<dns0-ip>:<dns1-ip>:<ntp0-ip>`。

如果这些参数在内核命令行缺失，所有区域将被认为是空白，则根据 kernel documentation 使用默认参数。通常这就意味这内核利用自动配置表来配置。

`<autoconf>` 参数单独作为 `ip` 的值出现 (没有上面提及的任一参数)。如果值是 `ip=off` 或 `ip=none`, 将不会执行自动配置，否则会执行自动配置。最常见的是 `ip=dhcp`.

获取参数具体说明，参见 kernel documentation.

例子：

```
ip=127.0.0.1:::::lo:none  --> 启用 loopback 接口。
ip=192.168.1.1:::::eth2:none --> 启用静态 eth2 接口。
```

`ip=:::::eth0:dhcp --> 为 eth0 启用 dhcp。`

注意：请确保使用内核设备命名 (例如 *eth0*) 作为 `<device>` 参数, 不要使用 网络配置#Network interfaces 命名 (例如 *enp2s0*) 因为这个不起作用。

BOOTIF= 使用多个网卡的时候，此参数可以指定要使用网卡的 Mac 地址。在接口号可能变化或与 IPAPPEND 2 、IPAPPEND 3 选项共用时比较有用。默认使用 eth0.

示例：

```
BOOTIF=01-A1-B2-C3-D4-E5-F6  # 注意前缀加"01-" 并且字母要大写。
```

nfsroot=

如果命令行中没有给出 `nfsroot` 参数，那么默认的 `/tftpboot/%s` 会被使用。

```
nfsroot=[<server-ip>:]<root-dir>[,<nfs-options>]
```

运行`mkinitcpio -H net` 查看参数详细解释。

使用 lvm

如果你的根分区在 LVM文件系统中, 参见 Install Arch Linux on LVM#Adding mkinitcpio hooks.

使用加密根目录

如果使用加密根目录，请参考 Dm-crypt/System configuration#mkinitcpio。

/usr 放到单独分区

如果将 /usr 放在单独分区，必须满足：

- 添加 `fsck` 钩子, `/usr`在`/etc/fstab`分区表中的`passno` 参数设置为`2` ，以便启动时检查该分区。作为常规推荐 如果想要`/usr`分区可以在启动时被检查，以上设置是中肯的。没有这个钩子 `/usr` 永远不能被检查（fsck）。
- 如果没使用systemd钩子，请添加`usr`钩子。这样将会在挂载根系统后挂载`/usr` 分区。

提示与技巧

生成 fallback initramfs 映像

自 mkinitcpio 版本 40 起，默认不生成 fallback initramfs 映像。若需要启用：

- 调整 `/etc/mkinitcpio.d/` 目录下相应的 **.preset** 文件：
- 注释掉 `PRESETS=('default')` 行，并取消注释 `PRESETS=('default' 'fallback')` 行
- 取消注释 `fallback_image="/boot/initramfs-linux-fallback.img"`

- 取消注释 `fallback_options="-S autodetect"`
- 重新生成 initramfs。
- 更新引导加载程序配置。

警告：缺少后备 initramfs 可能会使您在默认 initramfs 启动失败时，失去另一个启动系统的选项。请务必手头准备一个可启动的安装介质以备救援之用。

疑难解答

解压缩映像

如果你对 initrd 映像的内容感到好奇，你可以解压缩它然后看看里面的文件。

initrd 映像是一个 SVR4 CPIO 压缩包，通过 `find` 和 `bsdcpio` 命令生成，利用可被内核理解的压缩方式被有选择的压缩，参考#压缩方式(COMPRESSION))。

mkinitcpio 包含了一个叫做 lsinitcpio(1) 的工具，可以列出和/或解压缩出 initramfs 映像的内容。

你可以用下面的命令列出映像中的文件：

```
# lsinitcpio /boot/initramfs-linux.img
```

然后全部解压缩到当前文件夹：

```
# lsinitcpio -x /boot/initramfs-linux.img
```

你也可以用更对人类友好的方式列出映像中的重要的部分：

```
# lsinitcpio -a /boot/initramfs-linux.img
```

重新压缩解压之后修改过的映像

激活`/usr/bin/mkinitcpio`脚本的 `build_image` 功能使用此参数：

```
build_image outfile compression
```

将会产生一个拥有`build_image`功能的新脚本，用上面的参数运行它（*outfile*输出文件名字，*compression*压缩工具），将会把当前目录下的内容压缩到一个名字为`*outfile*`的新文件里。

警告：建议在运行上面的命令产生一个新的`/boot/initramfs-linux.img`之前把现有的映像重命名作为备份，以便情况不妙时有后悔药吃。如果因为失误导致系统无法启动，你需要用fallback映像或者boot CD启动，然后运行mkinitcpio来overwrite掉之前做的更改，或者你自己修好它然后重新压缩映像。

"/dev must be mounted" when it already is

*mkinitcpio*使用这个检测通过判断`/dev/fd/` 是否存在来判断`/dev`是否被挂载。如果其他一切看起来都很好，可以通过以下方式手动“创建”：

```
# ln -s /proc/self/fd /dev/
```

(显然, `/proc` 必须被安装. *mkinitcpio* 无论如何要求这样,指示它要检测的下一件事。)

Possibly missing firmware for module XXXX

当内核更新后，initramfs映像被重新构建时，你可能得到以下警告：

```
==> WARNING: Possibly missing firmware for module: wd719x
==> WARNING: Possibly missing firmware for module: aic94xx
==> WARNING: Possibly missing firmware for module: xhci_pci
```

如果产生 *default*映像时出现这些或相近的信息，那么按照警告要求安装额外的被需要的固件。绝大部分固件通过安装安装 linux-firmware包包获得。其他提供固件的包可以查看下表或在官方仓库或 AUR 中搜索固件模块名字。

除此以外，警告信息只出现在 *fallback* 映像生成时，可以按照下面的建议来：

- 如果不需要使用缺失模块的硬件，可以安全地忽略这些警告。
- 如果想要消除这些警告，安装缺失的固件，元包 mkinitcpio-firmwareAUR 包括绝大部分可选的固件。也可以手动安装需要的包：

: | 模块 | 包 |

如果想去掉警告，但不想浪费磁盘空间在不需要的固件包上，可以禁用 #生成 fallback initramfs 映像。

对于不可用的固件，也可以通过创建占位文件来抑制警告，例如：

```
# echo "设备不可用" > /usr/lib/firmware/qat_420xx.bin
# echo "设备不可用" > /usr/lib/firmware/qat_420xx_mmp.bin
```

没有发现 PS/2 控制器

在一些主板（主要是早期的但是也有一些新的主板）， i8042 控制器不能被自动侦测到。这很罕见，但是一些人确定没有键盘使用。 可以提前侦测到这种情况。PS/2 接口得到 `i8042: PNP: No PS/2 controller found. Probing ports directly` 信息，加 **atkbd** 到 `MODULES` 数组。

标准急救过程

使用不正确的初始化内存映像盘经常出现不可引导。因此遵守以下的系统急救过程：

在一台机器引导成功，但在另一台引导失败

在主要映像 initramfs 扫描 `/sys` 时 *mkinitcpio*的 `autodetect` 钩子 过滤掉了 kernel modules ，这些模块在启动被调用。 如果转移`/boot` 目录到其他机器，在早期用户空间造成启动失败，原因是新的硬件由于缺失内核模块不能被侦测到。注意USB 2.0 和 3.0 需要不同的模块。

修正这个缺陷，首先试着从引导记录bootloader 中选择 fallback 映像 ，因为这个映像不使用`autodetect`过滤模块。 一旦引导成功，在新机器上运行*mkinitcpio*用正确的

模块重建主要映像。如果fallback映像失败，试着用 Arch Linux live CD/USB，使用 arch-chroot 进入系统中, 在新机器上运行 *mkinitcpio* 。最后的手段，手工增加模块到 \[#MODULES|手动\]\]映像中。

无法访问控制台，root 用户被锁定

`systemd` 钩子会禁用 root 用户。要启用紧急 shell，请临时在内核参数中添加 `SYSTEMD_SULOGIN_FORCE=1`。

或者，也可以使用 initcpio-hook-shadowcopyAUR。安装后，在 `/etc/mkinitcpio.conf` 中将 `shadowcopy` 钩子置于 `systemd` 之后，并使用 `mkinitcpio -P` 重新生成 initramfs。更多文档请参见其 GitHub 仓库。

另见

- Linux Kernel documentation on initramfs, "What is rootfs?"
- Linux Kernel documentation on initrd
- Wikipedia article on initrd

配置文件

同一处桌面表现，负责的文件往往不同：有的设置由常驻进程统一发布，有的应用只认自己的文本资源，启动器的图标与名称另有出处，程序把文件写往哪个目录也有约定。本章分别说明这几类配置的结构、用法与排错，让你动手前先确认该改哪里。

阅读上，XDG 基本目录与桌面项两节建议必读：前者规定配置和数据的存放位置及工具行为，后者决定应用项能否被正确识别与显示；X 资源、Xsettingsd 用时再查即可。常见误解是把默认目录当成硬性规则，或图标、菜单异常时只盯着桌面项——先把路径与目录约定对上，往往比改文件管用。

10.1　Xsettingsd

难度 入门 | **篇幅** 4.2 KB | **所属** 配置文件

本节将讲到

- 安装
- 配置
- 用法
- 问题解决

翻译状态：

- 本文（或部分内容）译自 Xsettingsd，最近一次同步于 2024-8-5，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Xsettingsd 是一个轻量级的 **xsettings** 守护进程，通过 XSETTINGS 规范为 Xorg 应用程序提供设置。

某些桌面环境（如默认的 KDE 或自定义）不包含此功能。在这些环境中，运行 *xsettings* 守护进程对于某些应用程序（主要是基于GTK、Java 和 Wine）使用选定的主题、光标、字体和其他设置是必要的。

安装

安装 xsettingsd包 或 xsettingsd-gitAUR。

配置

xsettingsd(1) 只包含简要介绍，详情请参见 README 。

X FreeType 字体渲染配置示例（您可以使用自己喜欢的 config 文件路径）：：

```
~/.config/xsettingsd/xsettingsd.conf
```

```
Xft/Antialias   1
Xft/DPI         98304
Xft/Hinting     1
Xft/HintStyle   "hintfull"
Xft/lcdfilter   "lcddefault"
Xft/RGBA        "rgb"
```

注意：

- `Xft/DPI`是您的 DPI 乘以 1024。
- 使用 real DPI（如*94\1024*）*的字体可能比使用 artificial DPI（如*96*1024*）*的字体更好看，但在应用程序中可能会出现故障（如弹出提示没有边框）。
- 虽然 *X resources* 和 *xsettings* 参数名称看起来相似，但它们都区分大小写。例如，应将 `Xft.dpi: 96` 放在 `~/.Xresources` 中，将 `Xft/DPI 98304` 放在 `~/.xsettingsd.conf` 中。

用法

启动 `xsettingsd.service` 用户单元。

该单元是*静态*的，因此无法直接启用。您可以在 Xorg、桌面环境或窗口管理器启动时自动启动它（或`xsettingsd`二进制文件）。

注意：安装了 kde-gtk-config包 的 KDE 将自动启动二进制文件。

该单元被配置为 `graphical-session.target` 的一部分，因此当 `graphical-session.target` 停止（重启）时，它也会停止（重启），请参阅 [systemd.unit(5) § \[UNIT\] SECTION OPTIONS] (https://man.archlinux.org/man/systemd.unit.5#%5BUNIT%5D_SECTION_OPTIONS) 和 systemd.special(7) § Special Passive User Units。

问题解决

无法打开到 X 服务器的连接

检查是否设置了 `DISPLAY` 和 `XAUTHORITY` 环境变量。

如果正在启动 systemd 单元，请检查 `systemctl --user import-environment DISPLAY XAUTHORITY` 是否已执行（可通过从`~/.xinitrc`调用`/etc/X11/xinit/xinitrc.d/50-systemd-user.sh`来完成）。

10.2 X 资源

难度 进阶 | **篇幅** 11.1 KB | **所属** 配置文件

本节将讲到

- 安装
- Configuration
- 用法

- Troubleshooting
- 参见

翻译状态：

- 本文（或部分内容）译自 X resources，最近一次同步于 2024-8-1，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

X 资源 文件是一个用户级配置*点文件*，通常位于 `~/.Xresources`。 它可用于设置X资源，即 X 客户端程序的配置参数。

除其他用途外，它们可用于：

- 配置终端首选项（如终端颜色）
- 设置 DPI、抗锯齿、hinting 和其他 X 字体设置
- 更改 Xcursor 主题
- 主题 XScreenSaver
- 配置低级 X 应用程序，如 xorg-xclock包、xpdf包、rxvt-unicode)

安装

安装 xorg-xrdb包 软件包来获得 *X 服务器资源数据库工具*，安装 xorg-docs包 来获得 X.org 文档。

Configuration

X(7) § RESOURCES and XrmGetDatabase(3) § FILE SYNTAX provide detailed information on *X resources* mechanism and file syntax.

`~/.Xresources` is a conventional file name, `xrdb` does not claim it. You can use any other file names, like `~/.config/X11/Xresources` and `~/.config/X11/Xresources.d/*application-name*` (also see #Samples and #Include files).

Basic syntax

The syntax of an *X resources* file is a sequence of *resource lines* as follows:

```
application_name.Class.resourceName: value
      application_name.resourceName: value
                  Class.resourceName: value
      application_name*resourceName: value
                      *resourceName: value
```

`application_name` and `Class` substrings will never contain a dot (`.`), the `resourceName` substring may contain a dot. For example, `Dialog.bodyFont` is a XScreenSaver internal resource that is specified to set the body font and fallback font:

```
xscreensaver-auth.default.Dialog.bodyFont: times new roman 12, dejavu serif 12
```

application_name : The name of the application, such as `urxvt`, `xpdf`, `xterm`, etc. Also may be called the *instance name*.

Class : The classification used to group resources together. Class names are typically uppercase.

resourceName : The name of the resource whose value is to be set. Resources are typically lowercase with uppercase concatenation.

value : The actual value of the resource. This can be one of three types:

- Integer (whole numbers)
- Boolean (true/false, yes/no, on/off)
- String (a string of characters) — for example a word (`white`), a color (`#ffffff`), or a path (`/usr/bin/firefox`)

delimiters

- A dot (`.`) is a *tight binding* and is used to separate immediately adjacent components (in other words, to signify each step down into the hierarchy) — in the above example we start at application name, then descend into Class, and finally into the resource itself.
- An asterisk (`*`) is a *loose binding* and is used to represent any number of components, including none.
- A colon (`:`) is used to separate the resource name from the value.

注意：

- From resource file syntax point of view, everything before a colon (`:`) is a resource name. From the user perspective, we often call *resource* only the rightmost component. In other words, *resource name* is a string consisted of *application name*, *Class* and *resource name* substrings. That might be the source of confusion.
- Resource naming schema is totally application-dependent. While one application might use `application_name.Class.resourceName`, another might understand `application_name.resourceName` and `Class.resourceName` only.

Wildcard matching

Question mark (`?`) and asterisk (`*`) can be used as wildcards, making it easy to write a single rule that can be applied to many different applications or elements. `?` is used to match any single component name, while `*` is used to represent any number of intervening components including none.

Using the previous example, if you want to apply the same font to all programs (not just XScreenSaver) that contain the class name `Dialog` which contains the resource name `headingFont`, you could write:

```
?.Dialog.headingFont:     -*-fixed-bold-r-*-*-*-100-*-*-*-*-iso8859-1
```

If you want to apply this same rule to all programs that contain the resource `headingFont`, regardless of its class, you could write:

```
*headingFont:    -*-fixed-bold-r-*-*-*-100-*-*-*-*-iso8859-1
```

注意：

- Question mark (`?`) is like component name, so *binding character* — dot (`.`) or asterisk (`*`) — next to it is required.
- Asterisk (`*`) is a *binding character* itself, there is no need in placing dot next to it, and a sequence of dot(s) and asterisk(s) is replaced with a single asterisk during file processing.

See XrmGetResource(3) § MATCHING RULES for more information.

Comments

Lines starting with an exclamation mark (`!`) are ignored, for example:

```
! The following rule will be ignored because it has been commented out
!Xft.antialias: true
```

注意：

- The exclamation mark must be the first character on the line.
- If you have a C preprocessor, such as GNU CPP (gcc包), installed you can use C\-style (`//` and `/* … */`) comments (also see #Samples).

Include files

To spread resource configuration across multiple files (e.g. to use its own file for each application), use *C preprocessor* `#include` *directive*:

```
~/.config/X11/Xresources
#include "Xresources.d/fonts"
#include "Xresources.d/rxvt-unicode"
#include "Xresources.d/xscreensaver"
#include "Xresources.d/xterm"
```

If files that are referenced with *#include* are not reachable from the applied configuration file directory, you need to pass a directory to search for:

```
$ xrdb -load -I${HOME}/.config/X11 ~/.Xresources
```

Default settings

To see the default settings for your installed X11 applications, look in `/usr/share/X11/app-defaults/`.

Detailed information on program-specific resources is usually provided in the man page for the program. xterm(1) § RESOURCES is a good example, as it contains a list of X resources and their default values.

To see the currently loaded resources:

```
$ xrdb -query -all
```

Samples

- Color output in console#Terminal emulators
- Cursor themes#X resources
- Font configuration#Applications without fontconfig support
- rxvt-unicode#Configuration
- xpdf(1) § OPTIONS
- Xterm#Configuration
- klassiker (mrdotx) — Rxvt-unicode patch developer dotfiles

用法

加载资源文件

资源存储在 X 服务器中，因此只需读取一次。*远程*X11 客户端（如通过 SSH 转发）也可以访问这些资源。

加载资源文件（如传统的 `.Xresources`），替换任何当前设置：

```
$ xrdb ~/.Xresources
```

加载资源文件，并与当前设置合并：

```
$ xrdb -merge ~/.Xresources
```

注意：

- 大多数显示管理器会在登录时加载 `~/.Xresources` 文件。
- X11 程序启动时会读取较旧的 `~/.Xdefaults` 文件，但前提是当前会话中未使用过 *xrdb*。

xinitrc

如果使用默认的 xinitrc 副本作为 `.xinitrc`，它已经合并了 `~/.Xresources`。

如果您使用的是自定义的，请添加：

```
.xinitrc
[[ -f ~/.Xresources ]] && xrdb -merge -I$HOME ~/.Xresources
```

注意：请勿在 `~/.xinitrc` 中设置 xrdb 命令的后台。否则，在 xrdb 之后启动的程序可能会在加载完成前查找资源。

获取资源值

如果要获取某个资源的值（例如在 bash 脚本中使用），可以使用 xgetresAUR：

```
$ xgetres xscreensaver.Dialog.headingFont
-*-fixed-bold-r-*-*-*-100-*-*-*-*-iso8859-1
```

Troubleshooting

Parsing errors

Display managers such as GDM may use the `--nocpp` argument for *xrdb*.

No output from xrdb -query

It is not rare for `xrdb -query` to output nothing. Try following #Load resource file and #xinitrc from above. And note some of the files mentioned there could be empty.

参见

- Using the .Xdefaults File 由 Purdue Engineering Computer Network 提供
- Overlooked Points of X Resources 由 Thomas Dickey 提供

10.3 桌面项

难度 高级 | **篇幅** 23.5 KB | **所属** 配置文件

本节将讲到

- 基础
- 用法
- 应用程序项
- 图标
- 工具

XDG 桌面配置项规范定义了一个标准，用于在实现了XDG 桌面菜单规范的桌面环境里将应用程序集成到应用程序菜单中。（译注：下文中“XDG 桌面配置项”简称为“桌面项”）

基础

每个桌面项必须具有一个 `Type` 关键字和一个 `Name` 关键字。可以定义桌面项在应用程序菜单中的外观（非必须）。

桌面项可用的有效类型有三种：

Application：定义如何启动应用程序以及它支持的 MIME 类型（由 XDG MIME Applications 使用）。 使用 XDG Autostart 应用程序项可以通过将它们放在特定目录中以实现自动启动。应用程序项使用 *.desktop* 扩展名。 请参阅 #应用程序项。

Link：定义 `URL` 的快捷方式。链接项使用 `.desktop` 扩展名。

Directory：定义应用程序菜单中子菜单的外观。目录项使用 *.directory* 扩展名。

以下部分将大致解释这些是如何创建和验证的。

用法

使用像 dex包 这样的桌面启动器。

```
$ dex /usr/share/applications/firefox.desktop
```

桌面环境	启动器 （软件包）	Autostart	Application	Link
Cinnamon	*gio* (glib2包)	否	是	否
GNOME				
GNOME Flashback				
MATE				
Deepin	*dde-open*1 (deepin-api包)	是	部分	
Enlightenment	*enlightenment_open* (enlightenment包)			
KDE Plasma	*kde-open*2 (kde-cli-tools包)	是	是	是
LXDE	*pcmanfm* (pcmanfm包)			
LXQt	*pcmanfm-qt* (pcmanfm-qt包)			
Xfce	*exo-open*3 (exo包)			
环境无关	*gtk-launch*4 (gtk3包)	否	部分	否
dex (dex包)	是	是	否	
xdg-open (xdg-utils包)	否	存在 bug	否	

1. *dde-open* 用于启动 **Application** 类型的桌面项时要求具有可执行权限（`chmod +x`）。
2. KDE Plasma 桌面通常设置 `KDE_SESSION_VERSION` 环境变量用于识别会话环境。如果未设置，系统会回退使用 konqueror包 套件中的 *kfmclient*。
3. *gio*(源于 glib2包) 是 GNOME 相关的通用 I/O 工具，可以用来打开文件或应用。它会被用作另一个备用方法。
4. 前提是应用程序的 .desktop 文件位于 `/usr/share/applications` 目录中

应用程序项

应用程序的桌面项或 *desktop* 文件通常是元信息资源和应用程序快捷方式的组合。对于系统范围内安装的应用程序，这些文件通常位于 `/usr/share/applications/` 或 `/usr/local/share/applications/`。而 `~/.local/share/applications/` 用于用户特定的应用程序。后者（用户条目）优先于前者（系统条目）。

文件示例

下面是一个带有附加注释的结构示例。 这个例子只是为了给人一个快速的印象，并没有展示如何使用所有可能的关键字。完整的关键字清单可在 freedesktop 规范中找到。

```
[Desktop Entry]

# type关键字如上所述
Type=Application

# 本文件所遵循的桌面项规范版本
Version=1.0
```

```
# 应用程序的名称
Name=jMemorize

# 显示为工具提示的注释
Comment=Flash card based learning tool

# 可执行文件所在的目录
Path=/opt/jmemorise

# 可执行文件，可以带参
Exec=jmemorize

# 图标名称
Icon=jmemorize

# 应用程序是否需要运行在终端中
Terminal=false

# 本桌面项将显示在哪些分类中
Categories=Education;Languages;Java;
```

注意：只有`Type`和`Name`是必须的。

关键字赋值

所有可用的关键字都可以在 freedesktop 网站上找到。

例如， `Type` 关键字定义了三种类型的桌面项：应用程序（type 1）、链接（type 2）和目录（type 3）。

- `Version` 关键字不代表应用程序的版本，而是代表该文件符合的桌面项规范的版本。
- `Name`、`GenericName` 和 `Comment` 通常包含它们组合形式的冗余值，例如：

```
Name=Pidgin Internet Messenger
GenericName=Internet Messenger
```

或者

```
Name=NoteCase notes manager
Comment=Notes Manager
```

应该避免这种情况，因为它只会让用户感到困惑。`Name`关键字应仅包含名称，或者可能是缩写/首字母缩略词（如果可用）。

- `GenericName` 应该说明您通常如何描述该应用程序提供的功能（即 Firefox 是“Web 浏览器”）。
- `Comment` 旨在包含任何有用的附加信息。

验证

随着时间的推移，某些关键字已被弃用，您可能需要使用 desktop-file-validate(1) 来验证您的桌面项，它是 desktop-file-utils包 软件包的一部分。要验证，请运行：

```
$ desktop-file-validate <your desktop file>
```

这将为您提供非常详细且有用的警告和错误消息。

安装

使用 desktop-file-install(1) 将桌面文件安装到目标目录。例如：

```
$ desktop-file-install --dir=$HOME/.local/share/applications ~/app.desktop
```

这对于通过编辑选项自定义已有的桌面项（例如来自 `/usr/share/applications` 的文件）也很有用。

刷新桌面项数据库

通常，桌面环境会自动识别对桌面启动项的更改。 如果没有自动生效，而你希望强制更新定义在 `~/.local/share/applications` 中的桌面启动项，请运行以下命令：

```
$ update-desktop-database ~/.local/share/applications
```

提示：添加 `-v`（详细）参数以显示可能的桌面输入错误。

图标

参阅图标主题规范。

通用图标格式

以下是图标常用图像格式的概述。

<table class="wikitable" align="center"><caption>支持 freedesktop.org 标准指定的图标图像格式</caption><tbody><tr><th>扩展名</th><th>全称和/或说明</th><th>图像类型</th><th>是否容器型</th><th>是否支持</th></tr><tr><th align="left">.png</th><td>便携式网络图形</td><td>位图</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td></tr><tr><th align="left">.svg(z)</th><td>可缩放矢量图形</td><td>矢量图</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是（可选）

</td></tr><tr><th align="left">.xpm</th><td>X PixMap</td><td>位图</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td><td data-sort-value="3" class="archwiki-table-cell-background-yellow" style="vertical-align: middle; text-align: center;">是（不推荐）</td></tr><tr><th align="left">.gif</th><td>图像互换格式</td><td>位图</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td></tr><tr><th align="left">.ico</th><td>Windows图标格式</td><td>位图</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td></tr><tr><th align="left">.icns</th><td>Apple图标格式</td><td>位图</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td></tr></tbody></table>

图标格式转换

**本文或本章节可能需要合并到 ImageMagick#Usage**。

附注： 应归属该文。（在 Talk:桌面项 中讨论）

如果您偶然发现 freedesktop.org 标准不支持的图标格式（例如 `gif` 或 `ico`），您可以使用“magick”工具 （这是 imagemagick包 包的一部分）将其转换为支持/推荐的格式，例如：

```
$ magick <icon name>.gif <icon name>.png
```

如果您从 `ico` 之类的容器格式转换，您将获得以 `<icon name>-<number>` 形式封装在 `ico` 文件中的所有图像。如果你想知道图像的大小，或者像 `ico` 这样的容器文件中的图像数量，你可以使用 *identify* 工具（它也是 imagemagick包 的一部分）：

```
$ identify /usr/share/vlc/vlc48x48.ico
/usr/share/vlc/vlc48x48.ico ICO 32x32 32x32+0+0 8-bit DirectClass 84.3kb
/usr/share/vlc/vlc48x48.ico ICO 16x16 16x16+0+0 8-bit DirectClass 84.3kb
/usr/share/vlc/vlc48x48.ico ICO 128x128 128x128+0+0 8-bit DirectClass 84.3kb
/usr/share/vlc/vlc48x48.ico ICO 48x48 48x48+0+0 8-bit DirectClass 84.3kb
/usr/share/vlc/vlc48x48.ico ICO 32x32 32x32+0+0 8-bit DirectClass 84.3kb
/usr/share/vlc/vlc48x48.ico ICO 16x16 16x16+0+0 8-bit DirectClass 84.3kb
```

如您所见，示例 *ico* 文件，虽然它的名称可能暗示一个大小为 48x48 的图像，但包含不少于 6 个不同的大小，其中一个甚至大于 48x48，即 128x128。

或者，您可以使用 *icotool*（来自 icoutils包）从 ico 容器中提取 png 图像：

```
$ icotool -x <icon name>.ico
```

要从 .icns 容器中提取图像，可以使用 *icns2png*（由 libicns包 提供）：

```
$ icns2png -x <icon name>.icns
```

获取图标

附带“.desktop”文件的软件包肯定包含一个或一组图标。有时开发人员虽然没有创建“.desktop”文件，但可能会附带图标。所以一个好的开始是在源包中寻找图标。

您可以首先使用 **find** 过滤扩展名，然后使用 **grep** 进一步过滤某些常用关联词，如包名、“icon”、“logo”等，源码包中可能有很多图片。

```
$ find /path/to/source/package -regex ".*\.\(svg\|png\|xpm\|gif\|ico\)$"
```

如果应用程序的开发人员没有在他们的源包中包含图标，那么下一步就是在他们的网站上进行搜索。

一些项目，例如 tvbrowserAUR 有一个艺术品/徽标页面，可以在其中找到其他图标。如果一个项目是多平台的，可能会出现这样的情况，即使 linux/unix 包没有附带图标，Windows 包也可能提供图标。如果项目使用版本控制系统，如 CVS/SVN/等。并且您对它有一些经验，您也可以考虑浏览它以获取图标。

如果一切都失败了，该项目可能还没有图标/徽标。

图标文件路径

freedesktop.org 标准规定了程序应以何种顺序和目录查找图标：

1. `$HOME/.icons`（为了向后兼容）
2. `$XDG_DATA_DIRS/icons`
3. `/usr/share/pixmaps`

工具

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Use a list of Template:App entries instead of short sections.（在Talk:桌面项讨论）

Arronax

Arronax 是一个图形化程序，用于创建和修改应用程序或位置的桌面启动项。要使用它，请安装 arronaxAUR 软件包。

Alacarte

alacarte包 是一个用于 GNOME 的图形化菜单编辑器，遵循 freedesktop.org 的菜单规范。它也支持重写（覆盖）桌面启动项。

jdDesktopEntryEdit

jddesktopentryeditAUR 是一个用于编辑 Qt 桌面启动项的图形化程序。

MenuLibre

menulibreAUR 是一个使用 GTK 的图形化菜单编辑器，提供现代化功能，并具有简洁易用的界面。

LibreMenuEditor

libre-menu-editorAUR 是一个用于编辑桌面启动项的图形程序，旨在功能丰富的同时也易于上手。

它基于 GTK 和 libadwaita 开发，遵循 GNOME 的界面设计规范，但设计上可在任何符合 freedesktop.org 标准的桌面环境中使用。

gendesk

gendesk包 最初是一个 Arch Linux 特定的工具，用于通过直接从 PKGBUILD 文件中获取所需信息来生成 .desktop 文件。 现在它是一个接受命令行参数的通用工具。

可以从这个网站自动下载图标（如果网站可用的话）。（图标的来源是可配置的）。

用法

- 把 `gendesk` 添加到 makedepends
- 启动 `prepare()` 功能：

```
gendesk --pkgname "$pkgname" --pkgdesc "$pkgdesc"
```

- 或者，如果已经提供了一个图标（例如，$pkgname.png），可以指定 `-n` 参数限定不下载图标或使用默认图标。例如：

```
gendesk -n --pkgname "$pkgname" --pkgdesc "$pkgdesc"
```

- `$srcdir/$pkgname.desktop` 将被创建并安装在 `package()` 函数中：

```
install -Dm644 "$pkgname.desktop" "$pkgdir/usr/share/applications/$pkgname.desktop"
```

- 图标可以这样安装：

```
install -Dm644 "$pkgname.png" "$pkgdir/usr/share/pixmaps/$pkgname.png"
```

- 用 `--name='程序名称'` 这个参数指定程序在菜单中的名称。
- 用 `--exec='/opt/some_app/elf --some-arg --other-arg'` 这种形式设置可执行文件字段。
- 更多信息请参阅 gendesk 项目。

lsdesktopf

lsdesktopfAUR可列出可用的“.desktop”文件或搜索它们的内容。

```
$ lsdesktopf
$ lsdesktopf --list
$ lsdesktopf --list gtk zh_TW,zh_CN,en_GB
```

它还可以执行 MIME 类型相关的搜索。参阅XDG MIME Applications#lsdesktopf。

fbrokendesktop

fbrokendesktopAUR Bash 脚本用于检测指向不存在路径的无效`Exec`值。没有任何参数，它使用 `DskPath` 数组中的预设目录。它仅显示完整路径和文件名丢失的无效“.desktop”。

例如：

```
$ fbrokendesktop
$ fbrokendesktop /usr
$ fbrokendesktop /usr/share/xsessions/icewm.desktop
```

提示与技巧

编辑 .desktop 文件

对于系统范围的 *.desktop* 文件（例如，从软件包安装的文件），首先将相关的 *.desktop* 文件从 `/usr/share/applications/` 复制到 `$XDG_DATA_HOME/applications/`（例如 `~/.local/share/applications/`）。 这可以防止在系统升级期间更新软件包时覆盖您的更改。 本地用户特定的“.desktop”文件应自动优先于系统范围的文件。 现在您可以根据需要修改本地用户特定的“.desktop”文件。

注意：如果您已将此应用程序配置为自动启动，则 `$XDG_CONFIG_HOME/autostart` 中可能已经存在系统桌面条目的副本，如 XDG Autostart。 如果您只想在自动启动时修改行为，请继续编辑此文件。 否则，如果您想在所有情况下修改行为，您应该：

- 将桌面文件从 `$XDG_CONFIG_HOME/autostart/` 移动到 `$XDG_DATA_HOME/applications`。

- 使用 `ls -l`，确保此桌面条目是您可以编辑的常规文件，而不是指向系统位置的链接。如果是链接，请按照上述说明将其替换为原件的副本。
- 建立一个指向用户应用程序目录的符号链接（为方便起见，默认 XDG 目录被替换）：

```
ln -s ~/.local/share/applications/desktop_entry ~/.config/autostart/
```

现在，您的应用程序启动器中的文件将与自动启动的文件保持一致。

编辑环境变量

要设置环境变量，在 *.desktop* 文件中，编辑 `Exec=` 行以首先使用 env(1) 命令设置变量。例如，将原始行注释掉：

```
~/.local/share/applications/abiword.desktop
...
# Exec=abiword %U
Exec=env LANG=he_IL.UTF-8 abiword %U
...
```

如果存在 `DBusActivatable=true` ，也应将其删除（或设置为 `false` ），否则会导致 `Exec` 行被忽略。

编辑命令行参数

要更改或添加命令行参数，请编辑 `Exec=` 行以附加所需的选项。 例如，将原始行注释掉：

```
~/.local/share/applications/steam.desktop
...
# Exec=/usr/bin/steam-runtime %U
Exec=/usr/bin/steam-runtime -no-browser %U
...
```

如果存在 DBusActivatable=true ，也应将其删除（或设置为 false ），否则会导致 Exec 行被忽略。

隐藏桌面项

可以通过多种方式控制桌面条目的可见性。有关详细信息，请参阅桌面条目规范。 将以下行之一添加到您的 *.desktop* 文件中：

- *All* 桌面环境，选择以下一项（或两项）：
- - 为您不希望在菜单中显示的应用程序添加行 `NoDisplay=true`。
- - 为您认为已删除且不希望在菜单中显示的应用程序添加行 `Hidden=true`。
- *Specified* 桌面环境，选择以下之一，其中 `*desktop_names*` 是以分号分隔的桌面环境列表（例如 `GNOME`、`GNOME ;Xfce;KDE;`）：
- - 添加行 `NotShowIn=*desktop_names*` 以“隐藏”仅在指定桌面环境中的条目。

- - 将行 `OnlyShowIn=*desktop_names*` 添加到仅在指定桌面环境中的“显示”条目中。

参阅

- Wikipedia:.desktop
- Information for developers

10.4 XDG 基本目录

难度 高级 | **篇幅** 147.1 KB | **所属** 配置文件

本节将讲到

- 规范
- 支持
- 工具
- 库
- 提示与技巧

此文章在#规范中总结了 XDG 基本目录规范，并在#支持中跟进软件支持情况。

规范

请阅读完整规范。本节将尝试分解其试图实现的核心内容。

默认情况下，只有 `$XDG_RUNTIME_DIR` 会由 pam_systemd(8) 自动设置，其他变量用户可根据规范自行定义。更改 `XDG_RUNTIME_DIR` 可能会导致 PipeWire 和 Chromium 上的屏幕共享出现问题。

有关变量如何定义，请参见环境变量#全局。

用户目录

- `$XDG_CONFIG_HOME`
- 单个目录，用于存放用户配置文件（类似 `/etc`）。
- 默认应使用 `$HOME/.config`。
- `$XDG_CACHE_HOME`
- 单个目录，用于存放非必要（缓存）的用户数据（类似 `/var/cache`）。
- 默认应使用 `$HOME/.cache`。
- `$XDG_DATA_HOME`
- 单个目录，用于存放用户数据文件（类似 `/usr/share`）。
- 默认应使用 `$HOME/.local/share`。
- `$XDG_STATE_HOME`
- 单个目录，用于存放用户状态文件（类似 `/var/lib`）。

- 默认应使用 `$HOME/.local/state`。
- `$XDG_RUNTIME_DIR`
- 单个目录，用于存放运行时用户文件或其他文件对象（如 sockets、命名管道等）。
- 不需要提供默认值；如果未设置且没有其他等效配置，则应发出警告。
- 必须由用户拥有，访问模式为 `0700`。
- 文件系统符合 OS 标准。
- 必须位于本地文件系统上。
- 可能会定期清理。
- 设置粘滞位或每 6 小时修改一次防止被清理。
- 只能在用户登录期间存在。
- 可能挂载为 tmpfs，故不应用于存储大文件。
- `pam_systemd` 将其设置为 `/run/user/$UID`。
- `$HOME/.local/bin`
- 普遍用于存放用户可执行文件。

注意：

- 没有 `$XDG_BIN_HOME` 目录。
- `$HOME/.local` 应类似 `/usr/local` 使用。
- `$HOME` 可能在不同架构的系统间共享，造成问题：将已编译的二进制文件安装到 `$HOME/.local/bin` 目录后，不能在架构不同的系统上使用。虽然这通常没有什么关系，但需谨记：若将编译好的二进制文件存放在 `$HOME` 中，该目录便部分地架构相关。

系统目录

- `$XDG_DATA_DIRS`
- 由 `:` 分隔的目录列表，按优先级排列（类似 `$PATH`），用于搜索数据文件。
- 默认应为`/usr/local/share:/usr/share`。
- `$XDG_CONFIG_DIRS`
- 由 `:` 分隔的目录列表，按优先级排列（类似 `$PATH`），用于搜索配置文件。
- 默认应为`/etc/xdg`。

支持

此节旨在记录日益增长的使用 XDG 基本目录规范（于 2003 年提出）的软件，通过列出常见的 dotfiles 及其支持状态来展示此规范的可用性。 对于那些目前不支持 XDG 基本目录规范的软件，此处给出可模拟该规范的变通方案。

变通方案仅限于那些不涉及修改源代码、在环境变量中执行代码或修改编译时选项的方法。不修改编译时选项是因为配置应该能够在系统之间移植，而编译时选项会妨碍这一点。

希望这能作为识别 dotfiles 到底是干什么的、是来自哪个软件的信息源。

贡献

贡献时，请确保章节分类正确。

任何变通方案都不应需要修改源代码、在环境变量中执行代码或修改编译时选项，任何需要这些方法才能获得支持的软件都必须分类为“硬编码”。 此外，如果过程容易出错或困难，也应分类为“硬编码”。

- “应用程序”列应该使用指向本 wiki 条目的链接、模板:Pkg 或 模板:AUR。
- “旧路径”列是项目曾经拥有的所有遗留文件和目录（每行一个），即使这些文件或目录不再被读取，也应保留以便查找。
- 尽量找到项目采用 XDG 基本目录规范的首个 commit 或版本以及任何相关的公开讨论，分别收入“采用规范自”列和“讨论”列（每行两个）。
- “注释”列应包括适当的变通方案或解决方案。请确认提供的方案正确且可行。

完整支持

应用程序	旧路径	采用规范自	讨论	注释
act包	`~/.actrc`	1656 2195		 `XDG_CONFIG_HOME/act/actrc`

如果 `~/.actrc` 存在则会与XDG规范路径下的配置文件合并。

Google 的配置文件位置概述中没有提到XDG——路径有可能是硬编码的，尽管这种情况不太可能发生，因为 Android Studio 所基于的 Intellij IDEA 正确实现了本规范。

`$XDG_CONFIG_HOME/audacity` `$XDG_DATA_HOME/audacity`

优先遵循 `$BITWARDENCLI_APPDATA_DIR` 环境变量。

该目录包含一个存放所有保险库数据的 `data.json` 文件，理应存放到 `$XDG_DATA_HOME` 目录。

如果旧路径存在或者 `$XDG_CONFIG_HOME` 环境变量未设置，则优先使用旧路径。

如果旧路径 `~/.calcurse` 存在，优先使用。

可以在 `*proj*/.clangd` 中指定项目配置。 在合理的情况下，会组合配置。如果发生冲突，则用户配置具有最高优先级，其次是内部项目，最后是外部项目。

如果旧路径存在，优先使用。

请参阅 Option Files — Universal Ctags 0.3.0 Documentation。

27.1

旧路径优先于 XDG 路径。Emacs 不会自行创建 `$XDG_CONFIG_HOME/emacs/`。26.3 或更早版本的绕过方法：可以更改`HOME`，但它可能会产生意想不到的副作用。

如果旧路径 `~/.mozilla/firefox` 存在，优先使用。截至 Firefox 152.0.5 使用 XDG 路径存在一个 bug：

- C0 无法工作

0.20.0

`$XDG_CONFIG_HOME/FreeCAD` `$XDG_DATA_HOME/FreeCAD` `$XDG_CACHE_HOME/FreeCAD`

旧路径可通过 `FreeCAD --keep-deprecated-paths`使用

旧方法 `export STACK_ROOT="$XDG_DATA_HOME"/stack` 仍然有效且优先级更高。

完整支持：598

`$XDG_CONFIG_HOME/lesskey`

在 590 版本中**必须**设置环境变量 `$XDG_CONFIG_HOME` 和 `$XDG_DATA_HOME`。598 及以后版本不再需要。

如果旧路径 `~/.luarocks` 存在，优先使用。

默认使用旧目录，若 `$XDG_CONFIG_HOME/newsboat` 存在则使用 XDG 目录。旧目录下的文件/目录需要手动分离（参见 newsboat(1) § FILES）。

`~/.utoprc`

`$XDG_CONFIG_HOME/utop/utoprc`

如果旧路径存在，优先使用。

```
mkdir "$XDG_CONFIG_HOME"/scummvm/ "$XDG_DATA_HOME"/scummvm
mv ~/.scummvmrc "$XDG_CONFIG_HOME"/scummvm/scummvm.ini
mv ~/.scummvm "$XDG_DATA_HOME"/scummvm/saves
```

有关更多信息，请参阅 shellcheck(1) § RC FILES。

为了使 X11 Qt、GTK 程序或 Wayland Qt 程序使用 `$XDG_DATA_HOME/icons` 中的光标，需要配置 $XCURSOR_PATH 环境变量。

默认仅使用以上 XDG 目录/文件，其他需创建 $XDG_CONFIG_HOME/vim/ 目录后运行 $VIMRUNTIME/xdg.vim 启用。（见 :h xdg-base-dir） 以下文件/目录需要运行 $VIMRUNTIME/xdg.vim 启用：

$XDG_STATE_HOME/vim/viminfo

以下目录需要在 $VIMRUNTIME/xdg.vim 中取消注释部分行再运行启用：

$XDG_DATA_HOME/vim/undo/ $XDG_DATA_HOME/vim/（原 ~/.vim/pack/ 及 ~/.vim/ 目录下的 Netrw 数据） $XDG_DATA_HOME/vim/view/（原 ~/.vim/view/） $XDG_DATA_HOME/vim/spell/（原 ~/.vim/spell/） $XDG_STATE_HOME/vim/swap/ $XDG_STATE_HOME/vim/backup/

3.2

$XDG_CACHE_HOME/weechat $XDG_DATA_HOME/weechat $XDG_RUNTIME_DIR/weechat |

仅支持 Wayland 。

$XDG_CACHE_HOME/xmonad/ $XDG_CONFIG_HOME/xmonad/ $XDG_DATA_HOME/xmonad/

也可以使用更高优先级的 $XMONAD_CACHE_DIR、$XMONAD_CONFIG_DIR 和 $XMONAD_DATA_DIR 环境变量。

部分支持

应用程序	旧路径	采用规范自	讨论	注释
abookAUR	~/.abook			abook --config "$XDG_CONFIG_HOME"/abook/abookrc --datafile "$XDG_DATA_HOME"/abook/addressbook
ack包	~/.ackrc			export ACKRC="$XDG_CONFIG_HOME/ack/ackrc"
Ansible（英语：Ansible）	~/.ansible	2.14	 	 `` export ANSIBLE_HOME="${XDG_CONFIG_HOME}/ansible" export ANSIBLE_GALAXY_CACHE_DIR="${XDG_CACHE_HOME}/ansible/galaxy_cache" export ANSIBLE_LOCAL_TEMP="${XDG_CACHE_HOME}/ansible/tmp" export ANSIBLE_SSH_CONTROL_PATH_DIR="${XDG_CACHE_HOME}/ansible/cp" export ANSIBLE_ASYNC_DIR="${XDG_CACHE_HOME}/ansible_async" ``

```
export ASDF_CONFIG_FILE="${XDG_CONFIG_HOME}/asdf/asdfrc"
export ASDF_DATA_DIR="${XDG_DATA_HOME}/asdf"
export ASPELL_CONF="per-conf $XDG_CONFIG_HOME/aspell/aspell.conf; personal
$XDG_DATA_HOME/aspell/en.pws; repl $XDG_DATA_HOME/aspell/en.prepl"
export AWS_SHARED_CREDENTIALS_FILE="$XDG_CONFIG_HOME"/aws/credentials
export AWS_CONFIG_FILE="$XDG_CONFIG_HOME"/aws/config
HOME="$XDG_DATA_HOME" btcli
```

即使未设置 $BUP_DIR，也能正确使用 $XDG_CACHE_HOME 作为缓存目录。

```
export CALCHISTFILE="$XDG_CACHE_HOME"/calc_history
```

或者使用具有原生 XDG 支持的 fork：。

尽管 NSS 本身现在允许使用 XDG 规范，由于 Chromium 未正确设置 NSS，创建了 .pki。这导致其下游也无法正常工作（Qt WebEngine 尤其影响了许多情况，例如 KMail 等）

```
$XDG_CONFIG_HOME/clusterssh/config
extra_cluster_file=$HOME/.config/clusterssh/clusters
extra_tag_file=$HOME/.config/clusterssh/tags
```

即使使用以上方案，clusterssh 仍会创建 ~/.clusterssh/。

```
export CRAWL_DIR="$XDG_DATA_HOME"/crawl/
```

```
$XDG_CONFIG_HOME/clusterssh/config
extra_cluster_file=$HOME/.config/clusterssh/clusters
extra_tag_file=$HOME/.config/clusterssh/tags
```

尽管如此，clusterssh 仍然会创建 ~/.clusterssh/。

文档未记录，但常用： export DISCORD_USER_DATA_DIR="${XDG_DATA_HOME}"

来源：<discord_system_package_root>/resources/app.asar

```
export MIX_XDG="true"
```

```
mkdir "$XDG_CONFIG_HOME"/erlang mv ~/.erlang.cookie "$XDG_CONFIG_HOME"/erlang
```

```
__Game_Install_directory/config-path.cfg
use-system-read-write-data-directories=true
__Game_Install_directory/config/config.ini
[path]
read-data=__PATH__executable__/../../data
write-data=.local/share/factorio
```

```
export GDBHISTFILE="$XDG_DATA_HOME"/gdb/history
```

环境变量 GHCUP_USE_XDG_DIRS 可以设置为任何非空值。请参阅 。

请注意，目前使用 systemd 用户单元和基于 socket 的激活无法开箱即用，因为 socket 目录会根据 $GNUPGHOME 的哈希值发生变化。您可以使用 gpgconf --list-dirs socketdir 获取新的 socket 目录，并必须修改 systemd 用户单元以相应地监听正确的 socket。还必须使用以下 gpg-agent.service 插入文件（或以其他方式将 GNUPGHOME 环境变量传递给在 systemd 中运行的代理），否则您可能会遇到“丢失”私钥的问题：

```
[Service]
Environment="GNUPGHOME=%h/.local/share/gnupg"
```

如果您使用 GPG 作为您的 SSH 代理，请将 `SSH_AUTH_SOCK` 设置为 `gpgconf --list-dirs agent-ssh-socket` 的输出，而不是某些硬编码值。

`export GR_PREFS_PATH="$XDG_CONFIG_HOME"/gnuradio` GNU Radio Companion: `export GRC_PREFS_PATH="$XDG_CONFIG_HOME"/gnuradio/grc.conf`

如果未设置 `GOMODCACHE`，则默认为 `$GOPATH/pkg/mod`（参见 ）。 支持 `GOCACHE`，默认为 `$XDG_CACHE_HOME/go-build`（参见 ）。

```
~/.config/gopass/config.yml
root:
path: gpgcli-gitcli-fs+file:///home/<userid>/.config/password-store
```

`$PASSWORD_STORE_DIR` 仅在初始化时支持。

如果使用了 **Lxappearance**，`~/.gtkrc-2.0` 可能会不断被创建，因为点击“应用”时的定制设置会写入到这个文件。该路径在 Lxappearance 中是硬编码的，但作为一个输出文件，这些设置可以被反复移动到该位置。

要阻止 KDE Plasma 创建此文件，请禁用“GNOME/GTK 设置同步”后台服务。

这个变量的值必须包含子字符串 `__HVER__`，它将在运行时被替换成当前的 `*MAJOR*.*MINOR*` 版本号。

```
~/.profile
$ export IRBRC="$XDG_CONFIG_HOME"/irb/irbrc
"$XDG_CONFIG_HOME"/irb/irbrc
IRB.conf[:SAVE_HISTORY] ||= 1000
IRB.conf[:HISTORY_FILE] ||= File.join(ENV["XDG_DATA_HOME"], "irb", "history")
```

`export JUPYTER_CONFIG_DIR="$XDG_CONFIG_HOME"/jupyter`

v5.0.0 <= python-jupyter-core包 < v6.0.0：

`export JUPYTER_PLATFORM_DIRS="1"` (see)

python-jupyter-core包 >= v6.0.0：完整支持（通过python-platformdirs包），默认开启

KDE 4 使用 `$KDEHOME`。不建议在更新的版本上使用此变量。

要更改 Leiningen 使用的 m2 仓库位置，请参阅：Leiningen#m2 仓库位置（英语：Leiningen#m2 repo location）

确保提前设置 `XDG_CACHE_HOME`，使其指向运行 Xorg 的用户有写权限的目录。

不要使用 `XDG_RUNTIME_DIR`，因为它是在登录之后才可用的。否则启动 Xorg 的显示管理器（如 GDM）将会反复失败。

```
/opt/maptool/lib/app/MapTool.cfg
```

```
[JavaOptions]
-DMAPTOOL_DATADIR=.local/share/maptool-rptools
```

但是，没有办法改变这个配置文件的位置。

曾为 `$MATHEMATICA_USERBASE`，参见 从 Mathematica 升级到 Wolfram。

无论出于什么原因，这会在 `MINIKUBE_HOME` 中创建一个额外的 `.minikube` 目录。

```
/etc/mtpaint/mtpaintrc
userINI = ~/.config/mtpaint
export MYPY_CACHE_DIR="$XDG_CACHE_HOME"/mypy
1=export MYSQL_HISTFILE="$XDG_DATA_HOME"/mysql_history
```

`~/.my.cnf` 仅支持 mysql-server，不支持 mysql-client

`~/.mylogin.cnf` 不支持。

`export TERMINFO="$XDG_DATA_HOME"/terminfo`，`export TERMINFO_DIRS="$XDG_DATA_HOME"/terminfo:/usr/share/terminfo`

```
npmrc
prefix=${XDG_DATA_HOME}/npm
cache=${XDG_CACHE_HOME}/npm
init-module=${XDG_CONFIG_HOME}/npm/config/npm-init.js
logs-dir=${XDG_STATE_HOME}/npm/logs
```

如果 Node.js 是通过 nvmAUR 安装的，则 `prefix` 是不必要的（且不受支持）。

```
$XDG_CONFIG_HOME/octave/octaverc
source /usr/share/octave/site/m/startup/octaverc;
pkg prefix ~/.local/share/octave/packages ~/.local/share/octave/packages;
pkg local_list /home/<your username>/.local/share/octave/octave_packages;
```

`local_list` 选项必须给出绝对路径。

配置和状态数据都存储在 `$OPAMROOT` 中，因此该解决方案不完全兼容。

目前它硬编码 `~/.local/share`。

除非手动重新创建 `.packettracer` 文件，否则无论此设置如何，日志文件仍可能被写入 `~/pt/logs`。

以及 和

`export PHIVE_HOME="$XDG_DATA_HOME/phive"`

必须创建这两个目录：`mkdir "$XDG_CONFIG_HOME/pg" && mkdir "$XDG_STATE_HOME"`

`projectzomboid.sh` 脚本也通过 `GAMEDIR="${HOME}/Zomboid"` 定义了自己的游戏数据目录。如果你使用 Steam，该脚本会在游戏更新时被重置，因此不能直接手动修改。不过，你可以创建一个自定义脚本，在主脚本被 Steam 还原后自动对其进行修补：

```
start.sh
#!/usr/bin/env bash
```

```
SCRIPT="$(dirname "$0")/projectzomboid.sh"
sed 's|GAMEDIR|"${HOME}/Zomboid"|GAMEDIR="${XDG_DATA_HOME}/Zomboid"|' -i "$SCRIPT"
exec "$SCRIPT" -cachedir="${XDG_DATA_HOME}"/Zomboid/ "$@"
```

将其命名为 `start.sh`，并放在 `projectzomboid.sh` 所在的目录中。然后使用 `./start.sh # %command%"`，让 Steam 执行 `./start.sh` 而不是主脚本。请注意，这也会使游戏在不使用 Steam Linux Runtime 的情况下启动。

PYTHON_HISTORY：`export PYTHON_HISTORY=$XDG_STATE_HOME/python_history`

PYTHONPYCACHEPREFIX：`export PYTHONPYCACHEPREFIX=$XDG_CACHE_HOME/python` PYTHONUSERBASE：`export PYTHONUSERBASE=$XDG_DATA_HOME/python`

```
export RENPY_PATH_TO_SAVES="$XDG_DATA_HOME/renpy"
export RENPY_MULTIPERSISTENT="$XDG_DATA_HOME/renpy_shared"
export BUNDLE_USER_CACHE=$XDG_CACHE_HOME/bundle
export BUNDLE_USER_CONFIG=$XDG_CONFIG_HOME/bundle/config
export BUNDLE_USER_PLUGIN=$XDG_DATA_HOME/bundle
```

有关更多信息，请参阅 Bundler: bundle config。

`~/.ivy2`

配置和数据文件均存储在 `$SDKMAN_DIR` 中，因此该方案并不完全符合 XDG 基础目录规范。

`mv ~/.ssr "$XDG_CONFIG_HOME"/simplescreenrecorder`

`export SPACEMACSDIR="$XDG_CONFIG_HOME"/spacemacs; mv ~/.spacemacs "$SPACEMACSDIR"/init.el`

其他文件需要像 Emacs 一样进行配置。

```
export TEXMFHOME=$XDG_DATA_HOME/texmf
export TEXMFVAR=$XDG_CACHE_HOME/texlive/texmf-var
export TEXMFCONFIG=$XDG_CONFIG_HOME/texlive/texmf-config}}
```

`tiptop -W "$XDG_CONFIG_HOME"/tiptop`

设置此项会使编辑器在 `$VSCODE_PORTABLE/user-data` 中查找 `.config/Code - OSS` 的内容。

您还可以使用 `--extensions-dir` 标志运行 Visual Studio，例如 `code --extensions-dir "$XDG_DATA_HOME/vscode"`。这已记录在文档中并有其他用例，不大可能意外中断。

也可以修改 `product.json` 文件中 `dataFolderName` 的值，将其设为 `.local/share/codium` 或你指定的路径。不过，每次更新该软件包后都需要重

新进行此操作；为此，你可以安装 vscodium-xdg-dir-patchAUR，让系统自动完成这一过程。

该目录需要手动创建

```
mkdir "$XDG_CONFIG_HOME/wakatime"
```

```
mkdir -p "$XDG_DATA_HOME"/wineprefixes, export WINEPREFIX="$XDG_DATA_HOME"/wineprefixes/default
```

~/.x3270connect 目前不支持使用 XDG。

根据您配置 $XDG_CACHE_HOME 的位置，您可能需要自行扩展路径。

```
# xsession start script
USERXSESSION="$XDG_CACHE_HOME/X11/xsession"
USERXSESSIONRC="$XDG_CACHE_HOME/X11/xsessionrc"
ALTUSERXSESSION="$XDG_CACHE_HOME/X11/Xsession"
ERRFILE="$XDG_CACHE_HOME/X11/xsession-errors"
```

与此表中的大多数其他示例不同，实际的 X11 启动脚本在安装之间会有很大差异。

请注意，LightDM 不允许更改此变量。如果强行更改，将无法登录。请改用 startx 或配置 LightDM。根据 的信息，SLiM 将 ~/.Xauthority 路径硬编码在了程序中。SDDM 的 Xauthority 路径可以通过其配置文件进行更改（如下所示）。遗憾的是，该路径是相对于用户主目录的。

```
/etc/sddm.conf.d/xauth-path.conf
[X11]
UserAuthFile=.Xauthority
```

在 Wayland 环境下，覆盖此设置可能会导致 Xorg 程序无法连接到 Xwayland 服务器。例如，kwin包 和 mutter包 均使用随机生成的文件名，因此无法将其设置为固定值。

```
"$XDG_CONFIG_HOME"/zsh/.zshrc
# 把补全文件和历史记录文件存放到 XDG 目录
[ -d "$XDG_STATE_HOME"/zsh ] || mkdir -p "$XDG_STATE_HOME"/zsh
HISTFILE="$XDG_STATE_HOME"/zsh/history
[ -d "$XDG_CACHE_HOME"/zsh ] || mkdir -p "$XDG_CACHE_HOME"/zsh
zstyle ':completion:*' cache-path "$XDG_CACHE_HOME"/zsh/zcompcache
compinit -d "$XDG_CACHE_HOME"/zsh/zcompdump-$ZSH_VERSION
```

最后，如果您使用 zsh 作为登录 shell，并选择依赖启动文件 ~/.zshenv 或 ~/.zprofile 或 ~/.zlogin 来设置重要的环境变量（例如 ZDOTDIR）进行引导，则无法绕过将设置 ZDOTDIR 的一个文件放在默认位置。对于上下文，一个例外是，如果您的更广泛的系统配置在此之前设置了 ZDOTDIR 环境变量。

硬编码

应用程序	旧路径	讨论	注释
adb & Androi	~/.android/		尽管看起来并非如此，adb 将**始终**生成

应用程序	旧路径	讨论	注释
d Studio			~/.android/adbkeys，尽管它也会尝试 ADB_VENDOR_KEYS 中的密钥。
aegisub包	~/.aegisub/		
alpine（英语：alpine）	~/.pinerc ~/.addressbook ~/.pine-debug[1-4] ~/.newsrc ~/.mailcap ~/.mime.types ~/.pine-interrupted-mail		alias alpine="alpine -p $XDG_CONFIG_HOME/alpine/pinerc" 在上面的配置文件中，可以使用 newsrc-path= 和 address-book= 等选项定制一些位置。

在 arduino-cli.yaml 中指定 Arduino CLI 使用的新目录，如文档此处所述。 alias arduino-cli='arduino-cli --config-file $XDG_CONFIG_HOME/arduino15/arduino-cli.yaml'

export HISTFILE="$XDG_STATE_HOME"/bash/history

bashrc 可从 /etc/bash.bashrc 中的不同位置获取。 对于交互式 shell，请指定 --init-file <file> 作为 ~/.bashrc 的替代。

```
export JULIA_DEPOT_PATH="$XDG_DATA_HOME/julia:$JULIA_DEPOT_PATH"
export JULIAUP_DEPOT_PATH="$XDG_DATA_HOME/julia"
export KUBECONFIG="$XDG_CONFIG_HOME/kube"
export KUBECACHEDIR="$XDG_CACHE_HOME/kube"
```

由维护脚本 eval.php 和 sql.php 生成。

export OLLAMA_MODELS=$XDG_DATA_HOME/ollama/models

```
export R_PROFILE_USER="${XDG_CONFIG_HOME}/R/.Rprofile"
export R_ENVIRON_USER="${XDG_CONFIG_HOME}/R/.Renviron"
export R_HISTFILE="${XDG_STATE_HOME}/R/history"
export R_MAKEVARS_USER="${XDG_CONFIG_HOME}/R/Makevars"
export R_CHECK_ENVIRON="${XDG_CONFIG_HOME}/R/check.Renviron"
export R_BUILD_ENVIRON="${XDG_CONFIG_HOME}/R/build.Renviron"
export R_REPOSITORIES="${XDG_CONFIG_HOME}/R/repositories"
/etc/sbclrc
(require :asdf)
(setf sb-ext:*userinit-pathname-function*
      (lambda () (uiop:xdg-config-home #P"sbcl/sbclrc")))
```

请注意，这需要 root 权限，并将更改所有用户的 ~/.sbclrc 位置。可以通过检查 lambda 表单中是否存在 ~/.sbclrc 来缓解此问题。

export VIMPERATOR_RUNTIME="$XDG_CONFIG_HOME"/vimperator

export SSB_HOME="$XDG_DATA_HOME"/zoom

工具

工具 xdg-ninjaAUR 可检测 $HOME 中可移动到 XDG 基本目录的不需要的文件/目录。请参阅 README 了解示例。

工具 boxxy包 可用于包装不遵守 XDG 基本目录的应用程序并重定向任何不需要的文件。

工具 ephemeral 可用于将通常位于 `XDG_CONFIG_HOME` 中的 chromium/electron 缓存链接到 `XDG_CACHE_HOME` 中的位置。

库

C : libXDGdirs : libxdg-basedir : C99：Cloudef的简单实现

C#

: Xdg.Directories

C++ : xdg-utils-cxx : xdgpp

Go : adrg/xdg : go-appdir（弃用，已归档） : configdir（弃用，已停止维护） : kyoh86/xdg（弃用，已归档）

Haskell : 自 1.2.3.0 ab9d0810ce 起正式位于 directory。 : xdg-basedir

JVM：Java、Kotlin、Clojure、Scala⋯⋯ : directories-jvm

Perl : File-BaseDir（0.6 版旧标准） : File-XDG

Python : pyxdg : appdirs（弃用，已停止维护） : platformdirs

Ruby : bkuhlmann/xdg : rubyworks/xdg（弃用，已停止维护）

Rust : directories-rs : rust-xdg

Swift : swift-xdg

Vala : 通过 GLib.Environment 提供内置支持。 : 参见 `get_user_cache_dir`、`get_user_data_dir`、`get_user_config_dir` 等。

提示与技巧

隐藏不需要的目录

在某些桌面环境（例如 KDE）下可以隐藏无法改变位置的目录：

```
$ echo 路径 >> ~/.hidden
```

路径 是文件/目录的路径，相对于 `.hidden` 的父目录（此处即 `~/`）。

参见

- GNOME Goal: XDG Base Directory Specification Usage
- Rob Pike: "Dotfiles" being hidden is a UNIXv2 mistake

- systemd-path(1)
- file-hierarchy(7)
- Grawity 关于点文件的注释
- Grawity 关于环境变量的注释
- ploum.net: Modify Your Application to use XDG Folders
- PCGamingWiki 试图记录 Linux PC 游戏是否遵循 XDG 基本目录规范

非官方用户仓库

官方源里没有的包从哪装、装完怎么升级？本章讲中文社区仓库、它的历史存档，和针对指令集优化的 ALHP。必读《中文社区仓库》《ALHP》和《非官方用户仓库》：ALHP 讲配置、装密钥环和镜像列表、更新数据库并升级；《非官方用户仓库》讲加入你自己的仓库和已签名、未签名仓库的处理。历史存档只在翻旧包时看，可跳。

误区在签名：未签名仓库的信任没设对，装包升级就会失败；镜像别手写地址，用 arch4edu mirrorlist 或 archlinuxcn-mirrorlist-git 配。

11.1　Arch Linux 中文社区仓库

难度 入门 | **篇幅** 2.6 KB | **所属** 非官方用户仓库

Arch Linux 中文社区仓库（也可简称为 archlinuxcn 或 archcn 仓库）是由 Arch Linux 中文社区驱动的非官方软件仓库，包含许多官方仓库未提供的额外的软件包，以及已有软件的 git 版本等变种。一部分软件包的打包脚本来源于 AUR，但也有许多包与 AUR 不一样。

查看打包脚本及可用包列表、报告问题、申请打包新软件包，请访问在 GitHub 上的项目。

仓库主地址：`[https://repo.archlinuxcn.org/](https://repo.archlinuxcn.org/)`

（仓库服务器位于欧洲，中国大陆、香港、美国、欧洲等地均有镜像。）

使用方法：在 `/etc/pacman.conf` 文件末尾添加以下两行（或者从这里选择一个镜像）：

```
/etc/pacman.conf
[archlinuxcn]
Server = https://repo.archlinuxcn.org/$arch
```

之后安装 archlinuxcn-keyringCNRepo 包以导入 GPG 密钥。安装 archlinuxcn-mirrorlist-gitCNRepo 包可以获得一份镜像源列表，以便在 `pacman.conf` 中直接引入。推荐同时更新系统：

1. 更新数据库并安装 archlinuxcn-keyringCNRepo：

```
# pacman -Sy archlinuxcn-keyring
```

1. 更新系统并安装镜像仓库列表。分为两步是为了让安装的 keyring 生效：

```
# pacman -Su archlinuxcn-mirrorlist-git
```

1. 在 `/etc/pacman.conf` 文件`[archlinuxcn]`处添加以下内容，以启用镜像列表

```
    /etc/pacman.conf
    Include = /etc/pacman.d/archlinuxcn-mirrorlist
```

本仓库同时提供 debuginfod 服务（支持部分有调试符号的包；不可用镜像站点的地址），设置以下环境变量即可（此环境变量会被 archlinuxcn-mirrorlist-gitCNRepo 自动设置，但有些情况可能并不生效）：

```
DEBUGINFOD_URLS="https://debuginfod.archlinux.org https://repo.archlinuxcn.org"
```

11.2 Arch Linux 中文社区仓库历史存档

难度 入门 | **篇幅** 4.2 KB | **所属** 非官方用户仓库

本节将讲到

- 位置
- 目录
- 使用方法
- 历史

Arch Linux 中文社区仓库历史存档（*\[archlinuxcn\] 存档* 或*archcn-archive*），保存了 Arch Linux 中文社区仓库（即 \[archlinuxcn\] 仓库）的历史版本。

用途

- 将某个包降级到某个早期版本（最新版本不能用，我需要之前的版本）
- 将所有包恢复到某个指定的历史时刻（所有包都不能用，我要恢复到两个月之前的状态），此时需要配合 Arch 官方仓库的历史存档 Arch Linux Archive

目前该存档仓库由 User:Oldherl 维护，每天 UTC 0点钟抓取一次。由于服务器空间有限，预计未来仅保存近几个月（待定）的包，以及之前每个月1日的包。

位置

Arch Linux 中文社区仓库历史存档位于 https://archive.repo.archlinuxcn.org/ ，另有下列几个备选域名，对应不同的线路。如在欧洲、北美等网络互联较好的地区使用，应选择直连线路。在其他地区如中国境内使用，可自行测试不同线路并选择。下列地址中的 `yyyy/mm/dd` 应替换为年月日。

```
[archlinuxcn]
# 直连线路（加拿大）
Server = https://archive.repo.archlinuxcn.org/repos/yyyy/mm/dd/$arch
# Cloudflare CDN 转发
Server = https://archcn-archive-cf.oldherl.one/repos/yyyy/mm/dd/$arch
# HE 隧道，仅 IPv6
Server = https://archcn-archive-he.oldherl.one/repos/yyyy/mm/dd/$arch
```

```
# Route64 隧道，仅 IPv6
Server = https://archcn-archive-r6.oldherl.one/repos/yyyy/mm/dd/$arch
```

存档库服务器由这里的源代码搭建。

目录

存档库分为下列两个主目录：

```
├── packages
└── repos
```

/repos

repos 这个目录包含 \[archlinuxcn\] 仓库镜像的每日快照，按下例结构组织：

```
— repos
  └── 2026
      └── 04
          ├── 07
          │   ├── aarch64
          │   ├── any
          │   └── x86_64
          ├── 08
          │   ├── aarch64
          │   ├── any
          │   └── x86_64
          └── 09
              ├── aarch64
              ├── any
              └── x86_64
```

/packages

packages 这个目录包含每个包的所有版本及其相应的数字签名。每个包一个目录，按首字母排序。

```
├── packages
│   ├── a
│   │   ├── aria2-git
│   │   │   ├── aria2-git-1.37.0.r38.gdc89cd3d-1-aarch64.pkg.tar.zst
│   │   │   ├── aria2-git-1.37.0.r38.gdc89cd3d-1-aarch64.pkg.tar.zst.sig
│   │   │   ├── aria2-git-1.37.0.r38.gdc89cd3da-1-x86_64.pkg.tar.zst
│   │   │   └── aria2-git-1.37.0.r38.gdc89cd3da-1-x86_64.pkg.tar.zst.sig
│   │   ├── autojump
│   │   └── ...
│   ├── b
│   ├── ...
│   └── z
```

你可以使用“魔法目录”all 按包名访问所有包。这是一个没有子目录的结构。注意：与 Arch Linux Archive 不同（它使用 .all 目录）

```
├── packages
│   ├── all
│   │   ├── aria2-git-1.37.0.r38.gdc89cd3d-1-aarch64.pkg.tar.zst
│   │   └── ...
```

使用方法

请参阅 Arch Linux Archive 并适当修改其中的URL。注意：\[archlinuxcn\] 应与 Arch 官方仓库一起降级或升级。

历史

- 2026-04-09 开始服务，更早的包已经遗失无法收录。
- 2026-05-22 主域名由 https://archcn-archive.oldherl.one 更改为 https://archive.repo.archlinuxcn.org

11.3 ALHP

难度 入门 | **篇幅** 3.1 KB | **所属** 非官方用户仓库

本节将讲到

- 配置
- 安装密钥环和镜像列表
- 更新软件包数据库并升级
- 不想用ALHP了？移除ALHP仓库

第三方的 ALHP 仓库使用优化标志以及先进指令集（如SSE3、AVX512）重新编译了Arch Linux中的core、extra和multilib仓库中的部分软件包，使得设备获得性能提升。

警告：使用此仓库可能会出现部分包更新不及时、版本不一致从而导致问题的情况。

配置

警告：以下步骤为必需步骤。如果不执行此步骤，可能会导致您的系统无法启动，并且您将需要降级任何您可能已经升级的软件包。

执行以下命令以查询CPU支持的特性级别

```
/lib/ld-linux-x86-64.so.2 --help
```

如果输出为以下内容

```
Subdirectories of glibc-hwcaps directories, in priority order:
  x86-64-v4 (supported, searched)
  x86-64-v3 (supported, searched)
  x86-64-v2 (supported, searched)
```

那么请将 `pacman.conf` 修改为以下样式

```
# 省略以上内容

[core-x86-64-v4]
Include = /etc/pacman.d/alhp-mirrorlist

[extra-x86-64-v4]
Include = /etc/pacman.d/alhp-mirrorlist

[core-x86-64-v3]
Include = /etc/pacman.d/alhp-mirrorlist

[extra-x86-64-v3]
Include = /etc/pacman.d/alhp-mirrorlist

[core-x86-64-v2]
Include = /etc/pacman.d/alhp-mirrorlist
```

```
[extra-x86-64-v2]
Include = /etc/pacman.d/alhp-mirrorlist

[core]
Include = /etc/pacman.d/mirrorlist

[extra]
Include = /etc/pacman.d/mirrorlist

# 如果需要 [multilib] 支持

[multilib-x86-64-v4]
Include = /etc/pacman.d/alhp-mirrorlist

[multilib-x86-64-v3]
Include = /etc/pacman.d/alhp-mirrorlist

[multilib-x86-64-v2]
Include = /etc/pacman.d/alhp-mirrorlist

[multilib]
Include = /etc/pacman.d/mirrorlist

# 省略以下内容
```

如果输出为以下内容

```
Subdirectories of glibc-hwcaps directories, in priority order:
  x86-64-v4
  x86-64-v3 (supported, searched)
  x86-64-v2 (supported, searched)
```

那么请将pacman.conf修改为以下样式

```
# 省略以上内容

[core-x86-64-v3]
Include = /etc/pacman.d/alhp-mirrorlist

[extra-x86-64-v3]
Include = /etc/pacman.d/alhp-mirrorlist

[core-x86-64-v2]
Include = /etc/pacman.d/alhp-mirrorlist

[extra-x86-64-v2]
Include = /etc/pacman.d/alhp-mirrorlist

[core]
Include = /etc/pacman.d/mirrorlist

[extra]
Include = /etc/pacman.d/mirrorlist

# 如果需要 [multilib] 支持

[multilib-x86-64-v3]
Include = /etc/pacman.d/alhp-mirrorlist

[multilib-x86-64-v2]
Include = /etc/pacman.d/alhp-mirrorlist

[multilib]
Include = /etc/pacman.d/mirrorlist

省略以下内容
```

以此类推。

安装密钥环和镜像列表

从 AUR 安装 alhp-keyring 和 alhp-mirrorlist。

使用 `yay` 安装的示例：

```
yay -S alhp-keyring alhp-mirrorlist
```

`alhp-keyring` 提供 ALHP 当前使用的签名密钥，`alhp-mirrorlist` 提供一组镜像列表供选择。

更新软件包数据库并升级

```
sudo pacman -Syu
```

不想用ALHP了？移除ALHP仓库

要禁用 ALHP，删除 `/etc/pacman.conf` 中所有 *x86-64-vX* 条目，并删除 `alhp-keyring` 和 `alhp-mirrorlist`。

之后，使用以下命令刷新 pacman 的数据库并降级所有软件包：

```
sudo pacman -Syuu
```

11.4　非官方用户仓库

难度 高级 | **篇幅** 37.3 KB | **所属** 非官方用户仓库

本节将讲到

- 加入你自己的仓库
- 已签名
- or other mirrors in https://github.com/arch4edu/mirrorlist/blob/master/mirrorlist.arch4edu
- or install archlinuxcn-mirrorlist-git and use the mirrorlist
- 未签名

这篇文章列出了由社区创建的自由共享二进制软件包的软件仓库，其中很多的包都是由AUR中可以找到的PKGBUILD文件预编译打包而成。

警告：无论 Arch Linux 的开发者还是软件包维护者都不会对这些软件仓库做任何的测试与验证。需要每一个用户自己决定是否信任这些软件仓库的维护者，并且对软件源维护者做出的任何决定导致的后果负责。

想要使用这些软件仓库，你需要把他们添加到 `/etc/pacman.conf`，详情请看 pacman#软件仓库。如果一个软件仓库进行了签名，你必须要在本地签署这些key，详见Pacman/软件包签名#导入非官方密钥。

如果你想自己建立一个软件仓库，请看 Pacman/提示和技巧#自建本地仓库。

加入你自己的仓库

注意：本文为Unofficial user repositories的翻译，添加自己的仓库，建议同时更新英文原始页面。

如果你有自己的仓库，请添加到本页面中，这样其他用户就会知道哪里去找你的软件。 在添加新的仓库时请保持队形，并遵守下列规则：

- 请按罗马字首排序列表。
- 包含维护者的一些信息：包括至少一个名字（昵称也行）以及一些具有联系方式的信息（网址，邮箱，用户ArchWiki页面或者论坛帖子等等）。
- 如果仓库是签名的，请添加一个key-id，最好把它编辑成一个链到密匙服务器的连接，直接连到密钥文件上。
- 包含一些简短的描述（软件包的分类什么的）
- 如果你有一个介绍性的网页（无论是ArchWiki或者其他网站），请添加一个连接。
- 如果可能，不要在配置文件中添加注释，而是写在前面的描述中。
- 一些仓库也有除 x86_64 架构以外的软件包。 变量 `$arch` 将会被 pacman 自动设置。

已签名

ada

- **Maintainer:** Rod Kay <rodakay5 at gmail dot com>
- **Description:** Arch Ada Repository ~ all packages relating to the Ada and SPARK programming languages.
- **Key-ID:** 0xED55AF75B0330A2A3BAA9986B6120CD888A0DFD2

```
[ada]
Server = http://www.orthanc.site:8080/assets/arch_ada_repo
```

alerque

- **维护者:** Caleb Maclennan
- **描述:** 字体开发，排版和发布相关工具，例如 SILE，CaSILE，Fontship，字体和相关依赖项，包括许多字体，Lua Rocks 和 Python 模块。 以及 Asterisk，很多我（共同）维护的 AUR 包，以及很多我自己使用的 AUR 包。
- **Git 仓库:** https://github.com/alerque/aur
- **问题追踪:** https://github.com/alerque/aur/issues
- **Key-ID:** 63CC496475267693

```
[alerque]
Server = https://arch.alerque.com/$arch
```

ALHP

- **维护者:** Giovanni Harting

- **描述:** 使用 LTO、`-march=x86-64-vN -O3` 编译的官方仓库软件包
- **问题追踪:** https://somegit.dev/ALHP/ALHP.GO/issues
- **Keyring:** alhp-keyringAUR
- **镜像列表:** alhp-mirrorlistAUR
- **上游页面:** https://somegit.dev/ALHP/ALHP.GO
- **Debuginfod:** https://debuginfod.alhp.dev

```
[core-x86-64-v3]
Include = /etc/pacman.d/alhp-mirrorlist

[extra-x86-64-v3]
Include = /etc/pacman.d/alhp-mirrorlist
```

andontie-aur

- **维护者:** Holly M.
- **描述:** 包含最流行的 AUR 软件包以及我一直使用的软件包的仓库。在上游网站请求新软件包。
- **Key-ID:** 72BF227DD76AE5BF
- **上游页面:** https://aur.andontie.net

```
[andontie-aur]
Server = https://aur.andontie.net/$arch
```

arcanisrepo

- **维护者:** arcanis
- **描述:** 包含一些 AUR 软件包（包括来自 VCS 的软件包）的仓库，以开发和科学工具为主。每日更新。
- **Key-ID:** 0xBD2AC8C5E989490C
- **上游页面:** https://repo.arcanis.me/arcanisrepo/x86_64/index.html

```
[arcanisrepo]
Server = https://repo.arcanis.me/$repo/$arch
```

arch4edu

- **维护者:** Jingbei Li (petronny)和其他人
- **描述:** arch4edu 是 Arch Linux 和 Arch Linux ARM 的社区仓库，致力于提供大学生使用的大多数软件的最新版本。
- **Git 仓库:** https://github.com/arch4edu/arch4edu
- **问题追踪:** https://github.com/arch4edu/arch4edu/issues for packaging issues, out-of-date notifications, package requests, and related questions
- **镜像:** https://github.com/arch4edu/mirrorlist/blob/master/mirrorlist.arch4edu
- **Key-ID:** 7931B6D628C8D3BA

```
[arch4edu]
Server = https://mirrors.tuna.tsinghua.edu.cn/arch4edu/$arch
## or other mirrors in
https://github.com/arch4edu/mirrorlist/blob/master/mirrorlist.arch4edu
```

Arch Linux 中文社区仓库

- **维护者:** phoenixlzx, Felix Yan (felixonmars, dev), lilydjwg, farseerfc, 和其他人
- **描述:** 由 Arch Linux 中文社区提供的软件包，均已签名，又称 \[archlinuxcn\] 仓库。请注意，除了 x86_64 架构之外的软件包可能缺乏维护，如果发现问题，请创建 issue。
- **Git 仓库:** https://github.com/archlinuxcn/repo
- **问题追踪:** https://github.com/archlinuxcn/repo/issues
- **镜像:** https://github.com/archlinuxcn/mirrorlist-repo 。也可以从本仓库安装 archlinuxcn-mirrorlist-gitCNRepo。
- **Key-ID:** 添加 archlinuxcn 仓库之后，必须先安装 archlinuxcn-keyringCNRepo 钥匙环，之后才可以安装任何其它包，否则会出现 GPG 错误。archlinuxcn-keyring 包本身由 Arch Linux 开发者签名。
- **Debuginfod:** https://repo.archlinuxcn.org

```
[archlinuxcn]
Server = http://repo.archlinuxcn.org/$arch
## or install archlinuxcn-mirrorlist-git and use the mirrorlist
#Include = /etc/pacman.d/archlinuxcn-mirrorlist
```

archzfs

- **维护者:** Jan Houben (minextu)
- **描述:** Arch Linux 上 ZFS 的软件包。
- **上游页面:** https://github.com/archzfs/archzfs
- **Key-ID:** DDF7DB817396A49B2A2723F7403BD972F75D9D76

```
[archzfs]
# 源站 - 法国
Server = http://archzfs.com/$repo/x86_64
# 镜像 - 德国
Server = http://mirror.sum7.eu/archlinux/archzfs/$repo/x86_64
# 镜像 - 德国
Server = https://mirror.biocrafting.net/archlinux/archzfs/$repo/x86_64
# 镜像 - 印度
Server = https://mirror.in.themindsmaze.com/archzfs/$repo/x86_64
# 镜像 - 意大利
Server = https://mirror.emanuelebertolucci.com/$repo/$arch
# 镜像 - 美国
Server = https://zxcvfdsa.com/archzfs/$repo/$arch
```

artafinde

- **维护者:** Leonidas Spyropoulos
- **描述:** 包含 AUR 和自定义软件包的个人仓库
- **Key-ID:** 不需要，已经在 archlinux 密钥环里：

```
[artafinde]
Server = https://pkgbuild.com/~artafinde/repo
```

avr

- **维护者:** Vianney Bouchaud
- **描述:** 一些我自用的 AUR 包，主要和 kubernetes 有关，以及一些我自己的软件。
- **Git 仓库:** https://github.com/vbouchaud/aur
- **问题追踪:** https://github.com/vbouchaud/aur/issues
- **Key-ID:** 157B08346330029C
- **上游页面:** https://bouchaud.org/packages/avr

```
[avr]
Server = https://bouchaud.org/packages/$repo/$arch
```

bioarchlinux

- **维护者:** BioArchLinux 团队
- **描述:** Arch Linux 的生物学软件仓库
- **上游页面:** https://github.com/BioArchLinux/Packages
- **Key-ID:** B1F96021DB62254D
- **文献:** https://doi.org/10.1093/bioinformatics/btaf106

```
[bioarchlinux]
Server = https://repo.bioarchlinux.org/$arch
```

blackeagle-pre-community

- **维护者:** Ike Devolder
- **描述:** 我的包，在移动到 *extra* 仓库先行测试
- **Key-ID:** 不需要，维护者是软件包维护者

```
[blackeagle-pre-community]
Server = https://repo.herecura.eu/$repo/$arch
```

build.kilabit.info

- **维护者:** shulhan
- **包列表:** https://build.kilabit.info/karajo/app/
- **Key-ID:** 4A5360B500C9C4F0
- **网站:** https://build.kilabit.info

```
[build.kilabit.info]
Server = https://build.kilabit.info/aur
```

chaotic-aur

- **维护者:** dr460nf1r3, PedroHLC, LordKitsuna, Librewish, SolarAquarion, thotypous (前受信用户), 和 RustemB (in memoriam).
- **描述:** 自动构建的维护者们所使用的 AUR 软件包。每小时更新（有些是每天更新）。在全球拥有数个镜像。主要构建系统由巴西的圣卡洛斯联邦大学托管。仅 x86_64。

- **Key-ID:** FBA220DFC880C036，以及一些子密钥。为了便于使用，密钥环和镜像列表也可在项目主页获取。
- **注:** 见仓库的主页.

```
[chaotic-aur]
Server = https://geo-mirror.chaotic.cx/$repo/$arch
```

coderkun-aur

- **维护者:** coderkun
- **描述:** 带有随机软件的AUR软件包。支持软件包增量以及软件包和数据库签名。
- **上游页面:** https://www.suruatoel.xyz/arch
- **Key-ID:** 39E27199A6BEE374
- **Keyfile:** https://www.suruatoel.xyz/coderkun.key

```
[coderkun-aur]
Server = https://arch.suruatoel.xyz/$repo/$arch/
```

condorcore

- **维护者:** MrHacker
- **描述:** Wazuh-SIEM packages that I co-maintain and AUR. signed from the database.
- **上游页面:** https://aur.condorbs.net
- **Key-ID:** 3CA0B9DF1BE7CE09
- **Keyfile:**GPG

注意：该仓库有自己的密钥环： https://aur.centauricorex.net/x86_64/condorcore-keyring-20231117-1-any.pkg.tar.zst

```
[condorcore]
Server = https://aur.centauricorex.net/$arch
```

desolve

- **维护者:** desolve
- **描述:** 一些额外的东西
- **Git 仓库（其它 AUR 软件包）:** https://github.com/dvdesolve/pkgbuilds
- **Key-ID:** DD3BF75DCD96541AC723B7CD6A4CD3276CA8EBBD

```
[desolve]
Server = https://desolve.ru/archrepo/$arch
```

devkitpro

- **维护者:** wintermute
- **描述:** 提供 Nintendo Wii，Gamecube，DS，GBA，Gamepark gp32 和 Nintendo Switch 的 Homebrew 工具链
- **上游页面:** https://devkitpro.org/wiki/devkitPro_pacman
- **Key-ID:** F7FD5492264BB9D0

注意：仓库在 https://pkg.devkitpro.org/devkitpro-keyring.pkg.tar.xz 有自己的密钥环

```
[dkp-libs]
Server = https://downloads.devkitpro.org/packages
[dkp-linux]
Server = https://downloads.devkitpro.org/packages/linux
```

g14

- **维护者:** Luke Jones
- **描述:** A custom repository for ASUS Linux, it contains prebuilt binaries for all of the software they offer.
- **上游页面:** https://asus-linux.org/
- **包列表:** https://arch.asus-linux.org/
- **Key-ID:** 8F654886F17D497FEFE3DB448B15A6B0E9A3FA35

```
[g14]
# Germany, origin
Server = https://arch.asus-linux.org
# Republic of Korea
Server = https://naru.jhyub.dev/$repo
```

grawlinson

- **维护者:** George Rawlinson
- **描述:** 用户维护的AUR软件包以及一些实验软件包。
- **软件包列表:** https://mirror.little.kiwi
- **Key-ID:** 9D120F4AAF400B8313A87EF2369552B2069123EE
- **Keyfile:** https://mirror.little.kiwi/grawlinson.asc
- **源码:** https://git.little.kiwi/grawlinson/arch-pkgs

```
[grawlinson]
Server = https://mirror.little.kiwi
Server = https://pkgbuild.com/~grawlinson/repo/$repo
```

herecura

- **维护者:** Ike Devolder
- **描述:** 在 *community* 库没有的额外软件包
- **Key-ID:** 不需要，维护者是软件包维护者
- **Git 仓库:** https://gitlab.com/herecura/packages

```
[herecura]
Server = https://repo.herecura.eu/$repo/$arch
```

ivasilev

- **维护者:** Ianis G. Vasilev
- **描述:** 各种软件包，主要是我自己的软件和AUR构建。
- **上游页面:** https://ivasilev.net/pacman
- **Key-ID:** 17DAB671

```
[ivasilev]
Server = https://ivasilev.net/pacman/$arch
```

jk-aur

- **维护者:** JustKidding
- **描述:** Packages from the AUR I maintain and co-maintain.
- **上游页面:** https://github.com/jstkdng/aur
- **Key-ID:** 7627D0F8F60FBA35371A29E1AA6B2752759F9361

```
[home_justkidding_arch_Arch]
Server = https://download.opensuse.org/repositories/home:/justkidding:/arch/Arch/$arch
```

kawaii

- **维护者:** Leonid Pilyugin
- **描述:** Kawaii modified packages from AUR and myself.
- **上游页面:** https://github.com/LeonidPilyugin/kawaii-repo
- **Key-ID:** A308BDBE10D7C9C168AA2E055F2E4806FFE6B2CD

```
[kawaii]
Server = https://raw.githubusercontent.com/LeonidPilyugin/kawaii-repo/main/x86_64/
```

lahwaacz

- **维护者:** Jakub Klinkovský
- **描述:** Personal repository with AUR packages.
- **Key-ID:** Not needed, as key is in archlinux keyring.

```
[lahwaacz]
Server = https://pkgbuild.com/~lahwaacz/repo
```

linux-lts66

- **维护者:** bemxio
- **描述:** A repository for the linux-lts66AUR kernel packages, including linux-lts66-headersAUR and linux-lts66-docsAUR.
- **Git 仓库:** https://github.com/bemxio/linux-lts66-repo
- **问题追踪:** https://github.com/bemxio/linux-lts66-repo/issues
- **Key-ID:** D6BA2CBC66075B2B

```
[linux-lts66]
Server = https://linux-lts66.bemxio.xyz/
```

linux-nitrous

- **维护者:** Simao Gomes Viana (superboringdev or xdevs23)
- **描述:** linux-nitrous is a repository with prebuilt packages of the linux-nitrous kernel
- **Git 仓库:** https://gitlab.com/xdevs23/linux-nitrous
- **问题追踪:** https://github.com/xdevs23/linux-nitrous/issues for packaging issues and questions
- **Key-ID:** 3FFA4C23 (keyserver.ubuntu.com) | 3FFA4C23 (pgp.mit.edu) | 3FFA4C23 (pgp.net.nz)

```
[linux-nitrous]
```

```
Server = https://github.com/xdevs23/linux-nitrous/releases/latest/download
```

liquorix

- **维护者:** Steven Barrett (damentz)
- **描述:** Repository containing automated builds of linux-lqxAUR, linux-lqx-headersAUR, and linux-lqx-docsAUR
- **上游页面:** https://liquorix.net
- **Git 仓库:** https://github.com/damentz/liquorix-package, https://aur.archlinux.org/cgit/aur.git/log/?h=linux-lqx
- **问题追踪:** https://github.com/damentz/liquorix-package/issues
- **Key-ID:** 9AE4078033F8024D (keys.openpgp.org)

```
[liquorix]
Server = https://liquorix.net/archlinux/$repo/$arch
```

miffe

- **维护者:** miffe (AUR profile)
- **描述:** AUR packages maintained by miffe, e.g. linux-mainline
- **Key ID:** 313F5ABD

```
[miffe]
Server = https://arch.miffe.org/$arch/
```

mxmeinhold

- **维护者:** mxmeinhold
- **描述:** Packages I maintain in the AUR, currently just factorio-headless
- **Key ID:** B77D730E8D444707FA93320D72E05836F8252405
- **Keyfile:** https://gpg.mxmeinhold.com

```
[mxmeinhold]
Server = https://arch.mxmeinhold.com/$arch
```

orhun

- **维护者:** Orhun Parmaksiz
- **描述:** Personal repository with AUR and custom packages.
- **Key-ID:** 不需要，维护者是软件包维护者

```
[orhun]
Server = https://pkgbuild.com/~orhun/repo
```

oscloud

- **维护者:** bionade24
- **描述:** ~ROS melodic packages and necessary dependencies,~ various cmd-line tools, and other packages from the AUR
- **CI/CD 状态:** https://abs-cd.oscloud.info
- **Key-ID:** FF363C5F81664E2B

```
[oscloud]
Server = http://repo.oscloud.info/
```

ownstuff

- **维护者:** Martchus
- **描述:** A lot of packages from the AUR, e.g. a great number packages for mingw-w64 and Android cross compilation, fonts, Perl modules, tools like tageditorAUR, syncthingtrayAUR, subtitlecomposerAUR and qmplay2AUR
- **上游页面**: https://github.com/Martchus/PKGBUILDs (sources beside the AUR) and https://martchus.no-ip.biz/buildservice/#package-search-section (package browser/search)
- **Key-ID:** B9E36A7275FC61B464B67907E06FE8F53CDC6A4C

```
[ownstuff-testing]
Server = https://ftp.f3l.de/~martchus/$repo/os/$arch
Server = https://martchus.no-ip.biz/repo/arch/$repo/os/$arch

[ownstuff]
Server = https://ftp.f3l.de/~martchus/$repo/os/$arch
Server = https://martchus.no-ip.biz/repo/arch/$repo/os/$arch
```

注意：The testing repository is supposed to be used together with the official testing repositories.

pkgbuilder

- **维护者:** Chris Warrick
- **描述:** A repository for PKGBUILDer, a Python AUR helper.
- **上游页面:** https://github.com/Kwpolska/pkgbuilder
- **Key-ID:** 5EAAEA16

```
[pkgbuilder]
Server = https://pkgbuilder-repo.chriswarrick.com/
```

post-factum kernels

- **维护者**: Oleksandr Natalenko aka post-factum
- **上游页面**: https://pfkernel.natalenko.name
- **描述**: pf-kernel（英语：linux-pf） packages by its developer, post-factum
- **Key-ID:**: 95C357D2AF5DA89D
- **Keyfile**: https://download.opensuse.org/repositories/home:/post-factum:/kernels/Arch/x86_64/home_post-factum_kernels_Arch.key

```
[home_post-factum_kernels_Arch]
Server = https://download.opensuse.org/repositories/home:/post-factum:/kernels/Arch/$arch
```

pro-audio-legacy

- **维护者:** David Runge
- **描述:** Legacy tooling for pro-audio (e.g. jackAUR), mainly useful for old setups or CI
- **Key-ID:** Not needed, as maintainer is a developer/TU

```
[pro-audio-legacy]
Server = https://pkgbuild.com/~dvzrv/repos/pro-audio-legacy/$arch
```

proaudio

- **维护者:** OSAMC members (SpotlightKid, cbix, daniel.appelt et al.)
- **描述:** Pro Audio packages not (yet) in the official repos, built for x86_64 and aarch64
- **上游页面:** https://arch.osamc.de/
- **GitHub 仓库:** https://github.com/osam-cologne/archlinux-proaudio (PRs welcome)
- **Key-ID:** 762AE5DB2B38786364BD81C4B9141BCC62D38EE5
- **Keyfile:** https://arch.osamc.de/proaudio/osamc.gpg

```
[proaudio]
Server = https://arch.osamc.de/$repo/$arch
```

QOwnNotes

- **维护者:** Patrizio Bekerle (pbek), QOwnNotes author
- **描述:** QOwnNotes是一个开源记事本和待办事项列表管理器，具有降价支持和ownCloud integration.
- **Key-ID:** F2205FB121DF142B31450865A3BA514562A835DB
- **Keyfile:** https://download.opensuse.org/repositories/home:/pbek:/QOwnNotes/Arch_Extra/x86_64/home_pbek_QOwnNotes_Arch_Extra.key

```
[home_pbek_QOwnNotes_Arch_Extra]
Server = https://download.opensuse.org/repositories/home:/pbek:/QOwnNotes/Arch_Extra/$arch
```

quarry

- **维护者:** anatolik
- **描述:** Arch binary repository for Rubygems packages. See forum announcement for more information.
- **源码:** https://github.com/anatol/quarry
- **Key-ID:** Not needed, as maintainer is a developer

```
[quarry]
Server = https://pkgbuild.com/~anatolik/quarry/x86_64/
```

repo.mksscryertower.quest

- **维护者:** Klimenko Maxim Sergievich
- **描述:** Collection of AUR packages that I personally use: Crowdsec, libraries and etc.
- **Key-ID:** CF26478FD667CD54

```
[repo.mksscryertower.quest]

Server = https://repo.mksscryertower.quest/repo/x86_64/
```

rne

- **Maintainer:** Richard Neumann
- **Description:** Collection of AUR packages that I personally use.

- **Key-ID:** 4CA8D523BD386AF7

```
[rne]
Server = https://srv.richard-neumann.de/pub/repo/
```

seblu

- **维护者:** Sébastien Luttringer
- **描述:** 所有seblu有用的预制包，有些是自制的 (linux-seblu-meta, zfs-dkms, spotify, masterpdfeditor, etc).
- **Key-ID:** 8DBD63B82072D77A

```
[seblu]
Server = https://al1.seblu.net/$repo/$arch
Server = https://al2.seblu.net/$repo/$arch
```

seiichiro

- **维护者:** Stefan Brand (seiichiro0185)
- **描述:** AUR-packages I use frequently
- **Key-ID:** 805517CC

```
[seiichiro]
Server = https://www.seiichiro0185.org/repo/$arch
```

sergej-repo

- **维护者:** Sergej Pupykin
- **描述:** nextcloud, prosody, and some other stuff.
- **Key-ID:** 不需要，维护者是软件包维护者

```
[sergej-repo]
Server = http://repo.p5n.pp.ru/$repo/os/$arch
```

sublime-text

- **维护者:** Sublime Text developer
- **描述:** Sublime Text editor packages from developer's repository
- **上游页面:** https://www.sublimetext.com/docs/3/linux_repositories.html#pacman
- **Key-ID:** 8A8F901A

```
[sublime-text]
Server = https://download.sublimetext.com/arch/stable/x86_64
```

supermario

- **维护者:** Mario Finelli (supermario)
- **描述:** AUR packages that I use or maintain
- **Git Repository:** https://github.com/mfinelli/pkgs, https://github.com/mfinelli/aur-packages
- **Key-ID:** C3CD75B002978A8468CA7B1F6C3ADDDE36FDA306
- **Keyfile:** https://finelli.pub/36FDA306.asc

```
[supermario]
Server = https://pkgs.finelli.dev/arch/$arch
```

trinity

- **维护者:** Trinity Desktop Environment Developers
- **描述:** Trinity 桌面环境
- **Key-ID:** 8685AD8B

```
[trinity]
Server = https://mirror.ppa.trinitydesktop.org/trinity/archlinux/$arch
```

xyne-x86_64

- **维护者:** Xyne
- **描述:** A repository for Xyne's own projects.
- **上游页面:** https://xyne.dev/projects/
- **Key-ID:** 不需要，维护者是软件包维护者

```
[xyne-x86_64]
Server = https://xyne.dev/repos/xyne
```

未签名

注意：用户将需要在这些条目中添加以下内容: `SigLevel = PackageOptional`

警告：不要对未签名仓库使用 HTTP 协议。不用 HTTPS 的话，就没有任何正确性验证来保证你下载到的内容是服务器所提供的。通过中间人攻击，攻击者可能向你的系统上安装任意软件。HTTPS 避免了此类漏洞。

alucryd

- **维护者:** Maxime Gauduin
- **描述:** Various packages Maxime Gauduin maintains (or not) in the AUR, in particular the multilib repository is for various packages needed to run Steam without its runtime environment.

```
[core-alucryd]
Server = https://pkgbuild.com/~alucryd/$repo/$arch
[core-testing-alucryd]
Server = https://pkgbuild.com/~alucryd/$repo/$arch
[extra-alucryd]
Server = https://pkgbuild.com/~alucryd/$repo/$arch
[extra-testing-alucryd]
Server = https://pkgbuild.com/~alucryd/$repo/$arch
[multilib-alucryd]
Server = https://pkgbuild.com/~alucryd/$repo/$arch
[multilib-testing-alucryd]
Server = https://pkgbuild.com/~alucryd/$repo/$arch
```

arch-mact2

- **Maintainer:** Noa Himesaka
- **Description:** Kernel and utilities for use on Macs with T2 security chip.
- **Upstream page:** https://mirror.funami.tech/arch-mact2

```
[arch-mact2]
Server = https://mirror.funami.tech/arch-mact2/os/$arch
```

archlinuxgr

- **Maintainer:**

- **Description:** many interesting packages provided by the Hellenic (Greek) Arch Linux community

```
[archlinuxgr]

Server = https://archlinuxgr.tiven.org/archlinux/$arch
```

archlinux-phalcon

- **Maintainer:** Michal Sotolar <michal at sotolar dot com>
- **Description:** Phalcon Framework PHP 5.6 - 7.x extension builds
- **Upstream page:** https://archlinux-phalcon.gitlab.io

```
[archlinux-phalcon]
Server = https://archlinux-phalcon.gitlab.io/repository
```

archlinux-php

- **Maintainer:** Michal Sotolar <michal at sotolar dot com>
- **Description:** PHP 5.6 - 7.x old stable builds coexistable with mainline version
- **Upstream page:** https://archlinux-php.gitlab.io

```
[archlinux-php]
Server = https://archlinux-php.gitlab.io/repository
```

dx37essentials

- **维护者:** DragonX256
- **描述:** Personal repository. Contains packages from AUR, which I using every day.
- **Git repo:** https://gitlab.com/DX37/dx37essentials
- **Upstream page:** https://dx3756.ru/dx37essentials

```
[dx37essentials]
Server = https://dx3756.ru/$repo/$arch
Server = https://dx37.gitlab.io/$repo/$arch
```

ffy00 (python)

- **维护者:** Filipe Laíns
- **描述:** 提供几种不同的Python版本（例如3.5、3.6、3.7）。所有有效的Python版本均应在此处提供。
- **Git repo:** https://github.com/FFY00/arch-python-repo
- **上游页面:** https://github.com/FFY00/arch-python-repo

```
[python]
Server = https://ffy00.github.io/arch-python-repo/
```

heftig

- **维护者:** Jan Alexander Steffens
- **描述:** Includes firefox-nightly, freetype2-gitAUR and crawl-git
- **上游页面:** https://bbs.archlinux.org/viewtopic.php?id=117157

```
[heftig]
Server = https://pkgbuild.com/~heftig/repo/$arch
```

kernel

- **维护者:** blacksy3 <blacksky3@tuta.io>
- **描述:** Prebuilt packages for XanMod and Liquorix kernel with config tweak for performance
- **Git repo:** https://github.com/blacksky3/linux-xanmod.git
- **Git repo:** https://github.com/blacksky3/linux-lqx.git
- **上游页面:** https://repo.blacksky3.com/

```
[kernel]
Server = https://repo.blacksky3.com/$arch/$repo
```

kicad-nightly

- **Maintainer:** Rafael Silva <perigoso@riseup.net>
- **Description:** Prebuilt nightly packages for KiCad
- **Git repo:** https://gitlab.com/kicad/packaging/kicad-arch/kicad-arch-builder
- **Upstream page:** https://www.kicad.org/help/nightlies-and-rcs/

```
[kicad-nightly]
Server = https://kicad.gitlab.io/packaging/kicad-arch/kicad-arch-builder/
```

mesa-git

- **维护者:** Laurent Carlier
- **描述:** Mesa git builds for the *testing* repositories

```
[mesa-git]
Server = https://pkgbuild.com/~lcarlier/$repo/$arch
```

principia

- **Maintainer:** ROllerozxa <rollerozxa@voxelmanip.se>
- **Description:** Binary builds of the principiaAUR and principia-gitAUR packages.
- **Upstream page:** https://arch.principia-web.se

```
[principia]
Server = https://arch.principia-web.se/$arch
```

rayr

- **Maintainer:** Rayr
- **Description:** Prebuilt packages of my AUR packages, as an Arch repo.
- **Upstream page:** https://github.com/Rayrsn/ArchRepo

```
[rayr]
Server = https://rayrsn.github.io/ArchRepo/$arch
```

selinux

- **Maintainer:** Nicolas Iooss (nicolas <dot> iooss <at> m4x <dot> org)
- **Description:** Provide binary packages for SELinux.
- **Upstream Page:** https://github.com/archlinuxhardened/selinux

```
[selinux]
Server = https://github.com/archlinuxhardened/selinux/releases/download/ArchLinux-
SELinux
SigLevel = Never
```

stx4

- **维护者:** StarterX4 <starterx4\[at\]gmail.com>
- **描述:** Any – some fonts and fakepkgs; x86_64 – archival yet might useful packages (like PacmanXG4) and some AUR soft I use(d).
- **Upstream Page:** https://github.com/StarterX4/StarterX4.github.io

```
[stx4-any]
Server = https://starterx4.github.io/repos/arch/$arch/stx4
[stx4-x86_64]
Server = https://starterx4.github.io/repos/arch/$arch/stx4
```

vdr4arch

- **Maintainers:** CReimer, M-Reimer
- **Description:** A set of VDR packages for Arch Linux
- **Upstream page:** https://github.com/VDR4Arch/vdr4arch

注意：Packages are automatically built with repo-make-ci.sh.

```
[vdr4arch]
Server = https://vdr4arch.github.io/$arch
```

Packages search

警告：`pkgs.org` 将 Arch Linux ARM 的 aarch64 仓库列为 Arch Linux 组织的官方仓库，这是错误的。这些仓库不是 Arch Linux 官方的，使用它们请风险自担。

很多非官方仓库在 https://archlinux.pkgs.org/ 上有索引。此网站提供了仓库浏览和软件包搜索的功能。

AUR 助手

这一章解决一件事：官方仓库没有、只在 AUR 里的软件，怎么用 yay 装起来，装完又怎么继续管。全章只有《Yay》一节，安装部分交代 yay 从哪儿来、怎么装到位，命令部分列出日常真正用得上的那些。

单节即全部内容，没有可跳的分支，两块都建议读完；命令不必背，看过一遍留在手边当速查即可。最容易踩的坑是把"装上了 yay"当成"装好了包"：它只是把下载、构建、安装串成一条命令，结果对不对仍取决于构建脚本和依赖；另一个误解是以为经它装上的软件与官方仓库的包等价，AUR 的内容并没有人替你审过。

12.1　Yay

难度 入门 | **篇幅** 3.4 KB | **所属** AUR 助手

本节将讲到

- 安装
- 命令

Yay 是一个适用于Arch Linux的命令行软件，主要用于帮助用户从Arch User Repository(AUR)构建和安装软件包。它可以做的事情包括：

1. 自动解决软件包间的依赖关系。
2. 动态地搜索、编译和构建包。
3. 安装与管理 AUR中的包（ yay 的命令与 pacman 基本一致，例如 yay -S 对应 pacman -S，但 yay 能同时处理官方仓库和 AUR。 ）。

警告：Arch Linux 不对 AUR 助手引发的问题提供支持。您应熟悉自行构建过程以准备好解决遇到的问题。

安装

在开始之前，请确保您已安装 base-devel包 git包

安装base-devel, git：

```
# pacman -S git base-devel
```

克隆 yay 源码：

```
$ git clone https://aur.archlinux.org/yay.git
```

如果你在中国构建yay，强烈建议你克隆 yay-bin 而非 yay。这是由于克隆和构建 `yay-bin` 相比 `yay` 不会遇到被墙问题，克隆 `yay-bin` 构建yay是因为 `yay` 是以Go语言构建的yay，可能会受到被墙影响：

```
$ git clone https://aur.archlinux.org/yay-bin.git
$ cd yay-bin
```

进入yay文件夹：

```
$ cd yay
```

构建 yay 并在构建成功后安装生成的软件包：

```
$ makepkg -si
```

如果您想一次完成所有操作，请使用以下命令：

```
$ sudo pacman -S git base-devel && git clone https://aur.archlinux.org/yay.git && cd yay && makepkg -si
```

如果不想编译安装，也可以添加 [\[archlinuxcn\]](../非官方用户仓库/Arch%20Linux%20中文社区仓库.md) 仓库来安装 yayCNRepo。

命令

命令	描述
yay	升级系统，相当于yay -Syu
yay <搜索词>	显示包安装选择菜单
yay -Bi <目录>	安装依赖并构建本地PKGBUILD
yay -G <AUR Package>	从ABS或AUR下载PKGBUILD (yay v12.0+)
yay -Gp <AUR Package>	打印ABS或AUR的PKGBUILD到stdout
yay -Ps	打印系统统计信息
yay -Syu --devel	执行系统升级，但同时检查开发包的更新
yay -Syu --timeupdate	执行系统升级并使用PKGBUILD修改时间（不是版本号）来确定更新
yay -Wu <AUR Package>	取消对包的投票 (需要设置AUR_USERNAME和AUR_PASSWORD环境变量) (yay v11.3+)
yay -Wv <AUR Package>	投票支持包 (需要设置AUR_USERNAME和AUR_PASSWORD环境变量) (yay v11.3+)
yay -Y --combinedupgrade --save	使组合升级成为默认模式
yay -Y --gendb	生成用于开发更新的开发包数据库
yay -Yc	清理不需要的依赖
yay -S 包名	安装软件包
yay -Syu	升级所有包（含 AUR）
yay -Sc	清理缓存文件
yay -Rns 包名	删除软件包和依赖及配置
yay -Rs 包名	删除软件包和依赖
yay -R 包名	删除软件包

更多命令请通过 man yay 查看

如果发现命令介绍有错误请帮忙修改

函数式包管理器

依赖版本互相掐架、升级后想退回旧版本，传统做法多半只能手工救场。函数式包管理器换了模型：Nix 把每个包连同其依赖固化在独立路径里，安装、切换与回滚因此变得可预期，代价是得先接受一套不同的规则。

本章只有 Nix 一节，其中安装、配置、使用三段建议按顺序读透——配置决定后面用得顺不顺手，跳过它直接照抄使用部分的命令，是最常见的翻车方式。提示与技巧、命令补全可随手翻，用不上不通读也不影响主线；故障排查留到出问题时再查，不必顺读。另外别把 Nix 当成顺手多加的一个包管理器，它管的东西与系统自带的包各归各。

13.1　Nix

难度 入门 | **篇幅** 5.2 KB | **所属** 函数式包管理器

本节将讲到

- 安装
- 配置
- 使用
- 提示与技巧
- 命令补全

Nix 是一个纯函数式包管理器，旨在使包管理可重现、声明式且可靠。

安装

安装 nix包。

配置

要使 Nix 守护程序在系统启动时启动，启用 `nix-daemon.service`。

将需要运行 Nix 的用户添加到 `nix-users` 用户组中以获取与守护程序套接字连接的权限。重启会话以使更改生效。

添加一个频道并更新它。

```
$ nix-channel --add https://nixos.org/channels/nixpkgs-unstable
$ nix-channel --update
```

使用

注意：如果您从 官方仓库 安装了 Nix，您必须手动将 `~/.nix-profile/bin` 目录添加到您的 PATH 中。

在 shell 配置完成后，以下内容应该将 *hello* 安装到您已经更新过的 PATH 中：

```
$ nix-env -iA nixpkgs.hello
```
二进制文件本身将位于 `/nix/store/[hash]-hello-[version]/bin/hello`。

运行 `hello` 并确保它在正确的 PATH 中。如果它工作正常，您可以卸载它：

```
$ nix-env --uninstall hello
```
或者您可以检查已安装的程序列表：

```
$ nix-env --query
```
您还可以列出迭代版本：

```
$ nix-env --list-generations
```
更多详细信息请参阅 nix-env(1) 手册。

提示与技巧

最大任务数

nix 默认只同时进行一个构建。以下内容将可以使 nix 同时进行与 CPU 数量相同的构建：

```
/etc/nix/nix.conf
max-jobs = auto
```

图形加速

要运行 OpenGL 和 Vulkan 应用程序，请使用 NixGL。

桌面集成

要将 Nix 应用程序与您的桌面环境集成，请将 `~/.nix-profile/share` 目录添加到您的 `$XDG_DATA_DIRS` 中，例如使用 `export XDG_DATA_DIRS=$HOME/.nix-profile/share:$XDG_DATA_DIRS`。

Zsh 集成用于 nix-shell

`nix-shell` 默认启动 Bash。zsh-nix-shellAUR 允许您在 `nix-shell` 环境中使用 Zsh 作为默认 shell。一些提示插件如 zsh-theme-powerlevel10kAUR 和 zsh-pure-promptAUR 提供了 `nix-shell` 指示器。

命令补全

Zsh

nix-zsh-completionsAUR 提供了 Zsh 对 nix 命令的补全，如 `nix-env` 和 `nix-shell`。

故障排查

打开的文件太多

一些构建可能会遇到类似如下的问题：

```
error: opening directory '/nix/store/...': Too many open files
```

编辑 `nix-daemon.service` 并增大文件限制：

```
[Service]
LimitNOFILE=65536
```

关于 root 用户更新频道的警告

如果在使用 Nix 时出现以下错误：

```
warning: Nix search path entry '/nix/var/nix/profiles/per-user/root/channels' does not exist, ignoring
```

root 用户需要更新他们的频道：

```
# nix-channel --update
```

沙盒构建问题

其他沙盒问题

```
error: while setting up the build environment: mounting /proc: Operation not permitted
error: program '/usr/bin/nix-env' failed with exit code 1
1: package 'utils' in options("defaultPackages") was not found
2: package 'stats' in options("defaultPackages") was not found
Error: .onLoad failed in loadNamespace() for 'utils', details:
 call: system(paste(which, shQuote(names[i])), intern = TRUE, ignore.stderr = TRUE)
 error: cannot popen '/nix/store/fnkvlbls29d01jcx3wsdnhykyrl7087r-which-2.21/bin/which 'uname' 2>/dev/null', probable reason 'Cannot allocate memory'
```

这是已知的上游问题：#2311、#3000 和 #4636。

警告：通常不建议禁用沙盒，因为这会污染构建环境，且可能导致更多的构建错误。`nixpkgs` 中的所有内容都指望在启用沙盒的情况下构建。

最常见的解决方法是在配置文件中禁用沙盒：

```
/etc/nix/nix.conf
# 禁用沙盒
sandbox = false
```

然后重启 `nix-daemon` 服务。

区域设置警告

Nixos wiki 建议运行 `export LOCALE_ARCHIVE=/usr/lib/locale/locale-archive`。或者，导出环境变量 `LC_ALL=C`。

参见

- 上游 Wiki
- Nix 包管理器指南

启动屏幕

这一章处理开机画面的割裂感：从引导器交棒到登录界面这段路，原本滚动内核日志、再突然切进图形界面，改为用一张 Plymouth 启动图贯穿，让开机看起来连续。本章仅一节，准备、安装、配置三步环环相扣，建议按序读完；提示与技巧、疑难解答留待排查时翻查，另见可略。

最容易踩的坑是把 Plymouth 当成装上就生效的软件包：它必须被纳入启动流程，且每次改动后重新生成才会生效，否则看到的仍是文字滚动。另一个误解是任何显卡都能直接出图，条件不满足时往往不报错，只是黑屏或闪一下，动手前先想好怎样退回文本启动。

14.1　Plymouth

难度 进阶 | **篇幅** 10.1 KB | **所属** 启动屏幕

本节将讲到

- 准备
- 安装
- 配置
- 提示与技巧
- 疑难解答

Plymouth 是一个来自于 Fedora 社区的提供漂亮的启动界面的项目，现在它被列入了 freedesktop.org 的官方资源之中。它依靠 KMS 尽可能早的设置显示器的原始分辨率显示，之后产生漂亮的启动界面直至出现登录界面。

准备

Plymouth 依靠 KMS（Kernel Mode Setting，内核级显示模式设置）显示图形界面。在 UEFI 系统中，Plymouth 可以使用 EFI 帧缓冲。

如果使用了私有驱动, 无法启用 KMS，或者只是不想使用 EFI 帧缓冲，那么可以考虑使用支持大屏分辨率的 Uvesafb。如果既没有KMS也没有framebuffer，那么Plymouth将使用文本模式。

安装

需要稳定版本请安装 plymouth包， plymouth-gitAUR是开发版本。

默认情况下，Plymouth 将引导消息记录到 `/var/log/boot.log`，并且不显示启动画面。

- 要显示启动画面，将 `splash` 加入内核参数。

- 需要静默启动, 再添加 `quiet`。
- 要禁用日志，添加 `plymouth.nolog`。

安装 Plymouth 后，如果想要在启动早期阶段就启用，需要配置 initramfs 生成器，创建含有Plymouth的镜像。

mkinitcpio

把`plymouth`添加到 mkinitcpio.conf 的 `HOOKS` 列表中：

```
/etc/mkinitcpio.conf
HOOKS=(... plymouth ...)
```

然后使用 `mkinitcpio -P` 命令重新生成 initramfs。

如果你正在使用 `systemd` 钩子，它必须在 `plymouth` 之前。

如果你的系统使用 dm-crypt 加密，请确保将 `plymouth` 钩子置于 `crypt` 钩子之前。

dracut

安装 Plymouth 之后，dracut 会自动捕获并将其添加到 initramfs 镜像中。如果自动捕获失败，可以在 dracut 配置文件中添加下列几行来强制 dracut 引入 Plymouth：

```
/etc/dracut.conf.d/myflags.conf
add_dracutmodules+=" plymouth "
```

配置

通过 `/etc/plymouth/plymouthd.conf` 进行配置，通过 `/usr/share/plymouth/plymouthd.defaults` 文件可以查看配置的默认值。

更改主题

Plymouth 自带了一些主题：

1. **BGRT**: Spinner 的一个变种，如果 OEM 的图标可用便会显示。（BGRT 指的是 Boot Graphics Resource Table）
2. **Fade-in**: "简单的有淡出淡入的星星的主题"
3. **Glow**: "伴随着新兴标志的饼状引导进度条的企业主题"
4. **Script**: "脚本案例插件" (漂亮的 Arch Logo 主题)
5. **Solar**: "带有燃烧的蓝色星球的空间主题"
6. **Spinner**: "带有加载框的简单主题"
7. **Spinfinity**: "显示旋转的无穷大标志的主题"
8. **Tribar**: "带三色进度条的文本模式主题"
9. *(**Text**: "带三色进度条的文本模式主题")*
10. *(**Details**: "详细信息回退主题")*

默认选择 **bgrt**，可以通过修改配置文件设置其它主题, 例如:

```
/etc/plymouth/plymouthd.conf
[Daemon]
Theme=fade-in
```

或者执行

```
# plymouth-set-default-theme -R theme
```

每次更换主题，`initrd` 都必须重新构建。`-R`选项可确保其被重建（或者手动重新生成 initramfs）：

安装新主题

可以通过 AUR 安装其它主题，或者 plymouth-kcm包 提供与 KDE Plasma 设置的集成，并提供 AUR 上没有的主题。

使用以下命令获得已安装的主题列表：

```
$ plymouth-set-default-theme -l
```

或是通过：

```
$ ls /usr/share/plymouth/themes
bgrt  details  fade-in  glow  script  solar  spinfinity  spinner  text  tribar
```

显示延迟

Plymouth 可以配置选项以延迟启动画面显示：

```
/etc/plymouth/plymouthd.conf
[Daemon]
ShowDelay=5
```

对于启动很快的系统，可能在显示管理器或登录提示准备好前只能看到启动画面闪过。可以设置 `ShowDelay` 为一个比启动时间更长的值（以秒为单位）来避免这种闪烁以及黑屏。默认值为0秒，因此您不需要将其更改为其他值，就可以在引导过程中更早地看到您的启动。

HiDPI

编辑配置文件：

```
/etc/plymouth/plymouthd.conf
DeviceScale=an-integer-scaling-factor
```

然后重建 initrd。

提示与技巧

显示启动消息

启动时按 `Esc` 键可以切换到启动消息。

平滑过渡

GDM 支持开箱即用的*平滑过渡*，无需额外配置。

预览主题

可以通过在getty:中运行以下命令来预览当前配置的主题

```
# plymouthd; plymouth --show-splash; sleep 5; plymouth --quit
```

注意：此命令行以包含 sleep 调用的链式命令方式执行，可在设定的延迟后将控制权交还给用户，请勿将其移除。

更换背景图片

某些主题（如 spinner 和 BGRT）可以改变其背景图像。只需将 `/usr/share/plymouth/themes/spinner/background-tile.png` 替换为你想要的图片。一旦改变主题就要重新生成 initrd。

缺少 BGRT 图片

如果您使用的是 BGRT 主题，但 UEFI 没有提供供应商徽标，则可以将回退镜像放置到 `/usr/share/plymouth/themes/spinner/bgrt-fallback.png` 中以显示它。

或者，设置以下内容以保留固件背景：

```
/etc/plymouth/plymouthd.conf
UseFirmwareBackground=true
```

放慢启动速度以显示完整的动画

在启动时间非常快的系统上，如果需要显示整个动画，则可能需要使用包含 `ExecStartPre=/usr/bin/sleep 5` 的附加片段为 `plymouth-quit.service` 添加延迟。见这篇 reddit 帖子。

也可以使用一个新的 systemd 服务，它与 Plymouth 同时启动，并且等待了完整播放动画所需要的时间。这种方法将确保不会察觉到启动时间不一致，因为它不**在动画开始显示之后**添加时间，而是一个**在动画期间**开始的延迟。

```
/etc/systemd/system/plymouth-wait-for-animation.service
[Unit]
Description=Waits for Plymouth animation to finish
Before=plymouth-quit.service display-manager.service

[Service]
Type=oneshot
ExecStart=/usr/bin/sleep 动画所需的时间

[Install]
WantedBy=plymouth-start.service
```

然后启用这个服务。

注意：如果使用 initramfs 启动了 Plymouth，该方法将不起作用。

使用 SimpleDRM

对于使用 UEFI 的设备，Plymouth 默认使用 SimpleDRM 驱动程序在 UEFI 帧缓冲上显示启动画面。这使得在更快但加载 GPU 驱动耗时较长的机器上（例如，使用 AMD GPU 的系统），启动画面也能够显示出来，同时还能消除主板启动画面与 Plymouth 之间的闪烁。

在旧式 UEFI 系统上，如果 efifb 无法以原生模式启动，而只能以 640x32、800x600 或 1024x768 分辨率运行，Plymouth 将默认不使用 SimpleDRM。

警告：使用此方法时，次要的显示器在启动过程中不会点亮。如果您的笔记本使用扩展坞连接屏幕，全盘加密的密码提示可能无法显示。

要禁用 SimpleDRM，请在您的内核参数中添加 `plymouth.use-simpledrm=0`。有关此选项的更多信息，请参见 Plymouth 问题 #264。

疑难解答

禁用的内核参数

如果在启动过程中遇到问题，可以使用以下内核参数临时禁用 Plymouth：

```
plymouth.enable=0 disablehooks=plymouth
```

调试

要将调试输出写入 `/var/log/plymouth-debug.log`，请添加以下内核参数：

```
plymouth.debug
```

密码提示不更新

当在 Mkinitcpio 中使用 `systemd` 而不是 `udev` 钩子时，在通过 Plymouth 脚本处理它的主题上密码提示可能不会更新。

您可以尝试切换到开发版本 plymouth-gitAUR 或使用来自 Mkinitcpio#常用钩子的替代品。

显示未居中

当在启动过程中启用了多个监视器时，某些主题可能无法使显示居中。

您可以使用内核级显示模式设置#Forcing modes 禁用特定的监视器。

另见

- 原始规范
- 论坛相关帖子